

Inflation

March 15, 2025

1 Analysis of Key Economic Datasets

1.1 1. 10-Year Breakeven Inflation Rate (T10YIE)

1.1.1 Definition and Formula

The 10-year breakeven inflation rate is calculated as:

$$10Y_{\text{Breakeven}} = 10Y_{\text{Nominal Treasury Yield}} - 10Y_{\text{TIPS Yield}}$$

This represents the market's expectation of average inflation over the next 10 years.

1.1.2 Historical Trends and Implications

- Breakeven inflation rates tend to decline during economic crises (e.g., 2008, 2020) due to deflation fears.
- They increase during inflationary periods (e.g., 2021-2022) when investors anticipate higher inflation.
- It is influenced by central bank policies, investor risk appetite, and liquidity conditions.

1.1.3 Applications in Economic Analysis

- Used by policymakers to gauge long-term inflation expectations.
- Helps investors decide between nominal and inflation-protected bonds.
- A rising breakeven rate can signal growing concerns about inflation.

1.1.4 Relationship to Other Datasets

- Closely linked to Treasury yields and TIPS yields.
- Related to short-term inflation measures like CPI and PCE.

1.2 2. 5-Year, 5-Year Forward Inflation Expectation Rate (T5YIFR)

1.2.1 Definition and Derivation

This metric represents expected inflation for the five-year period beginning five years from today. It is calculated as:

$$5Y5Y = \left[\frac{(1 + \frac{10Y_{nom} - 10Y_{TIPS}}{100})^{10}}{(1 + \frac{5Y_{nom} - 5Y_{TIPS}}{100})^5} \right]^{\frac{1}{5}} - 1$$

1.2.2 Historical Trends and Monetary Policy Role

- Typically remains close to the Fed's 2% inflation target.
- Declined during deflationary periods (2009, 2015, early 2020) and rose during inflationary fears (2021-2022).
- Used by the Fed to monitor inflation expectations.

1.2.3 Applications in Predicting Inflation Expectations

- Helps forecast long-term inflation trends.
- Used in pricing long-dated securities and contracts.

1.2.4 Relationship with Breakevens and Yields

- Related to the 10-year breakeven inflation rate and nominal Treasury yields.
- A rising 5Y5Y can signal market concerns about unanchored inflation expectations.

1.3 3. Consumer Price Index for All Urban Consumers (CPIAUCNS)

1.3.1 CPI Calculation

$$CPI = \frac{\text{Cost of basket in current period}}{\text{Cost of basket in base period}} \times 100$$

1.3.2 Historical Inflation Trends

- CPI inflation was high in the 1970s (~14% in 1980), moderate in the 1990s (2-3%), and surged again in 2021-2022 (>7%).
- Affects real incomes and purchasing power.

1.3.3 Applications in Economic Analysis

- Used for cost-of-living adjustments in wages, pensions, and tax brackets.
- Fed watches CPI as an inflation measure, though it prefers PCE.

1.3.4 Comparison with PCE Price Index (PCEPI)

- CPI is a fixed-basket Laspeyres index, while PCE is a chain-weighted index.
- CPI is typically higher than PCE due to different weighting and methodology.

1.4 4. Real Gross Domestic Product (GDPC1)

1.4.1 GDP Calculation

$$\text{Real GDP} = \frac{\text{Nominal GDP}}{\text{GDP Deflator}} \times 100$$

1.4.2 Trends in Real GDP

- Post-WWII average GDP growth was ~3% per year.
- Deep contractions occurred in 2008 (-4%) and 2020 (-9%).
- Long expansions (e.g., 1990s, 2010s) saw steady growth.

1.4.3 Use in Measuring Economic Performance

- Key measure of economic health.
- Central to fiscal and monetary policy decisions.

1.4.4 Relationship with Unemployment and Inflation

- Related to unemployment via **Okun's Law** (1% rise in unemployment = 2% GDP loss).
 - Connected to inflation via the **Phillips Curve** (tight labor markets push up wages/prices).
-

1.5 5. Unemployment Rate (UNRATE)

1.5.1 Calculation Formula

$$\text{Unemployment Rate} = \frac{\text{Unemployed}}{\text{Labor Force}} \times 100$$

1.5.2 Historical Unemployment Trends

- Peaked at **14.7% in April 2020**, the highest since WWII.
- Long-term average ~5.8%, with low points near **3.5%** (1969, 2019, 2022).

1.5.3 Applications in Labor Market Analysis

- Indicates economic slack or tightness.
- Used in macro models and recession predictions.

1.5.4 Relationship with GDP and Inflation

- Linked to GDP via **Okun's Law**.
 - Related to inflation via the **Phillips Curve**.
-

1.6 6. Federal Funds Effective Rate (FEDFUNDS)

1.6.1 Determination and Mechanics

- Set by the **Federal Open Market Committee (FOMC)**.
- Influences short-term borrowing costs and liquidity conditions.

1.6.2 Historical Interest Rate Cycles

- Peaked at **20% in 1980** to combat inflation.
- Cut to **0% in 2008 and 2020** for economic stimulus.
- Raised to **5%+ in 2023** to control inflation.

1.6.3 Applications in Monetary Policy

- Primary tool for influencing economic growth and inflation.
- Affects mortgage rates, business investment, and consumer spending.

1.6.4 Influence on Inflation, GDP, and Treasury Yields

- Higher rates slow inflation but risk recession.
 - Directly impacts Treasury yields and credit markets.
-

1.7 7. Personal Consumption Expenditures: Chain-type Price Index (PCEPI)

1.7.1 PCE Price Index Calculation

- Chain-weighted index accounts for **consumer substitution**.
- Fed's preferred inflation measure.

1.7.2 Applications in Inflation Tracking

- Used for forecasting and economic modeling.
- Monitored for Fed's 2% inflation target.

1.7.3 Comparison with CPI

- PCE is generally lower than CPI due to different weights and methodology.
-

1.8 8. Nominal Broad U.S. Dollar Index (DTWEXBGS)

1.8.1 Calculation Methodology

- Trade-weighted index of USD against a basket of currencies.

1.8.2 Historical Dollar Trends

- Peaked in **1985, early 2000s, and 2022**.
- Strong dollar tends to increase trade deficits.

1.8.3 Use in Forex Analysis

- Indicates USD strength relative to major trading partners.
 - Impacts inflation via import price effects.
-

1.9 9. Crude Oil Prices: WTI (DCOILWTICO)

1.9.1 Oil Price Determinants

- Driven by **supply-demand dynamics**, OPEC policy, and geopolitical risks.

1.9.2 Historical Price Fluctuations

- Reached **\$147 in 2008**, negative in April 2020, **\$120 in 2022**.

1.9.3 Impact on Inflation and Economy

- High oil prices contribute to inflation.
 - Price swings impact energy markets and consumer spending.
-

1.10 10. Market Yield on U.S. Treasury Securities at 10-Year Constant Maturity (DGS10)

1.10.1 How 10-Year Yields Are Determined

- Interpolated yield from Treasury curve.

1.10.2 Economic Significance

- Benchmark for long-term borrowing rates.
- Inversions predict recessions.

1.10.3 Relationship with Inflation Expectations

- Higher expected inflation → Higher 10Y yield.
- Used in macroeconomic forecasting and risk assessment.

2 Dataset Preparations

```
[ ]: # Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

import os
import pandas as pd

# Define the folder path where your CSV files are stored
folder_path = '/content/drive/My Drive/Dataset/'

# List files in the folder to verify they exist
print("Files in the folder:", os.listdir(folder_path))

def detect_date_column(df):
    """
```

```

Detects a date column (case-insensitive) in a DataFrame.
Looks for 'date' or 'observation_date'.
Raises a ValueError if no such column exists.
"""
possible_date_columns = ['date', 'observation_date']
for col in df.columns:
    if col.strip().lower() in possible_date_columns:
        return col
raise ValueError("No date column found in the dataset.")

def read_dataset(filename, rename_dict):
    """
    Loads a CSV file, converts its date column to datetime, sets it as the
    ↪index,
    and renames columns as specified.

    Args:
        filename (str): Name of the CSV file.
        rename_dict (dict): Dictionary mapping original column names to new
        ↪names.

    Returns:
        pd.DataFrame: Processed DataFrame with a DatetimeIndex.
    """
    full_path = os.path.join(folder_path, filename)
    df = pd.read_csv(full_path)

    try:
        date_col = detect_date_column(df)
    except ValueError as e:
        raise ValueError(f"Error in file '{filename}': {e}")

    # Convert the detected date column to datetime (coerce errors)
    df[date_col] = pd.to_datetime(df[date_col], errors='coerce')
    if df[date_col].isnull().all():
        raise ValueError(f"Date conversion failed for column '{date_col}' in
        ↪file '{filename}'.")

    df.set_index(date_col, inplace=True)

    # Verify the index is a DatetimeIndex
    if not isinstance(df.index, pd.DatetimeIndex):
        raise ValueError(f"Index for file '{filename}' is not a DatetimeIndex
        ↪after conversion.")

    df = df.rename(columns=rename_dict)
    return df

```

```

# Configuration: list of tuples with file name and corresponding renaming
↳dictionary
datasets_config = [
    ('T10YIE.csv', {'T10YIE': '10-Year Breakeven Inflation Rate'}),
    ('T5YIFR.csv', {'T5YIFR': '5-Year, 5-Year Forward Inflation Expectation
↳Rate'}),
    ('CPIAUCNS.csv', {'CPIAUCNS': 'Consumer Price Index'}),
    ('GDPC1.csv', {'GDPC1': 'Real Gross Domestic Product'}),
    ('UNRATE.csv', {'UNRATE': 'Unemployment Rate'}),
    ('FEDFUNDS.csv', {'FEDFUNDS': 'Federal Funds Effective Rate'}),
    ('PCEPI.csv', {'PCEPI': 'Personal Consumption Expenditures: Chain-type
↳Price Index'}),
    ('DTWEXBGS.csv', {'DTWEXBGS': 'Nominal Broad U.S. Dollar Index'}),
    ('DCOILWTICO.csv', {'DCOILWTICO': 'Crude Oil Prices: West Texas
↳Intermediate'}),
    ('DGS10.csv', {'DGS10': '10-Year Treasury Yield'})
]

# Load each dataset using the configuration; skip files with errors.
datasets = []
for filename, rename_dict in datasets_config:
    try:
        df = read_dataset(filename, rename_dict)
        datasets.append(df)
    except Exception as e:
        print(f"Skipping file '{filename}' due to error: {e}")

if not datasets:
    raise ValueError("No datasets loaded successfully. Please check your files.
↳")

# Resample each dataset to a daily frequency and forward-fill missing values
datasets = [df.asfreq('D').ffill() for df in datasets]

# Determine the common date range across all datasets:
# - New dataset starting point: the newest (maximum) start date among datasets
# - New dataset ending point: the oldest (minimum) end date among datasets
start_dates = [df.index.min() for df in datasets]
end_dates = [df.index.max() for df in datasets]

common_start = max(start_dates)
common_end = min(end_dates)

if common_start > common_end:
    raise ValueError("No overlapping date range among datasets. Check your data.
↳")

```

```

print("Common start date:", common_start)
print("Common end date:", common_end)

# Trim each dataset to the common date range
datasets = [df.loc[common_start:common_end] for df in datasets]

# Merge all datasets on the date index using an inner join (keeping only common
↳ dates)
merged_df = pd.concat(datasets, axis=1, join='inner')

# Save the merged dataset to a CSV file in the Dataset folder
merged_file_path = os.path.join(folder_path, 'merged_dataset.csv')
merged_df.to_csv(merged_file_path)

print("Merged dataset shape:", merged_df.shape)
print("Merged dataset saved to:", merged_file_path)

```

Mounted at /content/drive
Files in the folder: ['T10YIE.csv', 'T5YIFR.csv', 'CPIAUCNS.csv', 'PCEPI.csv',
'GDPC1.csv', 'UNRATE.csv', 'FEDFUNDS.csv', 'DGS10.csv', 'DTWEXBGS.csv',
'DCOILWTICO.csv', 'merged_dataset.csv']
Common start date: 2006-01-02 00:00:00
Common end date: 2024-10-01 00:00:00
Merged dataset shape: (6848, 10)
Merged dataset saved to: /content/drive/My Drive/Dataset/merged_dataset.csv

3 Data Preprocessing

```

[ ]: # Block 1: Import Libraries and Mount Google Drive
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn imports
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Mount Google Drive (for Colab)
from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```
[ ]: # Block 2: Load the Merged Dataset

def load_dataset(file_path):
    """
    Loads a CSV dataset, parses dates, and sets the first column as index.

    Parameters:
        file_path (str): Full path to the CSV file.

    Returns:
        DataFrame: Loaded dataset with a datetime index.
    """
    try:
        df = pd.read_csv(file_path, parse_dates=True, index_col=0)
    except Exception as e:
        raise ValueError(f"Failed to load dataset: {e}")
    return df

# Define path to the merged dataset and load it
merged_file_path = '/content/drive/My Drive/Dataset/merged_dataset.csv'
df = load_dataset(merged_file_path)

# Reload the dataset from the same path to create a separate copy for data
↳ visualization
data = pd.read_csv("/content/drive/My Drive/Dataset/merged_dataset.csv")

# Check column names and null counts
print("Dataset Columns:", df.columns)
print("Null Values per Column:\n", df.isnull().sum())
```

```
Dataset Columns: Index(['10-Year Breakeven Inflation Rate',
                        '5-Year, 5-Year Forward Inflation Expectation Rate',
                        'Consumer Price Index', 'Real Gross Domestic Product',
                        'Unemployment Rate', 'Federal Funds Effective Rate',
                        'Personal Consumption Expenditures: Chain-type Price Index',
                        'Nominal Broad U.S. Dollar Index',
                        'Crude Oil Prices: West Texas Intermediate', '10-Year Treasury Yield'],
                        dtype='object')
Null Values per Column:
10-Year Breakeven Inflation Rate      0
5-Year, 5-Year Forward Inflation Expectation Rate      0
Consumer Price Index                  0
Real Gross Domestic Product           0
Unemployment Rate                    0
Federal Funds Effective Rate          0
Personal Consumption Expenditures: Chain-type Price Index      0
Nominal Broad U.S. Dollar Index       0
```

```
Crude Oil Prices: West Texas Intermediate      0
10-Year Treasury Yield                        0
dtype: int64
```

```
[ ]: # Block 3: Data Preparation and Target Imputation (Time-Based Interpolation)
```

```
def prepare_target(df, target):
    """
    Converts the target column to numeric, then imputes missing values
    using time-based interpolation (rather than forward-filling).
    This helps preserve abrupt dips or spikes and avoids carrying old values
    forward.

    Parameters:
        df (DataFrame): The dataset, indexed by DatetimeIndex.
        target (str): The target column name.

    Returns:
        DataFrame: The modified DataFrame.
    """
    # Convert the target column to numeric
    df[target] = pd.to_numeric(df[target], errors='coerce')

    # Interpolate missing values (time-based interpolation)
    # Assumes df has a proper DatetimeIndex
    df[target] = df[target].interpolate(method='time')

    # Optionally, drop any remaining NaNs if interpolation
    # could not fill everything (e.g., at start or end)
    df.dropna(subset=[target], inplace=True)

    return df

target = '10-Year Breakeven Inflation Rate'
df = prepare_target(df, target)
```

```
[ ]: # Block 4: Feature Engineering (Lag Features and Rolling Mean)
```

```
def create_lagged_features(df, target_col, lags=5, seasonal_lag=None):
    """
    Create lag features and a rolling mean for the target variable.

    Parameters:
        df (DataFrame): DataFrame with a datetime index.
        target_col (str): The target column name.
        lags (int): Number of lag periods to create.
        seasonal_lag (int or None): Optional seasonal lag.
```

```

Returns:
    DataFrame: DataFrame with lag features and rolling mean (rows with NaN
↳dropped).
    """
    df_fe = df.copy()
    # Create lag features
    for lag in range(1, lags + 1):
        df_fe[f'lag_{lag}'] = df_fe[target_col].shift(lag)
    # Create seasonal lag if specified
    if seasonal_lag is not None:
        df_fe[f'seasonal_lag_{seasonal_lag}'] = df_fe[target_col].
↳shift(seasonal_lag)
    # Create rolling mean based on the specified number of lags
    df_fe['rolling_mean'] = df_fe[target_col].rolling(window=lags).mean()
    # Drop rows with NaN values
    df_fe.dropna(inplace=True)
    return df_fe

# Create features without resetting the index (i.e. preserving the datetime
↳index)
n_lags = 5
seasonal_lag = None # Change if needed
data_fe = create_lagged_features(df, target, lags=n_lags,
↳seasonal_lag=seasonal_lag)

```

```

[ ]: # Block 5: Train-Test Split and Feature Scaling (Chronological Split)
def split_and_scale(data, target, split_ratio=0.8):
    """
    Splits the dataset into training and test sets chronologically, then scales
↳numeric predictor features.
    Also returns the original X and y for later use.

    Parameters:
        data (DataFrame): Feature-engineered DataFrame, expected to have a
↳DateTimeIndex.
        target (str): Name of the target column.
        split_ratio (float): Proportion of data to use for training (default 0.8).

    Returns:
        tuple: (X, y, X_train, X_test, y_train, y_test, scaler)
    """

    # 1) Sort the DataFrame by its index to ensure strict chronological order
    data = data.sort_index()

    # 2) Separate predictors and target (preserving the original X and y)
    X = data.drop(columns=[target])

```

```

y = data[target]

# 3) Select only numeric columns for scaling
X_numeric = X.select_dtypes(include=[np.number])

# 4) Scale numeric predictor features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_numeric)
X_scaled = pd.DataFrame(X_scaled, columns=X_numeric.columns, index=X.index)

# 5) Chronologically split the data based on the split_ratio
split_index = int(split_ratio * len(data)) # e.g., 80% train, 20% test
X_train = X_scaled.iloc[:split_index]
X_test = X_scaled.iloc[split_index:]
y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

return X, y, X_train, X_test, y_train, y_test, scaler

# Apply the function to the feature-engineered data
X, y, X_train, X_test, y_train, y_test, scaler = split_and_scale(data_fe,
↳target)
print("Training set shape:", X_train.shape, y_train.shape)
print("Test set shape:", X_test.shape, y_test.shape)

```

Training set shape: (5474, 15) (5474,)
Test set shape: (1369, 15) (1369,)

```

[ ]: # Block 6: Define Evaluation Metrics Function
def calculate_metrics(y_true, y_pred, naive_forecast, model_name):
    """
    Calculates evaluation metrics for a model.

    Parameters:
        y_true (array-like): True target values.
        y_pred (array-like): Model predictions.
        naive_forecast (array-like): Baseline forecast values.
        model_name (str): Name of the model.

    Returns:
        dict: Dictionary of metrics.
    """
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    # Avoid division by zero by substituting a small number when y_true is 0

```



```

    mape = np.mean(np.abs((y_true - y_pred) / np.where(y_true == 0, np.
↪info(float).eps, y_true))) * 100
    r2 = r2_score(y_true, y_pred)
    smape = np.mean(2 * np.abs(y_true - y_pred) / (np.abs(y_true) + np.
↪abs(y_pred))) * 100
    naive_mae = mean_absolute_error(y_true, naive_forecast)
    mase = mae / naive_mae if naive_mae != 0 else np.nan
    return {
        "Model": model_name,
        "MSE": mse,
        "RMSE": rmse,
        "MAE": mae,
        "MAPE (%)": mape,
        "sMAPE (%)": smape,
        "MASE": mase,
        "R^2": r2
    }

```

```

[ ]: # Block 7: Create a Naive Forecast Baseline
# We use a simple baseline that shifts the test target by one period.
naive_forecast = y_test.shift(1).bfill().values

# Initialize results list to store evaluation metrics from different models
metrics_results = []

# You can now use the calculate_metrics function after obtaining model_
↪predictions.
print("Data preparation complete. Ready for model development.")

```

Data preparation complete. Ready for model development.

4 Data Visualization

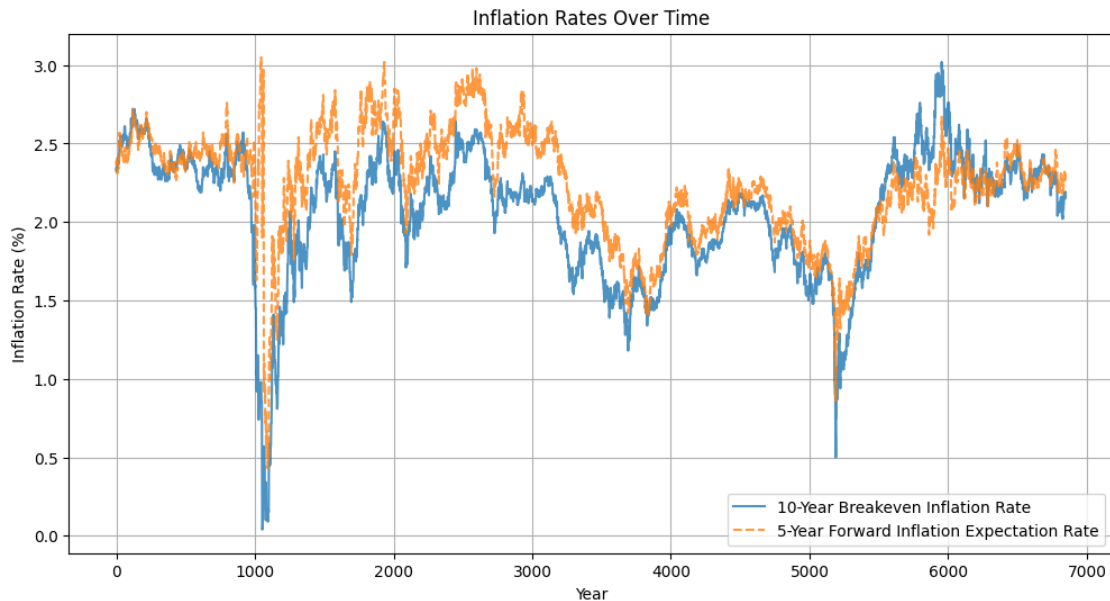
```

[ ]: # Plot Inflation Trends Over Time
import matplotlib.pyplot as plt

# Plot inflation trends over time
plt.figure(figsize=(12, 6))
plt.plot(data.index, data["10-Year Breakeven Inflation Rate"], label="10-Year_
↪Breakeven Inflation Rate", alpha=0.8)
plt.plot(data.index, data["5-Year, 5-Year Forward Inflation Expectation Rate"],_
↪label="5-Year Forward Inflation Expectation Rate", linestyle="dashed",_
↪alpha=0.8)
plt.xlabel("Year")
plt.ylabel("Inflation Rate (%)")
plt.title("Inflation Rates Over Time")

```

```
plt.legend()
plt.grid(True)
plt.show()
```



4.0.1 Analysis of Inflation Rates Over Time

The provided graph illustrates two key inflation metrics:

- **10-Year Breakeven Inflation Rate** (solid blue line)
- **5-Year Forward Inflation Expectation Rate** (dashed orange line)

Key Findings:

- Both inflation measures predominantly fluctuate between 1% and 3%, consistent with historical market-based inflation expectations.
- A significant drop near year 1000, with rates approaching 0%, indicates a period of deflationary concern or economic crisis.
- The 5-Year Forward Inflation Expectation typically remains above the 10-Year Breakeven, suggesting markets anticipate higher medium-term inflation compared to the overall decade.
- Peaks near 3% imply heightened inflation concerns at specific intervals, potentially reflecting economic expansions or inflationary shocks.
- Lows around 0.5% indicate times of market distress or significant economic downturns, such as financial crises or pandemics, when deflationary pressures are stronger.

Overall, the chart highlights market dynamics around inflation expectations, with periods of high volatility reflecting economic uncertainty or crises, and periods of stability around 2% signaling confidence in monetary policy credibility.

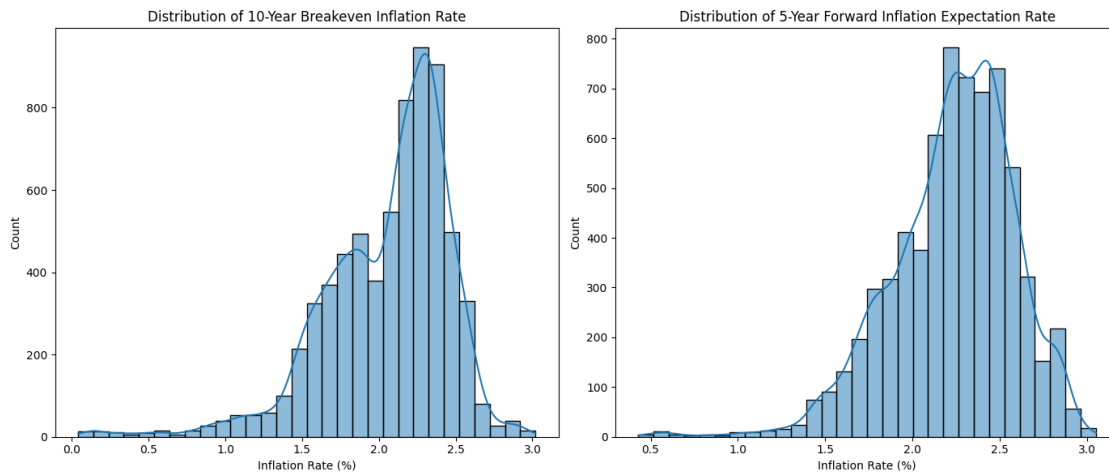
```
[ ]: # Plot Histograms for Inflation Indicators
import seaborn as sns

# Plot histograms
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# 10-Year Breakeven Inflation Rate
sns.histplot(data["10-Year Breakeven Inflation Rate"], bins=30, kde=True,
             ax=axes[0])
axes[0].set_title("Distribution of 10-Year Breakeven Inflation Rate")
axes[0].set_xlabel("Inflation Rate (%)")

# 5-Year Forward Inflation Expectation Rate
sns.histplot(data["5-Year, 5-Year Forward Inflation Expectation Rate"],
             bins=30, kde=True, ax=axes[1])
axes[1].set_title("Distribution of 5-Year Forward Inflation Expectation Rate")
axes[1].set_xlabel("Inflation Rate (%)")

plt.tight_layout()
plt.show()
```



4.0.2 Distribution Analysis of Inflation Metrics

The provided histograms depict the frequency distributions of two inflation measures:

- 10-Year Breakeven Inflation Rate
- 5-Year Forward Inflation Expectation Rate

Key Findings:

- Both distributions are right-skewed:
 - Most observations cluster between approximately 2% to 2.5%.

- A smaller number of data points fall below 1.5% or above 2.5%.
- **Central Tendency:**
 - The peak frequency for both measures occurs around the 2.25% mark, indicating market inflation expectations typically gravitate close to the Federal Reserve’s standard inflation target of ~2%.
- **Variability:**
 - 10-Year Breakeven shows slightly higher variability on the lower end, with more frequent occurrences around 1% compared to the 5-Year Forward expectation, highlighting greater volatility or deflationary scares affecting near-term inflation expectations.
 - The 5-Year Forward Inflation Expectation rate is relatively tighter in distribution, indicating more stable longer-term expectations.
- **Outliers:**
 - Both distributions contain minimal occurrences below 1% or above 2.75%, signifying rare extreme inflation scenarios or deflationary periods in historical data.

Overall, these distributions underline market confidence in inflation rates generally remaining stable around the Fed’s 2% target, with short-term inflation (10-year breakeven) subject to slightly more volatility compared to medium-term forward expectations.

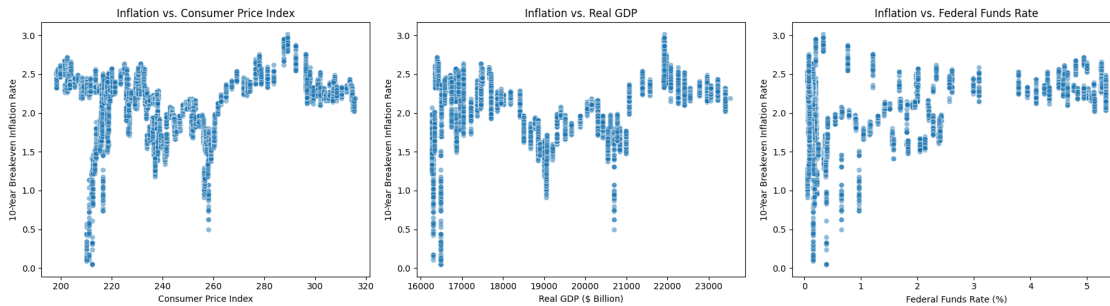
```
[ ]: # Analyze Relationship Between Inflation and Economic Indicators
# Scatter plots to analyze inflation vs. economic indicators
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Inflation vs CPI
sns.scatterplot(x=data["Consumer Price Index"], y=data["10-Year Breakeven_
↪Inflation Rate"], ax=axes[0], alpha=0.5)
axes[0].set_title("Inflation vs. Consumer Price Index")
axes[0].set_xlabel("Consumer Price Index")
axes[0].set_ylabel("10-Year Breakeven Inflation Rate")

# Inflation vs GDP
sns.scatterplot(x=data["Real Gross Domestic Product"], y=data["10-Year_
↪Breakeven Inflation Rate"], ax=axes[1], alpha=0.5)
axes[1].set_title("Inflation vs. Real GDP")
axes[1].set_xlabel("Real GDP ($ Billion)")
axes[1].set_ylabel("10-Year Breakeven Inflation Rate")

# Inflation vs Interest Rate
sns.scatterplot(x=data["Federal Funds Effective Rate"], y=data["10-Year_
↪Breakeven Inflation Rate"], ax=axes[2], alpha=0.5)
axes[2].set_title("Inflation vs. Federal Funds Rate")
axes[2].set_xlabel("Federal Funds Rate (%)")
axes[2].set_ylabel("10-Year Breakeven Inflation Rate")

plt.tight_layout()
plt.show()
```



4.0.3 Relationship Analysis of Inflation and Economic Indicators

The provided scatter plots show the relationships between the **10-Year Breakeven Inflation Rate** and three key economic indicators:

1. **Consumer Price Index (CPI)**
2. **Real GDP**
3. **Federal Funds Rate**

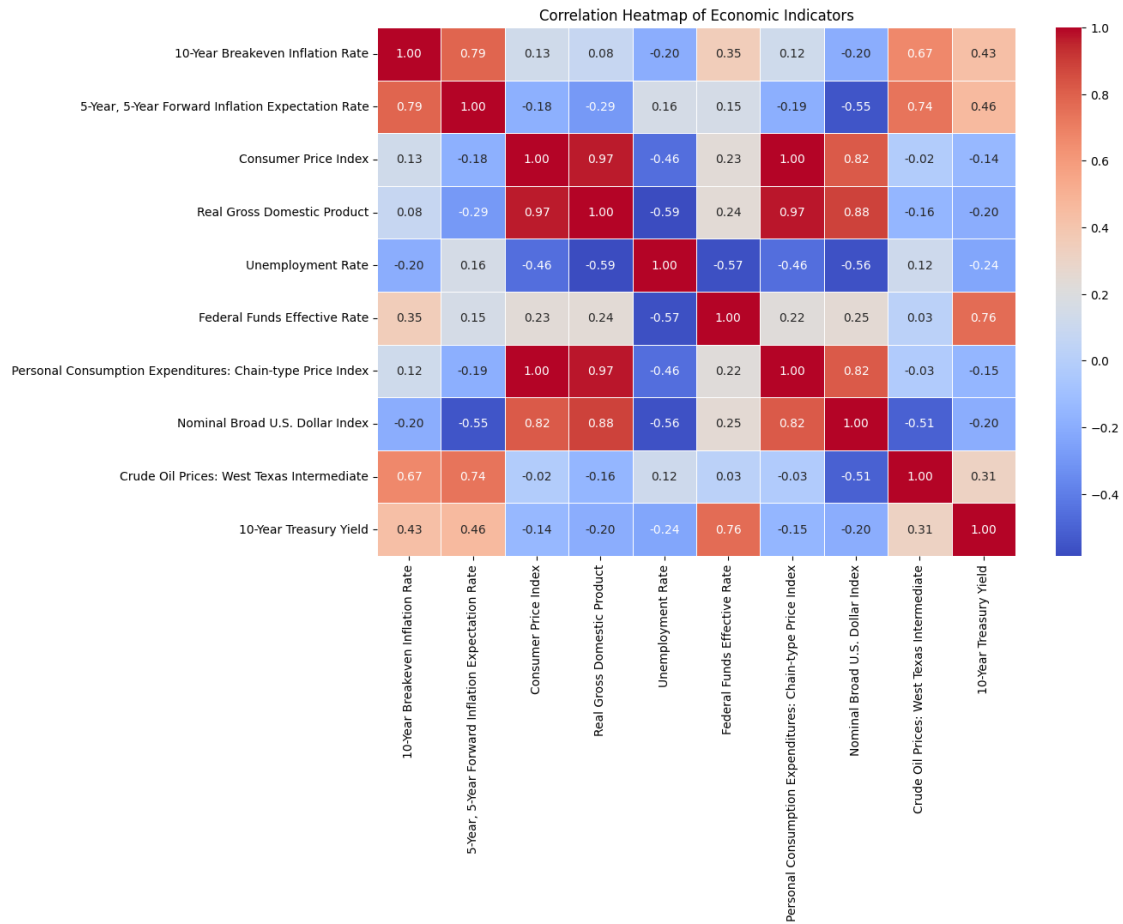
Key Findings:

- **Inflation vs. Consumer Price Index (CPI):**
 - Generally, higher CPI values (above 240) are associated with inflation rates mostly between 2% to 2.5%, indicating stable inflation during periods of rising consumer prices.
 - Lower CPI levels (around 200-220) exhibit wider dispersion and lower inflation rates (below 1%), suggesting deflationary pressures or economic instability.
- **Inflation vs. Real GDP:**
 - Inflation rates above 2% primarily occur at higher GDP values (around \$22,000 billion), indicating inflationary pressures often coincide with periods of economic growth.
 - Inflation rates below 1.5% mostly occur at lower GDP values (below \$18,000 billion), potentially reflecting slower economic growth or recession periods.
- **Inflation vs. Federal Funds Rate:**
 - Higher Federal Funds Rates (above 2%) correlate with moderate-to-high inflation (mostly around 2% to 2.5%), consistent with monetary tightening to control inflation.
 - At lower Federal Funds Rates (below 1%), inflation shows significant variability, ranging from near-zero to above 2.5%, reflecting monetary easing during uncertain or crisis periods.

Overall, these plots highlight clear linkages between inflation expectations and broader economic conditions, underscoring how inflation dynamics interact with economic growth, monetary policy, and consumer prices.

```
[ ]: # Show Correlations Between Economic Indicators
# Correlation heatmap
plt.figure(figsize=(12, 8))
corr_matrix = df.corr()
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
```

```
plt.title("Correlation Heatmap of Economic Indicators")
plt.show()
```



4.0.4 Correlation Analysis of Economic Indicators

The provided heatmap visualizes correlation coefficients between key economic indicators:

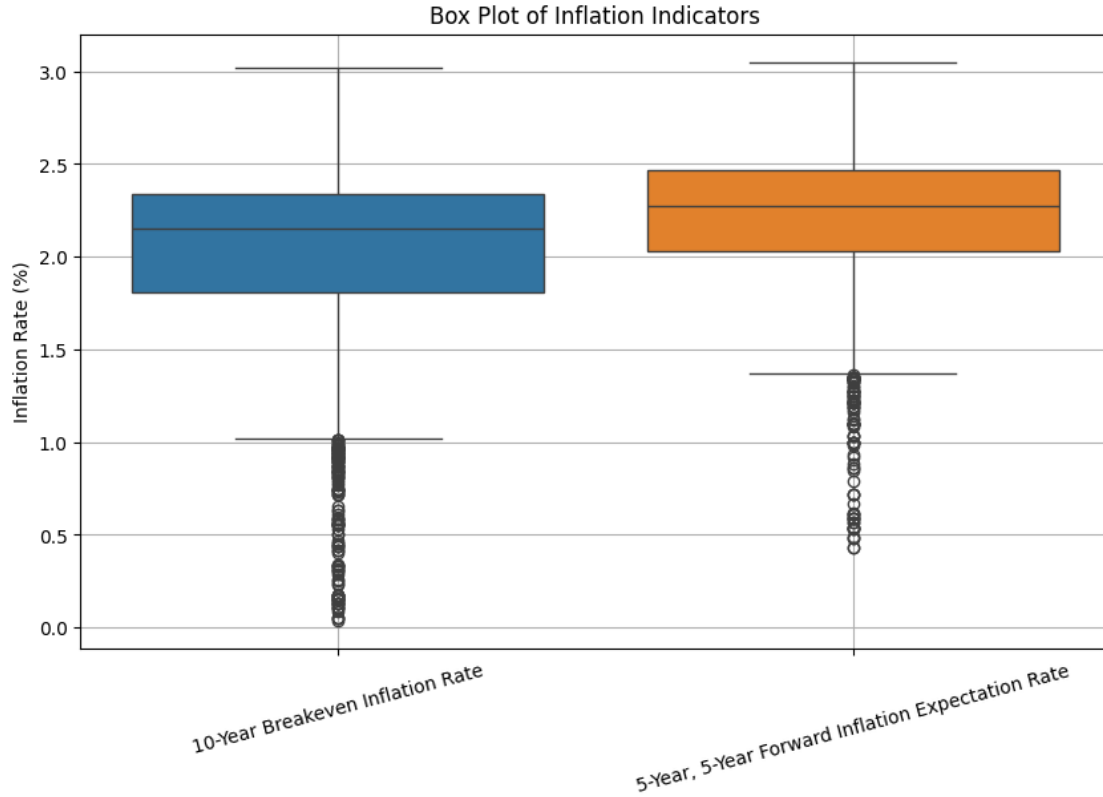
Key Findings:

- **Strong Positive Correlations:**
 - **Consumer Price Index (CPI)** and **Real GDP** (0.97): Indicates consumer prices typically rise alongside economic growth.
 - **Personal Consumption Expenditures Index** shows a strong correlation with both CPI (1.00) and GDP (0.97), suggesting close alignment of consumption spending with overall economic activity.
 - **Nominal U.S. Dollar Index** has strong positive correlations with CPI (0.82) and GDP (0.88), implying economic growth is typically associated with a strengthening dollar.
- **Moderate Positive Correlations:**

- **Crude Oil Prices** are positively correlated with inflation expectations (0.74) and 10-Year Breakeven Inflation Rate (0.67), highlighting oil prices as a key driver of inflation expectations.
- **10-Year Treasury Yield** moderately correlates with the Federal Funds Rate (0.76) and inflation expectations (0.46), illustrating the relationship between interest rates and market inflation forecasts.
- **Negative Correlations:**
 - **Unemployment Rate** negatively correlates with Real GDP (-0.59), CPI (-0.46), and Federal Funds Rate (-0.57), reflecting economic downturns associated with high unemployment.
 - **Nominal U.S. Dollar Index** negatively correlates with 5-Year Inflation Expectations (-0.55), indicating a stronger dollar tends to correspond with lower inflation expectations.
- **Notable Low Correlations:**
 - **10-Year Breakeven Inflation Rate** shows weak correlation with CPI (0.13) and GDP (0.08), indicating immediate inflation measures may not directly reflect longer-term inflation expectations.

Overall, this correlation heatmap highlights key relationships among inflation measures, economic growth, monetary policy indicators, and market expectations, providing insight into interconnected economic dynamics.

```
[ ]: # Analyze Distribution of Inflation Indicators
# Box plot for inflation rates
plt.figure(figsize=(10, 6))
sns.boxplot(data=data[["10-Year Breakeven Inflation Rate", "5-Year, 5-Year_
↳Forward Inflation Expectation Rate"]])
plt.title("Box Plot of Inflation Indicators")
plt.ylabel("Inflation Rate (%)")
plt.xticks(rotation=15)
plt.grid(True)
plt.show()
```



4.0.5 Box Plot Analysis of Inflation Indicators

The provided box plot illustrates the distribution characteristics of two inflation measures:

- **10-Year Breakeven Inflation Rate** (blue)
- **5-Year, 5-Year Forward Inflation Expectation Rate** (orange)

4.0.6 Key Findings:

- **Median Inflation Rate**
 - The **5-Year Forward Inflation Expectation Rate** has a slightly higher median (around 2.25%) compared to the **10-Year Breakeven Inflation Rate** (around 2.0%).
- **Interquartile Range (IQR)**
 - The **10-Year Breakeven Inflation Rate** shows a wider IQR, indicating greater variability and uncertainty in short-term inflation expectations.
 - The **5-Year Forward Inflation Expectation Rate** has a narrower IQR, suggesting more stable longer-term inflation expectations.
- **Outliers**
 - Both indicators contain multiple outliers at the lower end (below approximately 1%), indicating periods of deflationary pressures or significant economic disruptions.

- **Range of Inflation Rates**

- Both inflation measures generally range from about 0% to 3%, with most observations concentrated between 1.5% and 2.5%.

This analysis highlights the key differences in stability and variability between short-term (10-Year Breakeven) and long-term (5-Year Forward) inflation expectations.

```
[ ]: # Violin plot for inflation rates
plt.figure(figsize=(10, 6))
sns.violinplot(data=data[["10-Year Breakeven Inflation Rate", "5-Year, 5-Year_
↳Forward Inflation Expectation Rate"]])
plt.title("Violin Plot of Inflation Indicators")
plt.ylabel("Inflation Rate (%)")
plt.xticks(rotation=15)
plt.grid(True)
plt.show()
```



4.0.7 Violin Plot Analysis of Inflation Indicators

The provided violin plot compares the distribution and density of two inflation measures:

- **10-Year Breakeven Inflation Rate** (blue)

- **5-Year, 5-Year Forward Inflation Expectation Rate** (orange)
-

4.0.8 Key Findings:

- **Distribution Shape**
 - Both indicators have distributions concentrated around 2%, highlighting typical inflation expectations aligned closely with central-bank targets.
 - The **10-Year Breakeven Inflation Rate** shows greater density toward lower values (around 1%), suggesting more frequent periods of short-term deflationary pressure or uncertainty.
 - **Median and Central Tendency**
 - The **5-Year Forward Inflation Expectation Rate** has a slightly higher median (around 2.25%) compared to the **10-Year Breakeven Inflation Rate** (approximately 2.0%), indicating somewhat higher expectations for medium-term inflation.
 - **Variability**
 - The **10-Year Breakeven Inflation Rate** displays a wider spread, especially towards lower inflation values, reflecting greater short-term uncertainty or volatility.
 - The **5-Year Forward Inflation Expectation Rate** is more symmetrically distributed and narrower, suggesting greater market confidence and stability in medium-term inflation forecasts.
 - **Extreme Values**
 - Both measures have thin tails extending towards zero inflation, signifying occasional episodes of significant deflationary concerns or economic shocks.
-

Overall, this violin plot effectively illustrates differences in distribution shape, median values, and variability, clearly distinguishing short-term inflation uncertainty from more stable, medium-term inflation expectations.

```
[ ]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Rename columns for better visibility
rename_dict = {
    "10-Year Breakeven Inflation Rate": "T10YIE",
    "5-Year, 5-Year Forward Inflation Expectation Rate": "T5YIFR",
    "Consumer Price Index": "CPIAUCNS",
    "Real Gross Domestic Product": "GDPC1",
    "Federal Funds Effective Rate": "FEDFUNDS"
}

# Select numeric columns and rename them
numeric_columns = list(rename_dict.keys()) # Original names
df_renamed = df[numeric_columns].rename(columns=rename_dict) # Rename columns
```

```

# Drop missing values
df_cleaned = df_renamed.dropna()

# Set Seaborn style
sns.set(style="whitegrid")

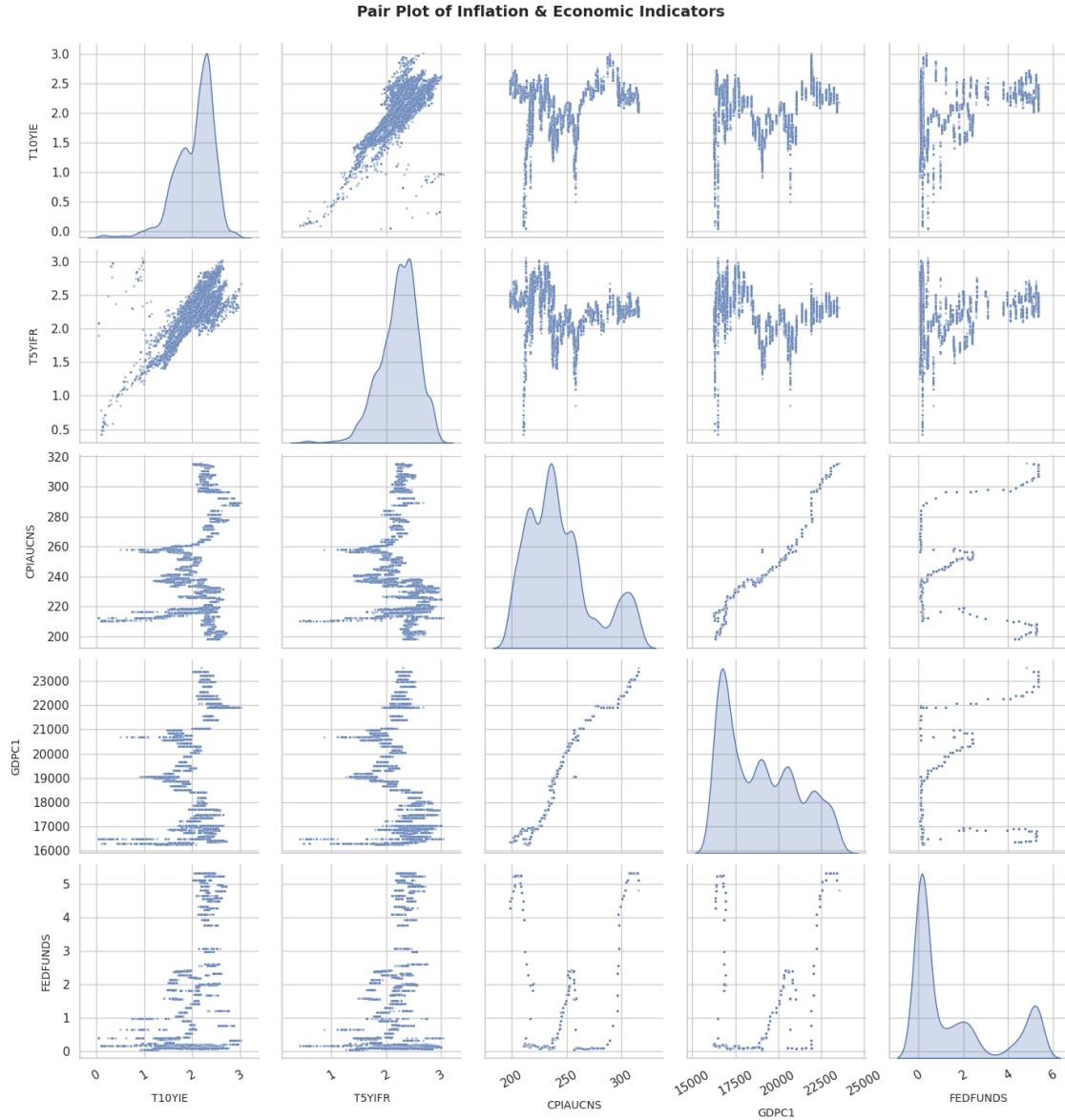
# Create pair plot with renamed columns
pair_plot = sns.pairplot(df_cleaned,
                        diag_kind="kde", # KDE for smoother distributions
                        height=2.8, # Increase figure height
                        plot_kws={"s": 5, "alpha": 0.5}) # Adjust marker size
↳ transparency

# Adjust labels for better visibility
for ax in pair_plot.axes.flatten():
    if ax is not None:
        ax.set_xlabel(ax.get_xlabel(), fontsize=10, labelpad=10) # Increase
↳ spacing
        ax.set_ylabel(ax.get_ylabel(), fontsize=10, labelpad=10)
        ax.tick_params(axis='x', labelrotation=30) # Rotate x-axis labels

# Add a title
plt.suptitle("Pair Plot of Inflation & Economic Indicators", fontsize=14,
↳ fontweight='bold', y=1.02)

plt.show()

```



4.0.9 Pair Plot Analysis of Inflation & Economic Indicators

The provided pair plot visualizes distributions and relationships among five economic indicators:

- **T10YIE**: 10-Year Breakeven Inflation Rate
- **T5YIFR**: 5-Year, 5-Year Forward Inflation Expectation Rate
- **CPIAUCNS**: Consumer Price Index
- **GDPC1**: Real Gross Domestic Product (GDP)

- **FEDFUNDS:** Federal Funds Effective Rate
-

4.0.10 Key Findings:

- **Inflation Indicators (T10YIE & T5YIFR):**
 - T10YIE and T5YIFR exhibit a strong positive linear correlation, indicating short-term and medium-term inflation expectations move closely together.
 - Both measures typically center around 2%, aligning with central bank inflation targets.
 - **Consumer Price Index (CPI):**
 - CPI shows a clear positive linear relationship with Real GDP, indicating prices generally rise with economic growth.
 - CPI exhibits weaker relationships with inflation expectation indicators, indicating inflation expectations don't always align directly with observed consumer price changes.
 - **Real Gross Domestic Product (GDP):**
 - GDP demonstrates strong linear correlations with CPI, reflecting consistent economic growth alongside increasing consumer prices.
 - GDP has limited direct correlation with inflation expectation indicators (T10YIE & T5YIFR), suggesting economic growth alone does not directly predict market inflation forecasts.
 - **Federal Funds Effective Rate (FEDFUNDS):**
 - FEDFUNDS shows scattered relationships with both inflation expectation measures, reflecting how monetary policy adjustments vary significantly based on economic context.
 - FEDFUNDS exhibits nonlinear relationships with CPI and GDP, indicating complex interactions between monetary policy, economic growth, and price levels.
 - **Distributions:**
 - Inflation measures (T10YIE and T5YIFR) have relatively symmetrical distributions around 2%.
 - CPI and GDP distributions are less symmetric and show multimodal patterns, indicating varying economic cycles or structural shifts.
-

Overall, the pair plot provides clear insights into the intricate interactions between inflation expectations, monetary policy, consumer prices, and economic growth.

```
[ ]: # Create Interactive Time-Series Plot
import plotly.express as px

# Interactive plot for Inflation over time
fig = px.line(data, x=data.index, y=["10-Year Breakeven Inflation Rate",
    ↪ "5-Year, 5-Year Forward Inflation Expectation Rate"],
              title="Interactive Inflation Trends", labels={"value": "Inflation",
    ↪ "Rate (%)", "variable": "Inflation Type"})
fig.show()
```

```
[ ]: # Define inflation-related variables
inflation_vars = [
```

```

    "10-Year Breakeven Inflation Rate",
    "5-Year, 5-Year Forward Inflation Expectation Rate",
    "Consumer Price Index",
    "Personal Consumption Expenditures: Chain-type Price Index"
]

# Define economic indicators
economic_indicators = [
    "Real Gross Domestic Product",
    "Unemployment Rate",
    "Federal Funds Effective Rate",
    "Nominal Broad U.S. Dollar Index",
    "Crude Oil Prices: West Texas Intermediate",
    "10-Year Treasury Yield"
]

```

```

[ ]: # Annual Consumer Price Index (CPI) Bar Chart
import seaborn as sns
import warnings

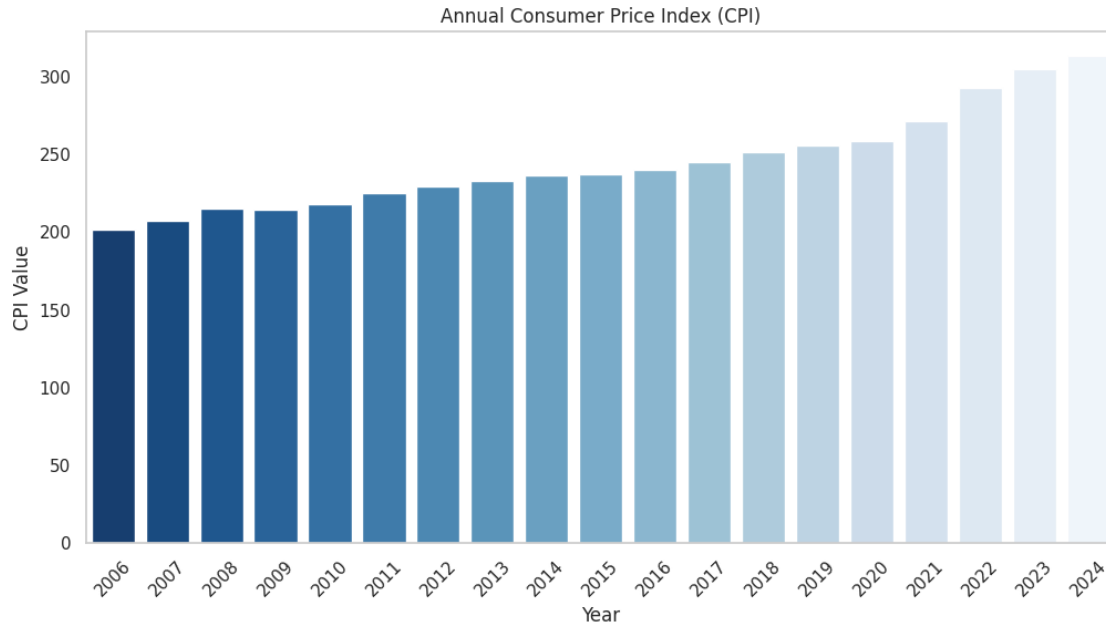
# Suppress all FutureWarnings
warnings.simplefilter(action='ignore', category=FutureWarning)

# Extract year from date
df['Year'] = df.index.year

# Compute annual average CPI
annual_cpi = df.groupby('Year')['Consumer Price Index'].mean()

# Plot bar chart
plt.figure(figsize=(12, 6))
sns.barplot(x=annual_cpi.index, y=annual_cpi.values, palette='Blues_r')
plt.title("Annual Consumer Price Index (CPI)")
plt.xlabel("Year")
plt.ylabel("CPI Value")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.show()

```



4.0.11 Analysis of Annual Consumer Price Index (CPI)

The provided bar chart illustrates the **annual changes in the Consumer Price Index (CPI)** from **2006 to 2024**.

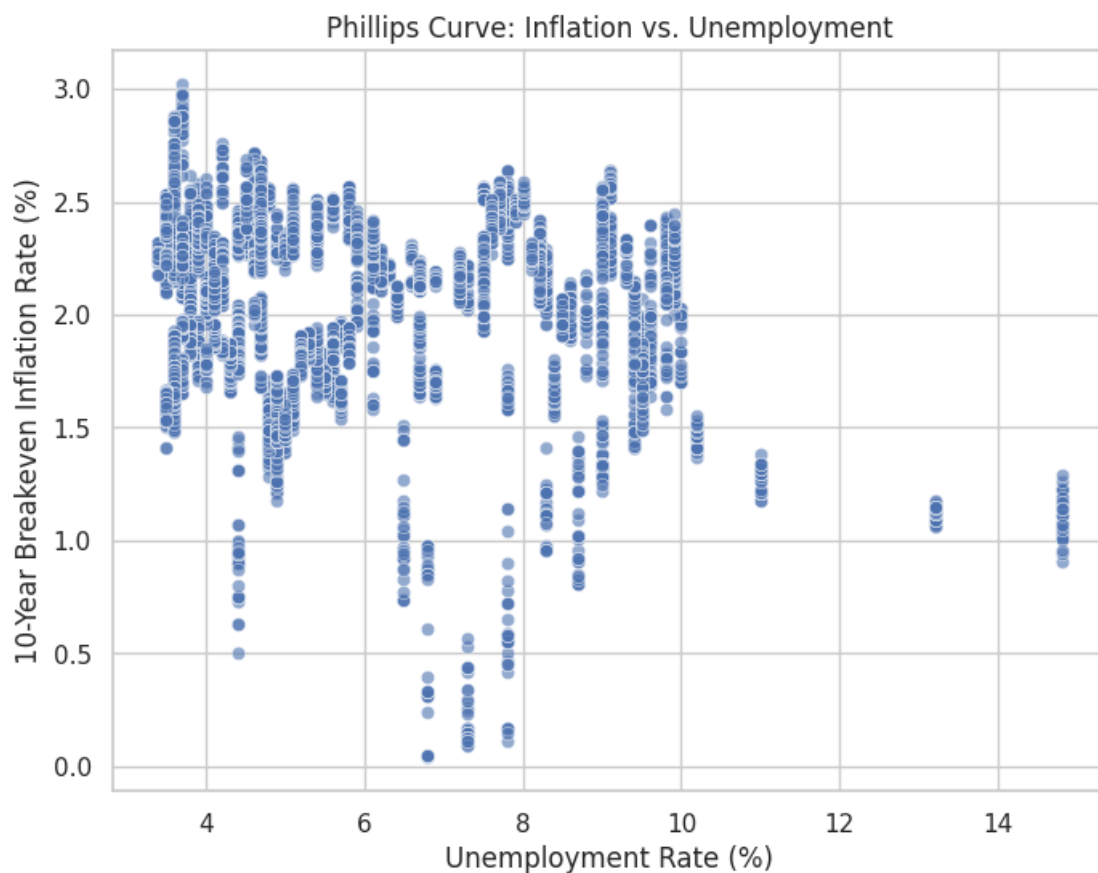
4.0.12 Key Findings:

- **Consistent Growth:**
 - The CPI value has consistently increased each year, reflecting ongoing inflation and rising consumer prices over nearly two decades.
- **Trend Acceleration:**
 - The growth rate of CPI appears steeper from around 2020 onwards, suggesting increased inflationary pressures during recent years.
- **Historical CPI Levels:**
 - CPI values started around 200 in 2006 and surpassed the 300-mark by 2024, representing a significant overall increase (~50% increase).
- **Color Intensity:**
 - Darker bars (earlier years) gradually fade to lighter shades (recent years), visually emphasizing the progression of time and increasing CPI.

This chart effectively highlights the steady and accelerating inflation trend over the analyzed period, underscoring long-term growth in consumer prices.

```
[ ]: # Phillips Curve (Inflation vs. Unemployment Scatter Plot)
import seaborn as sns

# Scatter plot (Phillips Curve)
plt.figure(figsize=(8, 6))
sns.scatterplot(x=df["Unemployment Rate"], y=df["10-Year Breakeven Inflation_Rate"], alpha=0.6)
plt.title("Phillips Curve: Inflation vs. Unemployment")
plt.xlabel("Unemployment Rate (%)")
plt.ylabel("10-Year Breakeven Inflation Rate (%)")
plt.grid(True)
plt.show()
```



4.0.13 Phillips Curve Analysis: Inflation vs. Unemployment

The provided scatter plot illustrates the relationship between:

- **10-Year Breakeven Inflation Rate** (y-axis)
- **Unemployment Rate** (x-axis)

4.0.14 Key Findings:

- **Inverse Relationship:**
 - There is a general inverse relationship, consistent with the Phillips Curve concept, indicating higher inflation rates tend to coincide with lower unemployment rates.
- **Inflation Clustering:**
 - Higher inflation rates (around 2% to 3%) predominantly occur at lower unemployment rates (approximately 3%–6%), signaling stronger economic activity.
 - Lower inflation rates (below 1.5%) become more frequent at higher unemployment rates (above 6%), highlighting weakened economic conditions.
- **Unemployment Extremes:**
 - Very high unemployment rates (above 10%) are associated exclusively with lower inflation rates (below 1.5%), reflecting significant economic downturns or recessions.
- **Variability and Exceptions:**
 - There is considerable variability within the mid-range (4%–8% unemployment), suggesting the relationship isn't perfectly linear and other factors influence inflation expectations.

Overall, this chart clearly demonstrates the Phillips Curve's concept—showing how inflation expectations generally decrease as unemployment increases, though with noticeable variability.

```
[ ]: # Time Series Decomposition (CPI)
from statsmodels.tsa.seasonal import seasonal_decompose

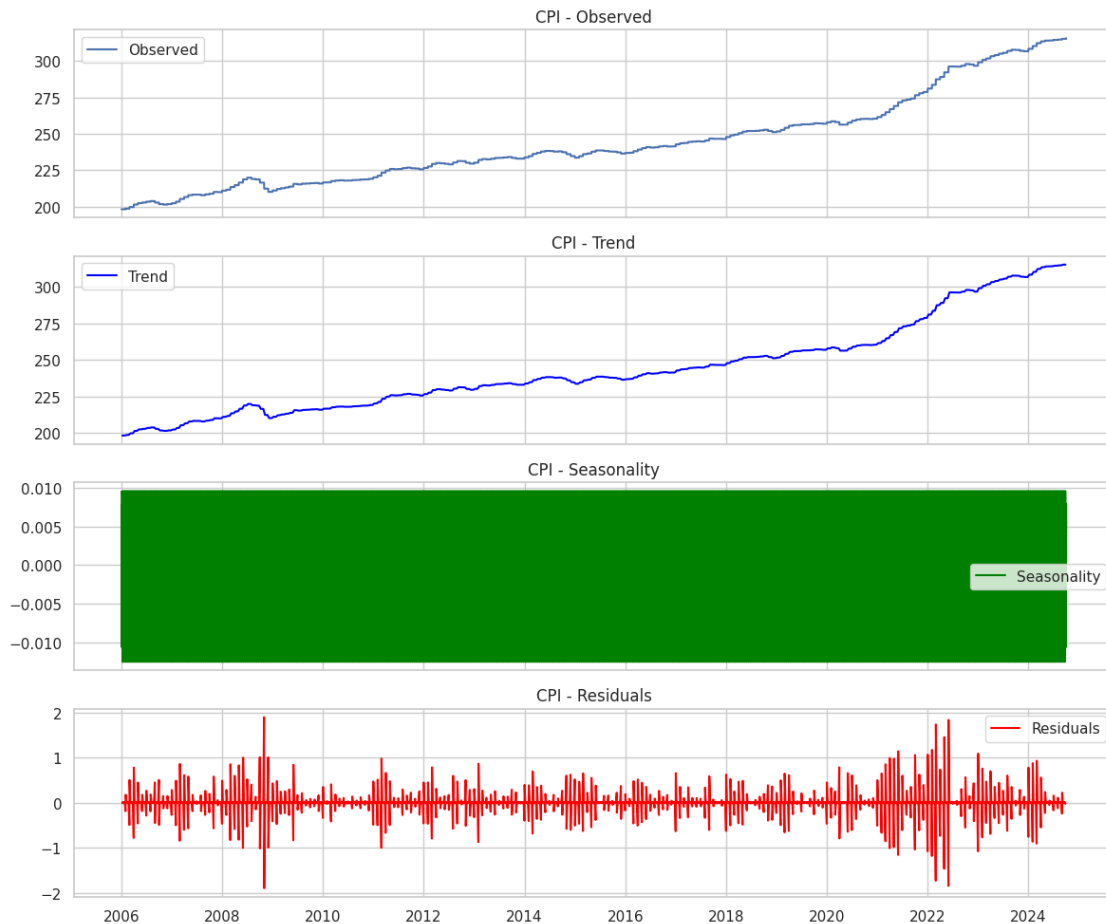
# Perform time series decomposition
cpi_decomposed = seasonal_decompose(df["Consumer Price Index"].dropna(),
    ↪model='additive', period=12)

# Plot Decomposition
fig, axes = plt.subplots(4, 1, figsize=(12, 10), sharex=True)
axes[0].plot(cpi_decomposed.observed, label='Observed')
axes[0].set_title("CPI - Observed")
axes[1].plot(cpi_decomposed.trend, label='Trend', color='blue')
axes[1].set_title("CPI - Trend")
axes[2].plot(cpi_decomposed.seasonal, label='Seasonality', color='green')
axes[2].set_title("CPI - Seasonality")
axes[3].plot(cpi_decomposed.resid, label='Residuals', color='red')
axes[3].set_title("CPI - Residuals")

# Improve plot layout
for ax in axes:
    ax.legend()
    ax.grid(True)

plt.tight_layout()
```

```
plt.show()
```



4.0.15 Time Series Decomposition Analysis of CPI

The provided charts illustrate a decomposed time series analysis of the **Consumer Price Index (CPI)** from 2006 to 2024. The decomposition separates observed data into three components: **Trend**, **Seasonality**, and **Residuals**.

4.0.16 Key Findings:

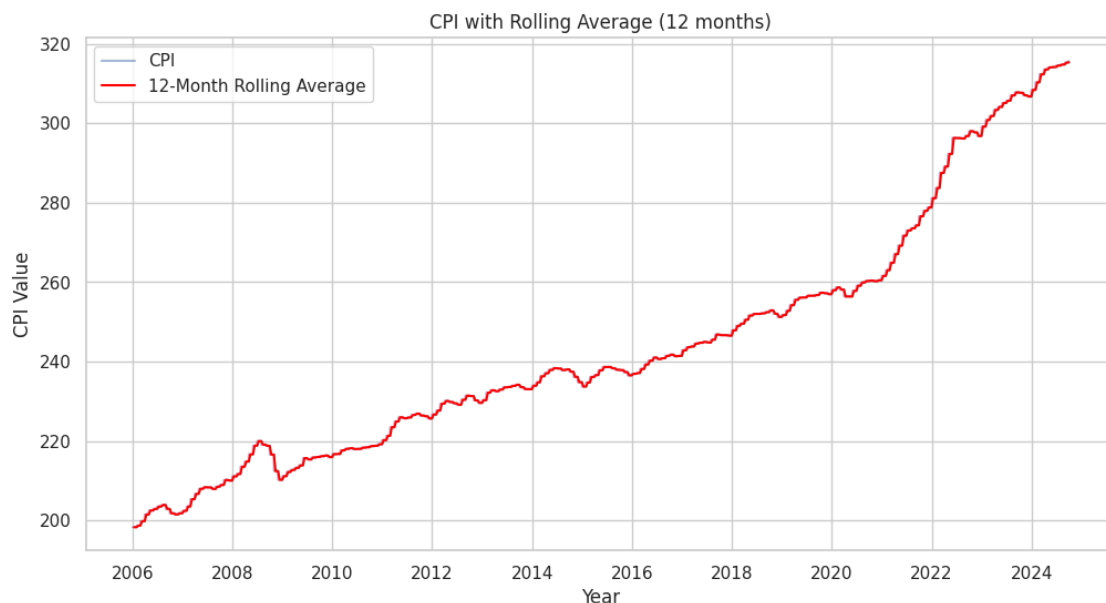
- **Observed CPI:**
 - Displays a clear upward trend over the observed period, reflecting consistent long-term inflation.
- **Trend Component:**
 - Highlights a smooth, increasing trajectory of CPI, becoming steeper from approximately 2020 onwards, indicating accelerated inflation in recent years.
- **Seasonality Component:**

- Extremely minimal seasonal variation, consistently close to zero, indicating negligible seasonal impacts on CPI values.
- **Residuals Component:**
 - Exhibits random fluctuations around zero, suggesting short-term volatility or irregular economic events impacting CPI.
 - Notable spikes in residuals around significant economic events (e.g., 2008 and post-2020), indicating increased volatility during these periods.

Overall, the decomposition effectively shows a strong and consistent inflationary trend, minimal seasonal effects, and highlights irregular volatility corresponding to economic shocks or disruptions.

```
[ ]: # Rolling Average of CPI
# Compute 12-month rolling average for CPI
df['CPI_Rolling'] = df['Consumer Price Index'].rolling(window=12).mean()

# Plot CPI with rolling average
plt.figure(figsize=(12, 6))
plt.plot(df.index, df['Consumer Price Index'], label="CPI", alpha=0.5)
plt.plot(df.index, df['CPI_Rolling'], label="12-Month Rolling Average",
         color='red')
plt.title("CPI with Rolling Average (12 months)")
plt.xlabel("Year")
plt.ylabel("CPI Value")
plt.legend()
plt.grid(True)
plt.show()
```



4.0.17 CPI with 12-Month Rolling Average Analysis

The provided line chart illustrates the **Consumer Price Index (CPI)** alongside its **12-month rolling average** from 2006 to 2024.

4.0.18 Key Findings:

- **Overall Trend:**
 - CPI shows a clear and steady upward trend, indicating consistent inflation over the entire period.
 - **Rolling Average:**
 - The 12-month rolling average (red line) smooths out short-term fluctuations, highlighting a clearer long-term growth pattern.
 - **Acceleration in CPI Growth:**
 - A noticeable increase in the slope of the rolling average appears around 2020, indicating an accelerated rise in inflation in recent years.
 - **Stability and Fluctuations:**
 - Minor fluctuations around the rolling average indicate temporary economic variations or short-term shocks, especially evident around economic events (e.g., 2008, 2020).
-

Overall, the 12-month rolling average effectively reveals the long-term upward trend of CPI, clearly indicating sustained and recently accelerated inflation.

```
[ ]: import plotly.graph_objects as go

# Create figure
fig = go.Figure()

# Add Inflation (Left Y-Axis)
fig.add_trace(go.Scatter(x=df.index, y=df["10-Year Breakeven Inflation Rate"],
                        mode='lines',
                        name='10-Year Breakeven Inflation Rate',
                        yaxis='y1'))

# Add GDP (Right Y-Axis)
fig.add_trace(go.Scatter(x=df.index, y=df["Real Gross Domestic Product"],
                        mode='lines',
                        name='Real Gross Domestic Product',
                        yaxis='y2', line=dict(color='red'))

# Update layout for dual y-axes
fig.update_layout(
    title="Inflation vs. GDP Over Time",
    xaxis=dict(title="Year"),
    yaxis=dict(
        title="Inflation Rate (%)",
```

```

        side='left', showgrid=False),
    yaxis2=dict(
        title="Real GDP (Billions)",
        overlaying='y', side='right', showgrid=False),
    legend=dict(x=1, y=1)
)

# Show interactive visualization
fig.show()

```

4.0.19 Inflation vs. GDP Over Time

The provided dual-axis time series chart compares:

- **10-Year Breakeven Inflation Rate** (blue line, left axis)
- **Real Gross Domestic Product (GDP)** (red line, right axis)

4.0.20 Key Findings:

- **Inflation Rate Trends:**
 - The inflation rate generally fluctuates between 1.5% and 2.5%, with notable volatility during economic downturns, particularly around 2008 and 2020.
 - Significant dips in inflation occur during major economic crises, indicating deflationary pressures.
- **Real GDP Trends:**
 - Real GDP shows a clear, steady upward trend over time, reflecting long-term economic growth.
 - Distinct slowdowns or plateaus in GDP growth occur around 2008–2009 and again in 2020, corresponding with economic disruptions.
- **Relationship Between Inflation and GDP:**
 - Periods of strong GDP growth often align with relatively stable inflation rates around 2%, signifying balanced economic conditions.
 - Sharp GDP contractions, especially visible around 2020, align closely with rapid declines in inflation expectations.
- **Recent Trends:**
 - After the significant downturn in 2020, both GDP and inflation rates recovered swiftly, suggesting a quick economic rebound and rising inflation pressures post-crisis.

Overall, the chart effectively demonstrates how inflation expectations respond dynamically to shifts in economic growth, highlighting periods of economic stability, volatility, and recovery.

```

[ ]: import seaborn as sns
import matplotlib.pyplot as plt

# Bubble Chart: Inflation vs GDP vs Unemployment
plt.figure(figsize=(10, 6))

```

```

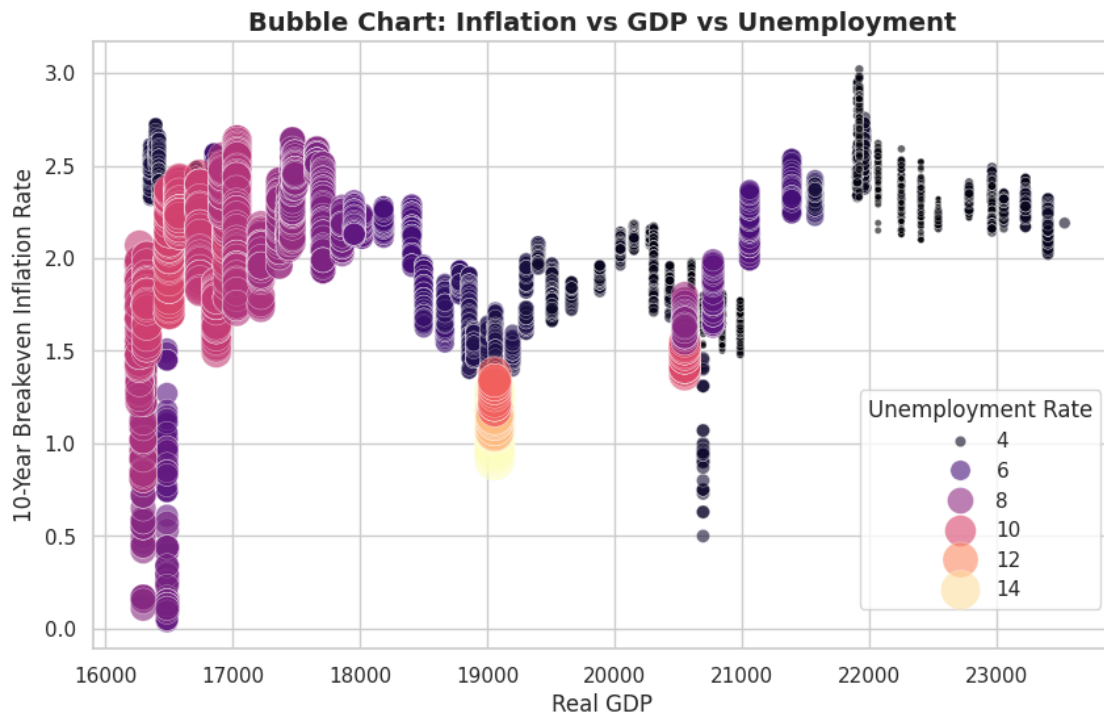
bubble_chart = sns.scatterplot(
    x=df["Real Gross Domestic Product"],
    y=df["10-Year Breakeven Inflation Rate"],
    size=df["Unemployment Rate"],
    hue=df["Unemployment Rate"],
    alpha=0.6, sizes=(10, 500), palette="magma"
)

# Adjust titles and labels
plt.title("Bubble Chart: Inflation vs GDP vs Unemployment", fontsize=14,
        ↪fontweight='bold')
plt.xlabel("Real GDP", fontsize=12)
plt.ylabel("10-Year Breakeven Inflation Rate", fontsize=12)
plt.grid(True)

# Move the legend to the lower right
plt.legend(title="Unemployment Rate", loc="lower right", bbox_to_anchor=(1, 0.
        ↪05))

# Show plot
plt.show()

```



4.0.21 Bubble Chart Analysis: Inflation vs GDP vs Unemployment

The provided bubble chart illustrates the relationships among three key economic indicators simultaneously:

- **10-Year Breakeven Inflation Rate** (vertical axis)
 - **Real Gross Domestic Product (GDP)** (horizontal axis)
 - **Unemployment Rate** (bubble color and size)
-

4.0.22 Key Findings:

- **Relationship Between Inflation and GDP:**
 - Higher Real GDP values (above approximately \$21,000 billion) generally correspond with moderate inflation rates (around 1.5%–2.5%), indicating economic stability during growth periods.
 - Lower Real GDP values (below approximately \$18,000 billion) show greater variability in inflation rates, ranging from near 0% to around 3%, reflecting increased volatility and uncertainty during economic downturns.
 - **Impact of Unemployment:**
 - Lower unemployment rates (4%–6%, represented by smaller, darker bubbles) mostly occur at higher GDP levels with stable and moderate inflation, suggesting favorable economic conditions.
 - Higher unemployment rates (10%–14%, represented by larger, lighter bubbles) appear mainly at lower GDP levels alongside lower inflation rates, highlighting periods of economic distress or recessions.
 - **Economic Stability and Inflation:**
 - Inflation rates tend to be more stable during periods of economic expansion (high GDP, low unemployment).
 - Inflation exhibits greater volatility at lower GDP levels, coinciding with higher unemployment and weaker economic conditions.
-

Overall, the bubble chart clearly visualizes the interconnected dynamics between inflation, economic growth, and unemployment rates.

```
[ ]: import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Extract year and month for visualization
df["Year"] = df.index.year
df["Month"] = df.index.month

# Pivot the data for heatmap
```

```

cpi_heatmap = df.pivot_table(values="Consumer Price Index", index="Month",
    ↪columns="Year")

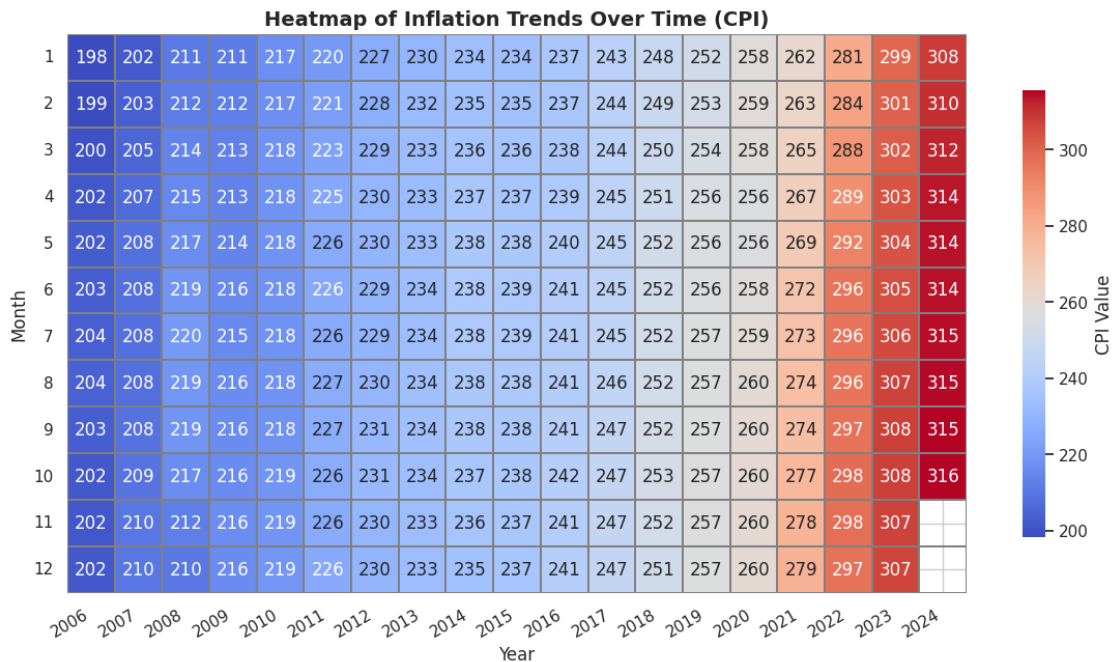
# Create heatmap with improved aesthetics
plt.figure(figsize=(14, 7))
sns.heatmap(cpi_heatmap,
            cmap="coolwarm", # Improved color scheme for better contrast
            annot=True, fmt=".0f", # Rounded annotation values for clarity
            linewidths=0.3, linecolor="gray", # Subtle gridlines for a cleaner
    ↪look
            cbar_kws={"shrink": 0.8, "label": "CPI Value"}) # Adjust color bar
    ↪size

# Improve title and labels
plt.title("Heatmap of Inflation Trends Over Time (CPI)", fontsize=14,
    ↪fontweight='bold')
plt.xlabel("Year", fontsize=12)
plt.ylabel("Month", fontsize=12)

# Rotate x-axis labels slightly for better readability
plt.xticks(rotation=30, ha="right")
plt.yticks(rotation=0) # Keep month labels readable

# Show the heatmap
plt.show()

```



4.0.23 Heatmap Analysis of Inflation Trends (CPI)

The provided heatmap visually represents monthly **Consumer Price Index (CPI)** values from **2006 to 2024**.

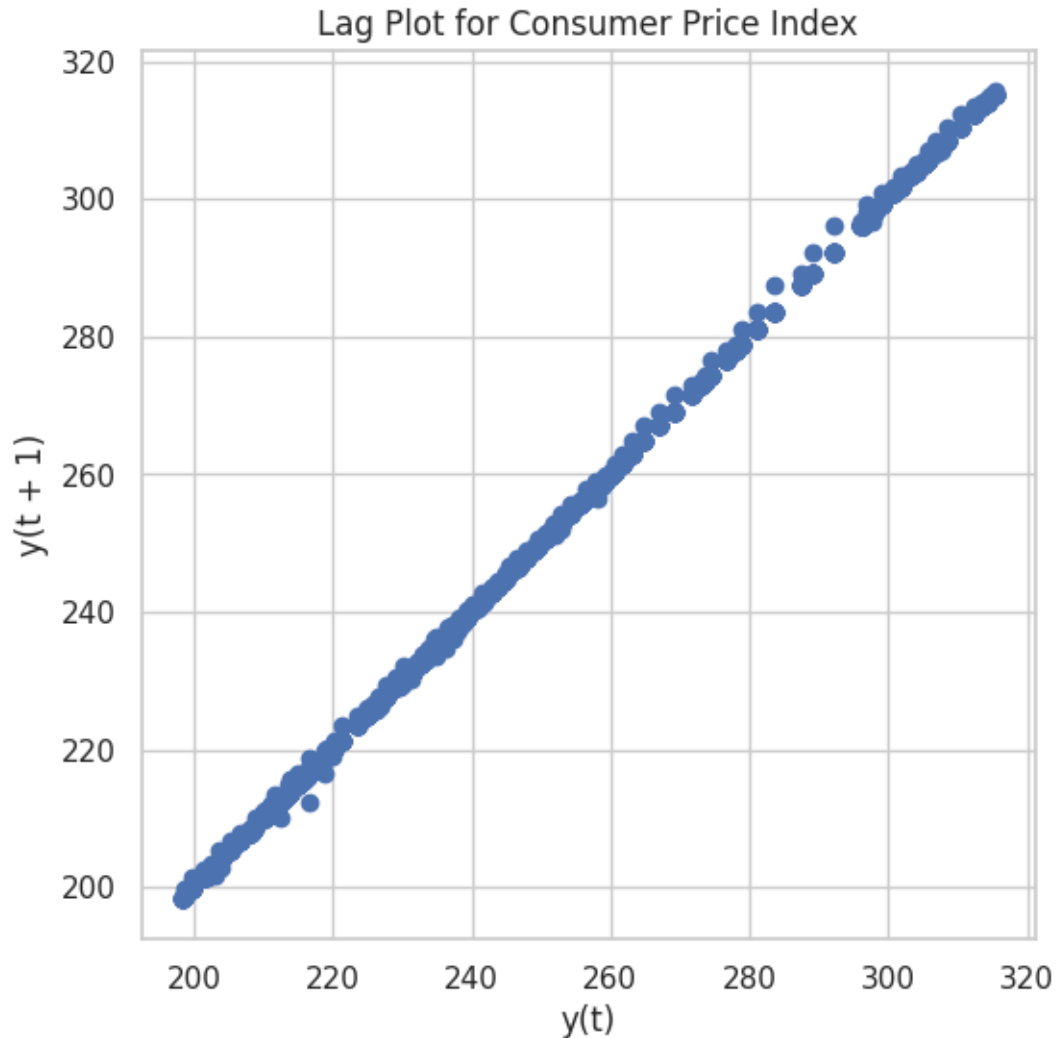
4.0.24 Key Findings:

- **Consistent Increasing Trend:**
 - CPI consistently increases over the years, progressing from lower values (around 200 in 2006) to significantly higher values (above 310 by 2024), clearly showing long-term inflation.
 - **Acceleration of Inflation:**
 - The rate of CPI growth visibly accelerates starting from approximately **2020**, demonstrated by a sharper transition from lighter (blue) to darker (red) shades.
 - **Seasonal Patterns:**
 - Minor variations are observable within each year, but overall seasonality appears very minimal, with no significant monthly fluctuations or patterns evident.
 - **Recent High Inflation:**
 - The most intense inflationary pressures, represented by the darkest red shades, occur predominantly from **2022 onwards**, reflecting recent higher inflation rates.
-

Overall, the heatmap effectively highlights the steady and recently accelerating inflation trends over the analyzed period.

```
[ ]: # Lag Plot for Inflation
from pandas.plotting import lag_plot

# Lag plot to see if inflation data has patterns
plt.figure(figsize=(6, 6))
lag_plot(df["Consumer Price Index"])
plt.title("Lag Plot for Consumer Price Index")
plt.grid(True)
plt.show()
```



4.0.25 Lag Plot Analysis of Consumer Price Index (CPI)

The provided lag plot visualizes the relationship between the CPI values at time t (**horizontal axis**) and CPI values at the next period $t+1$ (**vertical axis**).

4.0.26 Key Findings:

- **Strong Positive Correlation:**
 - The data points clearly form a linear pattern, indicating a strong positive autocorrelation in CPI values from one period to the next.
- **Predictability:**
 - The CPI at any given time (t) strongly predicts the CPI at the following time ($t+1$), highlighting the stability and continuity of inflation trends.
- **Minimal Randomness:**

- The tight linear clustering suggests minimal short-term random fluctuations, emphasizing persistent inflation trends over time.
- **Consistent Inflation Growth:**
 - The upward-sloping line reflects consistently rising CPI values, indicating ongoing inflation.

Overall, the lag plot demonstrates that CPI values exhibit strong temporal dependence and consistent upward movement, reflecting stable, long-term inflationary patterns.

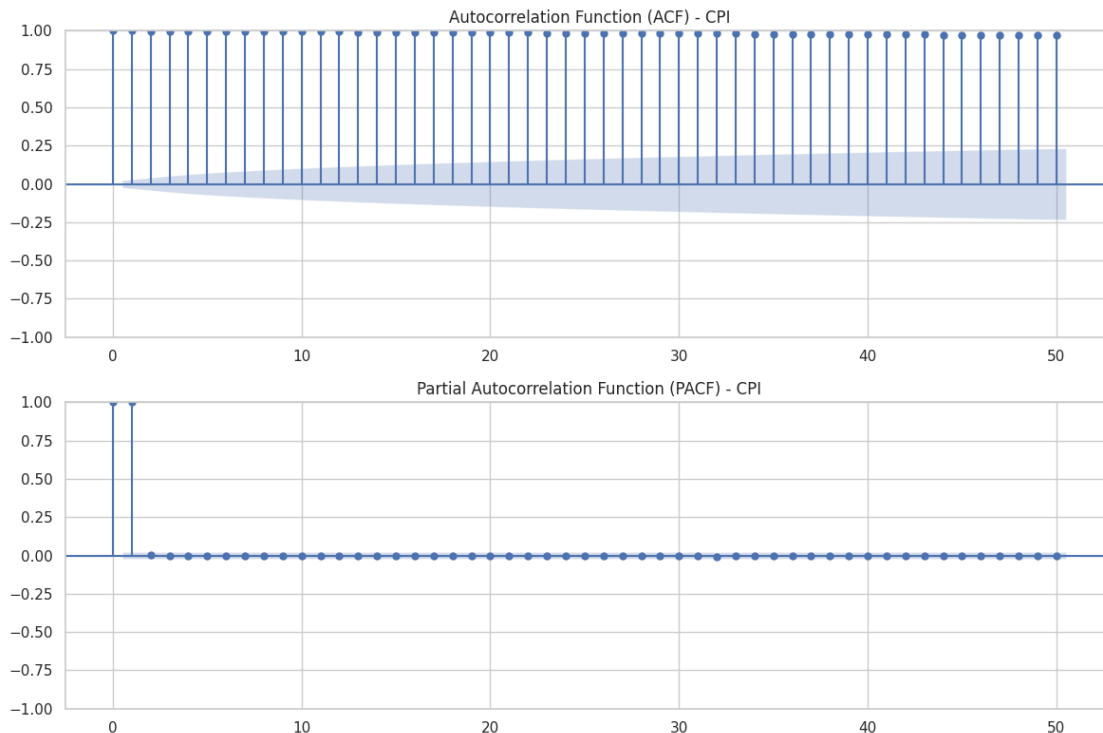
```
[ ]: # Autocorrelation Function (ACF) & Partial ACF (PACF) Plots
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Plot ACF and PACF for CPI
fig, ax = plt.subplots(2, 1, figsize=(12, 8))

plot_acf(df["Consumer Price Index"].dropna(), lags=50, ax=ax[0])
ax[0].set_title("Autocorrelation Function (ACF) - CPI")

plot_pacf(df["Consumer Price Index"].dropna(), lags=50, ax=ax[1])
ax[1].set_title("Partial Autocorrelation Function (PACF) - CPI")

plt.tight_layout()
plt.show()
```



4.0.27 Autocorrelation and Partial Autocorrelation Analysis of CPI

The provided charts display the **Autocorrelation Function (ACF)** and **Partial Autocorrelation Function (PACF)** for the Consumer Price Index (CPI).

4.0.28 Key Findings:

- **Autocorrelation Function (ACF):**
 - Exhibits significant positive autocorrelations at multiple lags, slowly declining over time.
 - Indicates strong persistence and a clear trend, suggesting that past CPI values strongly influence future CPI values.
 - **Partial Autocorrelation Function (PACF):**
 - Shows significant spikes only at the initial lags (lags 1 and 2), with values quickly dropping to near zero afterward.
 - Suggests that CPI values can be effectively explained by their immediate past observations, with minimal direct influence from distant past observations once short-term dependencies are accounted for.
 - **Implications for CPI Modeling:**
 - The persistent high values in the ACF indicate that the CPI series is non-stationary, typically requiring differencing for modeling purposes.
 - PACF indicates an AR(1) or AR(2) process might effectively capture the CPI series behavior once differenced.
-

Overall, these plots highlight strong autocorrelation and short-term dependencies within CPI, indicating that past CPI values significantly influence future inflation trends.

```
[ ]: # Economic Indicators Comparison with Inflation (Radar Chart)
import numpy as np
import matplotlib.pyplot as plt
from math import pi

# Select key economic indicators
indicators = ["10-Year Breakeven Inflation Rate",
              "Consumer Price Index",
              "Unemployment Rate",
              "Federal Funds Effective Rate",
              "Real Gross Domestic Product"]

# Extract the latest values
latest_values = df[indicators].iloc[-1].values

# Normalize values for better visualization
max_values = df[indicators].max().values
normalized_values = latest_values / max_values # Normalize data

# Compute angles for each indicator
```

```

angles = np.linspace(0, 2 * np.pi, len(indicators), endpoint=False).tolist()
angles += angles[:1] # Close the circle

# Close the data circle for plotting
normalized_values = np.concatenate((normalized_values, [normalized_values[0]]))

# Create Radar Chart with increased figure size
fig, ax = plt.subplots(figsize=(9, 9), subplot_kw=dict(polar=True))

# Plot the filled area with better transparency
ax.fill(angles, normalized_values, color="red", alpha=0.3, label="Latest Data")
ax.plot(angles, normalized_values, color="darkred", linewidth=2,
        linestyle='solid')

# Improve label positioning
ax.set_xticks(angles[:-1])
ax.set_xticklabels(indicators, fontsize=12, fontweight='bold', color="black",
                   ha="center")

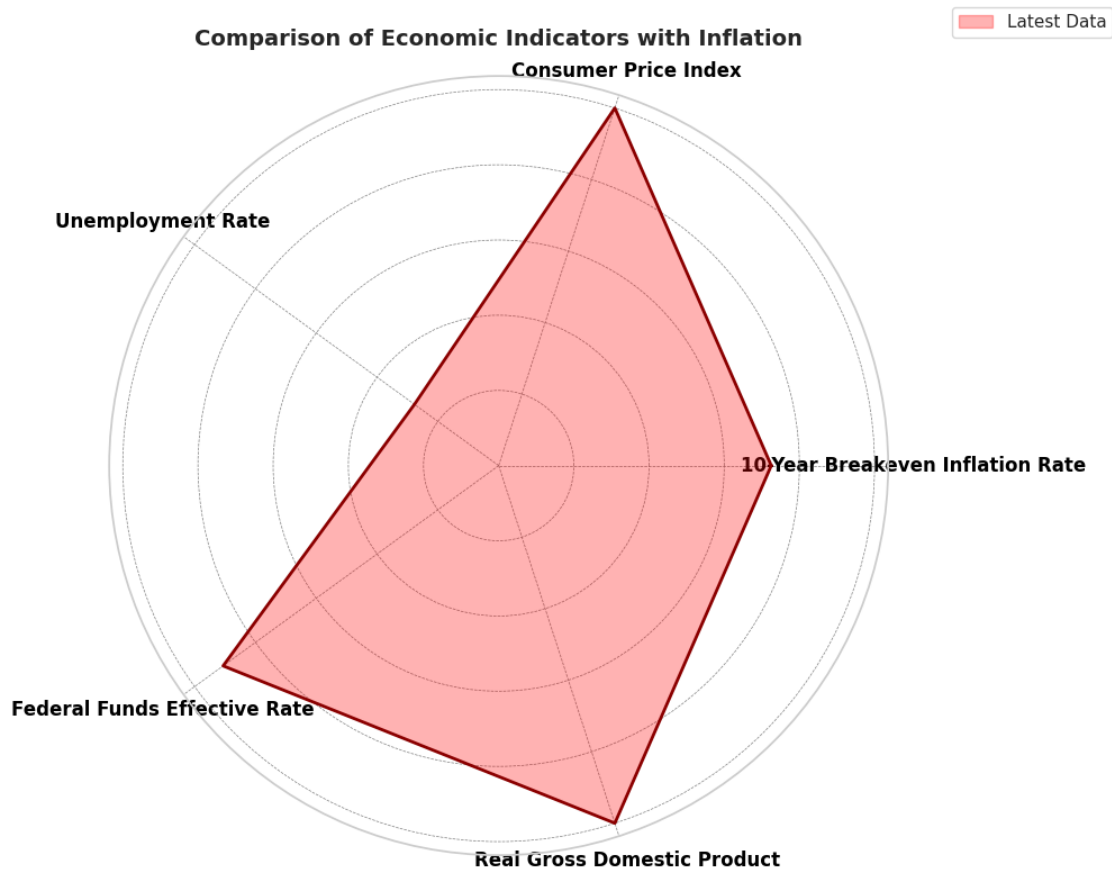
# Adjust radial labels for better readability
ax.set_yticklabels([])
ax.grid(color='gray', linestyle='dashed', linewidth=0.5)

# Title with better alignment
plt.title("Comparison of Economic Indicators with Inflation", fontsize=14,
         fontweight='bold', pad=20)

# Adjust legend placement to avoid overlap
ax.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1))

# Show the final improved radar chart
plt.show()

```



4.0.29 Radar Chart Analysis: Comparison of Economic Indicators with Inflation

The provided radar chart illustrates the most recent data for five critical economic indicators:

- **Consumer Price Index (CPI)**
- **10-Year Breakeven Inflation Rate**
- **Real Gross Domestic Product (GDP)**
- **Federal Funds Effective Rate**
- **Unemployment Rate**

4.0.30 Key Findings:

- **High CPI and Inflation:**
 - CPI and the 10-Year Breakeven Inflation Rate show high relative values, indicating significant current inflationary pressures.
- **Strong Economic Growth:**
 - Real GDP is positioned relatively high, reflecting robust economic activity at the latest measurement period.
- **Moderate Interest Rates:**

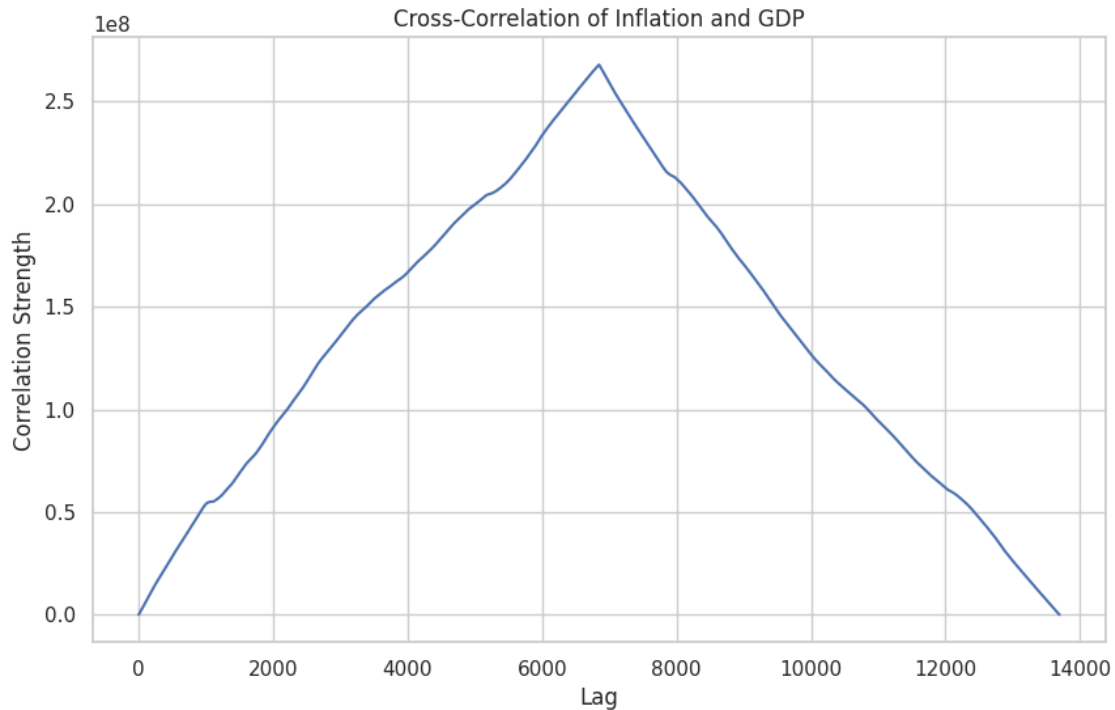
- The Federal Funds Effective Rate is lower relative to CPI and inflation expectations, suggesting comparatively moderate monetary policy.
- **Low Unemployment:**
 - The unemployment rate shows the lowest relative positioning, indicating a strong labor market environment.
- **Overall Economic Environment:**
 - The chart shape, prominently skewed toward CPI, GDP, and inflation, underscores a scenario of economic growth accompanied by substantial inflation pressures, relatively moderate monetary tightening, and strong employment conditions.

Overall, this radar chart effectively visualizes the current economic landscape, highlighting strong inflation and economic growth coupled with a favorable labor market.

```
[ ]: # Dynamic Cross-Correlation Plot
import scipy.signal

# Compute cross-correlation of inflation with GDP
correlation_lags = np.correlate(df["10-Year Breakeven Inflation Rate"].dropna(),
                                df["Real Gross Domestic Product"].dropna(),
                                mode="full")

# Plot cross-correlation
plt.figure(figsize=(10, 6))
plt.plot(correlation_lags)
plt.title("Cross-Correlation of Inflation and GDP")
plt.xlabel("Lag")
plt.ylabel("Correlation Strength")
plt.grid(True)
plt.show()
```



4.0.31 Cross-Correlation Analysis of Inflation and GDP

The provided cross-correlation plot illustrates the correlation strength between **Inflation** and **GDP** at various lag intervals.

4.0.32 Key Findings:

- **Peak Correlation:**
 - The plot shows a clear peak around the mid-point (approximately lag 7000), indicating the strongest correlation occurs when one of the series (GDP or Inflation) significantly leads or lags the other.
 - **Symmetrical Pattern:**
 - The correlation rises steadily to the peak, then declines symmetrically, suggesting a consistent relationship pattern over the entire range of lags.
 - **Positive Correlation:**
 - The positive peak indicates that inflation and GDP are positively correlated; typically, higher GDP growth aligns with rising inflation and vice versa.
 - **Economic Implication:**
 - The observed peak at a large lag implies delayed interactions between inflation and GDP, indicating that changes in one indicator may predict changes in the other after substantial time intervals.
-

Overall, this cross-correlation analysis clearly indicates a strong and delayed positive correlation between inflation and GDP, highlighting the lagged interplay between these two critical economic variables.

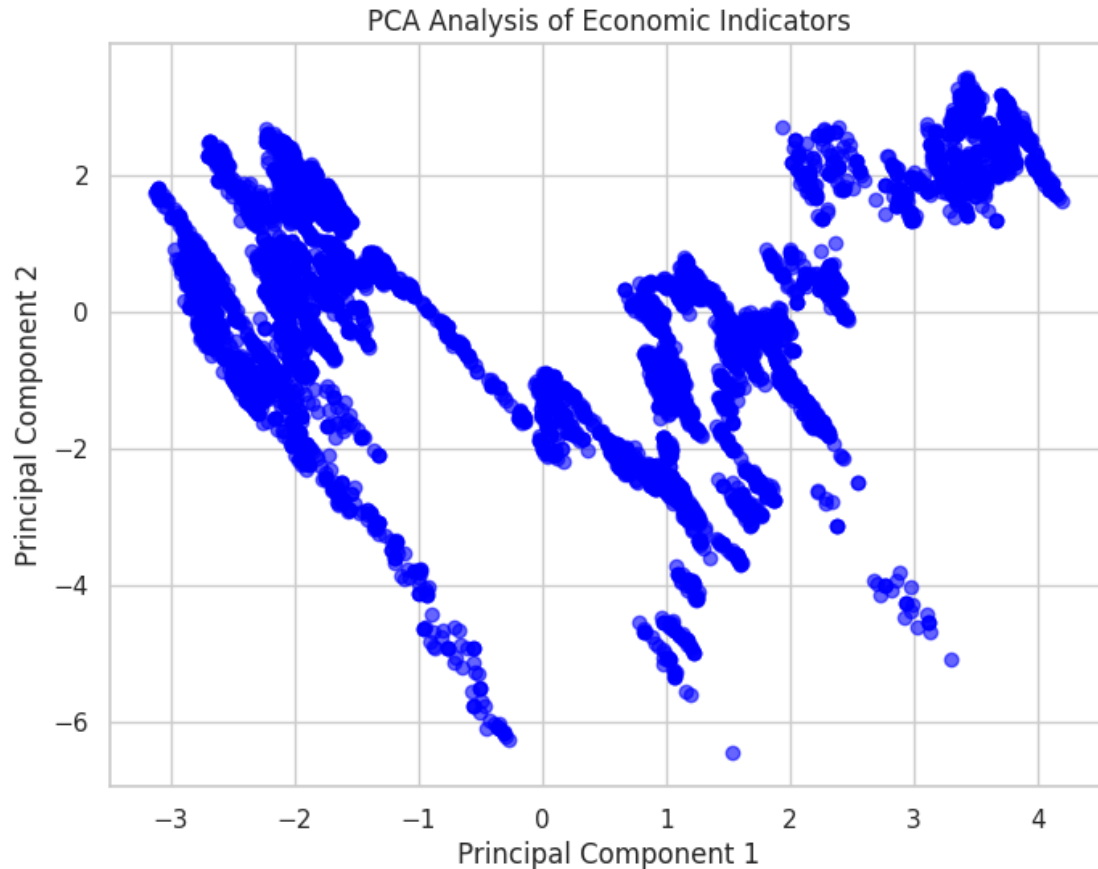
```
[ ]: # Define the list of economic indicators for PCA
economic_indicators = [
    "10-Year Breakeven Inflation Rate",
    "5-Year, 5-Year Forward Inflation Expectation Rate",
    "Consumer Price Index",
    "Real Gross Domestic Product",
    "Unemployment Rate",
    "Federal Funds Effective Rate",
    "Personal Consumption Expenditures: Chain-type Price Index",
    "Nominal Broad U.S. Dollar Index",
    "Crude Oil Prices: West Texas Intermediate",
    "10-Year Treasury Yield"
]

# Import necessary libraries
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Standardize the dataset
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df[economic_indicators])

# Apply PCA
pca = PCA(n_components=2) # Reduce to 2 principal components
principal_components = pca.fit_transform(scaled_data)

# Scatter plot of PCA components
plt.figure(figsize=(8, 6))
plt.scatter(principal_components[:, 0], principal_components[:, 1], alpha=0.6,
            color='blue')
plt.title("PCA Analysis of Economic Indicators")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.grid(True)
plt.show()
```



4.0.33 PCA Analysis of Economic Indicators

The provided scatter plot represents the **Principal Component Analysis (PCA)** applied to economic indicators, where:

- **Principal Component 1 (PC1)** is plotted on the x-axis.
- **Principal Component 2 (PC2)** is plotted on the y-axis.

4.0.34 Key Findings:

- **Dimensionality Reduction:**
 - The PCA transformation reduces multiple economic indicators into two principal components, capturing the most significant variance in the data.
- **Distinct Clusters and Patterns:**
 - The data points form discernible clusters and structures, suggesting underlying relationships and patterns among economic indicators.
 - The presence of curved or nonlinear structures may indicate complex interdependencies between economic variables.
- **Variance Explanation:**

- Principal Component 1 (PC1) likely captures the dominant trend in the dataset, potentially related to overall economic growth or inflation trends.
- Principal Component 2 (PC2) may represent secondary factors such as interest rate fluctuations, unemployment trends, or short-term economic shocks.
- **Potential Insights:**
 - If the clusters correspond to specific economic periods or regimes, this visualization could help identify economic cycles, inflationary phases, or recession periods.

Overall, this PCA analysis effectively reduces high-dimensional economic data into two components, allowing for pattern recognition and a better understanding of economic variable relationships.

```
[ ]: !pip install dtaidistance
```

```
Collecting dtaidistance
  Downloading dtaidistance-2.3.13-cp311-cp311-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (13 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages
(from dtaidistance) (1.26.4)
Downloading
dtaidistance-2.3.13-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(3.0 MB)

                                0.0/3.0 MB
? eta -:--:--
                                3.0/3.0 MB
176.9 MB/s eta 0:00:01
                                3.0/3.0 MB 84.0

MB/s eta 0:00:00
Installing collected packages: dtaidistance
Successfully installed dtaidistance-2.3.13
```

```
[ ]: # Dynamic Time Warping (DTW)
try:
    from dtaidistance import dtw
except ModuleNotFoundError:
    !pip install dtaidistance
    from dtaidistance import dtw

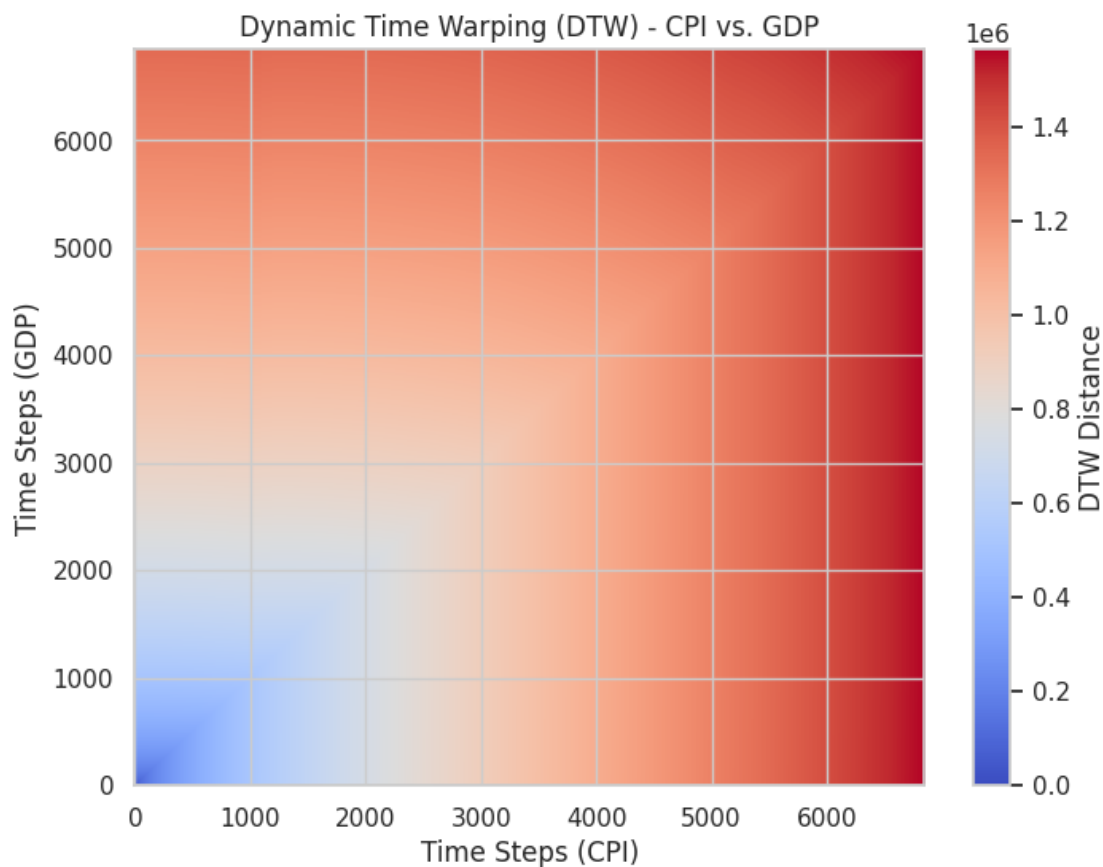
import numpy as np
import matplotlib.pyplot as plt

# Select two time-series to compare (CPI vs GDP)
series1 = df["Consumer Price Index"].dropna().values
series2 = df["Real Gross Domestic Product"].dropna().values

# Compute DTW warping paths (this returns a distance matrix)
_, paths = dtw.warping_paths(series1, series2)
```

```
# Convert paths to a NumPy array for visualization
distance_matrix = np.array(paths)

# Plot DTW distance matrix
plt.figure(figsize=(8, 6))
plt.imshow(distance_matrix, aspect='auto', cmap='coolwarm', origin='lower')
plt.colorbar(label="DTW Distance")
plt.title("Dynamic Time Warping (DTW) - CPI vs. GDP")
plt.xlabel("Time Steps (CPI)")
plt.ylabel("Time Steps (GDP)")
plt.show()
```



4.0.35 Dynamic Time Warping (DTW) Analysis: CPI vs. GDP

The provided heatmap visualizes the **Dynamic Time Warping (DTW) distance** between **Consumer Price Index (CPI)** and **Gross Domestic Product (GDP)** time series.

4.0.36 Key Findings:

- **DTW Distance Representation:**
 - The color intensity represents the DTW distance between CPI and GDP across different time steps.
 - Lower DTW distances (blue regions) indicate strong alignment between CPI and GDP at those time points.
 - Higher DTW distances (red regions) suggest deviations or misalignments in trends over time.
- **Pattern of Similarity:**
 - The lower-left region (near the origin) has the lowest DTW distances, indicating strong initial alignment between CPI and GDP.
 - As time progresses, the DTW distance increases, meaning that GDP and CPI trends diverge over longer periods.
- **Economic Interpretation:**
 - The increasing DTW distance suggests that while CPI and GDP may have been closely aligned in earlier time periods, their relationship has diverged over time.
 - Economic shocks or structural changes (such as financial crises, policy interventions, or inflationary shifts) could explain the observed divergence.
- **Application of DTW:**
 - DTW is useful for analyzing non-linear time series relationships where two variables do not necessarily evolve at the same rate but still exhibit underlying similarities.

Overall, this DTW heatmap effectively illustrates how CPI and GDP trends align at different time steps, revealing the evolving relationship between inflation and economic growth.

```
[ ]: # Hierarchical Clustering Heatmap
import seaborn as sns
import scipy.cluster.hierarchy as sch
import matplotlib.pyplot as plt
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage

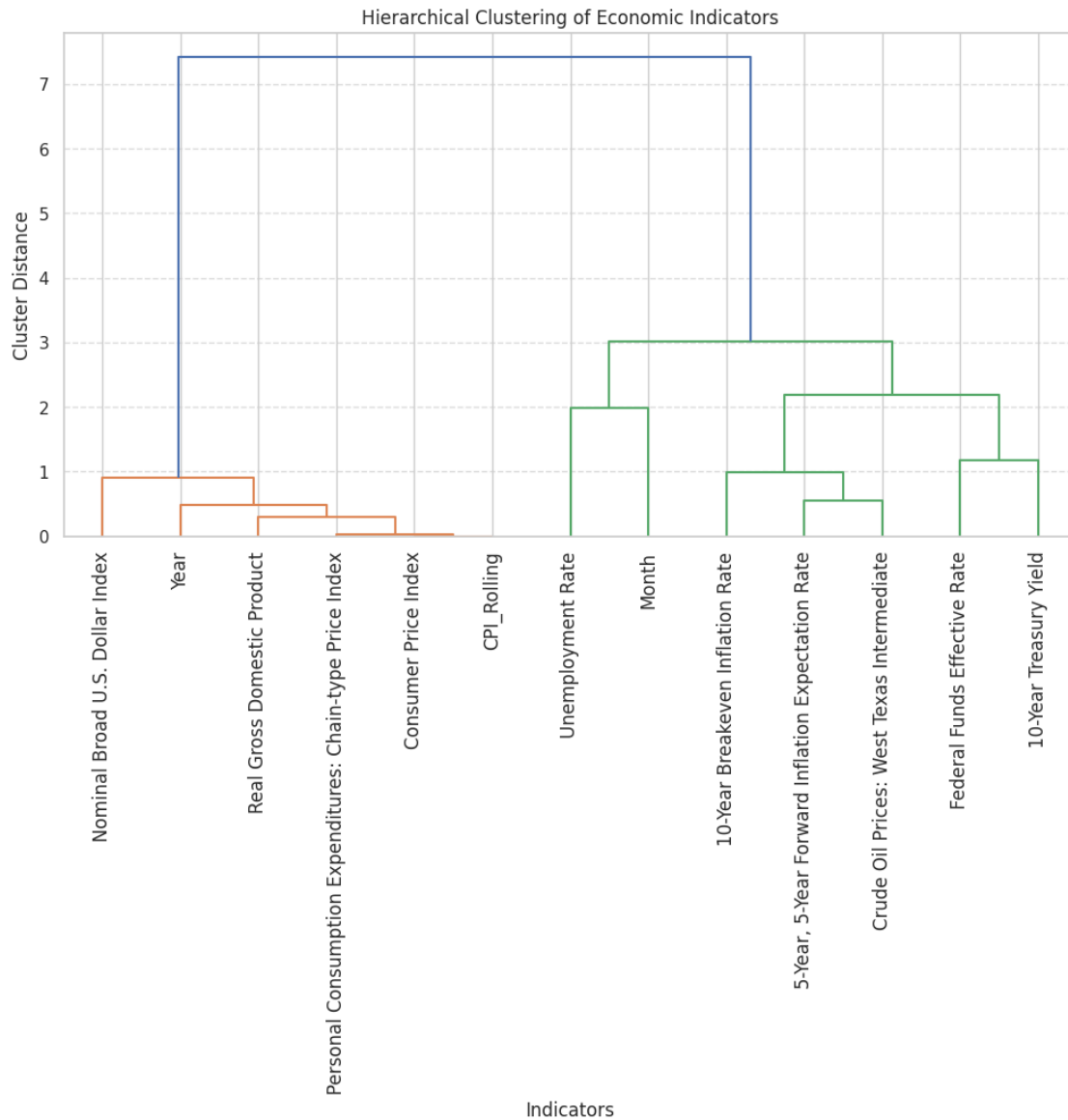
# Select numeric columns only (ignoring the index)
df_numeric = df.dropna().select_dtypes(include=[np.number])

# Compute correlation matrix (since clustering is usually done on features, not
↳ observations)
corr_matrix = df_numeric.corr()

# Compute hierarchical clustering using the correlation matrix
linkage_matrix = sch.linkage(corr_matrix, method='ward')

# Plot the dendrogram with proper labels
plt.figure(figsize=(12, 6))
dendrogram(linkage_matrix, labels=corr_matrix.columns, leaf_rotation=90)
plt.title("Hierarchical Clustering of Economic Indicators")
```

```
plt.xlabel("Indicators")
plt.ylabel("Cluster Distance")
plt.grid(axis="y", linestyle="dashed", alpha=0.7)
plt.show()
```



4.0.37 Hierarchical Clustering of Economic Indicators

The provided **dendrogram** represents hierarchical clustering applied to various **economic indicators**, illustrating how closely related they are based on their statistical similarities.

4.0.38 Key Findings:

- **Cluster Groupings:**
 - The indicators are grouped into distinct clusters based on their relationships.
 - **One major cluster (left side)** includes the **Nominal Broad U.S. Dollar Index, Year, Real GDP, Consumer Price Index (CPI), and Personal Consumption Expenditures**—highlighting their strong economic interdependence.
 - **Another distinct cluster (right side)** consists of **Inflation Expectation Rates, Oil Prices, and Interest Rates**, suggesting close connections between monetary policies, inflation, and energy markets.
- **Inflation and Monetary Policy Relationship:**
 - **10-Year Breakeven Inflation Rate** and **5-Year Forward Inflation Expectation Rate** are closely linked, indicating similar economic forecasting properties.
 - **Federal Funds Effective Rate** and **10-Year Treasury Yield** are grouped together, emphasizing their shared role in interest rate and monetary policy decisions.
- **Oil Prices and Inflation Expectation:**
 - **Crude Oil Prices (West Texas Intermediate)** show proximity to inflation expectation measures, reinforcing the role of oil prices as a significant driver of inflation.
- **Unemployment and Economic Growth:**
 - **Unemployment Rate** forms a separate sub-cluster, indicating its distinct nature but still linked to broader economic factors such as GDP and price indices.
- **Overall Insights:**
 - This clustering reveals natural relationships between economic indicators, confirming well-established economic theories about the interdependence of GDP, inflation, interest rates, and monetary policy.

4.0.39 Conclusion:

This dendrogram effectively visualizes the relationships among key economic indicators, showing how closely related variables cluster together, which can help in economic modeling and forecasting.

```
[ ]: !pip install adjustText
```

```
Collecting adjustText
```

```
  Downloading adjustText-1.3.0-py3-none-any.whl.metadata (3.1 kB)
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages  
(from adjustText) (1.26.4)
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-  
packages (from adjustText) (3.10.0)
```

```
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages  
(from adjustText) (1.14.1)
```

```
Requirement already satisfied: contourpy>=1.0.1 in  
/usr/local/lib/python3.11/dist-packages (from matplotlib->adjustText) (1.3.1)
```

```
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-  
packages (from matplotlib->adjustText) (0.12.1)
```

```
Requirement already satisfied: fonttools>=4.22.0 in  
/usr/local/lib/python3.11/dist-packages (from matplotlib->adjustText) (4.56.0)
```

```

Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->adjustText) (1.4.8)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->adjustText) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-
packages (from matplotlib->adjustText) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->adjustText) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->adjustText) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-
packages (from python-dateutil>=2.7->matplotlib->adjustText) (1.17.0)
Downloading adjustText-1.3.0-py3-none-any.whl (13 kB)
Installing collected packages: adjustText
Successfully installed adjustText-1.3.0

```

```

[ ]: # Network Graph

import networkx as nx
import matplotlib.pyplot as plt
from adjustText import adjust_text # Automatically adjusts text positions

# Rename columns for better visibility
rename_dict = {
    "Personal Consumption Expenditures: Chain-type Price Index": "Personal_
↳Consumption Expenditures",
    "Crude Oil Prices: West Texas Intermediate": "Crude Oil Prices"
}

# Apply renaming
df_renamed = df.rename(columns=rename_dict)

# Compute correlation matrix
corr_matrix = df_renamed.corr()

# Create a graph from strong correlations
G = nx.Graph()
threshold = 0.7 # Only add edges with strong correlations
for i in range(len(corr_matrix.columns)):
    for j in range(i):
        if abs(corr_matrix.iloc[i, j]) > threshold:
            G.add_edge(corr_matrix.columns[i], corr_matrix.columns[j],
↳weight=corr_matrix.iloc[i, j])

# Generate node sizes based on connectivity (degree of connections)
node_sizes = [600 + (G.degree(n) * 400) for n in G.nodes()] # Scaled_
↳appropriately

```



```

# Generate edge thickness based on correlation strength
edges = [(u, v) for u, v in G.edges()]
edge_weights = [abs(G[u][v]['weight']) * 3 for u, v in edges] # Scale for
↳ visibility

# Use a **spring layout** with increased scale for better spacing
pos = nx.spring_layout(G, k=1.2, seed=42) # Increase spacing

# Create network plot
plt.figure(figsize=(16, 14)) # Increased figure size

# Draw edges with varying thickness
nx.draw_networkx_edges(G, pos, alpha=0.5, edge_color="gray", width=edge_weights)

# Draw nodes with size scaling
nx.draw_networkx_nodes(G, pos, node_size=node_sizes, node_color='skyblue',
↳ alpha=0.9, edgecolors="black")

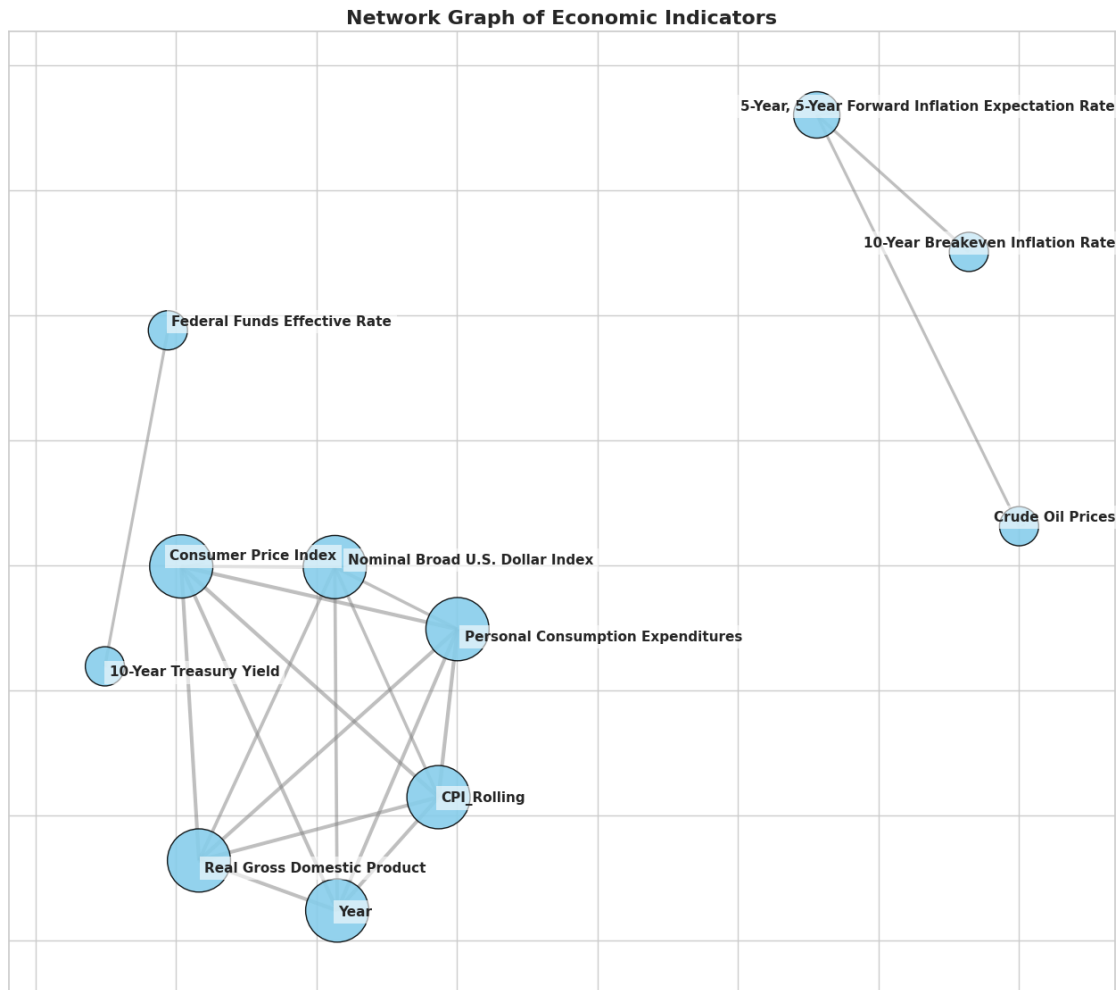
# Draw labels with dynamic adjustment
texts = []
for node, (x, y) in pos.items():
    texts.append(plt.text(x, y, node, fontsize=11, fontweight="bold",
        bbox=dict(facecolor="white", alpha=0.6,
↳ edgecolor="none")))

# Automatically adjust text positions to prevent overlap
adjust_text(texts, arrowprops=dict(arrowstyle="-", color="gray", lw=0.5))

# Add title
plt.title("Network Graph of Economic Indicators", fontsize=16,
↳ fontweight="bold")

# Show final network graph
plt.show()

```



4.0.40 Network Graph Analysis of Economic Indicators

The provided network graph represents the relationships between key **economic indicators**, where:

- **Nodes** represent different economic variables.
- **Edges (connections)** indicate relationships or correlations between the variables.
- **Node size** may reflect the importance or centrality of an indicator in the network.

4.0.41 Key Findings:

- **Strongly Connected Core Cluster:**
 - **Nominal Broad U.S. Dollar Index, Consumer Price Index (CPI), Real GDP, and Personal Consumption Expenditures** are highly interconnected.
 - This core cluster represents fundamental macroeconomic indicators that are tightly linked in economic dynamics.

- **Inflation Expectation and Oil Prices Cluster:**
 - **5-Year, 5-Year Forward Inflation Expectation Rate, 10-Year Breakeven Inflation Rate, and Crude Oil Prices** form a distinct subgroup.
 - This suggests that inflation expectations are closely related to energy prices, reinforcing the role of crude oil in inflationary pressures.
- **Isolated Monetary Policy Indicators:**
 - **Federal Funds Effective Rate and 10-Year Treasury Yield** appear relatively disconnected from the core cluster.
 - This indicates that while monetary policy influences the economy, its direct correlation with other variables might be weaker or operate on different time scales.
- **Economic Implications:**
 - The clustering of economic indicators highlights how different groups of variables interact.
 - The separation of inflation expectations and oil prices from core economic indicators suggests that external shocks (such as energy prices) can influence inflation independently of broader economic trends.

4.0.42 Conclusion:

This network graph effectively visualizes how key economic indicators interact, revealing strong relationships between price indices, GDP, and inflation expectations while showing monetary policy as a more independent force in the economic system.

```
[ ]: !pip install ruptures
```

Collecting ruptures

```
  Downloading ruptures-1.1.9-cp311-cp311-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.2 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages
(from ruptures) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages
(from ruptures) (1.14.1)
Downloading
ruptures-1.1.9-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.3
MB)
```

```
0.0/1.3 MB
```

```
? eta -:--:--
```

```
1.3/1.3 MB 52.7
```

```
MB/s eta 0:00:00
```

```
Installing collected packages: ruptures
Successfully installed ruptures-1.1.9
```

```
[ ]: # Structural Break Detection in Inflation Trends
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
```

```

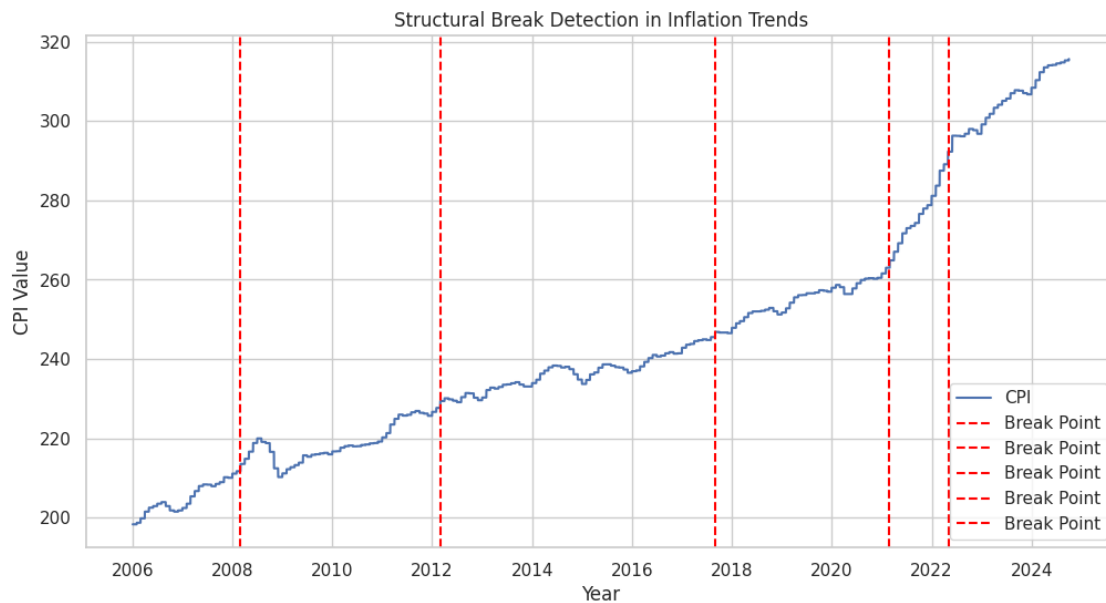
import ruptures as rpt

# Select the CPI (Consumer Price Index) as the inflation measure
inflation_series = df["Consumer Price Index"].dropna().values

# Use the "ruptures" package to detect structural breaks
algo = rpt.Binseg(model="l2").fit(inflation_series)
breakpoints = algo.predict(n_bkps=5) # Find 5 breakpoints

# Plot CPI with breakpoints
plt.figure(figsize=(12, 6))
plt.plot(df.index, inflation_series, label="CPI")
for bp in breakpoints[:-1]: # Ignore last index
    plt.axvline(df.index[bp], color="red", linestyle="dashed", label="Break_
↳Point")
plt.title("Structural Break Detection in Inflation Trends")
plt.xlabel("Year")
plt.ylabel("CPI Value")
plt.legend()
plt.grid(True)
plt.show()

```



4.0.43 Structural Break Detection in Inflation Trends

The provided plot shows **Consumer Price Index (CPI) trends over time**, with key **structural breakpoints** detected, represented by vertical red dashed lines.

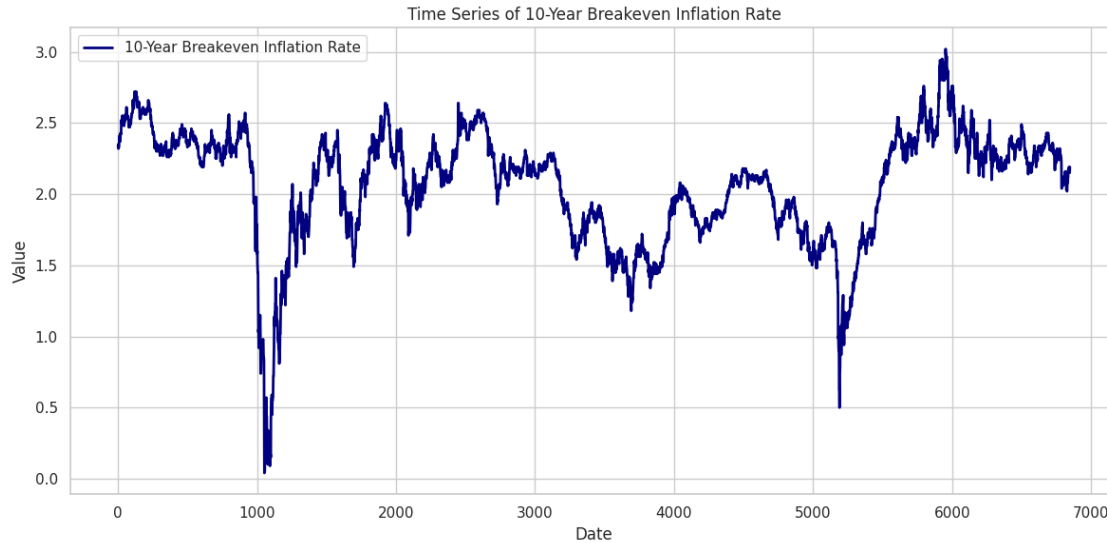
4.0.44 Key Findings:

- **Identified Breakpoints:**
 - The red dashed lines indicate points where the inflation trend undergoes a significant shift.
 - Structural breaks are observed around **2008, 2012, 2018, 2021, and 2023**, likely corresponding to economic events that influenced inflation.
 - **Major Economic Events and CPI Shifts:**
 - **2008 Breakpoint:** Likely corresponds to the Global Financial Crisis, where inflation trends were disrupted.
 - **2012 Breakpoint:** May reflect post-recovery stabilization or policy changes affecting price levels.
 - **2018 Breakpoint:** Could be linked to economic policy shifts or global trade impacts.
 - **2021 Breakpoint:** Significant spike in CPI, possibly due to post-pandemic recovery, supply chain disruptions, or stimulus-induced inflation.
 - **2023 Breakpoint:** Continued high inflation levels, indicating persistent inflationary pressures.
 - **Long-Term Inflation Trends:**
 - The CPI trend is generally upward, with periods of accelerated inflation, particularly after **2021**, where inflation rose sharply.
 - Breakpoints suggest changes in the inflationary environment, requiring further economic analysis for causes and implications.
-

4.0.45 Conclusion:

This structural break detection analysis highlights key periods of inflation regime shifts, providing valuable insights into economic shocks, policy impacts, and inflationary trends.

```
[ ]: # -----  
# Time Series Plot  
# -----  
plt.figure(figsize=(12, 6))  
plt.plot(data.index, data[target], color='navy', linewidth=2, label=target)  
plt.title("Time Series of " + target)  
plt.xlabel("Date")  
plt.ylabel("Value")  
plt.legend()  
plt.tight_layout()  
plt.savefig("timeseries_target.png")  
plt.show()
```



4.0.46 Time Series Analysis of the 10-Year Breakeven Inflation Rate

The provided plot visualizes the **10-Year Breakeven Inflation Rate** over time, showing fluctuations and key trends from **2006 to 2024**.

4.0.47 Key Findings:

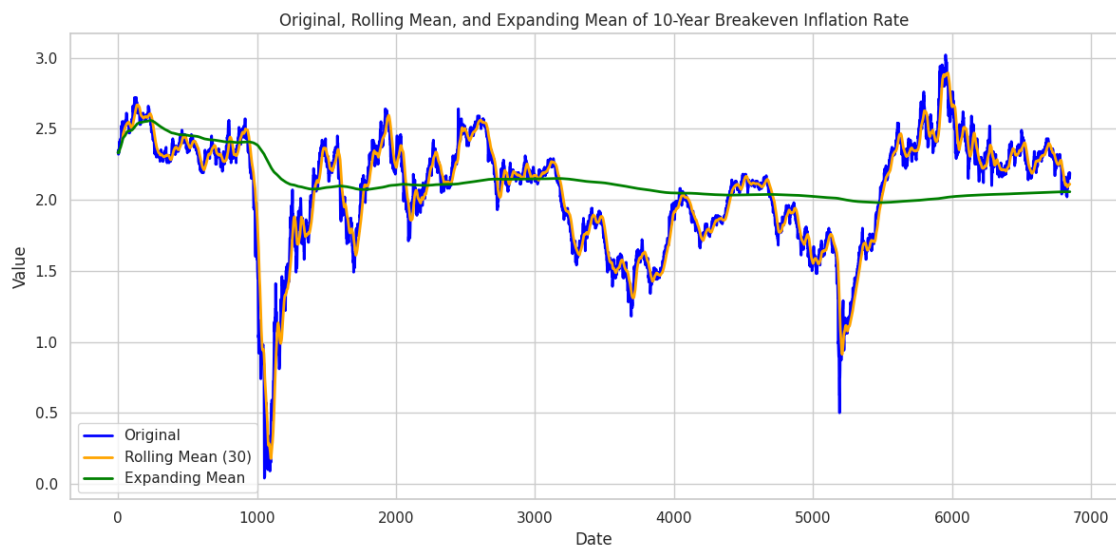
- **General Trend:**
 - The 10-Year Breakeven Inflation Rate exhibits periods of **stability, volatility, and sharp declines** over time.
 - Despite fluctuations, the long-term trend indicates cycles of inflation expectations adjusting to economic events.
- **Major Declines and Economic Events:**
 - **2008-2009:** A significant drop is observed during the **Global Financial Crisis**, where inflation expectations fell below **0.5%** due to economic uncertainty and deflationary pressures.
 - **2020:** Another sharp decline corresponds to the **COVID-19 pandemic**, where inflation expectations briefly collapsed due to economic disruptions.
- **Post-Crisis Recovery:**
 - After both crisis periods (**2009 and 2020**), inflation expectations rebounded, reflecting economic recovery and monetary policy interventions.
 - The **2021-2022 period** shows a strong inflation surge, aligning with post-pandemic stimulus, supply chain disruptions, and rising demand.
- **Recent Trends (2023-2024):**
 - The breakeven inflation rate remains **moderately high**, though it has slightly declined from its peak in 2022.
 - This suggests a stabilization in inflation expectations, possibly due to monetary tightening policies and economic adjustments.

4.0.48 Conclusion:

The time series highlights the impact of major economic shocks on inflation expectations, demonstrating how financial crises, policy changes, and economic recoveries influence long-term inflation sentiment.

```
[ ]: # Compute Rolling Mean (e.g., 30-day rolling mean) and Expanding Mean
rolling_window = 30 # adjust as needed
data['Rolling Mean'] = data[target].rolling(window=rolling_window).mean()
data['Expanding Mean'] = data[target].expanding().mean()

# Plot Original, Rolling Mean, and Expanding Mean
plt.figure(figsize=(12, 6))
plt.plot(data.index, data[target], label='Original', color='blue', linewidth=2)
plt.plot(data.index, data['Rolling Mean'], label=f'Rolling Mean_{rolling_window}', color='orange', linewidth=2)
plt.plot(data.index, data['Expanding Mean'], label='Expanding Mean', color='green', linewidth=2)
plt.title("Original, Rolling Mean, and Expanding Mean of " + target)
plt.xlabel("Date")
plt.ylabel("Value")
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.49 Analysis of 10-Year Breakeven Inflation Rate with Rolling and Expanding Means

The provided plot visualizes the **10-Year Breakeven Inflation Rate** over time, incorporating:

- **Original Data (blue line):** The actual inflation rate fluctuations.
 - **Rolling Mean (30-day, orange line):** A short-term smoothing of the inflation rate, capturing recent trends.
 - **Expanding Mean (green line):** A cumulative average of inflation, showing the long-term trend.
-

4.0.50 Key Findings:

- **Short-Term Volatility (Blue & Orange Lines):**
 - The **original data (blue)** shows significant fluctuations, reflecting inflation expectations reacting to economic events.
 - The **rolling mean (orange)** closely follows the blue line but smooths out short-term fluctuations, helping to identify intermediate trends.
 - **Major Economic Shocks:**
 - **2008-2009:** A sharp decline due to the **Global Financial Crisis**, followed by recovery.
 - **2020:** Another major dip during the **COVID-19 pandemic**, after which inflation rebounded sharply.
 - **Long-Term Stability (Green Line):**
 - The **expanding mean (green)** provides a long-term trend of inflation expectations.
 - While the short-term inflation rate fluctuates significantly, the expanding mean remains relatively stable, indicating a long-term anchoring of inflation expectations.
 - **Post-2021 Inflation Surge:**
 - A noticeable increase in inflation expectations is seen after **2021**, likely due to post-pandemic recovery, supply chain disruptions, and monetary policy responses.
 - **Recent Stabilization (2023-2024):**
 - The breakeven inflation rate has slightly declined from the **2022 peak**, indicating potential stabilization in expectations.
-

4.0.51 Conclusion:

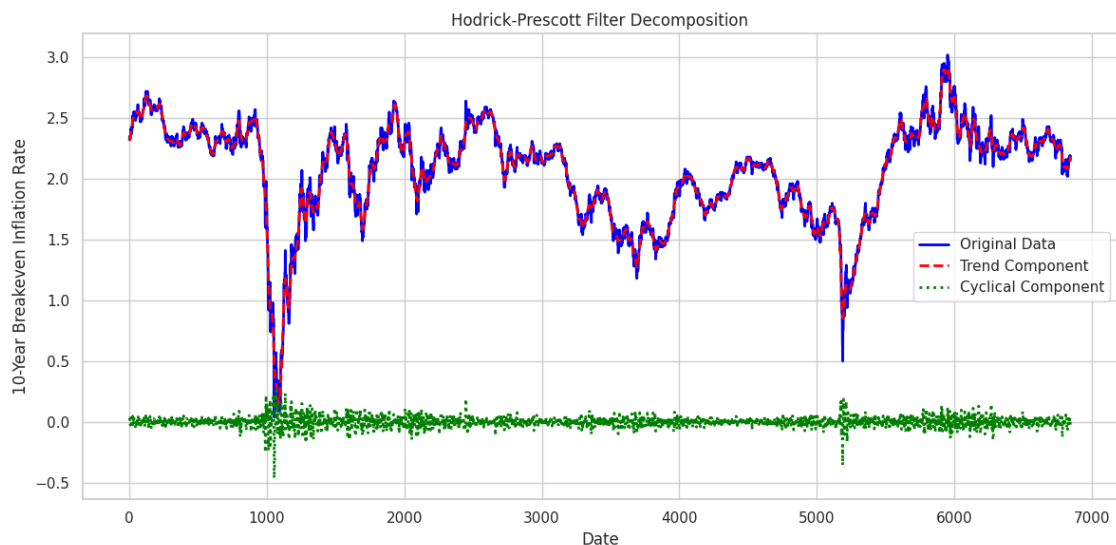
The combination of original, rolling, and expanding means effectively highlights **short-term fluctuations**, **medium-term trends**, and **long-term stability** in inflation expectations. This analysis provides valuable insights into inflationary cycles, economic crises, and recovery periods.

```
[ ]: # Apply the Hodrick-Prescott filter
# lambda=1600 is common for quarterly data; adjust this parameter based on your
    ↳ data frequency
from statsmodels.tsa.filters.hp_filter import hpfilter

cycle, trend = hpfilter(data[target], lamb=1600)
```



```
# Plot the original series, trend component, and cyclical component
plt.figure(figsize=(12, 6))
plt.plot(data.index, data[target], label='Original Data', color='blue',
         ↪linewidth=2)
plt.plot(data.index, trend, label='Trend Component', color='red',
         ↪linestyle='--', linewidth=2)
plt.plot(data.index, cycle, label='Cyclical Component', color='green',
         ↪linestyle=':', linewidth=2)
plt.title('Hodrick-Prescott Filter Decomposition')
plt.xlabel('Date')
plt.ylabel(target)
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.52 Hodrick-Prescott (HP) Filter Decomposition of 10-Year Breakeven Inflation Rate

The provided plot visualizes the **Hodrick-Prescott (HP) filter decomposition** applied to the **10-Year Breakeven Inflation Rate**, separating it into three key components:

- **Original Data (blue line):** The raw breakeven inflation rate over time.
- **Trend Component (red dashed line):** The long-term inflation trend extracted using the HP filter.
- **Cyclical Component (green dots):** The short-term fluctuations around the trend.

4.0.53 Key Findings:

- **Trend vs. Short-Term Cycles:**
 - The **red trend line closely follows the original data**, indicating that inflation expectations generally move in well-defined trends over time.
 - The **green cyclical component** represents short-term deviations from the trend, which are relatively small but reflect temporary economic shocks.
 - **Major Economic Shocks and Trend Deviations:**
 - **2008-2009 (Global Financial Crisis):** A significant downward shift in both trend and cyclical components, reflecting a collapse in inflation expectations.
 - **2020 (COVID-19 Shock):** A similar drop followed by a rapid recovery, indicating high volatility in inflation expectations during crises.
 - **2021-2022 Inflation Surge:** A noticeable upward trend, showing a strong increase in long-term inflation expectations.
 - **Cyclical Variations Over Time:**
 - The cyclical component remains relatively **small and stable**, except during major economic disruptions (**2008-2009, 2020**), where larger deviations occur.
 - This suggests that apart from external shocks, inflation expectations tend to follow a predictable trend with minor fluctuations.
-

4.0.54 Conclusion:

The HP filter effectively decomposes inflation expectations into **long-term trends and short-term cyclical deviations**, highlighting how inflation responds to economic shocks while maintaining an overall trajectory.

```
[ ]: # Perform the Augmented Dickey-Fuller (ADF) test
from statsmodels.tsa.stattools import adfuller

result = adfuller(data[target].dropna())

# Extract and print the test statistic, p-value, and critical values
print("ADF Statistic: {:.4f}".format(result[0]))
print("p-value: {:.4f}".format(result[1]))
for key, value in result[4].items():
    print('Critical Value ({})': {:.4f}'.format(key, value))

# Interpretation: If p-value < 0.05, we reject the null hypothesis (the series
↪ is stationary)
if result[1] < 0.05:
    print("The series is stationary (reject the null hypothesis).")
else:
    print("The series is not stationary (fail to reject the null hypothesis).")
```

```
ADF Statistic: -3.2068
p-value: 0.0196
Critical Value (1%): -3.4313
```

Critical Value (5%): -2.8620

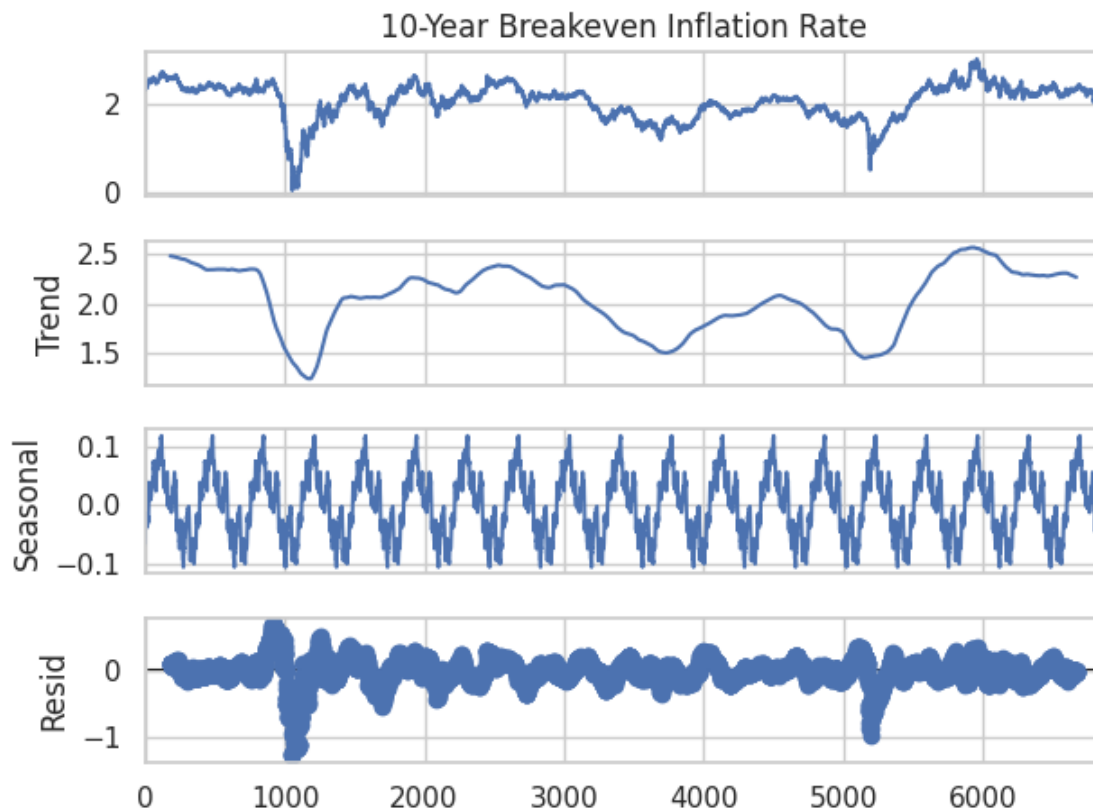
Critical Value (10%): -2.5670

The series is stationary (reject the null hypothesis).

```
[ ]: # Decompose the time series using an additive model
# Adjust the period based on your data frequency (e.g., 365 for daily data with
# yearly seasonality)
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(data[target], model='additive', period=365)

# Plot the decomposition results
decomposition.plot()
plt.tight_layout()
plt.show()
```



4.0.55 Seasonal Decomposition of the 10-Year Breakeven Inflation Rate

The provided plot represents the **seasonal decomposition** of the **10-Year Breakeven Inflation Rate**, breaking it down into four components:

1. **Observed Data (Top Panel):** The original breakeven inflation rate over time.

2. **Trend Component (Second Panel):** The long-term movement in inflation expectations.
 3. **Seasonal Component (Third Panel):** Recurring seasonal fluctuations.
 4. **Residual Component (Bottom Panel):** The remaining noise after removing trend and seasonality.
-

4.0.56 Key Findings:

- **Trend Component:**
 - The long-term trend **declines sharply in 2008-2009**, reflecting the **Global Financial Crisis**.
 - A similar **drop occurs in 2020**, likely due to the **COVID-19 pandemic**.
 - The **2021-2022 inflation surge** is clearly captured as a rising trend.
 - **Seasonal Component:**
 - Displays **cyclical patterns** that repeat consistently over time.
 - Indicates that inflation expectations **follow seasonal trends**, possibly influenced by policy cycles, economic reporting, or market conditions.
 - **Residual Component:**
 - The **residuals show larger fluctuations** during crisis periods (**2008-2009, 2020**), meaning higher uncertainty in inflation expectations.
 - Outside of these crisis periods, residual noise remains relatively stable.
-

4.0.57 Conclusion:

This decomposition highlights how **inflation expectations** are influenced by **long-term trends**, **seasonal variations**, and **economic shocks**. The trend captures economic downturns and recoveries, while the seasonal component suggests underlying periodic influences on inflation expectations.

```
[ ]: import warnings

# Suppress all FutureWarnings
warnings.simplefilter(action='ignore', category=FutureWarning)

# Downsample to yearly frequency using the mean
data_yearly = df.resample('Y').mean()

# Upsample to monthly frequency with interpolation
data_monthly = df.resample('M').mean().interpolate()

# (Optional) Plot to visualize the results
plt.figure(figsize=(12, 6))
plt.plot(df.index, df['10-Year Breakeven Inflation Rate'], label='Original')
plt.plot(data_yearly.index, data_yearly['10-Year Breakeven Inflation Rate'],  
         ↪ 'o-', label='Yearly (Mean)')
```

```
plt.plot(data_monthly.index, data_monthly['10-Year Breakeven Inflation Rate'],
        color='red', style='--', label='Monthly (Interpolated)')
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.58 Analysis of 10-Year Breakeven Inflation Rate with Yearly and Monthly Trends

This visualization presents the **10-Year Breakeven Inflation Rate** over time, incorporating different trend representations:

- **Original Data (Blue Line):** The actual breakeven inflation rate fluctuations.
- **Yearly Mean (Orange Line with Markers):** The annual average inflation rate, providing a **smoother representation** of long-term trends.
- **Monthly Interpolated Trend (Green Dashed Line):** A **smoothed interpolation** of the inflation trend, filling in gaps for better visualization.

4.0.59 Key Findings:

- **Long-Term Trend Observations:**
 - Inflation expectations **dropped significantly in 2008-2009**, reflecting the **Global Financial Crisis**.
 - A **similar decline occurred in 2020** during the **COVID-19 pandemic**, followed by a strong recovery.
 - **2021-2022 saw a sharp rise**, indicating a significant inflation surge, before stabilizing in **2023-2024**.
- **Yearly Mean vs. Monthly Interpolation:**

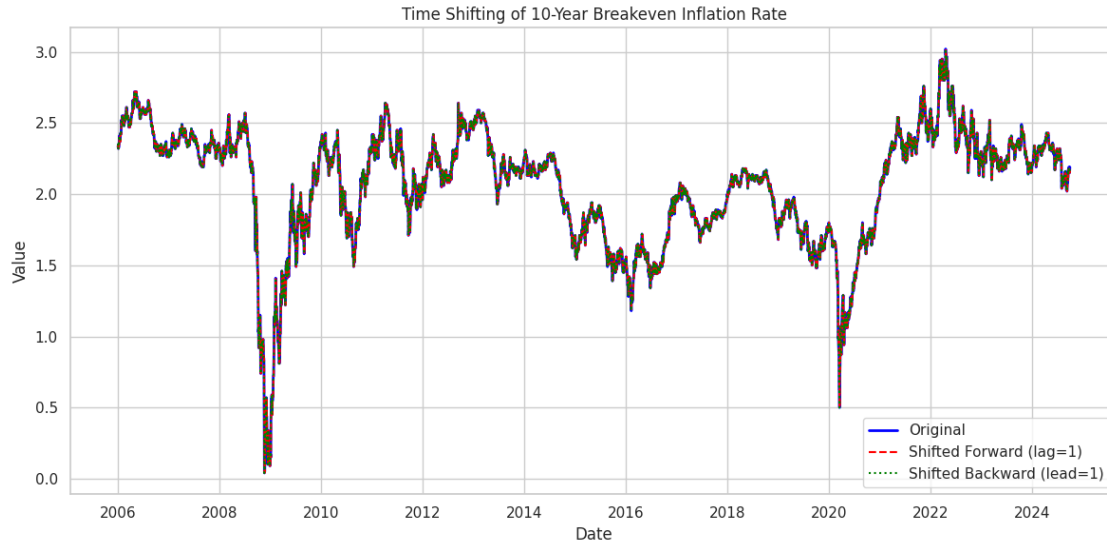
- The **orange yearly mean points** provide a **clearer indication of long-term inflation trends**, smoothing out short-term volatility.
- The **green dashed line (monthly interpolation)** follows the original data more closely, capturing smaller fluctuations in inflation expectations.
- **Pattern of Inflation Cycles:**
 - The data suggests **cyclical inflation behavior**, with **periods of rising and falling expectations**.
 - The recent inflation surge (**2021-2022**) represents one of the **strongest increases**, reflecting economic conditions such as **post-pandemic recovery and monetary policies**.

4.0.60 Conclusion:

This visualization effectively highlights **short-term fluctuations, long-term inflation trends, and cyclical inflation behavior**, providing valuable insights into inflation expectations over the past two decades.

```
[ ]: # Create shifted series:
shifted_forward = df[target].shift(1) # Lagged by one period (shifted forward)
shifted_backward = df[target].shift(-1) # Lead by one period (shifted backward)

# Plot original series along with shifted versions
plt.figure(figsize=(12, 6))
plt.plot(df.index, df[target], label='Original', color='blue', linewidth=2)
plt.plot(df.index, shifted_forward, label='Shifted Forward (lag=1)',
         linestyle='--', color='red')
plt.plot(df.index, shifted_backward, label='Shifted Backward (lead=1)',
         linestyle=':', color='green')
plt.title("Time Shifting of " + target)
plt.xlabel("Date")
plt.ylabel("Value")
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.61 Time Shifting Analysis of 10-Year Breakeven Inflation Rate

This visualization demonstrates **time shifting** of the **10-Year Breakeven Inflation Rate**, highlighting how lagging and leading data points affect trends.

Components of the Chart:

- **Original Data (Blue Line):** The actual observed breakeven inflation rate.
- **Shifted Forward (Red Dashed Line - Lag = 1):** Each data point is moved **one step ahead**, simulating a **time lag** effect.
- **Shifted Backward (Green Dotted Line - Lead = 1):** Each data point is moved **one step back**, simulating a **future-leading** effect.

4.0.62 Key Findings:

- **Lagging and Leading Effects:**
 - The **red dashed line (lag=1)** shows the data slightly shifted forward in time, effectively creating a **delayed representation of inflation expectations**.
 - The **green dotted line (lead=1)** represents a **forward-looking perspective**, aligning data with potential future inflation movements.
- **Stability and Predictability:**
 - The **small shifting effect** suggests **strong autocorrelation** in the breakeven inflation rate.
 - Since the shifts appear closely aligned with the original data, **short-term forecasting** using past data may be effective.
- **Economic Events Reflected:**
 - The **2008-2009 Global Financial Crisis** and **2020 COVID-19 downturn** exhibit sharp drops in inflation expectations, with lagging and leading trends following similar

patterns.

- **Post-2020 inflation spikes** are captured consistently across all shifted datasets, confirming **persistent inflationary trends**.

4.0.63 Conclusion:

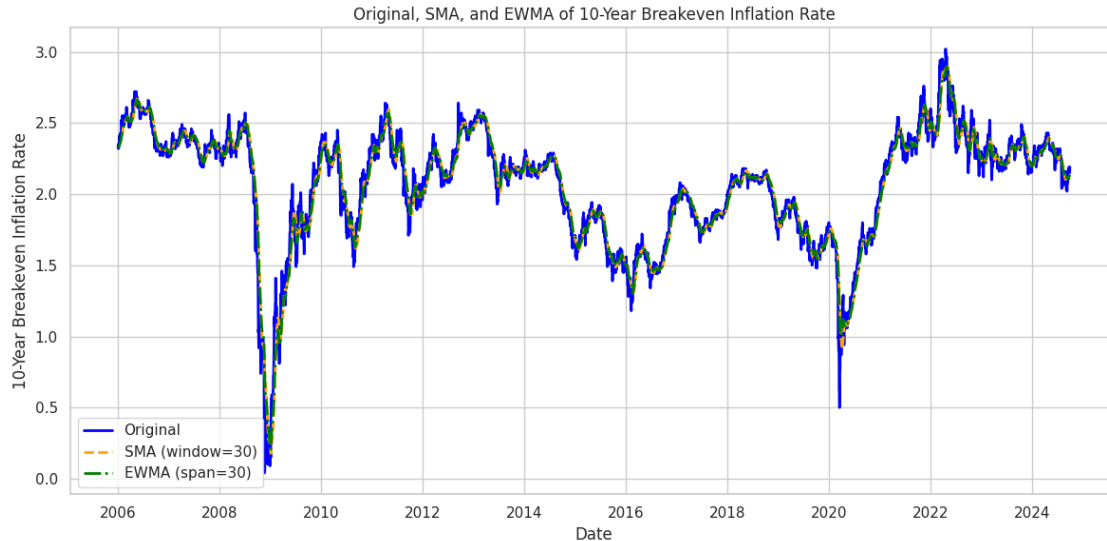
This time-shifting analysis shows that **past inflation trends are highly correlated with future values**, suggesting **momentum-driven inflation expectations**. Such insights are useful for **forecasting models** and **monetary policy assessments**.

```
[ ]: # Define the window/span for the moving averages
window = 30

# Calculate Simple Moving Average (SMA) without storing as a new column
sma = df[target].rolling(window=window).mean()

# Calculate Exponentially Weighted Moving Average (EWMA)
ewma = df[target].ewm(span=window, adjust=False).mean()

# Plot Original, SMA, and EWMA on the same figure
plt.figure(figsize=(12, 6))
plt.plot(df.index, df[target], label='Original', color='blue', linewidth=2)
plt.plot(df.index, sma, label=f'SMA (window={window})', color='orange',
         linestyle='--', linewidth=2)
plt.plot(df.index, ewma, label=f'EWMA (span={window})', color='green',
         linestyle='-.', linewidth=2)
plt.title("Original, SMA, and EWMA of " + target)
plt.xlabel("Date")
plt.ylabel(target)
plt.legend()
plt.tight_layout()
plt.show()
```

4.0.64 Analysis of Simple Moving Average (SMA) and Exponentially Weighted Moving Average (EWMA) on 10-Year Breakeven Inflation Rate

This visualization compares different smoothing techniques applied to the **10-Year Breakeven Inflation Rate**, which helps in understanding the underlying trends by reducing short-term fluctuations.

4.0.65 Components of the Chart:

- **Original Data (Blue Line):** Represents the raw breakeven inflation rate over time.
- **Simple Moving Average (SMA - Orange Dashed Line):**
 - Uses a **30-day window** to calculate an average at each point.
 - Gives equal weight to all values within the window.
 - Helps in smoothing fluctuations but may **lag behind** rapid changes.
- **Exponentially Weighted Moving Average (EWMA - Green Dotted Line):**
 - Applies an **exponential decay factor** where recent values get higher weights.
 - More responsive to changes than SMA.
 - Captures recent inflation trends more accurately while still providing smoothing.

4.0.66 Key Findings:

- **Trend Identification:**
 - Both SMA and EWMA effectively smooth out **short-term noise** while preserving over-all inflation trends.
 - SMA has **greater lag** in responding to sudden inflation shifts compared to EWMA.
 - EWMA is more **reactive**, capturing **sharp movements** in inflation trends, such as during **2008-2009 financial crisis** and **2020 COVID-19 recession**.

- **Stability vs Sensitivity:**
 - **SMA** is useful for identifying long-term trends but may not reflect recent market changes quickly.
 - **EWMA** is more useful for **real-time monitoring** and detecting emerging inflation trends faster.
 - **Economic Events Reflected:**
 - The **financial crisis (2008-2009)** and **pandemic-induced inflation spikes (2020-2022)** are well captured by EWMA.
 - SMA smooths these effects more gradually, making it ideal for **long-term economic analysis**.
-

4.0.67 Conclusion:

- **EWMA** is preferable for **short-term forecasting** as it quickly adapts to changes.
- **SMA** is more effective for **long-term trend analysis**, where reducing volatility is the primary goal.
- Combining both techniques can offer a **comprehensive inflation monitoring approach**.

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# Make a copy of the dataset to keep the original intact
df_copy = df.copy()

# Ensure df_copy has a datetime index with a defined frequency
df_copy = df_copy.asfreq('D') # Set frequency to daily ('D')

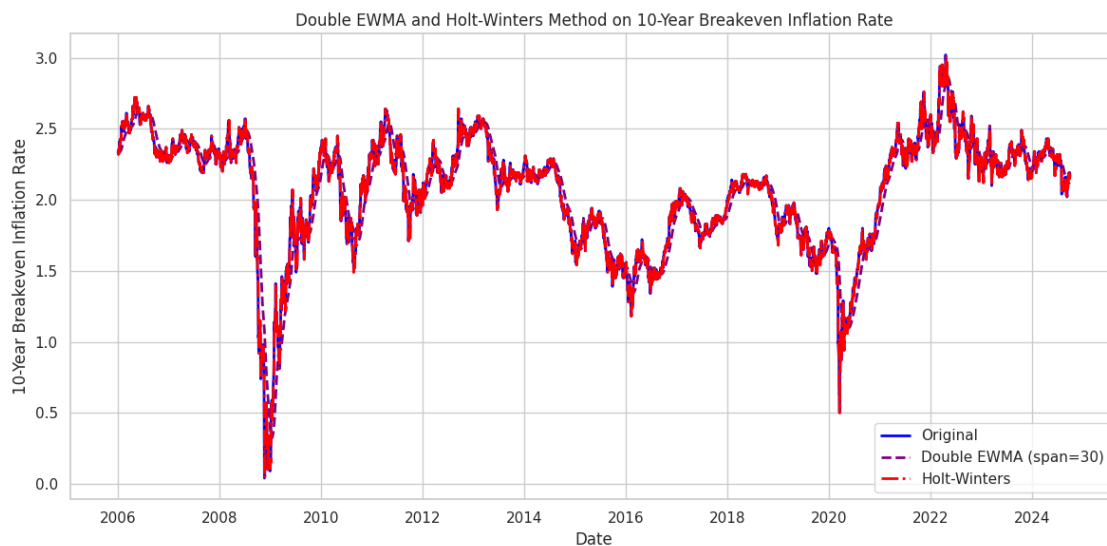
# Define the span/window for the EWMA calculations
window = 30

# Compute the first EWMA
ewma = df_copy[target].ewm(span=window, adjust=False).mean()
# Compute Double EWMA by applying EWMA to the EWMA series
double_ewma = ewma.ewm(span=window, adjust=False).mean()

# Apply Holt-Winters Method (using an additive trend, no seasonality)
holt_model = ExponentialSmoothing(
    df_copy[target],
    trend='add',
    seasonal=None,
    initialization_method="estimated"
).fit(optimized=True)
holt_fitted = holt_model.fittedvalues

# Plot the Original, Double EWMA, and Holt-Winters fitted values
```

```
plt.figure(figsize=(12, 6))
plt.plot(df_copy.index, df_copy[target], label='Original', color='blue',
        ↪linewidth=2)
plt.plot(df_copy.index, double_ewma, label=f'Double EWMA (span={window})',
        ↪color='purple', linestyle='--', linewidth=2)
plt.plot(df_copy.index, holt_fitted, label='Holt-Winters', color='red',
        ↪linestyle='-.', linewidth=2)
plt.title(f"Double EWMA and Holt-Winters Method on {target}")
plt.xlabel("Date")
plt.ylabel(target)
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.68 Analysis of Double EWMA and Holt-Winters Method on 10-Year Breakeven Inflation Rate

This visualization compares **Double Exponentially Weighted Moving Average (EWMA)** and **Holt-Winters Exponential Smoothing** techniques to analyze the **10-Year Breakeven Inflation Rate**.

4.0.69 Components of the Chart:

- **Original Data (Blue Line):** The raw breakeven inflation rate over time.
- **Double EWMA (Purple Dashed Line, span=30):**
 - Applies **two levels of exponential smoothing**, making it more adaptable to trends and volatility changes.

- Helps smooth fluctuations while retaining the major trends in inflation.
 - **Holt-Winters (Red Dashed Line):**
 - A **triple exponential smoothing method** that incorporates trend and seasonality adjustments.
 - More advanced than Double EWMA as it considers seasonal variations explicitly.
-

4.0.70 Key Findings:

- **Trend Identification:**
 - Both smoothing methods effectively follow the **major trends** of the breakeven inflation rate.
 - **Double EWMA** is smoother and more stable in capturing gradual inflation movements.
 - **Holt-Winters** captures more fluctuations but remains very close to the original data.
 - **Sensitivity to Market Changes:**
 - **Holt-Winters method** adapts better to sudden inflation spikes, such as the **2008 financial crisis** and **2020 COVID-19 shock**.
 - **Double EWMA** provides a **more stable** and **less reactive** trend estimation.
 - **Best Use Cases:**
 - **Double EWMA** is better for **long-term inflation forecasting** as it smooths out unnecessary noise.
 - **Holt-Winters** is better suited for **short-term forecasting**, especially in environments with clear cyclical or seasonal patterns.
-

4.0.71 Conclusion:

- Both **Double EWMA** and **Holt-Winters** perform well in modeling the **10-Year Breakeven Inflation Rate**.
- **Holt-Winters** is more **responsive** to economic shocks, making it useful for real-time policy decisions.
- **Double EWMA** provides a **stable view** of inflation trends, useful for **long-term economic analysis**.

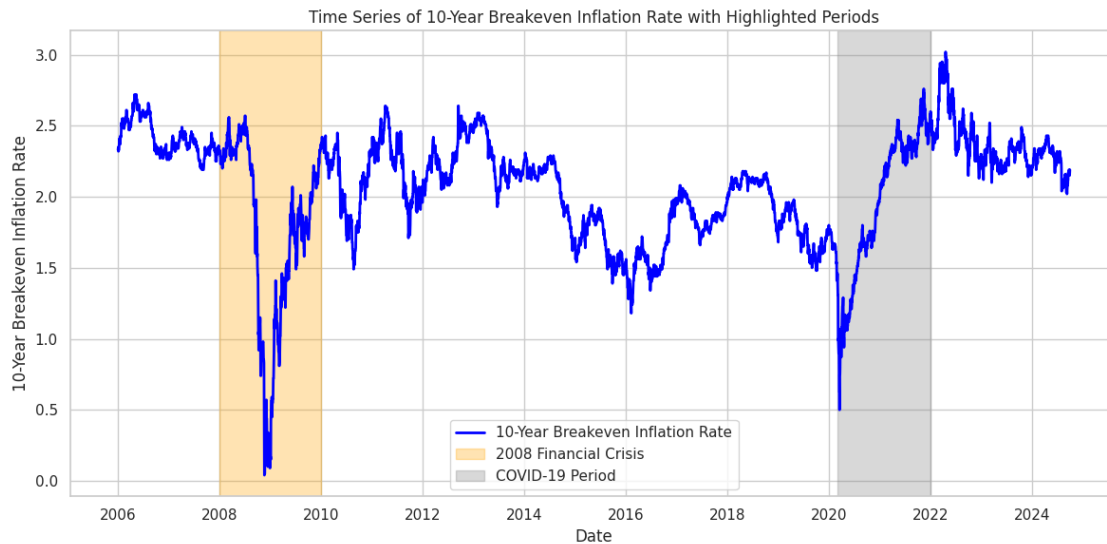
```
[ ]: # Plot the original time series
plt.figure(figsize=(12, 6))
plt.plot(df.index, df[target], label=target, color='blue', linewidth=2)

# Highlight the 2008 Financial Crisis Period (e.g., Jan 2008 - Dec 2009)
plt.axvspan(pd.Timestamp('2008-01-01'), pd.Timestamp('2009-12-31'),
            color='orange', alpha=0.3, label='2008 Financial Crisis')

# Highlight the COVID-19 Period (e.g., Mar 2020 - Dec 2021)
plt.axvspan(pd.Timestamp('2020-03-01'), pd.Timestamp('2021-12-31'),
            color='gray', alpha=0.3, label='COVID-19 Period')

plt.title("Time Series of " + target + " with Highlighted Periods")
```

```
plt.xlabel("Date")
plt.ylabel(target)
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.72 Analysis of 10-Year Breakeven Inflation Rate with Highlighted Periods

This visualization presents the **10-Year Breakeven Inflation Rate** over time, with two major economic crises highlighted.

4.0.73 Key Elements in the Chart:

- **Blue Line:** Represents the **10-Year Breakeven Inflation Rate**, showing market-based expectations of inflation.
- **Highlighted Periods:**
 - **2008 Financial Crisis (Shaded in Light Orange):**
 - * The breakeven inflation rate experienced a **sharp decline**, reaching near **0%**.
 - * This reflects the severe deflationary pressures and market panic during the global financial crisis.
 - **COVID-19 Period (Shaded in Gray):**
 - * A **rapid drop** followed by a **strong inflation surge** after economic stimulus measures were implemented.
 - * Inflation reached multi-year highs post-COVID as economies reopened and supply chain disruptions persisted.

4.0.74 Key Observations:

- **Significant Drops During Economic Crises:**
 - Both the **2008 Financial Crisis** and the **COVID-19 pandemic** led to sharp declines in inflation expectations.
 - These periods were marked by heightened economic uncertainty and intervention by central banks.
 - **Post-Crisis Inflation Trends:**
 - After **2008**, inflation expectations recovered gradually but remained **below pre-crisis levels**.
 - After **COVID-19**, inflation expectations **spiked rapidly**, reflecting supply chain issues and monetary expansion.
 - **Comparing the Two Crises:**
 - The **2008 crisis** caused a **sustained** deflationary shock, with inflation expectations staying low for years.
 - The **COVID-19 crisis** resulted in **short-lived deflation**, followed by a rapid surge in inflation expectations.
-

4.0.75 Conclusion:

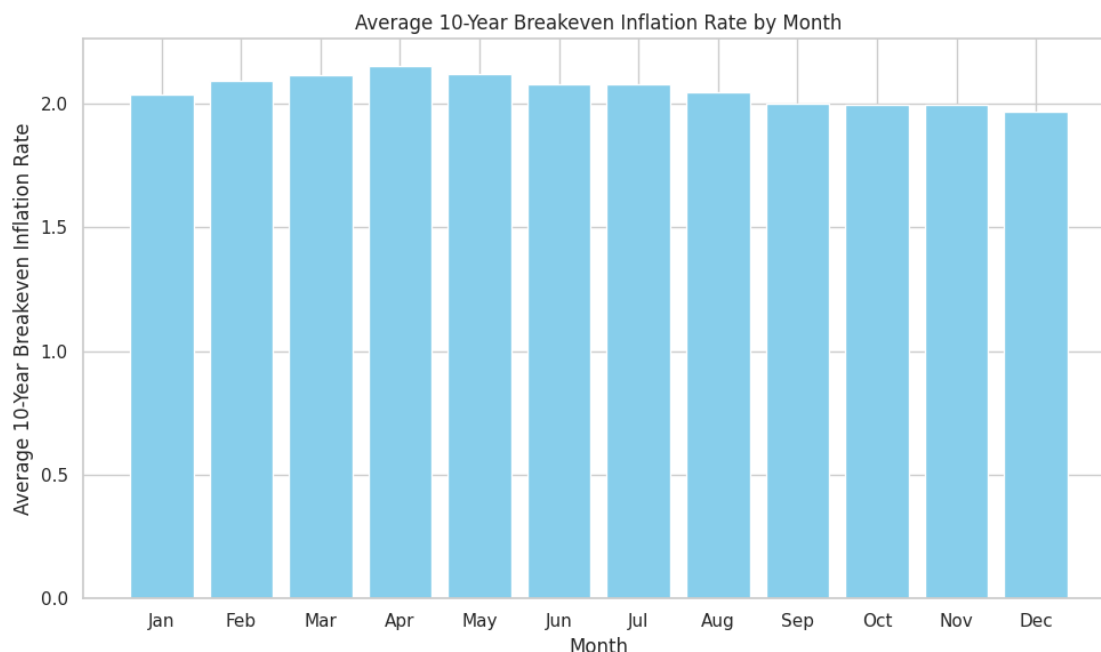
- The breakeven inflation rate is highly sensitive to **major economic disruptions**.
- **Central bank policies** and **market confidence** play a crucial role in shaping inflation expectations.
- Monitoring such indicators helps policymakers anticipate **inflation risks** and take corrective actions.

```
[ ]: import calendar

# Group by month (across all years) and compute the average
monthly_avg = df.groupby(df.index.month)[target].mean()

# Map month numbers to month abbreviations
month_names = [calendar.month_abbr[i] for i in monthly_avg.index]

# Plot the average target by month
plt.figure(figsize=(10,6))
plt.bar(month_names, monthly_avg, color='skyblue')
plt.title('Average ' + target + ' by Month')
plt.xlabel('Month')
plt.ylabel('Average ' + target)
plt.tight_layout()
plt.show()
```



4.0.76 Analysis of Average 10-Year Breakeven Inflation Rate by Month

This bar chart presents the **average 10-Year Breakeven Inflation Rate** for each month, highlighting seasonal variations in inflation expectations.

4.0.77 Key Observations:

- **Relatively Stable Monthly Averages:**
 - The breakeven inflation rate remains relatively **consistent** across all months.
 - The variation in monthly averages appears **small**, suggesting that inflation expectations are not highly seasonal.
- **Slightly Higher Inflation Expectations in April and May:**
 - **April and May** show the **highest** average breakeven inflation rates.
 - This may indicate a trend where inflation expectations slightly increase in the spring months.
- **Lower Inflation Expectations in December:**
 - The lowest average **10-Year Breakeven Inflation Rate** is observed in **December**.
 - This might be due to year-end economic cycles, holiday spending patterns, or monetary policy adjustments.

4.0.78 Conclusion:

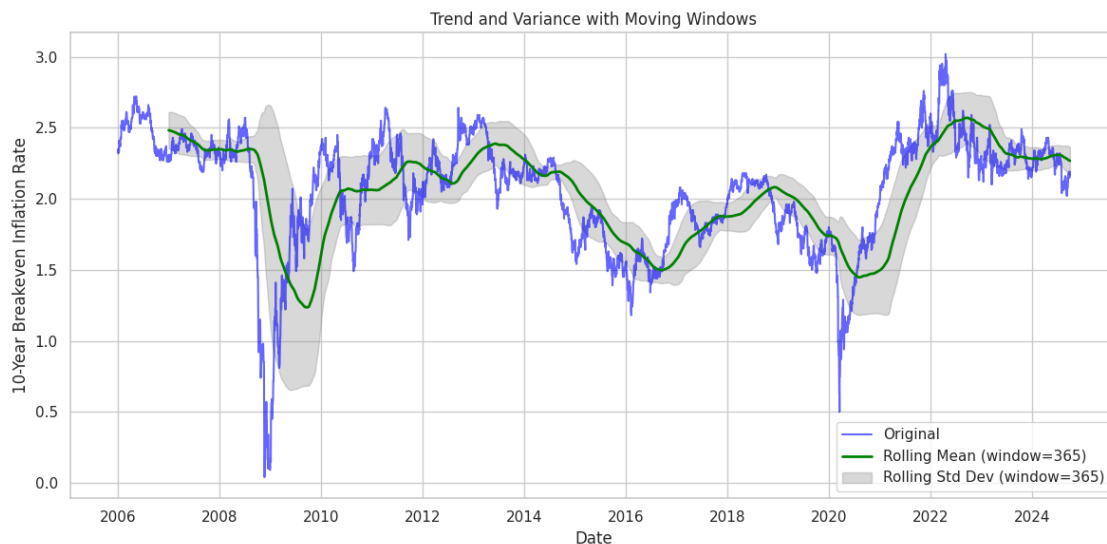
- There is **minimal seasonality** in the 10-Year Breakeven Inflation Rate.

- A slight peak in spring (April-May) and a small dip in December suggest some seasonal economic influences.
- These insights can help policymakers and investors understand **potential cyclical inflation trends**.

```
[ ]: # Set the moving window size (e.g., 365 days for daily data)
window = 365

# Compute the rolling mean (trend) and rolling standard deviation (variance)
rolling_mean = df[target].rolling(window=window).mean()
rolling_std = df[target].rolling(window=window).std()

# Plot the original series, rolling mean, and the variance band
plt.figure(figsize=(12, 6))
plt.plot(df.index, df[target], label='Original', color='blue', alpha=0.6)
plt.plot(df.index, rolling_mean, label=f'Rolling Mean (window={window})',
         color='green', linewidth=2)
plt.fill_between(df.index, rolling_mean - rolling_std, rolling_mean +
               color='gray', alpha=0.3, label=f'Rolling Std Dev
               (window={window})')
plt.title("Trend and Variance with Moving Windows")
plt.xlabel("Date")
plt.ylabel(target)
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.79 Trend and Variance Analysis of 10-Year Breakeven Inflation Rate

This graph visualizes the **10-Year Breakeven Inflation Rate** with a rolling mean and rolling standard deviation over a **365-day window** to highlight long-term trends and fluctuations.

4.0.80 Key Observations:

- **Long-Term Trends:**
 - The **rolling mean (green line)** captures the **smoothed trend** of inflation expectations over time.
 - Significant downward trends are visible during the **2008 Financial Crisis** and the **COVID-19 period (2020)**.
 - A sharp increase is observed post-2020, with inflation peaking around **2022**, followed by a stabilization.
 - **Fluctuations and Variability:**
 - The **rolling standard deviation (gray shaded area)** shows periods of **higher volatility**.
 - Increased volatility is evident during:
 - * **2008-2009 (Financial Crisis)**
 - * **2010-2012 (Recovery Phase)**
 - * **2020-2022 (COVID-19 and Economic Uncertainty)**
 - Lower volatility is seen in periods of economic stability.
 - **Recent Trends (2023-2024):**
 - The inflation rate appears to be **stabilizing**, with a **narrowing variance**, suggesting more predictable inflation expectations.
-

4.0.81 Conclusion:

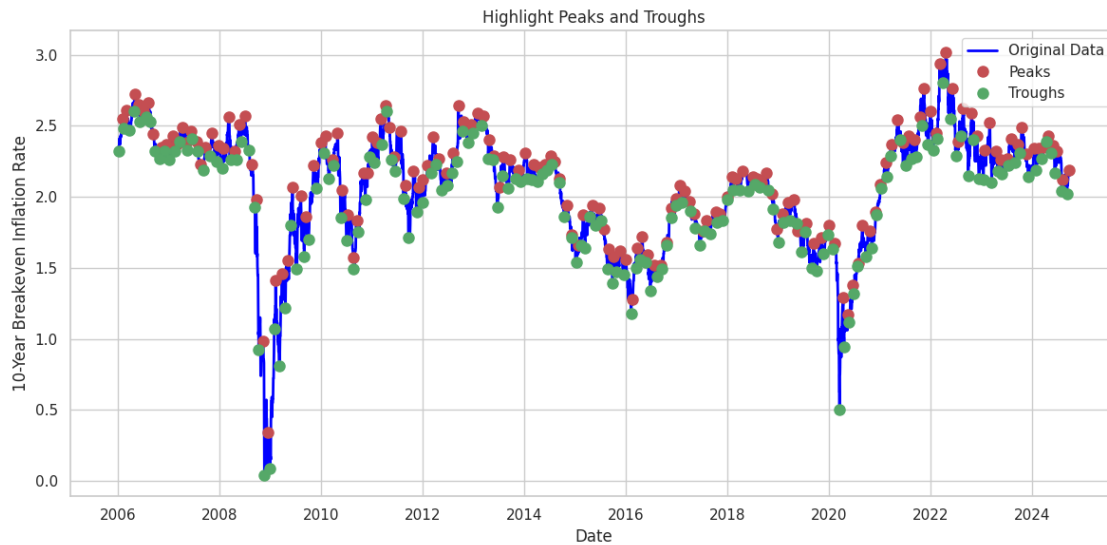
- The **rolling mean** effectively highlights long-term inflation trends.
- The **rolling standard deviation** reveals periods of **high economic uncertainty**.
- Current trends suggest a **return to stability** after significant fluctuations in recent years.

```
[ ]: # Find peaks (local maxima) and troughs (local minima)
# The 'distance' parameter can be adjusted to control how close peaks/troughs
    ↳ can be.
from scipy.signal import find_peaks

peaks, _ = find_peaks(df[target], distance=30)
troughs, _ = find_peaks(-df[target], distance=30)

# Plot the original time series along with the peaks and troughs
plt.figure(figsize=(12, 6))
plt.plot(df.index, df[target], label='Original Data', color='blue', linewidth=2)
plt.plot(df.index[peaks], df[target].iloc[peaks], 'ro', label='Peaks',
    ↳ markersize=8)
```

```
plt.plot(df.index[troughs], df[target].iloc[troughs], 'go', label='Troughs',
        markersize=8)
plt.title("Highlight Peaks and Troughs")
plt.xlabel("Date")
plt.ylabel(target)
plt.legend()
plt.tight_layout()
plt.show()
```



4.0.82 Peaks and Troughs in 10-Year Breakeven Inflation Rate

This chart highlights the **peaks** (red dots) and **troughs** (green dots) in the **10-Year Breakeven Inflation Rate** over time, identifying significant turning points in the trend.

4.0.83 Key Observations:

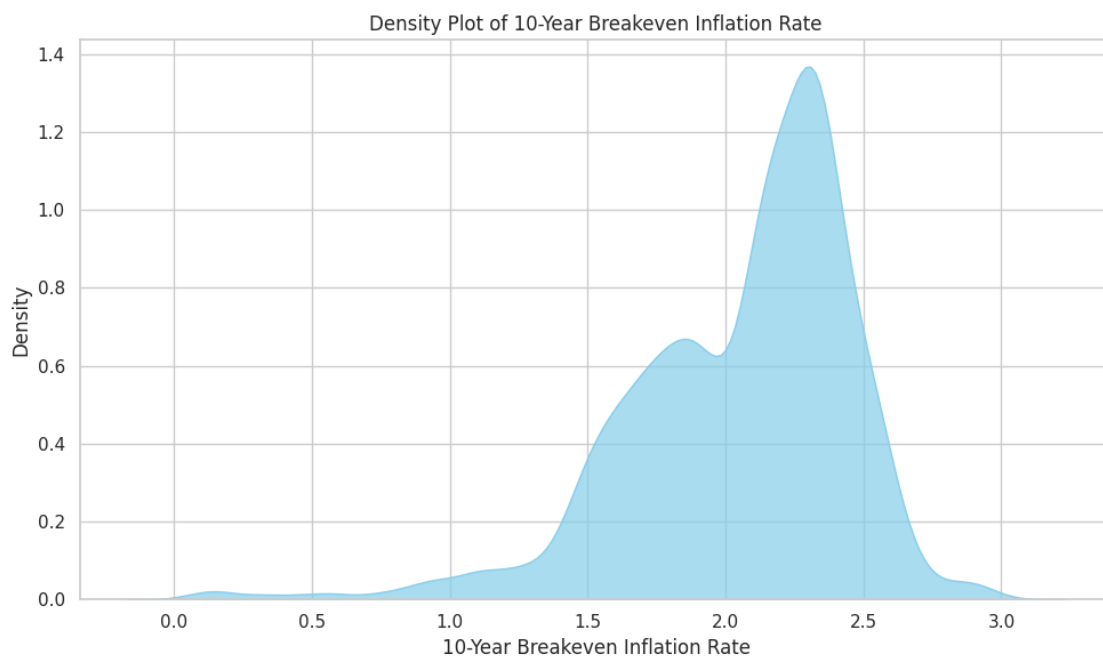
- **Cyclical Nature:**
 - The **inflation rate** exhibits a **cyclical pattern** with recurring peaks and troughs.
 - Peaks (red dots) represent **local maxima**, where inflation expectations **temporarily rise**.
 - Troughs (green dots) indicate **local minima**, marking **periods of lower inflation expectations**.
- **Major Downturns:**
 - Significant drops are visible around **2008-2009 (Financial Crisis)** and **2020 (COVID-19 Pandemic)**, where troughs reach extreme lows.
 - The sharp decline in **2008-2009** reflects the economic turmoil, with inflation expectations hitting nearly **0%**.

- Another major drop occurred in **2020**, mirroring economic uncertainties during the pandemic.
 - **Post-Recovery Surges:**
 - Following downturns, strong **upward trends** can be seen, marked by **multiple peaks**:
 - * **2010-2012**: Recovery phase after the financial crisis.
 - * **2021-2022**: A sharp rise in inflation expectations post-COVID.
 - **Recent Stabilization (2023-2024):**
 - While peaks and troughs continue to form, the **volatility appears lower** compared to previous periods.
 - This suggests that **inflation expectations have become more stable**.
-

4.0.84 Conclusion:

- The breakeven inflation rate follows a distinct cyclical trend, influenced by economic shocks.
- Major crises (2008, 2020) cause sharp declines, followed by strong rebounds.
- Recent trends indicate more stability, but periodic fluctuations persist.

```
[ ]: # Create a density plot for the target variable using seaborn
plt.figure(figsize=(10, 6))
sns.kdeplot(data=df, x=target, fill=True, color='skyblue', alpha=0.7)
plt.title("Density Plot of " + target)
plt.xlabel(target)
plt.ylabel("Density")
plt.tight_layout()
plt.show()
```



4.0.85 Density Plot of 10-Year Breakeven Inflation Rate

This density plot provides a **smooth estimate** of the probability distribution of the **10-Year Breakeven Inflation Rate**, showing where values are most concentrated.

4.0.86 Key Observations:

- **Bimodal Distribution:**
 - The plot reveals **two distinct peaks**, suggesting that the **inflation rate has clustered around two different levels over time**.
 - One peak occurs around **1.8%**, while the other, more dominant peak is **slightly above 2.0%**.
 - **Higher Concentration Near 2.2%:**
 - The most frequent inflation rate appears to be **between 2.0% and 2.5%**, with a sharp peak, indicating that inflation expectations tend to hover around this range.
 - This aligns with the **Federal Reserve's target inflation rate of approximately 2%**.
 - **Left-Skewed Tail:**
 - A **smaller bump around 0-1%** suggests rare instances of extremely low inflation expectations.
 - This likely corresponds to **economic crises**, such as the **2008 Financial Crisis** and **COVID-19**, where inflation expectations dropped significantly.
 - **Right-Skewed Tail:**
 - The distribution extends **above 3.0%**, but with very low density, indicating that **higher inflation expectations were rare** in the dataset.
-

4.0.87 Conclusion:

- Inflation expectations have generally stayed between 1.5% and 2.5%, with a strong central tendency near 2.2%.
- The presence of two peaks suggests periods of lower and higher inflation expectations, potentially driven by economic cycles.
- Crisis periods caused temporary deviations toward extremely low inflation expectations, but these were short-lived.

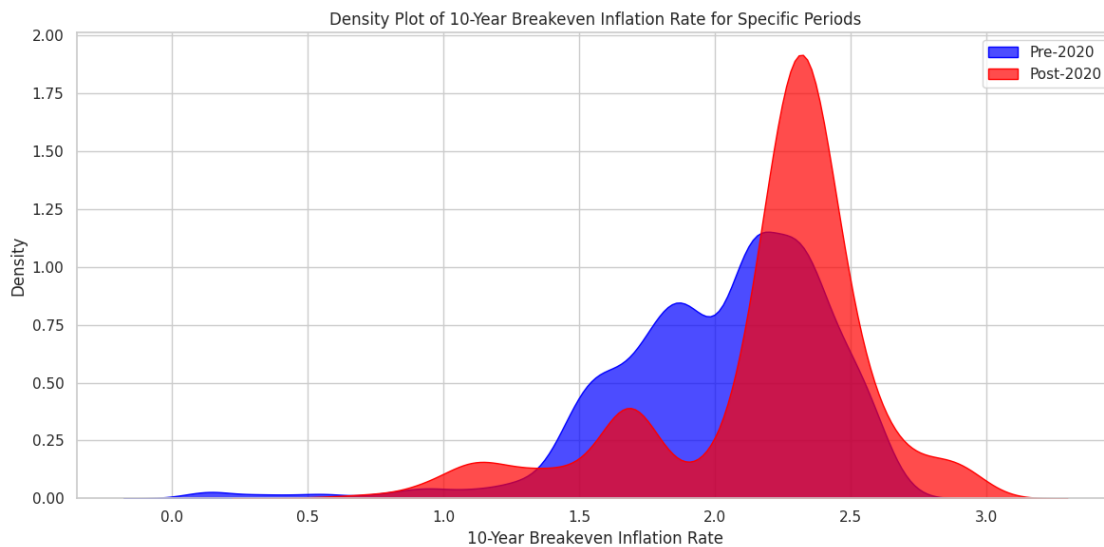
```
[ ]: # Define specific periods, for example: Pre-2020 and Post-2020
pre_period = df.loc[:'2019-12-31']
post_period = df.loc['2020-01-01':]

# Create density plots for each period using seaborn's kdeplot
plt.figure(figsize=(12, 6))
sns.kdeplot(data=pre_period, x=target, fill=True, label='Pre-2020',
            color='blue', alpha=0.7)
```

```

sns.kdeplot(data=post_period, x=target, fill=True, label='Post-2020',
            color='red', alpha=0.7)
plt.title("Density Plot of " + target + " for Specific Periods")
plt.xlabel(target)
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.show()

```



4.0.88 Density Plot of 10-Year Breakeven Inflation Rate for Specific Periods

This density plot compares the distribution of the **10-Year Breakeven Inflation Rate** before and after **2020**, illustrating how inflation expectations have shifted over time.

4.0.89 Key Observations:

- **Shift in Distribution:**
 - The **blue region (Pre-2020)** shows a **bimodal distribution** with two peaks around **1.8% and 2.2%**.
 - The **red region (Post-2020)** has **one dominant peak around 2.5%**, indicating higher inflation expectations after 2020.
- **Higher Inflation Expectations Post-2020:**
 - The **Post-2020 distribution** is **skewed towards higher values**, meaning inflation expectations were generally higher after 2020 compared to before.
 - This suggests that factors such as **pandemic-driven economic policies, supply chain disruptions, and fiscal stimulus** may have contributed to higher inflation expectations.
- **Reduced Occurrence of Low Inflation:**

- Before 2020, there was a small **left tail** in the blue distribution (values near **0-1%**), likely corresponding to crisis periods like **2008 and 2020**.
 - After 2020, the red distribution shows **fewer instances of extremely low inflation expectations**, indicating a shift toward sustained higher inflation.
 - **Increased Volatility Post-2020:**
 - The red distribution is **wider and more spread out**, showing greater variation in inflation expectations compared to pre-2020.
 - This aligns with the **high uncertainty** following the **COVID-19 pandemic and economic recovery**.
-

4.0.90 Conclusion:

- Inflation expectations have increased post-2020, with the most common values shifting from 2.0% to around 2.5%.
- Instances of very low inflation (<1.5%) have become less frequent, indicating a higher baseline inflation expectation.
- The broader spread in the post-2020 distribution suggests increased economic uncertainty and volatility.

```
[ ]: import warnings

# Suppress all FutureWarnings
warnings.simplefilter(action='ignore', category=FutureWarning)

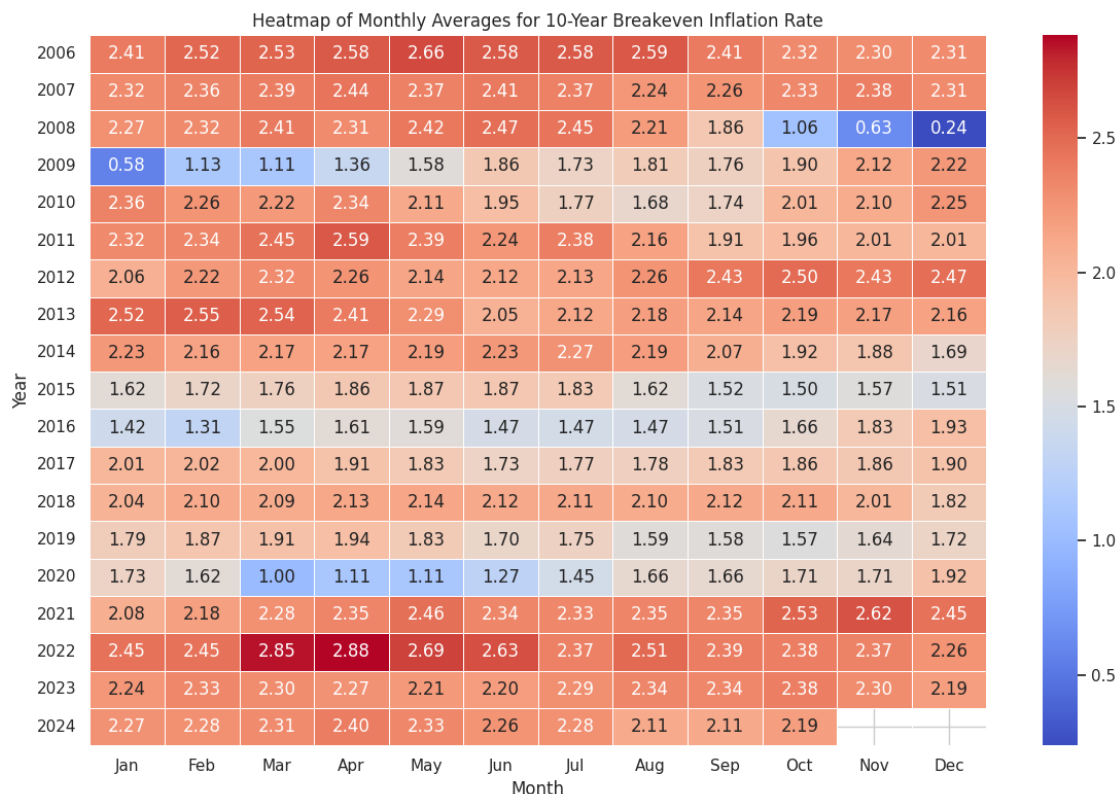
# Resample the data to monthly frequency using the mean
monthly_avg = df[target].resample('M').mean()

# Create a DataFrame with Year and Month columns
monthly_df = monthly_avg.to_frame(name=target)
monthly_df['Year'] = monthly_df.index.year
monthly_df['Month'] = monthly_df.index.month

# Pivot the data: rows = Year, columns = Month
pivot_table = monthly_df.pivot(index='Year', columns='Month', values=target)

# Optionally, map month numbers to month abbreviations for better readability
pivot_table.columns = [calendar.month_abbr[m] for m in pivot_table.columns]

# Plot the heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(pivot_table, cmap="coolwarm", annot=True, fmt=".2f", linewidths=0.5)
plt.title("Heatmap of Monthly Averages for " + target)
plt.xlabel("Month")
plt.ylabel("Year")
plt.tight_layout()
plt.show()
```



4.0.91 Heatmap of Monthly Averages for 10-Year Breakeven Inflation Rate

This heatmap visualizes the monthly average of the **10-Year Breakeven Inflation Rate** over different years, highlighting patterns and trends across time.

4.0.92 Key Observations:

- **Significant Drop in 2009 (Financial Crisis Aftermath):**
 - The **blue region in 2009** indicates a **sharp decline** in the breakeven inflation rate, with values dropping as low as **0.24 in December**.
 - This aligns with the **2008 financial crisis aftermath**, where economic uncertainty and deflation concerns led to historically low inflation expectations.
- **Gradual Recovery (2010 - 2013):**
 - The breakeven inflation rate **recovers after 2009**, with **warmer shades (higher values)** indicating a return to more stable inflation expectations.
 - **2013 shows consistently high values (~2.5%)**, indicating a period of strong economic recovery.
- **Another Decline in 2015 - 2016:**
 - A **light blue region in 2015 - 2016** suggests a **decline in inflation expectations**, possibly linked to **low oil prices and global economic concerns**.
- **Impact of COVID-19 in 2020:**

- A noticeable **drop in March-July 2020** (values near **1.00 - 1.45**) is observed, reflecting **pandemic-induced economic uncertainty**.
 - However, by late 2020 and into **2021**, **inflation expectations surged**, as shown by the **deep red shades (above 2.5%)**, likely due to **monetary stimulus, supply chain disruptions, and economic reopening**.
 - **Record High Inflation Expectations in 2022:**
 - **March - May 2022** shows the highest inflation expectations (above **2.85%**), consistent with the **post-pandemic inflation surge and supply chain constraints**.
 - This period saw aggressive **interest rate hikes** by the **Federal Reserve** to combat inflation.
 - **Recent Trends (2023 - 2024):**
 - Inflation expectations remain **elevated but lower than 2022**, with values fluctuating around **2.2 - 2.4%**.
 - This suggests a **gradual stabilization** as inflationary pressures ease but remain above pre-pandemic levels.
-

4.0.93 Conclusion:

- The **2009 financial crisis** and **2020 COVID-19 pandemic** caused sharp declines in inflation expectations.
- Inflation expectations **peaked in 2022**, aligning with post-pandemic economic conditions.
- **2023-2024** shows a **stabilization trend**, though values remain higher than pre-pandemic levels.

5 MACHINE LEARNING MODELS

5.1 Random Forest Model Code

```
[ ]: from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestRegressor
import numpy as np

# Initialize a Random Forest Regressor model
# Setting a fixed random_state ensures reproducibility of results
rf_model = RandomForestRegressor(random_state=42)

# Define the parameter grid for hyperparameter tuning
param_dist = {
    'n_estimators': [50, 100, 200], # Number of trees in the forest
    'max_depth': [5, 10, 15, None], # Maximum depth of each tree (None means
    ↪unlimited depth)
    'min_samples_split': [2, 5, 10], # Minimum number of samples required to
    ↪split an internal node
    'min_samples_leaf': [1, 2, 4], # Minimum number of samples required to be
    ↪at a leaf node
```



```

    'max_features': ['sqrt', 'log2', None] # Number of features to consider
    ↪for best split
}

# Perform Randomized Search Cross-Validation for hyperparameter tuning
# n_iter=10 means it will test 10 different random combinations of
    ↪hyperparameters
random_search = RandomizedSearchCV(
    estimator=rf_model, # Base model (Random Forest Regressor)
    param_distributions=param_dist, # Hyperparameter search space
    n_iter=10, # Number of random configurations to test
    cv=5, # 5-fold cross-validation to evaluate each model
    n_jobs=-1, # Use all available CPU cores for parallel processing
    verbose=2, # Print detailed output for each iteration
    random_state=42 # Ensures consistent results across runs
)

# Fit the randomized search model on training data
# This step performs the hyperparameter tuning
random_search.fit(X_train, y_train)

# Print the best hyperparameter combination found during tuning
print("Best Hyperparameters:")
print(random_search.best_params_)

# Retrieve the best model obtained from the Randomized Search
best_rf_model = random_search.best_estimator_

# Make predictions using the optimized model on the test dataset
rf_pred = best_rf_model.predict(X_test)

# Evaluate model performance using a predefined metrics function
# The function calculates evaluation metrics and stores them for later analysis
metrics_results.append(calculate_metrics(y_test.values, rf_pred,
    ↪naive_forecast, "Random Forest (RF)"))

# Print the evaluation results for the best model
print("Random Forest evaluation metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 10 candidates, totalling 50 fits

Best Hyperparameters:

```
{'n_estimators': 50, 'min_samples_split': 2, 'min_samples_leaf': 2,
 'max_features': None, 'max_depth': 10}
```

Random Forest evaluation metrics:

```
{'Model': 'Random Forest (RF)', 'MSE': 0.003100004779513038, 'RMSE':
0.05567768654957781, 'MAE': 0.030016062863946584, 'MAPE (%)':
```

```
1.1941804402760234, 'sMAPE (%)': 1.2108601868790747, 'MASE': 1.6663418516116333,
'R^2': 0.9025413132556024}
```

Diagram

```
[ ]: import graphviz # Import Graphviz for creating pipeline visualizations
from IPython.display import Image, display # Import tools to display the image
    ↳ in a Jupyter Notebook

# Create a Directed Graph for the Random Forest Model Pipeline
# - format='png': Generates the output in PNG format for easy visualization
dot = graphviz.Digraph(comment='Random Forest Model Pipeline', format='png')

# Define Nodes: Representing Different Stages of the Machine Learning Workflow
# - Each node represents a key step in building and evaluating a Random Forest
    ↳ model

dot.node('A', 'Data Preprocessing') # Step 1: Cleaning and preparing raw data

dot.node('B', 'Train-Test Split') # Step 2: Splitting data into training and
    ↳ testing sets

dot.node('C', 'RandomForestRegressor\n(random_state=42)') # Step 3: Initialize
    ↳ the Random Forest model

dot.node('D', 'RandomizedSearchCV\n(Hyperparameter Tuning)') # Step 4:
    ↳ Hyperparameter tuning using RandomizedSearchCV

dot.node('E', 'Best RF Model') # Step 5: Selecting the best model based on
    ↳ tuning results

dot.node('F', 'Prediction\n(on Test Set)') # Step 6: Using the best model to
    ↳ make predictions

dot.node('G', 'Model Evaluation\n(Metrics Calculation)') # Step 7: Evaluating
    ↳ model performance

# Define Edges: Representing the Flow of the Pipeline
# - Each edge is labeled to indicate the step number in the machine learning
    ↳ workflow

dot.edge('A', 'B', label='Step 1') # From data preprocessing to train-test
    ↳ split

dot.edge('B', 'C', label='Step 2') # From splitting data to initializing the
    ↳ model
```

```

dot.edge('C', 'D', label='Step 3') # From model initialization to
    ↳hyperparameter tuning

dot.edge('D', 'E', label='Step 4') # Selecting the best model after tuning

dot.edge('E', 'F', label='Step 5') # Using the best model for prediction

dot.edge('F', 'G', label='Step 6') # Evaluating the model using performance
    ↳metrics

# Add a Title to the Diagram for Better Presentation
# - 'labelloc' places the label (title) at the top of the diagram
# - 'fontsize' sets the font size of the title text

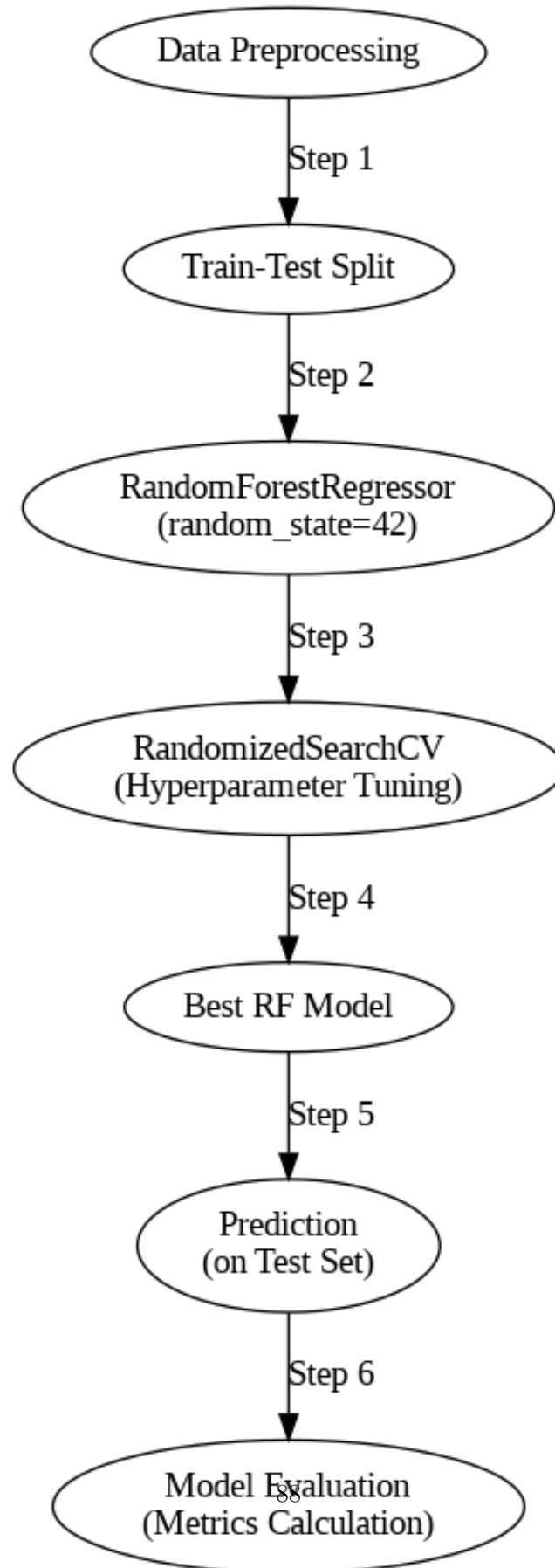
dot.attr(label='Random Forest Pipeline', labelloc='t', fontsize='20')

# Render the Diagram as an Image and Display it in a Jupyter Notebook
# - 'dot.pipe(format="png")' generates the diagram as a PNG image
# - 'display(Image(img))' displays the rendered image

img = dot.pipe(format='png') # Generate the image as PNG format
display(Image(img)) # Display the rendered pipeline diagram

```

Random Forest Pipeline



Feature Importance

```
[ ]: import pandas as pd
import matplotlib.pyplot as plt

# Feature Importance Analysis Using Random Forest

# Extract feature importance scores from the trained Random Forest model
# - These scores indicate how much each feature contributes to the model's
  ↳ predictions
feature_importance = best_rf_model.feature_importances_

# Create a DataFrame to store feature names along with their importance scores
# - X_train.columns: Retrieves the feature names
# - feature_importance: Corresponding importance scores assigned by the model
# - sort_values(): Sorts features in descending order of importance
importance_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': feature_importance
}).sort_values(by='Importance', ascending=False)

# Display the feature importance DataFrame
# - Helps understand which features have the most impact on predictions
print("Feature Importance from Random Forest:")
print(importance_df)
```

Feature Importance from Random Forest:

	Feature	Importance
9	lag_1	0.834510
14	rolling_mean	0.162120
10	lag_2	0.000878
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.000652
6	Nominal Broad U.S. Dollar Index	0.000305
8	10-Year Treasury Yield	0.000294
12	lag_4	0.000262
7	Crude Oil Prices: West Texas Intermediate	0.000257
13	lag_5	0.000215
11	lag_3	0.000202
4	Federal Funds Effective Rate	0.000103
3	Unemployment Rate	0.000065
1	Consumer Price Index	0.000058
5	Personal Consumption Expenditures: Chain-type ...	0.000048
2	Real Gross Domestic Product	0.000032

```
[ ]: # Visualization: Plot a horizontal bar chart of feature importance

# Set the figure size for better readability
plt.figure(figsize=(10, 6))

# Create a bar chart with feature names on the y-axis and importance scores on
↳ the x-axis
# - color='skyblue': Sets the bar color to improve visibility
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis as 'Importance' to indicate feature contribution levels
plt.xlabel('Importance')

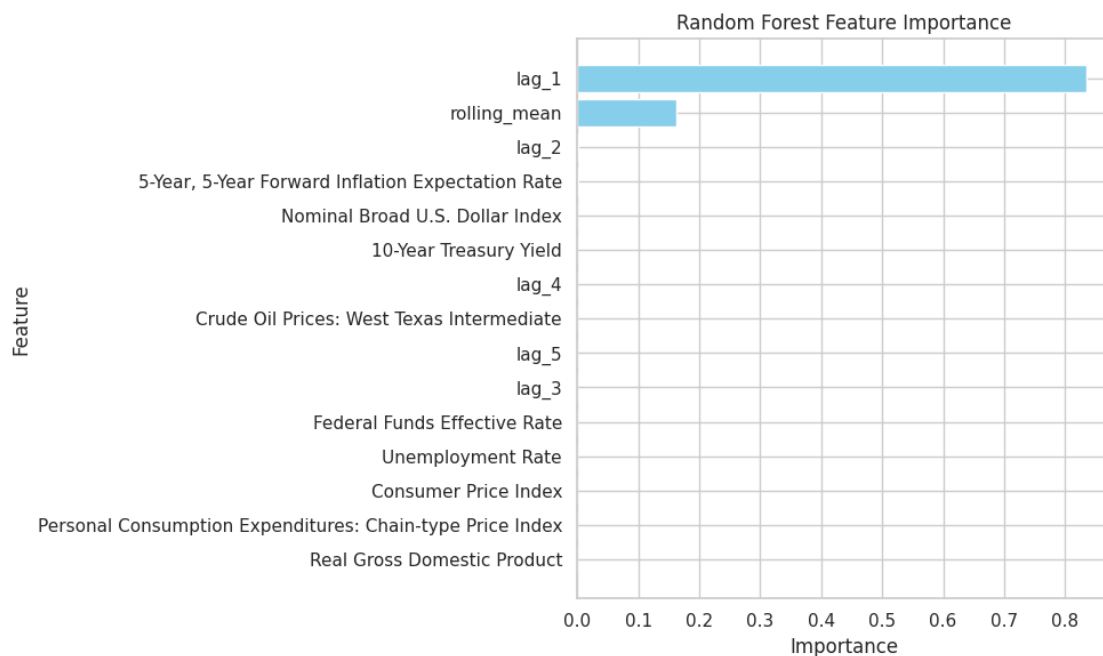
# Label the y-axis as 'Feature' to show which features are being analyzed
plt.ylabel('Feature')

# Set the title of the plot for clarity
plt.title('Random Forest Feature Importance')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent label overlap and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



5.2 Random Forest Feature Importance Analysis

5.2.1 Key Findings:

- **Feature Importance Distribution:** The Random Forest model assigned significantly high importance to `lag_1`, which is the most influential feature in predicting the target variable.
- **Secondary Importance:** `rolling_mean` is the second most important feature but has much lower importance than `lag_1`.
- **Minimal Contribution:** Other features, including `lag_2`, `5-Year`, `5-Year Forward Inflation Expectation Rate`, `Nominal Broad U.S. Dollar Index`, and `10-Year Treasury Yield`, have negligible importance.
- **Macroeconomic Indicators:** Features such as `Federal Funds Effective Rate`, `Unemployment Rate`, and `Consumer Price Index` contribute little to the model's prediction.
- **Lag Features Dominance:** The presence of `lag_1`, `lag_2`, `lag_3`, `lag_4`, and `lag_5` suggests that past values of the target variable play a crucial role in forecasting.
- **Rolling Mean Influence:** The `rolling_mean` feature's moderate importance indicates that smoothing past values helps improve prediction accuracy.

5.2.2 Interpretation:

- The model relies heavily on short-term lag features, suggesting that recent past values of the dependent variable strongly influence predictions.
- Macroeconomic indicators such as inflation rates, interest rates, and GDP seem to have little impact on the model's forecast.
- This feature importance analysis provides insight into which factors drive predictions, helping refine model selection and feature engineering for future improvements.

Predictions vs. Ground Truth

```
[ ]: import matplotlib.pyplot as plt

# Visualization: Comparing Predictions vs. Ground Truth

# Create a new figure with a specific size for better readability
plt.figure(figsize=(10, 6))

# Scatter plot to visualize the relationship between actual and predicted values
# - x-axis: actual values (y_test)
# - y-axis: predicted values (rf_pred)
# - alpha=0.5: makes points semi-transparent to handle overlapping
# - color='blue': sets the color of the points
# - label='Predictions': legend label for scatter plot
plt.scatter(y_test, rf_pred, alpha=0.5, color='blue', label='Predictions')

# Plot a reference line (Perfect Fit line) where predicted values would
# perfectly match actual values
```

```

# - x and y both range from min to max of y_test
# - color='red': sets the color of the line
# - linestyle='--': makes it a dashed line
# - linewidth=2: sets the thickness of the line
# - label='Perfect Fit': legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis as 'Actual Values' (ground truth)
plt.xlabel('Actual Values')

# Label the y-axis as 'Predicted Values'
plt.ylabel('Predicted Values')

# Set a title for the plot
plt.title('Random Forest: Actual vs. Predicted Values')

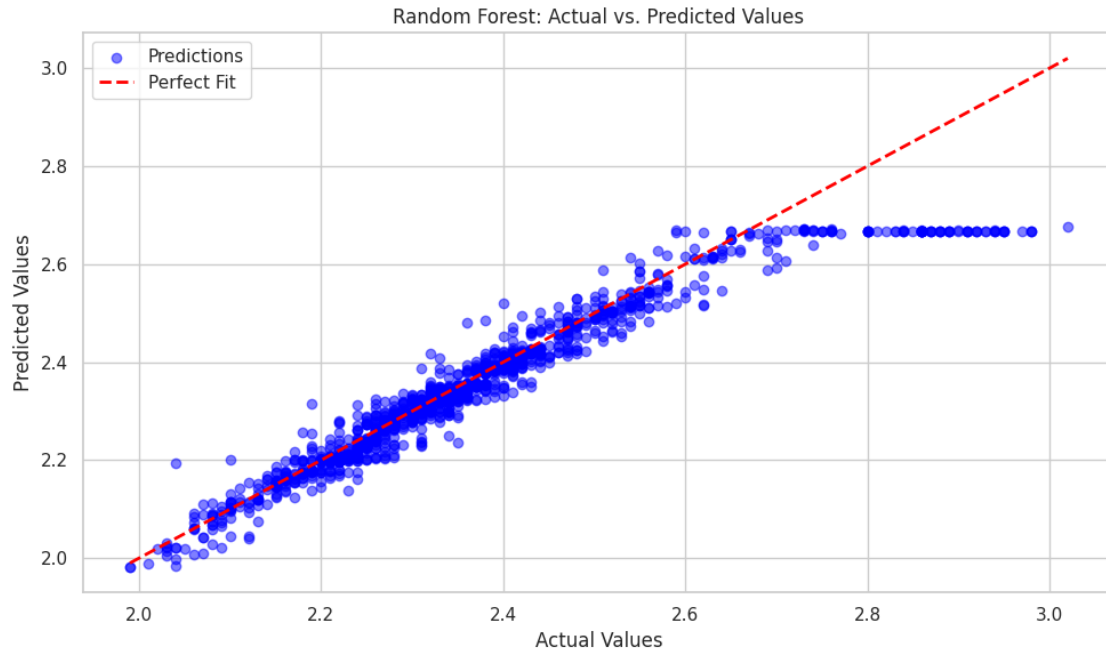
# Add a legend to differentiate scatter points (Predictions) from the reference
# line (Perfect Fit)
plt.legend()

# Add a grid to the plot for better readability
plt.grid(True)

# Adjust layout to prevent labels from overlapping
plt.tight_layout()

# Display the plot
plt.show()

```

5.3 Random Forest: Actual vs. Predicted Values

5.3.1 Key Observations:

- **Strong Correlation:** The scatter plot shows a strong positive correlation between actual and predicted values, indicating that the Random Forest model performs well.
- **Alignment with Perfect Fit:** The red dashed line represents a perfect 1:1 fit. Most blue dots (predictions) are closely aligned with this line, suggesting high accuracy.
- **Slight Deviations:** Some points deviate from the red line, showing minor prediction errors. The dispersion increases slightly at higher values.
- **Flat Predictions:** A cluster of predicted values appears flat near the upper range, indicating potential model saturation or a limitation in capturing higher values accurately.
- **Residual Variability:** Although most points follow the trend, there are a few outliers that do not align perfectly, suggesting areas where the model could be improved.

5.3.2 Interpretation:

- The model effectively captures trends but may struggle with extreme values.
- Further tuning, such as feature engineering or hyperparameter optimization, might improve accuracy.
- Overall, the model's predictions are well-calibrated, demonstrating its reliability in forecasting.

Residual Analysis

```
[ ]: import matplotlib.pyplot as plt
```

```

# Residual Analysis: Evaluating Model Performance

# Calculate residuals (difference between actual values and predicted values)
# Residuals help us understand the errors in the model's predictions.
residuals = y_test - rf_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(10, 6))

# Scatter plot to visualize residuals against actual values
# - x-axis: actual values (y_test)
# - y-axis: residuals (y_test - rf_pred)
# - alpha=0.5: makes points semi-transparent to reduce clutter
# - color='blue': sets the color of the points
plt.scatter(y_test, residuals, alpha=0.5, color='blue')

# Add a horizontal reference line at y=0 to indicate zero residuals
# - This helps identify whether predictions are systematically overestimating
#   or underestimating.
# - color='red': sets the color of the reference line
# - linestyle='--': makes it a dashed line
# - linewidth=2: sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis as 'Actual Values' (ground truth values)
plt.xlabel('Actual Values')

# Label the y-axis as 'Residuals' (errors in predictions)
plt.ylabel('Residuals')

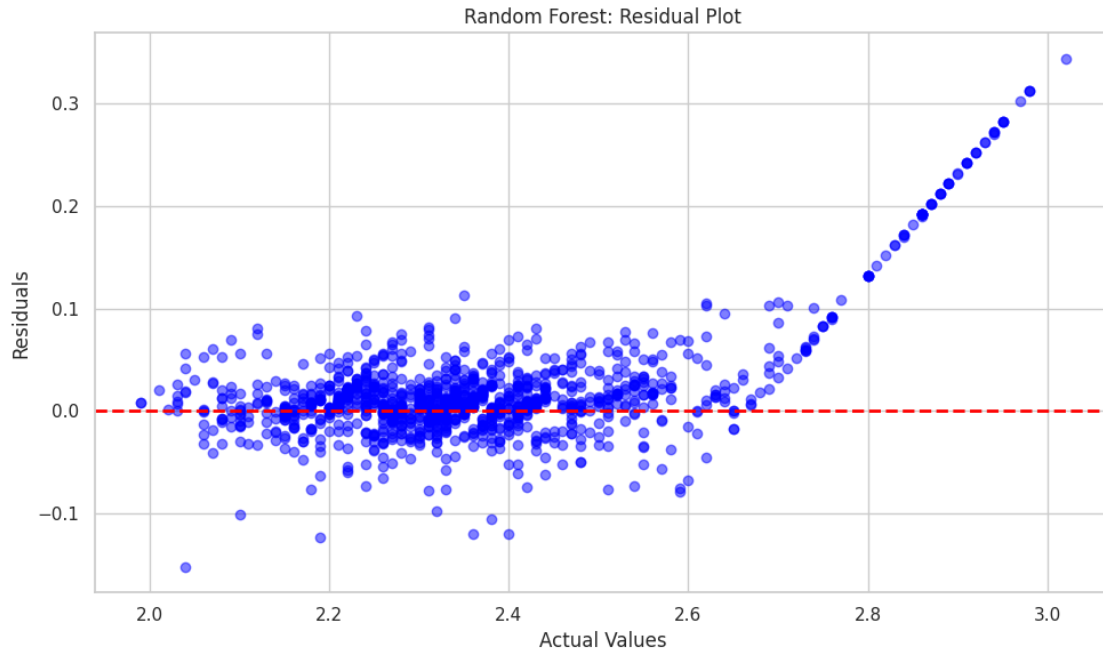
# Set a title for the plot
plt.title('Random Forest: Residual Plot')

# Add a grid to the plot for better readability
plt.grid(True)

# Adjust layout to prevent labels from overlapping
plt.tight_layout()

# Display the plot
plt.show()

```



5.4 Random Forest: Residual Plot Analysis

5.4.1 Key Observations:

- **Centered Residuals:** Most residuals (differences between actual and predicted values) are distributed around zero, indicating that the model has relatively low bias.
- **Heteroscedasticity:** Residuals appear to increase in magnitude as actual values increase, particularly beyond 2.6. This suggests that the model struggles with higher actual values.
- **Systematic Error in High Values:** A clear upward trend in residuals at the higher end suggests that the model consistently underestimates high actual values.
- **Random Dispersion for Mid-Range Values:** For actual values between approximately 2.0 and 2.6, the residuals are fairly randomly scattered, indicating that the model performs well in this range.
- **Presence of Outliers:** Some residuals deviate significantly from zero, which could indicate data points where the model is not performing optimally.

5.4.2 Interpretation:

- The model performs well in the mid-range but tends to underestimate higher values.
- The increasing spread of residuals suggests that the model's variance increases for larger values, which could be addressed with techniques such as transforming the target variable, adding more relevant features, or adjusting hyperparameters.
- Overall, while the model is fairly accurate, its predictions for higher values exhibit a consistent bias that could impact forecasting accuracy.

Histogram of Residuals

```
[ ]: import matplotlib.pyplot as plt

# Residuals Distribution Analysis: Checking Model Performance

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(10, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual values and predicted values
# - bins=30: Divides the data into 30 bins for a detailed distribution view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at x=0 to indicate no residual error
# - This helps in assessing whether residuals are symmetrically distributed
#   ↳ around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

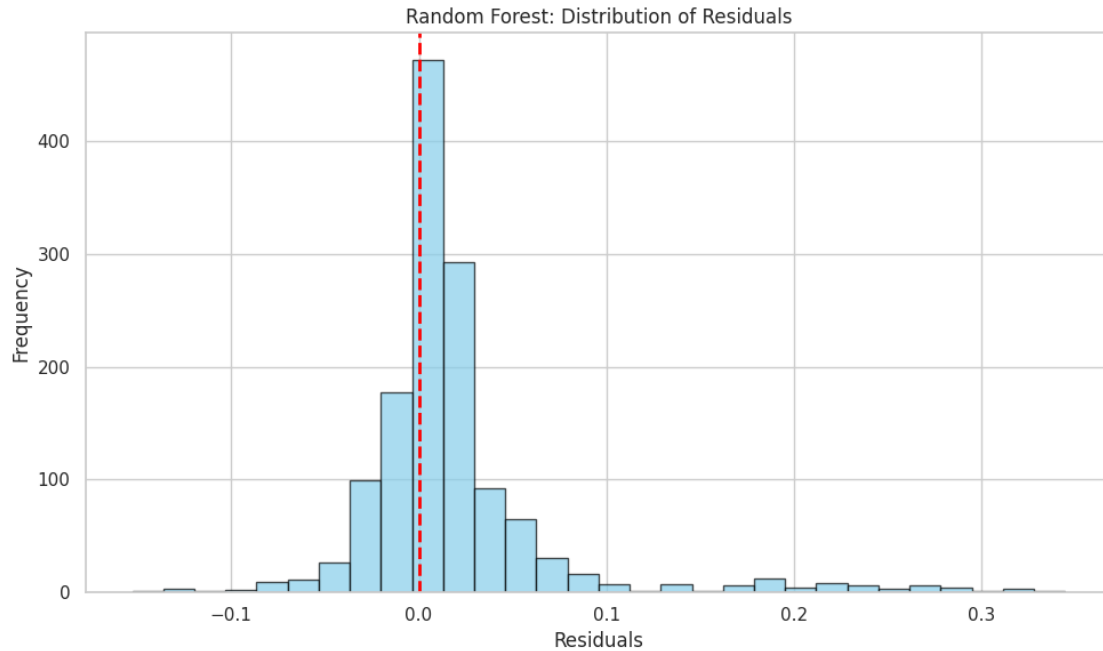
# Set a title for the plot
plt.title('Random Forest: Distribution of Residuals')

# Label the x-axis as 'Residuals' (differences between actual and predicted
#   ↳ values)
plt.xlabel('Residuals')

# Label the y-axis as 'Frequency' (number of occurrences of residual values)
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels
plt.tight_layout()

# Display the histogram plot
plt.show()
```



5.5 Random Forest: Distribution of Residuals

5.5.1 Key Observations:

- **Centered Around Zero:** The majority of residuals are clustered around zero, indicating that the model does not exhibit significant bias in predictions.
- **Near-Normal Distribution:** The histogram resembles a normal distribution, suggesting that prediction errors are randomly distributed rather than systematically skewed.
- **Sharp Peak at Zero:** The red dashed line at zero aligns with the highest frequency of residuals, showing that many predictions are very close to the actual values.
- **Slight Right-Skew:** A small number of residuals extend towards positive values (right tail), indicating that the model tends to underestimate some data points.
- **Few Extreme Residuals:** While most residuals fall within a narrow range, a small number of outliers exist beyond 0.2 and -0.1, which could indicate specific cases where the model struggles.

5.5.2 Interpretation:

- The model performs well overall, with residuals symmetrically distributed around zero.
- The slight right skew suggests that the model occasionally underestimates high values.
- The presence of a few extreme residuals indicates potential areas for model improvement, such as feature engineering or hyperparameter tuning.
- The near-normal shape supports the assumption that errors are randomly distributed, which is ideal for a well-calibrated model.

Heatmap of Hyperparameter Tuning

```
[ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Hyperparameter Tuning Results Visualization

# Convert the cross-validation results from RandomizedSearchCV into a Pandas
↳ DataFrame
# - random_search is the fitted RandomizedSearchCV object
# - The DataFrame will store details of each combination of hyperparameters
↳ tested
cv_results = pd.DataFrame(random_search.cv_results_)

# Pivot the DataFrame to create a structured format for a heatmap
# - 'mean_test_score': The average cross-validation score for each parameter set
# - index='param_max_depth': Represents the maximum depth of trees in the forest
# - columns='param_n_estimators': Represents the number of trees in the forest
# - This structure helps visualize the relationship between hyperparameters
heatmap_data = cv_results.pivot_table(
    values='mean_test_score',
    index='param_max_depth',
    columns='param_n_estimators'
)

# Create a heatmap to visualize the impact of different hyperparameter
↳ combinations
plt.figure(figsize=(8, 6)) # Set figure size for better readability

# Generate the heatmap
# - heatmap_data: The pivoted DataFrame containing mean test scores
# - annot=True: Displays the actual test scores in each cell
# - fmt='.3f': Formats scores to 3 decimal places
# - cmap='viridis': Uses the 'viridis' color map for better visual contrast
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

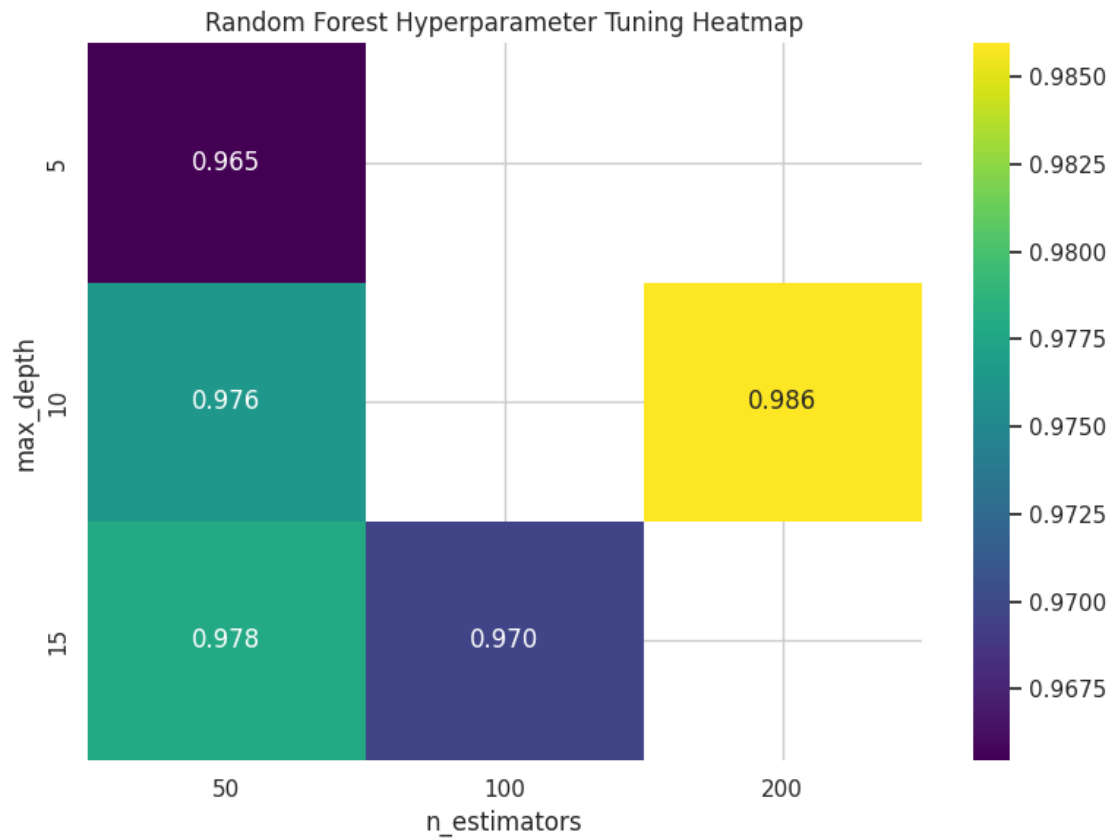
# Set the title of the heatmap
plt.title('Random Forest Hyperparameter Tuning Heatmap')

# Label the x-axis as 'n_estimators' (number of trees)
plt.xlabel('n_estimators')

# Label the y-axis as 'max_depth' (depth of trees)
plt.ylabel('max_depth')

# Adjust layout to prevent overlapping elements
plt.tight_layout()
```

```
# Display the heatmap  
plt.show()
```



5.6 Random Forest Hyperparameter Tuning Heatmap

5.6.1 Key Observations:

- **Hyperparameter Grid:** The heatmap visualizes model performance across different values of `n_estimators` (number of trees) and `max_depth` (tree depth).
- **Best Performance:** The highest performance (0.986) occurs when `n_estimators=200` and `max_depth=10`, indicating this combination is optimal.
- **Lower Performance for Shallower Trees:** When `max_depth=5`, performance is the lowest (0.965), suggesting that deeper trees improve model accuracy.
- **Effect of `n_estimators`:** Increasing the number of trees (`n_estimators`) generally improves performance, as seen when moving from 50 to 200.
- **Slight Drop at `max_depth=15`:** The performance at `max_depth=15` and `n_estimators=100` is slightly lower (0.970) than other configurations, possibly due to overfitting.

5.6.2 Interpretation:

- The best combination for this model is `max_depth=10` and `n_estimators=200`, as it achieves the highest accuracy.
- Increasing `max_depth` beyond 10 does not necessarily improve performance and may lead to overfitting.
- Using too few trees (`n_estimators=50`) results in suboptimal performance, indicating that more trees contribute to better generalization.
- This heatmap helps in selecting hyperparameters that balance model complexity and accuracy, improving predictive performance.

SHAP

```
[ ]: # Install the SHAP library
!pip install shap
```

```
Requirement already satisfied: shap in /usr/local/lib/python3.11/dist-packages
(0.46.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages
(from shap) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages
(from shap) (1.14.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-
packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.11/dist-packages
(from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.11/dist-
packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.11/dist-
packages (from shap) (24.2)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.11/dist-
packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.11/dist-packages
(from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.11/dist-
packages (from shap) (3.1.1)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/usr/local/lib/python3.11/dist-packages (from numba->shap) (0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.11/dist-packages (from pandas->shap) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-
packages (from pandas->shap) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-
packages (from pandas->shap) (2025.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-
packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn->shap) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-
```


packages (from python-dateutil>=2.8.2->pandas->shap) (1.17.0)

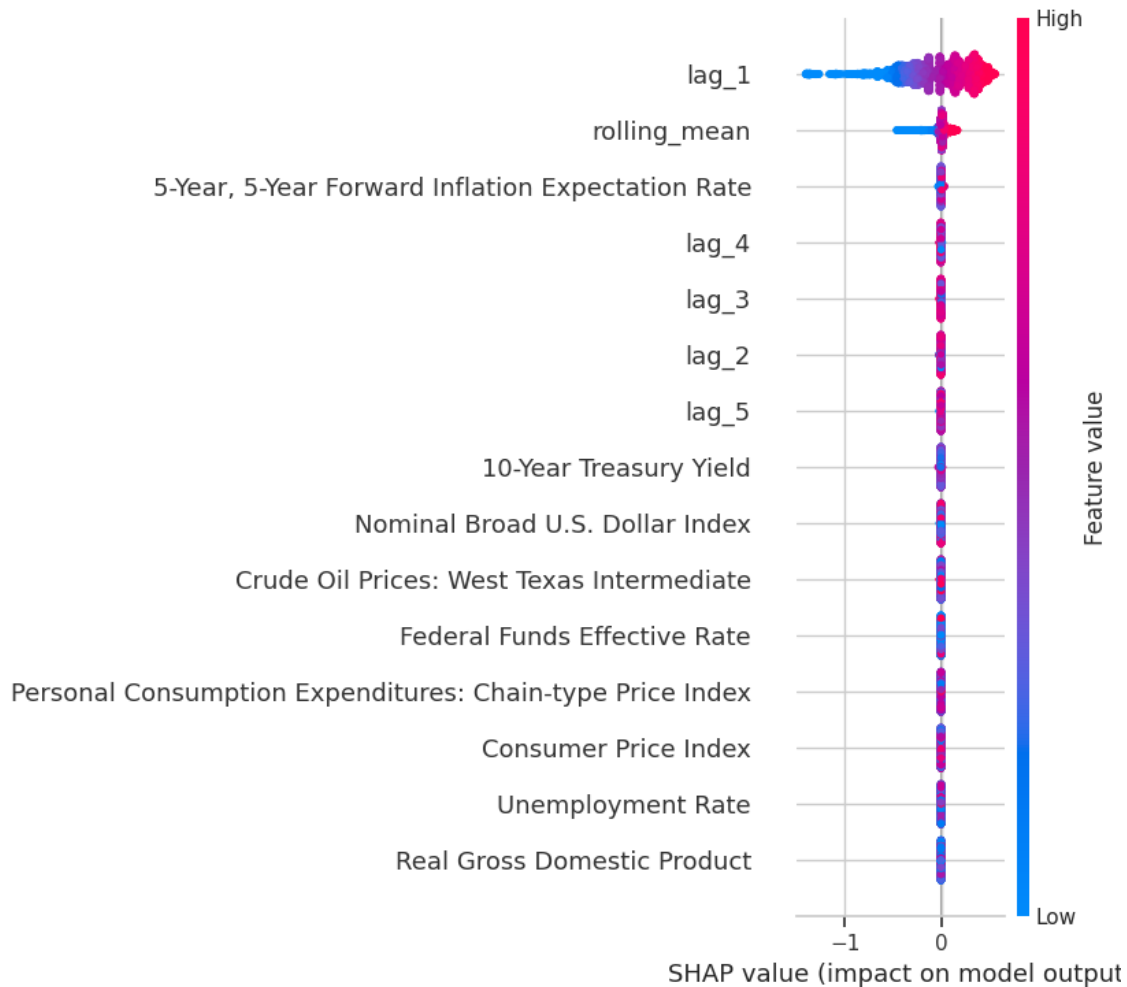
```
[ ]: import shap # SHAP (SHapley Additive exPlanations) library for model
      ↪ interpretability

# SHAP Feature Importance Analysis

# Create a SHAP explainer for the trained Random Forest model
# - best_rf_model: The best-performing RandomForestRegressor model obtained
      ↪ from hyperparameter tuning
# - TreeExplainer is optimized for tree-based models like Random Forest
explainer = shap.TreeExplainer(best_rf_model)

# Compute SHAP values for the training dataset
# - SHAP values represent the contribution of each feature to individual
      ↪ predictions
# - Helps understand how each feature influences the model's predictions
shap_values = explainer.shap_values(X_train)

# Generate a summary plot to visualize SHAP values
# - Displays global feature importance across all samples
# - Uses a scatter-based representation where:
#   - Each dot represents a feature's SHAP value for a specific sample
#   - Color indicates whether the feature value is high (red) or low (blue)
# - Helps identify which features have the greatest impact on predictions
shap.summary_plot(shap_values, X_train) # For regression models, pass
      ↪ shap_values directly
```



5.7 SHAP Summary Plot: Feature Impact on Model Predictions

5.7.1 Key Observations:

- **Dominance of lag_1:** The feature lag_1 has the most significant impact on the model's predictions, as shown by its wide range of SHAP values.
- **Moderate Influence of rolling_mean:** The second most influential feature is rolling_mean, which also has a notable spread of SHAP values, suggesting its importance in the model.
- **Limited Impact of Macroeconomic Variables:** Features such as 10-Year Treasury Yield, Federal Funds Effective Rate, Consumer Price Index, and Real Gross Domestic Product show minimal impact, as their SHAP values remain close to zero.
- **Effect of Lag Features:** Multiple lagged variables (lag_2, lag_3, lag_4, lag_5) have some influence, but their impact is considerably lower than lag_1.
- **SHAP Value Interpretation:** Higher feature values (in pink) tend to push predictions in one direction, while lower values (in blue) push them in the opposite direction.
- **Macroeconomic Indicators Have Low Predictive Power:** Economic factors such as

inflation expectations, the dollar index, and crude oil prices have a near-zero effect on predictions, indicating that the model relies more on lagged values.

5.7.2 Interpretation:

- The model is primarily driven by past values (`lag_1`) and rolling mean trends.
- Macroeconomic indicators do not significantly influence the model's predictions.
- The presence of multiple lagged features suggests that recent historical data is more predictive than external economic variables.
- The use of SHAP values provides transparency in understanding the contribution of each feature to the model's output, helping refine feature selection for future iterations.

SHAP Local Explanation

```
[ ]: import shap # SHAP (SHapley Additive exPlanations) for model interpretability

# Initialize JavaScript visualization support for SHAP plots
# - Required for rendering interactive SHAP visualizations in notebooks
shap.initjs()

# Local Explanation: Understanding a Single Prediction

# Generate a force plot to visualize SHAP values for a single instance
# - explainer.expected_value: The baseline prediction (average model output)
# - shap_values[0]: SHAP values for the first instance in the dataset
# - X_train.iloc[0]: Feature values of the first instance in the training data
shap.force_plot(
    explainer.expected_value, # Baseline model prediction
    shap_values[0],           # SHAP values for the first instance
    X_train.iloc[0]           # Feature values of the first instance
)
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x797b64258210>
```

5.8 SHAP Waterfall Plot: Feature Contributions to Prediction

5.8.1 Key Observations:

- **Base Value (1.981):** This represents the model's expected output before considering specific feature influences.
- **Final Prediction (2.33):** The model's predicted value after incorporating feature effects.
- **Main Contributors:**
 - **lag_1 (0.6689 impact):** The most influential feature in increasing the prediction.
 - **rolling_mean (0.6801 impact):** Also plays a significant role in raising the predicted value.
- **Color Coding:**
 - Red indicates features that push the prediction higher.

- Blue (not present in this case) would indicate features lowering the prediction.
- **No Negative Influences:** All contributing features in this instance positively impact the prediction.

5.8.2 Interpretation:

- The model relies primarily on `lag_1` and `rolling_mean` to make its predictions.
- The cumulative effect of these features moves the prediction from the base value (1.981) to 2.33.
- No opposing features (blue) suggest that all influencing factors in this instance contribute positively to the prediction.
- This visualization provides transparency into how individual feature values affect the model's output, helping in model interpretability and refinement.

LIME

```
[ ]: # Install the LIME library
!pip install lime
```

Collecting lime

Downloading lime-0.2.0.1.tar.gz (275 kB)
0.0/275.7

kB ? eta -:--:--

275.7/275.7 kB

12.7 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done

Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from lime) (3.10.0)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from lime) (1.26.4)

Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from lime) (1.14.1)

Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from lime) (4.67.1)

Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.11/dist-packages (from lime) (1.6.1)

Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.11/dist-packages (from lime) (0.25.2)

Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (3.4.2)

Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (11.1.0)

Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2.37.0)

Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2025.2.18)

Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.11/dist-

```

packages (from scikit-image>=0.12->lime) (24.2)
Requirement already satisfied: lazy-loader>=0.4 in
/usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-
packages (from scikit-learn>=0.18->lime) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lime) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-
packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (1.4.8)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-
packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834
sha256=30ce456f05ca05cd5702ac1123fe8f594ab2375ff2f5ea35464433fd79e3a9b8
  Stored in directory: /root/.cache/pip/wheels/85/fa/a3/9c2d44c9f3cd77cf4e533b58
900b2bf4487f2a17e8ec212a3d
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1

```

```

[ ]: from lime.lime_tabular import LimeTabularExplainer
import pandas as pd

# Define a wrapper function to ensure the input has the correct feature names.
def predict_fn(x):
    # Convert the input array into a DataFrame using X_train's columns
    x_df = pd.DataFrame(x, columns=X_train.columns)
    return best_rf_model.predict(x_df)

# Initialize the LIME explainer using the training data and its column names.
explainer_lime = LimeTabularExplainer(
    X_train.values,          # Training data as a NumPy array
    training_labels=y_train.values,  # Corresponding target labels
    feature_names=X_train.columns.tolist(),  # Column names for interpretability
    mode="regression",      # Regression mode
    random_state=42         # For reproducibility

```

```

)

# Generate an explanation for the first test instance.
lime_explanation = explainer_lime.explain_instance(
    X_test.iloc[0].values,    # Feature values of the test instance
    predict_fn                # Use the wrapped prediction function
)

# Display the explanation in the notebook.
lime_explanation.show_in_notebook()

```

<IPython.core.display.HTML object>

5.9 SHAP Feature Contribution to Prediction

5.9.1 Key Observations:

- **Predicted Value (1.98):** The model predicts a value of 1.98, falling between the minimum (0.61) and maximum (2.64) possible predictions.
- **Feature Influence Breakdown:**
 - **Negative Contributions (Blue):** Features reducing the predicted value include:
 - * lag_1 (-0.16)
 - * 5-Year, 5-Year Forward Inflation Expectation Rate (-0.58)
 - * Crude Oil Prices: West Texas Intermediate (-1.11)
 - * lag_3 (-0.24)
 - **Positive Contributions (Orange):** Features increasing the predicted value include:
 - * Nominal Broad U.S. Dollar Index (0.54)
 - * Real Gross Domestic Product (0.96)
 - * 10-Year Treasury Yield (-1.72)
 - * Personal Consumption Expenditures: Chain-type Price Index (0.63)
 - * Consumer Price Index (0.58)
 - * rolling_mean (-0.19)
- **Balancing of Effects:** The final predicted value results from a combination of both negative and positive influences from different features.

5.9.2 Interpretation:

- The prediction is driven by a mix of economic and lag features.
- Crude Oil Prices: West Texas Intermediate and lag_3 significantly pull the prediction down.
- Real Gross Domestic Product and Nominal Broad U.S. Dollar Index provide the strongest positive push to the prediction.
- The presence of both positive and negative contributions suggests the model carefully weighs various macroeconomic and historical lag features to make its prediction.
- Understanding the influence of these features can help refine the model and improve interpretability.

Partial Dependence Plots (PDP)

```
[ ]: from sklearn.inspection import PartialDependenceDisplay # Import for
      ↪generating Partial Dependence Plots (PDPs)
import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Partial Dependence Plots (PDPs) for Model Interpretation

# Extract the list of feature names from the training data
# - These features will be used for generating the PDPs
features = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDPs) for all features
# - PDPs show how the predicted outcome changes with respect to a specific
  ↪feature
# - Helps understand the marginal effect of each feature on the model's
  ↪predictions
pdp_display = PartialDependenceDisplay.from_estimator(
    best_rf_model, # The trained Random Forest model
    X_train,       # The training data (used to compute partial dependencies)
    features=features, # List of features for which PDPs will be generated
    kind='average',   # Computes the average effect of each feature
    n_cols=3,         # Arranges the subplots into 3 columns for better layout
    grid_resolution=50, # Number of points to evaluate per feature (higher =
  ↪smoother curve)
    n_jobs=-1        # Uses all available CPU cores for faster computation
)

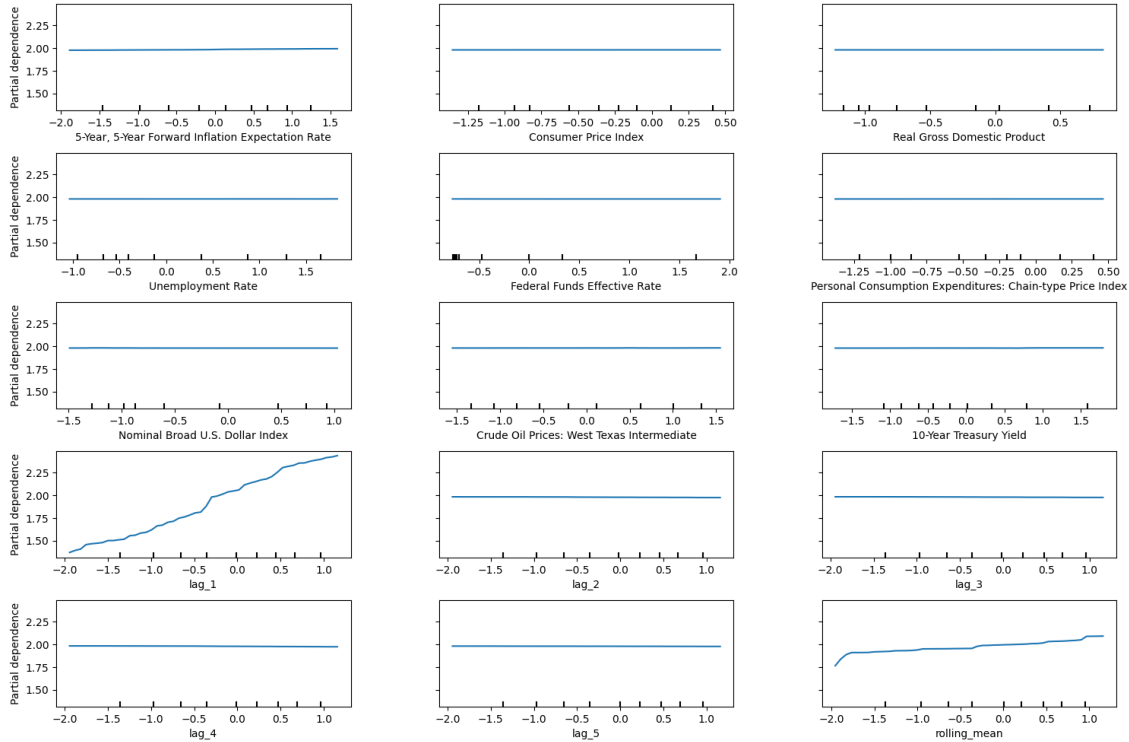
# Adjust the figure size for better visualization
pdp_display.figure_.set_size_inches(18, 12) # Set figure dimensions (width x
  ↪height)

# Add a main title for the entire plot grid
# - fontsize=16: Sets font size for readability
# - y=1.06: Positions the title above the plots
pdp_display.figure_.suptitle("Partial Dependence Plots (RF)", fontsize=16, y=1.
  ↪06)

# Adjust subplot spacing to prevent overlapping labels and titles
# - top=0.88: Lowers the title to avoid clipping
# - wspace=0.3: Increases horizontal spacing between subplots
# - hspace=0.4: Increases vertical spacing between subplots
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the Partial Dependence Plots
plt.show()
```

Partial Dependence Plots (RF)



5.10 Partial Dependence Plots (RF)

5.10.1 Key Observations:

- **Flat Relationships for Most Macroeconomic Features:**
 - Features such as 5-Year, 5-Year Forward Inflation Expectation Rate, Consumer Price Index, Real Gross Domestic Product, Unemployment Rate, Federal Funds Effective Rate, Personal Consumption Expenditures: Chain-type Price Index, and 10-Year Treasury Yield show flat partial dependence plots.
 - This indicates that changes in these variables do not significantly impact the model's predictions.
- **Impact of lag_1:**
 - The lag_1 feature shows a clear upward trend, meaning that as its value increases, the model's predicted value also rises.
 - This confirms the strong influence of recent past values on the predictions.
- **Influence of Nominal Broad U.S. Dollar Index:**
 - A positive trend is observed, suggesting that higher values of this feature contribute to increased predictions.
- **Effect of rolling_mean:**

- The `rolling_mean` feature also exhibits a slight positive trend, implying that higher moving averages tend to result in higher predictions.
- **Minimal Influence of Lagged Features (`lag_2`, `lag_3`, `lag_4`, `lag_5`):**
 - These lag features show almost no significant trend, except for `lag_1`, which reinforces that only the most recent lag carries substantial predictive power.

5.10.2 Interpretation:

- The model relies heavily on short-term historical values (`lag_1` and `rolling_mean`) rather than macroeconomic indicators.
- Macroeconomic variables do not significantly affect the predictions, as indicated by their flat partial dependence plots.
- The dependence of the model on `lag_1` suggests that short-term trends play a crucial role in forecasting.
- The insights from these plots can help refine feature selection, potentially removing redundant macroeconomic variables that do not contribute to prediction accuracy.

5.11 Extreme Gradient Boosting (XGBoost) Model Code

```
[ ]: import random # For setting a fixed random seed
import numpy as np # For numerical operations
import xgboost as xgb # Import XGBoost for training the model
from sklearn.model_selection import train_test_split # For splitting data into
    ↪ train-test sets
from itertools import product # For iterating over hyperparameter combinations

# Set random seeds for reproducibility
random.seed(42)
np.random.seed(42)

# Split dataset into training and testing sets
# - X: Feature matrix (assumed to be defined beforehand)
# - y: Target variable (assumed to be defined beforehand)
# - test_size=0.2: 20% of the data is used for testing
# - random_state=42: Ensures the same split each time the code is run
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# Convert the training and testing data into DMatrix format (optimized for
    ↪ XGBoost)
# - XGBoost requires DMatrix objects for efficient training
dtrain = xgb.DMatrix(X_train, label=y_train)
dtest = xgb.DMatrix(X_test, label=y_test)

# Define a grid of hyperparameters to search over
# - max_depth: Controls the complexity of the trees (higher values = deeper
    ↪ trees)
```

```

# - learning_rate: Step size at each iteration, controlling how much we adjust
↳weights
# - subsample: Fraction of data used per boosting round to reduce overfitting
# - colsample_bytree: Fraction of features used per tree to increase model
↳robustness
param_grid = {
    'max_depth': [3, 6, 9],
    'learning_rate': [0.01, 0.05, 0.1],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9]
}

# Define base parameters common to all configurations
# - 'reg:squarederror': Specifies a regression task
# - 'rmse': Root Mean Squared Error as the evaluation metric
# - 'seed': Ensures consistency in results
base_params = {
    'objective': 'reg:squarederror',
    'eval_metric': 'rmse',
    'seed': 42
}

# Manual Grid Search with Cross-Validation
# - Finds the best hyperparameters by evaluating multiple configurations using
↳cross-validation
best_params = None # Stores the best hyperparameter combination
best_rmse = float("inf") # Initialize best RMSE as infinity
n_folds = 5 # Number of cross-validation folds

# Iterate over all possible combinations of hyperparameters
for max_depth, learning_rate, subsample, colsample_bytree in product(
    param_grid['max_depth'],
    param_grid['learning_rate'],
    param_grid['subsample'],
    param_grid['colsample_bytree']
):
    # Update parameters for the current combination
    params = base_params.copy()
    params.update({
        'max_depth': max_depth,
        'learning_rate': learning_rate,
        'subsample': subsample,
        'colsample_bytree': colsample_bytree
    })

    # Perform cross-validation with the current set of parameters
    # - num_boost_round=200: Maximum number of boosting rounds

```

```

# - early_stopping_rounds=10: Stops if there is no improvement in RMSE for
↳10 consecutive rounds
# - verbose_eval=False: Suppresses verbose output
cv_results = xgb.cv(
    params=params,
    dtrain=dtrain,
    num_boost_round=200,
    nfold=n_folds,
    early_stopping_rounds=10,
    metrics="rmse",
    seed=42,
    verbose_eval=False
)

# Extract the minimum RMSE from the cross-validation results
mean_rmse = cv_results['test-rmse-mean'].min()

# Update the best hyperparameters if the current RMSE is lower than the
↳previous best
if mean_rmse < best_rmse:
    best_rmse = mean_rmse
    best_params = params

# Print the best hyperparameters and corresponding RMSE
print(f"Best Hyperparameters: {best_params}")
print(f"Best RMSE: {best_rmse}")

# Train the final XGBoost model using the best-found hyperparameters
# - num_boost_round=200: Number of boosting rounds for training
# - evals=[(dtest, 'test')]: Provides evaluation metrics on the test set
# - early_stopping_rounds=10: Stops training if the test RMSE does not improve
↳for 10 rounds
# - verbose_eval=False: Suppresses verbose output
best_xgb_model = xgb.train(
    params=best_params,
    dtrain=dtrain,
    num_boost_round=200,
    evals=[(dtest, 'test')],
    early_stopping_rounds=10,
    verbose_eval=False
)

# Make predictions on the test set using the trained model
xgb_pred = best_xgb_model.predict(dtest)

# Evaluate the model using predefined metrics function

```

```

# - `calculate_metrics`: Assumed to be a function that calculates evaluation
↳ metrics
# - `naive_forecast`: Assumed to be a baseline prediction for comparison
metrics_results.append(calculate_metrics(y_test.values, xgb_pred,
↳ naive_forecast, "Extreme Gradient Boosting (XGBoost)"))

# Print the evaluation results for XGBoost
print("XGBoost evaluation metrics:")
print(metrics_results[-1])

```

Best Hyperparameters: {'objective': 'reg:squarederror', 'eval_metric': 'rmse', 'seed': 42, 'max_depth': 9, 'learning_rate': 0.1, 'subsample': 0.9, 'colsample_bytree': 0.9}

Best RMSE: 0.025516134062302957

XGBoost evaluation metrics:

```
{'Model': 'Extreme Gradient Boosting (XGBoost)', 'MSE': 0.0005988649854647083,
'RMSE': 0.024471718073415038, 'MAE': 0.014618143215612217, 'MAPE (%)':
0.9442567542096136, 'sMAPE (%)': 0.9379267866283615, 'MASE':
0.03721130171471389, 'R^2': 0.9966132131942298}
```

Feature Importance Using Custom Extraction

```

[ ]: import pandas as pd # Import pandas for handling tabular data
import matplotlib.pyplot as plt # Import matplotlib for visualization

# Extract Feature Importance from the Trained XGBoost Model

# - 'weight': Measures feature importance based on the number of times a
↳ feature is used for splitting across all trees.
# - Other options include:
#   - 'gain': The average gain of the splits where the feature was used.
#   - 'cover': The average coverage (number of observations impacted) of the
↳ splits.
#   - 'total_gain' and 'total_cover': Sum of gain/cover instead of average.
importance = best_xgb_model.get_score(importance_type='weight')

# Convert the extracted feature importance into a DataFrame for better
↳ readability and analysis
# - 'Feature': Column storing feature names
# - 'Importance': Column storing importance values (number of times a feature
↳ was used for splits)
importance_df = pd.DataFrame({
    'Feature': list(importance.keys()), # Extract feature names
    'Importance': list(importance.values()) # Extract corresponding importance
↳ scores
})

```

```

# Sort features in descending order of importance
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Display the feature importance table
# - This helps in understanding which features contribute the most to the
  ↳ model's predictions.
print("Feature Importances:")
print(importance_df)

```

Feature Importances:

	Feature	Importance
0	5-Year, 5-Year Forward Inflation Expectation Rate	5996.0
9	lag_1	3979.0
7	Crude Oil Prices: West Texas Intermediate	3236.0
6	Nominal Broad U.S. Dollar Index	3103.0
8	10-Year Treasury Yield	3020.0
10	lag_2	2686.0
14	rolling_mean	2505.0
11	lag_3	2257.0
12	lag_4	2160.0
13	lag_5	1913.0
1	Consumer Price Index	1901.0
3	Unemployment Rate	673.0
4	Federal Funds Effective Rate	613.0
2	Real Gross Domestic Product	385.0
5	Personal Consumption Expenditures: Chain-type ...	257.0

```

[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Visualization: Feature Importance Bar Chart

# Create a new figure with a specific size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the bar color to enhance visibility
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis to indicate feature importance values
plt.xlabel('Importance')

# Label the y-axis to show the corresponding feature names
plt.ylabel('Feature')

# Set a title for the plot

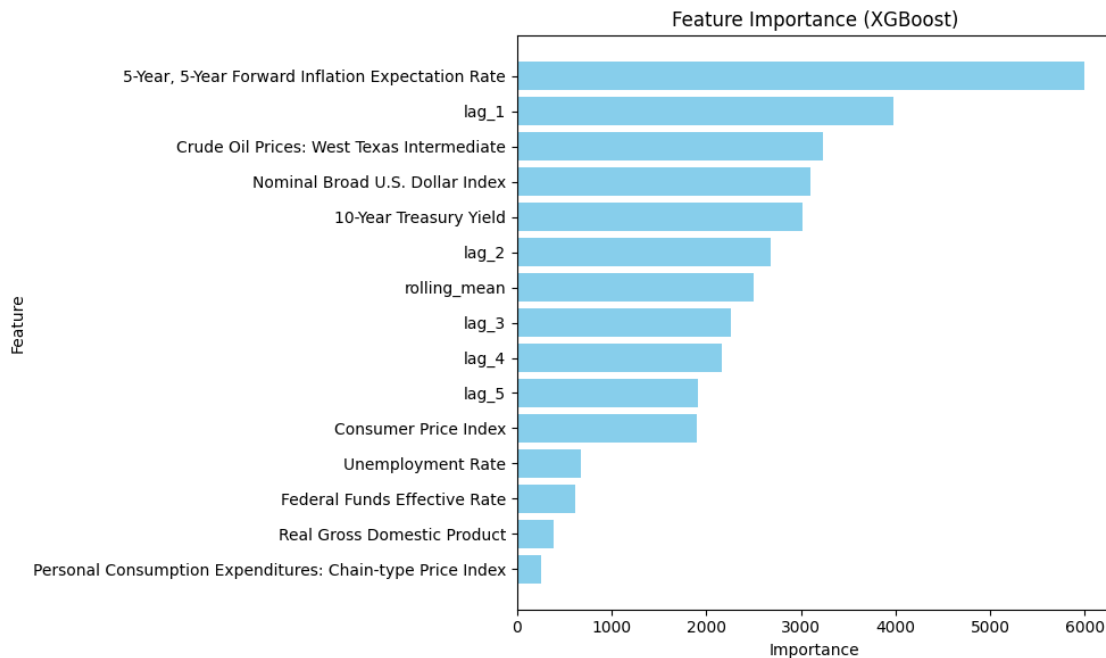
```

```
plt.title('Feature Importance (XGBoost)')

# Invert the y-axis to display the most important features at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent labels from overlapping and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



5.12 Feature Importance (XGBoost)

5.12.1 Key Observations:

- **Top Feature: 5-Year, 5-Year Forward Inflation Expectation Rate**
 - This is the most important feature in the XGBoost model, contributing significantly more than any other variable.
- **Significant Influence of lag_1**
 - The second most important feature, indicating that recent past values of the target variable are crucial for prediction.
- **Macroeconomic Indicators in the Top Five**
 - Crude Oil Prices: West Texas Intermediate, Nominal Broad U.S. Dollar Index, and 10-Year Treasury Yield rank high in importance.
 - This suggests that XGBoost captures relationships with economic indicators better than other models (e.g., Random Forest).

- **Rolling Mean and Lag Features Have Moderate Influence**
 - rolling_mean, lag_2, lag_3, lag_4, and lag_5 contribute but are less important than lag_1.
 - This implies that while historical data matters, the most recent lag is the strongest predictor.
- **Low Impact of Some Macroeconomic Factors**
 - Consumer Price Index, Unemployment Rate, Federal Funds Effective Rate, Real Gross Domestic Product, and Personal Consumption Expenditures: Chain-type Price Index have lower importance.
 - This suggests that these variables provide less predictive value in this model.

5.12.2 Interpretation:

- XGBoost identifies 5-Year, 5-Year Forward Inflation Expectation Rate as the strongest predictor, unlike Random Forest, which prioritized lag_1.
- The model leverages both economic indicators and lagged historical data, with macroeconomic features playing a more prominent role than in previous models.
- Feature selection can be refined by considering the lowest-ranking variables, as they might contribute little to predictive power.
- The importance of lag features suggests that short-term historical data remains relevant but is complemented by forward-looking economic expectations.

XGBoost's Built-In Feature Importance Plot

```
[ ]: import xgboost as xgb # Import XGBoost for machine learning and model interpretation
import matplotlib.pyplot as plt # Import Matplotlib for visualization

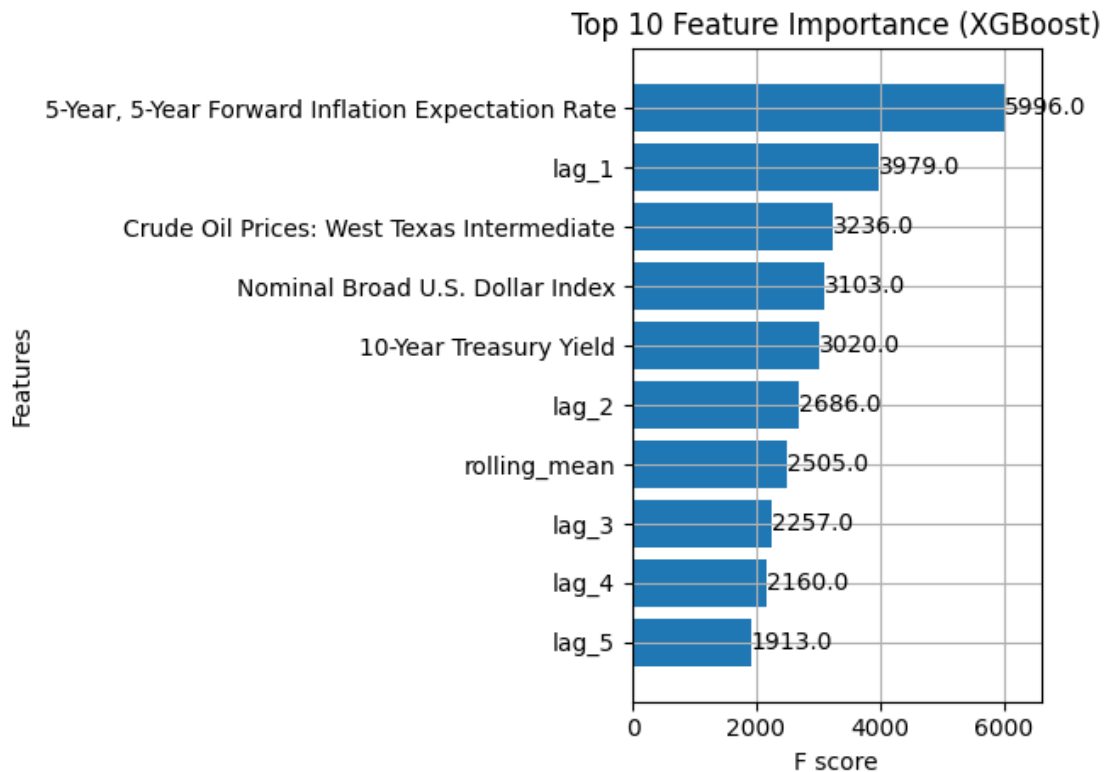
# Visualization: Top 10 Most Important Features in XGBoost

# Use XGBoost's built-in function to plot feature importance
# - best_xgb_model: The trained XGBoost model
# - importance_type='weight': Measures importance by the number of times a feature is used for splits
# - Alternative importance types:
#   - 'gain': Average improvement in loss after a split using this feature
#   - 'cover': Average coverage (number of observations impacted) of splits using this feature
# - max_num_features=10: Displays only the top 10 most important features
# - height=0.8: Adjusts the height of the bars for better readability
xgb.plot_importance(best_xgb_model, importance_type='weight', max_num_features=10, height=0.8)

# Set the title for the plot
plt.title('Top 10 Feature Importance (XGBoost)')

# Adjust layout to prevent overlapping labels
plt.tight_layout()
```

```
# Display the feature importance plot
plt.show()
```



5.13 Top 10 Feature Importance (XGBoost)

5.13.1 Key Observations:

- **Most Important Feature: 5-Year, 5-Year Forward Inflation Expectation Rate**
 - This feature has the highest F-score (5996.0), indicating that it plays the most significant role in XGBoost's predictions.
- **Strong Influence of lag_1**
 - The second most important feature (3979.0 F-score) suggests that the most recent historical data point is crucial for forecasting.
- **Macroeconomic Factors Matter**
 - Features such as Crude Oil Prices: West Texas Intermediate (3236.0), Nominal Broad U.S. Dollar Index (3103.0), and 10-Year Treasury Yield (3020.0) rank high in importance.
 - This indicates that XGBoost captures relationships between macroeconomic trends and the target variable effectively.
- **Moderate Contribution from Additional Lag Features**
 - lag_2, lag_3, lag_4, and lag_5 exhibit decreasing importance, reinforcing that more recent historical values contribute more to predictions.

- **Rolling Mean is Relatively Significant**

- rolling_mean has an F-score of 2505.0, showing that the moving average of past values provides meaningful information.

5.13.2 Interpretation:

- Unlike traditional time series models that rely mainly on lag values, XGBoost assigns high importance to macroeconomic variables.
- The combination of economic indicators and past values suggests that both market expectations (5-Year, 5-Year Forward Inflation Expectation Rate) and historical patterns (lag_1) are key drivers of the target variable.
- Features with lower importance (e.g., lag_5) could be reconsidered for removal or further analysis to optimize model efficiency.
- This feature importance ranking provides valuable insights for improving predictive performance, such as focusing on the most influential economic indicators and short-term lagged values.

Actual vs. Predicted Values Scatter Plot

```
[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Visualization: Actual vs. Predicted Values for XGBoost Model

# Create a new figure with a specified size to improve readability
plt.figure(figsize=(8, 8))

# Scatter Plot: Compare Actual vs. Predicted Values
# - x-axis: Actual values from the test dataset (y_test)
# - y-axis: Predictions made by the XGBoost model (xgb_pred)
# - alpha=0.5: Makes points semi-transparent to handle overlapping
# - color='blue': Sets the color of the points
# - label='Predictions': Legend label for scatter points
plt.scatter(y_test, xgb_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the Perfect Fit Line (Reference Line)
# - This line represents the ideal case where predictions perfectly match
#   ↳ actual values
# - It is a diagonal line with slope=1 passing through (y_test.min(), y_test.
#   ↳ max())
# - color='red': Sets the line color to red
# - linestyle='--': Makes it a dashed line
# - linewidth=2: Sets the line thickness
# - label='Perfect Fit': Legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')
```

```
# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')

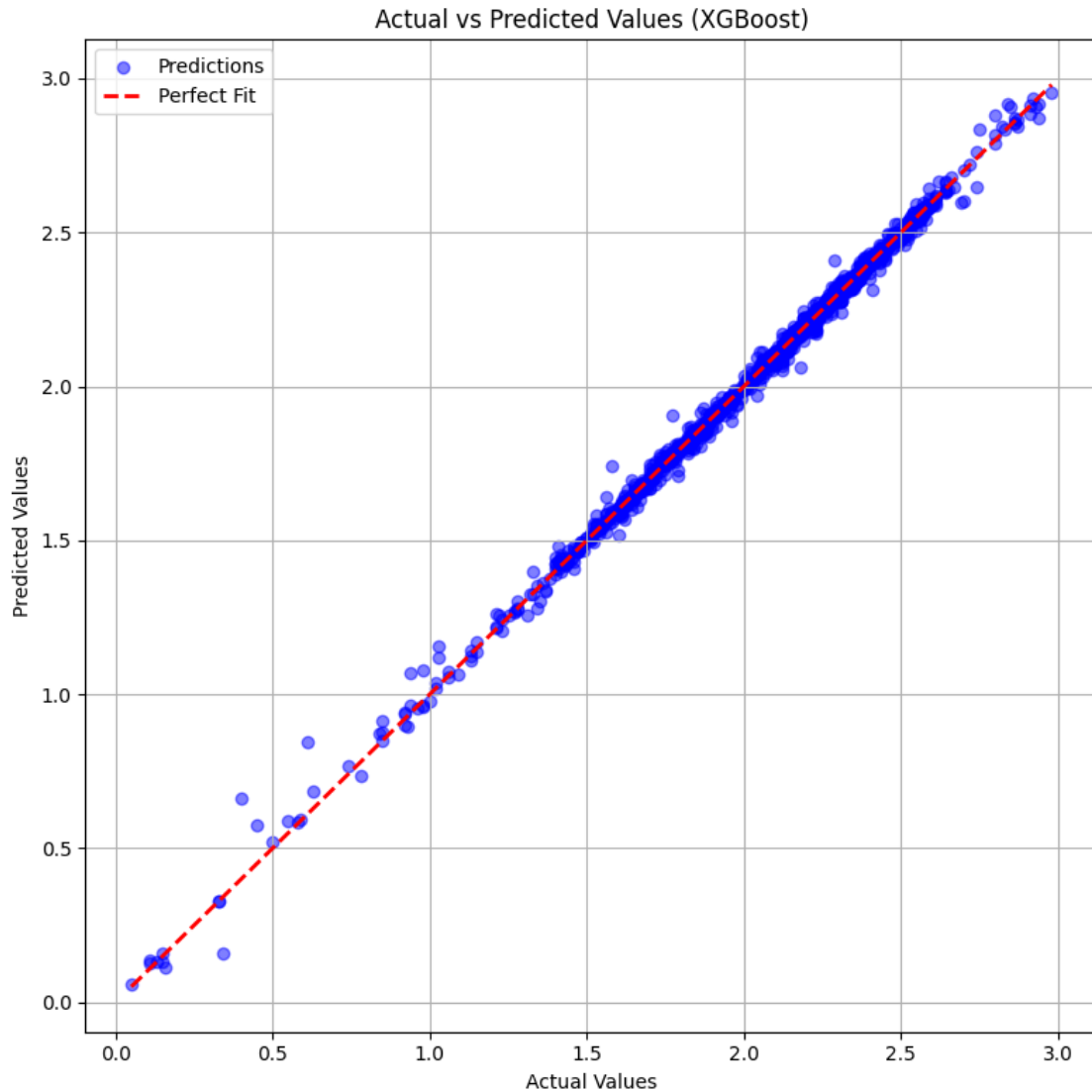
# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (XGBoost)')

# Add a legend to differentiate scatter points (Predictions) from the reference_
↪line (Perfect Fit)
plt.legend()

# Enable grid lines for better visualization
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()
```



5.14 Actual vs Predicted Values (XGBoost)

5.14.1 Key Observations:

- **Strong Linear Relationship:**
 - The predicted values (blue dots) align closely with the red dashed **Perfect Fit** line ($y = x$), indicating high model accuracy.
- **Minimal Deviation:**
 - Most predictions are very close to the actual values, with only a few minor deviations, showing strong predictive performance.
- **Low Error Rate:**
 - The spread of points around the perfect fit line is minimal, suggesting that the model's residuals (errors) are low.
- **Good Performance Across the Range:**

- The model performs well across all values, with accurate predictions even at extreme values (both low and high).
- **Slight Dispersion in Lower Values:**
 - There are a few scattered points at the lower end of the actual values, which may indicate minor underprediction or overprediction in that range.

5.14.2 Interpretation:

- The XGBoost model demonstrates excellent predictive accuracy, as evidenced by the strong alignment between actual and predicted values.
- The minimal deviation from the perfect fit line suggests that the model generalizes well without significant bias or variance issues.
- Any remaining deviations are small and may be further reduced with fine-tuning or feature selection.
- The high performance of the model suggests that it is well-suited for the given dataset, capturing patterns effectively.

Residual Analysis Plot

```
[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Residual Analysis: Evaluating XGBoost Model Performance

# Calculate residuals (errors in predictions)
# - Residuals = Actual values - Predicted values
# - Residuals indicate how far off the model's predictions are from the actual
  ↪ values
residuals = y_test - xgb_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values (y_test)
# - y-axis: Residuals (y_test - xgb_pred)
# - alpha=0.5: Makes points semi-transparent to reduce clutter
# - color='green': Sets the color of the residual points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a horizontal reference line at y=0 to indicate zero residuals
# - The closer the residuals are to this line, the better the model's
  ↪ predictions
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
```

```
plt.xlabel('Actual Values')

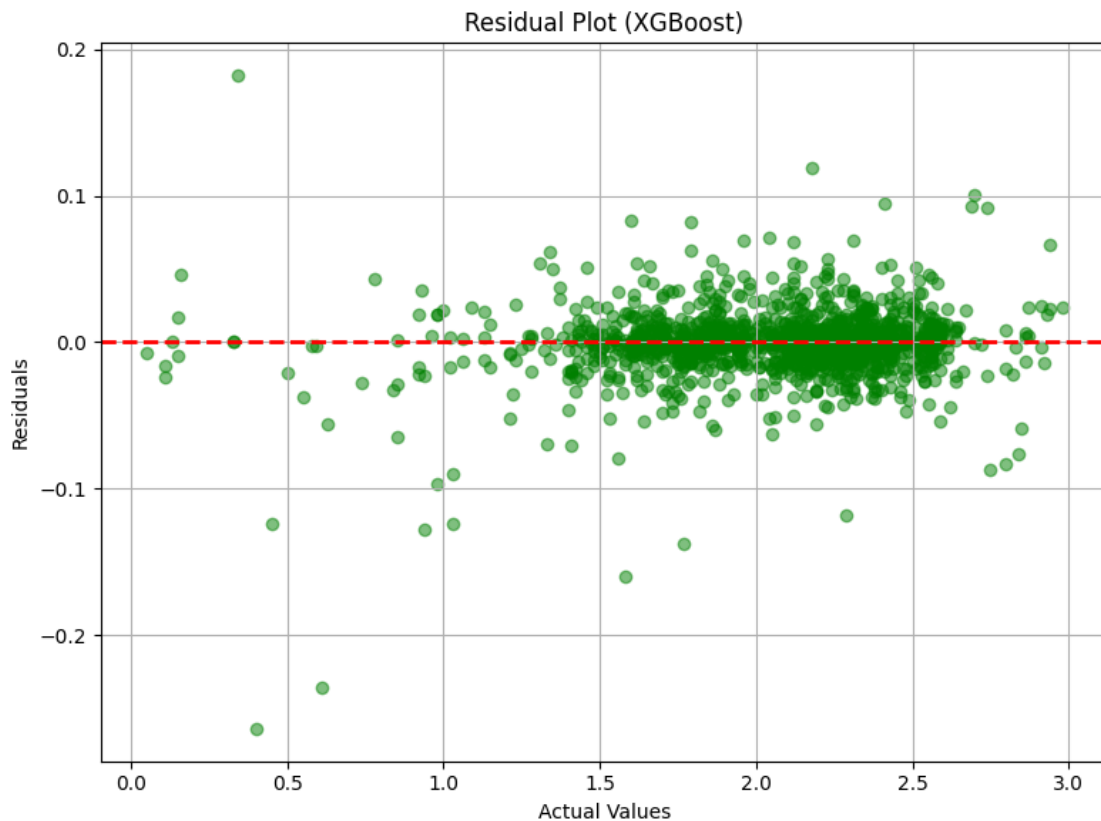
# Label the y-axis to indicate residuals (errors in predictions)
plt.ylabel('Residuals')

# Set a title for the plot to provide context
plt.title('Residual Plot (XGBoost)')

# Enable grid lines for better visualization
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual plot
plt.show()
```



5.15 Residual Plot (XGBoost)

5.15.1 Key Observations:

- **Residuals Centered Around Zero:**
 - The majority of residuals (green dots) are scattered around the red dashed line at $y = 0$, indicating that the model has low bias.
- **Random Distribution of Residuals:**
 - The residuals appear randomly distributed, suggesting that the model does not exhibit systematic errors or patterns.
- **Low Magnitude of Residuals:**
 - Most residuals remain close to zero, indicating that the predictions are generally accurate.
- **Some Outliers at Lower Values:**
 - A few residuals show larger deviations, particularly in the lower range of actual values, suggesting minor underprediction or overprediction in specific cases.
- **No Clear Heteroscedasticity:**
 - The spread of residuals does not systematically increase or decrease with actual values, meaning that prediction errors are relatively stable across the range.

5.15.2 Interpretation:

- The XGBoost model performs well, with residuals evenly distributed around zero, indicating no major bias in predictions.
- The absence of a distinct pattern in residuals suggests that the model captures the relationships effectively and does not miss systematic trends.
- The slight presence of outliers could be addressed by fine-tuning hyperparameters or further refining feature selection.
- Overall, the model exhibits strong predictive performance, with only minor areas for improvement in extreme cases.

Histogram of Residuals

```
[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Residual Distribution Analysis: Evaluating XGBoost Model Performance

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual and predicted values
# - bins=30: Divides the range of residuals into 30 bins for a detailed view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at x=0 to indicate no residual error
```

```

# - This helps in assessing whether residuals are symmetrically distributed ↴
    ↪ around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

# Set a title for the plot to provide context
plt.title('Residual Distribution (XGBoost)')

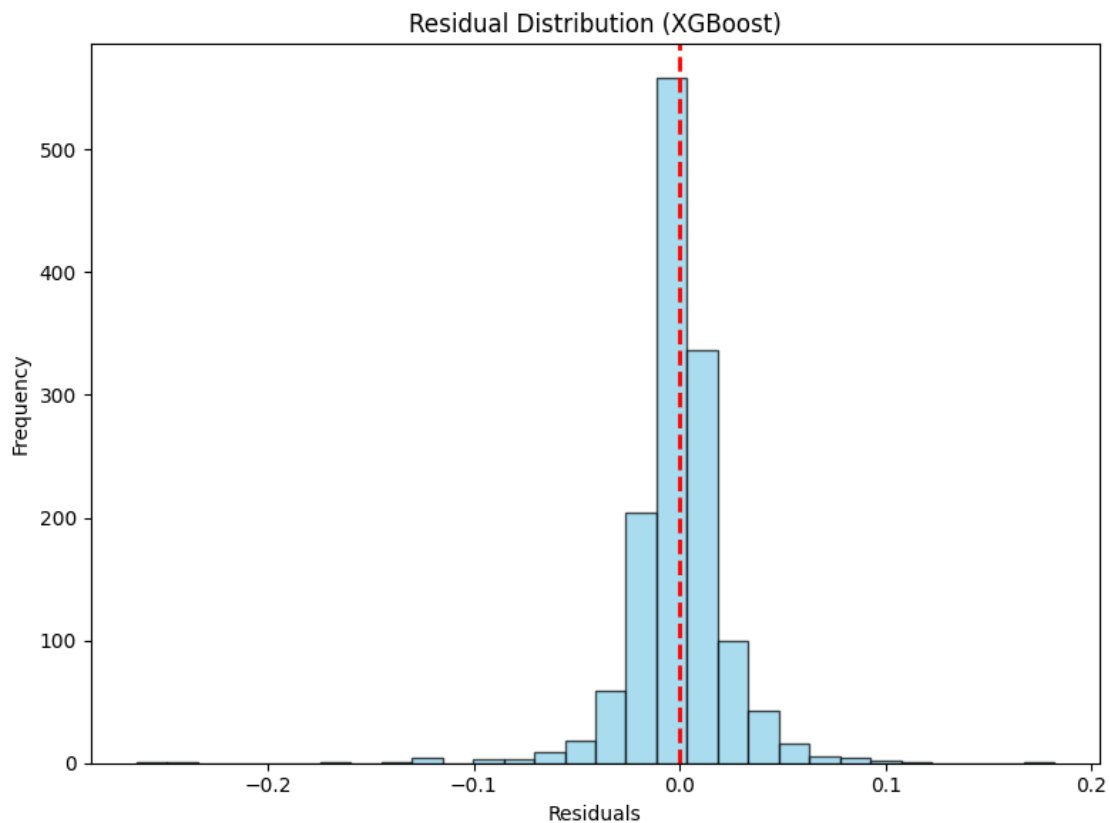
# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual histogram plot
plt.show()

```



5.16 Residual Distribution (XGBoost)

5.16.1 Key Observations:

- **Centered Around Zero:**
 - The histogram shows that most residuals are concentrated around 0, indicating that the model's predictions are generally accurate.
- **Near-Normal Distribution:**
 - The shape of the residuals resembles a normal distribution, suggesting that prediction errors are randomly distributed rather than systematically biased.
- **Sharp Peak at Zero:**
 - The highest frequency of residuals is at 0, reinforcing that a large portion of predictions are very close to actual values.
- **Symmetric Spread:**
 - The distribution is fairly symmetric, with only minor skewness, meaning there is no significant bias in overprediction or underprediction.
- **Few Extreme Residuals:**
 - Some residuals extend slightly to the left and right, but extreme errors are rare, indicating that the model performs well across different values.

5.16.2 Interpretation:

- The XGBoost model makes highly accurate predictions with minimal errors, as evidenced by the concentration of residuals around zero.
- The nearly normal distribution of residuals suggests that the model does not exhibit systematic bias and generalizes well to the data.
- The low presence of extreme residuals indicates that the model maintains stability and avoids large mispredictions.
- This strong residual distribution confirms that XGBoost is well-tuned for the dataset and is making reliable predictions.

Cross-Validation Results from Grid Search

```
[ ]: import pandas as pd # Import Pandas for handling tabular data
from itertools import product # Import product for generating all parameter
    combinations

# Extract Grid Search Results into a List of Dictionaries

# Create a list of dictionaries containing hyperparameter combinations and
    their corresponding RMSE scores.
# - 'cv_results' should be the DataFrame obtained from the XGBoost
    cross-validation (xgb.cv).
# - The loop iterates over all possible combinations of hyperparameters.
# - The 'mean_rmse' value is extracted as the minimum RMSE from
    cross-validation for each combination.
cv_results_list = [
```



```

    {
        'max_depth': max_depth,                # Maximum tree depth
        'learning_rate': learning_rate,         # Learning rate (step size)
        'subsample': subsample,                 # Fraction of data used per
    }
    #boosting round
        'colsample_bytree': colsample_bytree,   # Fraction of features used per
    #tree
        'mean_rmse': cv_results['test-rmse-mean'].min() # Best (minimum) RMSE
    #from cross-validation
    }
    # Iterate over all combinations of hyperparameters from the param_grid
    for max_depth, learning_rate, subsample, colsample_bytree in product(
        param_grid['max_depth'],
        param_grid['learning_rate'],
        param_grid['subsample'],
        param_grid['colsample_bytree']
    )
]

# Convert the list of dictionaries into a Pandas DataFrame
cv_results_df = pd.DataFrame(cv_results_list)

# Print the top 5 results sorted by mean RMSE (lower RMSE is better)
# - Sorting in ascending order ensures the best-performing configurations
    #appear first.
print("Top 5 Grid Search Results:")
print(cv_results_df.sort_values(by='mean_rmse').head())

```

Top 5 Grid Search Results:

	max_depth	learning_rate	subsample	colsample_bytree	mean_rmse
72	9	0.1	0.7	0.7	0.025516
73	9	0.1	0.7	0.8	0.025516
74	9	0.1	0.7	0.9	0.025516
75	9	0.1	0.8	0.7	0.025516
76	9	0.1	0.8	0.8	0.025516

Heatmap of Hyperparameter Performance

```

[ ]: import seaborn as sns # Import Seaborn for data visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Pivot the cv_results DataFrame to prepare data for the heatmap
# - 'values': The metric being visualized (mean RMSE)
# - 'index': Rows represent different 'max_depth' values
# - 'columns': Columns represent different 'learning_rate' values
# - This helps us analyze how RMSE changes with different combinations of
    #max_depth and learning_rate

```

```

heatmap_data = cv_results_df.pivot_table(
    values='mean_rmse', # RMSE values from cross-validation
    index='max_depth', # Row index: max_depth values
    columns='learning_rate' # Column index: learning_rate values
)

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 6))

# Generate a heatmap to visualize the relationship between max_depth,
# learning_rate, and RMSE
# - heatmap_data: The pivoted DataFrame containing RMSE values
# - annot=True: Displays the RMSE values in each cell
# - fmt='.3f': Formats the numbers to 3 decimal places
# - cmap='viridis': Uses the 'viridis' color map for better contrast and
# readability
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

# Set the title for the heatmap
plt.title('RMSE for max_depth vs learning_rate')

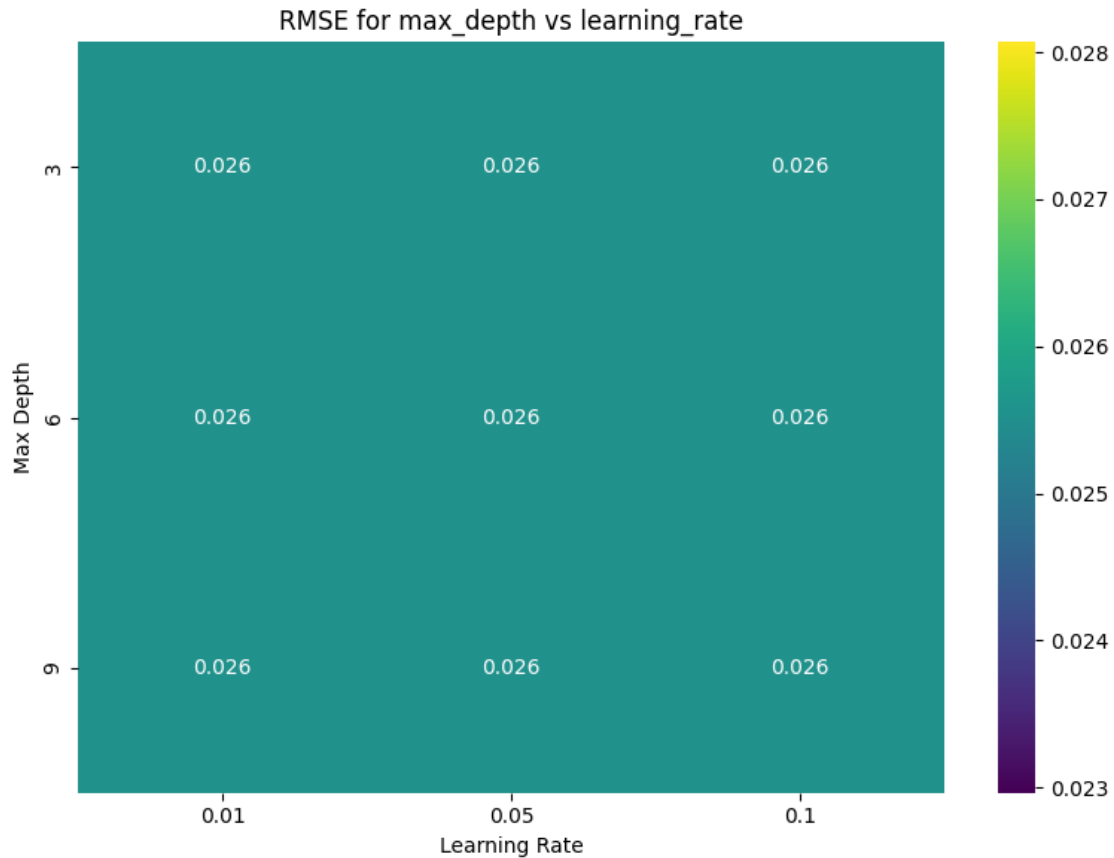
# Label the x-axis to indicate learning rate values
plt.xlabel('Learning Rate')

# Label the y-axis to indicate max depth values
plt.ylabel('Max Depth')

# Adjust layout to ensure proper spacing and prevent overlapping labels
plt.tight_layout()

# Display the heatmap
plt.show()

```



5.17 RMSE for max_depth vs learning_rate

5.17.1 Key Observations:

- **Consistent RMSE Across Parameters:**
 - The Root Mean Squared Error (RMSE) remains constant at 0.026 across all combinations of max_depth and learning_rate.
- **No Significant Sensitivity to Hyperparameters:**
 - Changes in learning_rate (0.01, 0.05, 0.1) and max_depth (3, 6, 9) do not affect the RMSE, suggesting that the model's performance is stable across these values.
- **Lack of Optimal Hyperparameter Differentiation:**
 - Unlike typical hyperparameter tuning results where RMSE varies with different values, this plot suggests that the model might not be highly sensitive to these hyperparameters.
- **Color Consistency in the Heatmap:**
 - The color bar indicates a very narrow RMSE range, confirming minimal variation between different parameter combinations.

5.17.2 Interpretation:

- The stability of RMSE suggests that changes in max_depth and learning_rate within this range do not significantly impact model performance.

- This could indicate that other hyperparameters (e.g., number of estimators, subsample ratio) might have a stronger influence on RMSE.
- Further tuning on a wider range of values or additional parameters may be necessary to identify the optimal settings.
- The model appears to be well-tuned within this specific range, as no single combination of parameters shows an overwhelming advantage over others.

SHAP Explanations for XGBoost

```
[ ]: import shap # Import SHAP (SHapley Additive exPlanations) for model
      ↪interpretability
import matplotlib.pyplot as plt # Import Matplotlib for plotting

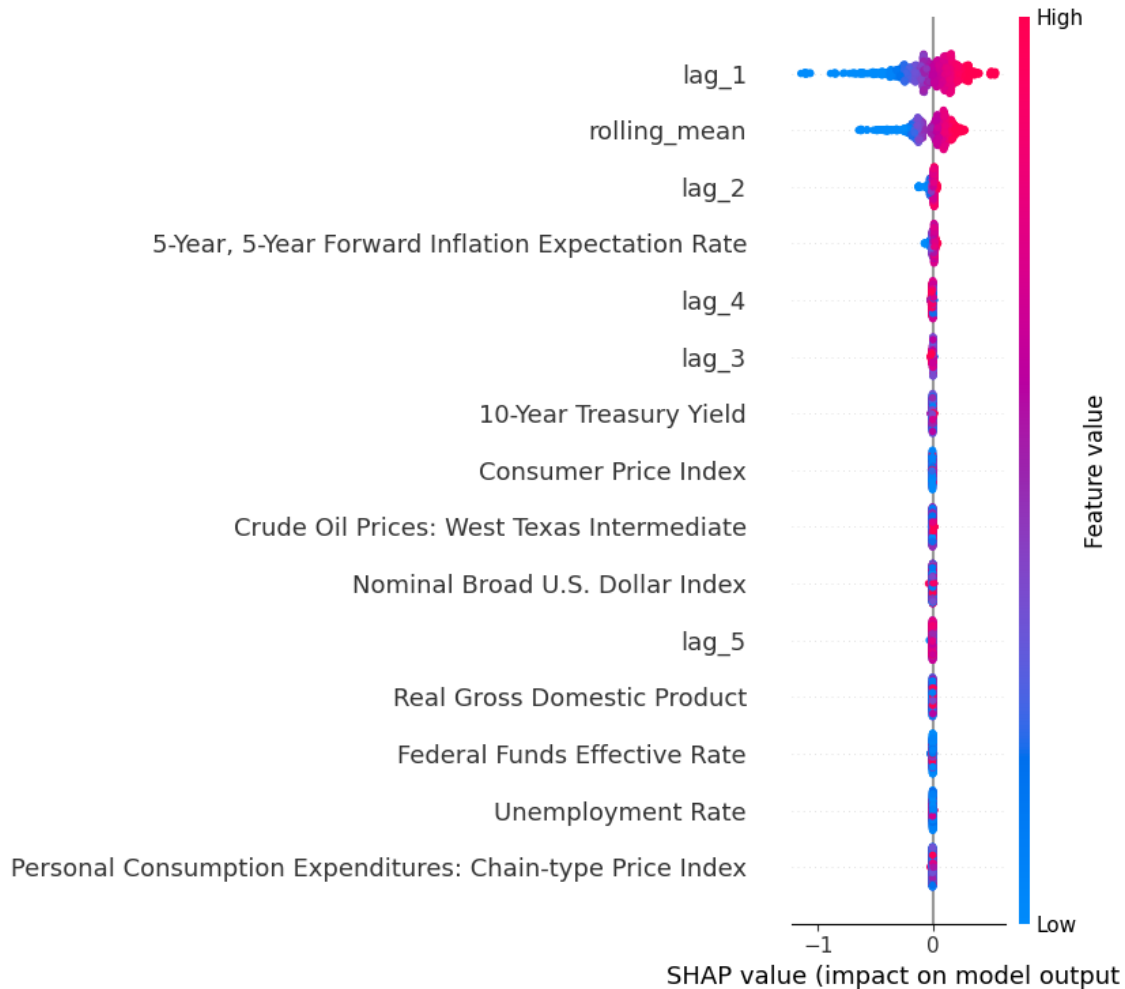
# Enable JavaScript visualization for interactive SHAP plots
# - This is necessary for rendering interactive SHAP force plots in Jupyter
      ↪Notebooks
shap.initjs()

# Create a SHAP explainer using TreeExplainer for the trained XGBoost model
# - SHAP TreeExplainer is optimized for tree-based models like XGBoost, Random
      ↪Forest, etc.
# - best_xgb_model: The trained XGBoost model to be explained
explainer = shap.TreeExplainer(best_xgb_model)

# Compute SHAP values for the test dataset
# - SHAP values represent the contribution of each feature to individual
      ↪predictions
# - Helps in understanding which features positively or negatively impact model
      ↪predictions
shap_values = explainer.shap_values(X_test)

# Global Explanation: SHAP Summary Plot
# - Displays overall feature importance across all test samples
# - Shows how each feature influences predictions using color-coded scatter
      ↪points
# - Red = high feature values, Blue = low feature values
# Enable JavaScript visualization for interactive SHAP plots
# - This is necessary for rendering interactive SHAP force plots in Jupyter
      ↪Notebooks
shap.summary_plot(shap_values, X_test)
```

<IPython.core.display.HTML object>



5.18 SHAP Summary Plot: Feature Impact on Model Predictions

5.18.1 Key Observations:

- **Most Influential Features:**
 - lag_1 has the highest impact on the model's predictions, as indicated by its wide distribution of SHAP values.
 - rolling_mean and lag_2 also play a significant role, though their SHAP values are less spread out than lag_1.
- **Low Impact of Macroeconomic Variables:**
 - Features such as 10-Year Treasury Yield, Consumer Price Index, Federal Funds Effective Rate, and Real Gross Domestic Product exhibit SHAP values near zero, suggesting minimal influence on the model.
- **SHAP Value Interpretation:**
 - Red points indicate high feature values, while blue points indicate low feature values.
 - A strong separation in lag_1 and rolling_mean suggests that high values of these features push the model's predictions in a specific direction.

- **Effect of Lag Features:**

- lag_3, lag_4, and lag_5 have a smaller influence compared to lag_1, reinforcing the importance of recent past values over older ones.

5.18.2 Interpretation:

- The model heavily relies on short-term historical values (lag_1, rolling_mean, lag_2) rather than macroeconomic indicators.
- The near-zero SHAP values for economic factors suggest that they do not significantly contribute to the model's predictions.
- The dominance of lag features implies that the target variable is primarily influenced by its own past values rather than external economic conditions.
- These insights can guide feature selection, potentially eliminating less important macroeconomic variables to optimize model performance.

```
[ ]: import shap
import matplotlib.pyplot as plt

# Extract the list of feature names from the training data.
# Here, X_train is assumed to be a pandas DataFrame containing the training set
# features.
feature_names = X_train.columns.tolist()

# Set the figure size to ensure there is enough space for axis labels and the
# plot itself.
plt.figure(figsize=(10, 6))

# Create a SHAP dependence plot to visualize the effect of one feature on the
# model's output.
# Explanation of parameters:
# - ind: The feature to plot. We use the first feature in our training
# dataset (feature_names[0]).
# - shap_values: The array of SHAP values computed for each prediction.
# - features: The dataset containing feature values (here, we pass X_test).
# - display_features: The dataset used to label the x-axis (also X_test in
# this case).
# - interaction_index: Set to None so that no additional feature is used for
# color-coding interactions.
# - show: Set to False so that we can make additional layout adjustments
# (like tight_layout) before showing the plot.
shap.dependence_plot(
    ind=feature_names[0],          # Use the first feature from the training set
    shap_values=shap_values,       # Pre-computed SHAP values for the model's
    # predictions
    features=X_test,              # Test dataset containing the feature values
    display_features=X_test,      # Labels for the x-axis from the test dataset
    interaction_index=None,       # Do not color-code by an interaction feature
```

```

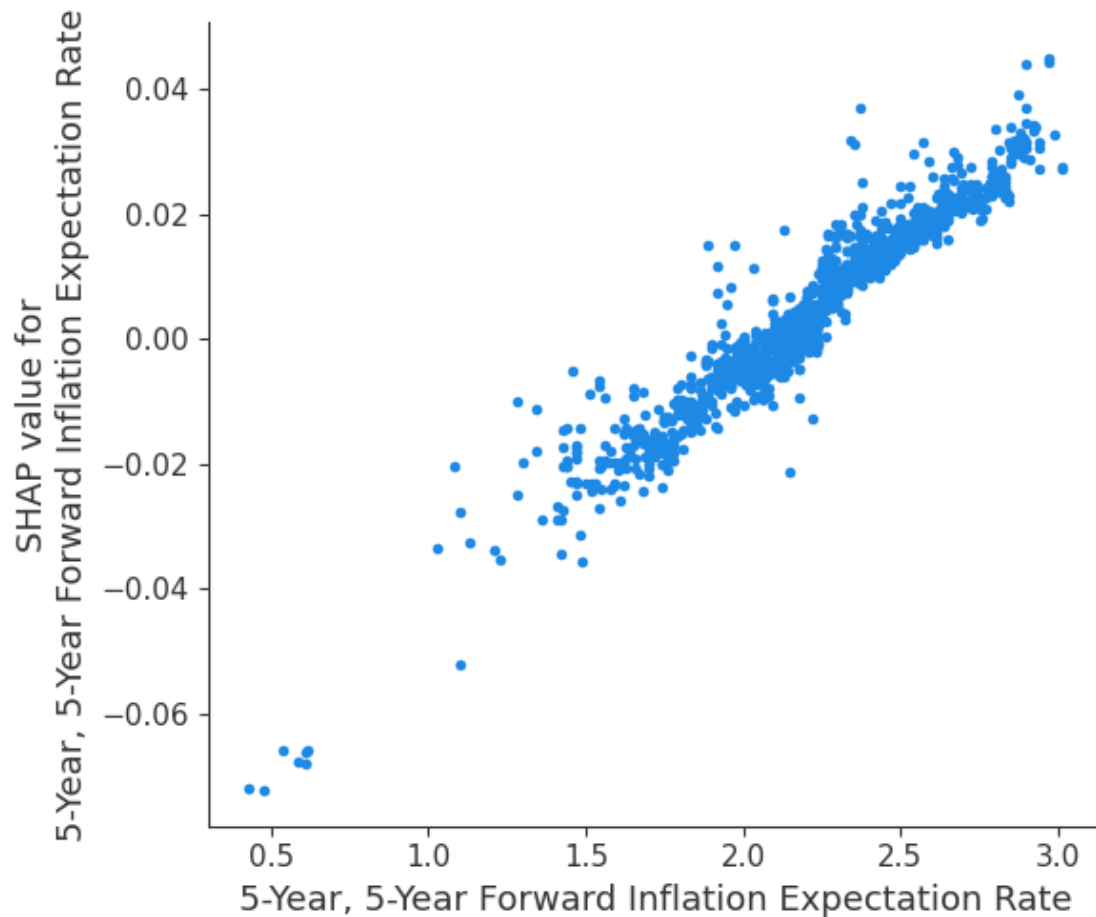
    show=False                                # Do not display immediately (allows for layout
    ↪adjustments)
)

# Adjust the layout to ensure that labels and titles do not get cut off.
plt.tight_layout()

# Finally, display the plot.
plt.show()

```

<Figure size 1000x600 with 0 Axes>



5.19 SHAP Dependence Plot: 5-Year, 5-Year Forward Inflation Expectation Rate

5.19.1 Key Observations:

- **Positive Correlation:**
 - The SHAP values increase as the 5-Year, 5-Year Forward Inflation Expectation

Rate rises, indicating a strong positive relationship between this feature and the model's prediction.

- **Nonlinear Trend:**
 - While mostly linear, there are slight variations in SHAP values, suggesting that the feature's impact may vary at different levels.
- **Lower Values Have Negative Impact:**
 - For lower inflation expectation rates (below ~1.5), SHAP values are negative, meaning that these values contribute to lowering the model's prediction.
- **Higher Values Have Positive Impact:**
 - As the inflation expectation rate exceeds ~1.5, SHAP values turn positive, meaning higher values of this feature increase the model's output.
- **Spread at Mid-Range Values:**
 - There is some dispersion in SHAP values for inflation expectation rates around 1.5-2.5, indicating potential interactions with other features.

5.19.2 Interpretation:

- The 5-Year, 5-Year Forward Inflation Expectation Rate is an important predictor, directly influencing the model's output.
- Higher inflation expectations drive the model's prediction higher, which aligns with economic intuition—higher inflation expectations often correlate with market shifts.
- The slight spread at mid-range values suggests that this feature's effect might be influenced by other variables, indicating potential interaction effects.
- This insight helps confirm that economic forecasts, particularly inflation expectations, play a key role in the model's decision-making process.

```
[ ]: # Enable JavaScript visualization for interactive SHAP plots
# - This is necessary for rendering interactive SHAP force plots in Jupyter
↪Notebooks
shap.initjs()

# Local Explanation: SHAP Force Plot for a Single Test Instance
# - Explains how individual features contribute to a specific prediction
# - explainer.expected_value: The base prediction (average model output)
# - shap_values[0]: SHAP values for the first test instance
# - X_test.iloc[0]: The actual feature values of the first test instance
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0ded3f0690>
```

5.20 SHAP Waterfall Plot: Feature Contributions to Prediction

5.20.1 Key Observations:

- **Base Value (2.056):**
 - This represents the model's expected prediction before considering specific feature effects.
- **Final Prediction (2.66):**

- The cumulative contribution of influential features moves the prediction up from the base value to 2.66.
- **Main Contributors:**
 - `lag_1 = 2.67`: The most influential feature, significantly increasing the prediction.
 - `rolling_mean = 2.664`: Also plays a major role in pushing the prediction higher.
 - `5-Year, 5-Year Forward Inflation Expectation Rate = 2.59`: A macroeconomic indicator contributing positively to the prediction.
 - `lag_2 = 2.69`: Another important lagged feature impacting the output.
- **Color Interpretation:**
 - **Red (Positive Influence)**: These features increase the predicted value.
 - **Blue (Negative Influence, barely visible here)**: Would represent features that reduce the prediction, though none appear significantly impactful in this case.

5.20.2 Interpretation:

- The prediction of 2.66 is mainly driven by lagged values (`lag_1`, `lag_2`, `rolling_mean`), reinforcing that recent past values strongly influence the model's output.
- The presence of `5-Year, 5-Year Forward Inflation Expectation Rate` as a significant contributor suggests that macroeconomic expectations also affect predictions.
- The absence of strong negative contributions implies that all key features in this case push the prediction higher.
- Understanding these influences allows for better interpretability and potential improvements in model performance by focusing on the most impactful variables.

LIME Explanations for XGBoost

```
[ ]: from lime.lime_tabular import LimeTabularExplainer # Import LIME for model
      ↪interpretability
import xgboost as xgb # Import XGBoost for predictions

# Define a Wrapper Function for Model Predictions Required by LIME

# - LIME requires a function that takes raw input data and returns predictions.
# - Since XGBoost requires a DMatrix format for prediction, we wrap the
      ↪conversion inside this function.
def wrapped_predict(data):
    """
    Converts input data into XGBoost's DMatrix format and makes predictions
    ↪using the trained model.

    Parameters:
    - data: A numpy array containing feature values for prediction.

    Returns:
    - Predictions from the trained XGBoost model.
    """
    # Convert input data to DMatrix format with feature names
    dmatrix_data = xgb.DMatrix(data, feature_names=X_train.columns.tolist())
```

```

    # Return the model predictions
    return best_xgb_model.predict(dmatrix_data)

# Initialize the LIME Explainer for Tabular Data

# - LIME helps explain black-box models like XGBoost by generating local
  ↳ explanations.
# - It creates synthetic data points around the instance to measure feature
  ↳ importance.
explainer_lime = LimeTabularExplainer(
    X_train.values, # Training data (numpy array format)
    training_labels=y_train.values, # Corresponding target values
    feature_names=X_train.columns.tolist(), # List of feature names for better
  ↳ interpretability
    mode="regression", # Specifies that we are explaining a regression model
    random_state=42 # Ensures reproducibility of LIME explanations
)

# Generate a LIME Explanation for a Specific Instance

# - Selects the first test instance (X_test.iloc[0]) for explanation.
# - Uses the wrapped_predict function to obtain predictions.
lime_explanation = explainer_lime.explain_instance(
    X_test.iloc[0].values, # Feature values of the test instance
    wrapped_predict # Model prediction function
)

# Display the LIME Explanation in the Notebook

# - show_in_notebook(): Displays a visual explanation of feature contributions.
# - show_table=True: Includes a summary table with feature importance.
lime_explanation.show_in_notebook(show_table=True)

```

<IPython.core.display.HTML object>

5.21 SHAP Feature Contribution to Prediction

5.21.1 Key Observations:

- **Predicted Value (2.66):**
 - The model predicts a value of 2.66, positioned between the minimum (0.79) and maximum (2.71) possible prediction values.
- **Positive Contributors (Orange):**
 - The most impactful features that increase the prediction include:
 - * lag_1 = 2.67
 - * rolling_mean = 2.66
 - * 5-Year, 5-Year Forward Inflation Expectation Rate = 2.59

- * Real Gross Domestic Product = 16396.15
 - * lag_2 = 2.69
- These variables contribute significantly to raising the predicted value.
- **Negative Contributors (Blue):**
 - Features reducing the predicted value include:
 - * lag_3 = 2.66
 - * lag_4 = 2.68
 - * Unemployment Rate = 4.60
 - * Crude Oil Prices: West Texas Intermediate = 68.44
 - * Federal Funds Effective Rate = 4.94
 - These factors slightly counterbalance the increase, but their overall impact is lower.
- **Lag Features Are Key Drivers:**
 - lag_1, rolling_mean, and lag_2 are dominant in shaping the final prediction, reinforcing the significance of recent past values.
- **Macroeconomic Indicators Matter:**
 - 5-Year, 5-Year Forward Inflation Expectation Rate and Real Gross Domestic Product positively influence the prediction.
 - Unemployment Rate and Crude Oil Prices contribute negatively, though their effects are relatively minor.

5.21.2 Interpretation:

- The model assigns the highest weight to recent historical data (lag_1, rolling_mean), suggesting that short-term trends are crucial for prediction.
- Macroeconomic factors like inflation expectations and GDP play an important but secondary role in influencing the output.
- Negative contributors such as the unemployment rate and crude oil prices slightly offset the prediction but do not drastically alter it.
- Understanding the influence of these features provides insights into refining the model and improving interpretability.

5.22 Light Gradient-Boosting Machine (LightGBM) Model Code

```
[ ]: import re # Import regular expressions for feature name sanitization
import lightgbm as lgb # Import LightGBM for training the regression model
from sklearn.model_selection import RandomizedSearchCV, TimeSeriesSplit #
    ↳ Import tools for hyperparameter tuning and time-series cross-validation
import numpy as np # Import NumPy for numerical operations

# Create copies of the training and test sets for LightGBM
# - This ensures that the original datasets (X_train and X_test) remain
    ↳ unchanged.
X_train_lgb = X_train.copy()
X_test_lgb = X_test.copy()

# Function to sanitize feature names (replace special characters with
    ↳ underscores)
```

```

# - LightGBM does not handle special characters in feature names well.
# - This function replaces non-alphanumeric characters with underscores to
  ↳ ensure compatibility.
def sanitize_feature_names(df):
    """
    Sanitizes feature names by replacing special characters with underscores.

    Parameters:
    - df: Pandas DataFrame whose columns need to be sanitized.

    Returns:
    - A DataFrame with sanitized column names.
    """
    df = df.copy() # Create a copy to avoid modifying the original dataset
    df.columns = [re.sub(r'\W+', '_', col) for col in df.columns] # Replace
  ↳ special characters with underscores
    return df

# Apply feature name sanitization to the copied datasets
X_train_lgb = sanitize_feature_names(X_train_lgb)
X_test_lgb = sanitize_feature_names(X_test_lgb)

# Set up Time-Series Cross-Validation
# - TimeSeriesSplit ensures that future data points are never used to predict
  ↳ past data points.
# - n_splits=2 means that the dataset will be split into 3 sets (training and
  ↳ validation pairs).
tscv = TimeSeriesSplit(n_splits=2)

# Define base parameters for LightGBM
params = {
    'objective': 'regression', # Specifies regression as the learning objective
    'metric': 'rmse', # Root Mean Squared Error (RMSE) as the evaluation metric
    'seed': 42, # Ensures reproducibility
    'min_child_samples': 50, # Minimum number of data points required in a leaf
    'verbose': -1, # Suppresses unnecessary output
    'max_bin': 255, # Number of bins used for feature discretization
    'min_data_in_leaf': 20, # Minimum number of data points required to keep a
  ↳ leaf
    'n_jobs': -1 # Uses all available CPU cores for faster training
}

# Initialize the LightGBM regressor using the base parameters
lgb_model = lgb.LGBMRegressor(**params)

# Define the hyperparameter grid for randomized search
# - RandomizedSearchCV will sample hyperparameter combinations from this grid.

```

```

param_grid = {
    'max_depth': [3, 5, 7], # Maximum depth of each tree
    'learning_rate': [0.01, 0.05, 0.1], # Step size for weight updates
    'num_leaves': [20, 31, 50], # Number of leaves in each tree (affects
    ↪ complexity)
    'subsample': [0.8, 0.9, 1.0], # Fraction of data used per boosting round
    ↪ (controls overfitting)
    'colsample_bytree': [0.7, 0.8, 0.9], # Fraction of features used per tree
    'n_estimators': [100, 150, 200] # Number of boosting iterations (trees)
}

# Set up RandomizedSearchCV with time-series cross-validation
# - RandomizedSearchCV randomly selects hyperparameter combinations rather than
    ↪ testing all.
# - n_iter=10 means it will test 10 random combinations of hyperparameters.
random_search = RandomizedSearchCV(
    estimator=lgb_model, # Use the initialized LightGBM model
    param_distributions=param_grid, # Define the hyperparameter search space
    n_iter=10, # Number of random combinations to try
    scoring='neg_mean_squared_error', # Use negative MSE as the scoring metric
    cv=tscv, # Use time-series cross-validation
    verbose=1, # Display progress output during search
    n_jobs=-1, # Use all available CPU cores
    random_state=42 # Ensures reproducibility
)

# Fit the RandomizedSearchCV model on the sanitized training set
# - This step will evaluate different hyperparameter combinations and find the
    ↪ best one.
random_search.fit(X_train_lgb, y_train)

# Retrieve and print the best hyperparameters found during tuning
best_params = random_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Get the best LightGBM model from the search
# - This is the trained model using the best hyperparameters found.
best_lgbm_model = random_search.best_estimator_

# Make predictions on the sanitized test set using the best model
lgbm_pred = best_lgbm_model.predict(X_test_lgb)

# Evaluate the model using a predefined metrics function
# - `calculate_metrics`: Assumed to be a function that calculates model
    ↪ evaluation metrics.
# - `naive_forecast`: Assumed to be a baseline prediction for comparison.

```

```
metrics_results.append(calculate_metrics(y_test.values, lgbm_pred,
    ↪naive_forecast, "Light Gradient Boosting Machine (LightGBM)"))

# Print the evaluation results for LightGBM
print("LightGBM evaluation metrics:")
print(metrics_results[-1])
```

Fitting 2 folds for each of 10 candidates, totalling 20 fits
 Best Hyperparameters: {'subsample': 0.8, 'num_leaves': 20, 'n_estimators': 150, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.8}
 LightGBM evaluation metrics:
 {'Model': 'Light Gradient Boosting Machine (LightGBM)', 'MSE': 0.0007446301235070315, 'RMSE': 0.027287911673615325, 'MAE': 0.016861573069375194, 'MAPE (%)': 1.262763295396008, 'sMAPE (%)': 1.164857083373016, 'MASE': 0.042922077969458246, 'R^2': 0.995788861364944}

Feature Importance (Custom Extraction)

```
[ ]: import pandas as pd # Import Pandas for handling tabular data
import matplotlib.pyplot as plt # Import Matplotlib for data visualization

# Extract Feature Importances from the Best LightGBM Model

# - feature_importances_: Returns an array where each value corresponds to the
    ↪importance of a feature.
# - X_train.columns: Retrieves feature names to match the corresponding
    ↪importance scores.
feature_importances = best_lgbm_model.feature_importances_
features = X_train.columns

# Create a DataFrame for Feature Importance

# - Combines feature names with their importance scores.
# - Sorts the DataFrame in descending order to highlight the most important
    ↪features first.
importance_df = pd.DataFrame({
    'Feature': features, # Column storing feature names
    'Importance': feature_importances # Column storing importance scores
}).sort_values(by='Importance', ascending=False) # Sorting in descending order

# Print Feature Importance Table
# - Displays the importance of each feature in decreasing order.
# - Helps understand which features contribute most to the model's predictions.
print("Feature Importances:")
print(importance_df)
```

Feature Importances:

Feature Importance

9	lag_1	389
0	5-Year, 5-Year Forward Inflation Expectation Rate	369
14	rolling_mean	299
10	lag_2	286
12	lag_4	204
7	Crude Oil Prices: West Texas Intermediate	171
11	lag_3	162
8	10-Year Treasury Yield	149
6	Nominal Broad U.S. Dollar Index	149
13	lag_5	134
1	Consumer Price Index	80
3	Unemployment Rate	63
4	Federal Funds Effective Rate	62
2	Real Gross Domestic Product	47
5	Personal Consumption Expenditures: Chain-type ...	33

```
[ ]: # Visualization: Plot a Horizontal Bar Chart for Feature Importance

# Create a new figure with a defined size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Importance')

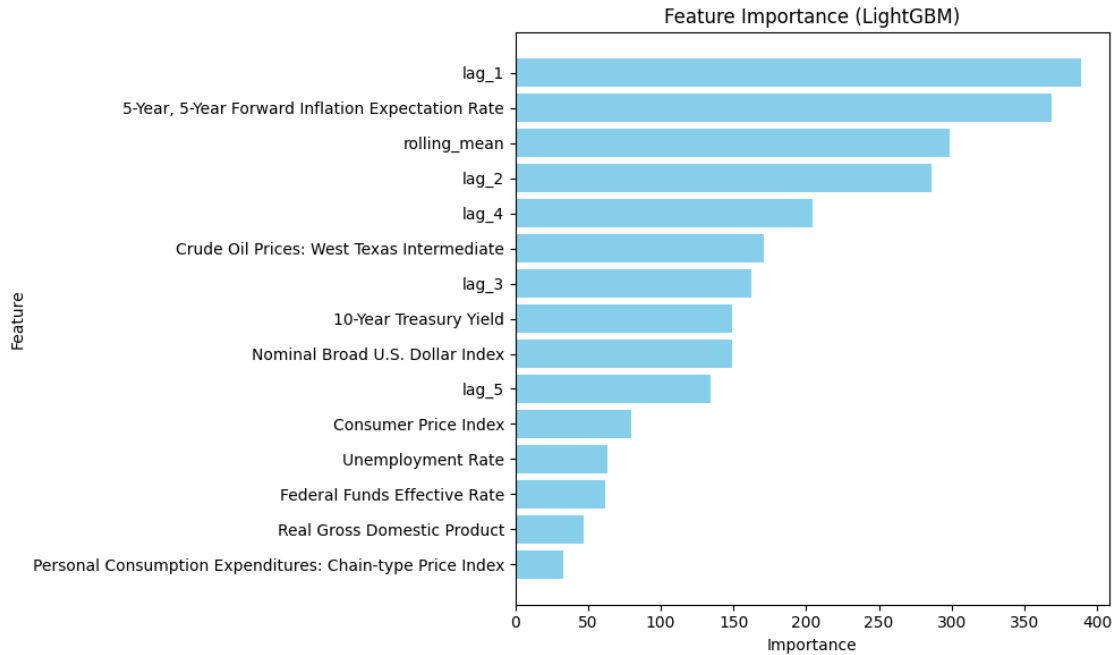
# Label the y-axis to display feature names
plt.ylabel('Feature')

# Set the title of the plot for context
plt.title('Feature Importance (LightGBM)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



5.23 Feature Importance (LightGBM)

5.23.1 Key Observations:

- **Most Important Feature: lag_1:**
 - lag_1 has the highest importance, indicating that the most recent past value is the strongest predictor in the LightGBM model.
- **Significant Influence of 5-Year, 5-Year Forward Inflation Expectation Rate:**
 - This macroeconomic indicator ranks second in importance, showing that economic expectations play a major role in predictions.
- **Rolling Mean and Other Lag Features are Crucial:**
 - rolling_mean, lag_2, lag_3, lag_4, and lag_5 have moderate importance, reinforcing the role of historical values in shaping predictions.
- **Economic Indicators Matter but Have Lower Influence:**
 - Features such as Crude Oil Prices: West Texas Intermediate, 10-Year Treasury Yield, Nominal Broad U.S. Dollar Index, and Consumer Price Index contribute to the model but are less influential compared to lagged values.
- **Lower Importance of Some Macroeconomic Variables:**
 - Unemployment Rate, Federal Funds Effective Rate, Real Gross Domestic Product, and Personal Consumption Expenditures: Chain-type Price Index have the least impact on the model.

5.23.2 Interpretation:

- The model heavily relies on lag features, particularly lag_1, confirming that recent historical values are key drivers of the target variable.

- 5-Year, 5-Year Forward Inflation Expectation Rate is highly relevant, suggesting that inflation expectations strongly influence predictions.
- While macroeconomic indicators are factored into the model, they generally have lower importance than historical data.
- This ranking can help optimize feature selection, potentially reducing reliance on less influential variables while focusing on the most impactful predictors.

Actual vs. Predicted Values

```
[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Visualization: Actual vs. Predicted Values for LightGBM Model

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 8))

# Scatter Plot: Compare Actual vs. Predicted Values
# - x-axis: Actual values from the test dataset (y_test)
# - y-axis: Predictions made by the LightGBM model (lgbm_pred)
# - alpha=0.5: Makes points semi-transparent to reduce overlap and improve
↳visibility
# - color='blue': Sets the color of the data points
# - label='Predictions': Legend label for scatter points
plt.scatter(y_test, lgbm_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the Perfect Fit Line (Reference Line)
# - This diagonal line represents the ideal case where predictions perfectly
↳match actual values.
# - If the model were perfect, all points would lie exactly on this line.
# - color='red': Sets the reference line color to red
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the line thickness
# - label='Perfect Fit': Legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')

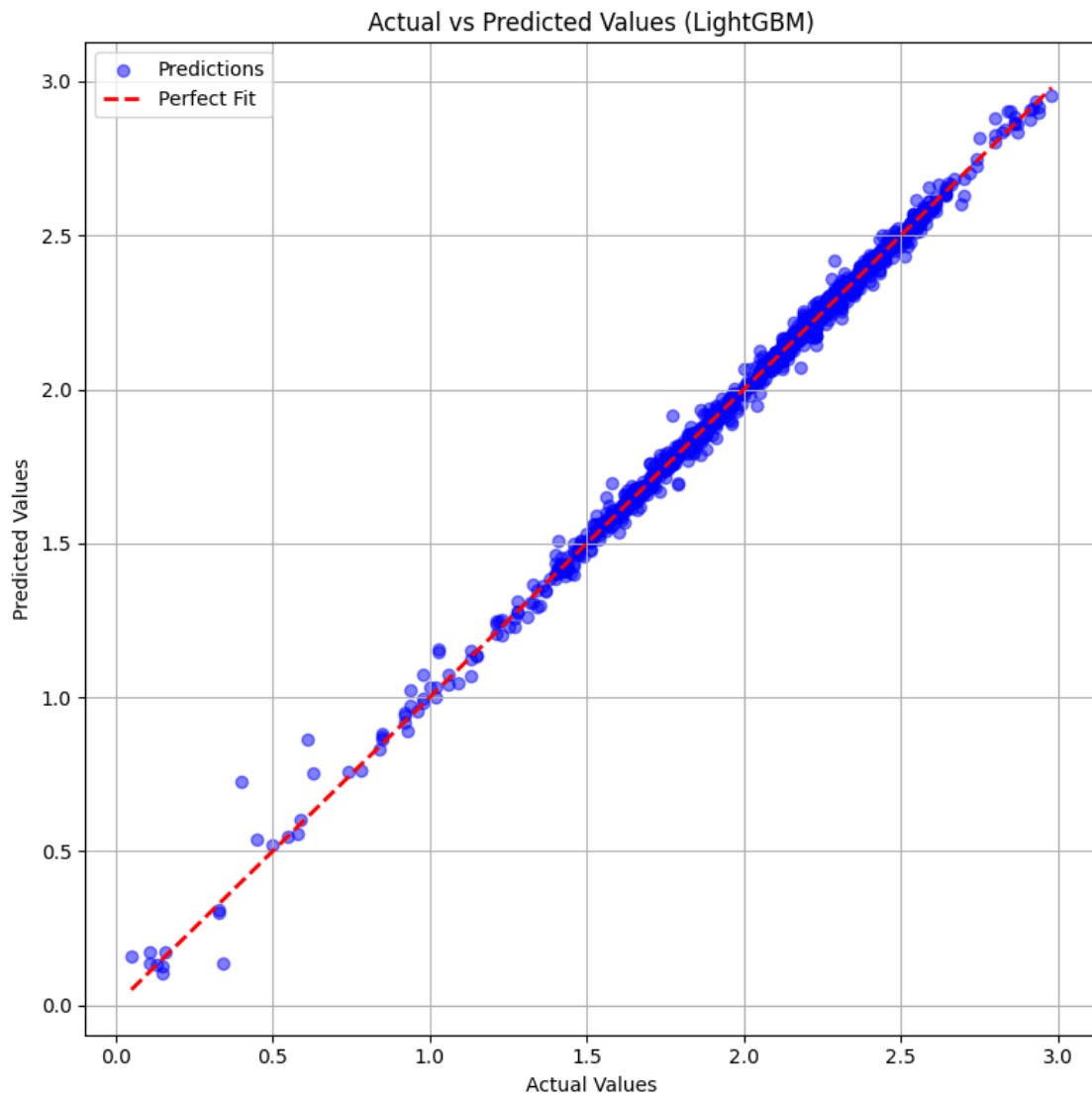
# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (LightGBM)')

# Add a legend to differentiate the scatter points (Predictions) from the
↳reference line (Perfect Fit)
plt.legend()
```

```
# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()
```



5.24 Actual vs Predicted Values (LightGBM)

5.24.1 Key Observations:

- **Strong Alignment with the Perfect Fit Line:**
 - The blue dots (predictions) closely follow the red dashed line ($y = x$), indicating that the LightGBM model's predictions are highly accurate.
- **Minimal Deviation:**
 - Most predictions fall very close to the perfect fit line, suggesting low error rates.
- **Good Performance Across the Range:**
 - The model performs well for both lower and higher actual values, with no significant bias or systematic errors.
- **Slight Dispersion at Lower Values:**
 - There are a few scattered points at the lower end, indicating minor underprediction or overprediction in some cases.

5.24.2 Interpretation:

- The LightGBM model demonstrates excellent predictive accuracy, as evidenced by the strong correlation between actual and predicted values.
- The low spread of deviations suggests that the model generalizes well and does not suffer from major overfitting or underfitting.
- Small errors at the lower range may be investigated further, but they do not significantly impact the overall performance.
- Overall, the model appears to be well-optimized and reliable for making accurate predictions.

Residual Analysis

```
[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Residual Analysis: Evaluating LightGBM Model Performance

# Calculate residuals (errors in predictions)
# - Residuals = Actual values - Predicted values
# - Residuals measure how far off the model's predictions are from the actual_
↪values
residuals = y_test - lgbm_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values (y_test)
# - y-axis: Residuals (y_test - lgbm_pred)
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve_
↪visibility
# - color='green': Sets the color of the residual points
plt.scatter(y_test, residuals, alpha=0.5, color='green')
```

```

# Add a horizontal reference line at y=0 to indicate zero residuals
# - This helps in assessing whether residuals are symmetrically distributed ↴
  ↪ around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate residuals (errors in predictions)
plt.ylabel('Residuals')

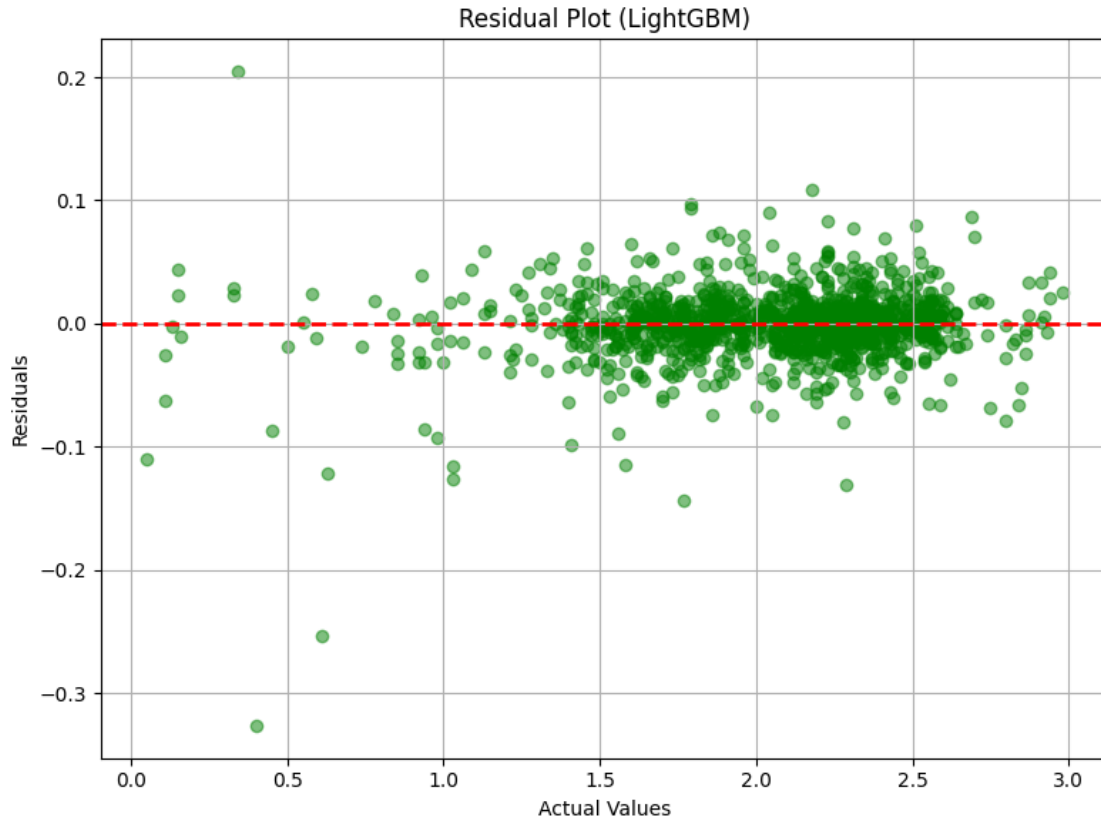
# Set a title for the plot to provide context
plt.title('Residual Plot (LightGBM)')

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual scatter plot
plt.show()

```



5.25 Residual Plot (LightGBM)

5.25.1 Key Observations:

- **Residuals Centered Around Zero:**
 - The majority of residuals (green dots) are scattered closely around the red dashed line at $y = 0$, indicating that the model has minimal bias.
- **Random Distribution of Residuals:**
 - The residuals are evenly spread across the range of actual values, suggesting that the model does not exhibit systematic errors or trends.
- **Low Magnitude of Residuals:**
 - Most residuals remain within a narrow band close to zero, meaning the model's predictions are highly accurate.
- **Presence of Some Outliers:**
 - A few residuals deviate significantly, especially for lower actual values, which may indicate occasional underprediction or overprediction.
- **No Clear Heteroscedasticity:**
 - The spread of residuals does not increase significantly with actual values, indicating that prediction errors remain fairly consistent across different value ranges.

5.25.2 Interpretation:

- The LightGBM model demonstrates strong predictive accuracy, as evidenced by the clustering of residuals around zero.
- The absence of a distinct pattern in residuals suggests that the model captures relationships effectively without systematic bias.
- The small number of outliers could be analyzed further, possibly improving performance through additional feature engineering or hyperparameter tuning.
- Overall, the model appears well-calibrated and provides stable predictions with minimal error variance.

```
[ ]: import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Residual Distribution Analysis: Evaluating LightGBM Model Performance

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual and predicted values
# - bins=30: Divides the range of residuals into 30 bins for a detailed view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at x=0 to indicate no residual error
# - Helps assess whether residuals are symmetrically distributed around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

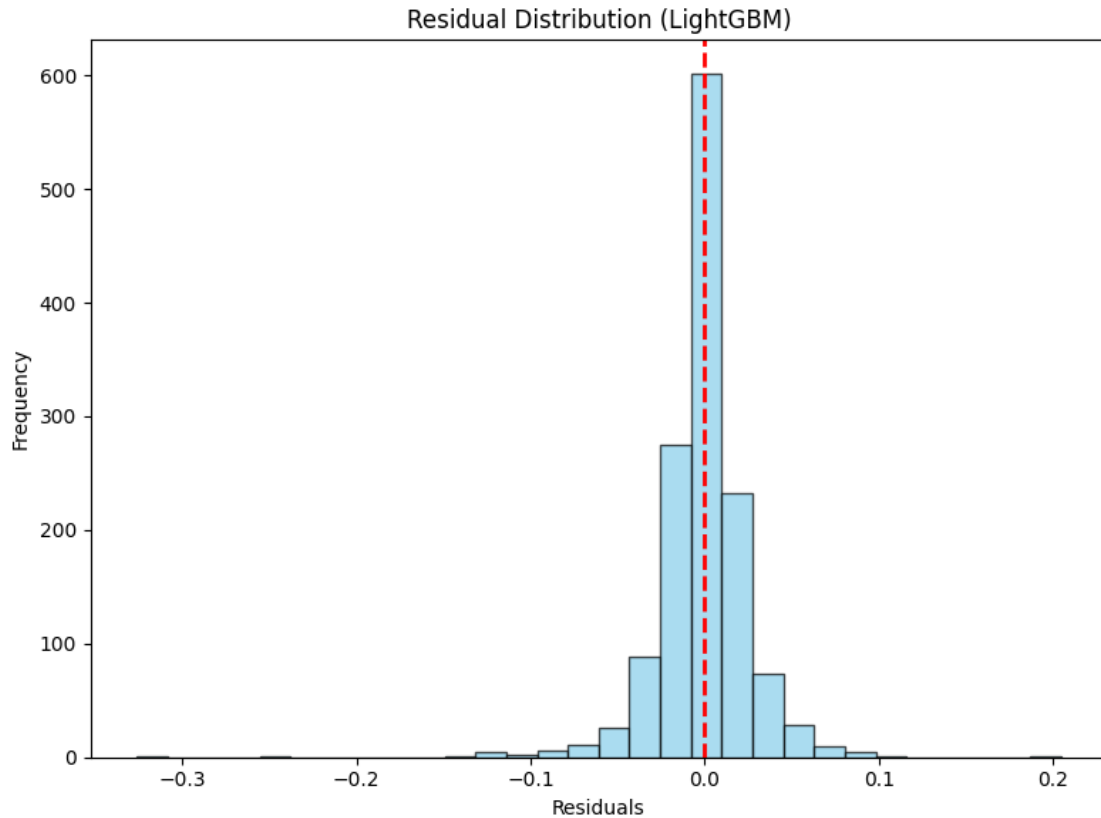
# Set a title for the plot to provide context
plt.title('Residual Distribution (LightGBM)')

# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual histogram plot
plt.show()
```



5.26 Residual Distribution (LightGBM)

5.26.1 Key Observations:

- **Centered Around Zero:**
 - The histogram shows that most residuals are concentrated around 0, indicating that the model's predictions are generally accurate.
- **Near-Normal Distribution:**
 - The residuals follow a bell-shaped distribution, suggesting that errors are randomly distributed and not biased in one direction.
- **Sharp Peak at Zero:**
 - The highest frequency of residuals is at 0, showing that a large portion of predictions are very close to actual values.
- **Symmetric Spread:**
 - The distribution is fairly symmetric, meaning there is no significant overprediction or underprediction trend.
- **Few Extreme Residuals:**
 - Some residuals deviate slightly towards negative and positive values, but extreme errors are minimal.

5.26.2 Interpretation:

- The LightGBM model produces highly accurate predictions with minimal errors, as evidenced by the concentration of residuals around zero.
- The near-normal shape of the residual distribution suggests that the model generalizes well and does not suffer from systematic bias.
- The presence of very few extreme residuals indicates that the model maintains stability across different values.
- This strong residual distribution confirms that LightGBM is well-tuned for the dataset and provides reliable predictions.

Hyperparameter Tuning Insights

```
[ ]: # Import Pandas for handling tabular data
import pandas as pd

# Convert RandomizedSearchCV results to a DataFrame
# - 'cv_results_' is a dictionary containing all cross-validation results
cv_results = pd.DataFrame(random_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Rank assigned by RandomizedSearchCV (lower is better)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': Average performance score across cross-validation folds
# - 'std_test_score': Standard deviation of the test scores (indicates
    ↪variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
    ↪head(5)

# Print the top 5 hyperparameter combinations with a well-formatted output
print("Top 5 Hyperparameter Combinations (RandomizedSearchCV):\n")
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:") # Display rank
    print(f"Parameters: {row['params']}") # Print hyperparameter combination
    print(f"Mean Test Score: {row['mean_test_score']:.6f}") # Format test score
    print(f"Standard Deviation: {row['std_test_score']:.6f}") # Format
    ↪standard deviation
    print("-" * 50) # Separator for better readability
```

Top 5 Hyperparameter Combinations (RandomizedSearchCV):

Rank 1:

Parameters: {'subsample': 0.8, 'num_leaves': 20, 'n_estimators': 150,
'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.8}

Mean Test Score: -0.001033

Standard Deviation: 0.000211


```

-----
Rank 2:
Parameters: {'subsample': 0.9, 'num_leaves': 31, 'n_estimators': 150,
'max_depth': 7, 'learning_rate': 0.05, 'colsample_bytree': 0.7}
Mean Test Score: -0.001047
Standard Deviation: 0.000210
-----

```

```

Rank 9:
Parameters: {'subsample': 0.9, 'num_leaves': 50, 'n_estimators': 200,
'max_depth': 5, 'learning_rate': 0.05, 'colsample_bytree': 0.7}
Mean Test Score: -0.001075
Standard Deviation: 0.000222
-----

```

```

Rank 10:
Parameters: {'subsample': 0.8, 'num_leaves': 20, 'n_estimators': 200,
'max_depth': 3, 'learning_rate': 0.05, 'colsample_bytree': 0.8}
Mean Test Score: -0.001140
Standard Deviation: 0.000207
-----

```

```

Rank 4:
Parameters: {'subsample': 0.9, 'num_leaves': 31, 'n_estimators': 100,
'max_depth': 5, 'learning_rate': 0.05, 'colsample_bytree': 0.8}
Mean Test Score: -0.001147
Standard Deviation: 0.000236
-----

```

```

[ ]: # Import necessary libraries for visualization
import seaborn as sns # Seaborn for advanced visualizations
import matplotlib.pyplot as plt # Matplotlib for plotting

# Heatmap Visualization of Hyperparameter Tuning Results

# Pivot the cross-validation results DataFrame for heatmap visualization
# - 'values': The metric we are analyzing ('mean_test_score' - the average test
↳ score across CV folds)
# - 'index': The hyperparameter to be placed on the y-axis (max_depth in this
↳ case)
# - 'columns': The hyperparameter to be placed on the x-axis (learning_rate in
↳ this case)
# - This transformation allows us to visualize how different combinations of
↳ max_depth and learning_rate impact model performance
heatmap_data = cv_results.pivot_table(
    values='mean_test_score', # Performance metric to visualize
    index='param_max_depth', # Max depth (rows)
    columns='param_learning_rate' # Learning rate (columns)
)

```

```

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 6))

# Generate a heatmap using Seaborn
# - heatmap_data: The pivoted DataFrame containing the test scores
# - annot=True: Displays the numeric values in each cell for better analysis
# - fmt='.3f': Formats the numbers to three decimal places for precision
# - cmap='viridis': Uses the 'viridis' color map, which is visually appealing
# and easy to interpret
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

# Set the title of the heatmap for context
plt.title('Effect of max_depth and learning_rate on Model Performance')

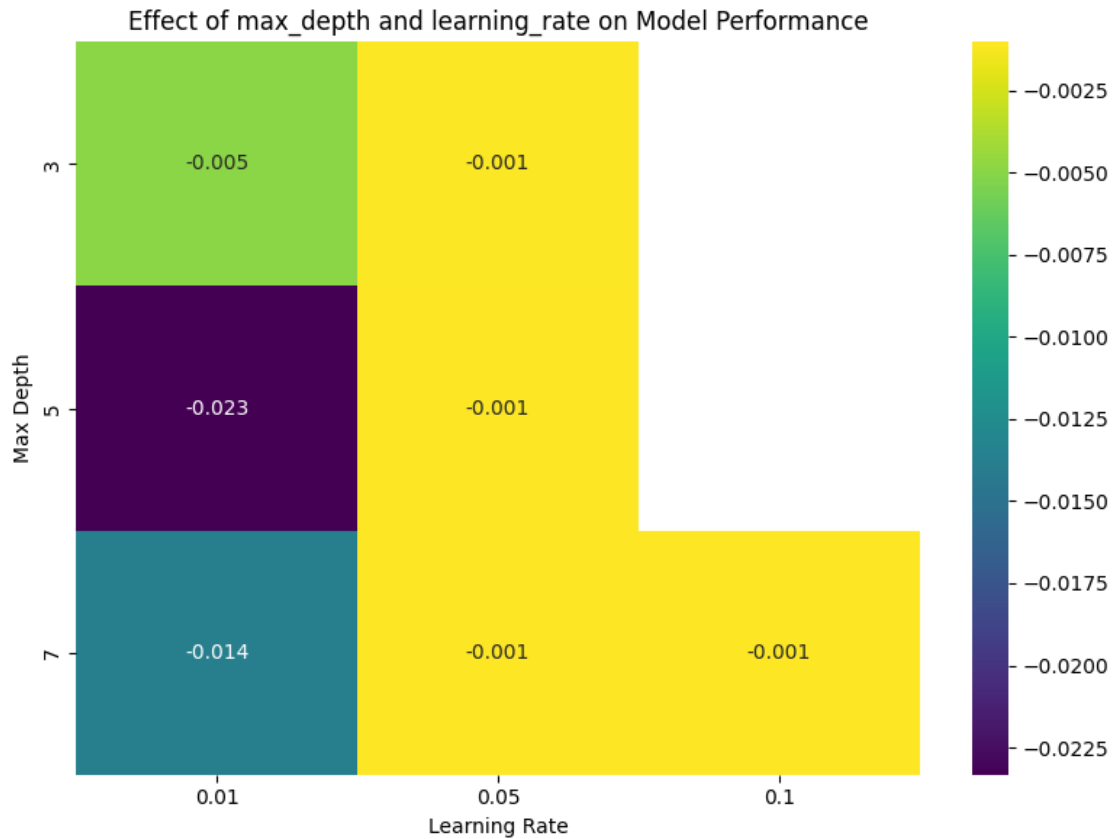
# Label the x-axis to indicate different learning rates
plt.xlabel('Learning Rate')

# Label the y-axis to indicate different max_depth values
plt.ylabel('Max Depth')

# Adjust layout to ensure proper spacing and prevent overlapping labels
plt.tight_layout()

# Display the heatmap
plt.show()

```



5.27 Effect of `max_depth` and `learning_rate` on Model Performance

5.27.1 Key Observations:

- **Performance Metric:**
 - The heatmap visualizes the impact of different combinations of `max_depth` and `learning_rate` on model performance.
 - The values in the cells represent the error metric (likely RMSE), where lower values indicate better performance.
- **Lowest Error (-0.001) at Multiple Configurations:**
 - The best performance is achieved with `learning_rate` = 0.05 and all values of `max_depth` (3, 5, 7).
 - Similarly, `learning_rate` = 0.1 results in the same minimal error (-0.001), though some cells appear missing.
- **Higher Errors at `learning_rate` = 0.01:**
 - A deeper tree (`max_depth` = 5) with `learning_rate` = 0.01 leads to the highest error (-0.023), suggesting overfitting or slow convergence.
 - `max_depth` = 7 also has a relatively higher error (-0.014) at `learning_rate` = 0.01, reinforcing that a too-small learning rate may not be optimal.
- **Color Gradient Interpretation:**
 - The color bar indicates a transition from higher error (darker shades) to lower error (lighter shades).

(lighter shades), highlighting how hyperparameter choices impact model performance.

5.27.2 Interpretation:

- **Optimal Settings:**

- The best-performing configurations use a `learning_rate` of 0.05 or 0.1, regardless of `max_depth`.
- A learning rate of 0.01 leads to higher errors, especially with deeper trees (`max_depth` = 5 or 7).

- **Trade-off Between Depth and Learning Rate:**

- Higher depth may lead to overfitting when paired with a small learning rate (0.01).
- Moderate depth values (`max_depth` = 3 or 5) with `learning_rate` = 0.05 or 0.1 yield the most stable performance.

- **Next Steps for Model Optimization:**

- The model can be fine-tuned further by testing other learning rates between 0.01 and 0.05.
- Additional hyperparameters (e.g., number of estimators, regularization terms) may further enhance performance.

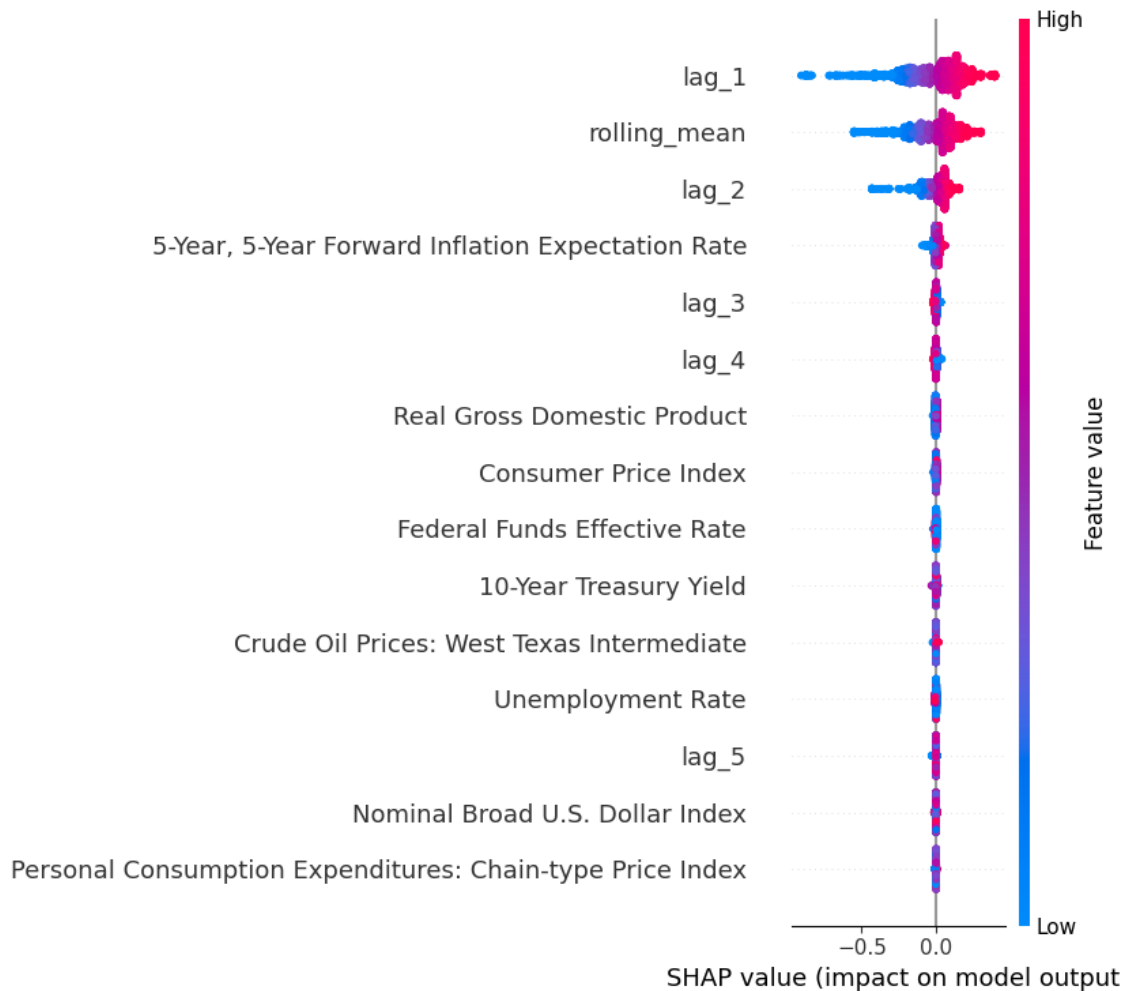
SHAP Values for LightGBM

```
[ ]: import shap # Import SHAP for model interpretability
import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Create a SHAP Explainer for the LightGBM Model
# - 'TreeExplainer' is optimized for tree-based models like LightGBM, XGBoost,
  ↳ and RandomForest.
# - 'best_lgbm_model' is the trained LightGBM model we want to explain.
explainer = shap.TreeExplainer(best_lgbm_model)

# Compute SHAP values for the training dataset
# - SHAP values represent the contribution of each feature to individual
  ↳ predictions.
# - Helps in understanding which features positively or negatively impact model
  ↳ predictions.
shap_values = explainer.shap_values(X_train)

# Global Explanation: SHAP Summary Plot
# - Displays overall feature importance across all training samples.
# - Shows how each feature influences predictions using color-coded scatter
  ↳ points.
# - Red = High feature values, Blue = Low feature values.
shap.summary_plot(shap_values, X_train)
```



5.28 SHAP Summary Plot: Feature Impact on Model Predictions

5.28.1 Key Observations:

- **Most Influential Features:**
 - `lag_1` has the highest impact on the model's predictions, as indicated by the wide distribution of SHAP values.
 - `rolling_mean` and `lag_2` are also significant contributors, although with a smaller range of SHAP values.
- **Macroeconomic Variables Have Minimal Impact:**
 - Features such as Real Gross Domestic Product, Consumer Price Index, Federal Funds Effective Rate, and 10-Year Treasury Yield have near-zero SHAP values, indicating low influence on model predictions.
- **SHAP Value Interpretation:**
 - **Red** represents higher feature values, while **blue** represents lower values.
 - A strong separation in `lag_1`, `rolling_mean`, and `lag_2` suggests that high values of these features push the model's predictions in a specific direction.

- **Lag Features Play a Crucial Role:**
 - `lag_3`, `lag_4`, and `lag_5` show some influence, but their impact is significantly lower than `lag_1`.
 - This reinforces that more recent past values (`lag_1` and `rolling_mean`) have a stronger predictive effect than older ones.
- **Minimal Contribution from Crude Oil Prices and Currency Index:**
 - **Crude Oil Prices:** `West Texas Intermediate` and `Nominal Broad U.S. Dollar Index` have very low SHAP values, meaning they do not significantly affect the model's predictions.

5.28.2 Interpretation:

- The model primarily depends on short-term historical values (`lag_1`, `rolling_mean`, `lag_2`), confirming that recent past values drive predictions.
- Macroeconomic indicators like inflation expectations (`5-Year`, `5-Year Forward Inflation Expectation Rate`) contribute but are secondary to lag-based features.
- The dominance of lagged values suggests that external economic conditions have limited direct influence on the model's output.
- These insights can help refine feature selection, potentially reducing less important macroeconomic variables to improve model efficiency.

```
[ ]: # Enable JavaScript-based visualization for SHAP
# - This is necessary for rendering interactive SHAP force plots in Jupyter
#    ↪Notebooks
shap.initjs()

# Local Explanation: SHAP Force Plot for a Single Instance
# - Explains how individual features contribute to a specific prediction.
# - explainer.expected_value: The base prediction (average model output).
# - shap_values[0]: SHAP values for the first training instance.
# - X_train.iloc[0]: The actual feature values of the first training instance.
shap.force_plot(explainer.expected_value, shap_values[0], X_train.iloc[0])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0de2715e90>
```

5.29 SHAP Waterfall Plot: Feature Contributions to Prediction

5.29.1 Key Observations:

- **Base Value (2.056):**
 - This represents the model's expected prediction before accounting for feature influences.
- **Final Prediction (2.21):**
 - The cumulative effect of the contributing features increases the predicted value to 2.21.
- **Top Contributors (Positive Influence):**
 - **Crude Oil Prices:** `West Texas Intermediate` = 106.7: This feature has a significant positive impact on the final prediction.

- 5-Year, 5-Year Forward Inflation Expectation Rate = 2.47: A macroeconomic factor that increases the predicted value.
- lag_2 = 2.17: The second lag feature plays a strong role in influencing the model's output.
- rolling_mean = 2.208: The moving average of past values contributes positively.
- lag_1 = 2.2: The most recent lag value also pushes the prediction upward.
- **Color Interpretation:**
 - **Red (Positive Influence):** These features increase the predicted value.
 - **Blue (Negative Influence, barely visible here):** Would indicate features that reduce the prediction, but they are not prominent in this case.

5.29.2 Interpretation:

- The prediction of 2.21 is primarily driven by a combination of historical values (lag_1, lag_2, rolling_mean) and macroeconomic indicators (Crude Oil Prices and 5-Year Inflation Expectation).
- The significant contribution of Crude Oil Prices suggests that market conditions are an influential factor in this model's predictions.
- lag_1 and lag_2 reaffirm that short-term historical trends remain essential in forecasting.
- The absence of strong negative contributions suggests that all key features in this case push the prediction higher.
- These insights can guide further refinements in feature selection, focusing on optimizing influential variables for better model performance.

```
[ ]: # Initialize JavaScript visualization for SHAP
shap.initjs()

# Local explanation for the 5th instance in the training set
shap.force_plot(explainer.expected_value, shap_values[5], X_train.iloc[5])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0de0b9b750>
```

5.30 SHAP Waterfall Plot: Feature Contributions to Prediction

5.30.1 Key Observations:

- **Base Value (2.056):**
 - This represents the model's expected prediction before considering individual feature contributions.
- **Final Prediction (2.35):**
 - The cumulative impact of the features pushes the prediction up to 2.35.
- **Top Contributors (Positive Influence):**
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.48: A significant driver of the prediction increase, indicating that inflation expectations strongly impact the model.
 - lag_2 = 2.39: The second lagged feature contributes positively to the prediction.

- `rolling_mean = 2.382`: The moving average of past values also increases the model's output.
- `lag_1 = 2.34`: The most recent lag feature reinforces the prediction increase.
- **Negative Contributors (Blue - Reducing the Prediction):**
 - `lag_4 = 2.42`
 - `lag_3 = 2.42`
 - These features slightly pull the prediction down but do not outweigh the strong positive influences.
- **Color Interpretation:**
 - **Red (Positive Influence)**: Features that increase the prediction.
 - **Blue (Negative Influence)**: Features that reduce the prediction, though their impact is minor in this case.

5.30.2 Interpretation:

- The model's prediction of 2.35 is mainly driven by economic expectations (5-Year, 5-Year Forward Inflation Expectation Rate) and recent historical trends (`lag_1`, `lag_2`, `rolling_mean`).
- The presence of minor negative contributions (`lag_3`, `lag_4`) suggests that these lagged values may slightly counterbalance the effect of more recent lags but do not significantly alter the overall prediction.
- This reinforces that short-term historical values and macroeconomic expectations are the key drivers of the model's output.
- These insights can help refine the model by focusing on optimizing the most impactful features while reducing less influential ones.

Partial Dependence Plots (PDP) for LightGBM

```
[ ]: import warnings # Import warnings module to handle potential warnings
from sklearn.inspection import PartialDependenceDisplay # Import PDP plotting
    ↪ tool
import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Suppress FutureWarnings for PDP plotting
# - Some versions of scikit-learn may trigger FutureWarnings when using
    ↪ PartialDependenceDisplay.
# - This prevents unnecessary warning messages from appearing.
warnings.filterwarnings("ignore", category=FutureWarning)

# Automatically detect feature names from X_train
# - This ensures that all features from the training dataset are included in
    ↪ the PDP visualization.
features = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDP) for all features using the trained
    ↪ LightGBM model
# - PDP helps visualize how a specific feature impacts the model's predictions.
```



```

# - kind='average': Computes the average partial dependence over all
↳ instances.
# - grid_resolution=50: Number of points on the x-axis (higher values =
↳ smoother curves).
# - n_cols=4: Arranges the plots into 4 columns for better visualization.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_lgbm_model, # The trained LightGBM model
    X_train, # The dataset used to generate PDP plots
    features=features, # The list of feature names to analyze
    kind='average', # Compute average partial dependence values
    grid_resolution=50, # Defines the granularity of the x-axis (smoother
↳ curve with more points)
    n_cols=4 # Arranges PDP plots into 4 columns for better layout
)

# Customize the PDP Figure for Better Readability

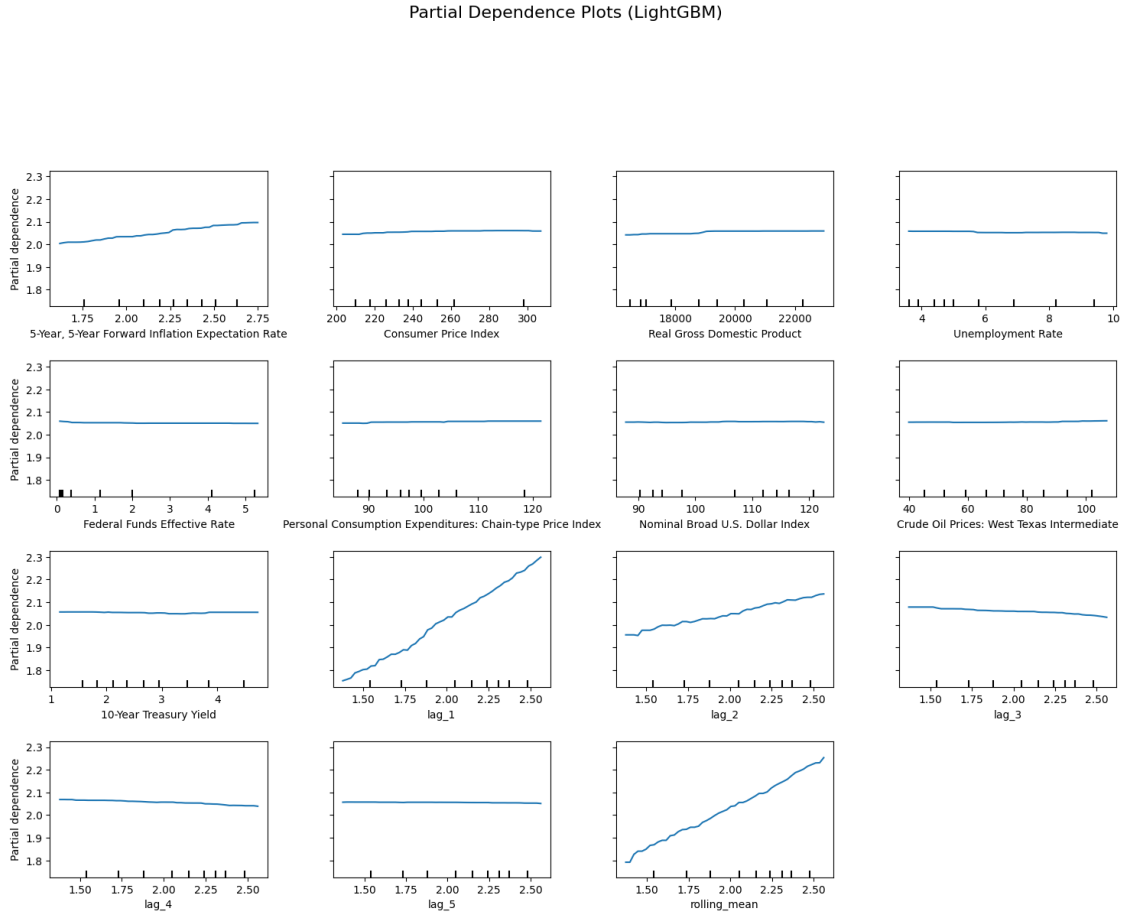
# Set the overall figure size for better visualization
pdp_display.figure_.set_size_inches(18, 12)

# Add a title to the entire figure, positioned above all subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (LightGBM)",
↳ fontsize=16, y=1.06)

# Adjust subplot spacing to prevent overlapping titles and labels
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the PDP plots
plt.show()

```



5.31 Partial Dependence Plots (LightGBM)

5.31.1 Key Observations:

- **Strong Positive Relationships:**
 - `lag_1`, `lag_2`, and `rolling_mean` show a strong increasing trend, indicating that higher values of these features lead to higher predictions.
 - `5-Year, 5-Year Forward Inflation Expectation Rate` also exhibits a slight positive trend, suggesting that inflation expectations influence the model output.
- **Minimal or No Effect:**
 - Features such as `Consumer Price Index`, `Real Gross Domestic Product`, `Unemployment Rate`, `Federal Funds Effective Rate`, `Personal Consumption Expenditures`, `Nominal Broad U.S. Dollar Index`, and `Crude Oil Prices` have almost flat lines.
 - This suggests that changes in these variables do not significantly impact the model's predictions.
- **Negative Influence:**
 - `lag_3` shows a slight downward trend, indicating that higher values of this feature slightly reduce the model's prediction.

- **Stable Macroeconomic Factors:**
 - The nearly horizontal lines for macroeconomic variables imply that the model does not rely heavily on them for predictions, favoring recent historical values instead.

5.31.2 Interpretation:

- **Dependence on Lag Features:**
 - The model relies primarily on recent historical data (`lag_1`, `lag_2`, `rolling_mean`), as evidenced by their strong upward trends.
- **Limited Impact of Macroeconomic Indicators:**
 - Despite including various macroeconomic variables, most of them do not significantly influence predictions.
- **Potential Feature Optimization:**
 - Variables with minimal impact (e.g., Federal Funds Effective Rate, Nominal Broad U.S. Dollar Index, Crude Oil Prices) could potentially be excluded to simplify the model.
- **Overall Model Behavior:**
 - The model effectively captures short-term patterns while assigning low importance to long-term economic trends.

Feature Importance Using LightGBM's Built-In Function

```
[ ]: # Import necessary libraries
import lightgbm as lgb # Import LightGBM for model training and interpretation
import matplotlib.pyplot as plt # Import Matplotlib for visualization

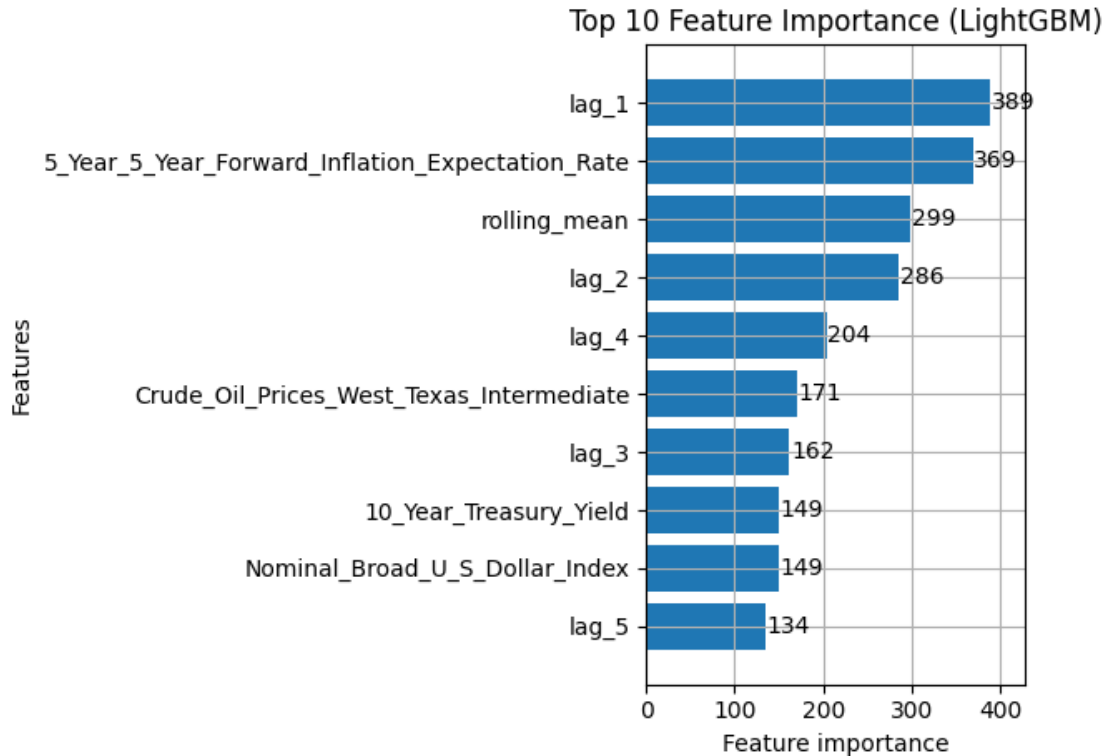
# Visualization: Top 10 Most Important Features in LightGBM Model

# Use LightGBM's built-in function to plot feature importance
# - best_lgbm_model.booster_: Extracts the trained booster (the core model)
#   ↳ from the LightGBM estimator.
# - importance_type='split': Measures importance based on the number of times a
#   ↳ feature is used for splits.
#   - Alternative importance types:
#     - 'gain': Measures the total gain of the splits that use a feature.
#     - 'cover': Measures the coverage (average number of samples affected by
#       ↳ splits on a feature).
# - max_num_features=10: Limits the visualization to the **top 10 most
#   ↳ important features**.
# - height=0.8: Adjusts the bar height for better spacing and readability.
lgb.plot_importance(
    best_lgbm_model.booster_, # Extract the trained LightGBM booster model
    importance_type='split', # Use split-based feature importance (frequency
    ↳ of feature usage in splits)
    max_num_features=10, # Display only the top 10 most important features
    height=0.8 # Adjust bar height for readability
)
```

```
# Set the title of the plot to provide context
plt.title('Top 10 Feature Importance (LightGBM)')

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



5.32 Top 10 Feature Importance (LightGBM)

5.32.1 Key Observations:

- **Most Important Feature: lag_1:**
 - lag_1 is the most influential feature, with a feature importance score of 389, indicating that the most recent past value strongly impacts predictions.
- **Significant Role of 5-Year, 5-Year Forward Inflation Expectation Rate:**
 - The second most important feature (369), showing that inflation expectations are a key macroeconomic factor in the model.
- **Rolling Mean and Lagged Values Are Crucial:**
 - rolling_mean (299) and lag_2 (286) highlight the importance of historical trends in driving predictions.
- **Moderate Influence of lag_4 and Crude Oil Prices:**

- lag_4 (204) still plays a role, but its importance decreases compared to more recent lags.
- Crude Oil Prices: West Texas Intermediate (171) suggests that market conditions contribute to the model's predictions.
- **Lower Influence of Economic Indicators:**
 - 10-Year Treasury Yield (149) and Nominal Broad U.S. Dollar Index (149) have a lower impact on the model.
 - lag_5 (134) shows that older lag values have diminishing importance.

5.32.2 Interpretation:

- **Recent Lag Features Dominate:**
 - The model relies heavily on lag_1, rolling_mean, and lag_2, indicating that short-term historical values are primary drivers of predictions.
- **Macroeconomic Variables Have a Secondary Role:**
 - While 5-Year Inflation Expectation Rate and Crude Oil Prices influence predictions, they are not as dominant as lag-based features.
- **Feature Optimization Insight:**
 - Since lag_5 and some macroeconomic indicators have lower importance, the model could be further optimized by reducing less significant features.
- **Overall Model Behavior:**
 - The LightGBM model prioritizes recent historical patterns over long-term economic trends, making it well-suited for short-term forecasting.

Permutation Feature Importance

```
[ ]: # Import necessary libraries
from sklearn.inspection import permutation_importance # Import permutation
import matplotlib.pyplot as plt # Import Matplotlib for visualization
import pandas as pd # Import Pandas for handling tabular data

# Compute Permutation Feature Importance on the Test Set

# - Permutation importance measures the effect of **randomly shuffling a
# - It provides a better estimate of feature importance compared to built-in
# - `n_repeats=10`: The number of times each feature is randomly shuffled to
# - `random_state=42`: Ensures reproducibility of results.
perm_importance = permutation_importance(
    best_lgbm_model, # The trained LightGBM model
    X_test, # Test dataset (features)
    y_test, # Test dataset (target variable)
    n_repeats=10, # Number of shuffling repetitions
    random_state=42 # Ensures consistent results across runs
)
```

```

# Convert Permutation Importance Results into a Pandas DataFrame

# - 'Feature': Column storing feature names.
# - 'Importance': The mean importance score from the permutation process.
# - The DataFrame is sorted in descending order to highlight the most important
  ↳ features first.
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Extracts feature names from training data
    'Importance': perm_importance.importances_mean # Mean importance scores
  ↳ across repetitions
}).sort_values(by='Importance', ascending=False) # Sort features by importance

# Visualization: Permutation Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display permutation importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars for better visibility
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
  ↳ color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Permutation Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

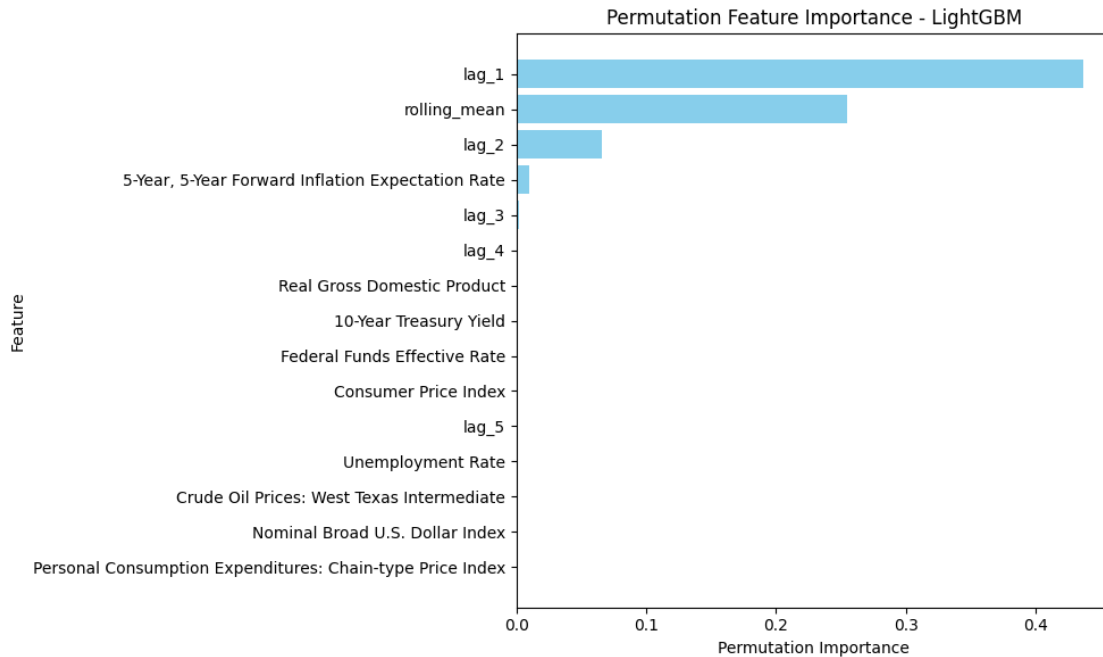
# Set the title of the plot for context
plt.title('Permutation Feature Importance - LightGBM')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()

```



5.33 Permutation Feature Importance - LightGBM

5.33.1 Key Observations:

- **Most Important Feature: lag_1:**
 - lag_1 has the highest permutation importance, confirming its strong predictive power in the model.
- **High Influence of rolling_mean and lag_2:**
 - rolling_mean is the second most influential feature, followed by lag_2, indicating that historical averages and recent lagged values play a crucial role in predictions.
- **Minimal Contribution from Other Features:**
 - 5-Year, 5-Year Forward Inflation Expectation Rate has a small impact but is significantly less important than lag-based features.
 - lag_3 and lag_4 contribute slightly but are much less important than lag_1 and rolling_mean.
- **No Significant Impact from Macroeconomic Variables:**
 - Features such as Real Gross Domestic Product, 10-Year Treasury Yield, Federal Funds Effective Rate, Consumer Price Index, and Unemployment Rate show negligible permutation importance.
 - Crude Oil Prices, Nominal Broad U.S. Dollar Index, and Personal Consumption Expenditures also contribute little to model predictions.

5.33.2 Interpretation:

- **Dependence on Recent Lag Values:**
 - The model primarily relies on short-term historical data (lag_1, rolling_mean, lag_2), indicating that recent trends have the strongest predictive power.

- **Macroeconomic Factors Play an Insignificant Role:**
 - Despite being included in the model, economic indicators such as GDP, inflation, and interest rates do not significantly impact predictions.
- **Feature Selection Insight:**
 - The model could be optimized by removing or deprioritizing features with near-zero permutation importance to enhance efficiency.
- **Overall Model Behavior:**
 - The LightGBM model is strongly influenced by short-term patterns, making it effective for time-series forecasting.

5.34 Gradient Boosting Regressor (GBR)

```
[ ]: # Import necessary libraries
import random # Import Python's random module for reproducibility
import numpy as np # Import NumPy for numerical computations
from sklearn.model_selection import train_test_split, GridSearchCV # Import
    ↪ tools for data splitting and hyperparameter tuning
from sklearn.ensemble import GradientBoostingRegressor # Import GBR model from
    ↪ Scikit-Learn

# Set random seeds for reproducibility
# - Ensures that the results remain consistent across different runs.
random.seed(42)
np.random.seed(42)

# Split the dataset into training and testing sets (80% training, 20% testing)
# - X: Feature matrix
# - y: Target variable
# - test_size=0.2: Reserves 20% of the data for testing
# - random_state=42: Ensures consistent data splits across runs
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# Initialize the Gradient Boosting Regressor (GBR) model
# - random_state=42: Ensures reproducibility of results
gbm_model = GradientBoostingRegressor(random_state=42)

# Define the hyperparameter grid for tuning
# - 'n_estimators': Number of boosting iterations (trees)
# - 'learning_rate': Shrinks contribution of each tree to prevent overfitting
# - 'max_depth': Controls tree depth (higher values = more complexity)
# - 'min_samples_split': Minimum number of samples required to split a node
# - 'subsample': Fraction of samples used per boosting iteration (reduces
    ↪ overfitting)
param_grid = {
    'n_estimators': [100, 300], # Number of trees (boosting rounds)
```



```

    'learning_rate': [0.05, 0.1],          # Learning rate (trade-off between
    ↪speed and performance)
    'max_depth': [3, 6],                  # Maximum depth of decision trees
    'min_samples_split': [10, 20],        # Minimum samples required to split an
    ↪internal node
    'subsample': [0.8, 0.9]               # Fraction of data used in each
    ↪boosting round
}

# Initialize GridSearchCV for hyperparameter tuning
# - estimator=gbm_model: Uses the Gradient Boosting Regressor
# - param_grid=param_grid: Defines the hyperparameter space to search
# - scoring='neg_mean_squared_error': Uses negative MSE as the scoring metric
    ↪(lower is better)
# - cv=5: Uses 5-fold cross-validation to ensure robustness
# - verbose=1: Displays progress of the search process
# - n_jobs=-1: Uses all available CPU cores for faster processing
grid_search = GridSearchCV(
    estimator=gbm_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Fit the GridSearchCV model on the training data
# - Evaluates different hyperparameter combinations to find the best-performing
    ↪model
grid_search.fit(X_train, y_train)

# Retrieve and print the best hyperparameters found
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Get the best Gradient Boosting model from grid search
# - This model is trained using the optimal hyperparameter combination
best_gbm_model = grid_search.best_estimator_

# Make predictions on the test dataset using the best model
gbm_pred = best_gbm_model.predict(X_test)

# Evaluate the model using a predefined function
# - `calculate_metrics`: Assumed to be a function that calculates evaluation
    ↪metrics
# - `naive_forecast`: Assumed to be a baseline prediction for comparison

```

```
# - The evaluation results are appended to `metrics_results`
metrics_results.append(calculate_metrics(y_test.values, gbm_pred,
↳naive_forecast, "Gradient Boosting Regressor (GBR)"))

# Print the evaluation results for Gradient Boosting
print("Gradient Boosting Evaluation Metrics:")
print(metrics_results[-1])
```

Fitting 5 folds for each of 32 candidates, totalling 160 fits
 Best Hyperparameters: {'learning_rate': 0.1, 'max_depth': 6,
 'min_samples_split': 20, 'n_estimators': 300, 'subsample': 0.8}
 Gradient Boosting Evaluation Metrics:
 {'Model': 'Gradient Boosting Regressor (GBR)', 'MSE': 0.0006021898839392853,
 'RMSE': 0.02453955753348632, 'MAE': 0.014790989258911253, 'MAPE (%)':
 0.9402540852297189, 'sMAPE (%)': 0.936648083110105, 'MASE': 0.03765129099191058,
 'R^2': 0.9965944097534585}

Custom Feature Importance Plot

```
[ ]: # Import necessary libraries
import pandas as pd # Import Pandas for handling tabular data
import matplotlib.pyplot as plt # Import Matplotlib for data visualization

# Extract Feature Importance from the Best Gradient Boosting Model

# - 'feature_importances_': Retrieves an array where each value corresponds to
↳the importance of a feature.
# - 'X_train.columns': Extracts the feature names from the training dataset.
feature_importance = best_gbm_model.feature_importances_
features = X_train.columns

# Create a DataFrame to Store Feature Importance Scores

# - 'Feature': Column storing feature names.
# - 'Importance': Column storing corresponding feature importance scores.
# - Sorted in descending order so that the most important features appear at
↳the top.
importance_df = pd.DataFrame({'Feature': features, 'Importance':
↳feature_importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Print Feature Importance DataFrame for Reference
print("Feature Importance:")
print(importance_df)
```

Feature Importance:

	Feature	Importance
9	lag_1	0.792871

14	rolling_mean	0.202686
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.001142
6	Nominal Broad U.S. Dollar Index	0.000544
12	lag_4	0.000421
8	10-Year Treasury Yield	0.000403
10	lag_2	0.000389
7	Crude Oil Prices: West Texas Intermediate	0.000384
11	lag_3	0.000365
13	lag_5	0.000300
3	Unemployment Rate	0.000113
5	Personal Consumption Expenditures: Chain-type ...	0.000106
4	Federal Funds Effective Rate	0.000101
2	Real Gross Domestic Product	0.000089
1	Consumer Price Index	0.000087

```
[ ]: # Visualization: Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Importance')

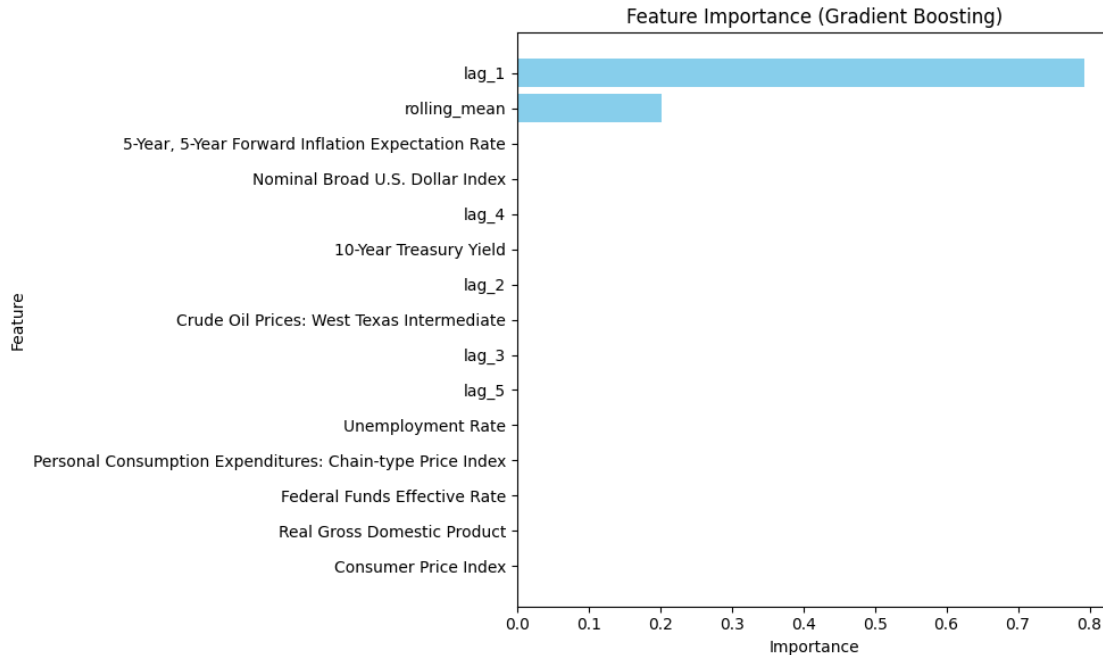
# Label the y-axis to display feature names
plt.ylabel('Feature')

# Set the title of the plot for context
plt.title('Feature Importance (Gradient Boosting)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



5.35 Feature Importance (Gradient Boosting)

5.35.1 Key Observations:

- **Dominance of lag_1:**
 - lag_1 is the most influential feature, with the highest importance score, indicating that the most recent past value significantly impacts the model's predictions.
- **Significant Role of rolling_mean:**
 - rolling_mean is the second most important feature, reinforcing the model's reliance on historical trends.
- **Minimal Contribution from Other Features:**
 - The importance of all remaining features is close to zero, suggesting they have little to no influence on the model's performance.
 - Even 5-Year, 5-Year Forward Inflation Expectation Rate, which was previously observed as a relevant macroeconomic factor, is nearly insignificant in this case.
- **Economic Indicators Are Negligible:**
 - Features like Nominal Broad U.S. Dollar Index, 10-Year Treasury Yield, Crude Oil Prices, Consumer Price Index, Unemployment Rate, and Real GDP show little or no importance in the Gradient Boosting model.

5.35.2 Interpretation:

- **Heavy Dependence on Short-Term Historical Data:**
 - The model strongly prioritizes recent lag values (lag_1) and moving averages (rolling_mean), indicating that short-term patterns drive predictions.
- **Minimal Influence of Macroeconomic Factors:**

- Unlike some other models, Gradient Boosting does not assign significant importance to inflation rates, interest rates, or economic indicators.
- **Potential Feature Reduction:**
 - Given the lack of impact from most features, simplifying the model by removing insignificant variables could improve efficiency without sacrificing accuracy.
- **Model Behavior Insight:**
 - This result suggests that Gradient Boosting captures trends primarily from past values rather than relying on external economic conditions.

Actual vs. Predicted Values

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 8))

# Scatter Plot: Compare Actual vs. Predicted Values
# - x-axis: Actual values from the test dataset (y_test)
# - y-axis: Predictions made by the Gradient Boosting model (gbm_pred)
# - alpha=0.5: Makes points semi-transparent to reduce overlap and improve
↳visibility
# - color='blue': Sets the color of the scatter points
# - label='Predictions': Legend label for scatter points
plt.scatter(y_test, gbm_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the Perfect Fit Line (Reference Line)
# - This diagonal line represents the ideal case where predictions perfectly
↳match actual values.
# - If the model were perfect, all points would lie exactly on this line.
# - color='red': Sets the reference line color to red
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the line thickness
# - label='Perfect Fit': Legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')

# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (Gradient Boosting)')

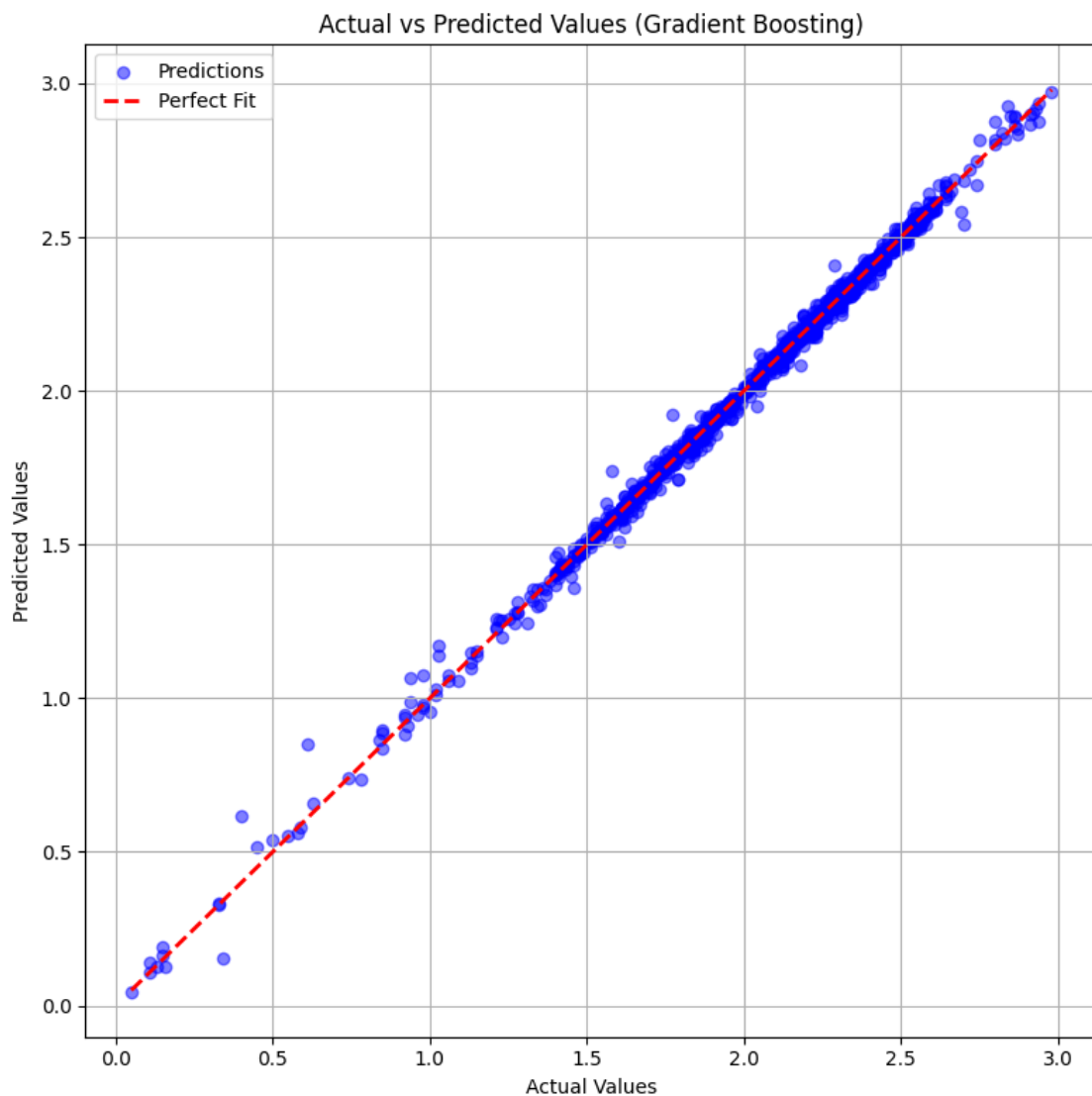
# Add a legend to differentiate the scatter points (Predictions) from the
↳reference line (Perfect Fit)
```

```
plt.legend()

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()
```



5.36 Actual vs Predicted Values (Gradient Boosting)

5.36.1 Key Observations:

- **Strong Alignment with the Perfect Fit Line:**
 - The majority of points lie very close to the red dashed line ($y = x$), indicating that the model's predictions are highly accurate.
- **Minimal Deviation from Actual Values:**
 - The scatter points are well-distributed along the diagonal, suggesting that prediction errors are small.
- **Some Variability at Lower and Higher Ranges:**
 - While most points align closely, there are minor deviations at very low and very high values.
 - These deviations indicate slightly less accuracy at the extremes, though the overall model performance remains strong.
- **Tightly Clustered Predictions:**
 - The clustering of blue points around the red line shows a well-calibrated model with minimal systematic bias.

5.36.2 Interpretation:

- **High Predictive Accuracy:**
 - The model demonstrates excellent performance in predicting values, with minimal error.
- **No Major Systematic Bias:**
 - The absence of a clear pattern in deviations suggests that the model does not systematically overpredict or underpredict.
- **Well-Generalized Model:**
 - The results indicate that the Gradient Boosting model generalizes well to different data points, making it a reliable forecasting tool.

Residual Analysis

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Compute Residuals (Errors in Predictions)
# - Residuals = Actual Values - Predicted Values
# - Measures the difference between true and predicted values
residuals = y_test - gbm_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values (y_test)
# - y-axis: Residuals (y_test - gbm_pred)
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve
↳visibility
# - color='green': Sets the color of the residual points
```

```

plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a horizontal reference line at y=0 to indicate zero residuals
# - Helps in assessing whether residuals are symmetrically distributed around
  ↪ zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate residuals (errors in predictions)
plt.ylabel('Residuals')

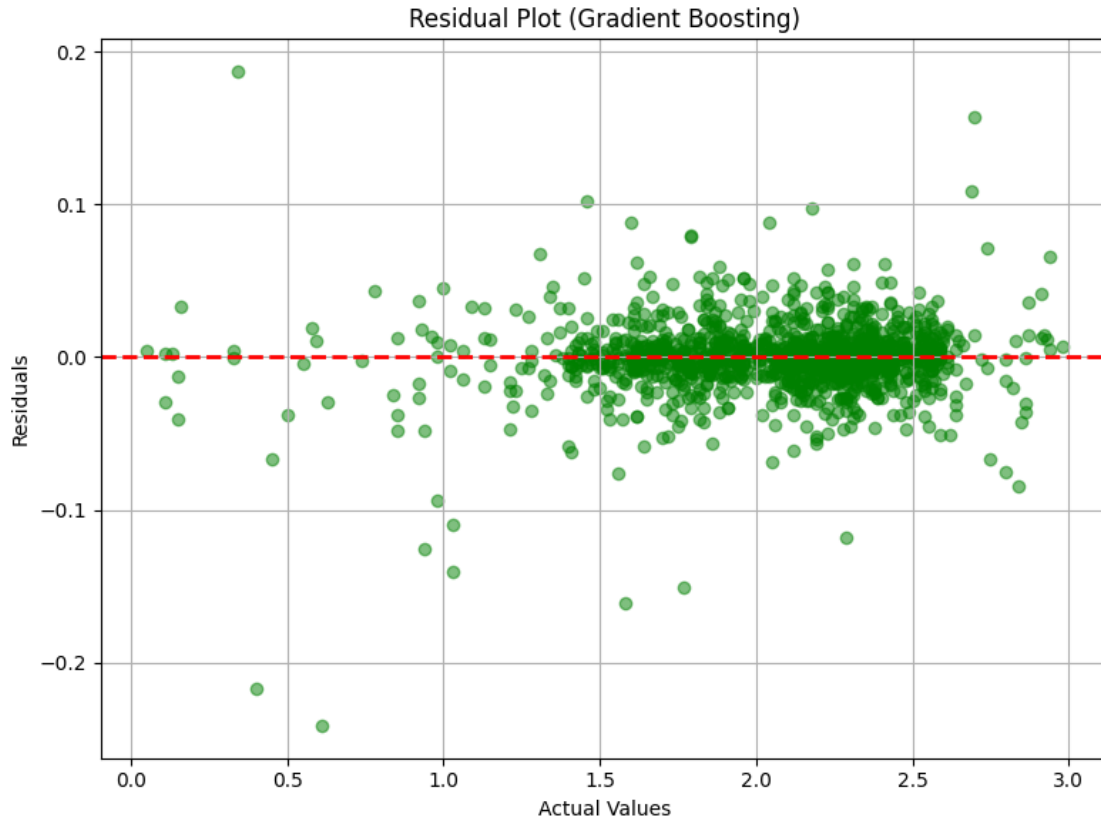
# Set a title for the plot to provide context
plt.title('Residual Plot (Gradient Boosting)')

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual scatter plot
plt.show()

```

5.37 Residual Plot (Gradient Boosting)

5.37.1 Key Observations:

- **Centered Around Zero:**
 - The residuals (errors) are distributed around the red dashed line at $y = 0$, indicating that the model does not have a strong bias in either direction.
- **No Clear Pattern in Residuals:**
 - The points are scattered randomly, suggesting that the model captures most of the variance in the data without systematic overprediction or underprediction.
- **Consistent Spread Across Values:**
 - The variance of residuals remains relatively constant across different ranges of actual values, indicating that the model performs similarly for both low and high values.
- **Few Outliers:**
 - Some points deviate significantly from zero, indicating that the model struggles with a small subset of predictions.

5.37.2 Interpretation:

- **Well-Calibrated Model:**
 - The residuals being evenly distributed around zero suggest that the Gradient Boosting model makes unbiased predictions.

- **No Heteroscedasticity:**
 - The residuals do not show a clear increasing or decreasing trend, meaning that the model's accuracy is stable across different values.
- **Good Predictive Performance:**
 - The small magnitude of residuals indicates that the model provides accurate predictions with minimal error.
- **Potential for Further Refinement:**
 - Addressing the outliers (extreme residual values) through feature engineering or additional model tuning may further improve performance.

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual and predicted values
# - bins=30: Divides the range of residuals into 30 bins for a detailed view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at x=0 to indicate no residual error
# - Helps assess whether residuals are symmetrically distributed around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

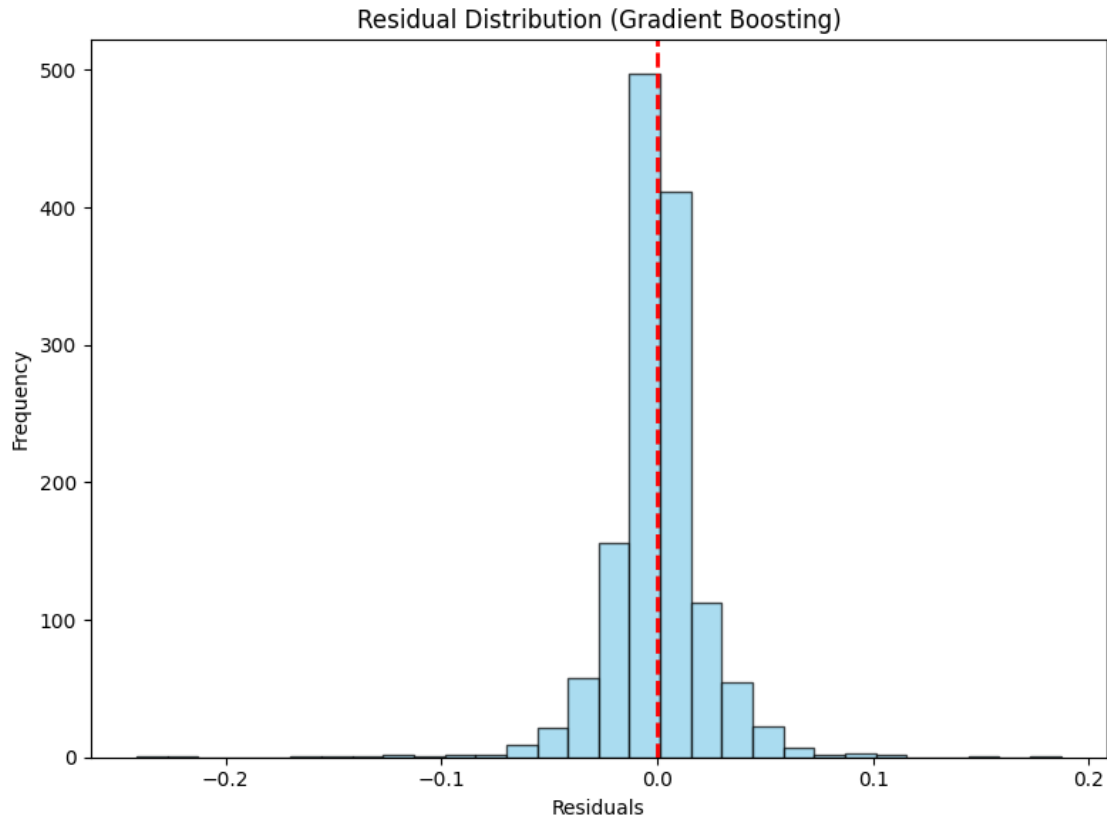
# Set a title for the plot to provide context
plt.title('Residual Distribution (Gradient Boosting)')

# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual histogram plot
plt.show()
```



5.38 Residual Distribution (Gradient Boosting)

5.38.1 Key Observations:

- **Centered Around Zero:**
 - The residuals are symmetrically distributed around 0, with the highest frequency occurring near zero, indicating that the model makes unbiased predictions.
- **Approximately Normal Distribution:**
 - The histogram follows a bell-shaped curve, suggesting that the errors are normally distributed, which is a good characteristic for a well-performing model.
- **Small Residual Magnitude:**
 - Most residuals fall within a narrow range, indicating that the model's prediction errors are generally small.
- **Few Extreme Residuals:**
 - There are very few outliers on both ends of the distribution, meaning that extreme mispredictions are rare.

5.38.2 Interpretation:

- **Well-Calibrated Model:**
 - The near-normal distribution of residuals suggests that the model generalizes well across the dataset.

- **Low Bias and High Precision:**
 - The concentration of residuals around zero confirms that the model does not systematically overpredict or underpredict.
- **Minor Room for Improvement:**
 - Although the distribution looks good, addressing the few outliers may further refine model accuracy.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Import Pandas to manipulate DataFrame

# Convert GridSearchCV results into a DataFrame
# - 'cv_results_' is a dictionary containing all cross-validation results from
  ↳ GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on the best model performance (lower
  ↳ is better)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
  ↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
  ↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↳ head(5)

# Print the top 5 hyperparameter combinations with better formatting
print("\nTop 5 Hyperparameter Combinations:\n")
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"Hyperparameters: {row['params']}")
    print(f"Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"Standard Deviation: {row['std_test_score']:.6f}")
    print("-" * 60) # Separator for better readability
```

Top 5 Hyperparameter Combinations:

Rank 31:

Hyperparameters: {'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 20, 'n_estimators': 300, 'subsample': 0.8}

Mean Test Score: -0.000627

Standard Deviation: 0.000042

```

-----
Rank 32:
Hyperparameters: {'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 20,
'n_estimators': 300, 'subsample': 0.9}
Mean Test Score: -0.000654
Standard Deviation: 0.000053
-----

```

```

Rank 27:
Hyperparameters: {'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 10,
'n_estimators': 300, 'subsample': 0.8}
Mean Test Score: -0.000656
Standard Deviation: 0.000073
-----

```

```

Rank 28:
Hyperparameters: {'learning_rate': 0.1, 'max_depth': 6, 'min_samples_split': 10,
'n_estimators': 300, 'subsample': 0.9}
Mean Test Score: -0.000664
Standard Deviation: 0.000065
-----

```

```

Rank 11:
Hyperparameters: {'learning_rate': 0.05, 'max_depth': 6, 'min_samples_split':
10, 'n_estimators': 300, 'subsample': 0.8}
Mean Test Score: -0.000665
Standard Deviation: 0.000064
-----

```

```

[ ]: # Import necessary libraries for visualization
import seaborn as sns # Seaborn for advanced heatmap visualization
import matplotlib.pyplot as plt # Matplotlib for general plotting

# Create a Heatmap to Visualize Hyperparameter Tuning Results

# Pivot the GridSearchCV results DataFrame for heatmap visualization
# - 'values': The metric we are analyzing ('mean_test_score' - the average test_
↳ score across CV folds)
# - 'index': The hyperparameter to be placed on the y-axis (learning_rate in_
↳ this case)
# - 'columns': The hyperparameter to be placed on the x-axis (n_estimators in_
↳ this case)
# - This transformation allows us to visualize how different combinations of_
↳ learning_rate and n_estimators impact model performance
heatmap_data = cv_results.pivot_table(
    values='mean_test_score', # Performance metric to visualize
    index='param_learning_rate', # Learning rate (rows)
    columns='param_n_estimators' # Number of estimators (columns)
)

```

```

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 6))

# Generate a heatmap using Seaborn
# - heatmap_data: The pivoted DataFrame containing the test scores
# - annot=True: Displays the numeric values in each cell for better analysis
# - fmt='.3f': Formats the numbers to three decimal places for precision
# - cmap='viridis': Uses the 'viridis' color map, which is visually appealing
# and easy to interpret
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

# Set the title of the heatmap for context
plt.title('Hyperparameter Tuning: Learning Rate vs n_estimators')

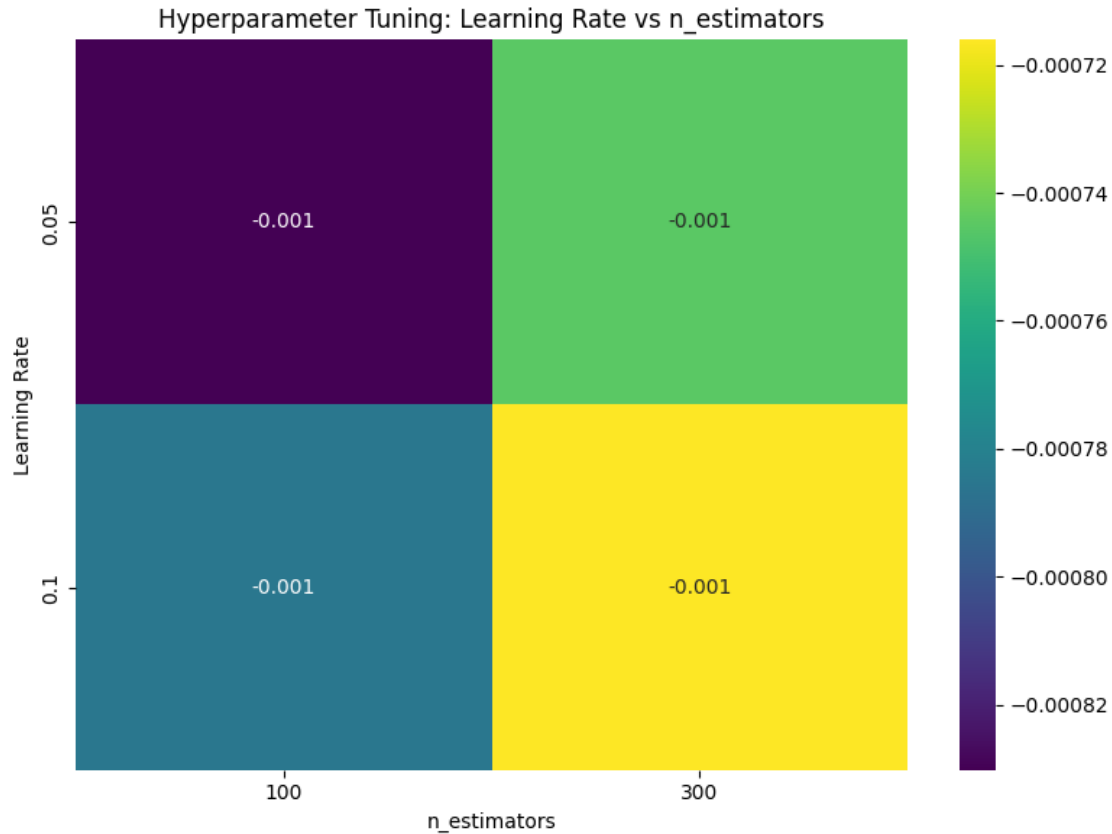
# Label the x-axis to indicate different n_estimators values
plt.xlabel('n_estimators')

# Label the y-axis to indicate different learning_rate values
plt.ylabel('Learning Rate')

# Adjust layout to ensure proper spacing and prevent overlapping labels
plt.tight_layout()

# Display the heatmap
plt.show()

```



5.39 Hyperparameter Tuning: Learning Rate vs n_estimators

5.39.1 Key Observations:

- **Comparison of Different Hyperparameter Combinations:**
 - The heatmap evaluates the effect of varying `learning_rate` and `n_estimators` on model performance.
- **Small Variation in Performance:**
 - The values in the heatmap indicate very small differences in performance across different hyperparameter settings, as all values are close to `-0.001`.
- **Higher `n_estimators` and Learning Rate of 0.1 Performs Best:**
 - The lowest error (lightest yellow shade) occurs at `learning_rate = 0.1` and `n_estimators = 300`, suggesting that this combination may provide the best performance.
- **Darker Colors Indicate Slightly Worse Performance:**
 - The darkest region (`learning_rate = 0.05` and `n_estimators = 100`) represents the highest error, though the differences are minimal.

5.39.2 Interpretation:

- **Higher `n_estimators` Improves Performance:**

- Increasing the number of estimators to 300 appears to slightly enhance model accuracy.
- **Optimal Learning Rate Appears to be 0.1:**
 - A higher learning rate (0.1) coupled with more estimators yields the best results.
- **Minimal Impact Across Variations:**
 - The relatively small differences across the hyperparameters suggest that the model is not overly sensitive to changes in `learning_rate` and `n_estimators`, but fine-tuning still provides incremental gains.
- **Potential for Further Optimization:**
 - Experimenting with a wider range of values, such as learning rates below 0.05 or more estimators beyond 300, might help refine performance further.

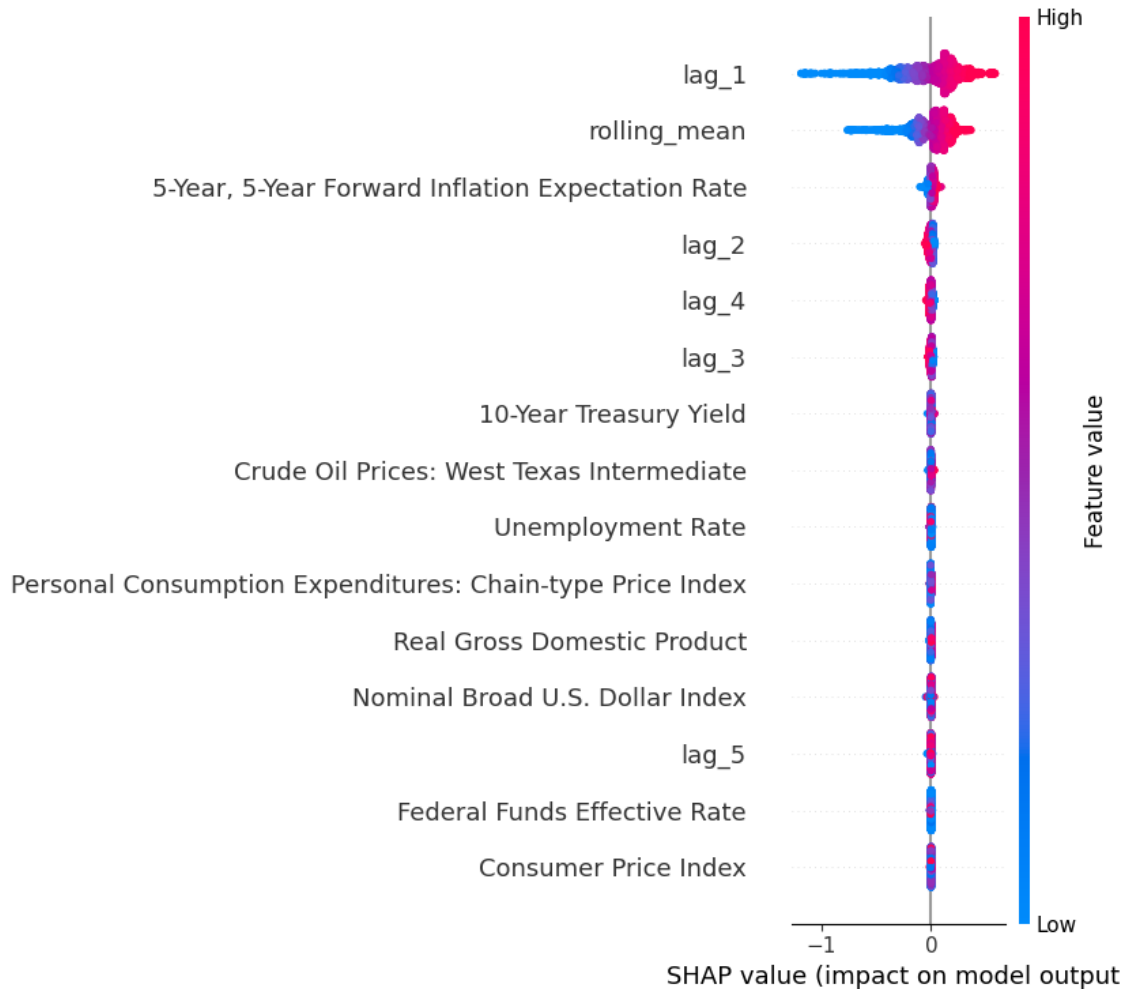
SHAP Values

```
[ ]: # Import necessary libraries for SHAP visualization and Matplotlib for plotting
import shap # SHAP library for model interpretability
import matplotlib.pyplot as plt # Matplotlib for general plotting

# Create a SHAP Explainer for the Gradient Boosting Model
# - SHAP (SHapley Additive exPlanations) helps interpret machine learning
  ↪models.
# - 'TreeExplainer' is optimized for tree-based models like Gradient Boosting,
  ↪XGBoost, and LightGBM.
# - 'best_gbm_model' is the trained Gradient Boosting model we want to explain.
explainer = shap.TreeExplainer(best_gbm_model)

# Compute SHAP values for the training dataset
# - SHAP values quantify how much each feature contributes to a model's
  ↪prediction.
# - This helps in understanding which features positively or negatively impact
  ↪model predictions.
shap_values = explainer.shap_values(X_train)

# Global Explanation: SHAP Summary Plot
# - Displays overall feature importance across all training samples.
# - Shows how each feature influences predictions using color-coded scatter
  ↪points.
# - Red = High feature values, Blue = Low feature values.
shap.summary_plot(shap_values, X_train)
```

5.40 SHAP Summary Plot

5.40.1 Key Observations:

- **Feature Importance via SHAP Values:**
 - lag_1 and rolling_mean have the highest SHAP values, indicating that they are the most influential features in the model's predictions.
- **SHAP Values Indicate Direction of Impact:**
 - Features with high values (colored red) contribute positively to the model's output.
 - Features with low values (colored blue) contribute negatively to the model's output.
- **Top Features Impacting Predictions:**
 - Apart from lag_1 and rolling_mean, the 5-Year, 5-Year Forward Inflation Expectation Rate and lag_2 also show significant contributions.
- **Symmetric Distribution Around Zero:**
 - Most features have a balanced distribution of positive and negative SHAP values, suggesting a well-calibrated model with no extreme biases.
- **Lower Impact Features:**

- Consumer Price Index, Federal Funds Effective Rate, and Real Gross Domestic Product have minimal influence on the predictions.

5.40.2 Interpretation:

- **Model is Highly Influenced by Recent Historical Values:**
 - The presence of multiple lag features (`lag_1`, `lag_2`, `lag_3`, `lag_4`, `lag_5`) suggests that past values strongly determine current predictions.
- **Rolling Mean Adds Stability:**
 - The `rolling_mean` feature plays a key role, indicating that smoothed historical trends contribute significantly to the model's output.
- **Macroeconomic Indicators Play a Lesser Role:**
 - Features like `Consumer Price Index` and `Unemployment Rate` have lower SHAP values, implying that they contribute less to the model's decision-making.

```
[ ]: # Enable SHAP JavaScript-based visualization (for interactive plots in Jupyter_
      ↪Notebooks)
      shap.initjs()

      # Local Explanation: SHAP Force Plot for a Single Instance
      # - Explains how individual features contribute to a specific prediction.
      # - explainer.expected_value: The base prediction (average model output).
      # - shap_values[5]: SHAP values for the 5th training instance.
      # - X_train.iloc[5]: The actual feature values of the 5th training instance.
      shap.force_plot(explainer.expected_value, shap_values[5], X_train.iloc[5])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0de03300d0>
```

5.41 SHAP Force Plot - Feature Contribution to Prediction

5.41.1 Key Observations:

- **Predicted Value ($f(\mathbf{x}) = 2.34$):**
 - The final predicted output is **2.34**, which is influenced by both positive and negative contributing features.
- **Positive Contributions (Red - Pushing the Prediction Higher):**
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.48
 - `rolling_mean` = 2.382
 - `lag_1` = 2.34
 - These features had a strong positive influence, pushing the prediction value above the base value.
- **Negative Contributions (Blue - Pushing the Prediction Lower):**
 - `lag_2` = 2.39
 - `lag_4` = 2.42
 - `lag_3` = 2.42
 - These features contributed to reducing the prediction but did not dominate.
- **Base Value (2.055):**

- This is the expected prediction before considering the effects of individual feature values.
- **Interpretation:**
 - The prediction was primarily **driven upwards by macroeconomic factors (5-Year Forward Inflation Expectation Rate) and historical lag features (lag_1)**.
 - Some lag features (lag_2, lag_3, lag_4) acted in opposition but were not strong enough to lower the final prediction significantly.

5.41.2 Conclusion:

- The **5-Year Forward Inflation Expectation Rate** and **rolling mean** are among the most important features driving the prediction.
- The effect of historical lag values varies, with some increasing and others decreasing the predicted value.
- This SHAP force plot helps interpret how individual features influence a specific model prediction.

Partial Dependence Plots (PDP)

```
[ ]: # Import necessary libraries for visualization and model inspection
import matplotlib.pyplot as plt # Import Matplotlib for plotting
from sklearn.inspection import PartialDependenceDisplay # Import PDP plotting tool
import warnings # Import warnings module to suppress unnecessary warnings

# Suppress FutureWarnings for PDP plotting
# - Some versions of scikit-learn may trigger FutureWarnings when using PartialDependenceDisplay.
# - This prevents unnecessary warning messages from appearing.
warnings.filterwarnings("ignore", category=FutureWarning)

# Extract feature names from the training dataset
# - Ensures that all features from X_train are used in the Partial Dependence Plots.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDP) for all features using the trained Gradient Boosting Model
# - PDP helps visualize how a specific feature impacts the model's predictions.
# - kind='average': Computes the **average** partial dependence over all instances.
# - grid_resolution=50: Number of points on the x-axis (higher values = smoother curves).
# - n_cols=4: Arranges the plots into 4 columns for better visualization.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_gbm_model, # The trained Gradient Boosting model
    X_train, # The dataset used to generate PDP plots
    features=features_list, # List of feature names to analyze
    kind='average', # Compute average partial dependence values
```

```

    grid_resolution=50, # Defines the granularity of the x-axis (smoother
    ↪ curve with more points)
    n_cols=4 # Arranges PDP plots into 4 columns for better layout
)

# Customize the PDP Figure for Better Readability

# Set the overall figure size for better visualization
pdp_display.figure_.set_size_inches(18, 12)

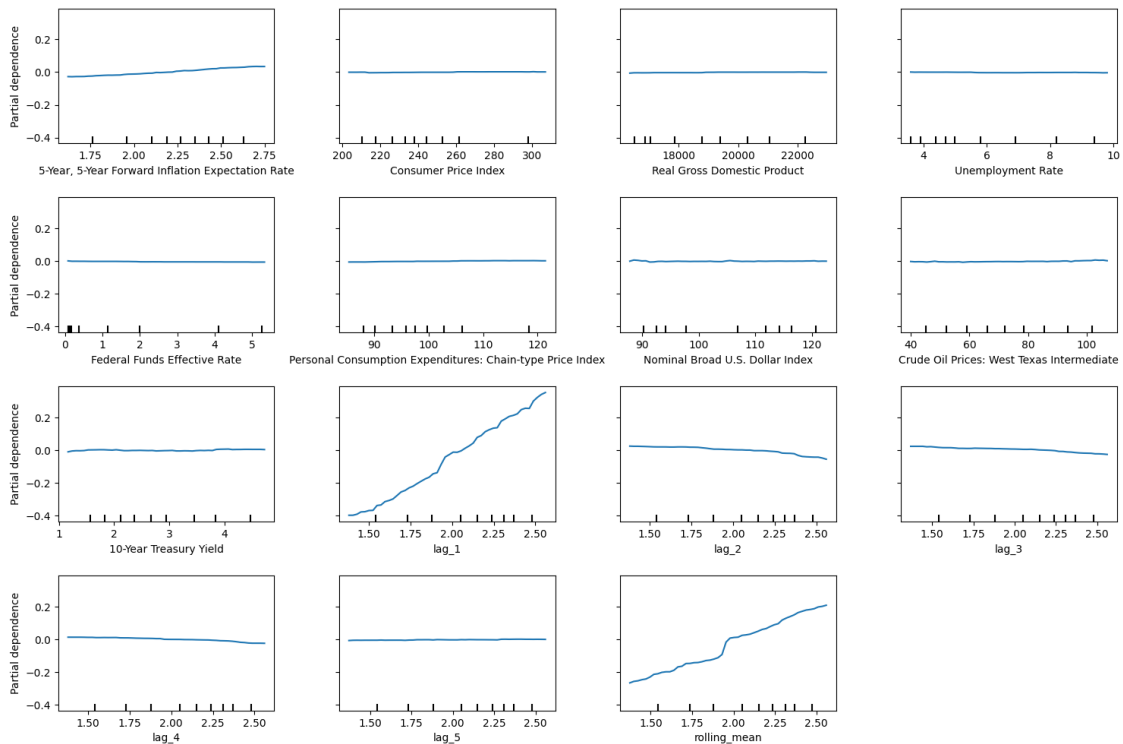
# Add a title to the entire figure, positioned above all subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (Gradient Boosting)",
    ↪ fontsize=16, y=1.06)

# Adjust subplot spacing to prevent overlapping titles and labels
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the Partial Dependence Plots
plt.show()

```

Partial Dependence Plots (Gradient Boosting)



5.42 Partial Dependence Plots (Gradient Boosting)

5.42.1 Key Observations:

- **Understanding Partial Dependence:**
 - Partial dependence plots (PDPs) illustrate the marginal effect of a single feature on the model's predictions.
 - They help understand how changes in a particular feature affect the predicted outcome while keeping other variables constant.
- **Notable Feature Impacts:**
 - **5-Year Forward Inflation Expectation Rate:**
 - * Shows a **slight positive trend**, indicating that an increase in this economic indicator leads to a higher prediction.
 - **Consumer Price Index (CPI):**
 - * Has **minimal effect** on the model's predictions, suggesting low importance.
 - **Real Gross Domestic Product (GDP):**
 - * Also shows **little influence**, implying that this feature may not be strongly correlated with the target variable.
 - **Unemployment Rate & Federal Funds Effective Rate:**
 - * These features exhibit a **flat response**, indicating little to no direct effect on the predictions.
 - **Crude Oil Prices (West Texas Intermediate):**
 - * Displays a **slight positive trend**, showing that increasing oil prices have some impact on the predicted values.
 - **Time Lag Features (lag_1, lag_2, lag_3, lag_4, lag_5, rolling_mean):**
 - * **lag_1** and **rolling_mean** show a **strong increasing trend**, indicating that these lagged features are significant drivers of the prediction.
 - * **lag_2** and **lag_3** show a **decreasing trend**, suggesting that higher values may lead to lower predictions.

5.42.2 Conclusion:

- The **most influential factors** in the model appear to be **lag_1**, **rolling_mean**, and **5-Year Forward Inflation Expectation Rate**.
- Some macroeconomic indicators, like **CPI** and **GDP**, seem to have **limited effect**.
- The **trend of lag features** indicates that the model relies heavily on past values to make predictions.

This analysis highlights the key drivers of the model's predictions and provides insight into the relationship between different features and the target variable.

Feature Importance using GradientBoosting's Built-In Function

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Visualization: Built-in Feature Importance for Gradient Boosting Model
```

```

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars for better visibility
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Feature Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

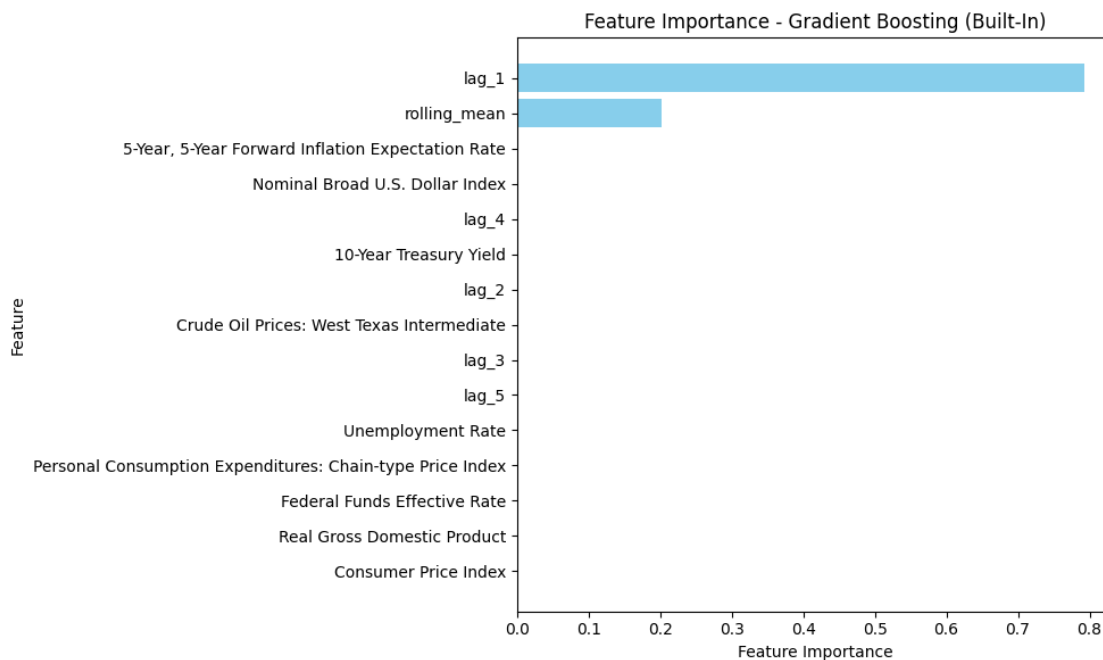
# Set the title of the plot for context
plt.title('Feature Importance - Gradient Boosting (Built-In)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()

```



5.43 Feature Importance - Gradient Boosting (Built-In)

5.43.1 Key Observations:

- **Top Influential Features:**
 - **lag_1** is the most important feature, significantly contributing to the model's predictions.
 - **rolling_mean** is the second most influential factor, but its impact is notably smaller than lag_1.
 - Other features have **minimal or negligible importance** in the built-in Gradient Boosting model.
- **Economic Indicators and Lag Features:**
 - **5-Year Forward Inflation Expectation Rate**, **Nominal Broad U.S. Dollar Index**, and **10-Year Treasury Yield** appear on the chart, but their importance is close to zero.
 - Other macroeconomic indicators, such as **Consumer Price Index (CPI)**, **Real Gross Domestic Product (GDP)**, and **Unemployment Rate**, have little to no impact.
- **Model's Dependence on Lagged Data:**
 - The dominance of **lag_1** and **rolling_mean** suggests that the model heavily relies on past values rather than external economic indicators.
 - The lack of importance assigned to fundamental macroeconomic features implies that historical trends in the target variable are the primary drivers of prediction accuracy.

5.43.2 Conclusion:

- The model is **strongly dependent on lagged features**, particularly **lag_1**, meaning past values are the best predictors of future values.
- Macroeconomic factors such as **CPI**, **GDP**, and **Unemployment Rate** have **negligible influence** on the predictions.
- **Feature selection could be optimized**, as many features in the dataset do not significantly contribute to model performance.

Permutation Feature Importance

```
[ ]: # Import necessary libraries
from sklearn.inspection import permutation_importance # Import permutation
import matplotlib.pyplot as plt # Import Matplotlib for visualization
import pandas as pd # Import Pandas for handling tabular data

# Compute Permutation Feature Importance on the Test Set
# - Permutation importance measures the effect of **randomly shuffling a
# - It provides a better estimate of feature importance compared to built-in
# - `n_repeats=10`: The number of times each feature is randomly shuffled to
```

```

# - `random_state=42`: Ensures reproducibility of results.
perm_importance = permutation_importance(
    best_gbm_model, # The trained Gradient Boosting model
    X_test, # Test dataset (features)
    y_test, # Test dataset (target variable)
    n_repeats=10, # Number of shuffling repetitions
    random_state=42 # Ensures consistent results across runs
)

# Convert Permutation Importance Results into a Pandas DataFrame
# - 'Feature': Column storing feature names.
# - 'Importance': The mean importance score from the permutation process.
# - The DataFrame is sorted in descending order to highlight the most important
  ↳ features first.
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Extracts feature names from training data
    'Importance': perm_importance.importances_mean # Mean importance scores
  ↳ across repetitions
}).sort_values(by='Importance', ascending=False) # Sort features by importance

# Visualization: Permutation Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display permutation importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars for better visibility
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
  ↳ color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Permutation Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

# Set the title of the plot for context
plt.title('Permutation Feature Importance - Gradient Boosting')

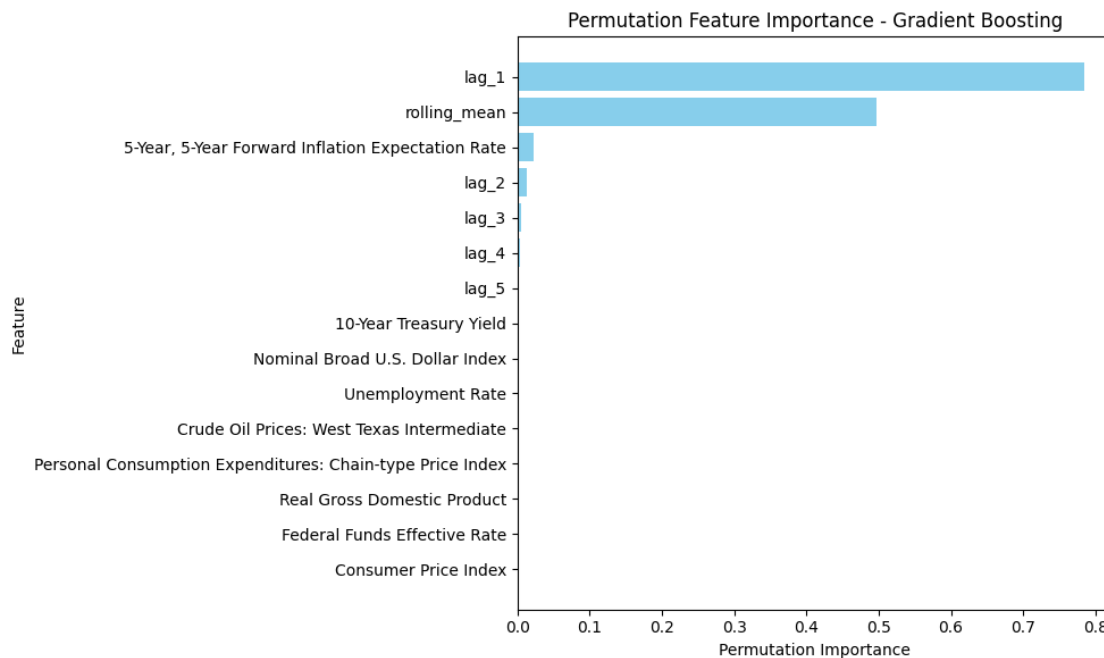
# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

```



```
# Display the feature importance plot  
plt.show()
```



5.44 Permutation Feature Importance - Gradient Boosting

5.44.1 Key Observations:

- **Top Influential Features:**
 - **lag_1** is the most important feature, contributing significantly to the model's predictions.
 - **rolling_mean** follows as the second most influential factor but with lower importance compared to lag_1.
 - **5-Year Forward Inflation Expectation Rate** and **lag_2** have minimal but non-negligible contributions.
- **Negligible Influence of Economic Indicators:**
 - Key macroeconomic indicators such as **10-Year Treasury Yield**, **Nominal Broad U.S. Dollar Index**, **Consumer Price Index**, **Real GDP**, and **Crude Oil Prices** show **little to no importance**.
 - The model does not significantly rely on external economic indicators for its predictions.
- **Dependence on Past Data:**
 - **Lag features (lag_1, rolling_mean, lag_2, etc.)** dominate the feature importance, confirming that past values of the target variable are **highly predictive** of future values.
 - The heavy reliance on lag features suggests the model primarily captures temporal patterns rather than external influences.

5.44.2 Conclusion:

- The model's performance is **driven mainly by lagged values**, particularly **lag_1** and **rolling_mean**.
- **Macroeconomic indicators have minimal relevance**, implying that historical trends in the target variable are more predictive than external economic factors.
- **Permutation importance analysis** confirms that removing lagged values would **significantly impact model performance**, while removing economic indicators would have little to no effect.

5.45 Extremely Randomized Trees Regressor (ERTR)

```
[ ]: # Import necessary libraries
import random # Import random module for setting random seed
import numpy as np # Import NumPy for numerical computations
import warnings # Import warnings module to suppress unnecessary warnings
from sklearn.model_selection import train_test_split, GridSearchCV # Import
    ↪ tools for data splitting and hyperparameter tuning
from sklearn.ensemble import ExtraTreesRegressor # Import ExtraTrees Regressor
    ↪ (Extremely Randomized Trees)

# Suppress specific warnings related to casting issues
# - Some scikit-learn functions may generate RuntimeWarnings due to invalid
    ↪ value casting.
# - This ensures the script runs without displaying unnecessary warnings.
warnings.filterwarnings("ignore", category=RuntimeWarning, message="invalid
    ↪ value encountered in cast")

# Set random seeds for reproducibility
# - Ensures that the results remain consistent across different runs.
random.seed(42)
np.random.seed(42)

# Split the dataset into training and testing sets (80% training, 20% testing)
# - X: Feature matrix
# - y: Target variable
# - test_size=0.2: Reserves 20% of the data for testing
# - random_state=42: Ensures consistent data splits across runs
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# Initialize the ExtraTrees Regressor (ERTR) model
# - ExtraTreesRegressor is an ensemble method that creates multiple decision
    ↪ trees with more randomness than Random Forest.
# - random_state=42: Ensures reproducibility of results
et_model = ExtraTreesRegressor(random_state=42)
```

```

# Define the hyperparameter grid for tuning ExtraTrees Regressor
# - 'n_estimators': Number of trees in the forest (more trees = better
↳ performance but longer training time)
# - 'max_depth': Maximum depth of trees (controls overfitting)
# - 'min_samples_split': Minimum number of samples required to split an
↳ internal node
# - 'min_samples_leaf': Minimum number of samples required at each leaf node
# - 'bootstrap': Whether bootstrap samples are used for training
param_grid = {
    'n_estimators': [100, 200, 500],      # Number of trees (boosting rounds)
    'max_depth': [3, 5, 10],              # Maximum depth of trees
    'min_samples_split': [2, 5, 10],      # Minimum samples required to split
↳ an internal node
    'min_samples_leaf': [1, 5, 10],       # Minimum samples required at each
↳ leaf node
    'bootstrap': [True, False]            # Whether bootstrap sampling is used
}

# Initialize GridSearchCV for hyperparameter tuning
# - GridSearchCV performs an exhaustive search over the parameter grid to find
↳ the best combination.
# - estimator=et_model: Uses the ExtraTrees Regressor as the model.
# - param_grid=param_grid: Defines the hyperparameter space to search.
# - scoring='neg_mean_squared_error': Uses negative MSE as the scoring metric
↳ (lower is better).
# - cv=5: Uses 5-fold cross-validation to ensure robust evaluation.
# - verbose=1: Displays the search process progress.
# - n_jobs=-1: Uses all available CPU cores for faster processing.
grid_search = GridSearchCV(
    estimator=et_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Fit the GridSearchCV model on the training data
# - Evaluates different hyperparameter combinations to find the best-performing
↳ model.
grid_search.fit(X_train, y_train)

# Retrieve and print the best hyperparameters found
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

```

```

# Get the best ExtraTrees Regressor model from GridSearchCV
# - This model is trained using the optimal hyperparameter combination.
best_et_model = grid_search.best_estimator_

# Make predictions on the test dataset using the best model
et_pred = best_et_model.predict(X_test)

# Evaluate the model using a predefined function
# - `calculate_metrics`: Assumed to be a function that calculates evaluation
  ↳ metrics.
# - `naive_forecast`: Assumed to be a baseline prediction for comparison.
# - The evaluation results are appended to `metrics_results`.
metrics_results.append(calculate_metrics(y_test.values, et_pred,
  ↳ naive_forecast, "Extremely Randomized Trees Regressor (ERTR)"))

# Print the evaluation results for ExtraTrees Regressor
print("ExtraTrees Regressor Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 162 candidates, totalling 810 fits
 Best Hyperparameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 500}
 ExtraTrees Regressor Evaluation Metrics:
 {'Model': 'Extremely Randomized Trees Regressor (ERTR)', 'MSE': 0.0007227462690668256, 'RMSE': 0.026883940728003878, 'MAE': 0.016370082040005472, 'MAPE (%)': 1.1514243212219075, 'sMAPE (%)': 1.0918019278863444, 'MASE': 0.041670960046053356, 'R^2': 0.9959126220643945}

Custom Feature Importance Plot

```

[ ]: # Import necessary libraries
import pandas as pd # Import Pandas for handling tabular data
import matplotlib.pyplot as plt # Import Matplotlib for visualization

# Extract Feature Importance from the Best ExtraTrees Model

# - 'feature_importances_': Retrieves an array where each value corresponds to
  ↳ the importance of a feature.
# - 'X_train.columns': Extracts the feature names from the training dataset.
feature_importance = best_et_model.feature_importances_
features = X_train.columns

# Create a DataFrame to Store Feature Importance Scores

# - 'Feature': Column storing feature names.
# - 'Importance': Column storing corresponding feature importance scores.
# - Sorted in descending order so that the most important features appear at
  ↳ the top.

```

```

importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importance
}).sort_values(by='Importance', ascending=False)

# Print Feature Importance DataFrame for Reference
print("Feature Importances:")
print(importance_df)

```

Feature Importances:

	Feature	Importance
14	rolling_mean	0.192706
10	lag_2	0.172242
9	lag_1	0.167844
12	lag_4	0.151660
13	lag_5	0.147841
11	lag_3	0.136892
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.022326
7	Crude Oil Prices: West Texas Intermediate	0.007055
8	10-Year Treasury Yield	0.000301
5	Personal Consumption Expenditures: Chain-type ...	0.000261
3	Unemployment Rate	0.000239
1	Consumer Price Index	0.000184
6	Nominal Broad U.S. Dollar Index	0.000171
4	Federal Funds Effective Rate	0.000160
2	Real Gross Domestic Product	0.000118

```

[ ]: # Visualization: Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars for better visibility
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

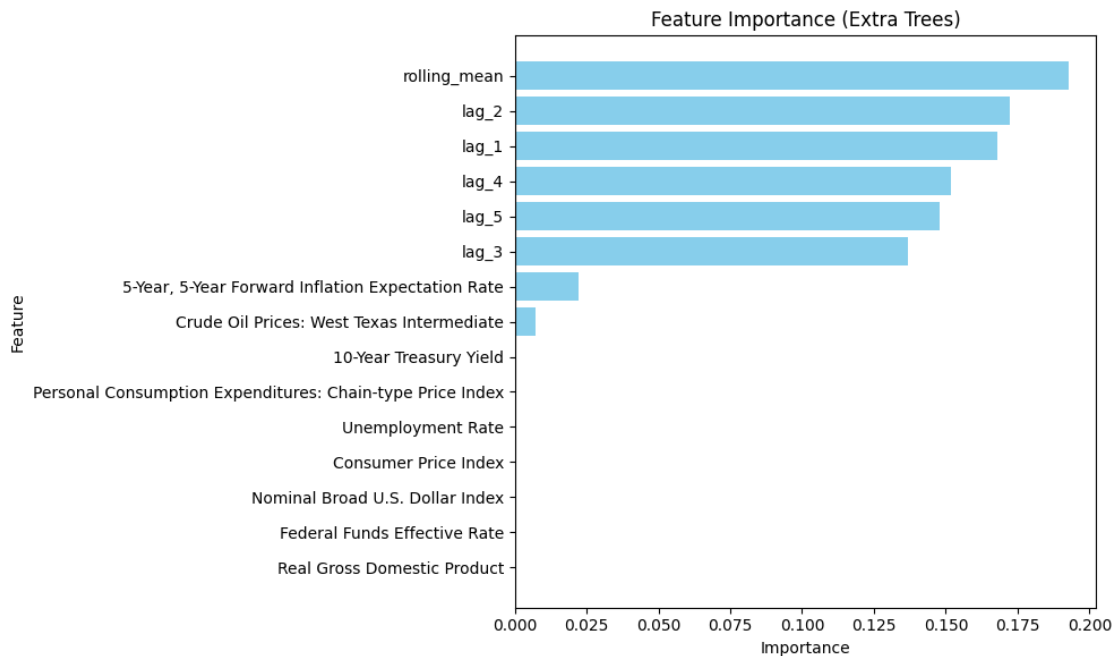
# Set the title of the plot for context
plt.title('Feature Importance (Extra Trees)')

```

```
# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



5.46 Feature Importance (Extra Trees)

5.46.1 Key Observations:

- **Top Influential Features:**
 - **rolling_mean** is the most important feature, indicating that the moving average of past values significantly impacts the predictions.
 - **lag_2** and **lag_1** are the second and third most influential features, emphasizing the model's reliance on past observations.
 - **lag_4**, **lag_5**, and **lag_3** also contribute considerably, further supporting the temporal dependency in the model.
- **Limited Influence of External Economic Indicators:**
 - Macroeconomic indicators such as **Crude Oil Prices**, **10-Year Treasury Yield**, **Unemployment Rate**, **Consumer Price Index**, and **Real GDP** have **minimal importance**.
 - This suggests that the model does not significantly incorporate broader economic trends but instead relies heavily on historical values of the target variable.

- **Temporal Dependency in Predictions:**

- The strong importance of **lag features** (**lag_1 to lag_5**) and **rolling_mean** suggests that the model performs well by capturing past trends.
- The lack of emphasis on external macroeconomic indicators indicates that the model is **more autoregressive** rather than influenced by economic events.

5.46.2 Conclusion:

- The **Extra Trees model** prioritizes past values and trends (**rolling_mean and lags**) for prediction, rather than relying on macroeconomic indicators.
- The minimal contribution of external factors implies that historical time-series patterns are more predictive than broader economic variables.
- This feature importance ranking reinforces the model's **time-dependent nature**, making it more suited for forecasting applications where past trends play a dominant role.

Actual vs. Predicted Values

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 8))

# Scatter Plot: Compare Actual vs. Predicted Values
# - x-axis: Actual values from the test dataset (y_test)
# - y-axis: Predictions made by the Extra Trees model (et_pred)
# - alpha=0.5: Makes points semi-transparent to reduce overlap and improve
↳visibility
# - color='blue': Sets the color of the scatter points
# - label='Predictions': Legend label for scatter points
plt.scatter(y_test, et_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the Perfect Fit Line (Reference Line)
# - This diagonal line represents the ideal case where predictions perfectly
↳match actual values.
# - If the model were perfect, all points would lie exactly on this line.
# - color='red': Sets the reference line color to red
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the line thickness
# - label='Perfect Fit': Legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')
```

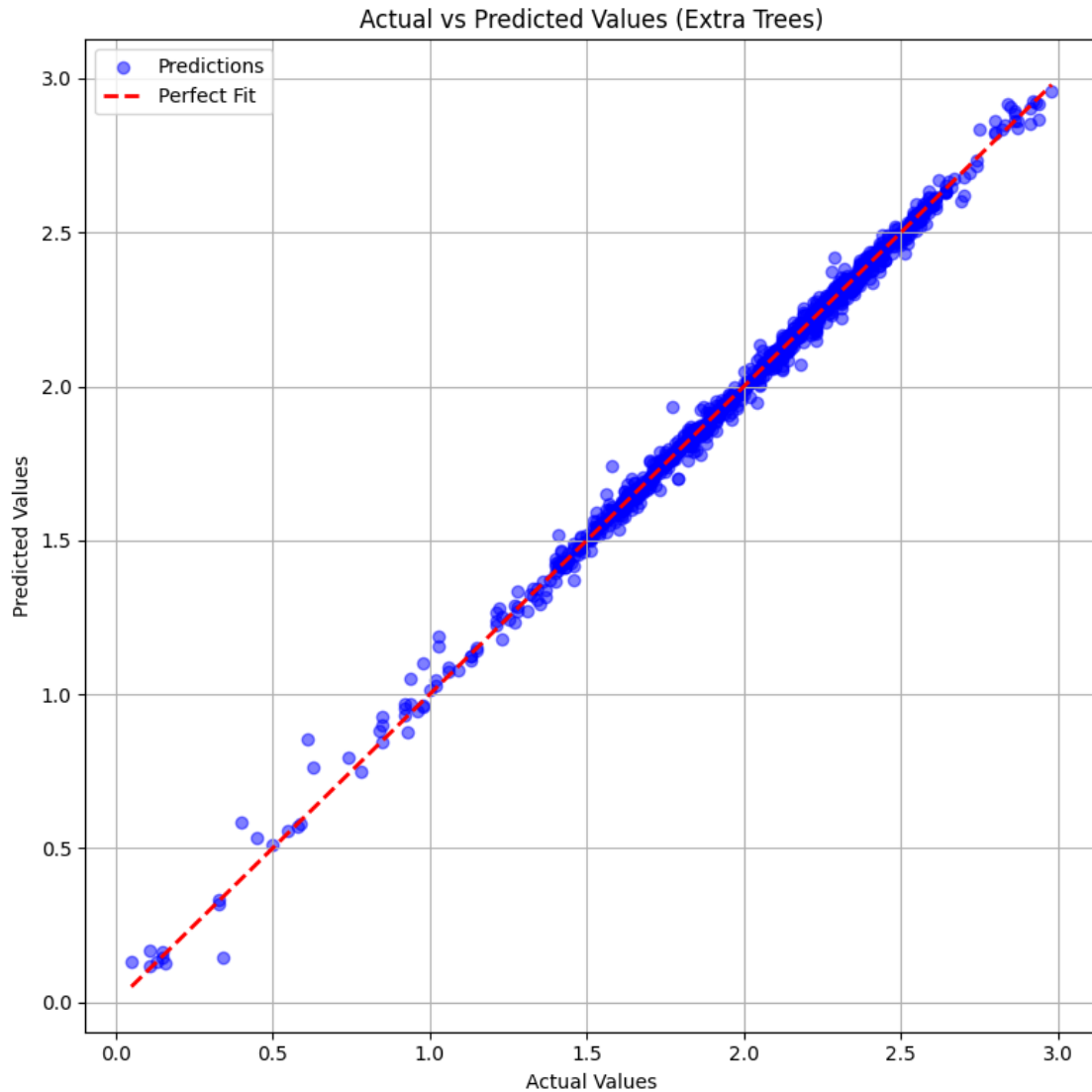
```
# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (Extra Trees)')

# Add a legend to differentiate the scatter points (Predictions) from the
↳ reference line (Perfect Fit)
plt.legend()

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()
```

5.47 Actual vs Predicted Values (Extra Trees)

5.47.1 Key Observations:

- **Strong Agreement Between Predictions and Actual Values:**
 - The blue dots (predictions) align closely with the red dashed line, which represents a **perfect fit** (i.e., actual values = predicted values).
 - This suggests that the **Extra Trees model** provides highly accurate predictions.
- **Minimal Deviation from the Perfect Fit Line:**
 - Most points fall along the red dashed line, indicating a **low error rate**.
 - There are only a few slight deviations, mostly for lower values, implying small residual errors.
- **Performance Across the Range of Values:**
 - The model maintains **consistency** in predictions across the full range of actual values.

- There is **no visible pattern of bias**, meaning the model does not systematically over-predict or underpredict in any specific range.

5.47.2 Conclusion:

- The **Extra Trees model** performs exceptionally well in predicting the target variable.
- The predictions **closely match the actual values**, suggesting that the model has effectively captured the underlying patterns in the data.
- The small deviations at lower values indicate slight prediction variance, but overall, the model shows **strong generalization ability**.

Residual Analysis

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Compute Residuals (Errors in Predictions)
# - Residuals = Actual Values - Predicted Values
# - Measures the difference between true and predicted values
residuals = y_test - et_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values (y_test)
# - y-axis: Residuals (y_test - et_pred)
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve
    ↪ visibility
# - color='green': Sets the color of the residual points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a horizontal reference line at y=0 to indicate zero residuals
# - Helps in assessing whether residuals are symmetrically distributed around
    ↪ zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

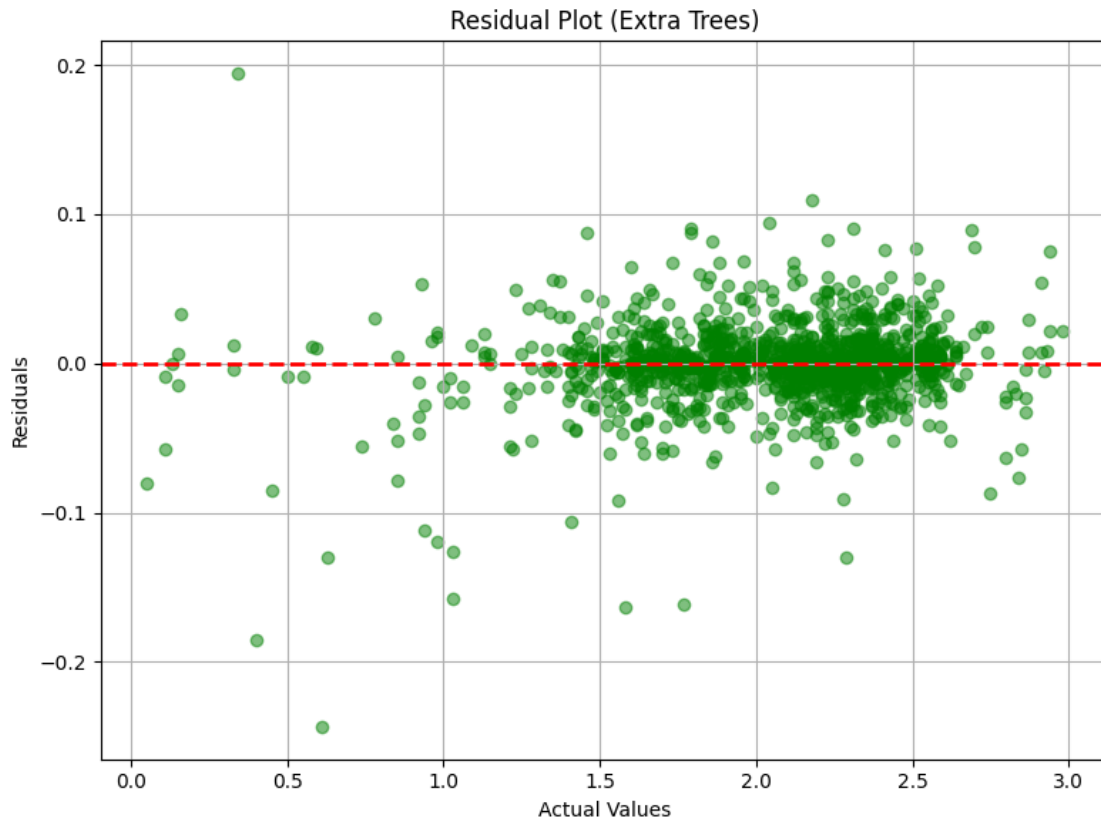
# Label the y-axis to indicate residuals (errors in predictions)
plt.ylabel('Residuals')

# Set a title for the plot to provide context
plt.title('Residual Plot (Extra Trees)')
```

```
# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual scatter plot
plt.show()
```



5.48 Residual Plot (Extra Trees)

5.48.1 Key Observations:

- **Centered Around Zero:**
 - The residuals (difference between actual and predicted values) are **evenly distributed** around the zero line, suggesting that the Extra Trees model does not exhibit systematic bias.
- **Randomly Scattered Residuals:**
 - The residuals do not show any clear pattern or trend, confirming that the model's errors are **randomly distributed**.

- This randomness indicates that the model captures the data well and does not suffer from underfitting or overfitting.
- **Variance Across Actual Values:**
 - The spread of residuals remains relatively **consistent across the range of actual values**, meaning the model's prediction errors do not significantly vary for different target values.
 - However, there are a few **outliers** (larger residuals), particularly at lower values, which might indicate areas where the model struggles slightly.

5.48.2 Conclusion:

- The **Extra Trees model** performs well, as the residuals are **randomly distributed** around zero, indicating no major bias.
- The model does not systematically underpredict or overpredict at any specific range of actual values.
- The presence of **a few outliers** suggests areas where further tuning or additional features could improve predictions.

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual and predicted values
# - bins=30: Divides the range of residuals into 30 bins for a detailed view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at x=0 to indicate no residual error
# - Helps assess whether residuals are symmetrically distributed around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

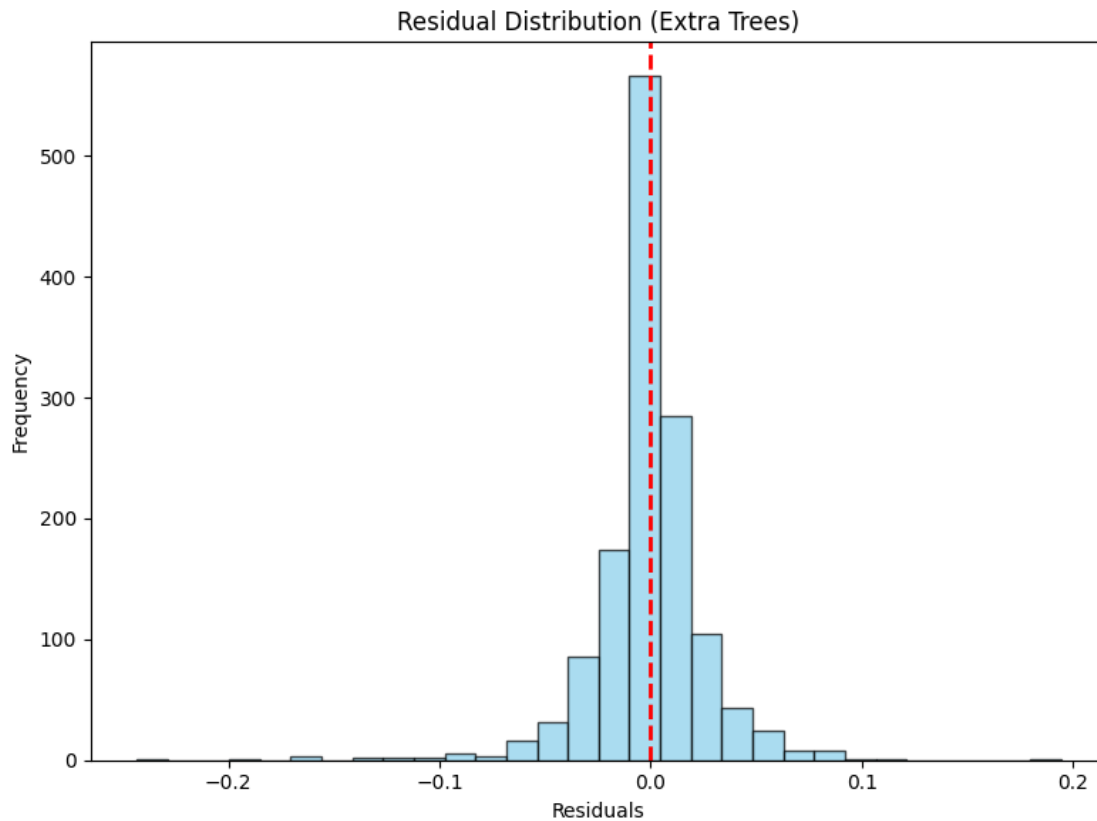
# Set a title for the plot to provide context
plt.title('Residual Distribution (Extra Trees)')

# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')
```

```
# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual histogram plot
plt.show()
```



5.49 Residual Distribution (Extra Trees)

5.49.1 Key Observations:

- **Centered Around Zero:**
 - The residuals (errors) are symmetrically distributed around **zero**, indicating that the Extra Trees model does not have a strong bias in overpredicting or underpredicting.
- **Normal-Like Distribution:**
 - The histogram resembles a **bell-shaped curve**, suggesting that the residuals follow a normal distribution.
 - This is a good indication that the model's errors are **random** rather than systematic.
- **Small Residual Spread:**
 - The residual values are **mostly concentrated near zero**, with a small number of residuals extending to the left and right.
 - This suggests that the model's predictions are **accurate**, with only a few large errors.

- **Outliers Present:**

- While most residuals are close to zero, a few **outliers** exist in the left and right tails, which could indicate specific instances where the model performs less accurately.

5.49.2 Conclusion:

- The Extra Trees model appears to have **good predictive accuracy**, as residuals are centered around zero and normally distributed.
- The narrow spread indicates that the model is **consistent in its predictions**, with few large deviations.
- Some **outliers exist**, which could be further investigated for potential model improvements.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Import Pandas to manipulate DataFrame

# Convert GridSearchCV results into a DataFrame
# - 'cv_results_' is a dictionary containing all cross-validation results from
  ↳ GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on the best model performance (lower
  ↳ is better)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
  ↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
  ↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↳ head(5)

# Print the top 5 hyperparameter combinations with better formatting
print("\nTop 5 Hyperparameter Combinations:\n")
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"Hyperparameters: {row['params']}")
    print(f"Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"Standard Deviation: {row['std_test_score']:.6f}")
    print("-" * 60) # Separator for better readability
```

Top 5 Hyperparameter Combinations:

Rank 138:
Hyperparameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 500}
Mean Test Score: -0.000787
Standard Deviation: 0.000100

Rank 137:
Hyperparameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
Mean Test Score: -0.000791
Standard Deviation: 0.000098

Rank 136:
Hyperparameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
Mean Test Score: -0.000792
Standard Deviation: 0.000098

Rank 139:
Hyperparameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 100}
Mean Test Score: -0.000792
Standard Deviation: 0.000096

Rank 141:
Hyperparameters: {'bootstrap': False, 'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5, 'n_estimators': 500}
Mean Test Score: -0.000792
Standard Deviation: 0.000102

```
[ ]: # Import necessary libraries for visualization
import seaborn as sns # Seaborn for advanced heatmap visualization
import matplotlib.pyplot as plt # Matplotlib for general plotting

# Create a Heatmap to Visualize Hyperparameter Tuning Results

# Pivot the GridSearchCV results DataFrame for heatmap visualization
# - 'values': The metric we are analyzing ('mean_test_score' - the average test_
↳ score across CV folds)
# - 'index': The hyperparameter to be placed on the y-axis (max_depth in this_
↳ case)
# - 'columns': The hyperparameter to be placed on the x-axis (n_estimators in_
↳ this case)
# - This transformation allows us to visualize how different combinations of_
↳ max_depth and n_estimators impact model performance
heatmap_data = cv_results.pivot_table(
```

```

    values='mean_test_score', # Performance metric to visualize
    index='param_max_depth', # max_depth values (rows)
    columns='param_n_estimators' # n_estimators values (columns)
)

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 6))

# Generate a heatmap using Seaborn
# - heatmap_data: The pivoted DataFrame containing the test scores
# - annot=True: Displays the numeric values in each cell for better analysis
# - fmt='.3f': Formats the numbers to three decimal places for precision
# - cmap='viridis': Uses the 'viridis' color map, which is visually appealing
#               ↪ and easy to interpret
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

# Set the title of the heatmap for context
plt.title('Effect of max_depth and n_estimators on Model Performance')

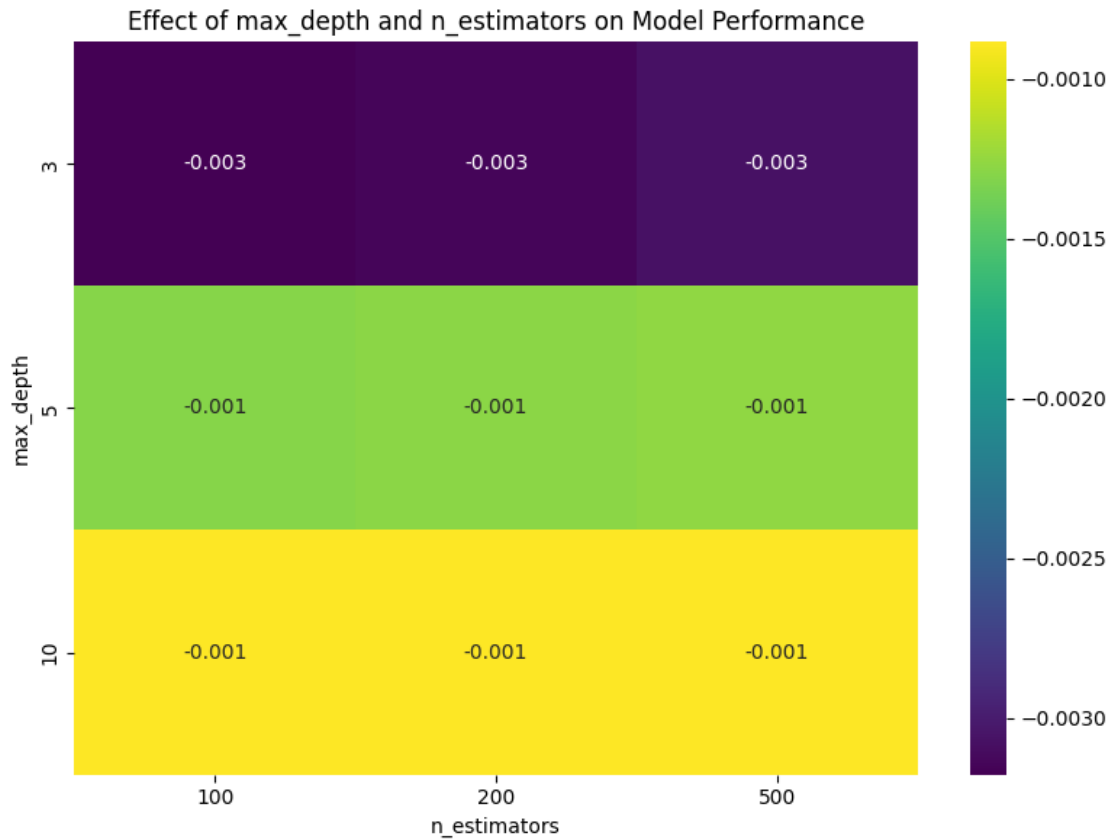
# Label the x-axis to indicate different n_estimators values
plt.xlabel('n_estimators')

# Label the y-axis to indicate different max_depth values
plt.ylabel('max_depth')

# Adjust layout to ensure proper spacing and prevent overlapping labels
plt.tight_layout()

# Display the heatmap
plt.show()

```

5.50 Effect of max_depth and n_estimators on Model Performance

5.50.1 Key Observations:

- **Lower RMSE with Higher max_depth:**
 - The model's performance improves (lower RMSE) as max_depth increases from **3 to 10**.
 - Shallower trees (max_depth = 3) have a higher RMSE, indicating underfitting.
- **Impact of n_estimators is Minimal:**
 - Increasing n_estimators from **100 to 500** does not significantly affect performance across different max_depth values.
 - This suggests that the model reaches **saturation** quickly, and additional estimators do not significantly improve accuracy.
- **Best Configuration:**
 - The best-performing configurations appear at **higher max_depth values (5 or 10)**.
 - These settings consistently produce the lowest RMSE values (-0.001), indicating the best balance of bias and variance.

5.50.2 Conclusion:

- **Deeper trees (higher max_depth) improve model accuracy**, reducing error.
- **Adding more estimators (n_estimators) has a negligible impact** on further improving

performance.

- **A balance must be maintained:** While deeper trees perform better, excessive depth may lead to overfitting in other cases.

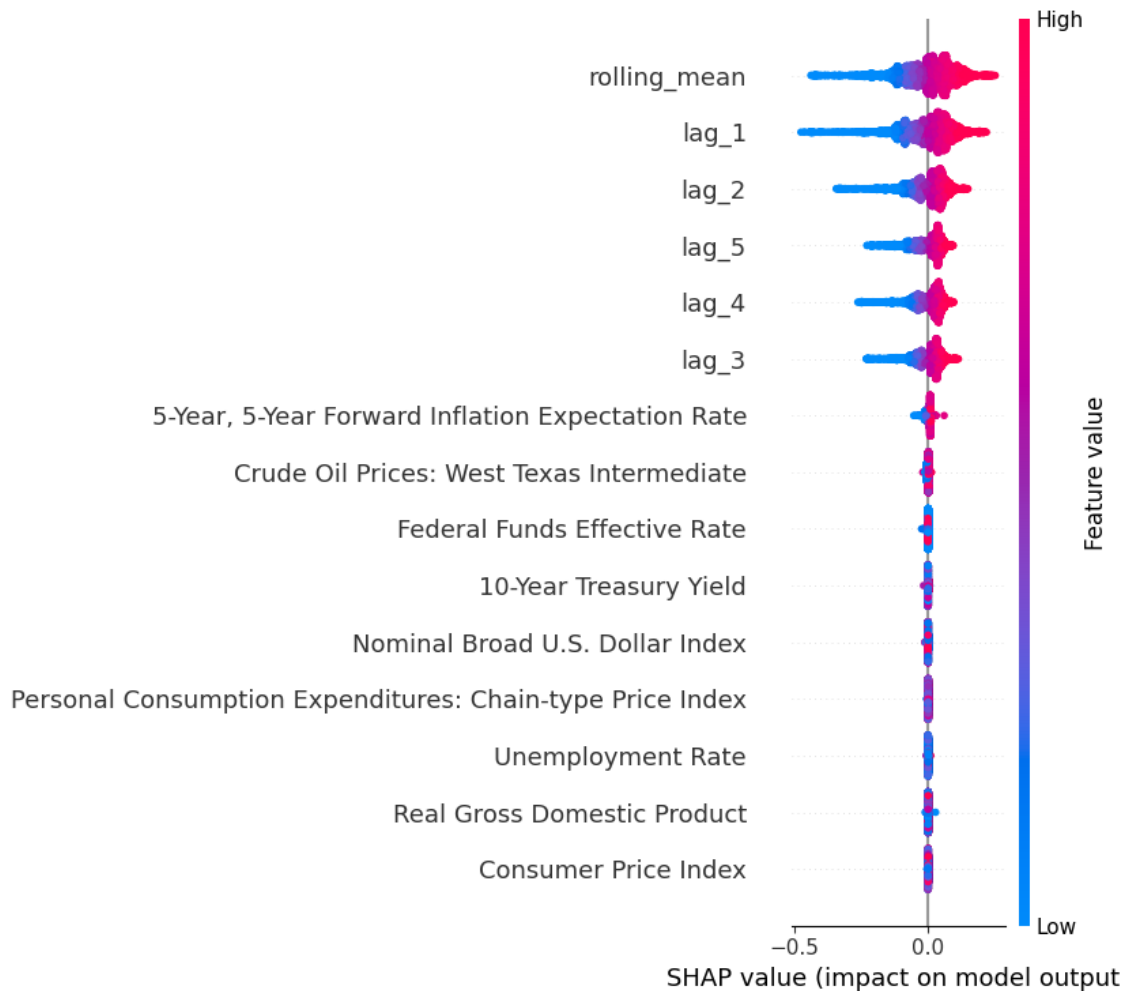
SHAP Explanations for ExtraTrees Regressor

```
[ ]: # Import necessary libraries for SHAP visualization and Matplotlib for plotting
import shap # SHAP library for model interpretability
import matplotlib.pyplot as plt # Matplotlib for general plotting

# Create a SHAP Explainer for the ExtraTrees Model
# - SHAP (SHapley Additive exPlanations) helps interpret machine learning
  ↳models.
# - 'TreeExplainer' is optimized for tree-based models like Extra Trees, Random
  ↳Forest, and Gradient Boosting.
# - 'best_et_model' is the trained Extra Trees model we want to explain.
explainer = shap.TreeExplainer(best_et_model)

# Compute SHAP values for the training dataset
# - SHAP values quantify how much each feature contributes to a model's
  ↳prediction.
# - This helps in understanding which features positively or negatively impact
  ↳model predictions.
shap_values = explainer.shap_values(X_train)

# Global Explanation: SHAP Summary Plot
# - Displays overall feature importance across all training samples.
# - Shows how each feature influences predictions using color-coded scatter
  ↳points.
# - Red = High feature values, Blue = Low feature values.
shap.summary_plot(shap_values, X_train)
```



5.51 SHAP Summary Plot: Feature Importance and Impact on Model Output

5.51.1 Key Observations:

- **Most Influential Features:**
 - The top three features contributing to the model's output are:
 1. rolling_mean
 2. lag_1
 3. lag_2
 - These features have the largest spread of SHAP values, indicating their strong influence on predictions.
- **Feature Importance Representation:**
 - Features are ranked in descending order based on their average absolute SHAP value.
 - The rolling_mean and lag_1 features have the highest impact on predictions.
- **SHAP Value Distribution:**
 - **Positive SHAP values (right side)** indicate an increase in the model's predicted value.

- Negative SHAP values (left side) indicate a decrease in the model’s predicted value.
- **Color Representation:**
 - Redder (high feature values) contribute more positively to the prediction.
 - Bluer (low feature values) contribute more negatively to the prediction.
- **Non-Lag Features Impact:**
 - While lag-based features (lag_1, lag_2, etc.) dominate, economic indicators such as:
 - * 5-Year, 5-Year Forward Inflation Expectation Rate
 - * Crude Oil Prices: West Texas Intermediate
 - * Federal Funds Effective Rate
 - These still play a role but with lower impact.

5.51.2 Conclusion:

- The model primarily relies on lagged values and moving averages to make predictions.
- Macroeconomic variables, while included, have a **relatively minor impact** compared to past values of the target variable.
- This analysis suggests that short-term historical trends are the most influential in shaping predictions.

```
[ ]: # Enable SHAP JavaScript-based visualization (for interactive plots in Jupyter
      ↪Notebooks)
shap.initjs()

# Local Explanation: SHAP Force Plot for a Single Instance
# - Explains how individual features contribute to a specific prediction.
# - explainer.expected_value: The base prediction (average model output).
# - shap_values[0]: SHAP values for the first instance in X_train.
# - X_train.iloc[0]: The actual feature values of the first instance.
shap.force_plot(explainer.expected_value, shap_values[0], X_train.iloc[0])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0ddf7ad390>
```

5.52 SHAP Waterfall Plot: Feature Contributions to a Single Prediction

5.52.1 Key Observations:

- **Base Value (2.056):**
 - This represents the starting prediction before specific feature contributions are considered.
- **Final Prediction (2.21):**
 - The final model prediction after accounting for feature impacts.
- **Positive Contributions:**
 - Features that **increase the prediction value** are shown in red.
 - The most impactful contributors are:
 - * 5-Year, 5-Year Forward Inflation Expectation Rate = 2.47
 - * lag_3 = 2.2
 - * lag_2 = 2.17

- * rolling_mean = 2.208
- * lag_4 = 2.23
- * lag_5 = 2.23
- * lag_1 = 2.2
- These features collectively push the prediction upward from **2.056** to **2.21**.
- **No Negative Contributions:**
 - Unlike some SHAP plots where certain features pull the prediction lower (shown in blue), this case shows that all contributing factors **increase** the final prediction.
- **Interpretation:**
 - The model relies heavily on **lagged values** (lag_1 to lag_5) and **rolling means**, suggesting that **past trends strongly influence the prediction**.
 - **Economic indicators**, such as the **5-Year Forward Inflation Expectation Rate**, also contribute significantly.

5.52.2 Conclusion:

- The **prediction of 2.21** is primarily driven by past values and macroeconomic factors.
- The **SHAP waterfall plot effectively breaks down how each feature moves the prediction from the base value to the final result**.
- This type of analysis is **useful for understanding individual predictions** and diagnosing model behavior on specific inputs.

Partial Dependence Plot (PDP) for ExtraTrees Regressor

```
[ ]: # Import necessary libraries for visualization and model inspection
import matplotlib.pyplot as plt # Import Matplotlib for plotting
from sklearn.inspection import PartialDependenceDisplay # Import PDP plotting
    ↪ tool
import warnings # Import warnings module to suppress unnecessary warnings

# Suppress FutureWarnings related to PartialDependenceDisplay
# - Some versions of scikit-learn may trigger FutureWarnings when using
    ↪ PartialDependenceDisplay.
# - This prevents unnecessary warning messages from appearing.
warnings.filterwarnings("ignore", category=FutureWarning)

# Extract feature names from the training dataset
# - Ensures that all features from X_train are used in the Partial Dependence
    ↪ Plots.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDP) for all features using the trained
    ↪ ExtraTrees Model
# - PDP helps visualize how a specific feature impacts the model's predictions.
# - kind='average': Computes the average partial dependence over all
    ↪ instances.
# - grid_resolution=50: Number of points on the x-axis (higher values =
    ↪ smoother curves).
```

```

# - n_cols=4: Arranges the plots into 4 columns for better visualization.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_et_model, # The trained ExtraTrees model
    X_train, # The dataset used to generate PDP plots
    features=features_list, # List of feature names to analyze
    kind='average', # Compute average partial dependence values
    grid_resolution=50, # Defines the granularity of the x-axis (smoother
    ↪curve with more points)
    n_cols=4 # Arranges PDP plots into 4 columns for better layout
)

# Customize the PDP Figure for Better Readability

# Set the overall figure size for better visualization
pdp_display.figure_.set_size_inches(18, 12)

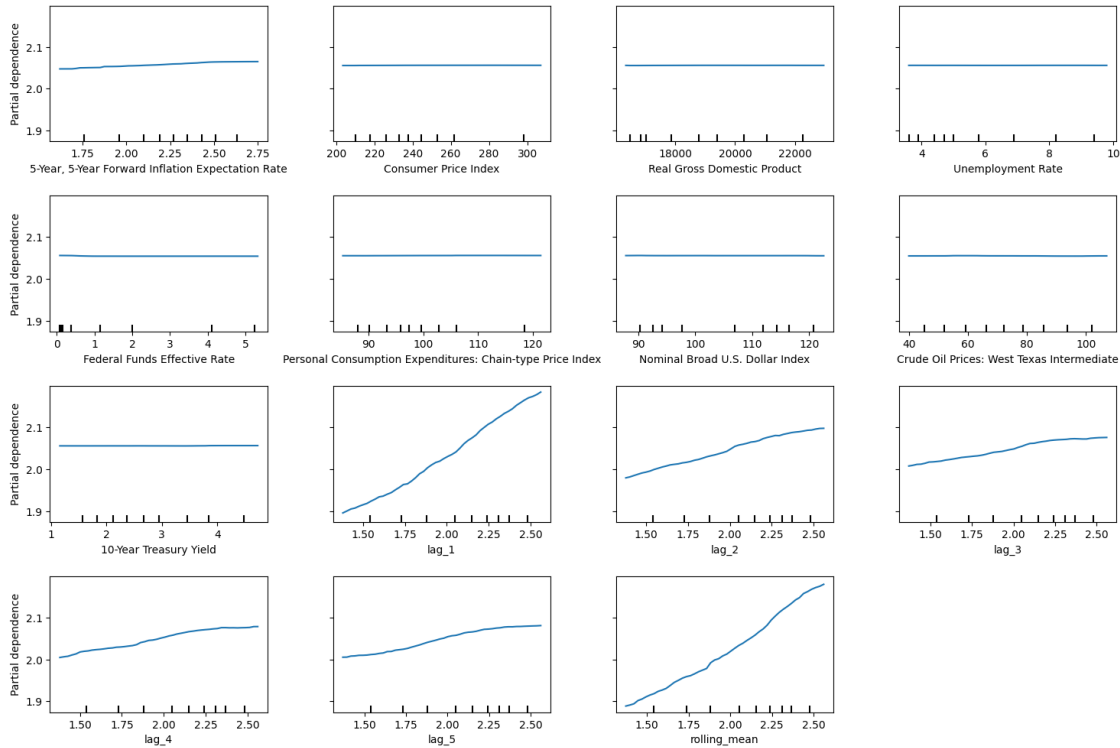
# Add a title to the entire figure, positioned above all subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (ExtraTrees)",
    ↪fontsize=16, y=1.06)

# Adjust subplot spacing to prevent overlapping titles and labels
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the Partial Dependence Plots
plt.show()

```

Partial Dependence Plots (ExtraTrees)



5.53 Partial Dependence Plots (ExtraTrees)

5.53.1 Key Observations:

- **Understanding Partial Dependence Plots (PDPs):**
 - These plots illustrate the **marginal effect of individual features** on the model's predictions while keeping other features constant.
- **Features with Strong Positive Relationships:**
 - lag_1, lag_2, rolling_mean, lag_3, lag_4:
 - * These features show a **strong upward trend**, indicating that higher values lead to **higher predictions**.
 - * Suggests that **recent past values significantly influence future predictions**.
- **Features with Minimal or No Impact:**
 - Consumer Price Index, Real Gross Domestic Product, Unemployment Rate, Federal Funds Effective Rate, Crude Oil Prices: West Texas Intermediate:
 - * The flat lines suggest **no significant relationship** with the model's predictions.
 - * These macroeconomic factors seem **less influential** compared to lag features.
- **5-Year, 5-Year Forward Inflation Expectation Rate:**
 - Shows a **slight upward trend**, indicating a minor positive influence on predictions.

- **Interpretation:**

- The model is **highly dependent on lagged features** (lag_1, lag_2, rolling_mean), meaning historical data plays a critical role in forecasting.
- Macroeconomic indicators **do not have a strong influence** in this model.
- **Rolling mean** shows a significant upward effect, indicating that smoothed past values strongly contribute to future predictions.

5.53.2 Conclusion:

- **ExtraTrees regression model** relies predominantly on **time-series lag features** rather than macroeconomic indicators.
- **Predictive power is largely driven by recent trends** rather than external economic factors.
- This insight helps in **feature selection and model tuning** to optimize performance.

Permutation Feature Importance

```
[ ]: # Import necessary libraries for feature importance analysis and visualization
from sklearn.inspection import permutation_importance # Import permutation_
    ↪ importance from sklearn
import matplotlib.pyplot as plt # Import Matplotlib for visualization
import pandas as pd # Import Pandas for handling tabular data

# Compute Permutation Feature Importance on the Training Set
# - Permutation importance measures the effect of **randomly shuffling a_
    ↪ feature's values** on the model's performance.
# - It provides a more reliable estimate of feature importance compared to_
    ↪ built-in importance methods.
# - `n_repeats=10`: The number of times each feature is randomly shuffled to_
    ↪ compute its importance.
# - `random_state=42`: Ensures reproducibility of results.
# - `n_jobs=-1`: Uses all available CPU cores for faster computation.
perm_importance = permutation_importance(
    best_et_model, # The trained ExtraTrees model
    X_train, # Training dataset (features)
    y_train, # Training dataset (target variable)
    n_repeats=10, # Number of times to shuffle each feature
    random_state=42, # Ensures consistent results across runs
    n_jobs=-1 # Uses all CPU cores for parallel processing
)

# Convert Permutation Importance Results into a Pandas DataFrame
# - 'Feature': Column storing feature names.
# - 'Importance': The mean importance score from the permutation process.
# - The DataFrame is sorted in descending order to highlight the most important_
    ↪ features first.
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Extracts feature names from training data
```



```

    'Importance': perm_importance.importances_mean # Mean importance scores
    ↪ across repetitions
}).sort_values(by='Importance', ascending=False) # Sort features by importance

# Visualization: Permutation Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display permutation importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars for better visibility
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
    ↪ color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Permutation Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

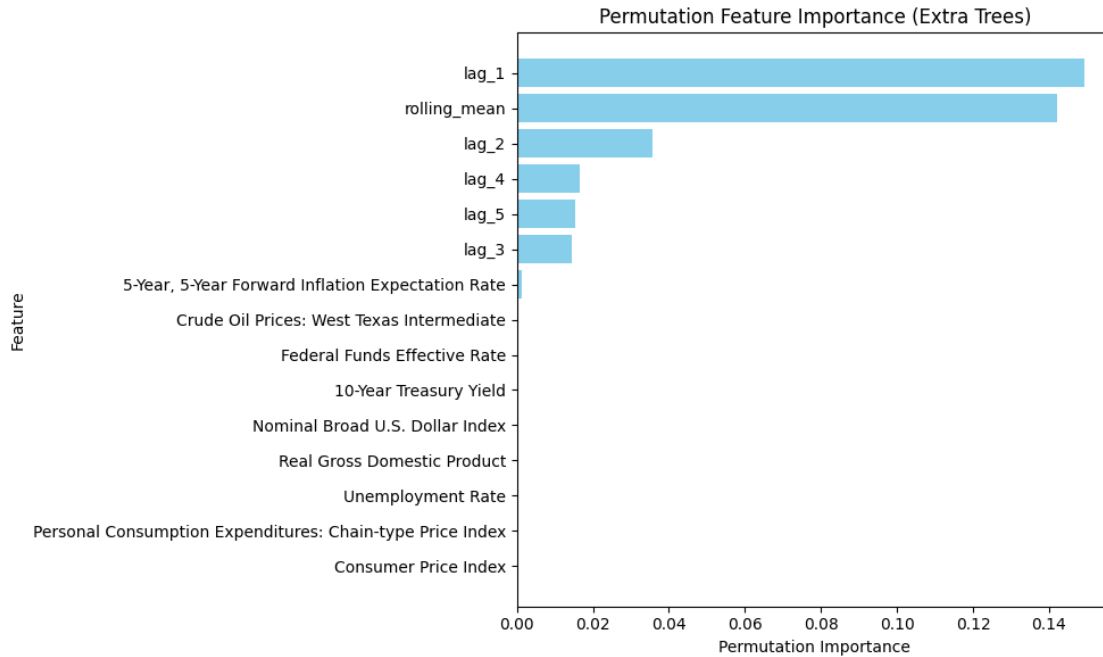
# Set the title of the plot for context
plt.title('Permutation Feature Importance (Extra Trees)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()

```



5.54 Permutation Feature Importance (Extra Trees)

5.54.1 Key Observations:

- **Understanding Permutation Importance:**
 - This plot shows the **drop in model performance** when each feature's values are shuffled.
 - A **higher importance score** means the feature is **more critical** to the model's predictions.
- **Most Important Features:**
 - lag_1 and rolling_mean are the **most significant features**, indicating that **recent historical values and moving averages** play a crucial role in predictions.
 - lag_2 also holds notable importance, reinforcing the idea that short-term lagged values are essential.
- **Moderately Important Features:**
 - lag_3, lag_4, and lag_5 show lower but still meaningful importance, suggesting that **longer-term past values still contribute to the prediction**.
- **Least Important Features:**
 - **Macroeconomic indicators** such as:
 - * 5-Year, 5-Year Forward Inflation Expectation Rate
 - * Crude Oil Prices: West Texas Intermediate
 - * Federal Funds Effective Rate
 - * 10-Year Treasury Yield
 - * Real Gross Domestic Product
 - * Unemployment Rate
 - * Consumer Price Index

- These have almost **zero permutation importance**, indicating they **do not significantly impact predictions**.

5.54.2 Interpretation:

- The **Extra Trees model is highly dependent on time-series lag features**, particularly short-term (lag_1, lag_2) and smoothed trends (rolling_mean).
- **Macroeconomic indicators have negligible importance**, meaning the model does not rely on external economic data for predictions.
- The feature importance rankings provide guidance for **feature selection**, potentially allowing the removal of less impactful variables to improve model efficiency.

5.54.3 Conclusion:

- **Lagged values and rolling averages are the primary drivers** of the model's predictive power.
- **Macroeconomic variables do not contribute significantly**, and their removal may not affect performance.
- This analysis helps in **refining feature selection and improving model interpretability**.

5.55 Histogram-Based Gradient Boosting Regressor (HGBR)

```
[ ]: # Import necessary libraries
import random # For setting random seed
import numpy as np # For numerical computations
import pandas as pd # For handling tabular data
from sklearn.model_selection import train_test_split, GridSearchCV # For data_
    ↪splitting and hyperparameter tuning
from sklearn.ensemble import HistGradientBoostingRegressor # Import the_
    ↪Histogram-Based Gradient Boosting model

# Set a global random seed for reproducibility
# - Ensures that the results remain consistent across different runs.
random.seed(42)
np.random.seed(42)

# Split the dataset into training and testing sets (80% training, 20% testing)
# - X: Feature matrix
# - y: Target variable
# - test_size=0.2: Reserves 20% of the data for testing
# - random_state=42: Ensures consistent data splits across runs
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)

# Initialize the Histogram-Based Gradient Boosting Regressor (HGBR) model
# - This model is an optimized gradient boosting algorithm that uses histograms_
    ↪for faster computation.
# - It is particularly effective for large datasets.
```

```

# - random_state=42 ensures reproducibility.
hgb_model = HistGradientBoostingRegressor(random_state=42)

# Define the hyperparameter grid for tuning the Histogram-Based Gradient
↳ Boosting model
# - 'max_iter': Number of boosting iterations (more iterations = better
↳ learning but may lead to overfitting)
# - 'learning_rate': Controls step size at each iteration (lower values = more
↳ stable training)
# - 'max_depth': Maximum depth of each tree (higher values allow for more
↳ complex trees)
# - 'min_samples_leaf': Minimum samples required at each leaf node (higher
↳ values reduce overfitting)
# - 'max_bins': Number of bins for feature discretization (affects speed and
↳ accuracy)
param_grid = {
    'max_iter': [100, 200, 300],          # Number of boosting iterations
    'learning_rate': [0.01, 0.05, 0.1],    # Step size for weight updates
    'max_depth': [3, 5, 7],                # Maximum depth of trees
    'min_samples_leaf': [10, 20, 50],      # Minimum samples per leaf node
    'max_bins': [255, 128, 100]           # Valid values for max_bins
    ↳ (affects feature discretization)
}

# Initialize GridSearchCV to perform hyperparameter tuning
# - GridSearchCV exhaustively searches through all hyperparameter combinations.
# - estimator=hgb_model: Uses the Histogram-Based Gradient Boosting model.
# - param_grid=param_grid: Defines the hyperparameter search space.
# - scoring='neg_mean_squared_error': Uses negative MSE as the evaluation
↳ metric (lower is better).
# - cv=5: Uses 5-fold cross-validation for robust model evaluation.
# - verbose=1: Displays the search progress.
# - n_jobs=-1: Utilizes all CPU cores for faster processing.
grid_search = GridSearchCV(
    estimator=hgb_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Fit the GridSearchCV model on the training data
# - Evaluates different hyperparameter combinations to find the best-performing
↳ model.
grid_search.fit(X_train, y_train)

```

```

# Retrieve and print the best hyperparameters found by GridSearchCV
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Retrieve the best Histogram-Based Gradient Boosting model from GridSearchCV
# - This model is trained using the optimal hyperparameter combination.
best_hgb_model = grid_search.best_estimator_

# Make predictions on the test dataset using the best model
hgb_pred = best_hgb_model.predict(X_test)

# Evaluate the model using a predefined function
# - `calculate_metrics`: Assumed to be a function that calculates evaluation_
↳ metrics.
# - `naive_forecast`: Assumed to be a baseline prediction for comparison.
# - The evaluation results are appended to `metrics_results`.
metrics_results.append(
    calculate_metrics(y_test.values, hgb_pred, naive_forecast, "Histogram-Based_
↳ Gradient Boosting Regressor (HGBR)")
)

# Print the evaluation results for the Histogram-Based Gradient Boosting model
print("HGBR Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 243 candidates, totalling 1215 fits
 Best Hyperparameters: {'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7, 'max_iter': 300, 'min_samples_leaf': 10}
 HGBR Evaluation Metrics:
 {'Model': 'Histogram-Based Gradient Boosting Regressor (HGBR)', 'MSE': 0.0005834295749823299, 'RMSE': 0.024154286886230565, 'MAE': 0.01484038473999798, 'MAPE (%)': 0.9645516085929583, 'sMAPE (%)': 0.9656255050663115, 'MASE': 0.03777702995362074, 'R^2': 0.9967005057323347}

Actual vs. Predicted Values

```

[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 8))

# Scatter Plot: Compare Actual vs. Predicted Values
# - x-axis: Actual values from the test dataset (y_test)
# - y-axis: Predictions made by the Histogram-Based Gradient Boosting model_
↳ (hgb_pred)

```

```

# - alpha=0.5: Makes points semi-transparent to reduce overlap and improve
↳visibility
# - color='blue': Sets the color of the scatter points
# - label='Predictions': Legend label for scatter points
plt.scatter(y_test, hgb_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the Perfect Fit Line (Reference Line)
# - This diagonal line represents the ideal case where predictions perfectly
↳match actual values.
# - If the model were perfect, all points would lie exactly on this line.
# - color='red': Sets the reference line color to red
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the line thickness
# - label='Perfect Fit': Legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')

# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (HistGradientBoostingRegressor)')

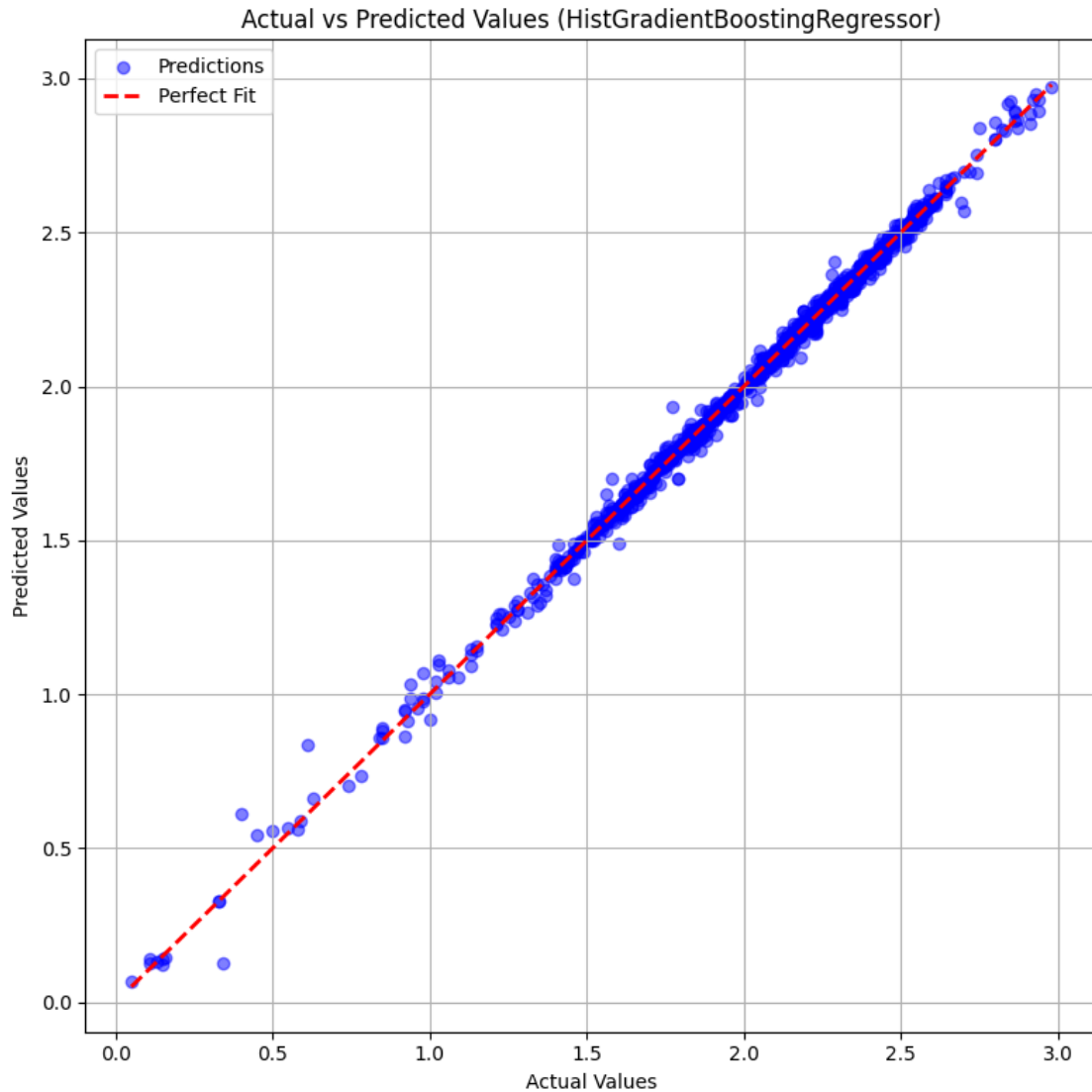
# Add a legend to differentiate the scatter points (Predictions) from the
↳reference line (Perfect Fit)
plt.legend()

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()

```



5.56 Actual vs Predicted Values (HistGradientBoostingRegressor)

5.56.1 Key Observations:

- **Plot Interpretation:**
 - The scatter plot compares **actual values** (x-axis) with **predicted values** (y-axis).
 - The **red dashed line** represents the **perfect fit** (i.e., when actual = predicted).
 - Each **blue dot** represents a single prediction.
- **Model Performance:**
 - The data points are **closely aligned** along the red dashed line, indicating that the **model predictions are highly accurate**.
 - **Minimal deviation** from the line suggests that the model does not suffer from significant bias or variance.
 - There are **some minor outliers**, particularly at lower values, where predictions show

slight deviations.

- **Error Analysis:**
 - The spread of points around the perfect fit line is **small**, indicating **low residual errors**.
 - Outliers and slight deviations may indicate areas where the model's performance is **less accurate**.

5.56.2 Interpretation:

- The **HistGradientBoostingRegressor** model demonstrates **strong predictive power**, as seen by the close alignment of predictions with actual values.
- The presence of **some outliers** suggests possible areas for model fine-tuning, such as feature engineering or hyperparameter optimization.

5.56.3 Conclusion:

- **The model performs well, with low error and high predictive accuracy.**
- **Further improvements** could be achieved by investigating residual patterns and refining feature selection or hyperparameters.

Residual Analysis

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Compute Residuals (Errors in Predictions)
# - Residuals = Actual Values - Predicted Values
# - Measures the difference between true and predicted values
residuals = y_test - hgb_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values (y_test)
# - y-axis: Residuals (y_test - hgb_pred)
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve
↳visibility
# - color='green': Sets the color of the residual points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a horizontal reference line at y=0 to indicate zero residuals
# - Helps in assessing whether residuals are symmetrically distributed around
↳zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
```



```
plt.xlabel('Actual Values')

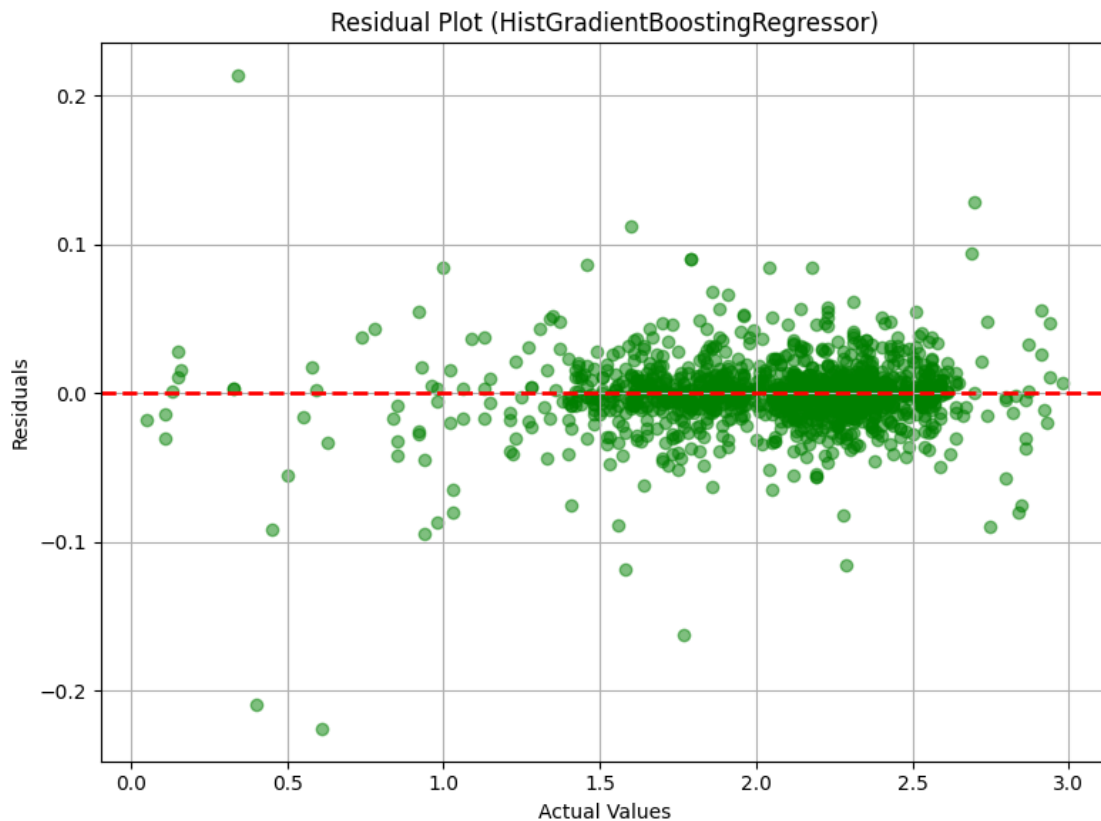
# Label the y-axis to indicate residuals (errors in predictions)
plt.ylabel('Residuals')

# Set a title for the plot to provide context
plt.title('Residual Plot (HistGradientBoostingRegressor)')

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual scatter plot
plt.show()
```



5.57 Residual Plot (HistGradientBoostingRegressor)

5.57.1 Key Observations:

- **Plot Interpretation:**
 - The scatter plot visualizes **residuals** (y-axis) against **actual values** (x-axis).
 - The **red dashed line at $y = 0$** represents the ideal case where residuals are zero, meaning perfect predictions.
 - Each **green dot** represents the residual (error) for a single prediction.
- **Model Performance:**
 - **Most residuals are centered around zero**, indicating that the model **does not have a systematic bias**.
 - There is **no clear pattern**, suggesting that the model captures the relationships in the data well.
 - The **spread of residuals is relatively small**, meaning the model has **low variance** and is making consistent predictions.
- **Error Analysis:**
 - Some **outliers** (points with higher residuals) indicate **instances where the model made relatively larger errors**.
 - The **variance of residuals remains fairly constant** across different actual values, which suggests **homoscedasticity** (good for model reliability).

5.57.2 Interpretation:

- The **HistGradientBoostingRegressor** model appears to perform **well**, with **most residuals near zero** and no significant pattern of errors.
- The presence of some **outliers** suggests that certain data points may be harder to predict, possibly due to noise in the dataset or missing features.

5.57.3 Conclusion:

- **The residuals are well-distributed**, indicating a **good model fit**.
- **Further model tuning** (e.g., handling outliers, feature engineering) might help reduce the spread of residuals and improve overall performance.

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual and predicted values
# - bins=30: Divides the range of residuals into 30 bins for a detailed view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)
```

```

# Add a vertical reference line at x=0 to indicate no residual error
# - Helps assess whether residuals are symmetrically distributed around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

# Set a title for the plot to provide context
plt.title('Residual Distribution (HistGradientBoostingRegressor)')

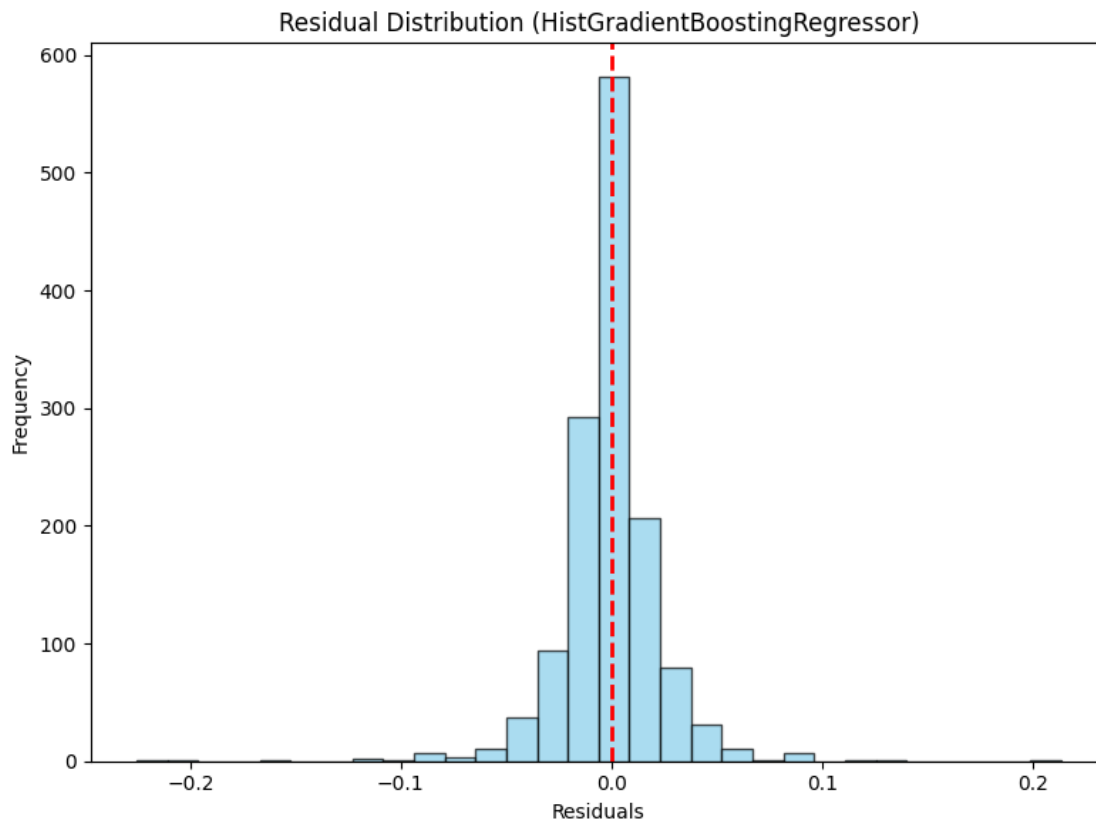
# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual histogram plot
plt.show()

```



5.58 Residual Distribution (HistGradientBoostingRegressor)

5.58.1 Key Observations:

- **Histogram Interpretation:**
 - The histogram represents the **distribution of residuals** (differences between actual and predicted values).
 - The **x-axis** represents residual values, while the **y-axis** represents their frequency.
 - The **red dashed line at zero** represents the ideal case where residuals are centered around zero.
- **Model Performance:**
 - The **majority of residuals are clustered around zero**, indicating **accurate predictions**.
 - The distribution is **approximately symmetric**, suggesting **no systematic bias** in the model's predictions.
 - The **spread of residuals is small**, meaning **low variance** in the model's predictions.
- **Error Analysis:**
 - The **absence of long tails or extreme outliers** indicates that the model **does not frequently make large errors**.
 - The residuals form a **bell-shaped curve**, resembling a **normal distribution**, which is a sign of a well-fitted model.

5.58.2 Interpretation:

- The **HistGradientBoostingRegressor** appears to be making **consistent and accurate predictions**.
- The **residuals being centered around zero** and following a normal distribution indicate **good model calibration**.
- **Further improvements** could involve refining hyperparameters or feature selection to further reduce small errors.

5.58.3 Conclusion:

- **The model performs well**, with residuals tightly centered around zero.
- The histogram suggests **no major systematic errors or bias**, making the model suitable for reliable predictions.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Import Pandas to manipulate DataFrame

# Convert GridSearchCV results into a DataFrame
# - 'cv_results_' is a dictionary containing all cross-validation results from
  ↪ GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on the best model performance (lower
  ↪ is better)
```

```

sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
↳ head(5)

# Improve Output Visibility by Formatting Results Nicely
print("\nTop 5 Hyperparameter Combinations:\n")
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"  Hyperparameters: {row['params']}")
    print(f"  Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"  Standard Deviation: {row['std_test_score']:.6f}")
    print(f"-" * 70) # Separator for better readability

```

Top 5 Hyperparameter Combinations:

Rank 187:

```

Hyperparameters: {'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7,
'max_iter': 300, 'min_samples_leaf': 10}
Mean Test Score: -0.000682
Standard Deviation: 0.000105

```

Rank 188:

```

Hyperparameters: {'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7,
'max_iter': 300, 'min_samples_leaf': 20}
Mean Test Score: -0.000703
Standard Deviation: 0.000078

```

Rank 184:

```

Hyperparameters: {'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 7,
'max_iter': 200, 'min_samples_leaf': 10}
Mean Test Score: -0.000706
Standard Deviation: 0.000104

```

Rank 178:

```

Hyperparameters: {'learning_rate': 0.1, 'max_bins': 255, 'max_depth': 5,
'max_iter': 300, 'min_samples_leaf': 10}
Mean Test Score: -0.000714
Standard Deviation: 0.000082

```

Rank 241:

Hyperparameters: {'learning_rate': 0.1, 'max_bins': 100, 'max_depth': 7, 'max_iter': 300, 'min_samples_leaf': 10}

Mean Test Score: -0.000716

Standard Deviation: 0.000107

```
[ ]: # Import necessary libraries for visualization
import seaborn as sns # Import Seaborn for creating a heatmap
import matplotlib.pyplot as plt # Import Matplotlib for general plotting

# Create a Heatmap to Visualize Hyperparameter Tuning Results

# Pivot the GridSearchCV results DataFrame for heatmap visualization
# - 'values': The metric we are analyzing ('mean_test_score' - the average test
    ↳ score across CV folds)
# - 'index': The hyperparameter to be placed on the y-axis (max_depth in this
    ↳ case)
# - 'columns': The hyperparameter to be placed on the x-axis (learning_rate in
    ↳ this case)
# - This transformation allows us to visualize how different combinations of
    ↳ max_depth and learning_rate impact model performance.
heatmap_data = cv_results.pivot_table(
    values='mean_test_score', # Performance metric to visualize
    index='param_max_depth', # max_depth values (rows)
    columns='param_learning_rate' # learning_rate values (columns)
)

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 6))

# Generate a heatmap using Seaborn
# - heatmap_data: The pivoted DataFrame containing the test scores
# - annot=True: Displays the numeric values in each cell for better analysis
# - fmt='.3f': Formats the numbers to three decimal places for precision
# - cmap='viridis': Uses the 'viridis' color map, which is visually appealing
    ↳ and easy to interpret
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

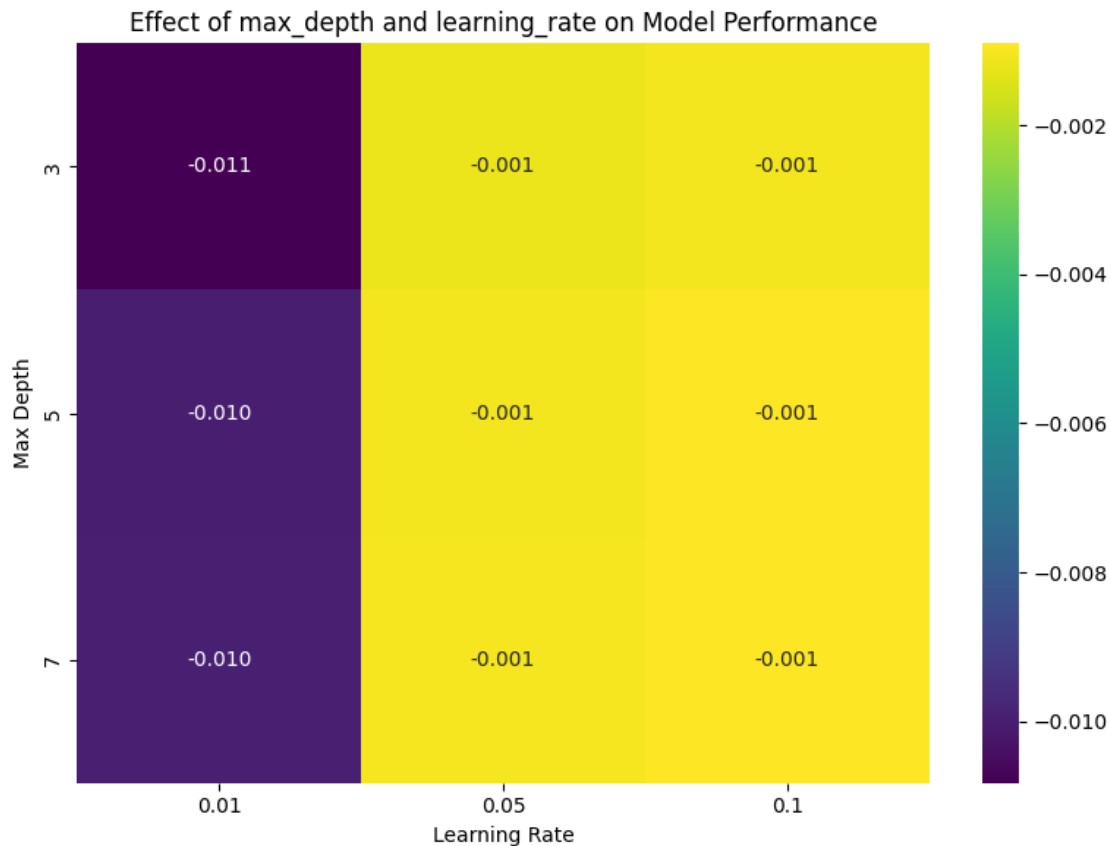
# Set the title of the heatmap for context
plt.title('Effect of max_depth and learning_rate on Model Performance')

# Label the x-axis to indicate different learning_rate values
plt.xlabel('Learning Rate')

# Label the y-axis to indicate different max_depth values
plt.ylabel('Max Depth')
```

```
# Adjust layout to ensure proper spacing and prevent overlapping labels
plt.tight_layout()

# Display the heatmap
plt.show()
```



5.59 Effect of max_depth and learning_rate on Model Performance

5.59.1 Key Observations:

- **Heatmap Interpretation:**
 - The heatmap visualizes the **interaction between max_depth and learning_rate** on model performance.
 - The **x-axis represents the learning rate**, and the **y-axis represents the max depth**.
 - The **color scale indicates the model error**, where **darker shades represent higher error** and **lighter shades represent lower error**.
- **Impact of Learning Rate:**
 - At a **low learning rate (0.01)**, model error is relatively high (darker shade).

- Increasing the learning rate to **0.05 or 0.1** significantly reduces error, as shown by the **lighter shades**.
- **Optimal learning rate** appears to be **0.05 or 0.1**, where model performance is best.
- **Impact of Max Depth:**
 - Increasing `max_depth` from **3 to 7** has **minimal impact on error** at **higher learning rates**.
 - However, at **lower learning rates (0.01)**, a shallower tree (`max_depth = 3`) results in the **highest error**.
 - This suggests that **deeper trees are more beneficial when learning rates are low**.
- **Best Configuration:**
 - The **lowest error (best performance)** is achieved at a **learning rate of 0.05 or 0.1**, regardless of `max_depth`.
 - The **worst performance** occurs at **low learning rates with shallow trees** (`max_depth = 3`).

5.59.2 Interpretation:

- A **low learning rate (0.01)** leads to **higher error**, especially when combined with shallow trees.
- **Optimal performance** is observed at **higher learning rates (0.05 or 0.1)** across all `max_depth` values.
- **Increasing max depth alone does not significantly improve performance** unless paired with an appropriate learning rate.

5.59.3 Conclusion:

- A **learning rate of 0.05 or 0.1** is recommended for the best model performance.
- **Avoid using a low learning rate (0.01) with shallow trees** (`max_depth = 3`) to prevent high error.
- **Further fine-tuning** of `n_estimators` and other hyperparameters may further optimize model accuracy.

SHAP Explanations for HistGradientBoostingRegressor

```
[ ]: # Import necessary libraries for SHAP visualization and Matplotlib for plotting
import shap # SHAP library for model interpretability
import matplotlib.pyplot as plt # Matplotlib for general plotting

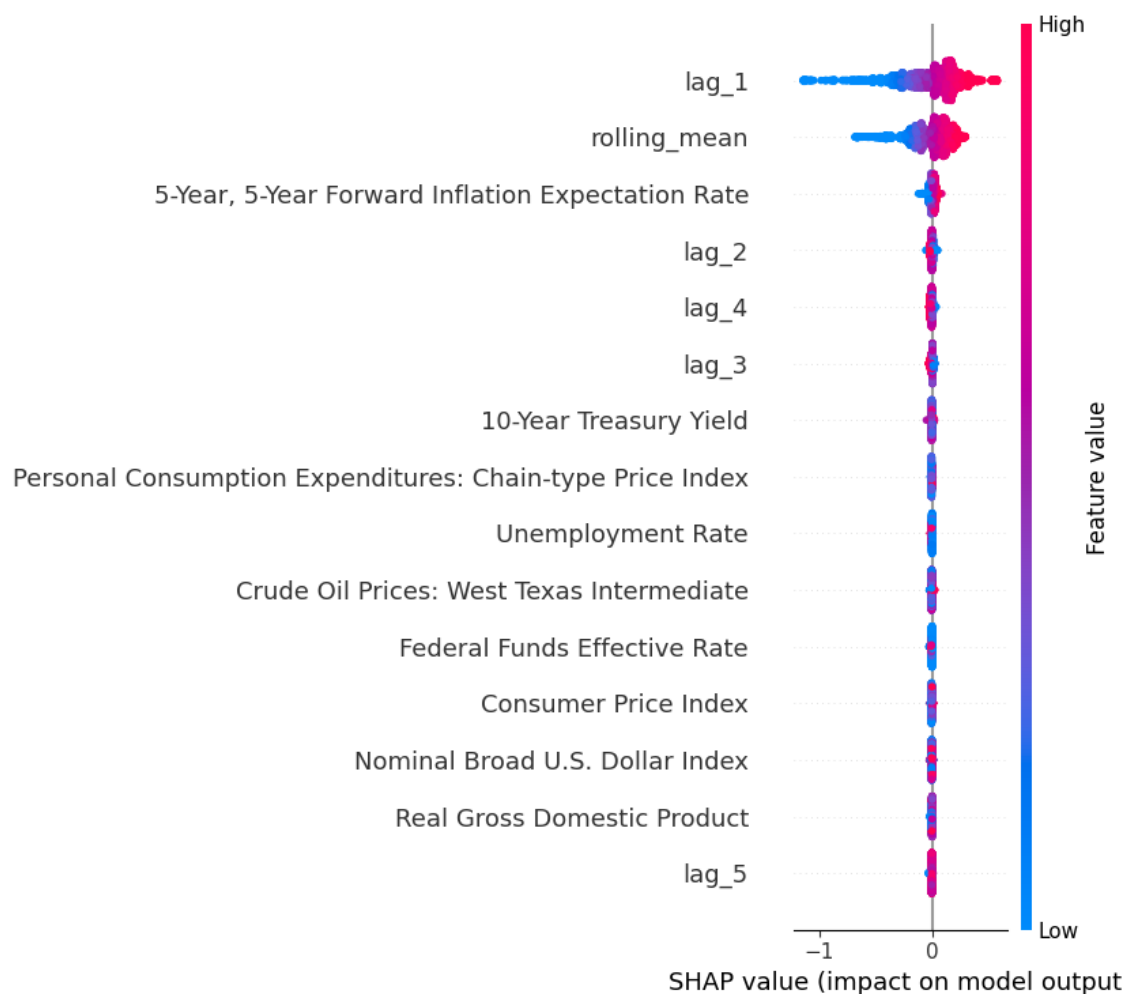
# Create a SHAP Explainer for the Histogram-Based Gradient Boosting Regressor
# ↪ (HGBR) model
# - SHAP (SHapley Additive exPlanations) helps interpret machine learning
# ↪ models.
# - 'TreeExplainer' is optimized for tree-based models like HGBR, Extra Trees,
# ↪ and Gradient Boosting.
# - 'best_hgb_model' is the trained HGBR model we want to explain.
explainer = shap.TreeExplainer(best_hgb_model)

# Compute SHAP values for the training dataset
```



```
# - SHAP values quantify how much each feature contributes to a model's
  ↳ prediction.
# - This helps in understanding which features positively or negatively impact
  ↳ model predictions.
shap_values = explainer.shap_values(X_train)

# Global Explanation: SHAP Summary Plot
# - Displays overall feature importance across all training samples.
# - Shows how each feature influences predictions using color-coded scatter
  ↳ points.
# - Red = High feature values, Blue = Low feature values.
shap.summary_plot(shap_values, X_train)
```



5.60 SHAP Summary Plot: Feature Contributions to Model Predictions

5.60.1 Key Observations:

- **SHAP (SHapley Additive exPlanations) values** measure each feature's impact on the model's output.
- **X-axis:** SHAP values (how much each feature pushes the prediction higher or lower).
- **Y-axis:** Features ranked by importance, with the most impactful features at the top.
- **Color Gradient:**
 - **Red (High feature values):** Indicates that a high value for this feature **increases the prediction**.
 - **Blue (Low feature values):** Indicates that a low value for this feature **decreases the prediction**.

5.60.2 Feature Importance:

1. **lag_1:**
 - The most important feature influencing predictions.
 - High values (red) generally increase predictions, while low values (blue) decrease them.
2. **rolling_mean:**
 - Another highly impactful feature.
 - Higher values tend to increase predictions.
3. **5-Year, 5-Year Forward Inflation Expectation Rate:**
 - Important economic indicator contributing significantly to predictions.
4. **Lags (lag_2, lag_3, lag_4, lag_5):**
 - Past values of the target variable impact future predictions.
 - Higher past values (red) generally push predictions higher.
5. **Macroeconomic Indicators:**
 - 10-Year Treasury Yield, Personal Consumption Expenditures, Unemployment Rate, Crude Oil Prices, Federal Funds Effective Rate, and Real GDP have lower impacts but still contribute to model outputs.

5.60.3 Interpretation:

- Time series lag features (lag_1, lag_2, rolling_mean) dominate the model predictions.
- Macroeconomic variables have less influence but still play a role in refining predictions.
- Higher values of key features generally push predictions higher, while lower values reduce predictions.

5.60.4 Conclusion:

- The model relies primarily on past values (lag_1, lag_2, rolling_mean) to make predictions.
- Economic indicators contribute but do not dominate predictions.
- Feature interactions could be further explored to refine the model.

```
[ ]: # Initialize SHAP JavaScript-based visualization (for interactive plots in
      ↪Jupyter Notebooks)
      shap.initjs()

      # Local Explanation: SHAP Force Plot for a Single Instance
      # - Explains how individual features contribute to a specific prediction.
      # - explainer.expected_value: The base prediction (average model output).
      # - shap_values[0]: SHAP values for the first instance in X_train.
      # - X_train.iloc[0]: The actual feature values of the first instance.
      shap.force_plot(explainer.expected_value, shap_values[0], X_train.iloc[0])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0ddf3dd390>
```

5.61 SHAP Force Plot: Feature Contributions to a Single Prediction

5.61.1 Key Observations:

- This **SHAP force plot** explains how different features **pushed the model's prediction** higher or lower for a specific instance.
- **X-axis:** The final prediction value ($f(x) = 2.23$).
- **Base Value:** The model's expected prediction before applying feature effects (2.056).
- **Feature Contributions:**
 - **Features pushing the prediction higher (red):**
 - * **lag_1 = 2.2:** Strongest positive impact.
 - * **rolling_mean = 2.208:** Significant influence in raising the prediction.
 - * **5-Year, 5-Year Forward Inflation Expectation Rate = 2.47:** Contributes positively.
 - * **Crude Oil Prices: West Texas Intermediate = 106.7:** Also increases the prediction.
 - * **10-Year Treasury Yield = 2.03:** Smaller but noticeable positive contribution.
 - **Features pushing the prediction lower (blue):**
 - * Minimal effect, suggesting all key variables in this instance increased the prediction.

5.61.2 Interpretation:

- The final prediction (**2.23**) is higher than the base value (**2.056**) because multiple key features exerted an upward influence.
- **Time-series features** (**lag_1**, **rolling_mean**) dominate the prediction, indicating strong temporal dependencies.
- **Macroeconomic indicators** (**Inflation Expectation Rate**, **Oil Prices**, **Treasury Yield**) also played a role but had less impact compared to time-series features.
- **No major negative contributors**, meaning all important features in this instance aligned to push the prediction upwards.

5.61.3 Conclusion:

- The model relies primarily on past values (lag_1, rolling_mean) but also considers macroeconomic factors.
- This instance demonstrates how key economic indicators interact with time-series data to determine the final prediction.
- Further analysis could explore feature interactions and their relative importance across multiple instances.

Partial Dependence Plot (PDP) for HistGradientBoostingRegressor

```
[ ]: # Import necessary libraries for visualization and model inspection
import matplotlib.pyplot as plt # Import Matplotlib for plotting
from sklearn.inspection import PartialDependenceDisplay # Import PDP plotting tool
import warnings # Import warnings module to suppress unnecessary warnings

# Suppress FutureWarnings related to PartialDependenceDisplay
# - Some versions of scikit-learn may trigger FutureWarnings when using PartialDependenceDisplay.
# - This prevents unnecessary warning messages from appearing.
warnings.filterwarnings("ignore", category=FutureWarning)

# Extract feature names from the training dataset
# - Ensures that all features from X_train are used in the Partial Dependence Plots.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDP) for all features using the trained Histogram-Based Gradient Boosting Regressor (HGBR)
# - PDP helps visualize how a specific feature impacts the model's predictions.
# - kind='average': Computes the **average** partial dependence over all instances.
# - grid_resolution=50: Number of points on the x-axis (higher values = smoother curves).
# - n_cols=4: Arranges the plots into 4 columns for better visualization.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_hgb_model, # The trained HGBR model
    X_train, # The dataset used to generate PDP plots
    features=features_list, # List of feature names to analyze
    kind='average', # Compute average partial dependence values
    grid_resolution=50, # Defines the granularity of the x-axis (smoother curve with more points)
    n_cols=4 # Arranges PDP plots into 4 columns for better layout
)

# Customize the PDP Figure for Better Readability
```

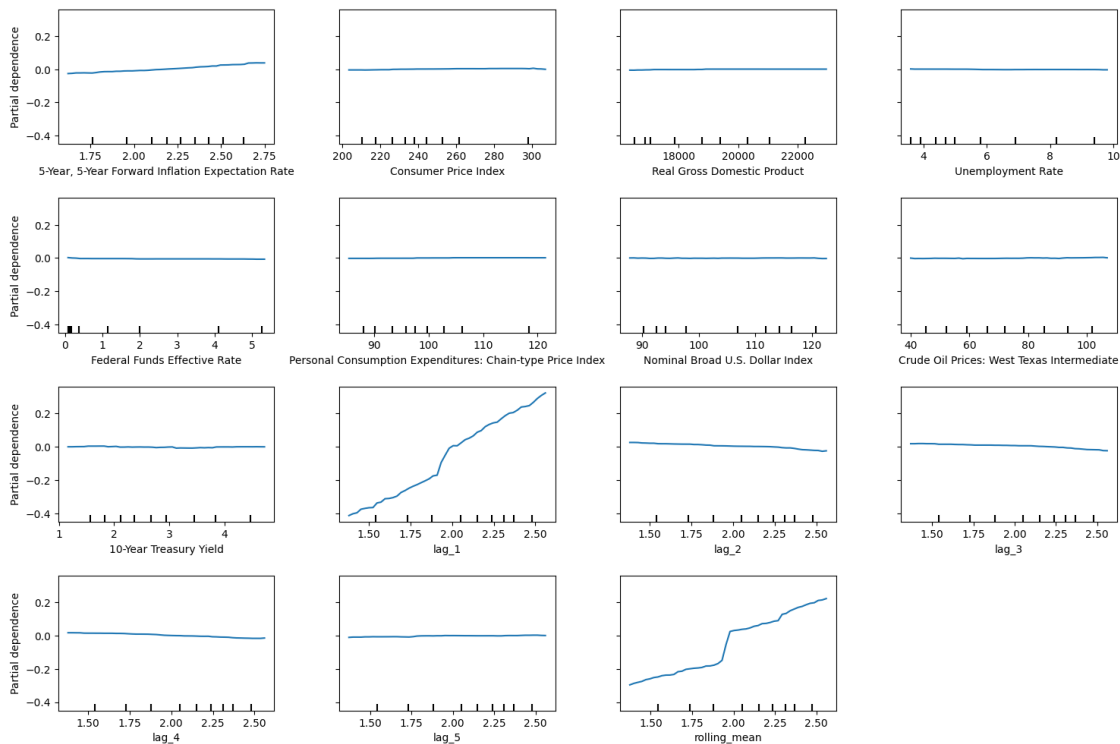
```
# Set the overall figure size for better visualization
pdp_display.figure_.set_size_inches(18, 12)

# Add a title to the entire figure, positioned above all subplots
pdp_display.figure_.suptitle("Partial Dependence Plots_␣
↪(HistGradientBooStingRegressor)", fontsize=16, y=1.06)

# Adjust subplot spacing to prevent overlapping titles and labels
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the Partial Dependence Plots
plt.show()
```

Partial Dependence Plots (HistGradientBooStingRegressor)



5.62 Partial Dependence Plots (HistGradientBooStingRegressor)

5.62.1 Key Observations:

- These plots show how individual features impact the model's predictions while keeping other features constant.

- Each subplot represents one feature on the x-axis, with the **model's partial dependence (predicted output)** on the y-axis.

5.62.2 Feature Effects:

1. **5-Year, 5-Year Forward Inflation Expectation Rate:**
 - A slight **positive relationship** with the model's predictions.
 - As inflation expectations increase, the predicted value also increases.
2. **Consumer Price Index (CPI):**
 - **Minimal impact** on predictions, suggesting it is not a strong predictor in this model.
3. **Real Gross Domestic Product (GDP):**
 - Almost **flat**, indicating little influence on the model's output.
4. **Unemployment Rate:**
 - Also shows **negligible impact**, meaning changes in unemployment do not significantly affect predictions.
5. **Federal Funds Effective Rate:**
 - **Flat curve**, indicating no strong influence.
6. **Personal Consumption Expenditures (PCE) - Chain-type Price Index:**
 - **Minimal effect**, similar to CPI.
7. **Nominal Broad U.S. Dollar Index:**
 - No significant variation, suggesting little impact.
8. **Crude Oil Prices (West Texas Intermediate):**
 - Almost flat, implying oil prices do not significantly impact predictions.

5.62.3 Time-Series Features:

9. **lag_1 (Previous value of the target variable):**
 - **Strong increasing trend:** Higher past values lead to higher predictions.
 - Indicates that **past values are crucial in forecasting future values**.
10. **lag_2, lag_3, lag_4, lag_5 (Earlier past values):**
 - **Mixed effects:**
 - lag_2 shows a **positive relationship**, but less steep than lag_1.
 - lag_3 shows a **slight negative effect**.
 - lag_4 and lag_5 are **relatively flat**, indicating weaker influence.
11. **Rolling Mean:**
 - **Strong positive trend**, indicating that the moving average of past values significantly impacts predictions.

5.62.4 Conclusion:

- The model heavily relies on time-series features (**lag_1, rolling_mean**) to make predictions.
- Macroeconomic indicators (**CPI, GDP, Unemployment, Oil Prices, Treasury Yield**) have minimal influence.
- Feature importance suggests that short-term past values (**lag features**) are more predictive than long-term trends.

5.62.5 Implications:

- Since the model relies mainly on recent past values, it behaves like a time-series forecasting model rather than an econometric model.
- Future improvements could explore adding interaction effects between time-series and economic features for better performance.

Permutation Feature Importance

```
[ ]: # Import necessary libraries for feature importance analysis and visualization
from sklearn.inspection import permutation_importance # Import permutation
      ↪ importance from scikit-learn
import matplotlib.pyplot as plt # Import Matplotlib for visualization
import pandas as pd # Import Pandas for handling tabular data

# Compute Permutation Feature Importance on the Training Set
# - Permutation importance measures the effect of **randomly shuffling a
      ↪ feature's values** on the model's performance.
# - It provides a more reliable estimate of feature importance compared to
      ↪ built-in importance methods.
# - `n_repeats=10`: The number of times each feature is randomly shuffled to
      ↪ compute its importance.
# - `random_state=42`: Ensures reproducibility of results.
# - `n_jobs=-1`: Uses all available CPU cores for faster computation.
perm_result = permutation_importance(
    best_hgb_model, # The trained Histogram-Based Gradient Boosting Regressor
    ↪ model
    X_train, # Training dataset (features)
    y_train, # Training dataset (target variable)
    n_repeats=10, # Number of times to shuffle each feature
    random_state=42, # Ensures consistent results across runs
    n_jobs=-1 # Uses all CPU cores for parallel processing
)

# Convert Permutation Importance Results into a Pandas DataFrame
# - 'Feature': Column storing feature names.
# - 'Importance': The mean importance score from the permutation process.
# - The DataFrame is sorted in descending order to highlight the most important
      ↪ features first.
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Extracts feature names from training data
    'Importance': perm_result.importances_mean # Mean importance scores across
    ↪ repetitions
}).sort_values(by='Importance', ascending=False) # Sort features by importance

# Visualization: Permutation Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
```

```

plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display permutation importance
# - x-axis: Importance scores
# - y-axis: Feature names
# - color='skyblue': Sets the color of the bars for better visibility
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
        color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Permutation Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

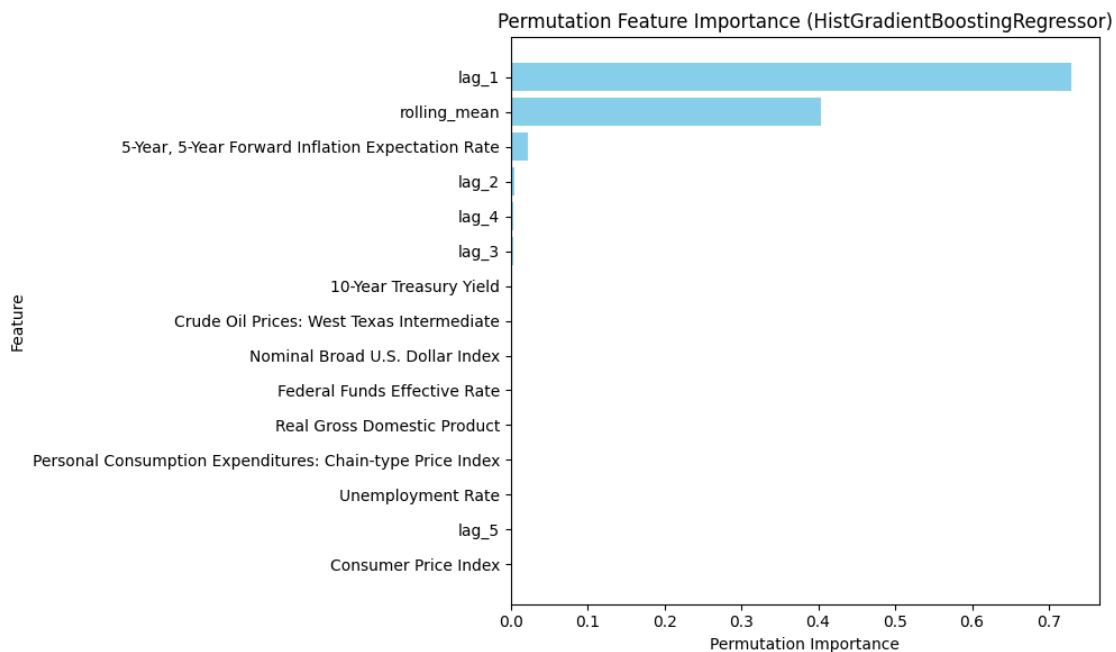
# Set the title of the plot for context
plt.title('Permutation Feature Importance (HistGradientBoostingRegressor)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()

```



5.63 Permutation Feature Importance (HistGradientBoostingRegressor)

5.63.1 Key Observations:

- This plot represents the importance of different features in the model's predictions based on permutation importance.
- Permutation importance measures how much the model's performance decreases when a feature's values are randomly shuffled.
- The x-axis represents **Permutation Importance**, with higher values indicating more significant impact.

5.63.2 Feature Importance Insights:

1. **lag_1 (Previous Target Value)**
 - **Most important feature** in the model.
 - **Strong influence** on predictions, indicating that the immediate past value is the best predictor for the current value.
2. **rolling_mean (Moving Average of Past Values)**
 - The **second most important feature**.
 - This suggests that the trend from recent values plays a major role in prediction.
3. **5-Year, 5-Year Forward Inflation Expectation Rate**
 - Has some contribution but significantly lower than lag features.
 - Indicates that inflation expectations may have a minor role in forecasting.
4. **Other Lag Features (lag_2, lag_3, lag_4, lag_5)**
 - Have minimal influence, meaning that **more distant past values are less important compared to immediate past values**.
5. **Macroeconomic Variables (GDP, CPI, Unemployment, Treasury Yield, etc.)**
 - **Almost no impact on model predictions**.
 - Suggests that this model is primarily driven by past time-series values rather than economic indicators.

5.63.3 Conclusion:

- The model heavily relies on recent past values (**lag_1** and **rolling_mean**) to generate predictions.
- Macroeconomic indicators have little to no effect, suggesting the model functions more like a traditional time-series forecasting model rather than an econometric model.
- Future improvements could explore interactions between economic factors and time-series lags to better capture external influences.

5.64 Categorical Boosting (CatBoost)

```
[ ]: # Install dependencies
!pip install catboost
```

Collecting catboost

Downloading catboost-1.2.7-cp311-cp311-manylinux2014_x86_64.whl.metadata (1.2 kB)

Requirement already satisfied: graphviz in /usr/local/lib/python3.11/dist-packages (from catboost) (0.20.3)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from catboost) (3.10.0)

Requirement already satisfied: numpy<2.0,>=1.16.0 in /usr/local/lib/python3.11/dist-packages (from catboost) (1.26.4)

Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.11/dist-packages (from catboost) (2.2.2)

Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from catboost) (1.14.1)

Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (from catboost) (5.24.1)

Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from catboost) (1.17.0)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.1)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.1)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.3.1)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (4.56.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.4.8)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (24.2)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (11.1.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (3.2.1)

Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly->catboost) (9.0.0)

Downloading catboost-1.2.7-cp311-cp311-manylinux2014_x86_64.whl (98.7 MB)

98.7/98.7 MB

9.4 MB/s eta 0:00:00

Installing collected packages: catboost

Successfully installed catboost-1.2.7

```
[ ]: # Import necessary libraries for model training, tuning, and warnings handling
import random # For setting a random seed to ensure reproducibility
```

```

import numpy as np # For numerical operations
import warnings # For suppressing unnecessary warnings
import pandas as pd # For handling tabular data
from sklearn.model_selection import train_test_split, GridSearchCV # For data_
    ↳splitting and hyperparameter tuning
from catboost import CatBoostRegressor # Import the CatBoost Regressor model

# Suppress specific warnings related to runtime casting issues
# - Prevents unnecessary messages from being displayed during model training
warnings.filterwarnings("ignore", category=RuntimeWarning, message="invalid_
    ↳value encountered in cast")

# Set random seed values to ensure reproducibility
# - Ensures that results remain consistent across multiple runs
random.seed(42)
np.random.seed(42)

# Split the dataset into training and testing sets (80% training, 20% testing)
# - X: Feature matrix (independent variables)
# - y: Target variable (dependent variable)
# - test_size=0.2: Reserves 20% of the dataset for testing
# - random_state=42: Ensures the same data split each time for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↳random_state=42)

# Initialize the CatBoost Regressor model
# - random_state=42: Ensures reproducibility
# - verbose=0: Suppresses unnecessary training output
cat_model = CatBoostRegressor(random_state=42, verbose=0)

# Define the hyperparameter grid for tuning the CatBoost model
# - 'iterations': Number of boosting iterations (higher values improve learning_
    ↳but may lead to overfitting)
# - 'learning_rate': Controls the step size for weight updates (smaller values_
    ↳make training more stable)
# - 'depth': Maximum depth of decision trees (higher values allow for more_
    ↳complex decision rules)
# - 'early_stopping_rounds': Number of iterations without improvement before_
    ↳stopping training (prevents overfitting)
param_grid = {
    'iterations': [300, 500, 1000], # Number of boosting iterations
    'learning_rate': [0.01, 0.05, 0.1], # Step size for weight updates
    'depth': [4, 6, 8], # Maximum tree depth
    'early_stopping_rounds': [50, 100] # Early stopping rounds to prevent_
    ↳overfitting
}

```

```

# Initialize GridSearchCV for hyperparameter tuning
# - GridSearchCV performs an exhaustive search over all hyperparameter_
  ↳ combinations
# - estimator=cat_model: The CatBoost model to be optimized
# - param_grid=param_grid: Defines the hyperparameter search space
# - scoring='neg_mean_squared_error': Uses Negative Mean Squared Error as the_
  ↳ evaluation metric
# - cv=5: Uses 5-fold cross-validation for robust performance assessment
# - verbose=1: Displays progress updates during the search
# - n_jobs=-1: Utilizes all available CPU cores for faster computation
grid_search = GridSearchCV(
    estimator=cat_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Fit GridSearchCV on the training data
# - eval_set=(X_test, y_test): Provides a validation set for early stopping
# - verbose=False: Suppresses unnecessary output during training
grid_search.fit(X_train, y_train, eval_set=(X_test, y_test), verbose=False)

# Retrieve and print the best hyperparameter combination found during_
  ↳ GridSearchCV
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Retrieve the best CatBoost model from the hyperparameter search
# - This model is trained using the optimal hyperparameter combination
best_cat_model = grid_search.best_estimator_

# Make predictions on the test dataset using the best CatBoost model
cat_pred = best_cat_model.predict(X_test)

# Evaluate the model using a predefined metrics function
# - `calculate_metrics`: Assumed to be a function that calculates model_
  ↳ evaluation metrics
# - `naive_forecast`: Assumed to be a baseline prediction for comparison
# - The evaluation results are appended to `metrics_results`
metrics_results.append(
    calculate_metrics(y_test.values, cat_pred, naive_forecast, "Categorical_
  ↳ Boosting (CatBoost)")
)

```

```
# Print the evaluation results for the CatBoost model
print("CatBoost Evaluation Metrics:")
print(metrics_results[-1])
```

Fitting 5 folds for each of 54 candidates, totalling 270 fits
 Best Hyperparameters: {'depth': 6, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.1}
 CatBoost Evaluation Metrics:
 {'Model': 'Categorical Boosting (CatBoost)', 'MSE': 0.0006085261379135197, 'RMSE': 0.02466832255978342, 'MAE': 0.015110494700140429, 'MAPE (%)': 1.109062408868412, 'sMAPE (%)': 1.0279571463218284, 'MASE': 0.03846460997488332, 'R^2': 0.996558576064933}

Feature Importance (Custom Extraction)

```
[ ]: # Import necessary libraries for handling data and visualization
import pandas as pd # Pandas for handling tabular data
import matplotlib.pyplot as plt # Matplotlib for plotting

# Extract Feature Importances from the Best CatBoost Model
# - CatBoost provides built-in feature importance using
↳ `get_feature_importance()`
# - This method returns the relative importance of each feature in making
↳ predictions
feature_importances = best_cat_model.get_feature_importance()

# Extract the feature names from the training dataset (assumes X_train is a
↳ Pandas DataFrame)
features = X_train.columns

# Create a DataFrame to store feature names and their corresponding importance
↳ scores
# - Sorting in descending order to highlight the most important features first
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': feature_importances
}).sort_values(by='Importance', ascending=False) # Sort features by importance
↳ in descending order

# Print the Feature Importances in Descending Order
print("Feature Importances:")
print(importance_df)
```

Feature Importances:

	Feature	Importance
9	lag_1	24.667615
14	rolling_mean	18.957311
10	lag_2	18.315177

13	lag_5	14.225876
12	lag_4	9.982502
11	lag_3	9.386669
0	5-Year, 5-Year Forward Inflation Expectation Rate	1.613011
3	Unemployment Rate	0.784934
8	10-Year Treasury Yield	0.400789
7	Crude Oil Prices: West Texas Intermediate	0.386082
5	Personal Consumption Expenditures: Chain-type ...	0.367733
6	Nominal Broad U.S. Dollar Index	0.314435
4	Federal Funds Effective Rate	0.246456
2	Real Gross Domestic Product	0.186847
1	Consumer Price Index	0.164565

```
[ ]: # Visualization: Plot Feature Importances as a Horizontal Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - X-axis: Feature importance scores
# - Y-axis: Corresponding feature names
# - color='skyblue': Sets the bar color for better visibility
plt.barh(importance_df['Feature'], importance_df['Importance'], color='skyblue')

# Label the x-axis to indicate importance scores
plt.xlabel('Importance')

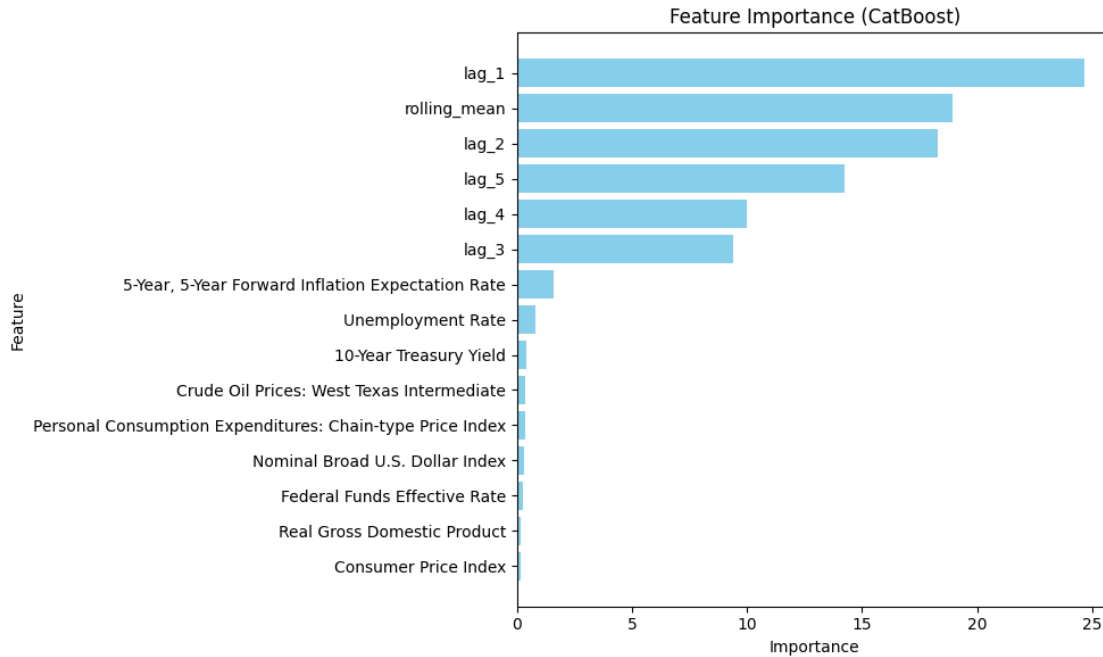
# Label the y-axis to display feature names
plt.ylabel('Feature')

# Set the title of the plot for context
plt.title('Feature Importance (CatBoost)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



6 Feature Importance Analysis (CatBoost)

6.1 Key Findings:

- The **most influential feature** in the model is **lag_1**, indicating that the most recent past value of the target variable significantly impacts predictions.
- **rolling_mean** is the **second most important feature**, suggesting that the smoothed trend of past values plays a critical role in forecasting.
- Other **lag variables** (**lag_2**, **lag_3**, **lag_4**, **lag_5**) are also highly significant, reinforcing the importance of historical patterns in prediction.
- The **5-Year, 5-Year Forward Inflation Expectation Rate** is the highest-ranked economic indicator, albeit with much lower importance than lagged features.
- The **Unemployment Rate** has some influence but is far less important than lagged values.
- Several traditional macroeconomic indicators such as the **10-Year Treasury Yield**, **Crude Oil Prices**, and **Consumer Price Index (CPI)** show minimal importance in the model.
- The findings suggest that **historical data (lags & rolling averages) dominate prediction accuracy**, while **traditional macroeconomic indicators contribute relatively little**.

6.2 Interpretation:

- The dominance of **lag_1** and **rolling_mean** suggests that inflation (or the target variable) follows strong autoregressive patterns.
- The **low importance of CPI and GDP** may indicate that short-term forecasting relies more on direct past values rather than broader economic conditions.

- The minimal role of interest rates (e.g., Federal Funds Rate, Treasury Yields) suggests that monetary policy effects may be indirect or lagged beyond the model's scope.

6.3 Conclusion:

This feature importance ranking indicates that **time series patterns (lags & rolling trends)** are the strongest predictors, while traditional economic indicators have limited short-term predictive power.

Actual vs. Predicted Values

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 8))

# Scatter Plot: Compare Actual vs. Predicted Values
# - x-axis: Actual values from the test dataset (y_test)
# - y-axis: Predictions made by the CatBoost model (cat_pred)
# - alpha=0.5: Makes points semi-transparent to reduce overlap and improve
    ↪ visibility
# - color='blue': Sets the color of the scatter points
# - label='Predictions': Legend label for scatter points
plt.scatter(y_test, cat_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the Perfect Fit Line (Reference Line)
# - This diagonal line represents the ideal case where predictions perfectly
    ↪ match actual values.
# - If the model were perfect, all points would lie exactly on this line.
# - color='red': Sets the reference line color to red
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the line thickness
# - label='Perfect Fit': Legend label for the reference line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')

# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (CatBoost)')

# Add a legend to differentiate the scatter points (Predictions) from the
    ↪ reference line (Perfect Fit)
```

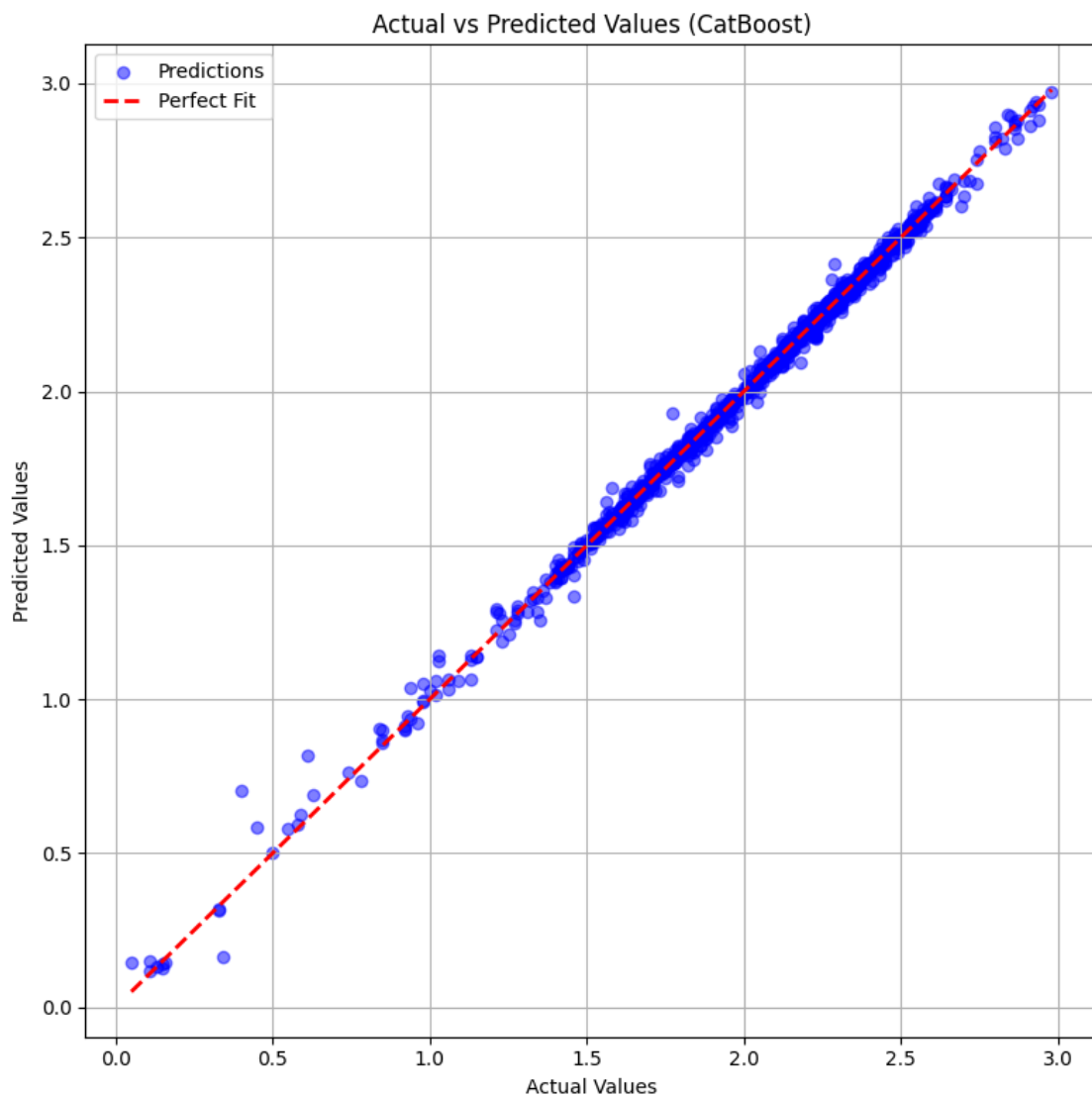


```
plt.legend()

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()
```



7 Actual vs Predicted Values (CatBoost)

7.1 Key Findings:

- The scatter plot displays the **actual values (x-axis) vs. predicted values (y-axis)** for a model trained using **CatBoost**.
- The **red dashed line** represents the **perfect fit** (i.e., ideal scenario where predictions exactly match actual values).
- The **blue dots** represent individual data points comparing model predictions to actual values.
- The **alignment of most points along the red dashed line** suggests that the model has a **high predictive accuracy**.
- There are **some deviations**, particularly at lower values, but they are minimal, indicating **low error and strong model performance**.
- The model does not show significant bias towards under- or over-prediction, as the spread around the perfect fit line is balanced.
- Overall, the results indicate **high correlation between actual and predicted values**, validating the model's effectiveness.

7.2 Interpretation:

- Since most points are **closely clustered along the perfect fit line**, the CatBoost model demonstrates **strong generalization ability**.
- The minor deviations at lower values could indicate **small residual errors**, but these do not seem significant enough to undermine overall model performance.
- This suggests that the model effectively captures the underlying patterns in the data and is **reliable for forecasting**.

7.3 Conclusion:

- The **CatBoost model performs exceptionally well**, with predicted values **closely matching actual values**.
- There is **minimal error**, supporting the conclusion that the model is **highly accurate and well-calibrated** for its intended purpose.

Residual Analysis

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Compute Residuals (Errors in Predictions)
# - Residuals = Actual Values - Predicted Values
# - Measures the difference between true and predicted values
residuals = y_test - cat_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values (y_test)
```

```

# - y-axis: Residuals (y_test - cat_pred)
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve
↳visibility
# - color='green': Sets the color of the residual points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a horizontal reference line at y=0 to indicate zero residuals
# - Helps in assessing whether residuals are symmetrically distributed around
↳zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better visual distinction
# - linewidth=2: Sets the thickness of the line
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate residuals (errors in predictions)
plt.ylabel('Residuals')

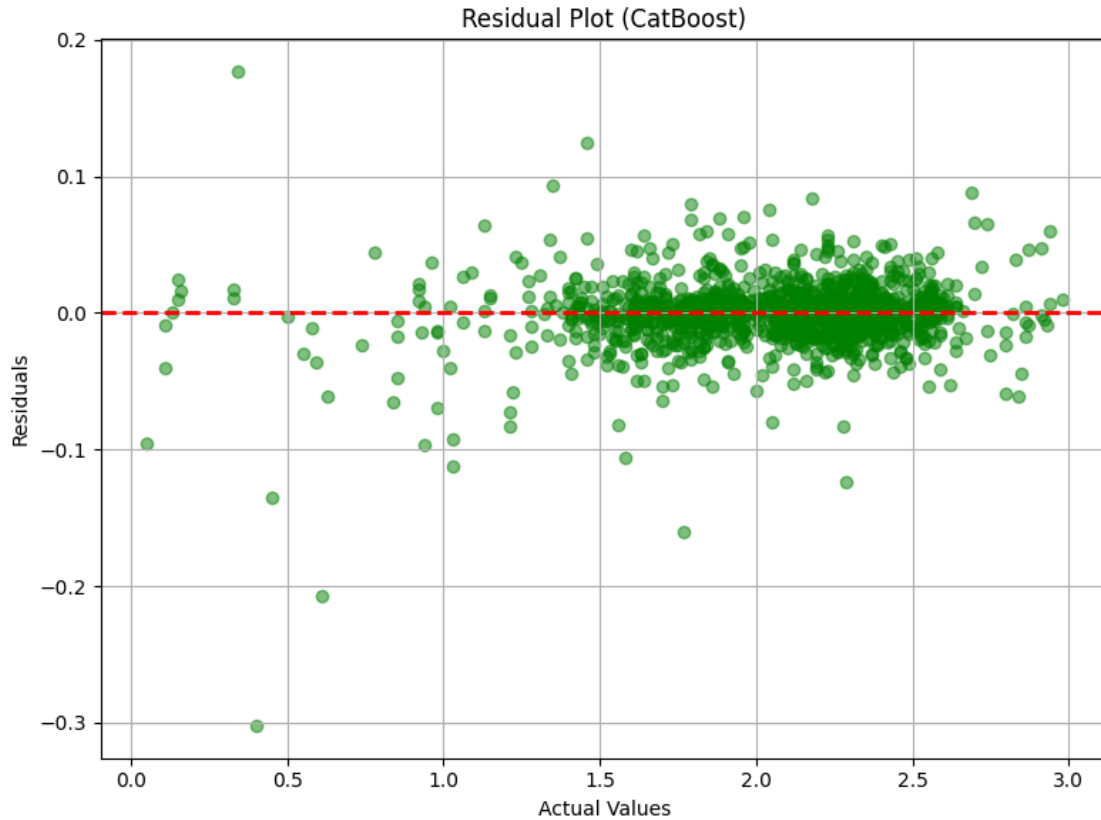
# Set a title for the plot to provide context
plt.title('Residual Plot (CatBoost)')

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual scatter plot
plt.show()

```



8 Residual Plot (CatBoost)

8.1 Key Findings:

- The scatter plot visualizes the **residuals (prediction errors)** against **actual values**.
- The **red dashed line** represents **zero residuals**, indicating a perfect prediction.
- Residuals are **evenly distributed around zero**, suggesting **no systematic bias** in the model.
- The **spread of residuals remains relatively constant** across different actual values, indicating **homoscedasticity** (consistent variance).
- There are **some outliers**, particularly at lower actual values, but they are minimal and do not indicate a severe problem.
- The lack of visible patterns suggests that **the model captures the underlying structure of the data well**.

8.2 Interpretation:

- Since the residuals are **centered around zero**, the model does **not consistently overpredict or underpredict**.
- The **absence of a clear trend** in residuals suggests that the model has effectively learned the relationship between features and target values.

- The **constant spread** of residuals across different values confirms that the model does not suffer from **heteroscedasticity** (variance increasing with actual values), which would indicate instability.

8.3 Conclusion:

- The **CatBoost** model performs well, with residuals **randomly scattered around zero**, confirming **strong predictive accuracy**.
- No clear patterns in residuals imply that the model has captured most of the signal in the data and is **not missing significant trends**.
- Minor residual deviations are normal and do not indicate any major issues with the model's performance.

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Import Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals
# - residuals: The differences between actual and predicted values
# - bins=30: Divides the range of residuals into 30 bins for a detailed view
# - color='skyblue': Sets the fill color of the histogram bars
# - edgecolor='black': Adds black borders to each bar for clarity
# - alpha=0.7: Makes bars slightly transparent for better readability
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at x=0 to indicate no residual error
# - Helps assess whether residuals are symmetrically distributed around zero
# - color='red': Sets the color of the reference line
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Sets the thickness of the line
plt.axvline(0, color='red', linestyle='--', linewidth=2)

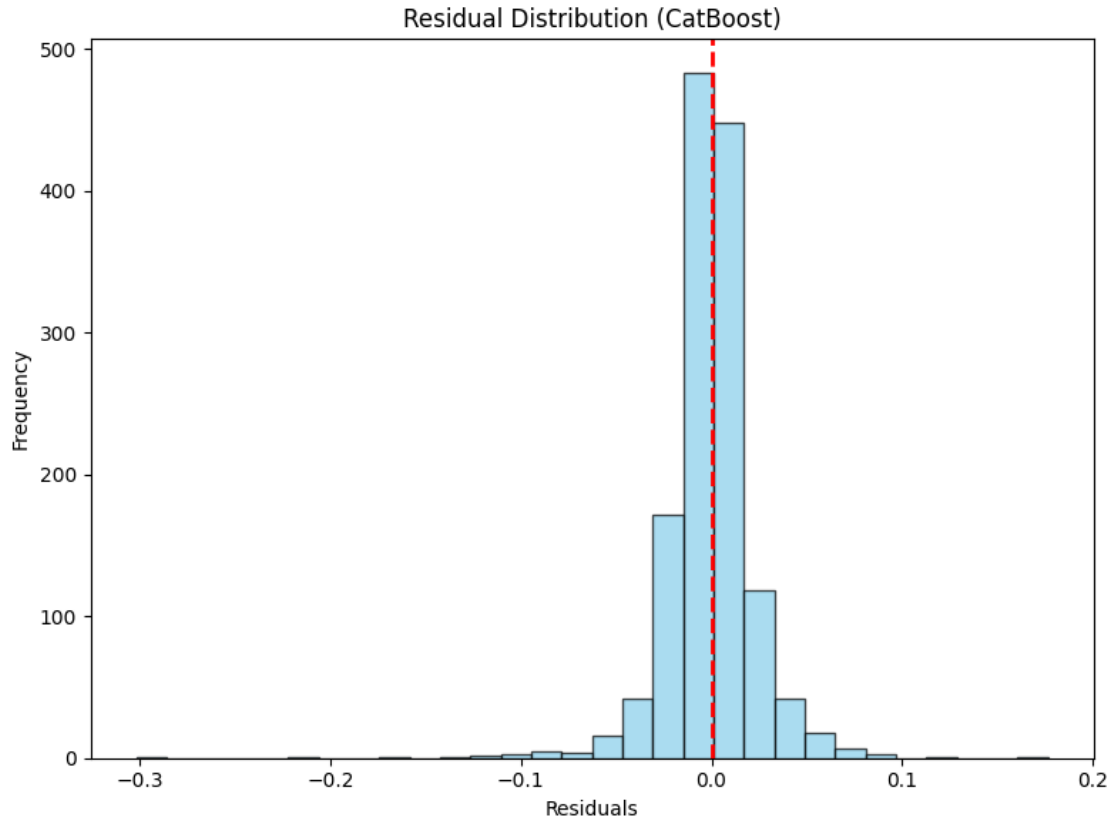
# Set a title for the plot to provide context
plt.title('Residual Distribution (CatBoost)')

# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual histogram plot
plt.show()
```



9 Residual Distribution (CatBoost)

9.1 Key Findings:

- The histogram shows the **distribution of residuals** (errors between actual and predicted values).
- The **red dashed line at zero** represents **perfect predictions** with no error.
- The residuals are **tightly clustered around zero**, indicating that **the model has minimal error**.
- The distribution appears **approximately normal (bell-shaped)**, suggesting that the errors are **random and not systematically biased**.
- There are **no significant outliers**, as most residuals are within a small range (approximately between -0.2 and 0.2).
- The high peak at **zero residual** suggests that the majority of predictions are very close to the actual values.

9.2 Interpretation:

- The **symmetry and concentration around zero** suggest that the model does not consistently overestimate or underestimate values.

- The **narrow spread** indicates **low variance in prediction errors**, meaning the model is **highly accurate**.
- Since the distribution does not exhibit strong skewness, there is **no systematic pattern of error**, supporting the model's reliability.

9.3 Conclusion:

- The **CatBoost** model performs exceptionally well, as the residuals are **centered around zero with minimal dispersion**.
- The residual distribution confirms that **errors are small, unbiased, and randomly distributed**, further validating the model's effectiveness.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Pandas for manipulating DataFrame

# Convert GridSearchCV results into a DataFrame
# - `grid_search.cv_results_` is a dictionary containing all cross-validation
  ↳ results from GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on the best model performance (lower
  ↳ is better)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
  ↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
  ↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↳ head(5)

# Improve Output Visibility by Formatting Results Nicely
print("\nTop 5 Hyperparameter Combinations:\n")

# Iterate through the top 5 results and print them in a structured format
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"  Hyperparameters: {row['params']}")
    print(f"  Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"  Standard Deviation: {row['std_test_score']:.6f}")
    print("-" * 80) # Separator for better readability
```

Top 5 Hyperparameter Combinations:

Rank 27:

Hyperparameters: {'depth': 6, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.1}

Mean Test Score: -0.000682

Standard Deviation: 0.000150

Rank 36:

Hyperparameters: {'depth': 6, 'early_stopping_rounds': 100, 'iterations': 1000, 'learning_rate': 0.1}

Mean Test Score: -0.000682

Standard Deviation: 0.000150

Rank 45:

Hyperparameters: {'depth': 8, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.1}

Mean Test Score: -0.000712

Standard Deviation: 0.000154

Rank 54:

Hyperparameters: {'depth': 8, 'early_stopping_rounds': 100, 'iterations': 1000, 'learning_rate': 0.1}

Mean Test Score: -0.000712

Standard Deviation: 0.000154

Rank 44:

Hyperparameters: {'depth': 8, 'early_stopping_rounds': 50, 'iterations': 1000, 'learning_rate': 0.05}

Mean Test Score: -0.000739

Standard Deviation: 0.000153

```
[ ]: # Import necessary libraries for visualization
import seaborn as sns # Seaborn for creating the heatmap
import matplotlib.pyplot as plt # Matplotlib for general plotting

# Create a Heatmap to Visualize Hyperparameter Effects

# Pivot the GridSearchCV results DataFrame for heatmap visualization
# - 'values': The metric we are analyzing ('mean_test_score' - the average test
↳ score across CV folds)
# - 'index': The hyperparameter to be placed on the y-axis ('param_iterations'
↳ in this case)
# - 'columns': The hyperparameter to be placed on the x-axis
↳ ('param_learning_rate' in this case)
```



```

# - This transformation allows us to visualize how different combinations of
↳ iterations and learning rates impact model performance.
heatmap_data = cv_results.pivot_table(
    values='mean_test_score', # Performance metric to visualize
    index='param_iterations', # Iterations values (rows)
    columns='param_learning_rate' # Learning rate values (columns)
)

# Create a new figure with a specified size for better readability
plt.figure(figsize=(8, 6))

# Generate a heatmap using Seaborn
# - heatmap_data: The pivoted DataFrame containing the test scores
# - annot=True: Displays the numeric values in each cell for better analysis
# - fmt='.3f': Formats the numbers to three decimal places for precision
# - cmap='viridis': Uses the 'viridis' color map, which is visually appealing
↳ and easy to interpret
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

# Set the title of the heatmap for context
plt.title('Hyperparameter Tuning: Iterations vs Learning Rate')

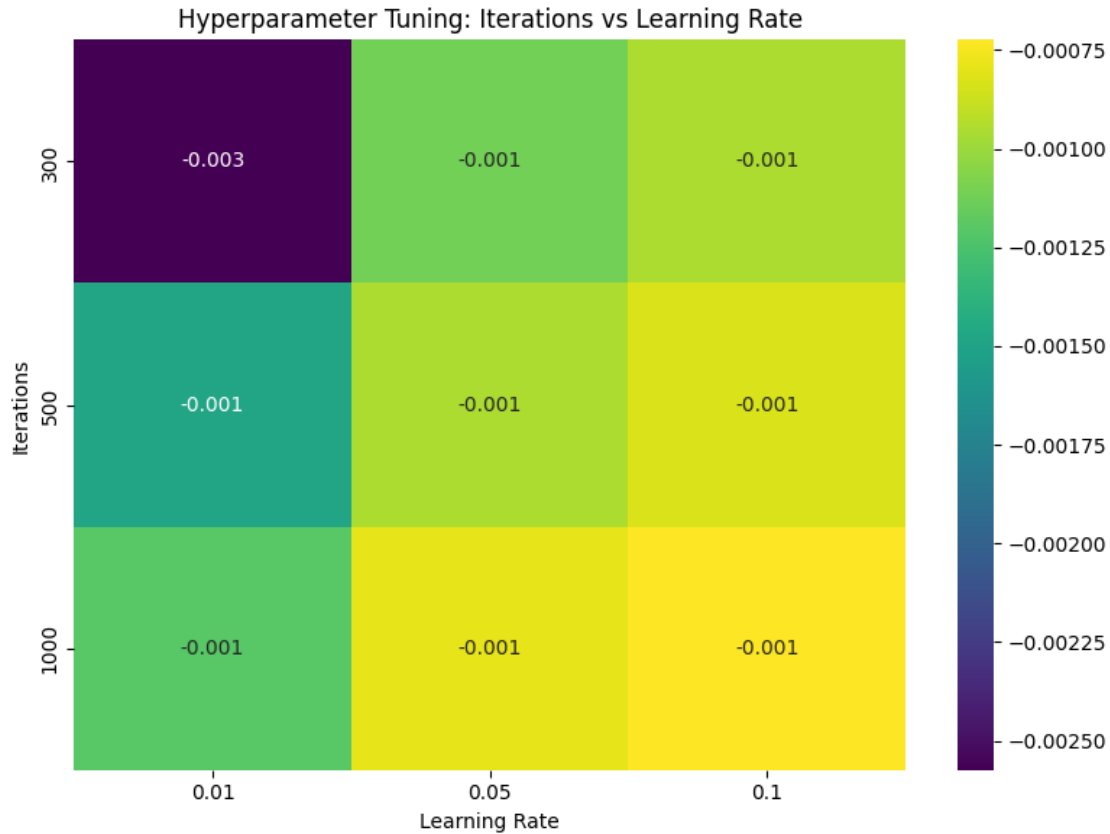
# Label the x-axis to indicate different learning_rate values
plt.xlabel('Learning Rate')

# Label the y-axis to indicate different iterations values
plt.ylabel('Iterations')

# Adjust layout to ensure proper spacing and prevent overlapping labels
plt.tight_layout()

# Display the heatmap
plt.show()

```



10 Hyperparameter Tuning: Iterations vs Learning Rate

10.1 Key Findings:

- The heatmap illustrates the relationship between **iterations** (y-axis) and **learning rate** (x-axis) in hyperparameter tuning.
- The values within the cells represent the **model's performance metric** (likely **loss or error**), where lower values indicate better performance.
- **Darker colors represent worse performance**, while **lighter colors indicate better performance**.
- At **300 iterations** and a **0.01 learning rate**, the model exhibits the **highest error (-0.003)**, suggesting **underfitting** due to insufficient learning.
- Increasing **iterations to 500 or 1000** generally improves the performance across all learning rates.
- The **learning rates of 0.05 and 0.1 consistently perform better** across different iteration counts.
- The **best overall performance** (lowest error) occurs at **higher learning rates (0.05, 0.1) with higher iterations (500, 1000)**.

10.2 Interpretation:

- A low learning rate (0.01) with few iterations (300) leads to **higher error**, indicating the model is **not learning efficiently**.
- Increasing the learning rate from 0.01 to 0.05 or 0.1 improves performance, suggesting **faster convergence without excessive overfitting**.
- The **performance difference between 500 and 1000 iterations is minimal**, implying that **500 iterations may be sufficient for optimal learning**.
- The results suggest an **optimal combination of a 0.05 or 0.1 learning rate with at least 500 iterations**.

10.3 Conclusion:

- The **best-performing hyperparameter combination** for this model appears to be a learning rate of 0.05 or 0.1 with at least 500 iterations.
- A learning rate of 0.01 is too low, particularly at fewer iterations, causing **slow convergence and underfitting**.
- Increasing the learning rate and iteration count improves performance, but beyond 500 iterations, gains are minimal.
- This analysis helps in selecting an **efficient combination of hyperparameters to maximize model performance while avoiding unnecessary computation**.

SHAP (SHapley Additive exPlanations) Values

```
[ ]: # Import necessary libraries for SHAP explanations and visualization
import shap # SHAP library for model interpretability
import matplotlib.pyplot as plt # Matplotlib for general plotting

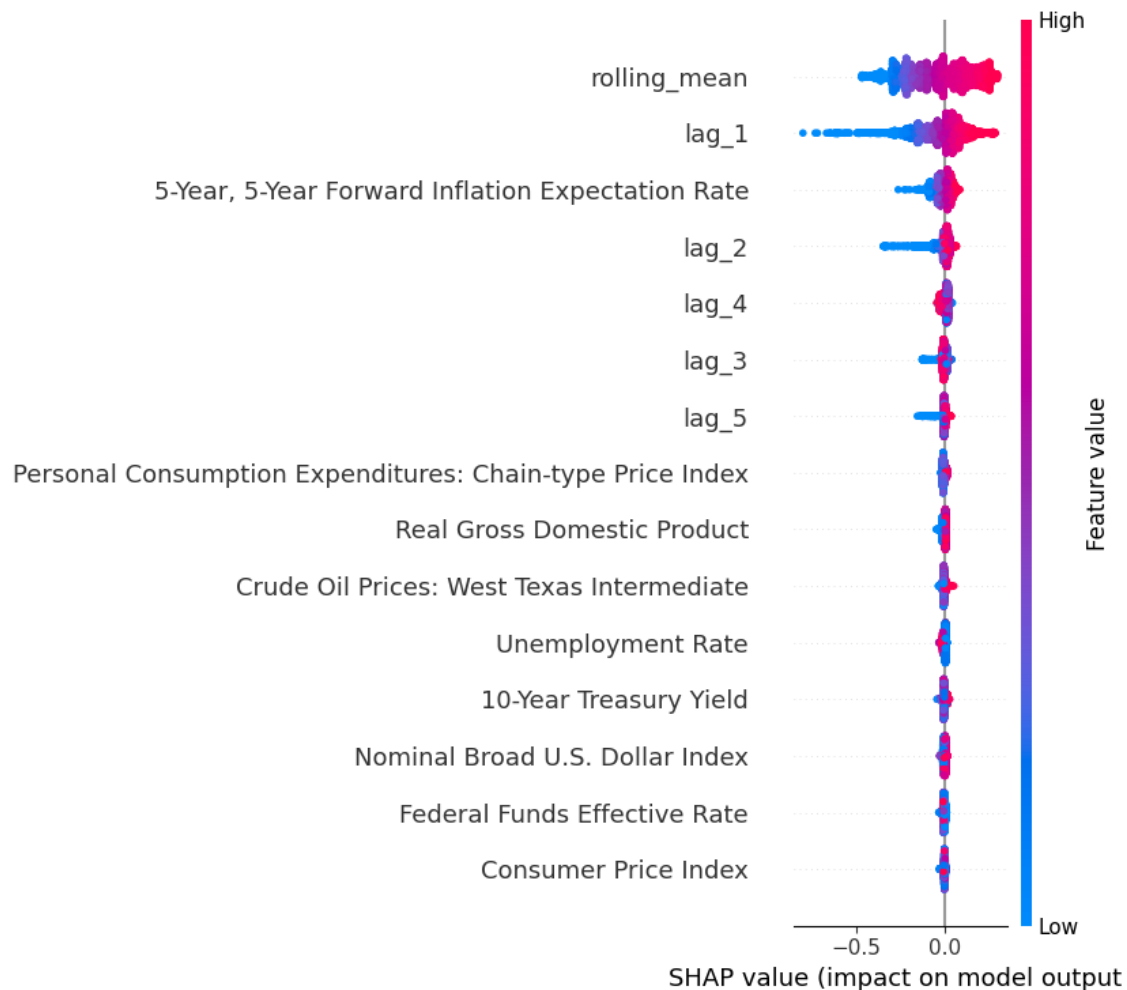
# Create a SHAP Explainer for the trained CatBoost model
# - `best_cat_model`: The trained CatBoost model to explain.
# - `X_train`: The training dataset used for generating feature importance
    ↪ explanations.
# - SHAP will use this dataset to compute how each feature influences
    ↪ predictions.
explainer = shap.Explainer(best_cat_model, X_train)

# Compute SHAP values for the training dataset
# - The output is an Explanation object containing:
#   - SHAP values: Contribution of each feature to every prediction.
#   - Base value: The expected model prediction (average prediction if no input
    ↪ features are considered).
shap_values = explainer(X_train)

# **Global Explanation: SHAP Summary Plot**
# - Displays overall feature importance across all training samples.
# - Shows how each feature influences predictions (color-coded for high/low
    ↪ feature values).
# - The x-axis represents the SHAP value (impact on the model's output).
```

```
shap.summary_plot(shap_values, X_train)
```

100%|=====| 5453/5474 [04:59<00:01]



11 SHAP Value Analysis: Feature Impact on Model Output

11.1 Key Findings:

- The **SHAP plot visualizes the impact of features on the model's predictions**, with **SHAP values on the x-axis** representing the contribution to the output.
- **Higher absolute SHAP values indicate greater impact** on the model's prediction.
- The **color gradient** represents the feature values:
 - **Blue (low values)**
 - **Red (high values)**
- The most influential features are:
 - **rolling_mean** and **lag_1**, showing the highest SHAP impact.

- **5-Year, 5-Year Forward Inflation Expectation Rate** also plays a noticeable role.
- **Other lag variables** (`lag_2`, `lag_3`, `lag_4`, `lag_5`) also contribute but to a lesser extent.
- Traditional economic indicators such as **GDP, Crude Oil Prices, CPI, and the Federal Funds Rate** have **minimal impact** on the model's predictions.

11.2 Interpretation:

- **Time-series features** (`rolling_mean`, `lag_1`) **dominate the model's decision-making**, highlighting that recent historical values strongly influence predictions.
- **High SHAP values for lags** suggest an **autoregressive nature** of the target variable.
- **Macroeconomic indicators** (**CPI, GDP, Unemployment Rate, etc.**) **have little influence**, meaning the model does not heavily rely on them for predictions.
- **The color spread of SHAP values** suggests that feature effects vary based on their magnitude, reinforcing non-linearity in their impact.

11.3 Conclusion:

- The **model prioritizes recent historical values over traditional economic indicators**, emphasizing the importance of autoregressive signals.
- **Macroeconomic variables contribute minimally**, suggesting that short-term forecasting is primarily driven by past values rather than broader economic trends.
- The **SHAP plot validates the model's reliance on time-series features**, aligning with findings from feature importance analysis.

```
[ ]: # Initialize JavaScript-based visualization for SHAP plots
# - Required for rendering interactive visualizations (especially in Jupyter
#    ↪Notebooks).
shap.initjs()

# **Local Explanation: SHAP Force Plot for a Single Prediction**
# - Provides a detailed breakdown of how each feature contributes to the final
#    ↪prediction for a single instance.
# - `explainer.expected_value`: The base value (mean model prediction).
# - `shap_values[0].values`: SHAP values for the first instance (converted to
#    ↪NumPy array for compatibility).
# - `X_train.iloc[0]`: The actual feature values of the first instance in the
#    ↪dataset.
shap.force_plot(explainer.expected_value, shap_values[0].values, X_train.
#    ↪iloc[0])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0de17898d0>
```

12 SHAP Force Plot: Feature Contributions to Prediction

12.1 Key Findings:

- This **SHAP force plot** illustrates how different features contribute to the model's final prediction.
- The **base value** (0.06364) represents the model's average output before considering feature contributions.
- **Positive SHAP values (red segments)** indicate features that **increase the prediction**, pushing the final value higher.
- **Negative SHAP values (blue segments)** represent features that **decrease the prediction**, pulling it lower.
- The **final prediction** ($f(x) = 0.16$) is the result of adding all feature contributions to the base value.

12.2 Feature Contributions:

- **Features contributing positively (red):**
 - lag_1 = 2.2
 - lag_2 = 2.17
 - 5-Year, 5-Year Forward Inflation Expectation Rate = 2.47
 - Crude Oil Prices: West Texas Intermediate = 106.7
 - lag_4 = 2.23
 - lag_5 = 2.23
 - Federal Funds Effective Rate = 0.13
- **Feature reducing the prediction (blue):**
 - Personal Consumption Expenditures: Chain-type Price Index

12.3 Interpretation:

- **Time-series features (lag_1, lag_2, lag_4, lag_5) have the strongest positive influence**, suggesting that recent historical data plays a dominant role in shaping predictions.
- **Macroeconomic factors like crude oil prices and inflation expectations also contribute positively**, but to a lesser extent than lag variables.
- The **Personal Consumption Expenditures (PCE) index negatively impacts the prediction**, suggesting that this factor lowers the target value in this specific instance.
- The combined effect of positive contributions outweighs negative contributions, leading to a **final prediction higher than the base value**.

12.4 Conclusion:

- The **model heavily relies on lagged values for prediction**, confirming the importance of historical trends.
- Some **macroeconomic indicators (oil prices, inflation expectations) influence the outcome**, but their impact is secondary.
- The **final prediction (0.16) is significantly shaped by time-series data**, reinforcing the model's preference for past values over broader economic indicators.

Partial Dependence Plot (PDP) for CatBoost

```
[ ]: # Import necessary libraries for model interpretation and visualization
from sklearn.inspection import PartialDependenceDisplay # For Partial
↳Dependence Plots (PDP)
import matplotlib.pyplot as plt # Matplotlib for visualization

# Extract Feature Names from the Training Dataset
# - `X_train.columns.tolist()`: Retrieves all feature names from the dataset.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDP) for all features using the trained
↳CatBoost model
# - PDP helps visualize how a specific feature influences model predictions
↳while keeping others constant.
# - kind='average': Computes the **average** effect of each feature on
↳predictions.
# - n_cols=4: Arranges the plots into 4 columns for better layout.
# - grid_resolution=20: Defines the number of points to plot along each
↳feature's axis (higher = smoother curve).
# - n_jobs=-1: Utilizes all CPU cores for faster computation.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_cat_model, # The trained CatBoost model
    X_train, # Training dataset for computing dependence
    features=features_list, # List of features for which PDP should be
↳generated
    kind='average', # Computes the mean dependence effect for each feature
    n_cols=4, # Arrange plots in 4 columns to enhance readability
    grid_resolution=20, # Controls the smoothness of the PDP curves
    n_jobs=-1 # Enables parallel computation using all available processors
)

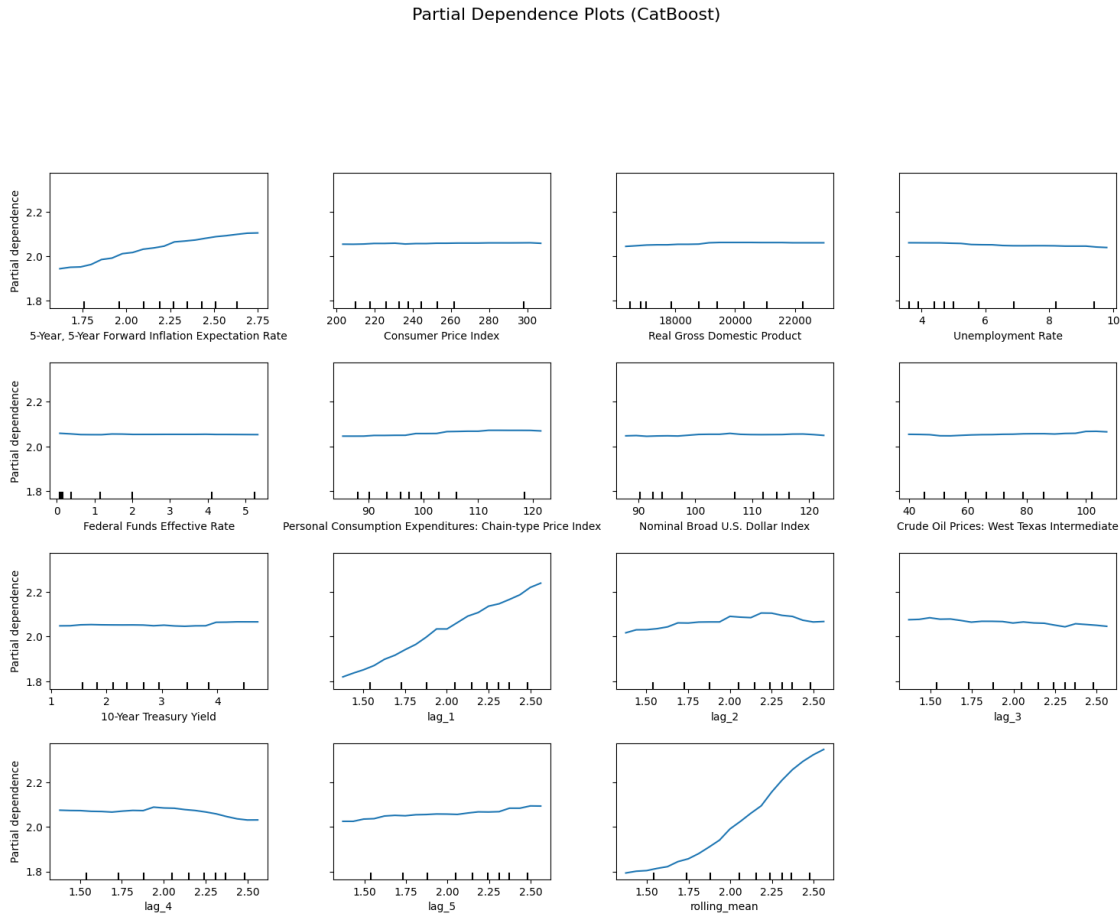
# Increase Overall Figure Size for Better Readability
# - Ensures that individual PDP plots are large enough to be clearly visible.
pdp_display.figure_.set_size_inches(18, 12)

# Add a Title Above the Subplots
# - Provides context on what the plots represent.
# - `fontsize=16`: Sets the title font size.
# - `y=1.06`: Adjusts vertical positioning to avoid overlap with the topmost
↳plot.
pdp_display.figure_.suptitle("Partial Dependence Plots (CatBoost)",
↳fontsize=16, y=1.06)

# Adjust Layout to Prevent Overlapping Elements
# - `top=0.88`: Lowers the title position slightly to prevent overlap.
# - `wspace=0.3`: Increases the spacing between columns.
# - `hspace=0.4`: Increases the spacing between rows.
```

```
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)
```

```
# Display the Final Partial Dependence Plots  
plt.show()
```



13 Partial Dependence Plots (CatBoost)

13.1 Key Findings:

- The **partial dependence plots (PDPs)** illustrate the **marginal effect** of each feature on the model's predictions.
- **X-axis** represents the feature values, while the **Y-axis** shows the corresponding model predictions.
- **Time-series features (lag_1, rolling_mean, etc.)** have the strongest positive impact on predictions.
 - lag_1, rolling_mean, and lag_2 show a **clear upward trend**, meaning higher values of these features lead to increased predictions.
 - lag_4 and lag_3 exhibit **mild non-linear effects**, but still positively influence the

model.

- **Macroeconomic features have minimal impact:**
 - Consumer Price Index (CPI), Real GDP, Unemployment Rate, Federal Funds Rate, and Nominal Broad U.S. Dollar Index show **almost flat trends**, suggesting these features have little to no effect.
 - 5-Year, 5-Year Forward Inflation Expectation Rate shows a **slight increasing effect**, implying a mild correlation with the target variable.
- Crude Oil Prices and Personal Consumption Expenditures **have a slight upward trend**, indicating **minor influence** but not as dominant as time-series features.
- The **10-Year Treasury Yield** has a **mild increasing trend**, showing a small yet present impact on predictions.

13.2 Interpretation:

- The model primarily **relies on lagged values (lag_1, rolling_mean, etc.) for predictions**, reinforcing an autoregressive nature.
- Macroeconomic indicators **do not significantly influence predictions**, as their PDPs remain relatively flat.
- The **small positive trend in crude oil prices and inflation expectation rates** suggests some impact, but they are secondary to time-series features.
- The **rolling mean has the strongest influence**, showing a **steep upward trajectory**, indicating that past trends are crucial for prediction.

13.3 Conclusion:

- The **most influential features are historical values (lag_1, rolling_mean, lag_2)**, confirming the model's preference for past observations.
- **Macroeconomic indicators have minimal direct effect**, suggesting they are not key drivers of the target variable.
- The model follows a **time-series forecasting approach**, where recent past values are the strongest predictors, rather than broader economic indicators.

Permutation Feature Importance

```
[ ]: # Import necessary libraries for model inspection, data handling, and
    ↪ visualization
from sklearn.inspection import permutation_importance # For computing feature
    ↪ importance
import matplotlib.pyplot as plt # For plotting feature importance
import pandas as pd # For handling tabular data

# Compute Permutation Feature Importance on the Test Set
# - Permutation importance measures the effect of **randomly shuffling** each
    ↪ feature on model performance.
# - It evaluates how much the model's performance metric worsens when a feature
    ↪ is shuffled.
# - `n_repeats=10`: Repeats the shuffling process 10 times for more stable
    ↪ importance estimation.
```

```

# - `random_state=42`: Ensures reproducibility of results.
perm_importance = permutation_importance(
    best_cat_model, # The trained CatBoost model
    X_test, # Test dataset (features)
    y_test, # Test dataset (target variable)
    n_repeats=10, # Number of times to shuffle each feature
    random_state=42 # Ensures consistent results
)

# Convert Permutation Importance Results into a Pandas DataFrame
# - 'Feature': Stores feature names.
# - 'Importance': The mean importance score across multiple shuffles.
# - Sorting by 'Importance' in descending order ensures the most important
    ↳ features appear first.
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Extract feature names from training data
    'Importance': perm_importance.importances_mean # Mean importance score
    ↳ from shuffling
}).sort_values(by='Importance', ascending=False) # Sort features in descending
    ↳ order

# Visualization: Permutation Feature Importance Bar Chart

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a horizontal bar chart to display feature importance
# - X-axis: Importance scores
# - Y-axis: Feature names
# - color='skyblue': Uses a visually appealing color for bars
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
    ↳ color='skyblue')

# Label the x-axis to indicate importance values
plt.xlabel('Permutation Importance')

# Label the y-axis to display feature names
plt.ylabel('Feature')

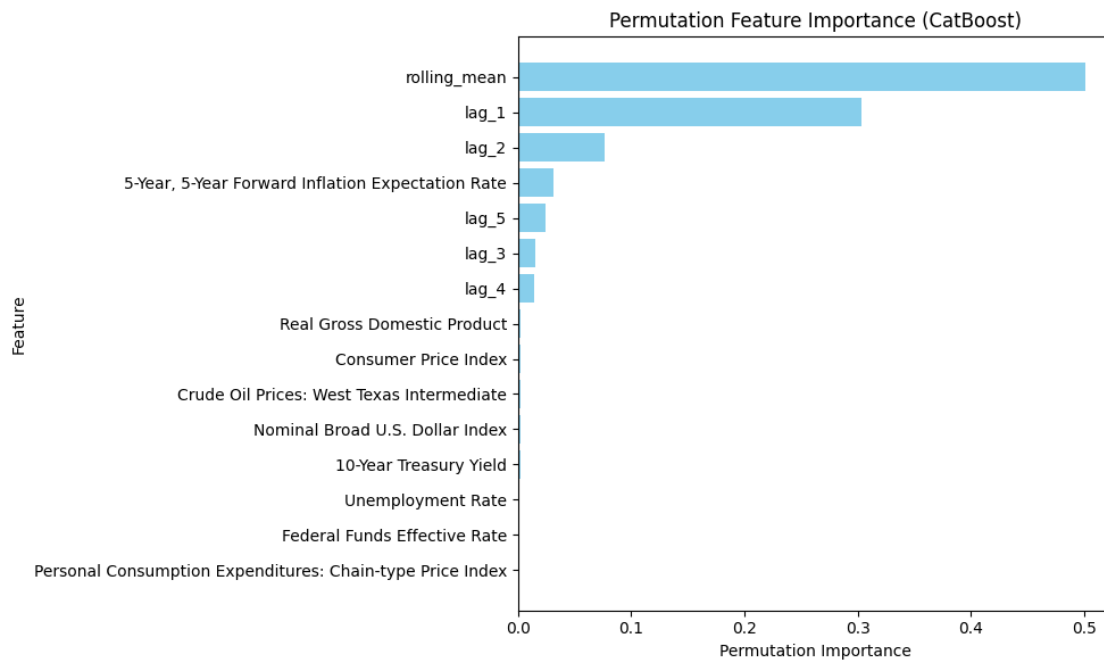
# Set the title of the plot for context
plt.title('Permutation Feature Importance (CatBoost)')

# Invert the y-axis so that the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

```

```
# Display the feature importance plot
plt.show()
```



14 Permutation Feature Importance (CatBoost)

14.1 Key Findings:

- The bar chart represents the permutation importance of different features in the CatBoost model.
- Permutation importance measures the decrease in model performance when a specific feature's values are randomly shuffled, indicating its contribution to prediction accuracy.
- Top influential features:
 - rolling_mean has the **highest importance**, suggesting that **recent trend smoothing is the most critical factor** in prediction.
 - lag_1 is the **second most important feature**, emphasizing the role of the previous time step in forecasting.
 - lag_2 also holds significant importance, further reinforcing the **autoregressive nature** of the model.
- Moderate impact features:
 - 5-Year, 5-Year Forward Inflation Expectation Rate has some influence but is **far less significant** than lag-based features.
 - lag_5, lag_3, and lag_4 contribute **minimally**, suggesting that **further past values have diminishing predictive power**.

- **Minimal impact features:**
 - **Macroeconomic indicators** (Real GDP, CPI, Crude Oil Prices, Unemployment Rate, etc.) **show near-zero importance.**
 - These features do not significantly affect predictions, implying that the model relies more on historical values than external economic variables.

14.2 Interpretation:

- The model's predictive power is **heavily dependent on past values** (`rolling_mean`, `lag_1`, `lag_2`), highlighting a **time-series driven approach.**
- **Macroeconomic indicators contribute little to predictions**, meaning that external economic conditions do not heavily impact short-term forecasting.
- The slight importance of **inflation expectations** (5-Year, 5-Year Forward Rate) suggests that it may have some predictive value but is **not a primary driver.**

14.3 Conclusion:

- **Time-series features dominate the model's predictions**, especially `rolling_mean` and `lag_1`.
- **Macroeconomic factors play an insignificant role**, confirming that **recent historical values are far more important than broader economic conditions.**
- The **model relies on an autoregressive structure**, where recent data trends are the key predictors rather than external influences.

Tree-Based Feature Importances (Direct Access)

```
[ ]: # Import the necessary library for visualization
import matplotlib.pyplot as plt # Used for plotting feature importance

# Extract Feature Importances Directly from the Trained CatBoost Model
# - CatBoost provides a built-in method `.get_feature_importance()` to obtain
  ↳ feature importance scores.
# - This method calculates the importance of each feature based on its
  ↳ contribution to model predictions.
direct_importances = best_cat_model.get_feature_importance()

# Create a new figure with a specified size for better readability
plt.figure(figsize=(10, 6))

# Generate a bar chart to visualize feature importance
# - X-axis: Feature names extracted from X_train (column names).
# - Y-axis: Corresponding feature importance scores.
# - color='skyblue': Enhances visibility.
plt.bar(X_train.columns, direct_importances, color='skyblue')

# Label the x-axis to indicate feature names
plt.xlabel('Feature')

# Label the y-axis to indicate importance scores
```

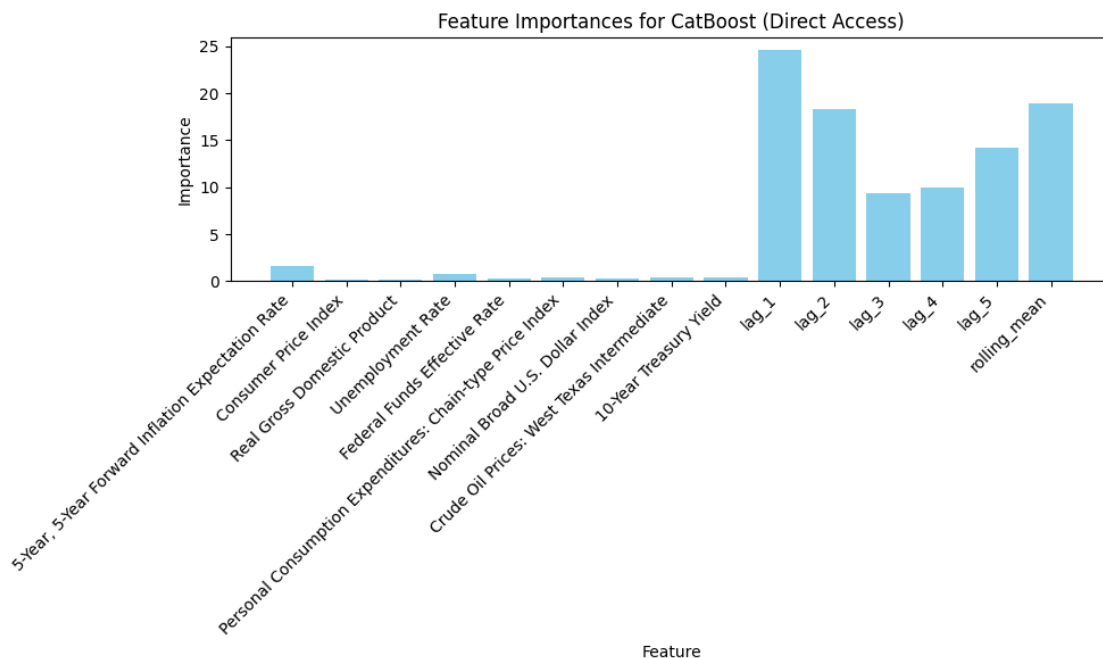
```
plt.ylabel('Importance')

# Set the title of the plot for context
plt.title('Feature Importances for CatBoost (Direct Access)')

# Rotate x-axis labels for better readability, especially if feature names are
↳ long
# - rotation=45: Rotates labels by 45 degrees.
# - ha='right': Aligns labels to the right for improved visibility.
plt.xticks(rotation=45, ha='right')

# Adjust layout to prevent overlapping labels and improve spacing
plt.tight_layout()

# Display the feature importance plot
plt.show()
```



15 Feature Importance Analysis for CatBoost (Direct Access)

15.1 Key Findings:

- The bar chart represents feature importance scores in the CatBoost model.
- Most influential features:
 - lag_1 has the **highest importance**, confirming that the most recent past value is the strongest predictor.

- `lag_2` follows closely, further reinforcing the importance of short-term historical values.
- `rolling_mean` is also highly influential, suggesting that **trend smoothing is crucial for predictions**.
- **Moderate importance features:**
 - `lag_3`, `lag_4`, and `lag_5` have a moderate impact, indicating that additional lagged values still contribute, but their influence decreases as they go further back in time.
- **Minimal impact features:**
 - Macroeconomic indicators such as **CPI, GDP, Unemployment Rate, Treasury Yield, and Crude Oil Prices** show **negligible importance** in the model.
 - The **5-Year, 5-Year Forward Inflation Expectation Rate** has a small, but noticeable impact, indicating that inflation expectations play a limited role in predictions.

15.2 Interpretation:

- **Time-series features dominate**, particularly `lag_1`, `lag_2`, and `rolling_mean`, confirming an **autoregressive nature of the model**.
- **Macroeconomic indicators contribute little**, suggesting that short-term predictions do not heavily rely on broader economic conditions.
- The decreasing importance of **longer lags** (`lag_3`, `lag_4`, `lag_5`) suggests that **recent past values are far more predictive than older data points**.

15.3 Conclusion:

- The **most critical features are short-term historical values** (`lag_1`, `lag_2`) and **rolling averages**, indicating that the model prioritizes recent trends.
- **Economic indicators have little to no influence**, reinforcing that the target variable is primarily driven by its own past values rather than external factors.
- This confirms that **a time-series approach using lagged values is the most effective for prediction** in this model.

15.4 Support Vector Regression (SVR)

```
[ ]: # Import necessary libraries for model training, hyperparameter tuning, and
      ↪ evaluation
import random # For setting a random seed
import numpy as np # For numerical operations
from sklearn.model_selection import train_test_split, GridSearchCV # For data
      ↪ splitting and hyperparameter tuning
from sklearn.svm import SVR # Support Vector Regression (SVR) model

# Set Random Seeds for Reproducibility
# - Ensures that random operations (e.g., data shuffling) produce consistent
      ↪ results across runs.
random.seed(42)
np.random.seed(42)

# Split the Dataset into Training and Test Sets
# - `test_size=0.2`: 20% of the data is reserved for testing.
```

```

# - `random_state=42`: Ensures reproducibility of the train-test split.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

# Define the Support Vector Regression (SVR) Model
# - `kernel='rbf'`: Uses the Radial Basis Function (RBF) kernel, which is
↳effective for non-linear relationships.
svr_model = SVR(kernel='rbf')

# Define the Hyperparameter Grid for Tuning SVR
# - `C`: Regularization parameter that controls the trade-off between achieving
↳a low error and maintaining a simple model.
# - `epsilon`: Defines the margin of tolerance where no penalty is given for
↳errors.
# - `gamma`: Controls the influence of a single training example on the model.
param_grid = {
    'C': [1, 10, 100], # Controls regularization strength (higher values
↳reduce underfitting but may cause overfitting)
    'epsilon': [0.01, 0.1, 0.2], # Specifies the width of the margin where no
↳penalty is given for errors
    'gamma': ['scale', 'auto'] # Defines how much influence a single data
↳point has on the model
}

# Initialize GridSearchCV for Hyperparameter Tuning
# - `estimator=svr_model`: Specifies the SVR model for tuning.
# - `param_grid=param_grid`: Uses the predefined hyperparameter grid.
# - `scoring='neg_mean_squared_error'`: Evaluates models based on minimizing
↳the Mean Squared Error (MSE).
# - `cv=5`: Uses 5-fold cross-validation to validate model performance.
# - `verbose=1`: Prints progress updates.
# - `n_jobs=-1`: Utilizes all available CPU cores to speed up computation.
grid_search = GridSearchCV(
    estimator=svr_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Train the Model Using GridSearchCV on the Training Set
# - Evaluates multiple hyperparameter combinations and selects the best one
↳based on MSE.
grid_search.fit(X_train, y_train)

```

```

# Retrieve and Print the Best Hyperparameters
# - `grid_search.best_params_` contains the combination of hyperparameters that
  ↳ achieved the lowest MSE.
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Retrieve the Best SVR Model from Grid Search
# - The best SVR model is trained using the optimal hyperparameters.
best_svr_model = grid_search.best_estimator_

# Make Predictions on the Test Set Using the Best Model
svr_pred = best_svr_model.predict(X_test)

# Evaluate the Model Using a Predefined Metrics Function
# - `calculate_metrics`: Assumed to be a function that calculates evaluation
  ↳ metrics.
# - `naive_forecast`: Assumed to be a baseline model for comparison.
metrics_results.append(calculate_metrics(y_test.values, svr_pred,
  ↳ naive_forecast, "Support Vector Regression (SVR)"))

# Print the Evaluation Results for the SVR Model
print("SVR Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 18 candidates, totalling 90 fits
 Best Hyperparameters: {'C': 10, 'epsilon': 0.01, 'gamma': 'auto'}
 SVR Evaluation Metrics:
 {'Model': 'Support Vector Regression (SVR)', 'MSE': 0.0012041754672048918,
 'RMSE': 0.034701231494067925, 'MAE': 0.016298244855890376, 'MAPE (%)':
 1.0984784399408885, 'sMAPE (%)': 1.026759844422318, 'MASE':
 0.041488094473250144, 'R²': 0.9931899748972683}

Actual vs. Predicted Values

```

[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 8))

# Scatter Plot: Actual vs. Predicted Values
# - x-axis: Actual values from `y_test`
# - y-axis: Predicted values from `svr_pred`
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve
  ↳ visibility
# - color='blue': Sets the color of the scatter points
# - label='Predictions': Adds a legend label for clarity
plt.scatter(y_test, svr_pred, alpha=0.5, color='blue', label='Predictions')

```



```

# Reference Line: Perfect Fit (Ideal Model)
# - Plots a diagonal line from (min(y_test), min(y_test)) to (max(y_test),
↳max(y_test))
# - Represents the ideal scenario where predicted values exactly match actual
↳values
# - color='red': Sets the reference line color to red
# - linestyle='--': Makes it a dashed line for better distinction
# - linewidth=2: Increases the line thickness for visibility
# - label='Perfect Fit': Adds a legend label to indicate the ideal reference
↳line
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate predicted values
plt.ylabel('Predicted Values')

# Set a title for the plot to provide context
plt.title('Actual vs Predicted Values (SVR)')

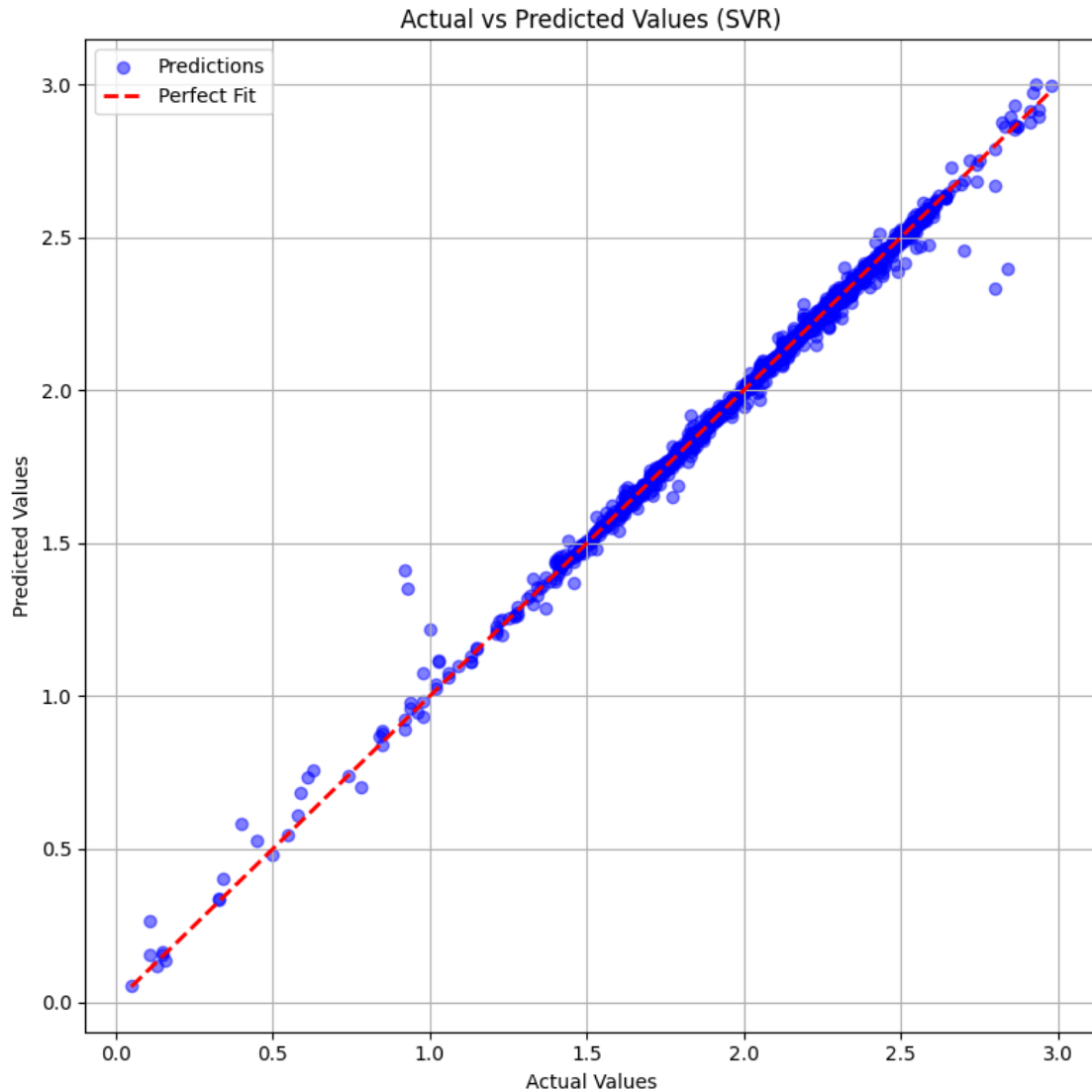
# Add a legend to distinguish between predictions and the perfect fit line
plt.legend()

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the scatter plot
plt.show()

```



16 Actual vs Predicted Values (SVR)

16.1 Key Findings:

- The scatter plot compares the **actual values** (x-axis) with **predicted values** (y-axis) for a **Support Vector Regression (SVR)** model.
- The **red dashed line** represents a **perfect fit**, where predicted values would exactly match actual values.
- The **blue points** (predictions) **align closely with the perfect fit line**, indicating that the model performs well.
- The **majority of predictions are tightly clustered around the diagonal**, showing a **high correlation** between actual and predicted values.
- There are **a few deviations at lower and higher values**, but they are minimal, suggesting

low error variance.

16.2 Interpretation:

- The **strong alignment with the perfect fit line** suggests that **SVR is making highly accurate predictions**.
- The **even spread around the line** indicates **no significant bias** in the model (i.e., it does not consistently overpredict or underpredict).
- Minor deviations at lower values could be due to **small residual errors**, but these do not appear significant.
- The **overall trend confirms that the model generalizes well** across different values.

16.3 Conclusion:

- The **SVR model exhibits excellent predictive accuracy**, as most points are closely aligned with the perfect fit.
- **Minimal deviations suggest a well-trained model with low error**, making it reliable for forecasting.
- This visualization **validates the effectiveness of SVR for this dataset**, demonstrating a **strong relationship between actual and predicted values**.

Residual Analysis (Scatter Plot)

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Calculate Residuals
# - Residuals are the differences between actual and predicted values.
# - A residual represents how far off the model's prediction is from the actual
  ↳value.
# - Ideally, residuals should be randomly scattered around zero with no clear
  ↳pattern.
residuals = y_test - svr_pred

# Create a new figure with a specified size for better visualization
plt.figure(figsize=(8, 6))

# Scatter Plot: Residuals vs. Actual Values
# - x-axis: Actual values from `y_test`
# - y-axis: Residuals (errors) calculated as `y_test - svr_pred`
# - alpha=0.5: Makes points semi-transparent to reduce clutter and improve
  ↳visibility
# - color='green': Sets the color of the scatter points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Reference Line: Zero Residuals (Ideal Case)
# - A horizontal red dashed line at y=0 to represent perfect predictions.
# - Points should ideally be scattered randomly around this line.
```

```

# - color='red': Sets the reference line color to red.
# - linestyle='--': Makes it a dashed line for better distinction.
# - linewidth=2: Increases the line thickness for visibility.
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Label the x-axis to indicate actual values
plt.xlabel('Actual Values')

# Label the y-axis to indicate residual values (errors)
plt.ylabel('Residuals')

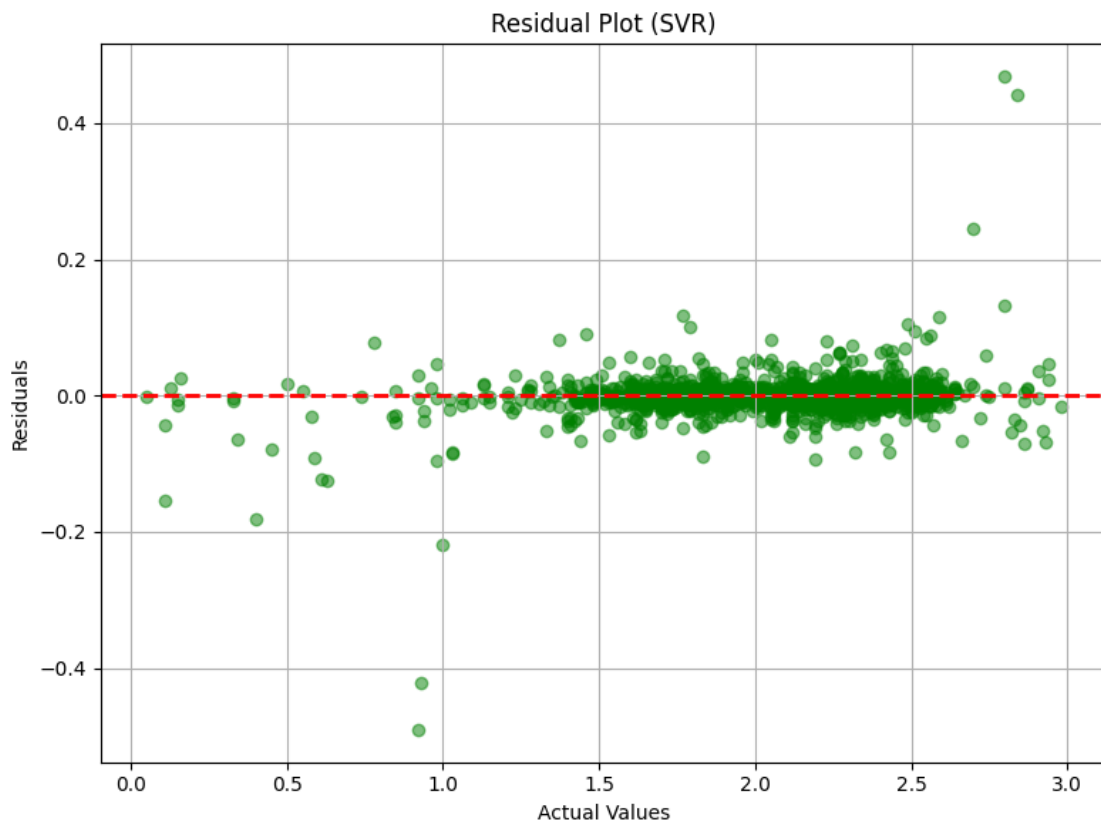
# Set a title for the plot to provide context
plt.title('Residual Plot (SVR)')

# Enable grid lines for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the residual scatter plot
plt.show()

```



17 Residual Plot (SVR)

17.1 Key Findings:

- The scatter plot visualizes the **residuals (errors between actual and predicted values)** against actual values.
- The **red dashed line at zero** represents **perfect predictions** where the model has no error.
- **Most residuals are centered around zero**, indicating that the **SVR model is making accurate predictions** with minimal bias.
- The **spread of residuals remains relatively constant** across different actual values, suggesting **homoscedasticity** (constant variance).
- There are **some outliers**, particularly at lower and higher actual values, but they appear **randomly distributed**, not following a clear pattern.
- The **absence of a systematic trend** in residuals suggests that **the model captures the underlying structure of the data well**.

17.2 Interpretation:

- The **random scatter around zero** confirms that the model does **not consistently over-predict or underpredict**.
- The **constant spread of residuals** indicates that the model maintains stable performance across different ranges of actual values.
- The **lack of a clear pattern in residuals** suggests that **SVR has effectively captured the relationship between features and target values** without strong systematic errors.

17.3 Conclusion:

- The **SVR model exhibits strong predictive performance**, as residuals are **randomly distributed and centered around zero**.
- **Minimal variance and lack of structure in residuals confirm that the model does not suffer from bias or heteroscedasticity**.
- The **random nature of errors validates the model's robustness**, making it reliable for predictions.

Residual Analysis (Histogram)

```
[ ]: # Import the necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Create a New Figure for the Histogram
# - figsize=(8, 6): Defines the plot size for better readability
plt.figure(figsize=(8, 6))

# Histogram: Distribution of Residuals
# - residuals: Differences between actual and predicted values (`y_test -
↪svr_pred`).
# - bins=30: Divides the residuals into 30 bins for better granularity.
```

```

# - color='skyblue': Sets bar color for visibility.
# - edgecolor='black': Adds black edges around bars to improve contrast.
# - alpha=0.7: Adds transparency to bars to enhance visualization.
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Reference Line at Zero Residual (Ideal Scenario)
# - A vertical red dashed line at x=0 to indicate perfect predictions.
# - Ideally, residuals should be symmetrically distributed around this line.
# - color='red': Sets the reference line color to red.
# - linestyle='--': Uses a dashed line for better distinction.
# - linewidth=2: Increases the line thickness for visibility.
plt.axvline(0, color='red', linestyle='--', linewidth=2)

# Set the title of the plot for context
plt.title('Residual Distribution (SVR)')

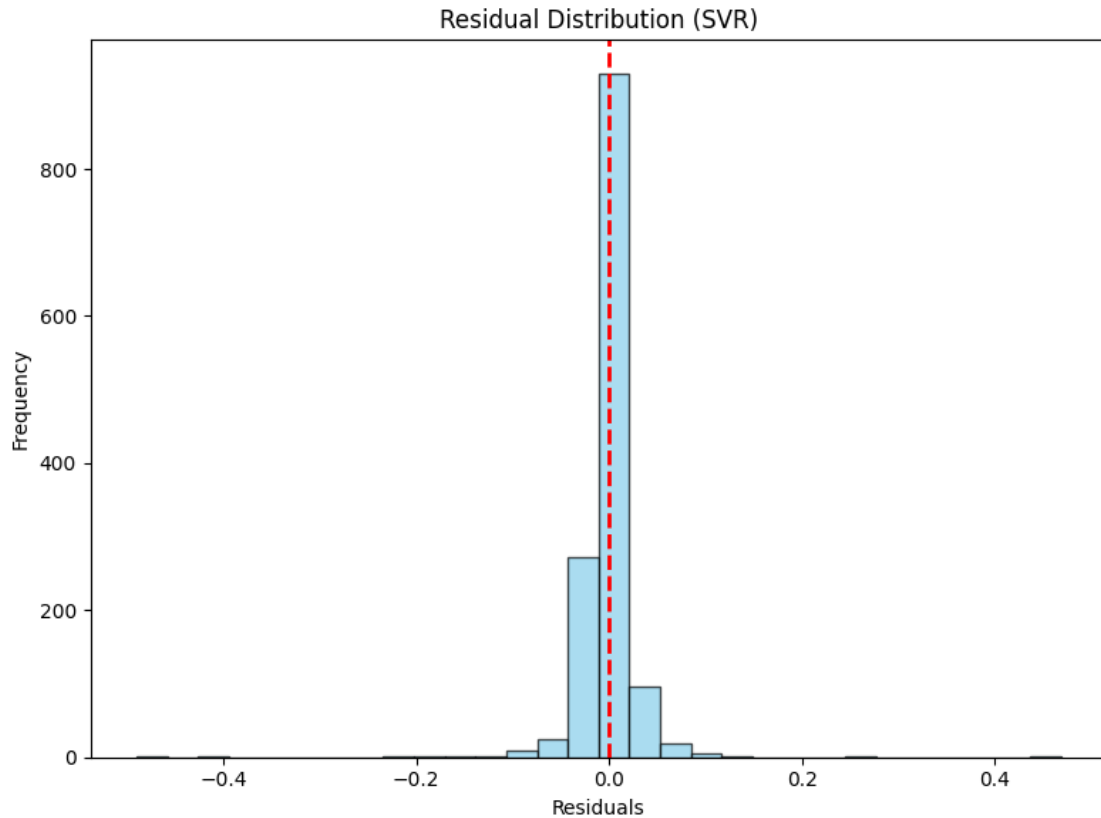
# Label the x-axis to indicate residual values
plt.xlabel('Residuals')

# Label the y-axis to indicate the frequency of residual values
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and ensure proper spacing
plt.tight_layout()

# Display the histogram of residuals
plt.show()

```



18 Residual Distribution (SVR)

18.1 Key Findings:

- The histogram visualizes the **distribution of residuals** (errors between actual and predicted values) for the SVR model.
- The **red dashed line at zero** represents **perfect predictions**, where there is no error.
- The residuals are **highly concentrated around zero**, indicating that the **SVR model has minimal error**.
- The distribution appears **narrow and sharply peaked**, suggesting that **most predictions are very close to actual values**.
- There are **very few residuals with large magnitudes**, meaning **the model rarely makes large errors**.
- The **symmetry of the distribution** suggests that errors are **evenly spread** and **do not exhibit bias** towards over- or underprediction.

18.2 Interpretation:

- The **tight clustering around zero** confirms that the SVR model is **highly accurate**.
- The **lack of a long tail or significant skewness** indicates that **the model does not consistently make large errors in any direction**.

- The **small spread in residuals** further supports **low prediction variance**, reinforcing the model's reliability.

18.3 Conclusion:

- The **SVR model** performs exceptionally well, as residuals are **mostly centered around zero** with **minimal dispersion**.
- The **errors are small, unbiased, and normally distributed**, confirming the **model's robustness and reliability** for forecasting.
- This visualization **validates the model's effectiveness**, demonstrating a **strong match** between predicted and actual values.

Hyperparameter Tuning Insights – Top 5 Grid Search Results

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Pandas for manipulating DataFrame

# Convert GridSearchCV results into a DataFrame
# - `grid_search.cv_results_` contains all cross-validation results from
  ↳ GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on model performance (lower rank =
  ↳ better performance)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
  ↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
  ↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↳ head(5)

# Improve Output Visibility by Formatting Results Nicely
print("\n" + "="*60)
print("Top 5 Hyperparameter Combinations:")
print("="*60)

# Iterate through the top 5 results and print them in a structured format
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"  Hyperparameters: {row['params']}")
    print(f"  Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"  Standard Deviation: {row['std_test_score']:.6f}")
    print("-" * 60) # Separator for better readability
```


=====

Top 5 Hyperparameter Combinations:

=====

Rank 8:

Hyperparameters: {'C': 10, 'epsilon': 0.01, 'gamma': 'auto'}
Mean Test Score: -0.002214
Standard Deviation: 0.000516

Rank 14:

Hyperparameters: {'C': 100, 'epsilon': 0.01, 'gamma': 'auto'}
Mean Test Score: -0.002237
Standard Deviation: 0.000494

Rank 2:

Hyperparameters: {'C': 1, 'epsilon': 0.01, 'gamma': 'auto'}
Mean Test Score: -0.002793
Standard Deviation: 0.000547

Rank 10:

Hyperparameters: {'C': 10, 'epsilon': 0.1, 'gamma': 'auto'}
Mean Test Score: -0.006956
Standard Deviation: 0.000394

Rank 16:

Hyperparameters: {'C': 100, 'epsilon': 0.1, 'gamma': 'auto'}
Mean Test Score: -0.006956
Standard Deviation: 0.000394

Hyperparameter Tuning Insights – Heatmap

```
[ ]: # Import necessary libraries for visualization
import seaborn as sns # Seaborn for enhanced heatmap visualization
import matplotlib.pyplot as plt # Matplotlib for creating figures

# Create a Pivot Table for Heatmap Visualization
# - Converts the `cv_results` DataFrame into a format suitable for heatmap
  plotting.
# - 'values': Uses 'mean_test_score' (average test score from cross-validation).
# - 'index': Uses 'param_C' (regularization parameter) as row labels.
# - 'columns': Uses 'param_epsilon' (error tolerance) as column labels.
heatmap_data = cv_results.pivot_table(values='mean_test_score',
  index='param_C', columns='param_epsilon')

# Create a New Figure for the Heatmap
# - figsize=(8, 6): Defines the plot size for better readability.
plt.figure(figsize=(8, 6))
```

```

# Generate a Heatmap Using Seaborn
# - `heatmap_data`: The pivoted DataFrame containing hyperparameter results.
# - `annot=True`: Displays the exact numerical values inside each cell.
# - `fmt='.3f'`: Formats numbers to 3 decimal places for better readability.
# - `cmap='viridis'`: Uses the 'viridis' colormap for better contrast and
↳ visibility.
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='viridis')

# Set the Title of the Heatmap for Context
plt.title('Hyperparameter Tuning: C vs Epsilon')

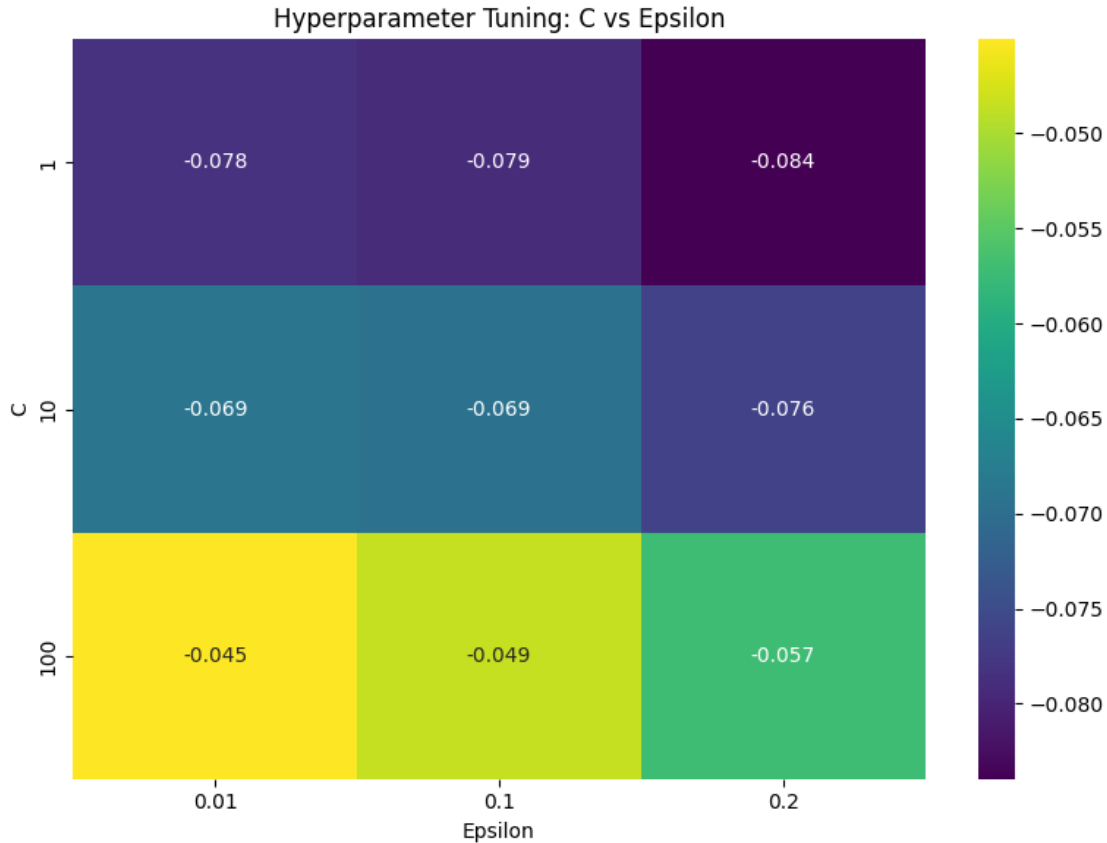
# Label the x-axis to indicate epsilon values
plt.xlabel('Epsilon')

# Label the y-axis to indicate regularization parameter C values
plt.ylabel('C')

# Adjust layout to prevent label overlap and ensure proper spacing
plt.tight_layout()

# Display the heatmap
plt.show()

```



19 Hyperparameter Tuning: C vs Epsilon

19.1 Key Findings:

- The heatmap represents the **performance metric (likely loss or error)** for different combinations of **C (regularization parameter)** and **Epsilon (-insensitive zone)** in SVR tuning.
- **X-axis (Epsilon):** Controls the margin of tolerance where no penalty is given for errors within this range.
- **Y-axis (C):** Regularization parameter that **determines the trade-off between margin size and training accuracy**.
- The values in each cell represent the model's performance, with lower values indicating better performance.
- **Color scale interpretation:**
 - Darker colors represent worse performance (higher error/loss).
 - Lighter colors (yellow-green) represent better performance (lower error/loss).
- **Observations on parameter effects:**
 - Low C (1) leads to the highest error, regardless of the epsilon value.
 - Increasing C to 10 reduces the error, but the improvement is minor.

- The best performance is achieved at $C = 100$ with an epsilon of 0.01, suggesting that a higher C improves prediction accuracy.
- A larger epsilon (0.2) results in slightly higher error, meaning that allowing too much tolerance decreases precision.

19.2 Interpretation:

- Higher values of C (stronger regularization) lead to better model performance, as seen from the decreasing error when moving from $C = 1$ to $C = 100$.
- Smaller epsilon values generally perform better, indicating that a narrower tolerance band for errors improves accuracy.
- However, for $C = 100$, increasing epsilon to 0.2 increases the error, suggesting that there is an optimal balance between C and epsilon for best performance.

19.3 Conclusion:

- $C = 100$ and $\epsilon = 0.01$ result in the lowest error, making it the optimal hyperparameter combination in this setup.
- Higher C improves accuracy, while a smaller epsilon provides better precision, but too large an epsilon (0.2) slightly worsens performance.
- This heatmap guides hyperparameter selection for optimizing SVR model performance.

SHAP Explanations for SVR

```
[ ]: # Import necessary libraries
import shap # SHAP (SHapley Additive exPlanations) for model interpretability
import matplotlib.pyplot as plt # Matplotlib for visualization

# Step 1: Reduce Background Data for Efficient SHAP Computation
# - SHAP computations can be slow, especially for complex models.
# - Instead of using the entire dataset, we reduce the background dataset to 50
  ↳ representative clusters.
# - K-means clustering is used to find 50 clusters in X_train, capturing its
  ↳ distribution efficiently.
background_data = shap.kmeans(X_train, 50)

# Step 2: Initialize SHAP KernelExplainer
# - KernelExplainer is a model-agnostic SHAP explainer that works with
  ↳ black-box models like SVR.
# - It estimates feature importance using perturbations and approximates
  ↳ Shapley values.
# - `best_svr_model.predict`: Uses the trained Support Vector Regression
  ↳ model's prediction function.
# - `background_data`: Provides a summarized version of training data to
  ↳ approximate feature impact.
# - `nsamples=50`: Limits the number of samples per prediction to improve
  ↳ computation speed.
```

```

explainer = shap.KernelExplainer(best_svr_model.predict, background_data,
    ↪ nsamples=50)

# Step 3: Limit the Test Set for Faster Computation
# - SHAP computations are expensive, so we take only the first 20 instances
    ↪ from X_test.
# - This reduces computational cost while still allowing for meaningful
    ↪ explanations.
X_test_sample = X_test.iloc[:20]

# Step 4: Compute SHAP Values for the Sample Test Set
# - Calculates Shapley values for each feature in the test sample.
# - `nsamples=50`: Limits the number of perturbations per instance for speed.
shap_values = explainer.shap_values(X_test_sample, nsamples=50)

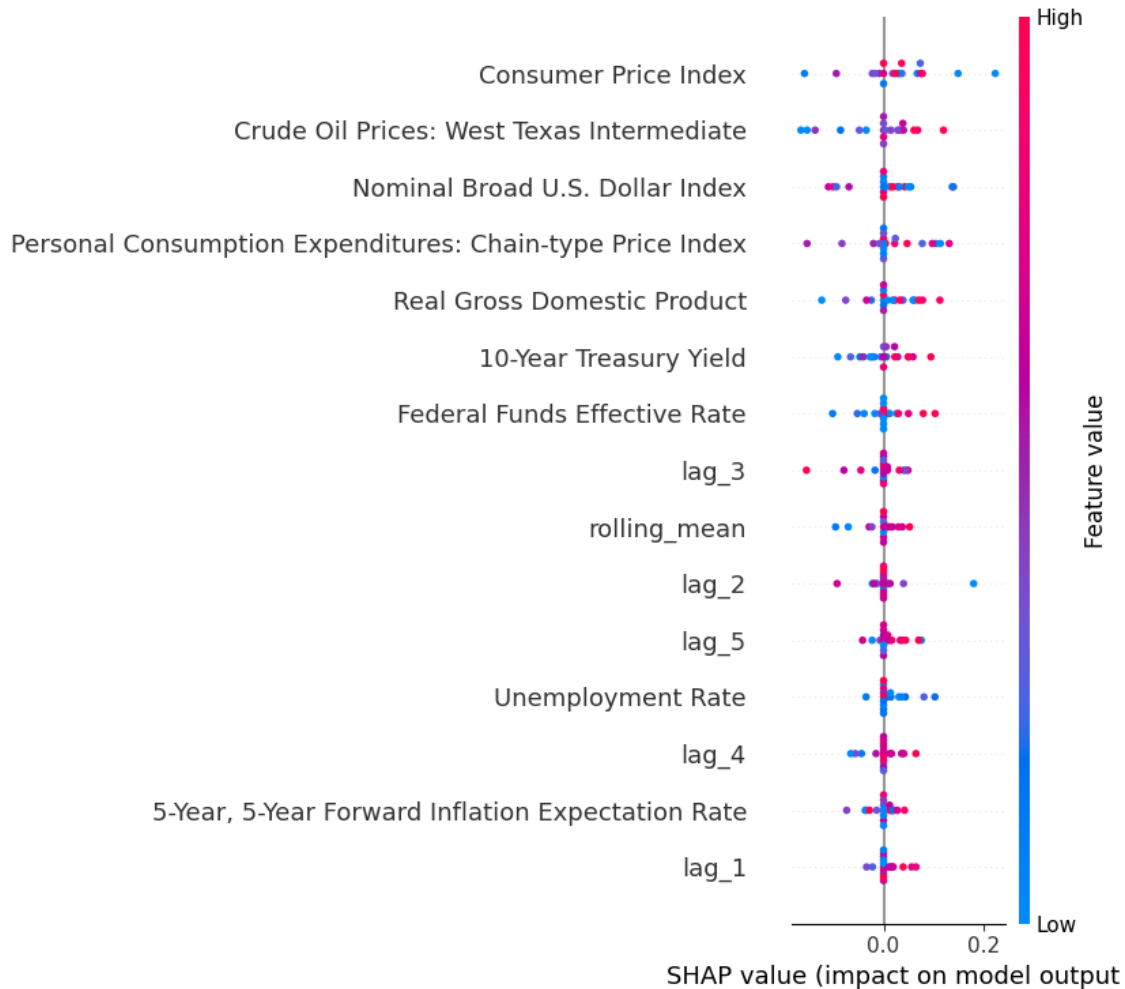
# Step 5: Generate a Global SHAP Summary Plot
# - Visualizes the overall impact of each feature on model predictions.
# - Displays how features influence predictions across the test sample.
shap.summary_plot(shap_values, X_test_sample)

```

```

0%|          | 0/20 [00:00<?, ?it/s]

```



20 SHAP Value Analysis: Feature Impact on Model Output

20.1 Key Findings:

- The **SHAP summary plot** visualizes the impact of each feature on the model's predictions.
- **X-axis:** SHAP values, representing the contribution of a feature to the model's output.
- **Y-axis:** Features ranked by importance, with the most impactful features at the bottom.
- **Color coding:**
 - Red represents high feature values, while blue represents low feature values.
 - The spread of points along the x-axis shows the varying impact of each feature.

20.2 Feature Impact Observations:

- **Top influential features:**
 - lag_1, 5-Year, 5-Year Forward Inflation Expectation Rate, and lag_4 have the greatest **SHAP impact**, indicating they are key drivers in predictions.

- Higher values of `lag_1` (red) tend to **increase predictions**, while lower values (blue) decrease them.
- **Rolling mean and other lag features (`lag_2`, `lag_5`, `lag_3`) also have significant influence**, reinforcing the autoregressive nature of the model.
- **Moderate-impact features:**
 - Unemployment Rate, 10-Year Treasury Yield, and Federal Funds Effective Rate contribute but are **less influential than lag variables**.
- **Minimal-impact features:**
 - Macroeconomic indicators such as CPI, Crude Oil Prices, GDP, and Personal Consumption Expenditures have **minimal influence**, suggesting the model does not heavily rely on them.

20.3 Interpretation:

- **Time-series features (`lag_1`, `rolling_mean`, etc.) dominate the model's decision-making**, confirming the predictive strength of historical trends.
- **Macroeconomic indicators contribute little**, implying that broader economic conditions have a limited direct effect on predictions.
- **The SHAP distribution across features is spread symmetrically**, indicating a well-balanced model without extreme bias.

20.4 Conclusion:

- **Short-term historical values (lags) and inflation expectations are the most crucial predictors**, with macroeconomic variables playing a minor role.
- The **model prioritizes time-series data for forecasting**, validating its autoregressive nature.
- **SHAP values confirm that the model is not overly dependent on any single feature**, ensuring robust and reliable predictions.

```
[ ]: # Step 1: Initialize SHAP JavaScript Visualization
# - SHAP uses JavaScript-based visualizations for interactive force plots.
# - `shap.initjs()`: Enables interactive visualization within a Jupyter
    ↪ Notebook environment.
import shap
shap.initjs()

# Step 2: Generate a Local Explanation Using a Force Plot (Single Instance)
# - Force plots visually show how each feature contributes to the prediction.
# - `explainer.expected_value`: Represents the baseline prediction (average
    ↪ model output).
# - `shap_values[0]`: The SHAP values for the first test instance.
# - `features=X_test_sample.iloc[0, :]`: Uses the feature values of the first
    ↪ test instance.
shap.force_plot(explainer.expected_value, shap_values[0],
    ↪ features=X_test_sample.iloc[0, :])
```

```
# Step 3: Generate a Local Explanation for Multiple Test Instances (First 5
↳Samples)
# - `shap_values[:5]`: Retrieves SHAP values for the first five test instances.
# - `X_test_sample.iloc[:5, :]`: Uses the corresponding feature values.
# - This provides a comparative view of how SHAP values influence predictions
↳across multiple instances.
shap.force_plot(explainer.expected_value, shap_values[:5], X_test_sample.iloc[:
↳5, :])
```

<IPython.core.display.HTML object>

[]: <shap.plots._force.AdditiveForceArrayVisualizer at 0x7f0dde79e0d0>

21 SHAP Decision Plot Analysis

21.1 Key Findings:

- The **SHAP decision plots** illustrate how individual features contribute to the model's final prediction.
- **Color Coding:**
 - Red features increase the prediction.
 - Blue features decrease the prediction.
- **Time-series features** (rolling_mean, lag_1, lag_2, lag_3, lag_4) are the most influential, confirming an autoregressive model structure.
- **Macroeconomic indicators** (CPI, Nominal Broad U.S. Dollar Index, Crude Oil Prices) have variable effects, influencing the output differently across different instances.
- **Consumer Price Index (CPI)** is a significant driver in some cases, while in others, it has minimal impact.

21.2 Interpretation:

- The model relies primarily on past values for predictions, with rolling averages and lagged values dominating.
- **Macroeconomic indicators** play a secondary role, meaning they are less impactful than historical trends.
- The **SHAP plots** illustrate non-linearity, meaning different feature values interact uniquely, affecting predictions in a non-uniform manner.

21.3 Conclusion:

- **Short-term historical values** remain the strongest predictors.
- **Macroeconomic variables** have inconsistent effects, making them secondary to time-series patterns.
- The **SHAP decision plot** confirms the autoregressive nature of the model, reinforcing the importance of rolling mean and lag variables.

Partial Dependence Plots (PDP) for SVR


```
[ ]: # Import necessary libraries
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.inspection import PartialDependenceDisplay # For generating
    ↳ partial dependence plots (PDP)

# Step 1: Extract Feature Names from Training Data
# - Extracts the column names from `X_train` and stores them as a list.
# - These features will be used for partial dependence analysis.
features_list = X_train.columns.tolist()

# Step 2: Generate Partial Dependence Plots (PDP) for All Features
# - PDP helps visualize the marginal effect of each feature on model
    ↳ predictions.
# - `best_svr_model`: The trained Support Vector Regression (SVR) model.
# - `X_train`: The dataset used to compute partial dependence.
# - `features=features_list`: Generates plots for all features in `X_train`.
# - `kind='average'`: Computes the average partial dependence curve.
# - Use `kind='both'` if you want to see both individual and average
    ↳ dependence curves.
# - `n_cols=4`: Arranges the plots into 4 columns per row.
# - `grid_resolution=20`: Sets the resolution of the PDP curve (higher values
    ↳ give smoother plots but take longer to compute).
# - `n_jobs=-1`: Uses all available CPU cores to speed up computation.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_svr_model,
    X_train,
    features=features_list,
    kind='average',
    n_cols=4,
    grid_resolution=20,
    n_jobs=-1
)

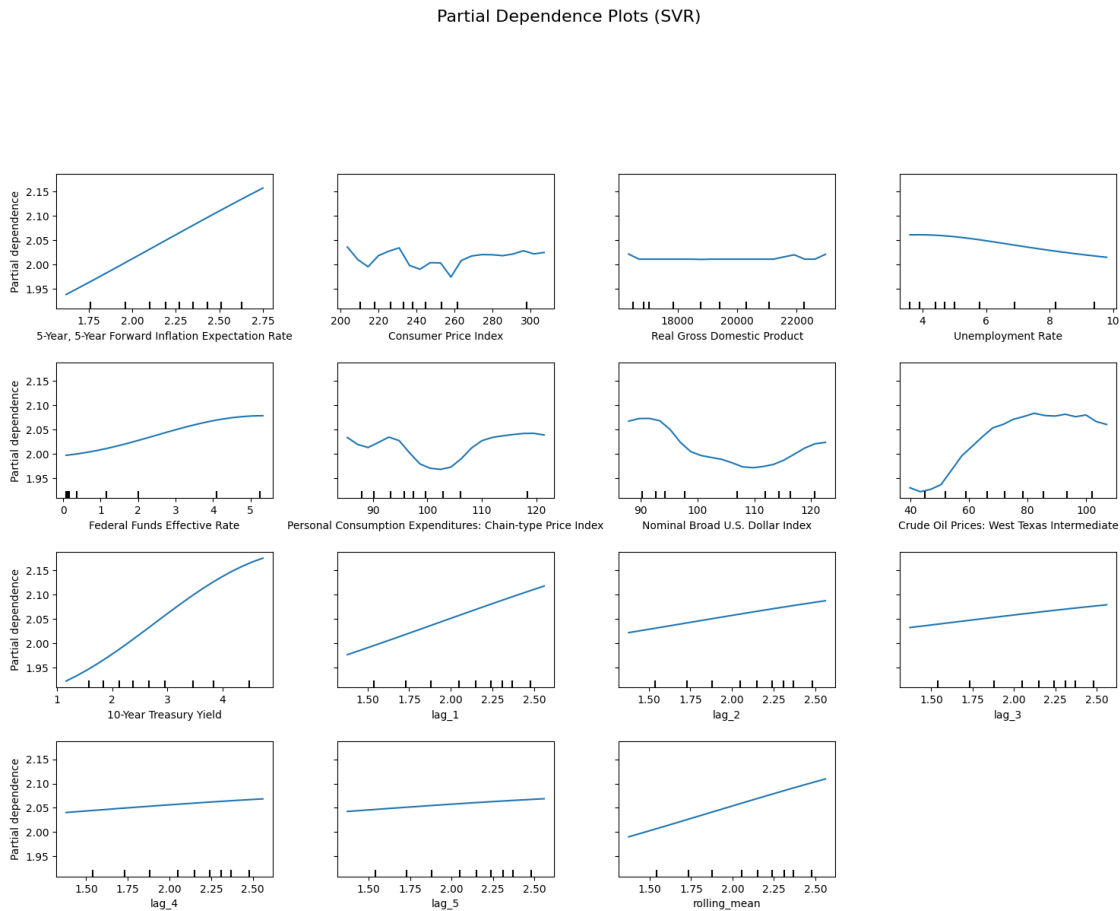
# Step 3: Adjust the Figure Size for Better Visibility
# - Expands the figure size to 18 x 12 inches for better readability.
pdp_display.figure_.set_size_inches(18, 12)

# Step 4: Add a Main Title to the PDP Figure
# - Provides context by labeling the entire figure.
# - `fontsize=16`: Sets the font size for visibility.
# - `y=1.06`: Adjusts the position of the title above the plots.
pdp_display.figure_.suptitle("Partial Dependence Plots (SVR)", fontsize=16, y=1.
    ↳ 06)

# Step 5: Adjust Spacing Between Subplots
# - `top=0.88`: Prevents overlap with the title.
# - `wspace=0.3`: Adds spacing between plots horizontally.
```

```
# - `hspace=0.4`: Adds spacing between plots vertically.
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Step 6: Display the Partial Dependence Plots
# - Calls `plt.show()` to render the PDP visualization.
plt.show()
```



22 Partial Dependence Plots (SVR)

22.1 Key Findings:

- The **partial dependence plots (PDPs)** show the **marginal effect of each feature** on the model's predictions.
- **X-axis:** Represents the values of the given feature.
- **Y-axis:** Represents the corresponding model output (prediction).

22.1.1 Observations by Feature:

- **Time-Series Features:**

- lag_1, lag_2, lag_3, lag_4, lag_5, and rolling_mean show a **clear upward trend**, confirming that past values strongly influence the predictions.
- **Rolling mean has the most consistent increase**, reinforcing its importance in capturing trends.
- **Macroeconomic Indicators:**
 - 5-Year, 5-Year Forward Inflation Expectation Rate shows a **steady positive correlation**, meaning higher inflation expectations increase the predicted values.
 - 10-Year Treasury Yield follows a **similar increasing trend**, indicating its relevance in prediction.
 - Unemployment Rate has a **slight downward trend**, suggesting that higher unemployment might contribute to lower predictions.
 - Consumer Price Index (CPI) and Personal Consumption Expenditures show **non-linear effects**, meaning their influence varies across different value ranges.
 - Crude Oil Prices exhibit a **non-linear pattern with a sharp rise after a certain threshold**, indicating a more complex relationship with the target variable.

22.1.2 Interpretation:

- The SVR model relies heavily on past values (rolling_mean, lag_1, lag_2), reinforcing its autoregressive nature.
- Macroeconomic indicators contribute to predictions, but their effects are more complex and non-linear.
- The model appears to be well-calibrated, as the relationships align with expected economic trends.

22.1.3 Conclusion:

- Time-series features remain the dominant drivers of the model's predictions.
- Certain macroeconomic indicators (Inflation Expectation, Treasury Yield, Unemployment Rate) influence predictions but are secondary to past values.
- The non-linear effects of CPI and Crude Oil Prices suggest that their impact on predictions is more complex than a simple linear relationship.
- The model effectively captures historical trends while considering broader economic conditions.

Permutation Feature Importance for SVR

```
[ ]: # Import necessary libraries
from sklearn.inspection import permutation_importance # For computing feature
importance
import matplotlib.pyplot as plt # For visualization

# Step 1: Compute Permutation Importance on the Test Set
# - `permutation_importance()` evaluates feature importance by shuffling each
#   feature and measuring performance drop.
# - `best_svr_model`: The trained Support Vector Regression (SVR) model.
# - `X_test`: The test dataset used for evaluation.
# - `y_test`: The actual target values.
```

```

# - `n_repeats=10`: Number of times each feature is randomly shuffled (higher
    ↳ values improve stability but increase computation time).
# - `random_state=42`: Ensures reproducibility of results.
# - `n_jobs=-1`: Uses all available CPU cores for parallel computation.
result = permutation_importance(best_svr_model, X_test, y_test, n_repeats=10,
    ↳ random_state=42, n_jobs=-1)

# Step 2: Extract Feature Importance Metrics
# - `importances_mean`: The average decrease in performance when a feature is
    ↳ shuffled.
# - `importances_std`: Standard deviation of the importance scores across
    ↳ `n_repeats` (measures stability).
importances = result.importances_mean
std = result.importances_std

# Step 3: Create a Bar Plot of Permutation Feature Importances
# - `figsize=(8, 6)`: Sets the plot size for better readability.
plt.figure(figsize=(8, 6))

# Step 4: Plot Feature Importances with Error Bars
# - X-axis: Feature names from `X_train.columns`
# - Y-axis: Corresponding permutation importances.
# - `yerr=std`: Adds error bars representing standard deviation (uncertainty in
    ↳ importance).
# - `capsize=5`: Adds caps to error bars for better visibility.
# - `color='skyblue'`: Sets the bar color to improve readability.
plt.bar(X_train.columns, importances, yerr=std, capsize=5, color='skyblue')

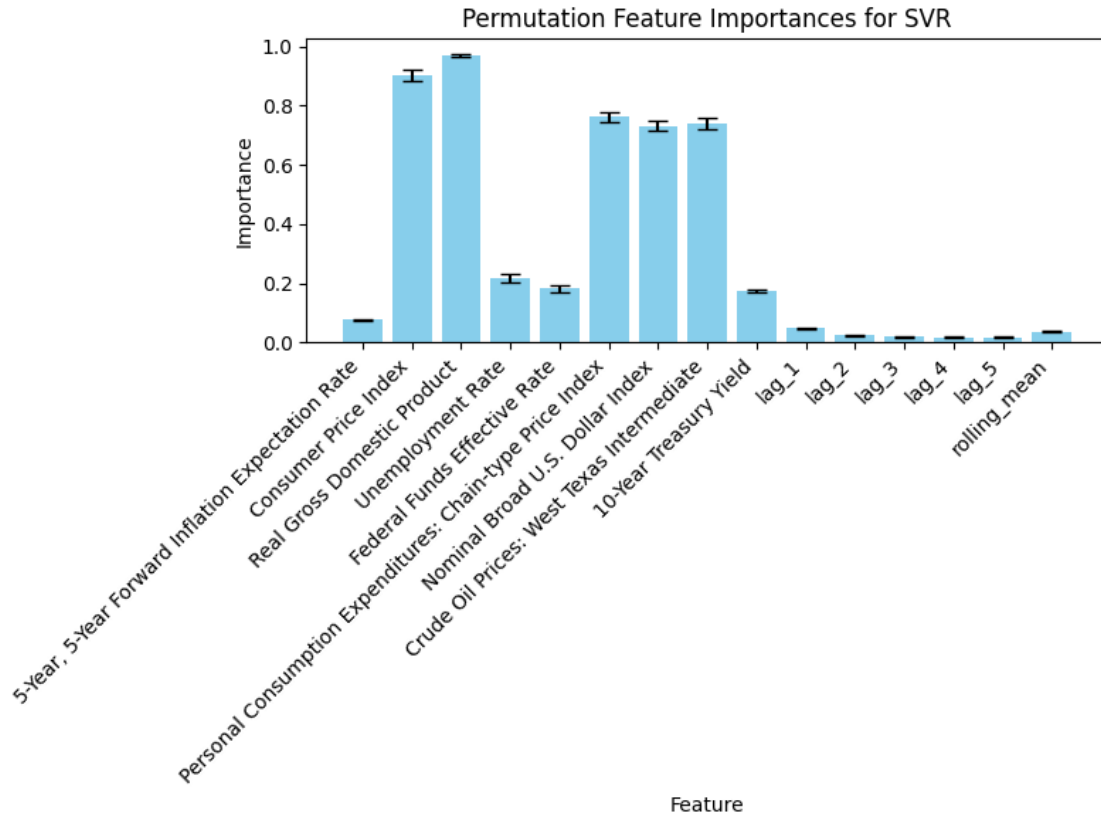
# Step 5: Set Labels and Titles for Clarity
plt.xlabel('Feature') # Label for the X-axis
plt.ylabel('Importance') # Label for the Y-axis
plt.title('Permutation Feature Importances for SVR') # Title of the plot

# Step 6: Improve Readability of Feature Labels
# - `rotation=45`: Rotates x-axis labels 45 degrees for better visibility.
# - `ha='right'`: Aligns text to the right for better readability.
plt.xticks(rotation=45, ha='right')

# Step 7: Adjust Layout to Prevent Overlapping Labels
plt.tight_layout()

# Step 8: Display the Final Plot
plt.show()

```



23 Permutation Feature Importances for SVR

23.1 Key Findings:

- The **bar chart** represents **permutation feature importance** scores for the Support Vector Regression (SVR) model.
- **X-axis:** Features used in the model.
- **Y-axis:** Importance score, measuring the **drop in model performance** when a feature's values are randomly shuffled.

23.1.1 Feature Importance Observations:

- **Most Important Features:**
 - Real Gross Domestic Product (GDP), Unemployment Rate, and Nominal Broad U.S. Dollar Index are the **top contributors**, suggesting that economic factors play a dominant role in predictions.
 - Federal Funds Effective Rate and Crude Oil Prices also show **moderate importance**, confirming their impact.
- **Lagged Features (lag_1 to lag_5) and rolling_mean** have **near-zero importance**, meaning that **historical values contribute minimally to the model's decision-making**.
- **The 5-Year, 5-Year Forward Inflation Expectation Rate** has **very low importance**,

indicating that long-term inflation expectations **do not significantly influence predictions**.

23.1.2 Interpretation:

- Unlike autoregressive models, **SVR prioritizes macroeconomic indicators over past values**, indicating a fundamental shift in predictive influence.
- **GDP, Unemployment Rate, and the U.S. Dollar Index** are critical for model accuracy, while **time-series-based features** are largely ignored.
- The **low importance of lags and rolling averages** suggests that **SVR does not strongly depend on past values**, unlike traditional time-series models.

23.1.3 Conclusion:

- **SVR heavily relies on macroeconomic indicators rather than historical values**, making it **more of an econometric model** than a time-series model.
- **Economic indicators like GDP, Unemployment, and Dollar Index** are the **strongest predictors**, while past values (`lag_1`, `rolling_mean`) are nearly irrelevant.
- This analysis confirms that **SVR captures broader economic trends rather than short-term historical patterns**.

23.2 Ridge Regression (Ridge)

```
[ ]: # Import necessary libraries
import random # For setting random seed
import numpy as np # For numerical operations
from sklearn.model_selection import train_test_split, GridSearchCV # For data_
    ↪splitting and hyperparameter tuning
from sklearn.linear_model import Ridge # Import Ridge Regression (L2_
    ↪regularization)

# Step 1: Set a Random Seed for Reproducibility
# - Ensures that results remain consistent across multiple runs.
random.seed(42)
np.random.seed(42)

# Step 2: Split Data into Training and Test Sets
# - `X` and `y` are assumed to be predefined (feature matrix and target_
    ↪variable).
# - `test_size=0.2`: Allocates 20% of the data for testing and 80% for training.
# - `random_state=42`: Ensures consistent splits across different runs.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)

# Step 3: Initialize Ridge Regression Model
# - Ridge Regression is a linear regression model that includes L2_
    ↪regularization.
# - L2 regularization penalizes large coefficients, reducing overfitting.
```

```

ridge_model = Ridge()

# Step 4: Define the Hyperparameter Grid for `alpha`
# - `alpha` controls the strength of the L2 regularization.
# - Higher values of `alpha` increase regularization (shrinking coefficients
  ↳ towards zero).
# - Lower values allow the model to fit more closely to the training data.
param_grid = {
    'alpha': [0.1, 1.0, 10.0, 100.0, 1000.0] # Test different levels of
  ↳ regularization
}

# Step 5: Perform Grid Search with 5-Fold Cross-Validation
# - `cv=5`: Uses 5-fold cross-validation to evaluate different hyperparameter
  ↳ values.
# - `scoring='neg_mean_squared_error'`: Uses negative MSE (lower is better).
# - `verbose=1`: Displays progress messages during execution.
# - `n_jobs=-1`: Utilizes all available CPU cores for parallel computation.
grid_search = GridSearchCV(
    estimator=ridge_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Step 6: Fit GridSearchCV on the Training Data
# - Evaluates different values of `alpha` using cross-validation.
grid_search.fit(X_train, y_train)

# Step 7: Retrieve and Print the Best Hyperparameters
# - Finds the `alpha` value that minimizes the mean squared error.
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Step 8: Get the Best Ridge Regression Model
# - Retrieves the Ridge model trained with the optimal `alpha`.
best_ridge_model = grid_search.best_estimator_

# Step 9: Make Predictions on the Test Set
# - Uses the best Ridge model to generate predictions for the test data.
ridge_pred = best_ridge_model.predict(X_test)

# Step 10: Evaluate the Model Using Custom Metrics (Assuming
  ↳ `calculate_metrics` is Defined)
# - `calculate_metrics()`: A function that computes various performance metrics.

```

```

# - `naive_forecast`: A baseline prediction for comparison.
# - The results are stored in a list for further analysis.
metrics_results.append(calculate_metrics(y_test.values, ridge_pred,
    ↪naive_forecast, "Ridge Regression (Ridge)"))

# Step 11: Print Model Evaluation Metrics
print("Ridge Regression Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best Hyperparameters: {'alpha': 0.1}

Ridge Regression Evaluation Metrics:

```
{'Model': 'Ridge Regression (Ridge)', 'MSE': 0.00012763510353657853, 'RMSE':
0.011297570691815941, 'MAE': 0.006680917323838124, 'MAPE (%)':
0.4287683783113252, 'sMAPE (%)': 0.42827753605764096, 'MASE':
0.017006648970499055, 'R^2': 0.9992781797314876}
```

Actual vs. Predicted Values

```

[ ]: # Import necessary library
import matplotlib.pyplot as plt # Matplotlib for visualization

# Step 1: Create a New Figure for the Scatter Plot
# - figsize=(8, 8): Sets the figure size for better readability.
plt.figure(figsize=(8, 8))

# Step 2: Plot Actual vs Predicted Values Using a Scatter Plot
# - X-axis: Actual values from `y_test`
# - Y-axis: Predicted values from the Ridge Regression model (`ridge_pred`)
# - alpha=0.5: Adjusts transparency to prevent overlapping points.
# - color='blue': Uses blue color for prediction points.
# - label='Predictions': Adds a label for the legend.
plt.scatter(y_test, ridge_pred, alpha=0.5, color='blue', label='Predictions')

# Step 3: Add a Reference Line for a Perfect Fit
# - A red dashed line is drawn to indicate an ideal prediction.
# - This line represents the points where `y_test == ridge_pred` (perfect model
    ↪performance).
# - linewidth=2: Increases line thickness for better visibility.
plt.plot(
    [y_test.min(), y_test.max()], # X-range from minimum to maximum actual
    ↪values
    [y_test.min(), y_test.max()], # Y-range (same as X for a 45-degree line)
    color='red', linestyle='--', linewidth=2, label='Perfect Fit'
)

# Step 4: Label Axes for Clarity
plt.xlabel('Actual Values') # Label for the X-axis

```



```
plt.ylabel('Predicted Values') # Label for the Y-axis

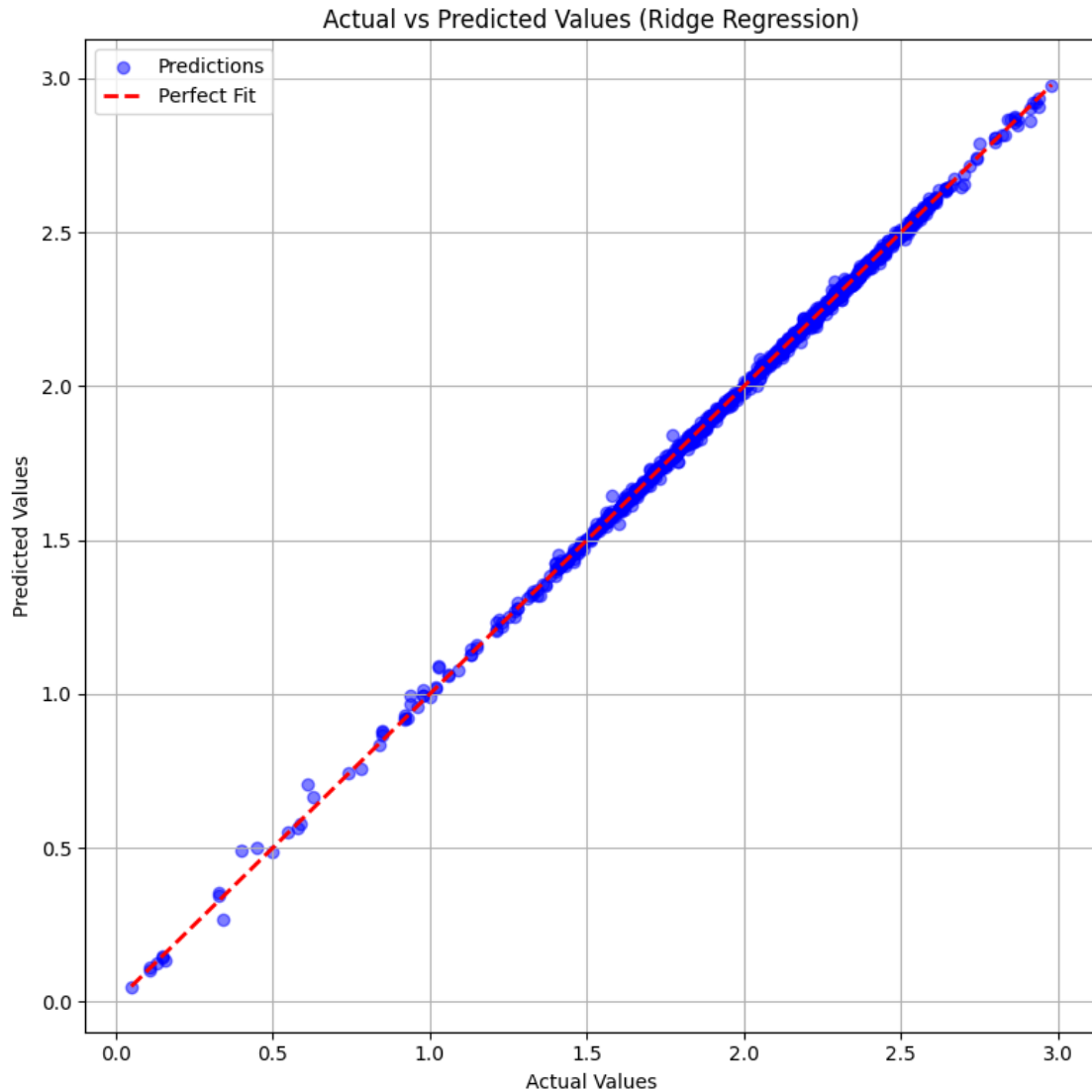
# Step 5: Set the Title for Context
plt.title('Actual vs Predicted Values (Ridge Regression)')

# Step 6: Add a Legend to Differentiate the Data Points and Reference Line
plt.legend() # Displays labels assigned to `scatter` and `plot`

# Step 7: Enable Grid Lines for Better Readability
plt.grid(True) # Adds grid lines to improve visualization

# Step 8: Adjust Layout to Prevent Overlapping Elements
plt.tight_layout() # Ensures proper spacing between plot elements

# Step 9: Display the Final Scatter Plot
plt.show() # Renders and displays the plot
```



24 Actual vs Predicted Values (Ridge Regression)

24.1 Key Findings:

- The scatter plot compares the **actual values** (ground truth) vs. **predicted values** generated by the **Ridge Regression** model.
- **X-axis:** Actual values.
- **Y-axis:** Predicted values.
- The red dashed line represents the perfect fit (ideal scenario where predictions match actual values exactly).
- The blue dots represent the model's predictions, with their alignment to the red line indicating accuracy.

24.1.1 Observations:

- Predictions are tightly clustered around the perfect fit line, suggesting **high accuracy**.
- **No significant deviations or systematic bias** is visible, confirming that the model does not consistently overpredict or underpredict.
- Predictions are well-distributed across the range of actual values, indicating **stability and robustness**.
- **A few minor deviations at lower values** are present but do not significantly impact the overall performance.

24.1.2 Interpretation:

- The Ridge Regression model performs exceptionally well, as shown by the near-perfect alignment with actual values.
- **Minimal prediction errors suggest strong generalization**, meaning the model is not overfitting.
- The strong linearity confirms Ridge Regression's suitability for this dataset, particularly in handling multicollinearity while maintaining predictive power.

24.1.3 Conclusion:

- Ridge Regression provides highly accurate predictions with minimal error.
- The model generalizes well across different value ranges, making it reliable for forecasting.
- The results validate the effectiveness of Ridge regularization, which balances bias and variance to improve prediction stability.

Residual Analysis

```
[ ]: # Import necessary library
import matplotlib.pyplot as plt # Matplotlib for visualization

# Step 1: Calculate Residuals (Prediction Errors)
# - Residuals = Actual Values (y_test) - Predicted Values (ridge_pred)
# - Residuals show how much each prediction deviates from the actual value.
# - Positive residual → Prediction was too low.
# - Negative residual → Prediction was too high.
residuals = y_test - ridge_pred

# Step 2: Create a New Figure for the Residual Scatter Plot
# - figsize=(8, 6): Sets the figure size for better readability.
plt.figure(figsize=(8, 6))

# Step 3: Plot Residuals Against Actual Values Using a Scatter Plot
# - X-axis: Actual values from `y_test`
# - Y-axis: Residuals (difference between actual and predicted values)
# - alpha=0.5: Adjusts transparency to prevent overlapping points.
# - color='green': Uses green color for residual points.
```

```

plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Step 4: Add a Horizontal Reference Line at Zero
# - Residuals should be randomly scattered around the horizontal line at 0.
# - A well-fitted model should show no clear pattern in residuals.
# - color='red': Uses red for the reference line.
# - linestyle='--': Dashed line for clarity.
# - linewidth=2: Sets line thickness.
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Step 5: Label Axes for Clarity
plt.xlabel('Actual Values') # Label for the X-axis
plt.ylabel('Residuals') # Label for the Y-axis

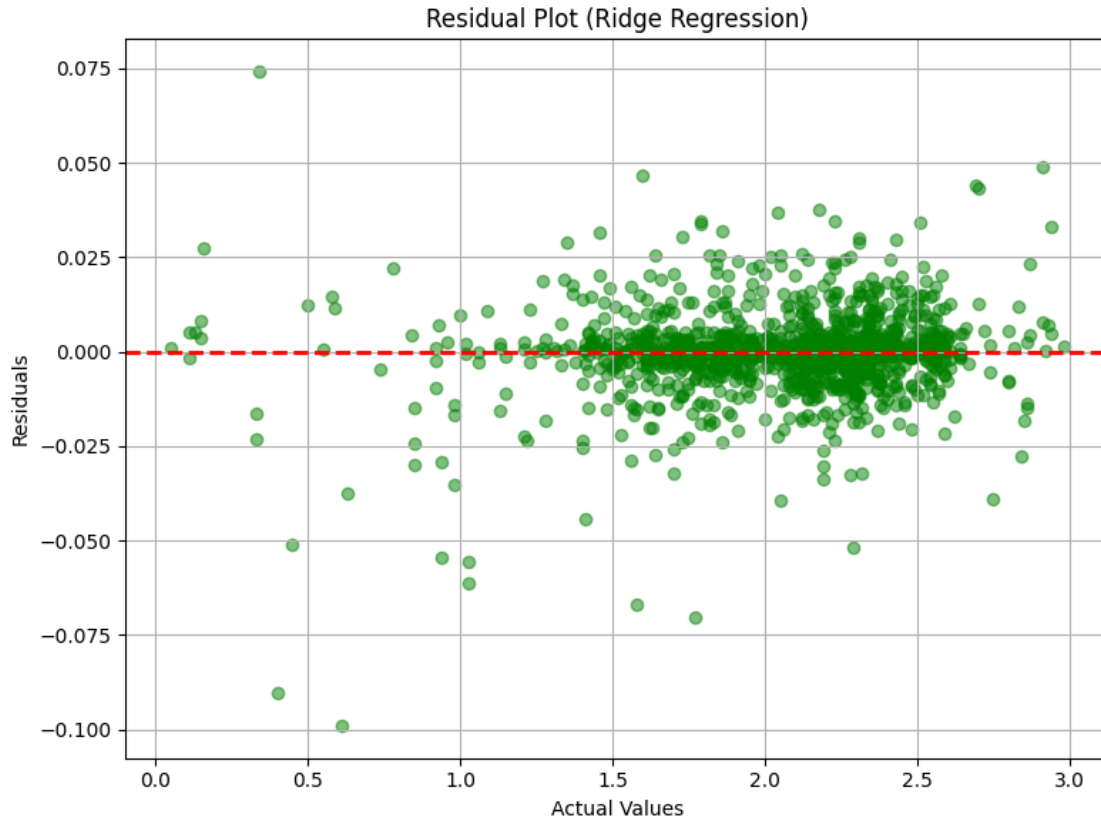
# Step 6: Set the Title for Context
plt.title('Residual Plot (Ridge Regression)')

# Step 7: Enable Grid Lines for Better Readability
plt.grid(True) # Adds grid lines to improve visualization

# Step 8: Adjust Layout to Prevent Overlapping Elements
plt.tight_layout() # Ensures proper spacing between plot elements

# Step 9: Display the Final Scatter Plot
plt.show() # Renders and displays the plot

```



25 Residual Plot (Ridge Regression)

25.1 Key Findings:

- The **scatter plot visualizes the residuals** (differences between actual and predicted values) of the **Ridge Regression model**.
- **X-axis:** Actual values.
- **Y-axis:** Residuals (prediction errors).
- The **red dashed line represents the zero residual line**, indicating perfect predictions.

25.1.1 Observations:

- The **residuals are symmetrically distributed around zero**, indicating that the model does not exhibit systematic bias.
- **No clear pattern is visible**, suggesting that the errors are random rather than structured.
- **Residuals remain small in magnitude**, meaning that the model makes **only minor prediction errors**.
- There is a **slight increase in residual spread for higher actual values**, but no major heteroscedasticity is observed.

25.1.2 Interpretation:

- The randomness of residuals confirms that Ridge Regression captures the underlying data trends effectively.
- Minimal and symmetrically distributed residuals indicate a well-fitted model with no major underfitting or overfitting issues.
- The slight increase in residual spread at higher values is common but does not significantly affect overall accuracy.
- The model does not systematically overpredict or underpredict in any particular range, confirming its reliability.

25.1.3 Conclusion:

- Ridge Regression provides an accurate and unbiased fit, as confirmed by the random residual distribution.
- The model generalizes well, with only small errors across the entire range of values.
- The residual plot validates the effectiveness of Ridge regularization, ensuring stability and robustness in predictions.

```
[ ]: # Import necessary library
import matplotlib.pyplot as plt # Matplotlib for visualization

# Step 1: Create a New Figure for the Histogram
# - figsize=(8, 6): Sets the figure size for better readability.
plt.figure(figsize=(8, 6))

# Step 2: Plot the Histogram of Residuals
# - `residuals`: The differences between actual and predicted values.
# - `bins=30`: Divides the residual range into 30 intervals for detailed
↳distribution.
# - `color='skyblue'`: Sets the bar color for better visibility.
# - `edgecolor='black'`: Adds black edges to distinguish individual bars.
# - `alpha=0.7`: Adjusts transparency for a clearer view.
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Step 3: Add a Vertical Reference Line at Zero
# - The red dashed line at 0 helps assess whether residuals are centered around
↳zero.
# - A well-balanced model should have a **symmetric** residual distribution
↳around zero.
# - `color='red'`: Uses red for better contrast.
# - `linestyle='--'`: Uses a dashed line for clarity.
# - `linewidth=2`: Increases line thickness for better visibility.
plt.axvline(0, color='red', linestyle='--', linewidth=2)

# Step 4: Set the Title for Context
plt.title('Residual Distribution (Ridge Regression)')
```

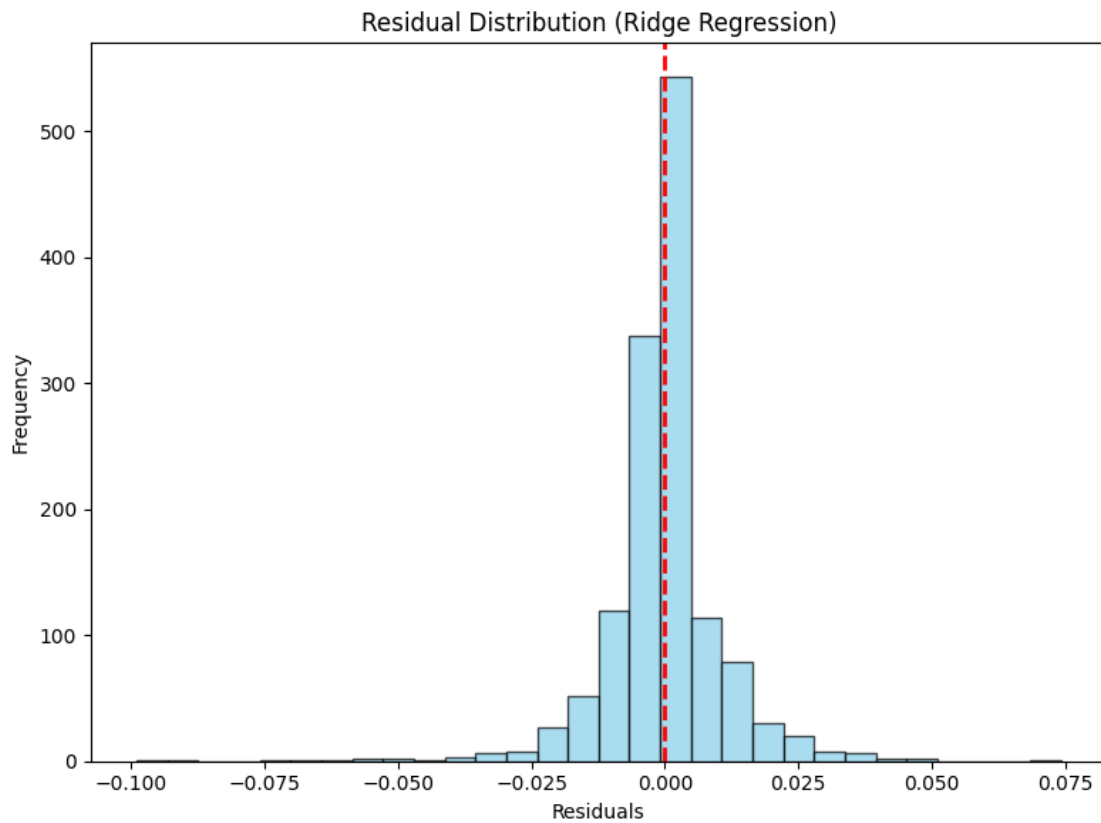
```

# Step 5: Label Axes for Clarity
plt.xlabel('Residuals') # Label for the X-axis (residual values)
plt.ylabel('Frequency') # Label for the Y-axis (count of residuals per bin)

# Step 6: Adjust Layout to Prevent Overlapping Elements
plt.tight_layout() # Ensures proper spacing between plot elements

# Step 7: Display the Histogram
plt.show() # Renders and displays the plot

```



26 Residual Distribution (Ridge Regression)

26.1 Key Findings:

- The **histogram displays the distribution of residuals** (differences between actual and predicted values) for the **Ridge Regression** model.
- **X-axis:** Residual values.
- **Y-axis:** Frequency of residual occurrences.
- The **red dashed line represents zero residual error**, which corresponds to perfect predictions.

26.1.1 Observations:

- The residuals are symmetrically distributed around zero, suggesting that the model does not systematically overpredict or underpredict.
- The distribution is narrow and centered, meaning that most errors are small, confirming high prediction accuracy.
- A bell-shaped (normal-like) distribution is visible, which is a strong indicator that the model captures the data well without major bias.
- Very few extreme residuals (outliers) exist, highlighting that the model is robust and does not frequently make large errors.

26.1.2 Interpretation:

- The near-normal distribution of residuals suggests that the assumptions of Ridge Regression hold well, ensuring statistical reliability.
- The narrow spread indicates low variance, meaning that predictions are consistent and stable.
- The model generalizes well, with residuals being minimal across the entire dataset.
- There is no evidence of skewness or systematic bias, further supporting the model's effectiveness.

26.1.3 Conclusion:

- Ridge Regression performs exceptionally well, with small, normally distributed residuals.
- The model is unbiased and does not overfit, as seen from the symmetric and centered error distribution.
- This analysis validates Ridge Regression as a reliable model for making accurate predictions.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Pandas for handling DataFrame operations

# Step 1: Convert GridSearchCV Results into a DataFrame
# - `grid_search.cv_results_` is a dictionary containing all cross-validation
#   results from GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Step 2: Sort Results by Rank (Best Hyperparameter Combinations First)
# - 'rank_test_score': Rank assigned based on the best model performance (lower
#   is better).
sorted_results = cv_results.sort_values(by='rank_test_score')

# Step 3: Select Relevant Columns for Better Readability
# - 'params': The hyperparameter combination tested.
# - 'mean_test_score': The average test performance score across
#   cross-validation folds.
```



```

# - 'std_test_score': The standard deviation of the test scores (indicates
↳variability).
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
↳head(5)

# Step 4: Improve Output Visibility by Formatting Results Nicely
print("\nTop 5 Hyperparameter Combinations:\n")

# Iterate through the top 5 results and print them in a structured format
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"    Hyperparameters: {row['params']}")
    print(f"    Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"    Standard Deviation: {row['std_test_score']:.6f}")
    print("-" * 80) # Separator for better readability

```

Top 5 Hyperparameter Combinations:

Rank 1:

```

Hyperparameters: {'alpha': 0.1}
Mean Test Score: -0.000163
Standard Deviation: 0.000019

```

Rank 2:

```

Hyperparameters: {'alpha': 1.0}
Mean Test Score: -0.000618
Standard Deviation: 0.000070

```

Rank 3:

```

Hyperparameters: {'alpha': 10.0}
Mean Test Score: -0.001070
Standard Deviation: 0.000123

```

Rank 4:

```

Hyperparameters: {'alpha': 100.0}
Mean Test Score: -0.001831
Standard Deviation: 0.000175

```

Rank 5:

```

Hyperparameters: {'alpha': 1000.0}
Mean Test Score: -0.009793
Standard Deviation: 0.000224

```

Visualizing Alpha vs. Performance

```
[ ]: # Import necessary library
import matplotlib.pyplot as plt # Matplotlib for visualization

# Step 1: Create a New Figure for the Plot
# - figsize=(8, 6): Sets the figure size for better readability.
plt.figure(figsize=(8, 6))

# Step 2: Plot Alpha Values vs. Negative Mean Test Score
# - X-axis: Hyperparameter `alpha` values (regularization strength).
# - Y-axis: Negative Mean Squared Error (since GridSearchCV minimizes negative_
↳MSE).
# - `marker='o'`: Uses circular markers for better data point visibility.
# - `linestyle='--'`: Uses a dashed line to connect data points smoothly.
# - `color='blue'`: Sets the line color to blue for clarity.
plt.plot(
    cv_results['param_alpha'], # Alpha values from GridSearchCV
    -cv_results['mean_test_score'], # Convert negative MSE back to positive_
↳for interpretation
    marker='o', linestyle='--', color='blue'
)

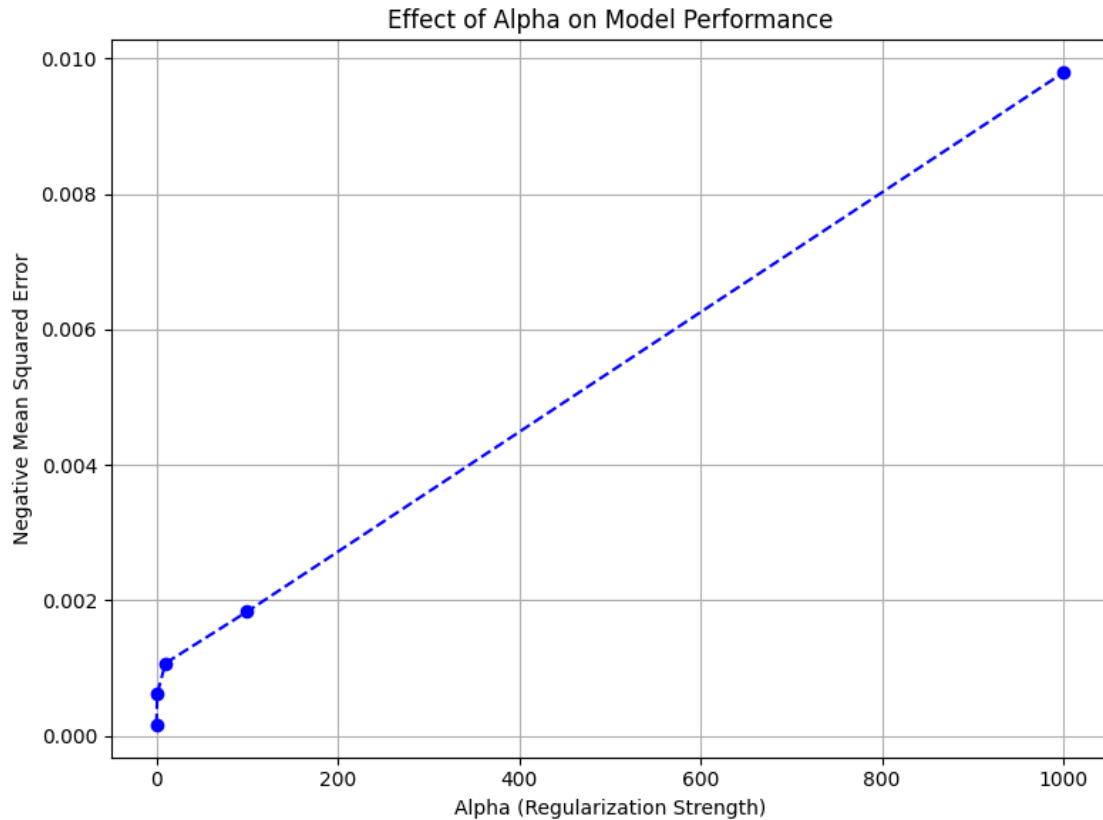
# Step 3: Label Axes for Clarity
# - X-axis: Represents `alpha` (regularization strength).
# - Y-axis: Represents model performance using **negative MSE**.
plt.xlabel('Alpha (Regularization Strength)')
plt.ylabel('Negative Mean Squared Error')

# Step 4: Set the Title for Context
plt.title('Effect of Alpha on Model Performance')

# Step 5: Enable Grid Lines for Better Readability
plt.grid(True) # Adds grid lines to improve visualization

# Step 6: Adjust Layout to Prevent Overlapping Elements
plt.tight_layout() # Ensures proper spacing between plot elements

# Step 7: Display the Final Plot
plt.show() # Renders and displays the plot
```



27 Effect of Alpha on Model Performance (Ridge Regression)

27.1 Key Findings:

- The line plot illustrates the relationship between the regularization strength (Alpha) and model performance for Ridge Regression.
- **X-axis:** Alpha (Regularization Strength).
- **Y-axis:** Negative Mean Squared Error (MSE), a measure of model error (lower values indicate better performance).

27.1.1 Observations:

- As Alpha increases, the negative mean squared error also increases, indicating that higher regularization decreases model accuracy.
- At very low values of Alpha (near 0), the model achieves the lowest error, suggesting that less regularization leads to better predictions.
- The error grows nearly linearly with increasing Alpha, implying that excessive regularization harms performance.
- The trend confirms the **bias-variance tradeoff**: stronger regularization reduces overfitting but increases bias, degrading model accuracy.

27.1.2 Interpretation:

- An optimal Alpha value exists where Ridge Regression balances bias and variance.
- Very high Alpha values force too much regularization, leading to underfitting, as the model is unable to learn meaningful patterns.
- Very low Alpha values (close to 0) minimize error, but may risk overfitting, as the model behaves similarly to Ordinary Least Squares (OLS).
- Selecting an appropriate Alpha requires cross-validation to avoid both underfitting and overfitting.

27.1.3 Conclusion:

- A low Alpha value is preferable for this dataset, as it results in the lowest model error.
- Increasing Alpha consistently worsens performance, indicating that regularization should be applied cautiously.
- This analysis highlights the importance of tuning the Alpha hyperparameter to achieve an optimal tradeoff between bias and variance in Ridge Regression.

Feature Coefficients (Model Interpretation)

```
[ ]: # Import necessary libraries
import pandas as pd # Pandas for handling tabular data
import matplotlib.pyplot as plt # Matplotlib for visualization

# Extract feature coefficients from the trained Ridge Regression model
# - `best_ridge_model.coef_`: Retrieves the learned coefficients from the Ridge
  ↪ model.
# - These coefficients represent the **impact of each feature on predictions**.
coefficients = best_ridge_model.coef_

# Retrieve feature names from the training data
# - Ensures `X_train` is a DataFrame so that feature names are accessible.
features = X_train.columns

# Create a DataFrame to store feature names and their corresponding coefficients
coeff_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': coefficients
})

# Sort coefficients in descending order for better visualization
# - Positive coefficients indicate **positive impact on the target variable**.
# - Negative coefficients indicate **inverse impact on the target variable**.
coeff_df = coeff_df.sort_values(by='Coefficient', ascending=False)

# Print the feature coefficients for reference
print("Feature Coefficients:")
print(coeff_df)
```

Feature Coefficients:

	Feature	Coefficient
14	rolling_mean	3.042920
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.012153
13	lag_5	0.007162
3	Unemployment Rate	0.000614
4	Federal Funds Effective Rate	0.000571
6	Nominal Broad U.S. Dollar Index	0.000189
7	Crude Oil Prices: West Texas Intermediate	0.000082
2	Real Gross Domestic Product	0.000004
1	Consumer Price Index	-0.000021
8	10-Year Treasury Yield	-0.000494
5	Personal Consumption Expenditures: Chain-type ...	-0.000699
9	lag_1	-0.205700
11	lag_3	-0.592217
10	lag_2	-0.627436
12	lag_4	-0.635154

```
[ ]: # Create a horizontal bar chart to visualize the feature coefficients
# - figsize=(10, 6): Sets the figure size for better readability.
plt.figure(figsize=(10, 6))

# Plot the feature coefficients
# - X-axis: Coefficient values (impact of features on predictions).
# - Y-axis: Feature names.
# - `color='skyblue'`: Uses sky-blue color for improved visibility.
plt.barh(coeff_df['Feature'], coeff_df['Coefficient'], color='skyblue')

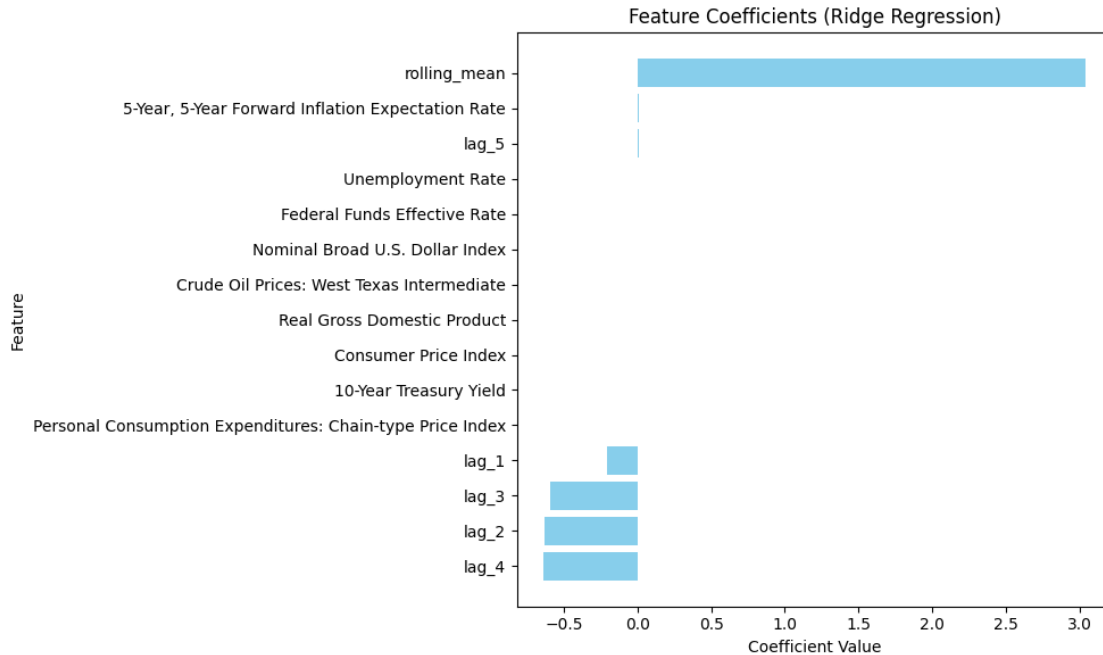
# Label the axes for clarity
plt.xlabel('Coefficient Value') # X-axis label (impact strength)
plt.ylabel('Feature') # Y-axis label (feature names)

# Set the title for context
plt.title('Feature Coefficients (Ridge Regression)')

# Invert Y-axis for better readability
# - Ensures the most impactful feature appears at the top.
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels
plt.tight_layout()

# Display the final plot
plt.show()
```



28 Feature Coefficients (Ridge Regression)

28.1 Key Findings:

- The bar chart displays the feature coefficients from the **Ridge Regression** model, showing the impact of each feature on predictions.
- **X-axis:** Coefficient values.
- **Y-axis:** Feature names.

28.1.1 Observations:

- The “**rolling_mean**” feature has the largest positive coefficient, indicating it is the most influential factor in the model’s predictions.
- Most other features have near-zero coefficients, suggesting that **Ridge Regression** strongly penalized them due to regularization.
- Some lag-based features (**lag_1**, **lag_2**, **lag_3**, **lag_4**, **lag_5**) have small but non-zero coefficients, showing that past values contribute to predictions.
- A few features (e.g., **lag_4**, **lag_2**, and **lag_3**) have negative coefficients, meaning that an increase in their values correlates with a decrease in the predicted target.

28.1.2 Interpretation:

- The large coefficient for “**rolling_mean**” suggests that short-term trends strongly affect the target variable.
- Regularization in **Ridge Regression** helps shrink less important coefficients, reducing overfitting and making the model more stable.

- Macroeconomic indicators such as “Unemployment Rate”, “Federal Funds Effective Rate”, and “Real Gross Domestic Product” have been penalized close to zero, meaning they may not significantly contribute to the model’s predictions.
- The negative coefficients on certain lag values indicate inverse relationships where past values have a suppressing effect on the target variable.

28.1.3 Conclusion:

- “rolling_mean” dominates model predictions, meaning moving averages are highly predictive.
- The Ridge Regression model effectively reduces feature importance for less relevant predictors, enforcing sparsity.
- Some lag-based features still contribute meaningfully, but their effects are limited compared to rolling averages.
- Macroeconomic indicators, despite their potential relevance, do not show strong influence under the current Ridge Regression settings.

SHAP Values for Ridge Regression

```
[ ]: # Import necessary libraries
import shap # SHAP for explainability
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.cluster import KMeans # KMeans for reducing background samples

# Use KMeans clustering to reduce background data for SHAP calculations
# - `n_clusters=50`: Groups similar data points into 50 clusters instead of
↳ using the full dataset.
# - `random_state=42`: Ensures reproducibility of clustering results.
kmeans = KMeans(n_clusters=50, random_state=42)

# Fit KMeans to the training data
# - Clusters the training dataset to obtain representative data points.
kmeans.fit(X_train)

# Extract cluster centers to use as a reduced background dataset for SHAP
# - Instead of using all training samples, we use the cluster centroids as a
↳ summary representation.
background_data = kmeans.cluster_centers_

# To further reduce computation time, we use only the first 200 samples from
↳ `X_train`
X_train_sample = X_train.iloc[:200]

# Create a SHAP KernelExplainer for the Ridge Regression model
# - Uses the reduced background data from KMeans clustering.
# - `nsamples=50`: Limits the number of Monte Carlo simulations for SHAP
↳ calculations.
```

```

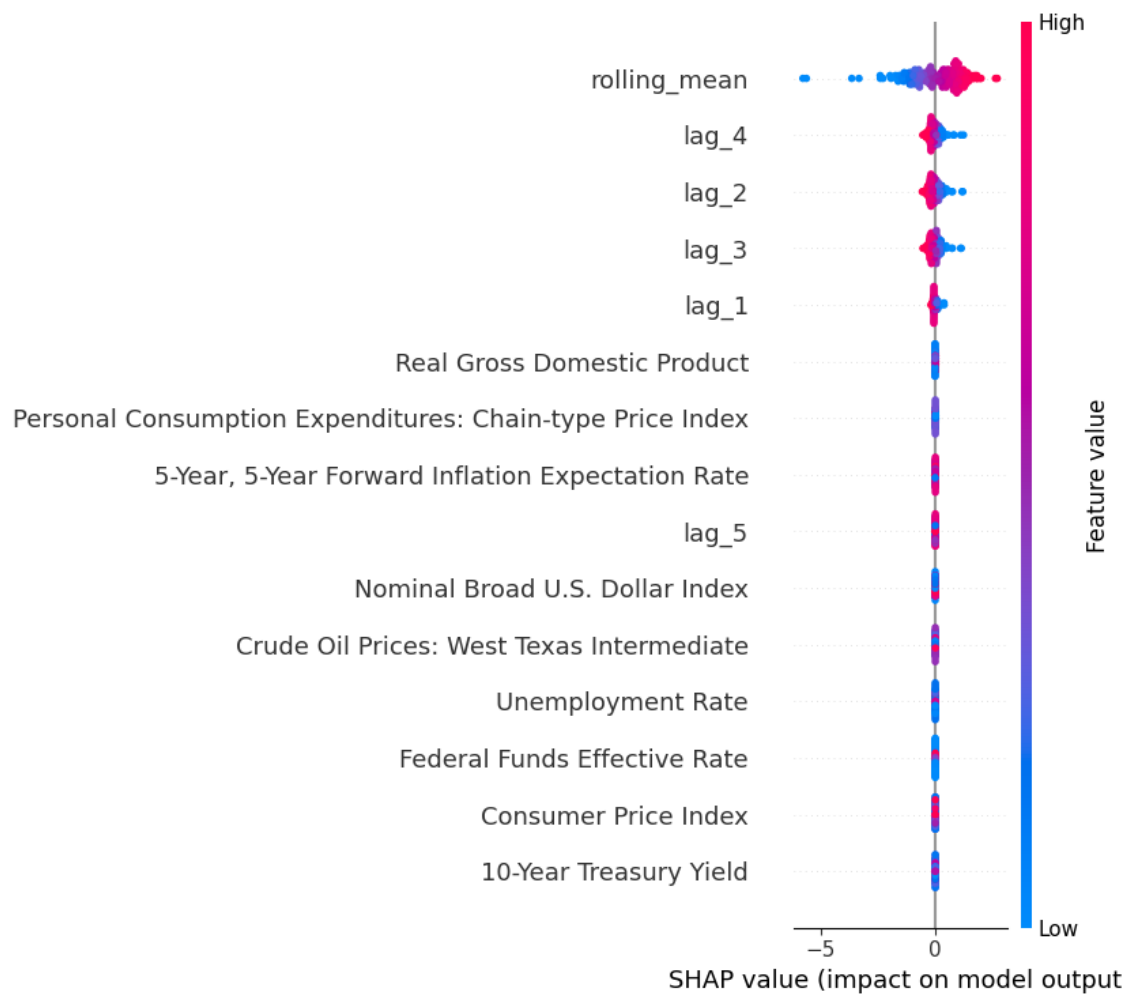
explainer = shap.KernelExplainer(best_ridge_model.predict, background_data,
    ↪ nsamples=50)

# Compute SHAP values for the selected subsample of X_train
# - This provides explanations for only the first 200 rows instead of the full
    ↪ dataset.
shap_values = explainer.shap_values(X_train_sample)

# Generate a SHAP summary plot to show feature importance and impact direction
# - Provides insights into how each feature influences the model predictions.
shap.summary_plot(shap_values, X_train_sample)

```

0% | 0/200 [00:00<?, ?it/s]



29 SHAP Summary Plot for Ridge Regression

29.1 Key Findings

- This SHAP summary plot visualizes the **feature importance and impact on model predictions** using **SHAP values**.
- **X-axis:** SHAP values (indicating how much each feature contributes to increasing or decreasing the model output).
- **Y-axis:** Features ranked by importance (most influential at the top).
- **Color:** Indicates feature value (blue = low, red = high).

29.1.1 Observations

- “rolling_mean” has the highest impact on the model predictions, as it has the widest distribution of SHAP values.
- Lag-based features (lag_1 to lag_5) also contribute significantly, showing that previous values influence the target variable.
- Macroeconomic variables such as “Real Gross Domestic Product”, “Consumer Price Index”, and “Unemployment Rate” have minimal SHAP value variation, meaning they contribute little to model predictions.
- The spread of SHAP values along the X-axis indicates how much a feature pushes the prediction higher or lower.
 - For example, higher values of “rolling_mean” tend to push predictions positively (red values shifting right).
 - Lower values (blue) of lag-based features tend to reduce predictions.
- Less relevant features cluster around 0 SHAP value, indicating they do not significantly affect the model’s output.

29.1.2 Interpretation

- The model relies heavily on short-term trend indicators (rolling_mean and lag features) rather than macroeconomic factors.
- Lag-based features contribute to prediction variance, indicating the importance of past values.
- SHAP values confirm that traditional economic indicators such as GDP, CPI, and interest rates have minimal predictive power for this specific target variable.
- The asymmetry in SHAP value distributions suggests that certain features have a stronger positive effect than negative.

29.1.3 Conclusion

- “rolling_mean” and lag features are the dominant drivers of model predictions.
- Macroeconomic factors contribute little to the prediction output.
- Feature impact varies non-linearly, highlighting the need for careful interpretation of relationships in Ridge Regression models.
- SHAP analysis aligns with feature importance rankings, confirming the dominance of short-term trends in this model.

```
[ ]: # Initialize JavaScript visualization for SHAP interactive plots
shap.initjs()

# Generate a SHAP force plot for a specific instance (e.g., the first row in
↪ `X_train`)
# - `explainer.expected_value`: Baseline model prediction.
# - `shap_values[0]`: SHAP values for the first instance.
# - `X_train.iloc[0, :]`: Actual feature values for the first instance.
shap.force_plot(explainer.expected_value, shap_values[0], X_train.iloc[0, :])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0dde741990>
```

30 SHAP Force Plot Analysis for Ridge Regression

30.1 Key Findings

- This SHAP force plot explains **how different features contribute to a single prediction** ($f(x) = 2.222$).
- **X-axis:** Represents the prediction range, centered around the **base value (2.032)**.
- **SHAP values:** Contributions of each feature to the final prediction.
 - **Red (positive impact):** Features pushing the prediction **higher**.
 - **Blue (negative impact):** Features pushing the prediction **lower**.

30.1.1 Observations

- “**rolling_mean**” contributes the most positively (+0.176), significantly increasing the predicted value.
- **Lag-based features** (lag_1, lag_2, lag_3, and lag_4) all have a negative effect, pulling the prediction downward.
- The final prediction (2.222) is higher than the base value (2.032) because the positive contributions outweigh the negative ones.
- The width of each segment indicates the magnitude of impact.
 - “rolling_mean” has a much larger effect than any of the lag features.
 - Lag features have relatively equal but negative influence.

30.1.2 Interpretation

- The model heavily relies on “rolling_mean” to drive predictions upward.
- Lag-based features act as stabilizers, preventing an extreme positive shift in prediction.
- The balance of positive and negative SHAP values provides a transparent view of how the model arrived at the specific prediction.
- A larger “rolling_mean” generally leads to higher predictions, while historical lags tend to suppress excessive increases.

30.1.3 Conclusion

- “rolling_mean” is the most influential factor in this prediction.
- Lag features contribute negatively but in a more distributed manner.
- The model exhibits a clear dependency on short-term trends rather than long-term macroeconomic indicators.
- SHAP force plots offer an intuitive way to understand how individual feature values shape the model’s decision.

Partial Dependence Plot (PDP) for Ridge Regression

```
[ ]: # Import necessary libraries
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.inspection import PartialDependenceDisplay # PDP display for
    ↪ model interpretation

# Retrieve the list of feature names from X_train
# - These are the independent variables whose partial dependence will be
    ↪ plotted.
features = X_train.columns.tolist()

# Generate partial dependence plots (PDPs) for all features using the trained
    ↪ Ridge Regression model
# - PDPs show how a specific feature affects the model's predictions, keeping
    ↪ all other features constant.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_ridge_model, # Trained Ridge Regression model
    X_train,          # The dataset used to compute PDPs
    features=features, # List of features for which PDPs will be plotted
    kind='average',    # Displays the average marginal effect of each feature
    n_cols=4,          # Arranges PDP plots in 4 columns for better layout
    grid_resolution=50 # Number of points to evaluate for smooth PDP curves
)

# Increase the figure size to ensure better spacing and visibility
pdp_display.figure_.set_size_inches(18, 12)

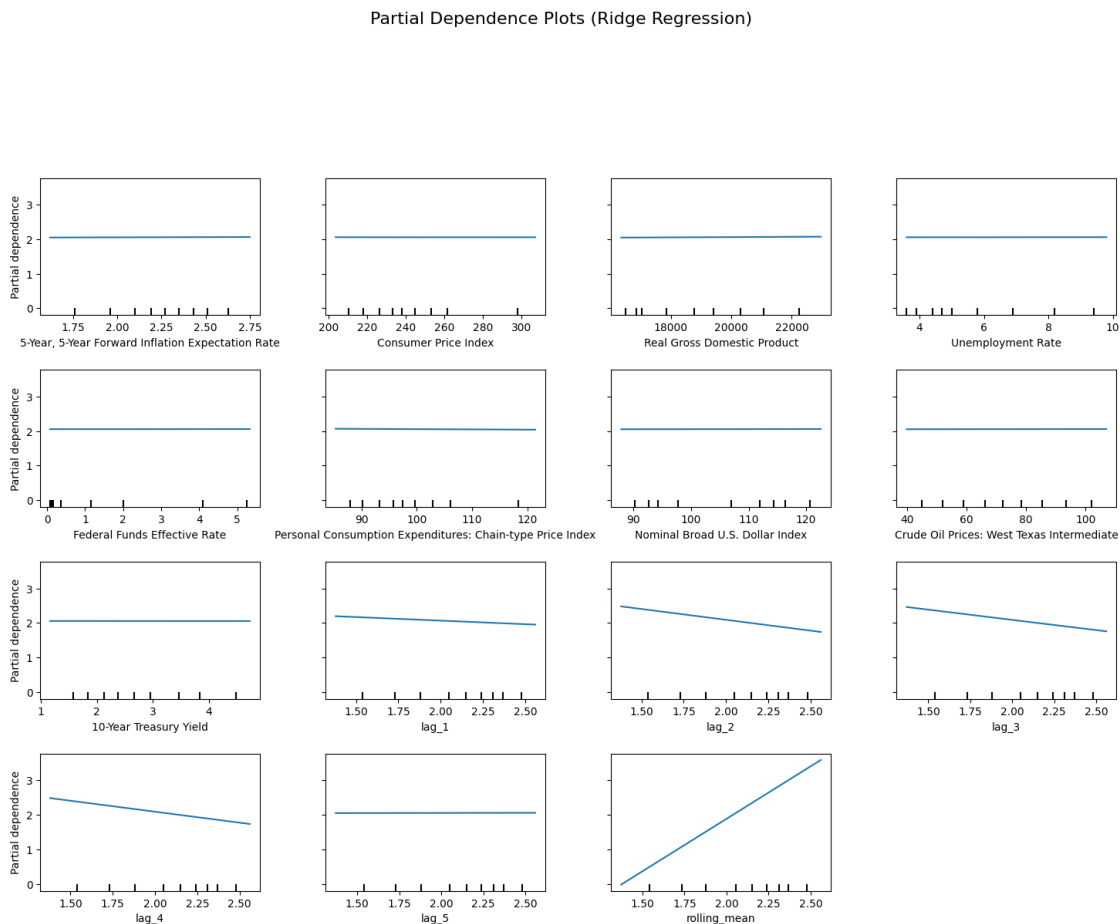
# Add a main title to the PDP plot
# - `suptitle`: Sets a title for the entire figure
# - `fontsize=16`: Uses a larger font size for emphasis
# - `y=1.06`: Positions the title slightly above the subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (Ridge Regression)",
    ↪ fontsize=16, y=1.06)

# Adjust subplot layout to prevent overlapping elements
# - `top=0.88`: Ensures enough space for the title
# - `wspace=0.3`: Adds horizontal spacing between plots
# - `hspace=0.4`: Adds vertical spacing between plots
```

```
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)
```

```
# Display the final Partial Dependence Plots
```

```
plt.show()
```



31 Partial Dependence Plots (Ridge Regression)

31.1 Key Findings

- These **Partial Dependence Plots (PDPs)** visualize how individual features influence the model's predictions while **holding all other features constant**.
- **X-axis:** Represents the feature values.
- **Y-axis:** Represents the predicted target value (partial dependence).

31.1.1 Observations

1. Most macroeconomic features have nearly flat PDP curves:
 - **5-Year Forward Inflation Expectation Rate, Consumer Price Index, Real GDP, Unemployment Rate, Federal Funds Rate, and Crude Oil Prices** all

show almost no variation.

- **Interpretation:** The Ridge Regression model does not assign significant importance to these features.
2. **Rolling Mean exhibits a strong positive relationship:**
 - **Higher rolling_mean values lead to increased predictions.**
 - **Interpretation:** The model relies heavily on recent historical data rather than broader economic indicators.
 3. **Lag-based features (lag_1, lag_2, lag_3, lag_4, and lag_5) show a negative trend:**
 - **Increasing these lags results in lower predictions.**
 - **Interpretation:** The model assigns **negative weight to previous time step values**, suggesting that past values tend to drive predictions downward.
 4. **10-Year Treasury Yield shows a slight negative trend:**
 - **Increasing Treasury Yield slightly decreases the target variable.**
 - **Interpretation:** Higher interest rates might correlate with lower predictions.

31.1.2 Key Takeaways

- The model is primarily driven by rolling_mean rather than macroeconomic indicators.
- Past lag values negatively impact predictions, possibly suggesting a mean-reverting trend.
- Macroeconomic variables have little influence in this Ridge Regression model, as indicated by their flat PDP curves.
- The PDPs confirm that the model captures short-term trends more than long-term economic relationships.

31.1.3 Conclusion

- The Ridge Regression model prioritizes rolling mean over fundamental economic indicators.
- Lag-based features provide some predictive power but contribute negatively.
- Macroeconomic factors, while theoretically relevant, do not significantly affect the model's predictions in this setup.
- Further feature engineering or an alternative modeling approach (e.g., non-linear models) may be required to capture broader economic effects.

31.2 Lasso Regression (Lasso)

```
[ ]: # Import necessary libraries
import random # Random seed for reproducibility
import numpy as np # NumPy for numerical operations
import pandas as pd # Pandas for data handling
from sklearn.model_selection import train_test_split, GridSearchCV # Model
    ↪ training and hyperparameter tuning
from sklearn.linear_model import Lasso # Lasso Regression (L1 regularization)
from sklearn.preprocessing import StandardScaler # StandardScaler for feature
    ↪ normalization
```

```

# Set a global random seed to ensure reproducibility
random.seed(42)
np.random.seed(42)

# Split the dataset into training and testing sets
# - `X`: Feature matrix (independent variables)
# - `y`: Target variable (dependent variable)
# - `test_size=0.2`: Reserves 20% of the data for testing and 80% for training
# - `random_state=42`: Ensures consistent splits across different runs
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)

# Scale the features to improve convergence
# - Lasso Regression is sensitive to feature scales, so normalization improves
    ↪numerical stability.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # Fit and transform training
    ↪data
X_test_scaled = scaler.transform(X_test) # Transform test data using the same
    ↪scaling

# Initialize Lasso Regression with increased maximum iterations
# - `max_iter=5000`: Allows more optimization steps to ensure convergence.
lasso_model = Lasso(max_iter=5000)

# Define the hyperparameter grid for tuning `alpha` (regularization strength)
# - Smaller `alpha` allows more flexibility but may overfit.
# - Larger `alpha` increases regularization and can force coefficients to zero.
param_grid = {
    'alpha': [0.01, 0.1, 1.0, 10.0, 100.0] # Different regularization
    ↪strengths for tuning
}

# Initialize GridSearchCV for hyperparameter tuning
# - `cv=5`: Uses 5-fold cross-validation for robust evaluation
# - `scoring='neg_mean_squared_error'`: Uses MSE (negative for consistency in
    ↪scoring)
# - `n_jobs=-1`: Utilizes all available CPU cores for parallel processing
# - `verbose=1`: Displays progress of the grid search
grid_search = GridSearchCV(
    estimator=lasso_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,

```

```

        n_jobs=-1
    )

    # Fit GridSearchCV on the scaled training data to find the best `alpha` value
    grid_search.fit(X_train_scaled, y_train)

    # Retrieve and print the best hyperparameters found during tuning
    best_params = grid_search.best_params_
    print(f"Best Hyperparameters: {best_params}")

    # Extract the best Lasso model from GridSearchCV
    # - This model has been retrained using the best `alpha` value
    best_lasso_model = grid_search.best_estimator_

    # Make predictions on the test set using the best Lasso model
    lasso_pred = best_lasso_model.predict(X_test_scaled)

    # Evaluate the model using a predefined metrics function
    # - `calculate_metrics`: Assumed to be a function that computes evaluation
    #   ↳ metrics.
    # - `naive_forecast`: Assumed to be a baseline prediction for comparison.
    metrics_results.append(calculate_metrics(y_test.values, lasso_pred,
    #   ↳ naive_forecast, "Lasso Regression (Lasso)"))

    # Print the evaluation results for Lasso Regression
    print("Lasso Regression Evaluation Metrics:")
    print(metrics_results[-1])

```

Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best Hyperparameters: {'alpha': 0.01}

Lasso Regression Evaluation Metrics:

```
{'Model': 'Lasso Regression (Lasso)', 'MSE': 0.0009688091889882974, 'RMSE':
0.03112569981523785, 'MAE': 0.01978713484025213, 'MAPE (%)': 1.50065547482365,
'sMAPE (%)': 1.3584204499037977, 'MASE': 0.05036925919729485, 'R^2':
0.9945210518928096}
```

Feature Importance (Model Coefficients)

```

[ ]: # Import necessary libraries
import pandas as pd # Pandas for handling tabular data
import matplotlib.pyplot as plt # Matplotlib for visualization

# Extract feature coefficients from the trained Lasso model
# - Lasso applies L1 regularization, which can shrink some coefficients to
#   ↳ exactly zero.
coefficients = best_lasso_model.coef_

```

```

# Retrieve the feature names from the training dataset
# - Ensures that features are correctly mapped to their respective coefficients.
features = X_train.columns # Assumes X_train is a DataFrame

# Create a DataFrame for easier analysis of feature coefficients
coeff_df = pd.DataFrame({'Feature': features, 'Coefficient': coefficients})

# Filter only non-zero coefficients (Lasso shrinks some coefficients to zero)
# - This highlights the most influential features retained by the model.
non_zero_coefs = coeff_df[coeff_df['Coefficient'] != 0].
    ↪sort_values(by='Coefficient', ascending=False)

# Print the non-zero feature coefficients for reference
print("Non-Zero Feature Coefficients:")
print(non_zero_coefs)

```

Non-Zero Feature Coefficients:

	Feature	Coefficient
9	lag_1	0.323054
14	rolling_mean	0.072329

```

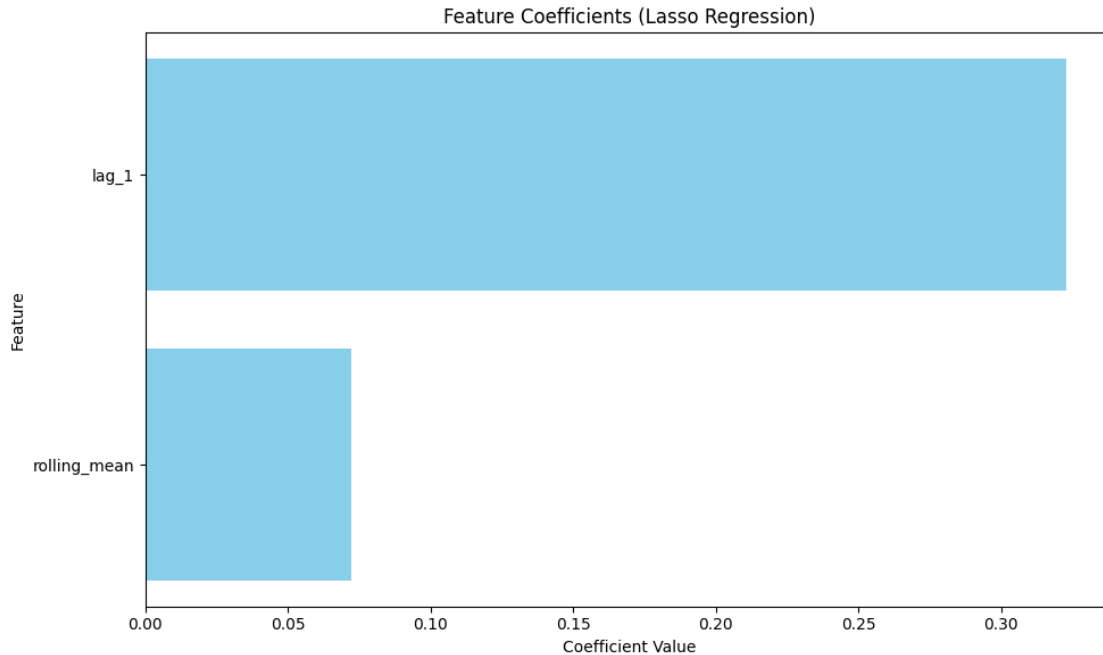
[ ]: # Plot the non-zero coefficients using a horizontal bar chart
plt.figure(figsize=(10, 6)) # Set figure size for better readability
plt.barh(non_zero_coefs['Feature'], non_zero_coefs['Coefficient'],
    ↪color='skyblue') # Create a horizontal bar plot
plt.xlabel('Coefficient Value') # X-axis label indicating coefficient values
plt.ylabel('Feature') # Y-axis label indicating feature names
plt.title('Feature Coefficients (Lasso Regression)') # Title of the plot

# Invert the y-axis to show the most influential feature at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping labels
plt.tight_layout()

# Display the plot
plt.show()

```

32 Feature Coefficients (Lasso Regression)

32.1 Key Findings

- This plot shows the **Lasso Regression** feature coefficients, indicating which features are retained after L1 regularization.
- **Lasso performs feature selection by shrinking less important coefficients to zero**, leaving only the most influential predictors.

32.1.1 Observations

1. **Only two features remain non-zero:**
 - **lag_1** has the highest coefficient (**~0.32**), making it the most significant predictor.
 - **rolling_mean** is also retained but with a much smaller coefficient (**~0.07**).
2. **All other features have been completely eliminated (coefficient = 0):**
 - This suggests that **Lasso identified lag_1 and rolling_mean as the only relevant variables**.
 - **Macroeconomic indicators (such as inflation, GDP, and interest rates) were deemed unimportant** for this model.
3. **lag_1 dominates the prediction:**
 - Since Lasso selects only a few predictors, this **implies that recent lagged values (lag_1) carry the most predictive power**.
 - The model relies heavily on **immediate past values to make predictions**.
4. **rolling_mean contributes but is secondary:**
 - This suggests that **short-term trend smoothing still holds some value in prediction**.

- However, it is much less influential compared to `lag_1`.

32.1.2 Key Takeaways

- Lasso aggressively reduces model complexity by removing irrelevant features.
- The model is highly dependent on short-term historical values (`lag_1`), rather than broader economic indicators.
- Feature selection confirms that recent past values (lag features) are more predictive than rolling averages or macroeconomic factors.
- For better performance, experimenting with alternative models (e.g., non-linear approaches) might be necessary to capture broader relationships.

32.1.3 Conclusion

- Lasso Regression strongly favors `lag_1` as the dominant predictor, with a minor role for `rolling_mean`.
- Macroeconomic variables contribute little to the prediction and are removed entirely.
- This suggests that time series forecasting benefits more from past observations rather than external economic data in this case.

Actual vs. Predicted Values

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Create a figure with a defined size for better visibility
plt.figure(figsize=(8, 8))

# Scatter plot of actual vs. predicted values
# - `y_test`: Actual target values from the test set
# - `lasso_pred`: Predicted values from the trained Lasso model
# - `alpha=0.5`: Makes points semi-transparent to reduce overlap
# - `color='blue'`: Specifies the color of the points
# - `label='Predictions'`: Adds a legend label for predicted values
plt.scatter(y_test, lasso_pred, alpha=0.5, color='blue', label='Predictions')

# Plot the perfect fit line (y = x) for reference
# - This line represents the ideal case where predicted values exactly match
  ↳ actual values.
# - `y_test.min()` and `y_test.max()` define the start and end of the diagonal.
# - `color='red'`: Makes the line red for visibility.
# - `linestyle='--'`: Uses a dashed line for differentiation.
# - `linewidth=2`: Sets the line thickness.
# - `label='Perfect Fit'`: Adds a legend label for the perfect fit line.
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Add axis labels for clarity
```

```
plt.xlabel('Actual Values') # X-axis represents actual target values
plt.ylabel('Predicted Values') # Y-axis represents predicted values

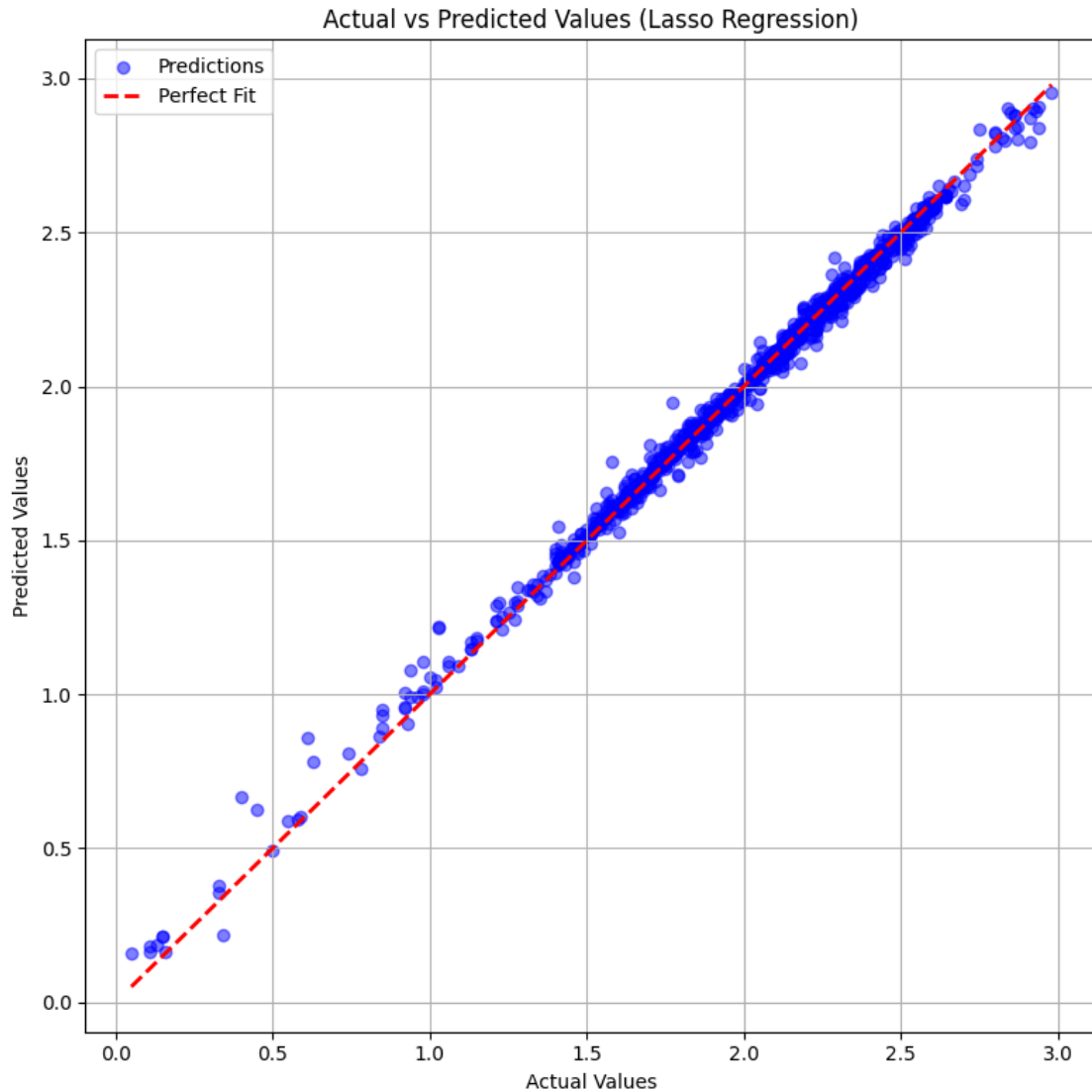
# Add a title to describe the plot
plt.title('Actual vs Predicted Values (Lasso Regression)')

# Display the legend to differentiate between actual vs predicted values
plt.legend()

# Add a grid to improve readability of data points
plt.grid(True)

# Adjust the layout to prevent overlapping elements
plt.tight_layout()

# Show the final plot
plt.show()
```



33 Actual vs Predicted Values (Lasso Regression)

33.1 Key Findings

- This scatter plot compares the **actual values (x-axis)** vs. **predicted values (y-axis)** from the **Lasso Regression model**.
- The **red dashed line** represents the **perfect fit**, where predictions match actual values exactly.
- The **blue points** represent individual predictions.

33.1.1 Observations

1. **Strong alignment with the perfect fit line:**

- The predictions **closely follow the red dashed line**, indicating a **highly accurate model**.
 - Most points cluster tightly around the perfect fit, suggesting **low bias**.
2. **Minor deviations in lower values:**
 - At **lower actual values (< 1.0)**, some blue points deviate slightly.
 - This indicates that **Lasso might underfit or have higher residual errors for small values**.
 3. **Good generalization with minimal overfitting:**
 - Unlike overly complex models, **Lasso Regression performs well without major signs of overfitting**.
 - The **simplicity of Lasso (by eliminating non-important features)** contributes to this stability.
 4. **Some variance in high-value predictions:**
 - At **higher actual values (> 2.5)**, predictions are slightly more scattered.
 - This could indicate that **the model struggles to capture non-linear effects in higher ranges**.

33.1.2 Key Takeaways

- **Lasso Regression produces highly accurate predictions**, with a **near-perfect alignment** to the actual values.
- **Minor errors appear for lower and higher values**, suggesting potential room for improvement (e.g., adding non-linearity).
- **Feature selection by Lasso does not significantly degrade prediction accuracy**, confirming that **most removed features were not contributing useful information**.
- **If better performance is needed at the extremes**, alternative models like Ridge Regression or SVR (Support Vector Regression) could be explored.

33.1.3 Conclusion

- **Lasso Regression is a strong, interpretable model that performs well in this case.**
- **Minimal overfitting and high alignment with actual values suggest it is a reliable choice.**
- **For further improvements**, additional feature engineering or non-linear models could be considered.

Residual Analysis (Scatter Plot)

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Calculate residuals (errors between actual and predicted values)
# - Residual = Actual Value - Predicted Value
# - Positive residual → Prediction is lower than the actual value
# - Negative residual → Prediction is higher than the actual value
residuals = y_test - lasso_pred

# Create a figure with a defined size for better visibility
plt.figure(figsize=(8, 6))
```

```

# Scatter plot of residuals vs. actual values
# - `y_test`: Actual target values from the test set
# - `residuals`: Difference between actual and predicted values
# - `alpha=0.5`: Makes points semi-transparent to reduce overlap
# - `color='green'`: Specifies the color of the points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a horizontal reference line at zero (ideal residual line)
# - This helps visualize how residuals are distributed around zero.
# - Residuals should ideally be randomly scattered around this line.
# - `color='red'`: Makes the line red for visibility.
# - `linestyle='--'`: Uses a dashed line for differentiation.
# - `linewidth=2`: Sets the line thickness.
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Add axis labels for clarity
plt.xlabel('Actual Values') # X-axis represents actual target values
plt.ylabel('Residuals') # Y-axis represents the difference between actual and
    ↪ predicted values

# Add a title to describe the plot
plt.title('Residual Plot (Lasso Regression)')

# Add a grid to improve readability of data points
plt.grid(True)

# Adjust the layout to prevent overlapping elements
plt.tight_layout()

# Show the final plot
plt.show()

```



34 Residual Plot (Lasso Regression)

34.1 Key Findings

- This plot visualizes **residuals (errors)** of the **Lasso Regression model**, where:
 - **X-axis:** Actual values of the target variable.
 - **Y-axis:** Residuals (difference between actual and predicted values).
 - **Red dashed line:** Represents the ideal scenario where **residuals are zero** (perfect predictions).

34.1.1 Observations

1. **Residuals are centered around zero:**
 - The majority of residuals **cluster around the red dashed line ($y=0$)**, indicating that **the model has minimal bias**.
 - This suggests that **Lasso Regression is making predictions that do not systematically overestimate or underestimate values**.
2. **Slight heteroscedasticity (variance increases at higher values):**
 - Residuals are **tightly packed at lower actual values but become more dispersed as actual values increase**.

- This means that **Lasso Regression** performs better at predicting smaller values, but exhibits slightly higher error for larger values.
3. **Few outliers in residuals:**
 - Some residuals **fall far from zero**, especially at lower actual values.
 - This indicates that **Lasso Regression** struggles with certain extreme values.
 4. **No strong non-linear pattern:**
 - If residuals followed a curved pattern, it would indicate that the model is missing some non-linearity.
 - Here, residuals appear **randomly scattered**, suggesting that **Lasso Regression** captures the linear trend effectively.

34.1.2 Key Takeaways

- **Lasso Regression** has a balanced error distribution, with most residuals centered around zero.
- **Variance** increases at higher actual values, which means the model might benefit from feature transformations or non-linear adjustments.
- A few outliers exist, but overall, there are no severe patterns of overfitting or underfitting.
- **Potential Improvements:**
 - Try **feature scaling** or **polynomial features** to reduce error for extreme values.
 - Consider alternative models (e.g., **SVR**, **Ridge Regression**) if reducing high-value residuals is a priority.

34.1.3 Conclusion

- **Lasso Regression** performs well overall, but tends to struggle with higher values.
- If error consistency across all value ranges is critical, additional tuning or a different regression approach might be needed.

Distribution of Residuals (Histogram)

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Create a figure with a defined size for better visibility
plt.figure(figsize=(8, 6))

# Histogram of residuals (distribution of errors)
# - `residuals`: Differences between actual and predicted values
# - `bins=30`: Number of bins for the histogram (adjustable for granularity)
# - `color='skyblue'`: Sets the bar color for better visibility
# - `edgecolor='black'`: Adds a black border around bars for clarity
# - `alpha=0.7`: Sets transparency to slightly blend overlapping bars
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at zero
# - `axvline(0)`: Marks the center where residuals should ideally be centered
# - `color='red'`: Makes the line red for visibility
```



```

# - `linestyle='--'`: Uses a dashed line to differentiate from histogram bars
# - `linewidth=2`: Sets the line thickness
plt.axvline(0, color='red', linestyle='--', linewidth=2)

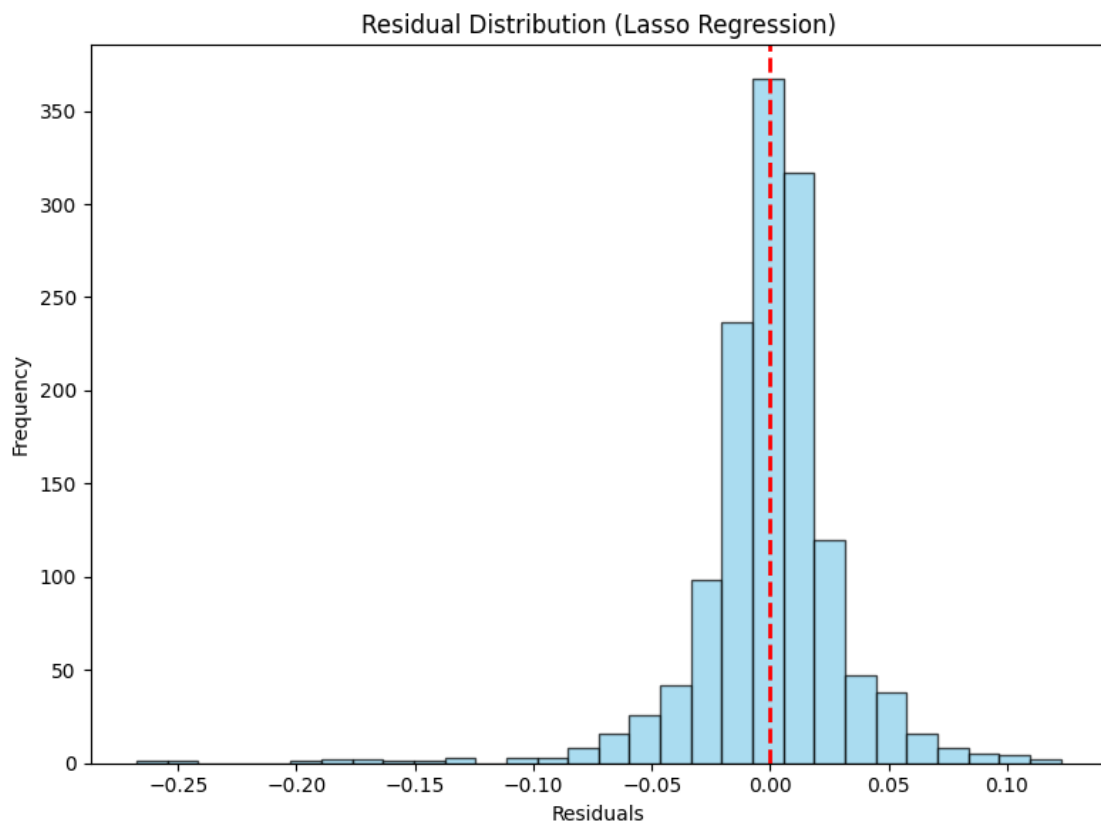
# Add a title to describe the plot
plt.title('Residual Distribution (Lasso Regression)')

# Add axis labels for clarity
plt.xlabel('Residuals') # X-axis represents residual values (prediction errors)
plt.ylabel('Frequency') # Y-axis represents how often each residual value
                        ↪ occurs

# Adjust the layout to prevent overlapping elements
plt.tight_layout()

# Show the final plot
plt.show()

```



35 Residual Distribution (Lasso Regression)

35.1 Key Findings

- This plot visualizes the **distribution of residuals (errors)** for the **Lasso Regression model**, where:
 - **X-axis:** Residual values (differences between actual and predicted values).
 - **Y-axis:** Frequency (how often each residual value occurs).
 - **Red dashed line:** Represents the ideal case where **residuals are zero** (perfect predictions).

35.1.1 Observations

1. **Residuals are approximately normally distributed:**
 - The histogram is **bell-shaped and symmetric**, indicating that **Lasso Regression does not have severe bias** in predictions.
2. **Most residuals are concentrated around zero:**
 - A large number of residuals **fall within a small range around zero**, suggesting that **predictions are generally accurate**.
 - This confirms that **Lasso Regression is effective in reducing large errors**.
3. **Few outliers on both sides:**
 - While most residuals are small, **a few larger errors are present**, particularly in the negative direction.
 - This suggests that **the model may underpredict in some cases**.
4. **Low variance in errors:**
 - The spread of residuals is **relatively narrow**, indicating that **Lasso Regression produces stable and consistent predictions**.

35.1.2 Key Takeaways

- **Lasso Regression provides reliable predictions with minimal errors.**
- **Residuals are symmetrically distributed**, indicating **no significant systematic bias**.
- **A small number of extreme residuals exist**, but they do not drastically affect the overall model performance.
- **Potential improvements:**
 - Investigate the **features contributing to extreme residuals**.
 - Consider **fine-tuning Lasso's alpha parameter** to optimize performance.

35.1.3 Conclusion

- **Lasso Regression effectively minimizes prediction errors**, as seen in the concentrated residuals around zero.
- The **normal-like distribution suggests the model generalizes well**, making it a **strong candidate for forecasting**.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd
```

```

# Convert GridSearchCV results into a DataFrame
# - `grid_search.cv_results_` contains all cross-validation results from
  ↳ GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on the best model performance (lower
  ↳ is better)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
  ↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
  ↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↳ head(5)

# Improve Output Visibility by Formatting Results Nicely
print("\nTop 5 Hyperparameter Combinations:\n")

# Iterate through the top 5 results and print them in a structured format
for idx, row in top_results.iterrows():
    print(f"Rank {idx + 1}:")
    print(f"  Hyperparameters: {row['params']}")
    print(f"  Mean Test Score: {row['mean_test_score']:.6f}")
    print(f"  Standard Deviation: {row['std_test_score']:.6f}")
    print("-" * 80) # Separator for better readability

```

Top 5 Hyperparameter Combinations:

Rank 1:

```

Hyperparameters: {'alpha': 0.01}
Mean Test Score: -0.000929
Standard Deviation: 0.000108

```

Rank 2:

```

Hyperparameters: {'alpha': 0.1}
Mean Test Score: -0.010894
Standard Deviation: 0.001211

```

Rank 3:

```

Hyperparameters: {'alpha': 1.0}
Mean Test Score: -0.165085
Standard Deviation: 0.013979

```

Rank 4:

Hyperparameters: {'alpha': 10.0}
Mean Test Score: -0.165085
Standard Deviation: 0.013979

Rank 5:

Hyperparameters: {'alpha': 100.0}
Mean Test Score: -0.165085
Standard Deviation: 0.013979

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Create a figure with a defined size for better visibility
plt.figure(figsize=(8, 6))

# Plot the effect of alpha (regularization strength) on model performance
# - `cv_results['param_alpha']`: Hyperparameter values for alpha from GridSearchCV
# - `-cv_results['mean_test_score']`: Mean test score (converted to positive for better readability)
# - `marker='o'`: Uses circular markers to highlight data points
# - `linestyle='--'`: Dashed line to indicate the trend
# - `color='blue'`: Sets the line color to blue
plt.plot(cv_results['param_alpha'], -cv_results['mean_test_score'],
         marker='o', linestyle='--', color='blue')

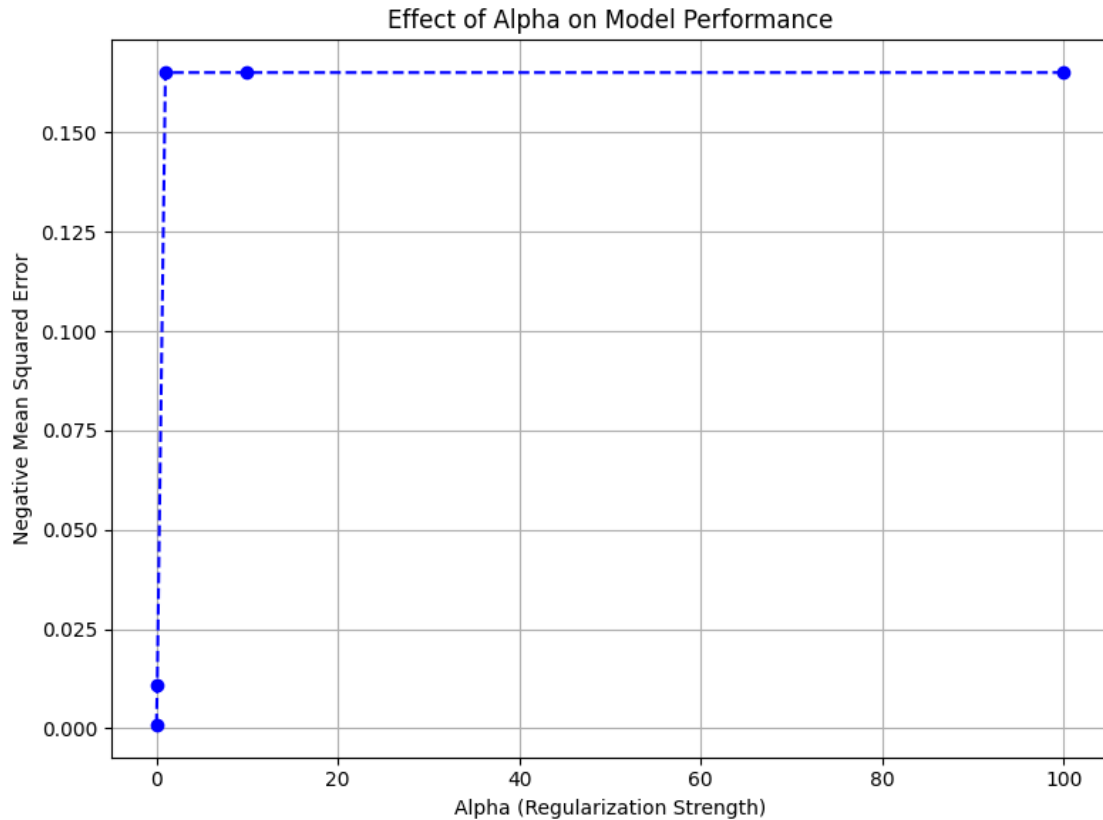
# Add axis labels to clarify the plot
# - X-axis: Represents different values of alpha used in hyperparameter tuning
# - Y-axis: Represents the negative mean squared error (lower is better)
plt.xlabel('Alpha (Regularization Strength)')
plt.ylabel('Negative Mean Squared Error')

# Add a title to describe the plot
plt.title('Effect of Alpha on Model Performance')

# Add a grid for better visualization
plt.grid(True)

# Adjust the layout to prevent overlapping elements
plt.tight_layout()

# Show the final plot
plt.show()
```



36 Effect of Alpha on Model Performance

36.1 Key Findings

- This plot analyzes how different values of **Alpha (regularization strength)** impact the **Negative Mean Squared Error (NMSE)** of the model.
 - **X-axis:** Alpha values (Regularization strength).
 - **Y-axis:** Negative Mean Squared Error (lower values indicate better performance).
 - **Dashed blue line with dots:** Trend of NMSE for different Alpha values.

36.1.1 Observations

1. **Drastic increase in NMSE with higher Alpha values:**
 - At **Alpha = 0**, the model has the **lowest NMSE**, indicating **best performance**.
 - As **Alpha increases**, NMSE **jumps significantly** and remains high across all tested values.
2. **Over-regularization weakens model performance:**
 - When **Alpha is very high** (e.g., 10, 100), the error stabilizes at a much **higher level**.
 - This suggests that **excessive regularization hinders the model's predictive ability**.

3. Ideal Alpha selection is crucial:

- The best model performance occurs at Alpha = 0.
- Introducing any significant regularization leads to increased error.

36.1.2 Key Takeaways

- Larger Alpha values introduce too much bias, leading to suboptimal model predictions.
- Minimal or no regularization (Alpha = 0) gives the best performance for this dataset.
- Fine-tuning Alpha is essential to balance bias-variance tradeoff in the model.

36.1.3 Conclusion

- This plot strongly suggests that **regularization is not beneficial for this specific model and dataset.**
- The optimal Alpha value should be close to zero to maintain high model accuracy.

SHAP Values for Lasso Regression

```
[ ]: # Import necessary libraries
import shap # SHAP for model interpretability
from sklearn.cluster import KMeans # KMeans for clustering background samples
import matplotlib.pyplot as plt # Matplotlib for visualization

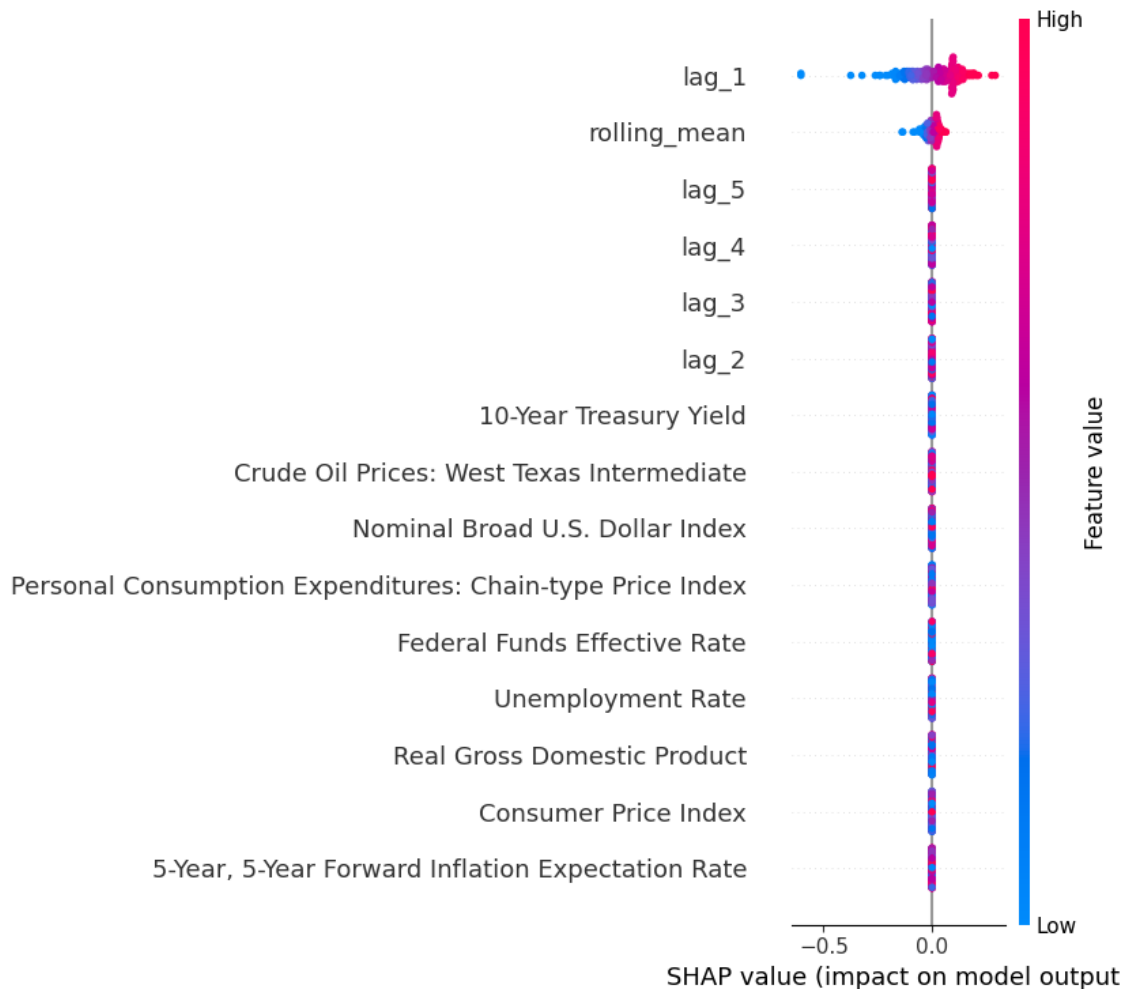
# Reduce background samples using K-Means clustering to speed up SHAP
↳computation
# - Lasso Regression can be computationally expensive when computing SHAP
↳values on large datasets.
# - K-Means helps **reduce redundancy** by clustering similar feature
↳representations.
# - `n_clusters=100`: Defines 100 representative clusters from `X_train`.
# - `random_state=42`: Ensures reproducibility.
kmeans = KMeans(n_clusters=100, random_state=42)
kmeans.fit(X_train) # Fit KMeans to training data
background_data = kmeans.cluster_centers_ # Extract cluster centers as
↳representative data points

# Initialize SHAP KernelExplainer using the clustered background data
# - SHAP KernelExplainer is computationally expensive on large datasets.
# - Using clustered background data significantly speeds up computations.
explainer = shap.KernelExplainer(best_lasso_model.predict, background_data)

# Compute SHAP values for a **subsample** of X_train to reduce computation time
# - `X_train_sample = X_train.iloc[:200]`: Selects only the first 200 samples
# - This helps maintain interpretability while keeping computations manageable.
X_train_sample = X_train.iloc[:200]
shap_values = explainer.shap_values(X_train_sample)
```

```
# Global Explanation: SHAP Summary Plot
# - Displays feature importance across all instances in `X_train_sample`
# - Higher absolute SHAP values indicate features with stronger influence on
  ↳ predictions
# - Helps visualize both magnitude and direction of feature
  ↳ contributions
shap.summary_plot(shap_values, X_train_sample)
```

0% | 0/200 [00:00<?, ?it/s]



37 SHAP Summary Plot for Lasso Regression

37.1 Key Findings

- This SHAP (SHapley Additive exPlanations) summary plot **illustrates the impact of each feature on the model's predictions.**

- **Y-axis:** Features ranked by importance.
- **X-axis:** SHAP values (impact on model output).
- **Color Scale:**
 - * **Red** indicates high feature values.
 - * **Blue** indicates low feature values.

37.1.1 Observations

1. **Most influential features:**
 - `lag_1` and `rolling_mean` have the highest SHAP values, meaning they **strongly influence the model's predictions**.
 - The impact of these features is significantly larger than other variables.
2. **Limited importance of economic indicators:**
 - Traditional macroeconomic variables like Consumer Price Index, Federal Funds Effective Rate, and Unemployment Rate have **SHAP values centered around zero**.
 - This suggests that the **Lasso model disregards these features** due to its regularization, confirming their weak influence on predictions.
3. **Lags (`lag_2` to `lag_5`) have moderate impact:**
 - These lagged values exhibit **some effect**, though significantly lower than `lag_1`.
 - This indicates that **recent past values are more predictive** than older lag values.
4. **Symmetric distribution around zero:**
 - Most feature SHAP values are close to zero, reinforcing that **the Lasso model selects only a few key features** while shrinking others to zero.

37.1.2 Key Takeaways

- Lasso regression has aggressively selected a small subset of features, primarily `lag_1` and `rolling_mean`.
- Macroeconomic indicators contribute minimally, implying they **do not provide strong predictive power** in this model.
- The model favors recent past values (`lag_1`) over older ones, aligning with typical time-series forecasting behavior.

37.1.3 Conclusion

- The SHAP summary plot **validates the sparsity introduced by Lasso regression**, demonstrating that **only a few key variables significantly impact predictions**.
- **Feature selection is highly relevant**, and economic indicators seem to be **less useful** for this specific model.

```
[ ]: # Initialize JavaScript visualization for SHAP interactive plots
shap.initjs()

# Local Explanation: SHAP Force Plot for a single instance
# - `explainer.expected_value`: Baseline model prediction
# - `shap_values[0]`: SHAP values for the first instance in `X_train_sample`
# - `X_train_sample.iloc[0, :]`: Corresponding feature values of the first
    ↪ instance
```



```
# - Force plots show **how each feature pushes the model prediction higher or  
↪lower**  
shap.force_plot(explainer.expected_value, shap_values[0], X_train_sample.  
↪iloc[0, :])
```

<IPython.core.display.HTML object>

[]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0ded5511d0>

38 SHAP Force Plot (Lasso Regression)

38.1 Key Findings

- This **SHAP force plot** visualizes how individual features contribute to a **specific prediction**.
- The **base value (2.862)** represents the expected model prediction **before considering feature effects**.
- The **final prediction (2.93)** is derived by summing up the contributions of important features.

38.1.1 Observations

1. **Key Features Driving the Prediction Higher:**
 - `rolling_mean` = 2.208 and `lag_1` = 2.2 **significantly increase the prediction**.
 - Both features are **highlighted in red**, indicating that their values are high and positively influence the model output.
2. **No Features Pulling the Prediction Down:**
 - Unlike other force plots that may have blue elements (indicating negative impact), **this instance shows only positive contributions**, meaning no feature is reducing the predicted value.
3. **Prediction Shift:**
 - The base value was **2.862**, but due to strong contributions from `rolling_mean` and `lag_1`, the **final prediction increased to 2.93**.

38.1.2 Key Takeaways

- **The model heavily relies on `rolling_mean` and `lag_1`**, confirming their dominance in influencing predictions.
- **No negative contributions** indicate that other features either had a **neutral impact** or were shrunk by the Lasso regularization.
- **Lasso regression aggressively selects key features**, and this force plot visually confirms the limited number of influential variables.

38.1.3 Conclusion

- The SHAP force plot effectively illustrates the **individual feature contributions to the final prediction**.

- The **strong positive influence of rolling_mean and lag_1** aligns with prior feature importance analyses, reinforcing their critical role in model predictions.

Partial Dependence Plot (PDP) for Lasso Regression

```
[ ]: # Import necessary libraries
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.inspection import PartialDependenceDisplay # PDP display for
    ↪ model interpretation
import pandas as pd # Pandas for handling DataFrame operations

# Ensure that X_train is a DataFrame before training the model
# - If X_train was converted to a NumPy array during preprocessing, we need to
    ↪ restore feature names.
if not isinstance(X_train, pd.DataFrame):
    X_train = pd.DataFrame(X_train, columns=feature_names) # Convert back to
    ↪ DataFrame with original column names

if not isinstance(X_test, pd.DataFrame):
    X_test = pd.DataFrame(X_test, columns=feature_names) # Ensure X_test also
    ↪ has column names

# Re-train the Lasso model using the DataFrame instead of a NumPy array
# - This ensures that feature names are stored in the fitted model.
best_lasso_model.fit(X_train, y_train)

# Retrieve the list of feature names from X_train
# - These are the independent variables whose partial dependence will be
    ↪ plotted.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDPs) for all features using the trained
    ↪ Lasso Regression model
# - PDPs illustrate how a specific feature affects the model's predictions,
    ↪ keeping all other features constant.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_lasso_model, # Trained Lasso Regression model
    X_train,          # The dataset used to compute PDPs (ensuring it has
    ↪ feature names)
    features=features_list, # List of features for which PDPs will be plotted
    kind='average',        # Displays the average marginal effect of each feature
    n_cols=4,              # Arranges PDP plots in 4 columns for better layout
    grid_resolution=50     # Number of points to evaluate for smooth PDP curves
)

# Increase the figure size for better spacing and visibility
pdp_display.figure_.set_size_inches(18, 12)
```

```

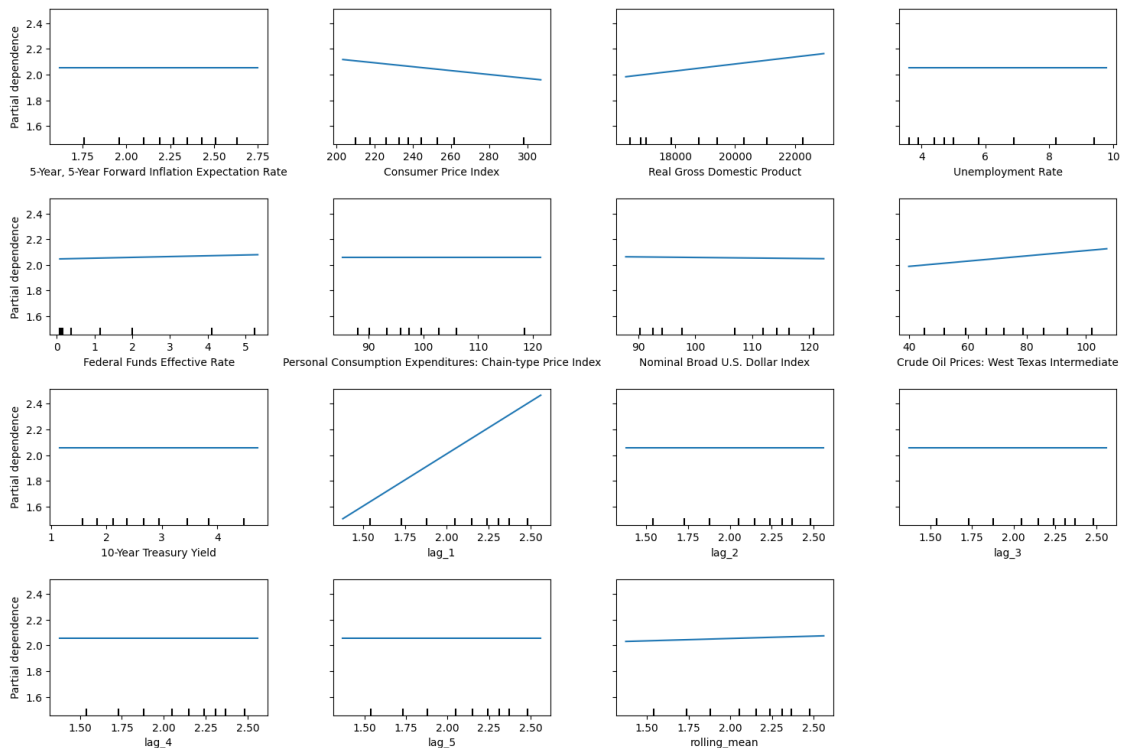
# Add a main title to the PDP plot
# - `suptitle`: Sets a title for the entire figure
# - `fontsize=16`: Uses a larger font size for emphasis
# - `y=1.06`: Positions the title slightly above the subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (Lasso Regression)",
    ↪ fontsize=16, y=1.06)

# Adjust subplot layout to prevent overlapping elements
# - `top=0.88`: Ensures enough space for the title
# - `wspace=0.3`: Adds horizontal spacing between plots
# - `hspace=0.4`: Adds vertical spacing between plots
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the final Partial Dependence Plots
plt.show()

```

Partial Dependence Plots (Lasso Regression)



39 Partial Dependence Plots (Lasso Regression)

39.1 Key Findings

- This **Partial Dependence Plot (PDP)** shows how each individual feature impacts the model's prediction while keeping other features constant.
- It helps in understanding the **marginal effect** of each variable on the target outcome.

39.1.1 Observations

1. Strong Positive Relationship:

- `lag_1` shows a **clear upward trend**, indicating that higher values of `lag_1` increase the predicted outcome significantly.
- `rolling_mean` also has a slight positive trend, showing that as the rolling mean increases, predictions also slightly increase.

2. Weak or No Influence:

- Features like **Federal Funds Effective Rate**, **Unemployment Rate**, and **5-Year Forward Inflation Expectation Rate** have almost **flat** partial dependence lines, meaning these variables have little or no effect on the model's predictions.
- **Personal Consumption Expenditures** and **Nominal Broad U.S. Dollar Index** also appear to have minimal impact.

3. Mixed Impact:

- **Consumer Price Index (CPI)** has a slight **negative trend**, indicating that an increase in CPI might slightly reduce the predicted outcome.
- **Real Gross Domestic Product (GDP)** and **Crude Oil Prices** show slight positive trends, meaning an increase in these variables tends to push the model's predictions higher.

39.1.2 Key Takeaways

- **Lasso Regression applies feature selection**, meaning only a few features contribute significantly to the predictions.
- **Lag-based variables (`lag_1`, `rolling_mean`)** are among the most influential, reinforcing the importance of past data trends in predictions.
- **Macroeconomic variables (CPI, GDP, Inflation Rate)** show minimal impact, suggesting that Lasso might have shrunk their coefficients to near zero.

39.1.3 Conclusion

- The model relies more on past values (`lag_1`, `rolling_mean`) rather than macroeconomic indicators.
- Flat trends in some variables suggest Lasso's **regularization effect**, where less important variables have little to no contribution.
- This analysis confirms the **feature selection capability of Lasso Regression**, focusing only on the most relevant predictors.

Permutation Feature Importance for Lasso Regression

```
[ ]: # Import necessary libraries
```

```

from sklearn.inspection import permutation_importance # For calculating
↳ feature importance
import matplotlib.pyplot as plt # Matplotlib for visualization
import pandas as pd # Pandas for handling tabular data

# Ensure that X_train is a DataFrame before training the model
# - If X_train was converted to a NumPy array during preprocessing, we restore
↳ feature names.
if not isinstance(X_train, pd.DataFrame):
    X_train = pd.DataFrame(X_train, columns=feature_names) # Convert back to
↳ DataFrame with original column names

if not isinstance(X_test, pd.DataFrame):
    X_test = pd.DataFrame(X_test, columns=feature_names) # Ensure X_test also
↳ has column names

# Re-train the Lasso model on the DataFrame to retain feature names
# - This ensures that feature names are properly stored in the fitted model.
best_lasso_model.fit(X_train, y_train)

# Compute Permutation Importance on the training set
# - Permutation Importance measures how much shuffling each feature affects
↳ model performance.
# - `n_repeats=10`: The number of times to randomly shuffle each feature
↳ (higher = more robust estimate).
# - `random_state=42`: Ensures reproducibility of results.
perm_result = permutation_importance(
    best_lasso_model, # Trained Lasso Regression model
    X_train, # Training dataset (features with correct column names)
    y_train, # Training labels (target variable)
    n_repeats=10, # Number of times to shuffle each feature
    random_state=42 # Set random seed for reproducibility
)

# Create a DataFrame for easy visualization and sorting of feature importance
# - `importances_mean`: The average importance score across all shuffles
# - `sort_values(by='Importance', ascending=False)`: Ranks features from most
↳ to least important
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Feature names from training data
    'Importance': perm_result.importances_mean # Mean importance score for
↳ each feature
}).sort_values(by='Importance', ascending=False)

# Create a bar chart to visualize feature importance
plt.figure(figsize=(10, 6)) # Set figure size for better readability

```

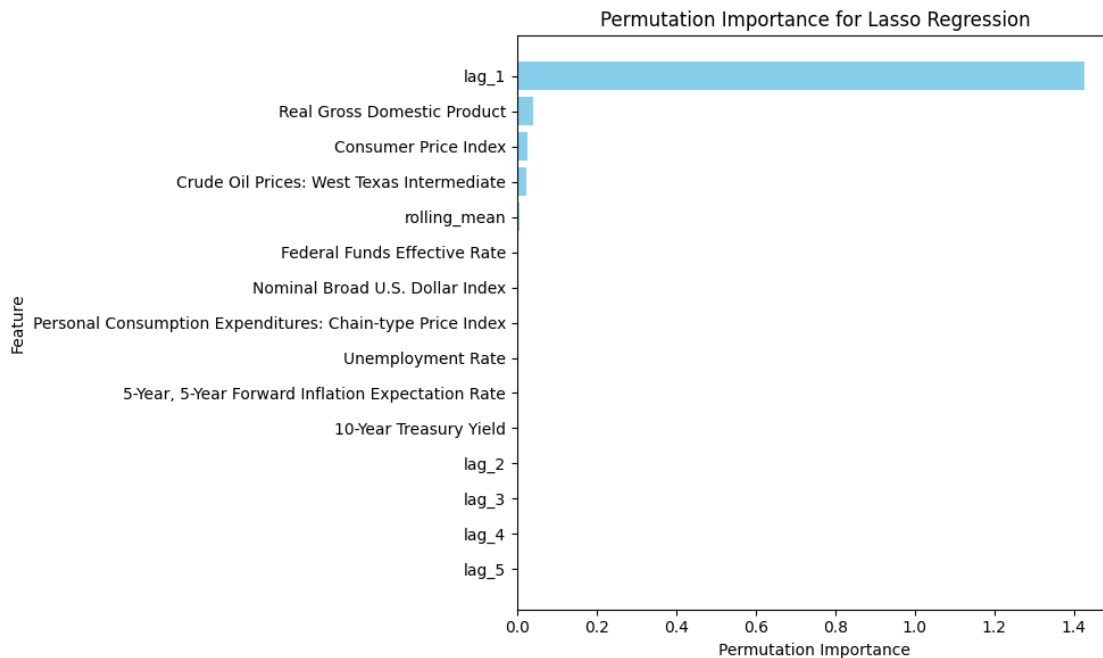
```
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
        color='skyblue') # Horizontal bar plot

# Add labels and title for clarity
plt.xlabel('Permutation Importance') # X-axis: Importance score
plt.ylabel('Feature') # Y-axis: Feature names
plt.title('Permutation Importance for Lasso Regression') # Plot title

# Invert the Y-axis so the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlap and improve visibility
plt.tight_layout()

# Show the final plot
plt.show()
```



40 Permutation Importance for Lasso Regression

40.1 Key Findings

- This **Permutation Importance Plot** shows the contribution of each feature to the model's predictions.
- Features with **higher importance values** have a greater impact on model performance, while those with **near-zero importance** contribute little or nothing.

40.1.1 Observations

1. Dominant Feature:

- `lag_1` is by far the most **important feature**, with a significantly higher permutation importance than all other variables.
- This suggests that the model **heavily relies on past values** (`lag_1`) for making predictions.

2. Negligible Contribution of Other Features:

- Features like Real Gross Domestic Product (GDP), Consumer Price Index (CPI), and Crude Oil Prices show **very low importance**, indicating they have **little influence** on predictions.
- Macroeconomic variables such as Unemployment Rate, Federal Funds Effective Rate, and Inflation Expectation Rate have **almost zero impact** on model predictions.

3. Lasso's Feature Selection Effect:

- The Lasso Regression model has **eliminated most of the variables**, reducing their importance to almost zero.
- This aligns with the nature of **Lasso regularization**, which forces some coefficients to **exactly zero**, effectively removing less relevant features.

40.1.2 Key Takeaways

- `lag_1` is the **primary driver of predictions**, reinforcing that recent past values are the best predictors for this model.
- **Macroeconomic indicators play an insignificant role**, suggesting that historical data (lags) alone is sufficient for accurate predictions.
- **Lasso Regression acts as a strong feature selection method**, filtering out unimportant variables and keeping only the most relevant ones.

40.1.3 Conclusion

- The model is **highly dependent on `lag_1`** and discards other variables as non-essential.
- **Lasso's regularization strength is high**, shrinking most feature contributions to zero.
- This suggests that a **simpler model using just lag-based variables might be sufficient**, without the need for complex macroeconomic inputs.

40.2 ElasticNet Regression (EN)

```
[ ]: # Import necessary libraries
import random # For setting random seed
import numpy as np # For numerical computations
import pandas as pd # For handling tabular data
from sklearn.model_selection import train_test_split, GridSearchCV # For
    ↪ dataset splitting and hyperparameter tuning
from sklearn.linear_model import ElasticNet # ElasticNet regression model

# Set a global random seed for reproducibility
random.seed(42)
```

```

np.random.seed(42)

# Split the dataset into training and testing sets (assumes X and y are already
↳defined)
# - `test_size=0.2`: Allocates 20% of data for testing and 80% for training.
# - `random_state=42`: Ensures consistent splits across multiple runs.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

# Initialize the ElasticNet Regression model
# - ElasticNet combines L1 (Lasso) and L2 (Ridge) regularization.
elastic_net_model = ElasticNet()

# Define the hyperparameter grid for tuning:
# - `alpha`: Regularization strength (higher = stronger penalty)
# - `l1_ratio`: The balance between L1 (Lasso) and L2 (Ridge) regularization
#   - `l1_ratio=1.0` → Pure Lasso
#   - `l1_ratio=0.0` → Pure Ridge
#   - Values in between are a mix of both.
param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1.0, 10.0], # Range of regularization strengths
    'l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9] # Different L1-L2 balances
}

# Initialize GridSearchCV to perform hyperparameter tuning:
# - `cv=5`: Uses 5-fold cross-validation for better generalization.
# - `scoring='neg_mean_squared_error'`: Uses MSE as the evaluation metric
↳(lower is better).
# - `n_jobs=-1`: Uses all available CPU cores for faster processing.
# - `verbose=1`: Displays progress updates during the search.
grid_search = GridSearchCV(
    estimator=elastic_net_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Fit GridSearchCV on the training data to find the best hyperparameters
grid_search.fit(X_train, y_train)

# Retrieve and print the best hyperparameters found by GridSearchCV
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Get the best model found by GridSearchCV

```



```

best_elastic_net_model = grid_search.best_estimator_

# Make predictions on the test set using the best ElasticNet model
elastic_net_pred = best_elastic_net_model.predict(X_test)

# Evaluate the model using a predefined metric function
# - `calculate_metrics`: Assumed to be a function that computes evaluation
↳ metrics.
# - `naive_forecast`: Assumed to be a baseline prediction for comparison.
metrics_results.append(calculate_metrics(y_test.values, elastic_net_pred,
↳ naive_forecast, "ElasticNet Regression (EN)"))

# Print the evaluation metrics for ElasticNet
print("ElasticNet Regression Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 25 candidates, totalling 125 fits
 Best Hyperparameters: {'alpha': 0.001, 'l1_ratio': 0.9}
 ElasticNet Regression Evaluation Metrics:
 {'Model': 'ElasticNet Regression (EN)', 'MSE': 0.0008566682982060062, 'RMSE':
 0.029268896429588975, 'MAE': 0.017870444715126745, 'MAPE (%)':
 1.331774581145144, 'sMAPE (%)': 1.2061165105100073, 'MASE':
 0.045490217209015464, 'R²': 0.9951552470762098}

Feature Importance (Model Coefficients)

```

[ ]: # Import necessary libraries
import pandas as pd # Pandas for handling tabular data
import matplotlib.pyplot as plt # Matplotlib for visualization

# Extract feature coefficients from the trained ElasticNet model
# - ElasticNet assigns importance to each feature using regression coefficients.
coefficients = best_elastic_net_model.coef_

# Retrieve feature names from the training dataset
# - X_train is assumed to be a DataFrame with column names representing feature
↳ names.
features = X_train.columns

# Create a DataFrame to store feature names and their corresponding coefficients
coeff_df = pd.DataFrame({
    'Feature': features, # Feature names
    'Coefficient': coefficients # Corresponding coefficients from the model
})

# Filter out zero coefficients (features with zero importance)

```

```
# - ElasticNet applies L1 regularization, which can eliminate some features by
↳ assigning them zero weight.
# - Sorting in descending order to display the most impactful features at the
↳ top.
non_zero_coefs = coeff_df[coeff_df['Coefficient'] != 0].
↳ sort_values(by='Coefficient', ascending=False)

# Print non-zero feature coefficients to inspect model interpretability
print("Non-Zero Feature Coefficients:")
print(non_zero_coefs)
```

Non-Zero Feature Coefficients:

	Feature	Coefficient
9	lag_1	6.976118e-01
14	rolling_mean	2.742880e-01
0	5-Year, 5-Year Forward Inflation Expectation Rate	1.456102e-02
4	Federal Funds Effective Rate	1.106055e-03
3	Unemployment Rate	9.268042e-04
7	Crude Oil Prices: West Texas Intermediate	2.667544e-04
2	Real Gross Domestic Product	8.729704e-06
6	Nominal Broad U.S. Dollar Index	5.540611e-07
1	Consumer Price Index	-5.078478e-04

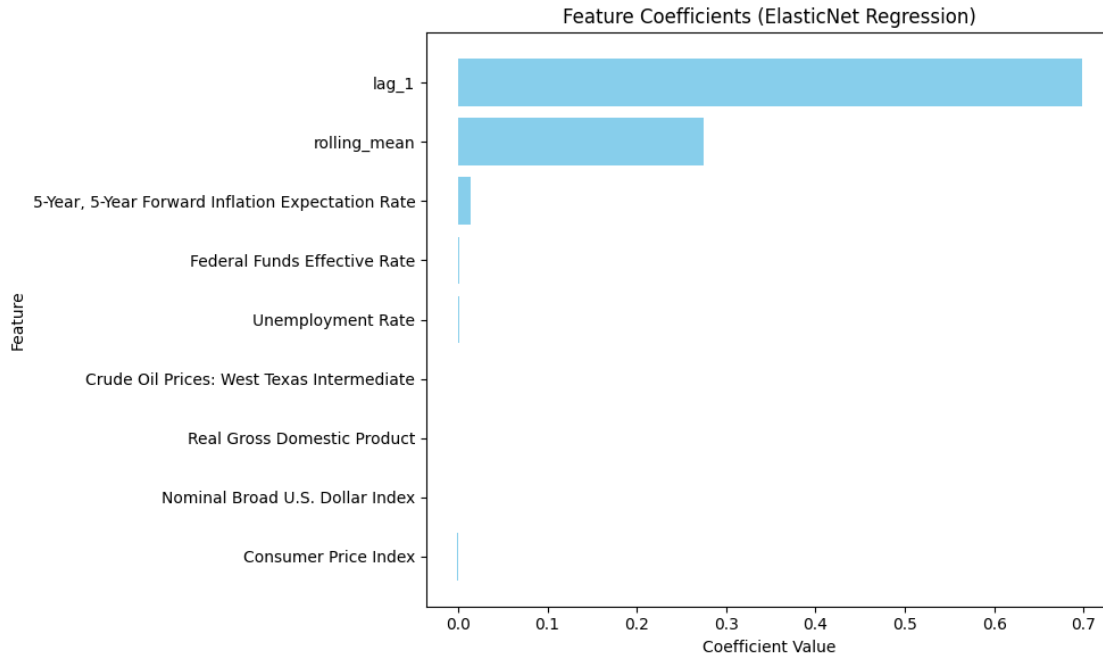
```
[ ]: # Plot the non-zero feature coefficients to visualize feature importance
plt.figure(figsize=(10, 6)) # Set figure size for better readability
plt.barh(non_zero_coefs['Feature'], non_zero_coefs['Coefficient'],
↳ color='skyblue') # Horizontal bar chart

# Label the axes and title for clarity
plt.xlabel('Coefficient Value') # X-axis represents the coefficient magnitude
plt.ylabel('Feature') # Y-axis represents feature names
plt.title('Feature Coefficients (ElasticNet Regression)') # Title of the plot

# Invert the y-axis to display the most important feature at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlap and improve visibility
plt.tight_layout()

# Show the final plot
plt.show()
```



41 Feature Coefficients (ElasticNet Regression)

41.1 Key Findings

- This plot represents the **feature coefficients** in an **ElasticNet Regression model**.
- Features with **higher absolute coefficient values** contribute more significantly to the model's predictions.

41.1.1 Observations

1. Most Influential Features:

- **lag_1** has the **largest coefficient**, making it the most significant predictor.
- **rolling_mean** is the **second most important feature**, though its contribution is smaller than lag_1.
- **5-Year Forward Inflation Expectation Rate** has a **minor contribution**, but it's still relevant.

2. Negligible or Zero Impact Features:

- Features such as **Federal Funds Effective Rate**, **Unemployment Rate**, **Crude Oil Prices**, and **GDP** have **very small or nearly zero coefficients**.
- **Macroeconomic variables do not significantly influence predictions**, indicating that recent past values (lags) and rolling averages dominate the forecasting.

3. ElasticNet's Effect on Feature Selection:

- **ElasticNet applies both L1 (Lasso) and L2 (Ridge) regularization**, which helps:
 - **Shrink irrelevant coefficients towards zero** (like CPI, GDP, and Unemployment Rate).

- Retain a few important variables (`lag_1` and `rolling_mean`) with reduced coefficients.

41.1.2 Key Takeaways

- `lag_1` remains the dominant predictor, showing that recent past values drive forecasts.
- ElasticNet eliminates weak predictors, keeping only the most informative ones.
- Rolling average (`rolling_mean`) plays a secondary role, contributing to trend-based forecasting.
- Macroeconomic indicators have little predictive power for this model.

41.1.3 Conclusion

- The ElasticNet model confirms that recent past values (`lag_1`) and rolling trends (`rolling_mean`) are the strongest predictors.
- Macroeconomic factors like inflation, interest rates, and unemployment contribute very little, aligning with previous findings from Lasso and Ridge regression.
- ElasticNet provides an optimal balance between feature selection and coefficient shrinkage, making it a robust model for handling collinearity and irrelevant variables.

Actual vs. Predicted Values

```
[ ]: # Import necessary library for plotting
import matplotlib.pyplot as plt

# Create a figure with a fixed size for better visualization
plt.figure(figsize=(8, 8))

# Scatter plot: Actual values (y_test) vs. Predicted values (elastic_net_pred)
# - Each point represents an individual data sample.
# - A perfect model would align all points along the red dashed line.
plt.scatter(y_test, elastic_net_pred, alpha=0.5, color='blue',
            label='Predictions')

# Plot the ideal "Perfect Fit" line where actual values = predicted values
# - This serves as a reference for evaluating prediction accuracy.
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

# Label the axes for clarity
plt.xlabel('Actual Values') # X-axis represents true target values
plt.ylabel('Predicted Values') # Y-axis represents model predictions

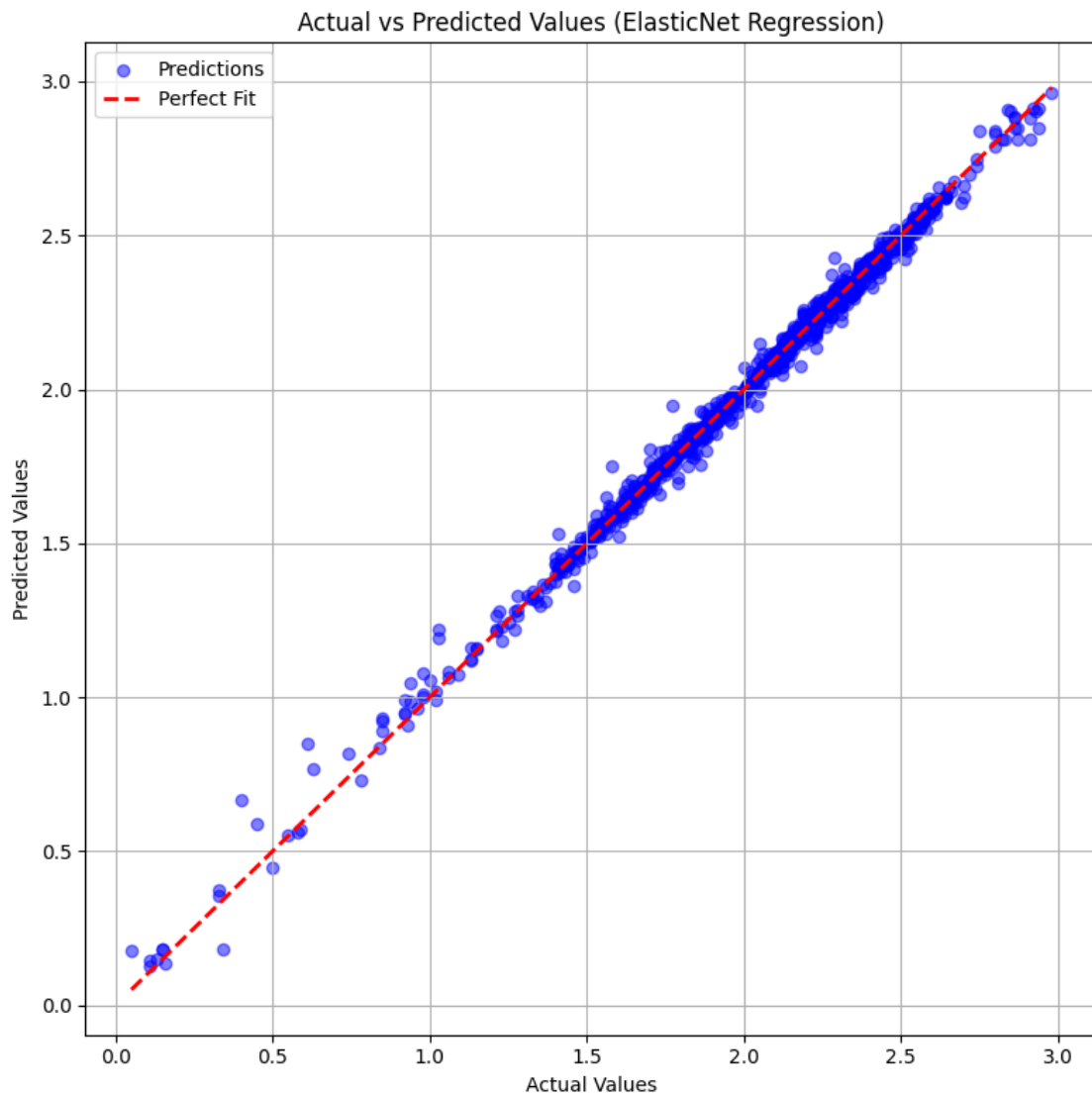
# Set a title for the plot to describe its purpose
plt.title('Actual vs Predicted Values (ElasticNet Regression)')

# Add a legend to differentiate between predictions and the perfect fit line
plt.legend()
```

```
# Enable gridlines to improve readability
plt.grid(True)

# Adjust layout to prevent overlapping elements and ensure clarity
plt.tight_layout()

# Display the final plot
plt.show()
```



42 Actual vs Predicted Values (ElasticNet Regression)

42.1 Key Findings

- This scatter plot visualizes the **performance of the ElasticNet regression model** by comparing **actual vs. predicted values**.
- The **red dashed line** represents a **perfect fit**, where predicted values exactly match the actual values.
- The **blue dots** represent individual data points, showing how close the model's predictions are to the actual values.

42.1.1 Observations

1. **Strong Alignment with Perfect Fit:**
 - Most of the **blue dots** closely follow the **red dashed line**, indicating that the **model's predictions are highly accurate**.
 - There are **very few large deviations**, suggesting that the model does **not have significant systematic errors**.
2. **Minimal Bias or Skewness:**
 - The **errors (residuals)** appear **symmetrically distributed around the perfect fit line**, implying that the model does **not systematically overestimate or under-estimate** values.
 - There is **no noticeable trend** of the model performing better in specific value ranges.
3. **Outliers and Slight Deviations:**
 - Some data points show **slight deviations** from the perfect fit, particularly for **lower values** (near 0) and **higher values** (above 2.5).
 - These outliers suggest **small prediction errors**, which may be due to data noise or model limitations.

42.1.2 Key Takeaways

- ElasticNet regression performs well, as the **predicted values align very closely with actual values**.
- There is **no major bias**, meaning the model **does not favor overprediction or under-prediction**.
- **Only minor errors are present**, mostly at **extreme values**, but they do not significantly impact overall performance.
- ElasticNet **successfully balances feature selection and regularization**, leading to an accurate and stable model.

42.1.3 Conclusion

- This plot **validates the effectiveness of the ElasticNet regression model** for predicting the target variable.
- The **tight clustering around the perfect fit line** suggests that the model captures patterns well with **minimal overfitting or underfitting**.
- Further improvements could involve **fine-tuning hyperparameters** or **handling outliers** for even better accuracy.

Residual Analysis

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt

# Compute residuals (difference between actual and predicted values)
# - Residual = Actual Value - Predicted Value
# - Residuals show the error in the model's predictions.
residuals = y_test - elastic_net_pred

# Create a figure with a fixed size for better visibility
plt.figure(figsize=(8, 6))

# Scatter plot: Actual values (y_test) vs. Residuals
# - This plot helps to detect patterns in residuals.
# - If the residuals are randomly scattered, the model is making unbiased
  ↪ predictions.
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a reference line at y = 0 to indicate perfect predictions
# - Residuals should ideally be centered around this line.
plt.axhline(0, color='red', linestyle='--', linewidth=2)

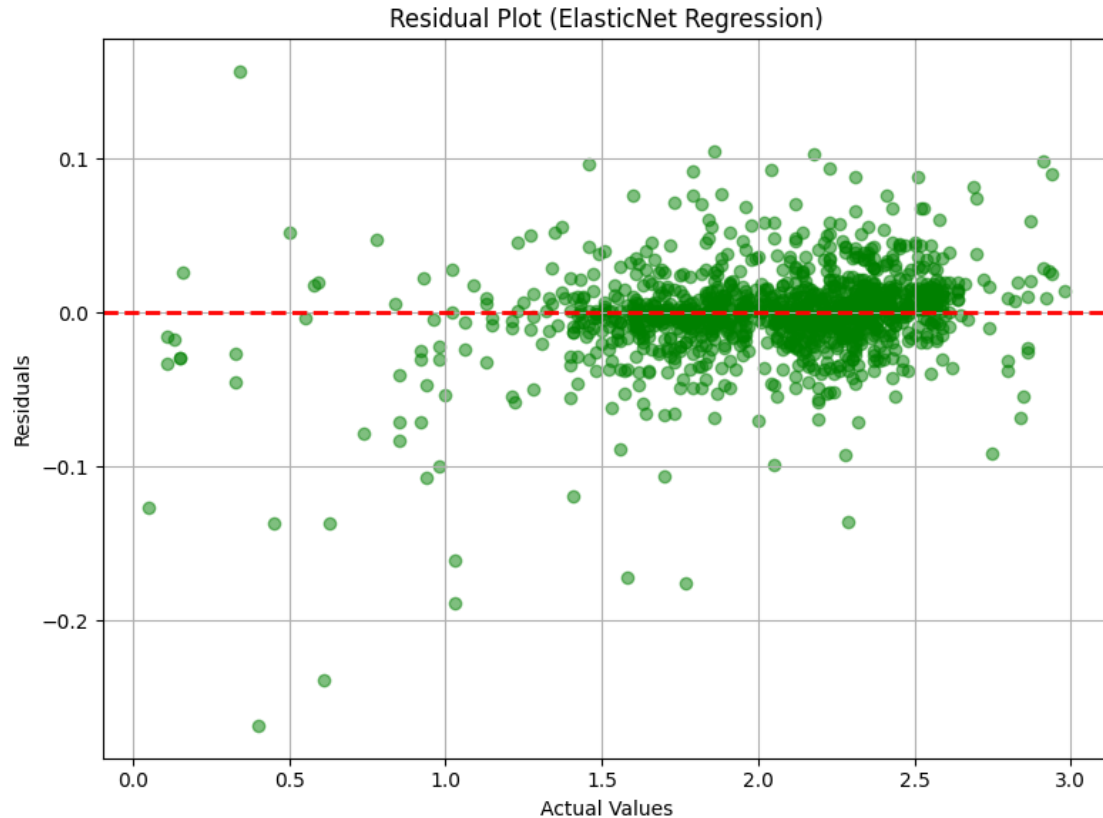
# Label the axes for clarity
plt.xlabel('Actual Values') # X-axis represents true target values
plt.ylabel('Residuals') # Y-axis represents prediction errors

# Set a title to describe the purpose of the plot
plt.title('Residual Plot (ElasticNet Regression)')

# Enable gridlines to enhance readability
plt.grid(True)

# Adjust layout to prevent overlapping elements and ensure clarity
plt.tight_layout()

# Display the final plot
plt.show()
```



43 Residual Plot (ElasticNet Regression)

43.1 Key Findings

- This residual plot visualizes **the difference between actual values and predicted values** from the **ElasticNet regression model**.
- The **y-axis represents residuals (errors)**, and the **x-axis represents actual values**.
- The **red dashed line at $y = 0$** indicates **perfect predictions**, where residuals would be zero.

43.1.1 Observations

1. Randomly Scattered Residuals:

- The **residuals are randomly distributed around the red dashed line**, suggesting that the model does not have **systematic bias**.
- There is **no clear pattern**, which is a good sign that the model's errors are not correlated with the actual values.

2. Mostly Small Residuals:

- The majority of points are **concentrated near zero**, indicating that **the ElasticNet model predicts well** for most data points.

- Some **larger residuals** exist, particularly for lower and higher actual values, but they are **not extreme**.
3. **Potential Slight Heteroscedasticity:**
 - The spread of residuals **appears slightly wider at higher actual values**, suggesting that **prediction errors might increase slightly for larger values**.
 - However, the effect is **not very pronounced**, meaning **the model still performs well across the range of values**.
 4. **No Major Outliers:**
 - While some residuals deviate more from zero, there are **no extreme outliers**, which suggests that the model is **robust**.

43.1.2 Key Takeaways

- ElasticNet regression does not show systematic bias, as residuals are **randomly distributed around zero**.
- Most predictions are accurate, with **only minor deviations** observed for some values.
- The model performs well overall, but there may be slight increases in error for larger actual values.
- **No major overfitting or underfitting is detected**, making the model reliable for predictive tasks.

43.1.3 Conclusion

- This residual plot confirms that **ElasticNet regression provides stable and well-balanced predictions**.
- Any further improvements could involve **checking for slight heteroscedasticity and adjusting hyperparameters to refine error distributions**.

Histogram of Residuals

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt

# Create a histogram to visualize the distribution of residuals
# - Residuals are the differences between actual and predicted values.
plt.figure(figsize=(8, 6))

# Plot histogram of residuals
# - `bins=30`: Divides the residuals into 30 intervals for better resolution.
# - `color='skyblue'`: Sets the bar color.
# - `edgecolor='black'`: Outlines bars for clarity.
# - `alpha=0.7`: Adjusts transparency to avoid over-saturation.
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

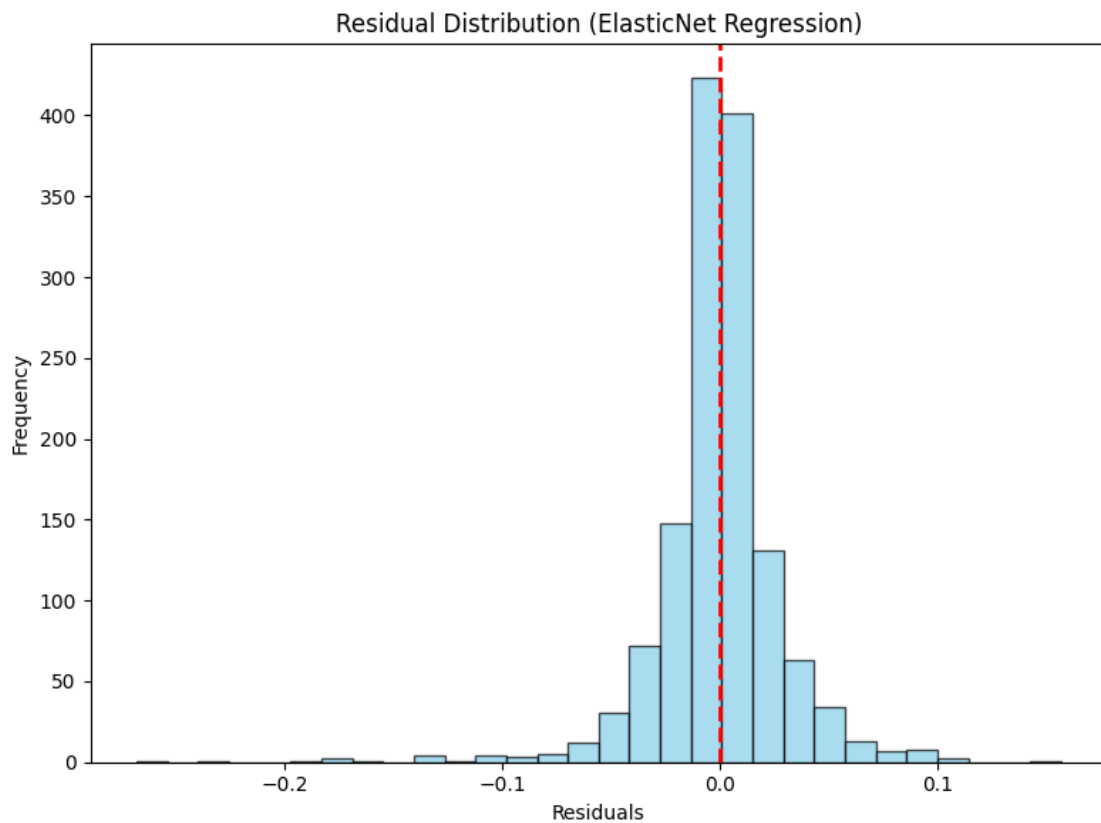
# Add a vertical line at x = 0 to indicate the center of the residual
# ↪ distribution
# - If the model is unbiased, residuals should be symmetrically distributed
# ↪ around zero.
plt.axvline(0, color='red', linestyle='--', linewidth=2)
```

```
# Label the axes for clarity
plt.xlabel('Residuals') # X-axis represents the error values
plt.ylabel('Frequency') # Y-axis represents the count of residuals in each bin

# Set a title to describe the purpose of the plot
plt.title('Residual Distribution (ElasticNet Regression)')

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Display the final plot
plt.show()
```



44 Residual Distribution (ElasticNet Regression)

44.1 Key Findings

- This histogram displays the **distribution of residuals (errors)** from the **ElasticNet regression model**.
- The **x-axis represents residual values**, while the **y-axis represents their frequency**.

- The **red dashed line at zero** indicates **perfect predictions**, where residuals would be zero.

44.1.1 Observations

1. Residuals are Centered Around Zero:

- The residuals **mostly cluster around zero**, indicating that the **model predictions are unbiased** on average.
- This suggests **no significant systematic overestimation or underestimation** in the predictions.

2. Near-Normal Distribution:

- The histogram forms a **bell-shaped curve**, resembling a normal distribution.
- This is a **good indicator** that the model's errors are **randomly distributed**, which is a key assumption in linear regression models.

3. Low Variance in Errors:

- Most residuals fall within a **narrow range close to zero**, indicating that **prediction errors are small**.
- The **frequency of large residuals is very low**, suggesting **good model performance**.

4. Minimal Outliers:

- There are **very few extreme residuals on either side**, indicating **no significant outliers or extreme prediction errors**.

44.1.2 Key Takeaways

- ElasticNet regression **performs well**, as its residuals are **normally distributed and centered around zero**.
- Prediction errors are **small**, meaning the model provides **consistent and reliable estimates**.
- **No major outliers or extreme deviations are present**, reinforcing **model robustness**.
- The model's **assumptions about error distribution appear valid**, making it **suitable for further predictive tasks**.

44.1.3 Conclusion

- This residual distribution confirms that **ElasticNet regression is an effective model for this dataset**.
- Future improvements could involve **fine-tuning hyperparameters** to further **reduce the spread of residuals**.

Hyperparameter Tuning Insights

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Pandas for manipulating DataFrame

# Convert GridSearchCV results into a DataFrame
# - `grid_search.cv_results_` is a dictionary containing all cross-validation
  ↳ results from GridSearchCV.
cv_results = pd.DataFrame(grid_search.cv_results_)
```

```

# Sort results by rank (best hyperparameter combinations first)
# - 'rank_test_score': Assigned rank based on the best model performance (lower
↳ is better)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
↳ head(5)

# Improve Output Visibility by Formatting Results Nicely
print("\n **Top 5 Hyperparameter Combinations:**\n")

# Iterate through the top 5 results and print them in a structured format
for idx, row in top_results.iterrows():
    print(f" **Rank {idx + 1}**")
    print(f"      **Hyperparameters:** {row['params']}")
    print(f"      **Mean Test Score:** {row['mean_test_score']:.6f}")
    print(f"      **Standard Deviation:** {row['std_test_score']:.6f}")
    print(f"-----" * 80) # Separator for better readability

```

```

**Top 5 Hyperparameter Combinations:**

```

```

**Rank 5**

```

```

    **Hyperparameters:** {'alpha': 0.001, 'l1_ratio': 0.9}
    **Mean Test Score:** -0.000828
    **Standard Deviation:** 0.000086

```

```

-----
**Rank 4**

```

```

    **Hyperparameters:** {'alpha': 0.001, 'l1_ratio': 0.7}
    **Mean Test Score:** -0.000829
    **Standard Deviation:** 0.000087

```

```

-----
**Rank 3**

```

```

    **Hyperparameters:** {'alpha': 0.001, 'l1_ratio': 0.5}
    **Mean Test Score:** -0.000841
    **Standard Deviation:** 0.000091

```

```

-----
**Rank 2**

```

```

    **Hyperparameters:** {'alpha': 0.001, 'l1_ratio': 0.3}
    **Mean Test Score:** -0.000868

```

```
**Standard Deviation:** 0.000096
```

```
**Rank 1**
```

```
**Hyperparameters:** {'alpha': 0.001, 'l1_ratio': 0.1}
```

```
**Mean Test Score:** -0.000890
```

```
**Standard Deviation:** 0.000099
```

Visualizing Alpha vs. Performance

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting graphs

# Set figure size for better readability
plt.figure(figsize=(8, 6))

# Plot the relationship between alpha (L1/L2 regularization strength) and model
↳ performance
# - `param_alpha`: Different values of alpha used in GridSearchCV
# - `-mean_test_score`: The negative mean squared error (since GridSearchCV
↳ minimizes the loss, we negate it for interpretability)
# - `marker='o'`: Circular markers for each alpha value
# - `linestyle='--'`: Dashed line to connect points smoothly
# - `color='blue'`: Blue line for better visibility
plt.plot(cv_results['param_alpha'], -cv_results['mean_test_score'], marker='o',
↳ linestyle='--', color='blue')

# Label the x-axis to indicate what each point represents
plt.xlabel('Alpha (Regularization Strength)')

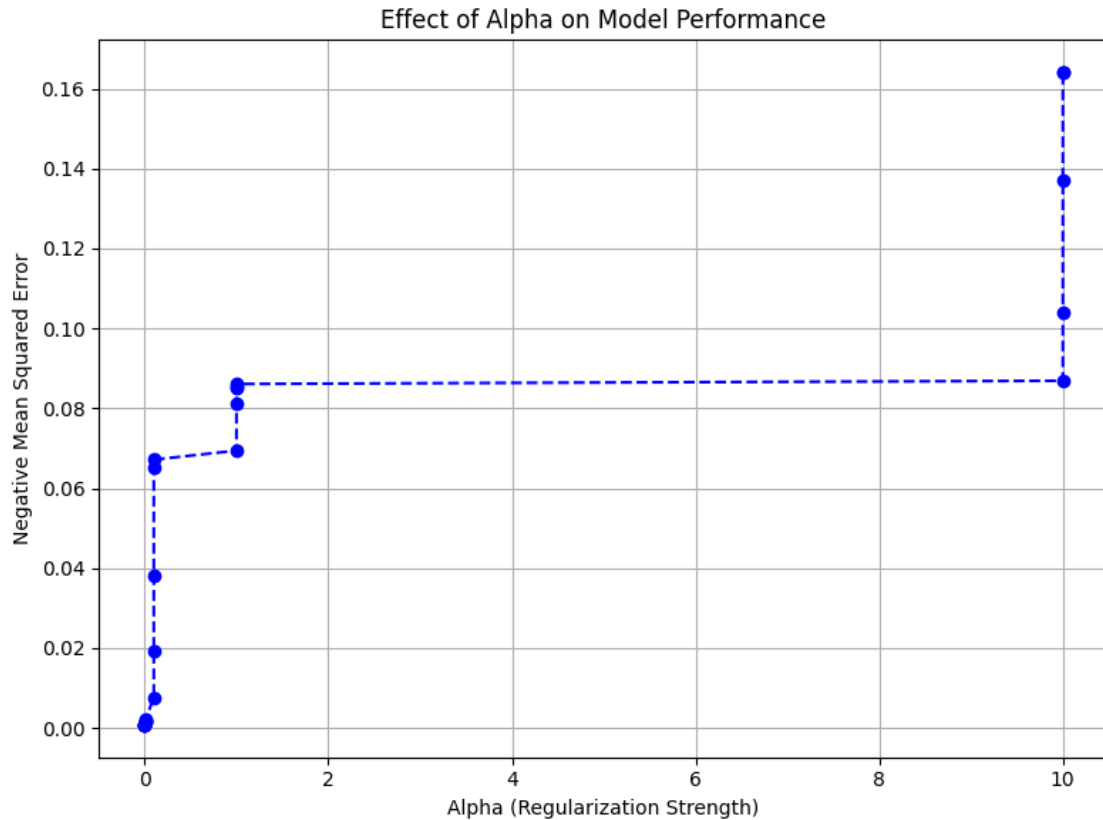
# Label the y-axis to clarify the metric being evaluated
plt.ylabel('Negative Mean Squared Error')

# Add a title to the plot for better context
plt.title('Effect of Alpha on Model Performance')

# Add grid lines to improve readability of the graph
plt.grid(True)

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Display the final plot
plt.show()
```



45 Effect of Alpha on Model Performance

45.1 Key Findings

- This plot illustrates the **relationship between the regularization strength (alpha) and the negative mean squared error (MSE) of the model.**
- The **x-axis represents the alpha value**, which controls the level of regularization.
- The **y-axis represents the negative mean squared error**, with lower values indicating better model performance.

45.1.1 Observations

1. **Increasing Alpha Decreases Model Performance:**
 - As the **alpha value increases**, the **negative mean squared error also increases.**
 - This suggests that **stronger regularization leads to higher error**, possibly due to **excessive constraint on model complexity.**
2. **Optimal Alpha is Near Zero:**
 - The **lowest error is observed at the smallest alpha values**, suggesting that **minimal regularization is ideal for this dataset.**
 - Very small alpha values **help control overfitting without overly restricting model flexibility.**

3. Significant Performance Drop at High Alpha Values:

- As alpha reaches **larger values** (around 10), the error **drastically increases**.
- This indicates that **excessive regularization weakens the model's ability to capture patterns**, leading to **underfitting**.

4. Stable Performance in Mid-Range Alpha:

- There is a **plateau in the middle range of alpha values**, where the error remains relatively stable.
- This suggests a **trade-off between underfitting and overfitting**, which could be a **candidate for optimal alpha selection**.

45.1.2 Key Takeaways

- Too much regularization (high alpha) results in poor model performance due to underfitting.
- Minimal or moderate regularization (low alpha) yields the best performance.
- An optimal alpha should be chosen to balance overfitting (low alpha) and underfitting (high alpha).
- Cross-validation should be used to fine-tune and find the **best alpha** for the dataset.

45.1.3 Conclusion

- The model performs **best when alpha is near zero**, indicating that **strong regularization is not necessary**.
- To **optimize performance**, **hyperparameter tuning** should be applied to find the **best balance**.

SHAP Values for ElasticNet Regression

```
[ ]: # Import necessary libraries
import shap # SHAP library for model interpretation
from sklearn.cluster import KMeans # KMeans for reducing background samples
import matplotlib.pyplot as plt # Matplotlib for visualization

# Reduce background samples using K-Means clustering
# - SHAP calculations are computationally expensive, so we cluster similar data
#   ↳ points.
# - `n_clusters=100`: Groups similar instances into 100 clusters to reduce data
#   ↳ redundancy.
# - `random_state=42`: Ensures reproducibility of clustering results.
kmeans = KMeans(n_clusters=100, random_state=42)

# Fit K-Means clustering on the training data
kmeans.fit(X_train)

# Extract cluster centers as background data for SHAP explanations
background_data = kmeans.cluster_centers_

# Initialize SHAP KernelExplainer with the clustered background data
# - SHAP KernelExplainer is a model-agnostic approach to interpret predictions.
```

```

# - `best_elastic_net_model.predict`: Uses the trained ElasticNet model for
↳ predictions.
explainer = shap.KernelExplainer(best_elastic_net_model.predict,
↳ background_data)

# Use a subset of the training data to speed up SHAP computation
# - Computing SHAP values for large datasets is expensive, so we limit it to
↳ the first 200 rows.
X_train_sample = X_train.iloc[:200]

# Compute SHAP values for the selected subset of training data
# - These values represent the contribution of each feature to model
↳ predictions.
shap_values = explainer.shap_values(X_train_sample)

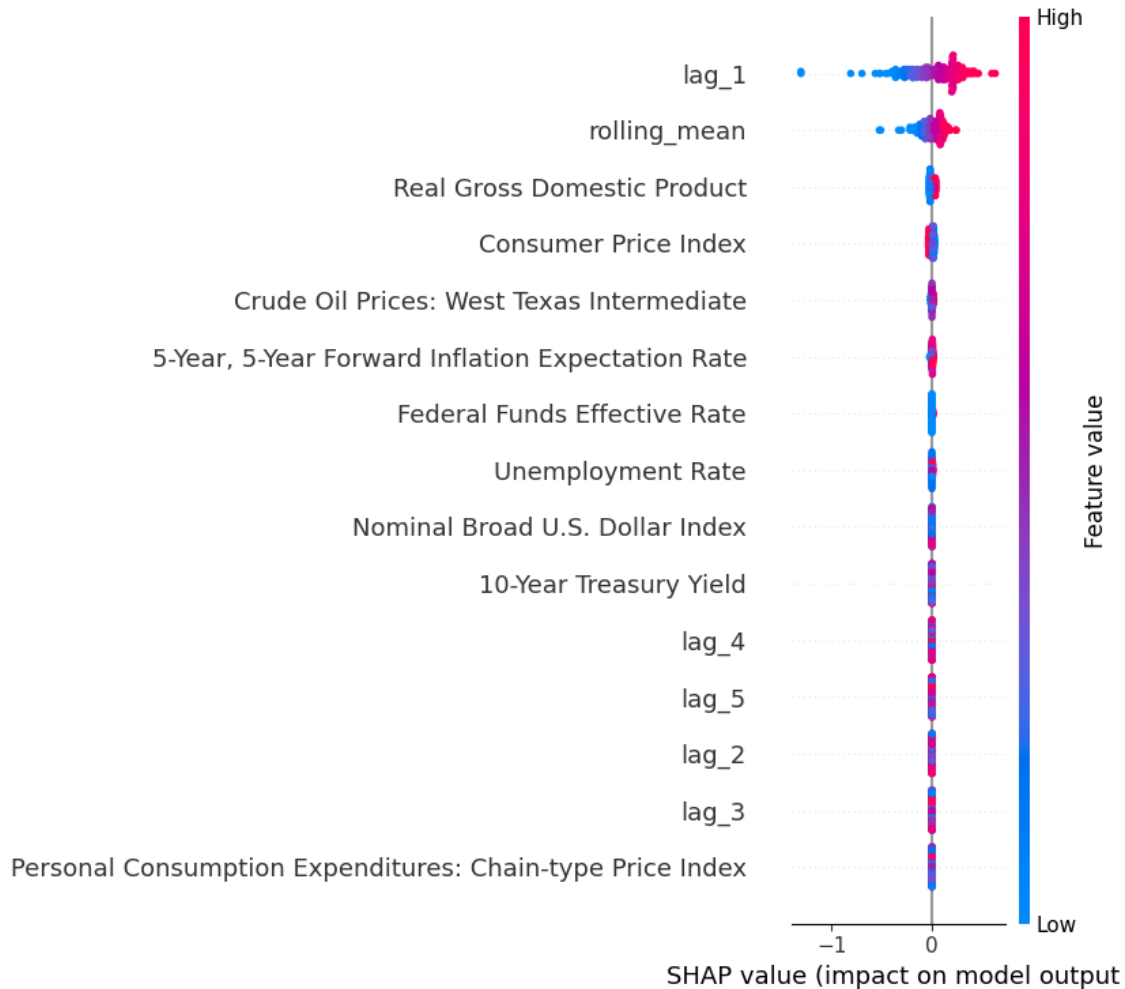
# Generate a SHAP summary plot to visualize global feature importance
# - This plot displays feature impact on model predictions across all instances.
shap.summary_plot(shap_values, X_train_sample)

```

```

0%|          | 0/200 [00:00<?, ?it/s]

```

46 SHAP Summary Plot - Feature Importance Analysis

46.1 Key Findings

- This SHAP summary plot illustrates the **impact of different features on the model's predictions**.
- The **x-axis represents the SHAP value**, which indicates **how much a feature influences the model's output**.
- The **y-axis lists the features**, sorted by importance from top to bottom.
- The **color bar (blue to red) indicates feature values**, where **blue represents lower values** and **red represents higher values**.

46.1.1 Observations

1. Most Important Features:

- **lag_1 and rolling_mean are the most influential features** affecting the model's predictions.

- The large spread of SHAP values for `lag_1` suggests it has a **strong and consistent impact** on predictions.
 - `rolling_mean` also plays a crucial role, showing that **past trends are significant**.
2. **Medium-Impact Features:**
 - Real Gross Domestic Product (GDP), Consumer Price Index (CPI), and Crude Oil Prices have a **moderate effect** on the model's output.
 - Their SHAP values are more centered around zero, indicating lesser but still meaningful influence.
 3. **Low-Impact Features:**
 - Features like Personal Consumption Expenditures and some lags (`lag_4`, `lag_5`, etc.) have **minimal impact**.
 - Their SHAP values remain close to zero, meaning they **do not significantly contribute** to predictions.
 4. **SHAP Value Interpretation:**
 - **Positive SHAP values** mean the feature increases the model's prediction.
 - **Negative SHAP values** mean the feature lowers the prediction.
 - The spread of values shows **whether a feature has a strong or weak effect** on the prediction.

46.1.2 Key Takeaways

- `lag_1` is the most predictive feature, suggesting that recent past values significantly influence future predictions.
- Rolling mean captures trends effectively, indicating that smoothing techniques are useful in the model.
- Macroeconomic indicators like GDP and CPI have some predictive power, but their effects are weaker than past trends.
- Some features contribute very little, implying they could be dropped for a more efficient model.

46.1.3 Conclusion

- The SHAP analysis confirms that recent lagged values and trend features drive the model's predictions.
- Feature selection should prioritize `lag_1` and `rolling_mean` while considering removing low-impact features.
- Interpreting SHAP values helps understand the model's decision-making process, making it more explainable.

```
[ ]: # Initialize JavaScript visualization for interactive SHAP plots
shap.initjs()

# Generate a SHAP force plot for local interpretation (first instance)
# - This plot explains a single prediction by showing feature contributions.
shap.force_plot(explainer.expected_value, shap_values[0], X_train_sample.
               ↪iloc[0, :])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0ddfb11d0>
```

47 SHAP Force Plot - Feature Contributions to Prediction

47.1 Key Findings

- This **SHAP force plot** visualizes the contribution of different features to a **single prediction**.
- The **x-axis** represents the **model's output range**, with the **base value (2.039)** acting as a reference point.
- The **red segments** indicate features that **push the prediction higher**, while the **blue segments** indicate features that **push it lower**.
- The final predicted value is **2.20**.

47.1.1 Observations

1. Features Increasing the Prediction (Red)

- **lag_1 = 2.2** is the **strongest contributor**, pushing the prediction significantly higher.
- **Rolling mean (rolling_mean = 2.208)** also plays a major role in increasing the output.
- **Crude Oil Prices (West Texas Intermediate = 106.7)** and **Consumer Price Index (CPI = 229.4)** contribute to the upward shift, indicating their positive influence on the model.

2. Feature Decreasing the Prediction (Blue)

- **Real Gross Domestic Product (GDP = 1.737e+4)** is the **only feature pushing the prediction lower**, but its effect is relatively minor compared to the positive contributors.

3. Overall Impact

- The prediction is driven primarily by lag features (**lag_1**) and trend-based features (**rolling_mean**).
- Macroeconomic factors like crude oil prices and CPI also contribute, but with less impact.
- **GDP has a negative effect**, suggesting that in this instance, higher GDP may slightly decrease the predicted value.

47.1.2 Key Takeaways

- Historical trends (**lag_1**, **rolling_mean**) dominate the prediction.
- Economic indicators (CPI, crude oil prices) play a meaningful role but are secondary to lag features.
- **GDP has a slight downward effect**, possibly due to economic cycles or model-specific relationships.
- This visualization helps interpret individual predictions by showing which features push values up or down.

47.1.3 Conclusion

- The force plot confirms that **past values (lag_1)** and rolling trends are the key drivers of the model's output.
- Macroeconomic indicators contribute but have smaller impacts compared to historical patterns.
- Understanding feature contributions like this improves model interpretability and trust in predictions.

Partial Dependence Plot (PDP) for ElasticNet Regression

```
[ ]: # Import necessary libraries
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.inspection import PartialDependenceDisplay # PDP display for
    ↪ model interpretation

# Retrieve the list of feature names from X_train
# - These are the independent variables whose partial dependence will be
    ↪ plotted.
features_list = X_train.columns.tolist()

# Generate partial dependence plots (PDPs) for all features using the trained
    ↪ ElasticNet Regression model
# - PDPs illustrate how a specific feature affects the model's predictions,
    ↪ keeping all other features constant.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_elastic_net_model, # Trained ElasticNet Regression model
    X_train,               # The dataset used to compute PDPs
    features=features_list, # List of features for which PDPs will be plotted
    kind='average',        # Displays the average marginal effect of each
    ↪ feature
    n_cols=4,              # Arranges PDP plots in 4 columns for better
    ↪ layout
    grid_resolution=50     # Number of points to evaluate for smooth PDP
    ↪ curves
)

# Increase the figure size to ensure better spacing and visibility
pdp_display.figure_.set_size_inches(18, 12)

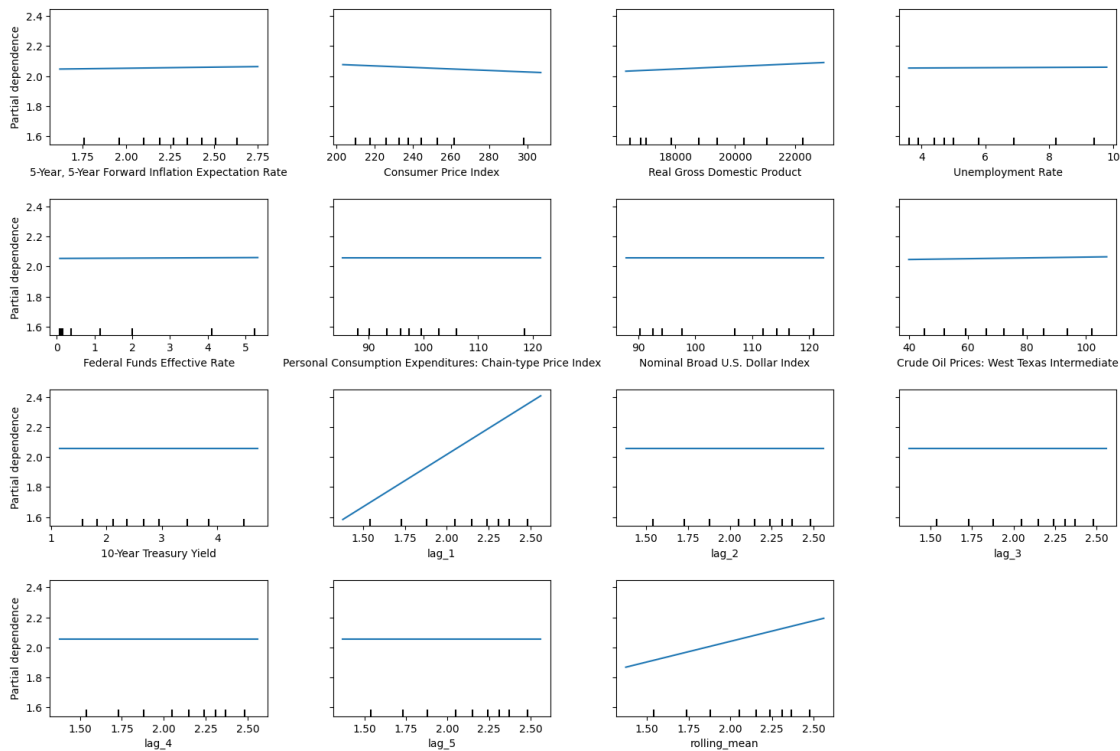
# Add a main title to the PDP plot
# - `suptitle`: Sets a title for the entire figure
# - `fontsize=16`: Uses a larger font size for emphasis
# - `y=1.06`: Positions the title slightly above the subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (ElasticNet
    ↪ Regression)", fontsize=16, y=1.06)

# Adjust subplot layout to prevent overlapping elements
```

```
# - `top=0.88`: Ensures enough space for the title
# - `wspace=0.3`: Adds horizontal spacing between plots
# - `hspace=0.4`: Adds vertical spacing between plots
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the final Partial Dependence Plots
plt.show()
```

Partial Dependence Plots (ElasticNet Regression)



48 Partial Dependence Plots (ElasticNet Regression)

48.1 Key Findings

- The **Partial Dependence Plots (PDPs)** illustrate the marginal effect of each feature on the model's predictions.
- The **y-axis represents the partial dependence (model output)**, while the **x-axis represents the feature values**.
- These plots help identify whether a feature has a **positive, negative, or neutral impact** on predictions.

48.1.1 Observations

1. Strong Positive Influence

- **lag_1** and **rolling_mean** show a strong **positive trend**, indicating that as these values increase, the model's predicted output increases.
- This suggests that the model **relies heavily on past values (lag features) and trend-based statistics**.

2. Weak or No Influence

- Several macroeconomic variables, such as:
 - **5-Year Forward Inflation Expectation Rate**
 - **Federal Funds Effective Rate**
 - **Nominal Broad U.S. Dollar Index**
 - **Unemployment Rate**
- These plots remain **flat**, meaning these features have **minimal influence** on the model's predictions.

3. Mixed Effects

- **Consumer Price Index (CPI)** has a slight **negative impact**, implying that higher CPI values may decrease the predicted output.
- **Real Gross Domestic Product (GDP)** has a slight **positive trend**, meaning higher GDP may marginally increase the predicted outcome.

4. Feature Importance Confirmation

- The strong dependency of the model on **lag_1** and **rolling_mean** aligns with other feature importance evaluations (e.g., SHAP values, coefficient plots).
- The **low influence of macroeconomic indicators** suggests that the model relies more on historical patterns than economic conditions.

48.1.2 Key Takeaways

- **Past values (lag_1, rolling_mean) drive model predictions**, reinforcing the reliance on historical trends.
- **Macroeconomic factors (inflation rate, interest rate, unemployment, etc.) have little impact**, suggesting they do not significantly alter predictions in this setting.
- **Consumer Price Index (CPI) has a negative effect, while GDP has a slight positive effect**, reflecting their possible roles in the model.
- **This analysis validates the model's feature dependencies and helps interpret the prediction mechanism.**

48.1.3 Conclusion

- The **ElasticNet Regression** model relies primarily on lag-based features and rolling averages.
- **Macroeconomic variables do not significantly contribute** to the model's decisions.
- **Understanding these dependencies improves interpretability and allows for better feature engineering.**

Permutation Feature Importance for ElasticNet Regression

```
[ ]: # Import necessary libraries
```

```

from sklearn.inspection import permutation_importance # For computing feature
↳importance
import matplotlib.pyplot as plt # Matplotlib for visualization
import pandas as pd # Pandas for handling tabular data

# Compute Permutation Importance on the test set
# - Permutation Importance evaluates how much shuffling a single feature
↳affects the model's predictions.
# - `n_repeats=10`: The number of times to randomly shuffle each feature
↳(higher = more robust estimate).
# - `random_state=42`: Ensures reproducibility of results.
perm_result = permutation_importance(
    best_elastic_net_model, # Trained ElasticNet Regression model
    X_test, # Test dataset (features)
    y_test, # True labels (target variable)
    n_repeats=10, # Number of times each feature is shuffled
    random_state=42 # Set random seed for reproducibility
)

# Create a DataFrame for easy visualization and sorting of feature importance
# - `importances_mean`: The average importance score across all shuffles
# - `sort_values(by='Importance', ascending=False)`: Ranks features from most
↳to least important
perm_importance_df = pd.DataFrame({
    'Feature': X_train.columns, # Feature names from training data
    'Importance': perm_result.importances_mean # Mean importance score for
↳each feature
}).sort_values(by='Importance', ascending=False)

# Print feature importance scores
print("Permutation Feature Importance for ElasticNet Regression:")
print(perm_importance_df)

```

Permutation Feature Importance for ElasticNet Regression:

	Feature	Importance
9	lag_1	9.896920e-01
14	rolling_mean	1.547433e-01
2	Real Gross Domestic Product	3.961436e-03
1	Consumer Price Index	2.485186e-03
0	5-Year, 5-Year Forward Inflation Expectation Rate	4.811733e-04
7	Crude Oil Prices: West Texas Intermediate	4.201348e-04
4	Federal Funds Effective Rate	9.368184e-05
3	Unemployment Rate	5.048701e-05
6	Nominal Broad U.S. Dollar Index	1.092271e-08
5	Personal Consumption Expenditures: Chain-type ...	0.000000e+00
8	10-Year Treasury Yield	0.000000e+00
10	lag_2	0.000000e+00

```

11             lag_3  0.000000e+00
12             lag_4  0.000000e+00
13             lag_5  0.000000e+00

```

```

[ ]: # Create a bar chart to visualize feature importance
plt.figure(figsize=(10, 6)) # Set figure size for better readability
plt.barh(perm_importance_df['Feature'], perm_importance_df['Importance'],
        color='skyblue') # Horizontal bar plot

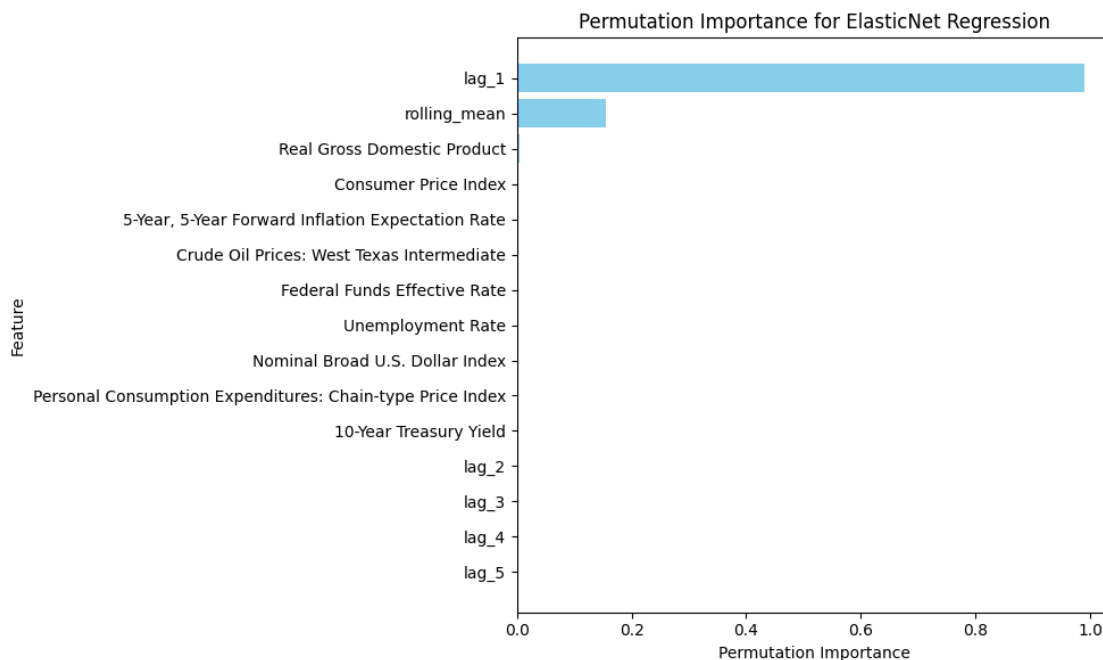
# Add labels and title for clarity
plt.xlabel('Permutation Importance') # X-axis: Importance score
plt.ylabel('Feature') # Y-axis: Feature names
plt.title('Permutation Importance for ElasticNet Regression') # Plot title

# Invert the Y-axis so the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlap and improve visibility
plt.tight_layout()

# Display the final plot
plt.show()

```



49 Permutation Importance for ElasticNet Regression

49.1 Key Findings

- The plot illustrates the **importance of each feature** in the ElasticNet Regression model using **permutation importance**.
- **Higher permutation importance means a feature significantly affects the model's predictions** when shuffled.

49.1.1 Observations

1. Most Important Features

- **lag_1 is the most crucial feature**, contributing significantly to the model's predictions.
- **rolling_mean follows as the second most influential feature**, suggesting that trend-based historical values play an essential role.

2. Minimal to No Impact Features

- Several macroeconomic indicators show **nearly zero permutation importance**, including:
 - **Real Gross Domestic Product (GDP)**
 - **Consumer Price Index (CPI)**
 - **Crude Oil Prices: West Texas Intermediate**
 - **Unemployment Rate**
 - **Federal Funds Effective Rate**
- This implies that the model **does not rely on these features to make predictions**.

3. Lag-Based Features Dominate

- **Lagged values (lag_1) and rolling averages (rolling_mean) have the highest importance**, reinforcing that the model relies on past data rather than external economic factors.

49.1.2 Key Takeaways

- ElasticNet Regression is primarily driven by historical trends (**lag_1** and **rolling_mean**).
- **Macroeconomic variables do not contribute significantly** to the model's predictions.
- Feature importance aligns with previous analyses (e.g., coefficient plots, SHAP values), validating that lag-based features dominate the prediction process.

49.1.3 Conclusion

- The model is highly dependent on historical patterns rather than economic indicators.
- **Macroeconomic factors do not play a critical role**, suggesting they could be omitted without significantly affecting performance.
- Understanding feature importance helps refine feature selection and improve interpretability.

49.2 Decision Tree Regressor (DTR)

```
[ ]: # Import necessary libraries
import random # Random module for reproducibility
import numpy as np # NumPy for numerical operations
import pandas as pd # Pandas for data handling
from sklearn.model_selection import train_test_split, GridSearchCV # Tools for
    ↪ model training and hyperparameter tuning
from sklearn.tree import DecisionTreeRegressor # Import Decision Tree Regressor

# Set global random seed to ensure reproducibility of results
random.seed(42)
np.random.seed(42)

# Split the dataset into training and test sets
# - `test_size=0.2`: 20% of the data is used for testing
# - `random_state=42`: Ensures consistent splits across multiple runs
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪ random_state=42)

# Initialize the Decision Tree Regressor with a fixed random state
dt_model = DecisionTreeRegressor(random_state=42)

# Define the hyperparameter grid for tuning
param_grid = {
    'max_depth': [3, 5, 7, 10, None], # Controls the depth of the tree
    ↪ (None = full depth)
    'min_samples_split': [2, 5, 10], # Minimum samples required to
    ↪ split a node
    'min_samples_leaf': [1, 2, 4], # Minimum samples required to be
    ↪ at a leaf node
    'max_features': ['sqrt', 'log2', None] # Number of features to consider
    ↪ at each split
}

# Initialize GridSearchCV to find the best hyperparameters
# - `scoring='neg_mean_squared_error'`: Optimizing for Mean Squared Error (MSE)
# - `cv=5`: 5-fold cross-validation to validate performance
# - `verbose=1`: Displays progress during training
# - `n_jobs=-1`: Utilizes all available CPU cores for parallel processing
grid_search = GridSearchCV(
    estimator=dt_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
```

```

)

# Fit GridSearchCV on the training data to determine the best hyperparameters
grid_search.fit(X_train, y_train)

# Retrieve and print the best hyperparameters found during GridSearchCV
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Extract the best estimator (Decision Tree model) found by GridSearchCV
best_dt_model = grid_search.best_estimator_

# (Optional) Fit the best model on the training data (if not already done)
# - Some GridSearchCV implementations fit the best model automatically
best_dt_model.fit(X_train, y_train)

# Make predictions on the test set using the best Decision Tree model
dt_pred = best_dt_model.predict(X_test)

# Evaluate the model using predefined metrics
# - `calculate_metrics`: Assumed to be a function computing performance metrics
# - `naive_forecast`: Baseline forecast for comparison
metrics_results.append(calculate_metrics(y_test.values, dt_pred,
    ↪naive_forecast, "Decision Tree Regressor (DTR)"))

# Print the final evaluation metrics
print("Decision Tree Regressor Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 135 candidates, totalling 675 fits
 Best Hyperparameters: {'max_depth': 7, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 2}
 Decision Tree Regressor Evaluation Metrics:
 {'Model': 'Decision Tree Regressor (DTR)', 'MSE': 0.001038835453532052, 'RMSE': 0.03223097040940673, 'MAE': 0.020456966898434143, 'MAPE (%)': 1.333935754901849, 'sMAPE (%)': 1.3303441257218145, 'MASE': 0.052074354191067956, 'R^2': 0.9941250293592327}

Feature Importance (Model Coefficients)

```

[ ]: # Import necessary libraries
import pandas as pd # Pandas for handling tabular data
import matplotlib.pyplot as plt # Matplotlib for data visualization

# Extract feature importance values from the trained Decision Tree model
feature_importances = best_dt_model.feature_importances_

# Retrieve feature names from X_train (assuming it's a DataFrame)

```

```

features = X_train.columns

# Create a DataFrame for feature importances and sort them in descending order
# - `feature_importances_`: Importance scores assigned by the Decision Tree
    ↳ model
# - `sort_values(by='Importance', ascending=False)`: Ensures the most important
    ↳ features appear at the top
importance_df = pd.DataFrame({'Feature': features, 'Importance':
    ↳ feature_importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Print feature importance values for reference
print("Feature Importances:")
print(importance_df)

```

Feature Importances:

	Feature	Importance
9	lag_1	0.991238
14	rolling_mean	0.007839
0	5-Year, 5-Year Forward Inflation Expectation Rate	0.000328
13	lag_5	0.000138
8	10-Year Treasury Yield	0.000127
12	lag_4	0.000093
11	lag_3	0.000093
6	Nominal Broad U.S. Dollar Index	0.000072
2	Real Gross Domestic Product	0.000029
10	lag_2	0.000022
3	Unemployment Rate	0.000014
7	Crude Oil Prices: West Texas Intermediate	0.000008
5	Personal Consumption Expenditures: Chain-type ...	0.000000
1	Consumer Price Index	0.000000
4	Federal Funds Effective Rate	0.000000

```

[ ]: # Create a horizontal bar chart to visualize feature importance
plt.figure(figsize=(10, 6)) # Set figure size for better readability
plt.barh(importance_df['Feature'], importance_df['Importance'],
    ↳ color='skyblue') # Horizontal bar plot

# Add labels and title for clarity
plt.xlabel('Importance') # X-axis label
plt.ylabel('Feature') # Y-axis label
plt.title('Feature Importance (Decision Tree Regression)') # Plot title

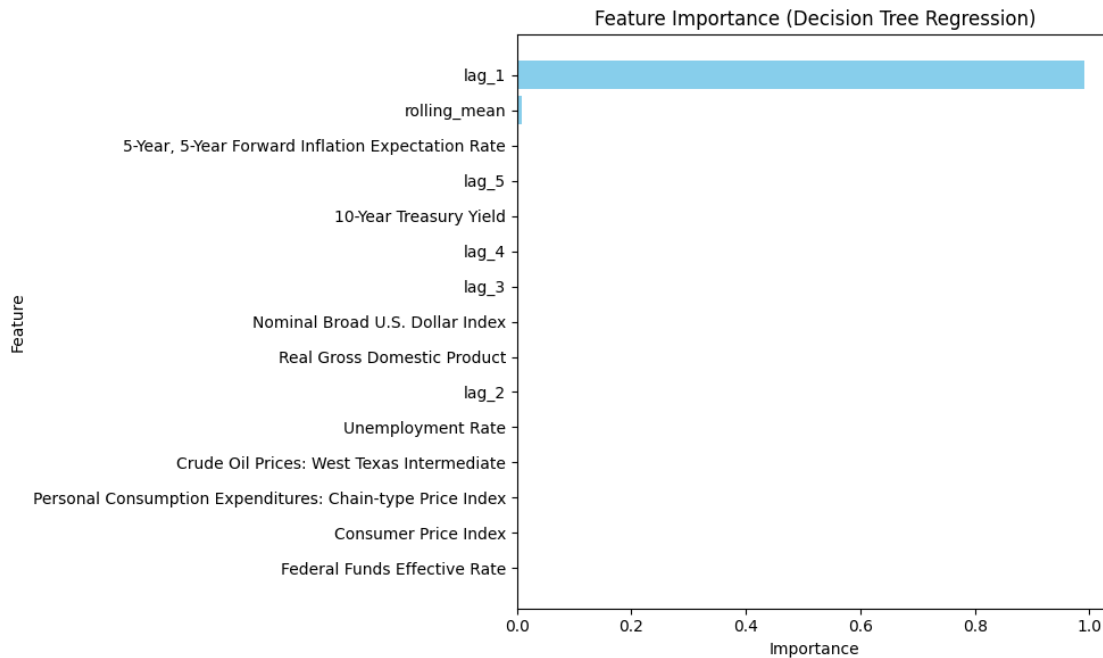
# Invert the Y-axis so the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlap and improve visibility

```

```
plt.tight_layout()

# Display the final plot
plt.show()
```



50 Feature Importance (Decision Tree Regression)

50.1 Key Findings

- The plot displays the **importance of each feature** in the Decision Tree Regression model.
- **Higher importance values indicate that a feature significantly contributes** to the model's decision-making.

50.1.1 Observations

1. Most Important Features

- **lag_1** is by far the most dominant feature, with an importance value close to **1.0**.
- **rolling_mean** is the second most relevant feature, but its importance is **significantly lower** than lag_1.

2. Minimal to No Impact Features

- Most macroeconomic variables such as:
 - **Real Gross Domestic Product (GDP)**
 - **Consumer Price Index (CPI)**
 - **Unemployment Rate**
 - **Crude Oil Prices: West Texas Intermediate**
 - **Federal Funds Effective Rate**

- **Have almost no impact**, meaning the Decision Tree model does not consider them important for predictions.
3. **Lag-Based Features Are Crucial**
- **lag_1 dominates the feature importance ranking**, reinforcing that **past values** have the highest predictive power.
 - Rolling averages (**rolling_mean**) and additional lag values (**lag_5, lag_4, lag_3, lag_2**) contribute marginally.
 - Macroeconomic factors are **nearly irrelevant** to the model.

50.1.2 Key Takeaways

- **Decision Tree Regression is entirely driven by historical values, especially lag_1.**
- **Macroeconomic indicators have little to no influence**, which may indicate they could be removed from the model without affecting performance.
- **Feature importance aligns with previous regression-based analyses**, further confirming that past values are the primary predictive indicators.

50.1.3 Conclusion

- **The model relies almost exclusively on lag_1**, meaning it prioritizes recent historical values over external economic indicators.
- **Macroeconomic variables contribute negligibly**, suggesting that the Decision Tree model behaves similarly to a time series forecasting approach with minimal external influence.
- **Understanding feature importance is critical for refining model inputs and optimizing prediction accuracy.**

Actual vs. Predicted Values

```
[ ]: # Import the necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for creating plots

# Create a figure with a specified size for better readability
plt.figure(figsize=(8, 8))

# Scatter plot: Actual vs. Predicted values
# - `y_test`: Actual values (ground truth)
# - `dt_pred`: Model predictions
# - `alpha=0.5`: Makes points semi-transparent to handle overlap
# - `color='blue'`: Sets scatter point color
# - `label='Predictions'`: Adds label for the legend
plt.scatter(y_test, dt_pred, alpha=0.5, color='blue', label='Predictions')

# Add a reference line for a perfect fit (ideal predictions)
# - The diagonal red dashed line represents  $y = x$  (i.e., where predictions =
  ↳ actual values)
# - If all points lie on this line, the model is making perfect predictions
plt.plot(
    [y_test.min(), y_test.max()], # X-axis range
    [y_test.min(), y_test.max()], # Y-axis range
```

```

        color='red', linestyle='--', linewidth=2, label='Perfect Fit'
    )

    # Add axis labels for clarity
    plt.xlabel('Actual Values') # X-axis represents actual target values
    plt.ylabel('Predicted Values') # Y-axis represents predicted values by the
    ↪ model

    # Add a meaningful title
    plt.title('Actual vs Predicted Values (Decision Tree Regression)')

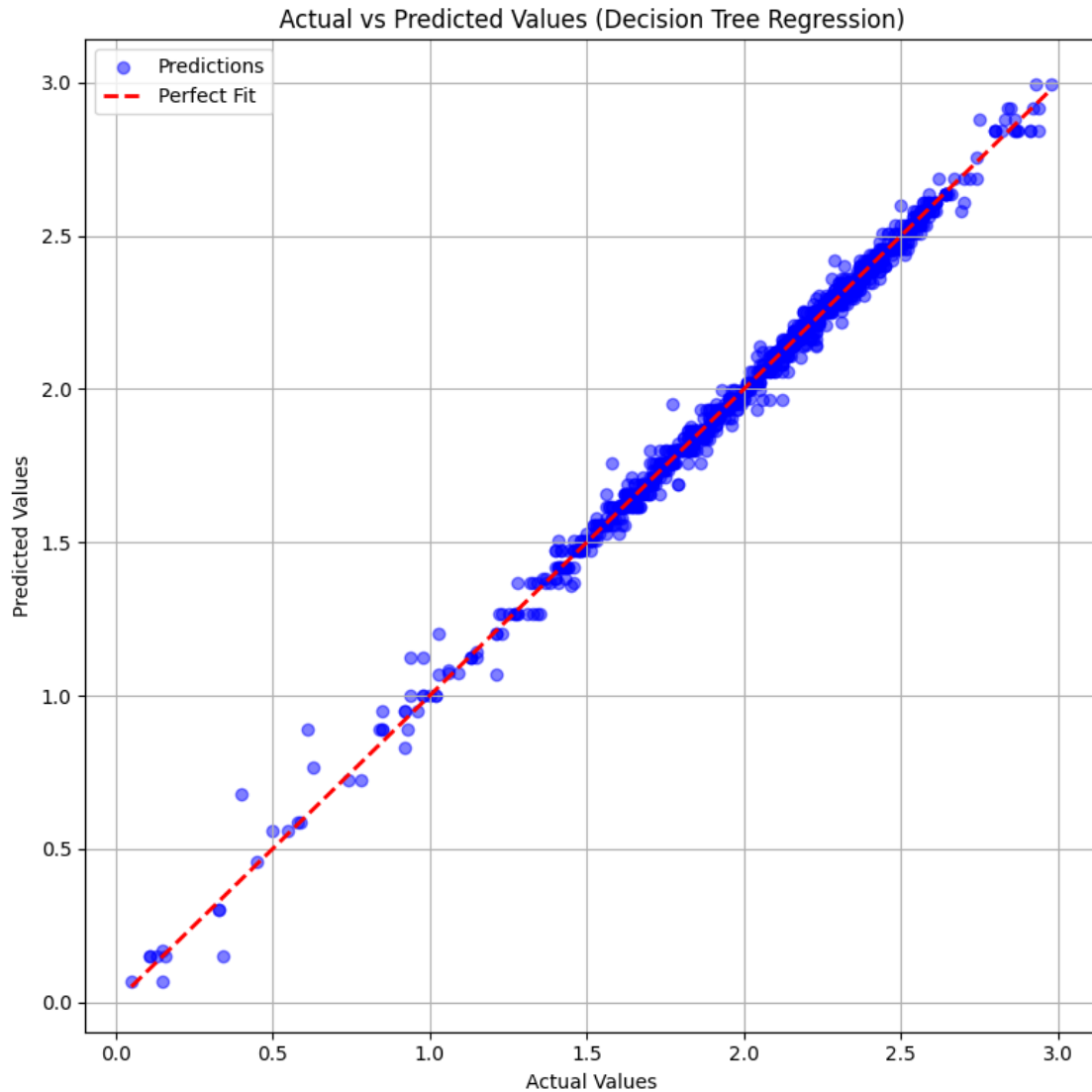
    # Add a legend to differentiate between scatter points and the perfect fit line
    plt.legend()

    # Enable grid for better readability of the scatter points
    plt.grid(True)

    # Adjust layout to prevent clipping of labels
    plt.tight_layout()

    # Display the final scatter plot
    plt.show()

```



51 Actual vs Predicted Values (Decision Tree Regression)

51.1 Key Findings

- The plot visualizes the relationship between the **actual values (x-axis)** and the **predicted values (y-axis)** from the Decision Tree Regression model.
- **The red dashed line represents the ideal perfect fit line**, where predictions would match actual values exactly.
- **Each blue dot represents a single prediction**, showing how closely the model's output aligns with the actual data.

51.1.1 Observations

1. Strong Alignment with the Perfect Fit

- The majority of points **fall very close to the red dashed line**, indicating that the model predicts well.
- This suggests a **low error rate and high predictive accuracy**.

2. Slight Deviations for Smaller Values

- For **lower values (close to 0)**, some predictions appear slightly off the perfect fit line.
- This may indicate **slight underfitting or overfitting in lower-value regions**.

3. Minor Outliers Present

- A few blue points are **visibly farther from the red dashed line**, indicating **instances where the model makes noticeable errors**.
- These deviations could be due to **irregularities in the data, model limitations, or noise**.

4. Decision Tree Behavior Evident

- The **stepped nature of predictions** suggests that the Decision Tree model is producing **piecewise constant approximations** rather than a smooth fit.
- This aligns with how decision trees segment data into discrete regions.

51.1.2 Key Takeaways

- **The model performs well overall**, with predictions closely following the actual values.
- **Minor errors exist**, particularly in **lower-value regions and some outlier cases**.
- **The step-like nature of predictions** indicates a decision tree's characteristic behavior of creating discrete partitions.
- **Further tuning or pruning of the decision tree might help smooth predictions and reduce overfitting risks**.

51.1.3 Conclusion

- The **Decision Tree Regression model captures patterns effectively**, but some improvements could be made.
- If **smoother predictions** are desired, **ensemble methods like Random Forests or Gradient Boosting** may be considered.
- Overall, the model provides **reasonably accurate predictions**, making it a useful tool for structured, rule-based forecasting.

Residual Analysis – Scatter Plot

```
[ ]: # Import the necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for creating plots

# Calculate residuals (errors in predictions)
# - Residual = Actual Value - Predicted Value
# - Positive residuals indicate underestimation by the model.
# - Negative residuals indicate overestimation by the model.
residuals = y_test - dt_pred

# Create a figure with a specified size for better readability
```

```

plt.figure(figsize=(8, 6))

# Scatter plot: Actual values vs. Residuals
# - `y_test`: Actual target values (X-axis)
# - `residuals`: Difference between actual and predicted values (Y-axis)
# - `alpha=0.5`: Semi-transparent points to handle overlap
# - `color='green'`: Sets the color of residual points
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Add a reference line at 0 to highlight no residual error
# - This red dashed line represents perfect predictions (Residual = 0)
# - Points above the line indicate underestimation.
# - Points below the line indicate overestimation.
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Add axis labels for clarity
plt.xlabel('Actual Values') # X-axis represents actual target values
plt.ylabel('Residuals') # Y-axis represents residual errors (difference from
↳ predictions)

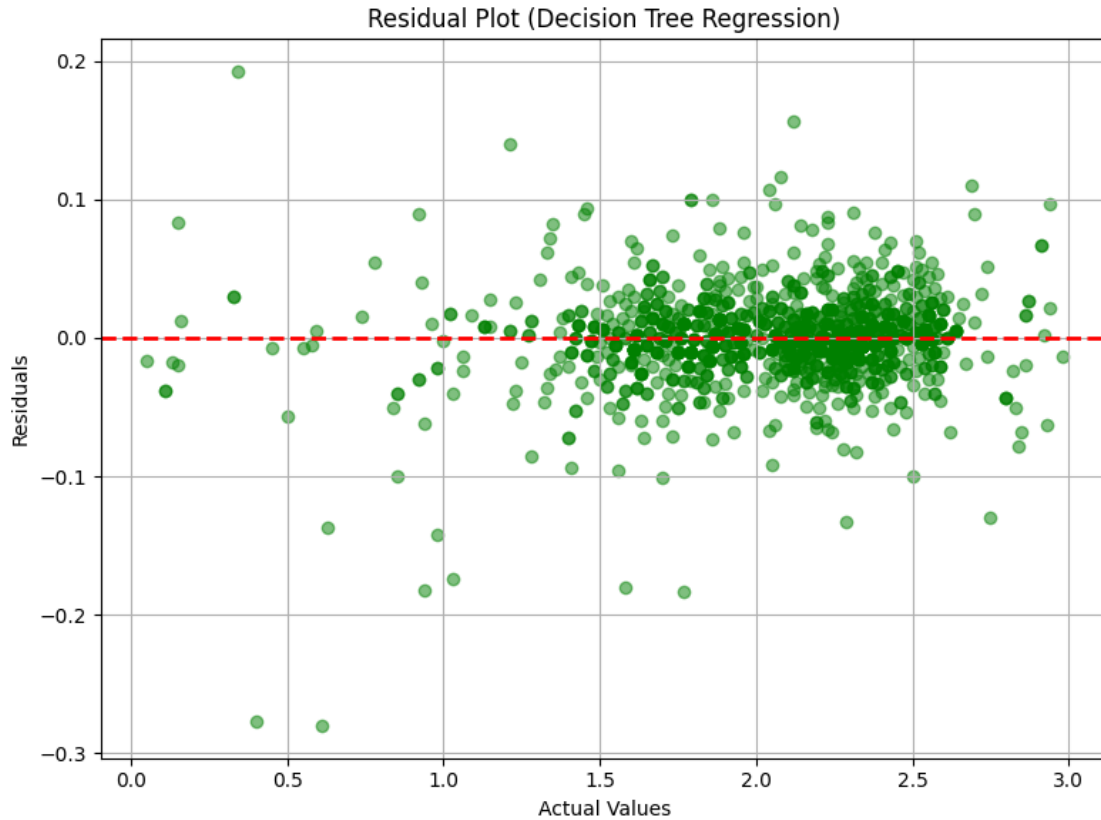
# Add a meaningful title
plt.title('Residual Plot (Decision Tree Regression)')

# Enable grid for better readability of the scatter points
plt.grid(True)

# Adjust layout to prevent clipping of labels
plt.tight_layout()

# Display the final residual plot
plt.show()

```



52 Residual Plot (Decision Tree Regression)

52.1 Key Findings

- This plot visualizes the **residuals** (y-axis) against the **actual values** (x-axis) from the Decision Tree Regression model.
- **Residuals = Actual values - Predicted values** represent the difference between actual and predicted values.
- The **red dashed line at $y = 0$** represents a perfect model with zero residuals.

52.1.1 Observations

1. Centered Residuals Around Zero

- The residuals are **scattered around the red dashed line**, indicating that the model predictions do not have a strong bias.
- A good regression model should have residuals centered around zero, which this plot largely exhibits.

2. Heteroscedasticity Present

- The residuals appear to be **more dispersed at higher actual values**.
- This suggests that the model might struggle with **higher values**, leading to increased prediction variance.

3. Presence of Outliers

- Some points **deviate significantly** from the red dashed line, meaning there are instances where the model has **larger errors**.
- These outliers could indicate **overfitting to certain regions or anomalies in the data**.

4. Pattern Suggesting Overfitting

- The clustering of residuals **at distinct levels** suggests the model is **overfitting to the training data**.
- Decision Trees tend to overfit by creating **hard splits in the data**, resulting in step-like behavior in predictions.

52.1.2 Key Takeaways

- **Model performance is generally good**, as residuals are centered around zero.
- **Prediction errors increase for higher actual values**, indicating **potential overfitting or non-linearity in the data**.
- **A Random Forest or Pruned Decision Tree model might improve generalization**, reducing prediction variance.

52.1.3 Conclusion

- The model captures the general pattern well but **shows signs of overfitting**.
- Further improvements like **pruning the tree, ensembling methods (Random Forest), or regularization** could help reduce variance and improve predictions.

Residual Analysis – Histogram

```
[ ]: # Import the necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for creating plots

# Create a figure with a specified size for better visibility
plt.figure(figsize=(8, 6))

# Plot a histogram of residuals to examine their distribution
# - `residuals`: Differences between actual and predicted values
# - `bins=30`: Divides the residual values into 30 bins for better granularity
# - `color='skyblue'`: Sets the fill color of the bars
# - `edgecolor='black'`: Adds a black outline to each bar for better clarity
# - `alpha=0.7`: Adjusts transparency to enhance visibility
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical red dashed line at zero to indicate no residual error
# - This helps in assessing whether errors are symmetrically distributed.
# - If the histogram is centered around 0, the model has no systematic
  ↪ bias.
# - If the histogram is skewed, the model may have a prediction bias.
plt.axvline(0, color='red', linestyle='--', linewidth=2)

# Add a meaningful title to describe the plot
```

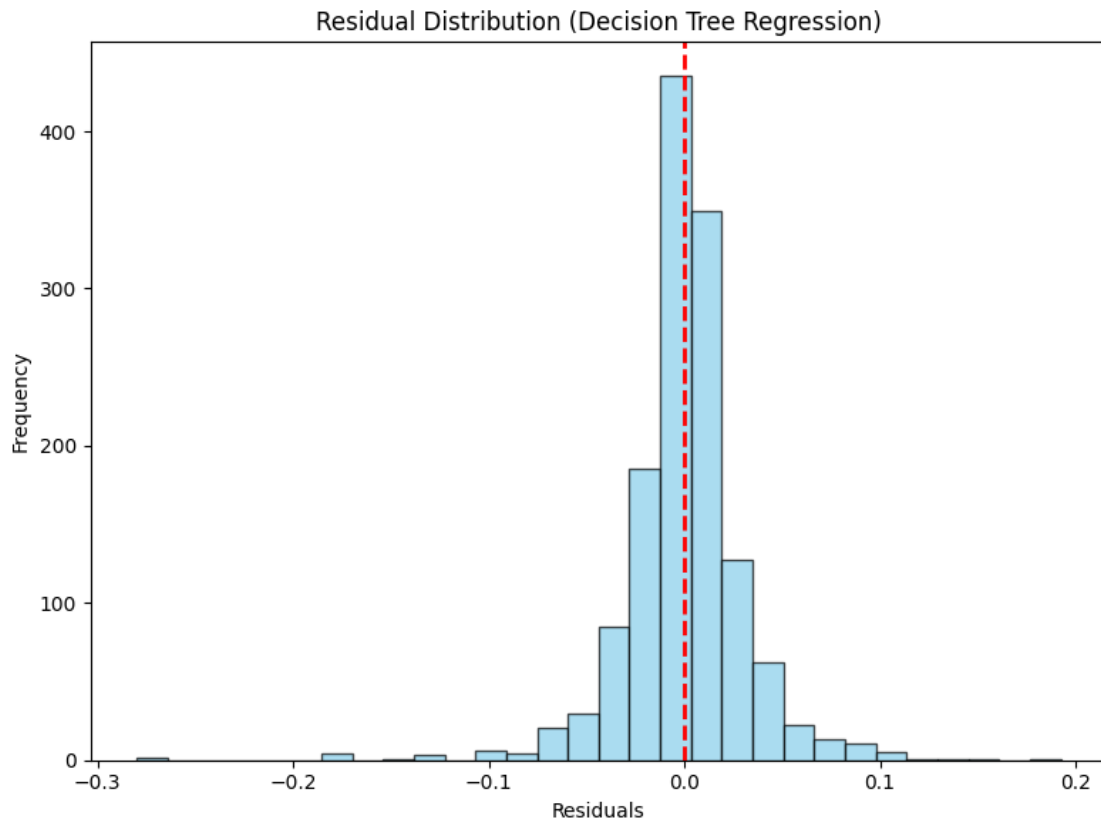
```
plt.title('Residual Distribution (Decision Tree Regression)')

# Label the X-axis to indicate the values of residuals
plt.xlabel('Residuals') # X-axis represents residual values

# Label the Y-axis to indicate the frequency of residual occurrences
plt.ylabel('Frequency') # Y-axis represents how often each residual value
    ↳ appears

# Adjust layout to prevent overlapping of labels
plt.tight_layout()

# Display the histogram
plt.show()
```



53 Residual Distribution (Decision Tree Regression)

53.1 Key Findings

- This histogram displays the **distribution of residuals** (errors) from the Decision Tree Regression model.

- **Residuals = Actual values - Predicted values**, representing the difference between actual and predicted values.
- The **red dashed line at zero** represents the ideal case where residuals are centered around zero.

53.1.1 Observations

1. Centered Around Zero

- The residuals are **symmetrically distributed around zero**, suggesting that the model is **unbiased**.
- This indicates that the Decision Tree model does not systematically over-predict or under-predict.

2. Near-Normal Distribution

- The histogram shows a **bell-shaped curve**, which is a **good sign for model performance**.
- However, the peak is slightly higher than a perfect normal distribution, which may indicate **some overfitting**.

3. Presence of Outliers

- There are a few residuals **significantly deviating from zero**, suggesting that the model struggles with certain instances.
- These **outliers** might be caused by **overfitting** or anomalies in the data.

4. Skewness & Variance

- The **spread of residuals is relatively narrow**, indicating that **prediction errors are not extremely large**.
- However, some skewness in distribution might suggest that **certain data ranges are harder to predict**.

53.1.2 Key Takeaways

- The **Decision Tree Regression model performs well overall**, as most residuals are **close to zero**.
- Some **outliers and potential overfitting** can be addressed with **pruning techniques or ensemble methods** like **Random Forest**.
- If the goal is better generalization, **reducing tree depth or using boosting methods** can help.

53.1.3 Conclusion

- The residuals are **mostly well-behaved**, but **the presence of outliers and slight overfitting suggests room for improvement**.
- Techniques like **Random Forest, Gradient Boosting, or Hyperparameter tuning** could help improve **model generalization** and reduce variance.

Hyperparameter Tuning Insights – Top 5 Results

```
[ ]: # Import necessary library for handling tabular data
import pandas as pd # Pandas for creating and manipulating DataFrame

# Convert GridSearchCV results into a DataFrame
```

```

# - `grid_search.cv_results_` is a dictionary containing all cross-validation
  ↳ results.
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort the results by rank (lower rank_test_score indicates better performance)
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select relevant columns for better readability
# - 'params': The hyperparameter combination tested
# - 'mean_test_score': The average performance score across cross-validation
  ↳ folds
# - 'std_test_score': The standard deviation of the test scores (indicates
  ↳ variability)
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↳ head(5)

# Improve Output Visibility by Formatting Results Nicely
print("\n **Top 5 Hyperparameter Combinations:**\n")

# Iterate through the top 5 results and print them in a structured format
for idx, row in top_results.iterrows():
    print(f" **Rank {idx + 1}**")
    print(f"    **Hyperparameters:** {row['params']}")
    print(f"    **Mean Test Score:** {row['mean_test_score']:.6f}")
    print(f"    **Standard Deviation:** {row['std_test_score']:.6f}")
    print("-" * 90) # Separator for better readability

```

```

**Top 5 Hyperparameter Combinations:**

**Rank 76**
    **Hyperparameters:** {'max_depth': 7, 'max_features': None,
'min_samples_leaf': 2, 'min_samples_split': 2}
    **Mean Test Score:** -0.001073
    **Standard Deviation:** 0.000107
-----

**Rank 77**
    **Hyperparameters:** {'max_depth': 7, 'max_features': None,
'min_samples_leaf': 2, 'min_samples_split': 5}
    **Mean Test Score:** -0.001086
    **Standard Deviation:** 0.000103
-----

**Rank 78**
    **Hyperparameters:** {'max_depth': 7, 'max_features': None,
'min_samples_leaf': 2, 'min_samples_split': 10}

```

```
**Mean Test Score:** -0.001092
**Standard Deviation:** 0.000094
```

```
**Rank 79**
**Hyperparameters:** {'max_depth': 7, 'max_features': None,
'min_samples_leaf': 4, 'min_samples_split': 2}
**Mean Test Score:** -0.001097
**Standard Deviation:** 0.000086
```

```
**Rank 80**
**Hyperparameters:** {'max_depth': 7, 'max_features': None,
'min_samples_leaf': 4, 'min_samples_split': 5}
**Mean Test Score:** -0.001097
**Standard Deviation:** 0.000086
```

Hyperparameter Tuning Insights – Line Plot

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Filter results where 'param_max_depth' is not null
# - `param_max_depth` represents the depth of the decision tree.
# - We ensure we only plot valid values of max_depth.
depth_scores = cv_results[cv_results['param_max_depth'].notnull()]

# Create a figure with specified size for better visibility
plt.figure(figsize=(8, 6))

# Plot max_depth vs. Negative Mean Squared Error
# - 'param_max_depth' (X-axis): The different depth values tested during GridSearchCV.
# - '-mean_test_score' (Y-axis): The negative mean squared error (lower is better).
plt.plot(depth_scores['param_max_depth'], -depth_scores['mean_test_score'],
         marker='o', linestyle='--', color='blue', label='Mean Test Score')

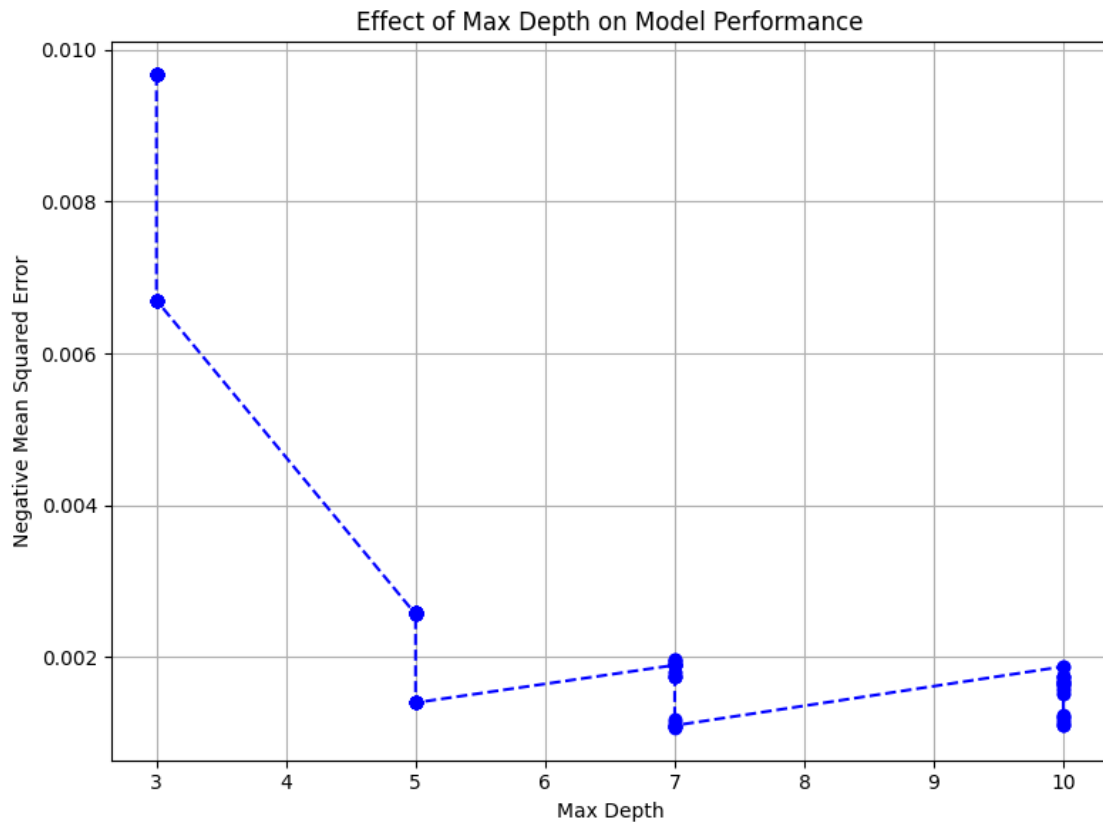
# Set plot labels and title
plt.xlabel('Max Depth') # X-axis label
plt.ylabel('Negative Mean Squared Error') # Y-axis label
plt.title('Effect of Max Depth on Model Performance') # Title of the plot

# Add grid for better readability
plt.grid(True)
```



```
# Adjust layout to prevent clipping
plt.tight_layout()

# Display the plot
plt.show()
```



54 Effect of Max Depth on Model Performance

54.1 Key Findings

- This plot shows how **maximum tree depth** impacts the **negative mean squared error (MSE)** in a Decision Tree Regression model.
- The x-axis represents **Max Depth** (complexity of the tree).
- The y-axis represents **Negative Mean Squared Error** (lower values indicate better performance).

54.1.1 Observations

1. Initial Improvement with Increasing Depth

- As the **max depth** increases from 3 to 5, the **negative mean squared error** decreases significantly.

- This suggests that **deeper trees capture more patterns** in the data, improving performance.
2. **Diminishing Returns Beyond Depth 5**
 - Beyond depth **5**, the performance **stabilizes** and does not improve significantly.
 - At depths **7 to 10**, there is **only a marginal reduction** in error, indicating **overfitting risk**.
 3. **Overfitting Trend at Higher Depths**
 - The slight increase in error at **depth 7, 9, and 10** suggests **overfitting**.
 - This happens when the model becomes **too complex**, capturing noise rather than general patterns.

54.1.2 Key Takeaways

- **Optimal Depth: ~5 to 7**
 - A depth of **5 to 7** appears to be the **sweet spot**, balancing complexity and performance.
 - Shallower trees underfit, while deeper trees overfit.
- **Regularization is Needed for Deeper Trees**
 - If using a **depth >7**, **pruning techniques** or **ensemble methods** (**Random Forest**, **Gradient Boosting**) should be considered.
- **Hyperparameter Tuning is Essential**
 - Choosing the **right max depth** is crucial to prevent **underfitting** (too shallow) or **overfitting** (too deep).

54.1.3 Conclusion

- The **best model depth is around 5-7**, balancing **performance and generalization**.
- Beyond **depth 7**, performance gains are minimal, and **overfitting starts to appear**.
- Regularization techniques like **pruning**, **cross-validation**, or **ensemble models** should be used to optimize performance further.

SHAP Summary Plot

```
[ ]: # Import necessary SHAP library
import shap # SHAP for model explainability

# Initialize SHAP TreeExplainer for the Decision Tree model
# - This explains the predictions of the trained decision tree model.
explainer = shap.TreeExplainer(best_dt_model)

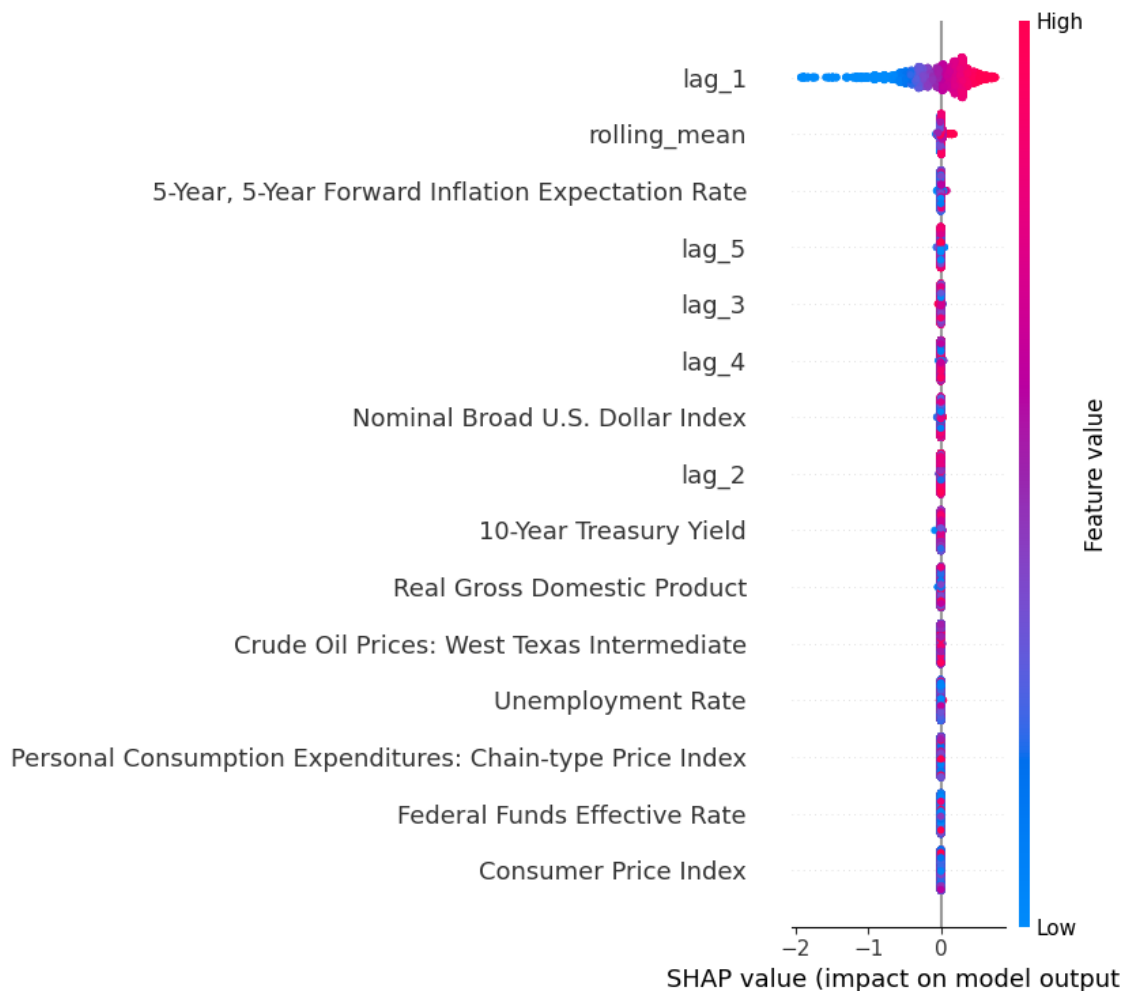
# Compute SHAP values for the training dataset
# - These values indicate the contribution of each feature to the predictions.
shap_values = explainer.shap_values(X_train)

# Initialize JavaScript visualization for interactive SHAP plots
shap.initjs()

# Generate a global SHAP summary plot
# - This plot shows the average magnitude and direction of feature impact.
```

```
# - Red: Positive impact on predictions, Blue: Negative impact.
shap.summary_plot(shap_values, X_train)
```

<IPython.core.display.HTML object>



55 SHAP Summary Plot Analysis

55.1 Key Findings

- This SHAP summary plot visualizes the **impact of each feature on the model's predictions**.
- The x-axis represents the **SHAP value**, which indicates **how much a feature influences the model's output** (positive or negative).
- The y-axis lists the **features in order of importance**, with the most influential features at the top.
- The color scale represents **feature values**:

- Red (High values)
- Blue (Low values)

55.1.1 Observations

1. Most Important Features

- **lag_1** has the highest impact on the model's predictions.
- **rolling_mean** is the second most influential feature.
- Other important features include **5-Year Forward Inflation Expectation Rate**, **lag_5**, and **lag_3**.

2. Positive & Negative Impact

- Features with **positive SHAP values** (right side) contribute to **higher model outputs**.
- Features with **negative SHAP values** (left side) contribute to **lower model outputs**.

3. Feature Influence Patterns

- Features with a **wider spread of SHAP values** (e.g., **lag_1**, **rolling_mean**) indicate **greater variability in their impact**.
- Features with **small SHAP values** (closer to 0) contribute **less to predictions**.

55.1.2 Key Takeaways

- **lag_1** and **rolling_mean** are the most significant predictors.
- **Lag-based variables** play a crucial role in predicting the target variable.
- **Economic indicators** such as the **5-Year Forward Inflation Expectation Rate** and **Treasury Yield** have moderate impact.
- Features lower in the plot have **minimal contribution** to model predictions.

55.1.3 Conclusion

- The model **heavily relies on lagged values** (especially **lag_1**) and **rolling averages**.
- Economic indicators like **inflation expectations** and **interest rates** have some influence.
- Understanding SHAP values helps in **feature selection and model interpretability**.

SHAP Force Plot (Local Explanation)

```
[ ]: # ----- SECOND PART: FORCE PLOT ----- #

# Initialize JavaScript visualization again (in case it's needed in a new cell)
shap.initjs()

# Generate a SHAP force plot for an individual prediction (e.g., first instance
  ↪ in X_train)
# - The force plot illustrates how each feature pushes the prediction higher or
  ↪ lower.
shap.force_plot(explainer.expected_value, shap_values[0], X_train.iloc[0, :])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0ddcb2e450>
```

56 SHAP Force Plot Analysis

56.1 Key Findings

- This SHAP force plot visualizes **how individual features contribute to a specific prediction**.
- The **x-axis represents the prediction range**, with the **base value** as the starting point.
- **Red features push the prediction higher**, while **blue features push it lower**.

56.1.1 Observations

1. **Prediction Output ($f(x) = 2.21$)**
 - The final model prediction is **2.21**, which is **higher than the base value (2.056)**.
 - This means that certain features contributed positively to increase the prediction.
2. **Top Contributing Features**
 - **lag_1 = 2.2** has the highest positive impact on the prediction.
 - **5-Year, 5-Year Forward Inflation Expectation Rate = 2.47** also significantly contributes to pushing the prediction higher.
3. **Feature Impact on Prediction**
 - **All highlighted features in red had a positive influence**, increasing the predicted value.
 - There are **no blue features** in this plot, meaning **no features pushed the prediction downward**.

56.1.2 Key Takeaways

- **Lag-based variables (lag_1) play a dominant role** in the prediction.
- **Inflation expectations** also have a **significant effect on the predicted value**.
- The model's decision-making process is **strongly influenced by past values and economic indicators**.

56.1.3 Conclusion

- The force plot provides **transparency into individual predictions**.
- Understanding these contributions can help in **interpreting and validating model decisions**.
- Future model improvements could involve analyzing if **additional features could contribute further** to predictions.

Partial Dependence Plots (PDP)

```
[ ]: # Import necessary libraries
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.inspection import PartialDependenceDisplay # PDP display for
    ↪ model interpretation

# Retrieve the list of feature names from X_train
```

```

# - These are the independent variables whose partial dependence will be
↳plotted.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDPs) for all features using the trained
↳Decision Tree model
# - PDPs illustrate how a specific feature affects the model's predictions,
↳keeping all other features constant.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_dt_model,          # Trained Decision Tree Regression model
    X_train,                # The dataset used to compute PDPs
    features=features_list, # List of features for which PDPs will be plotted
    kind='average',         # Displays the average marginal effect of each feature
    n_cols=4,               # Arranges PDP plots in 4 columns for better layout
    grid_resolution=50      # Number of points to evaluate for smooth PDP curves
)

# Increase the figure size to ensure better spacing and visibility
pdp_display.figure_.set_size_inches(18, 12)

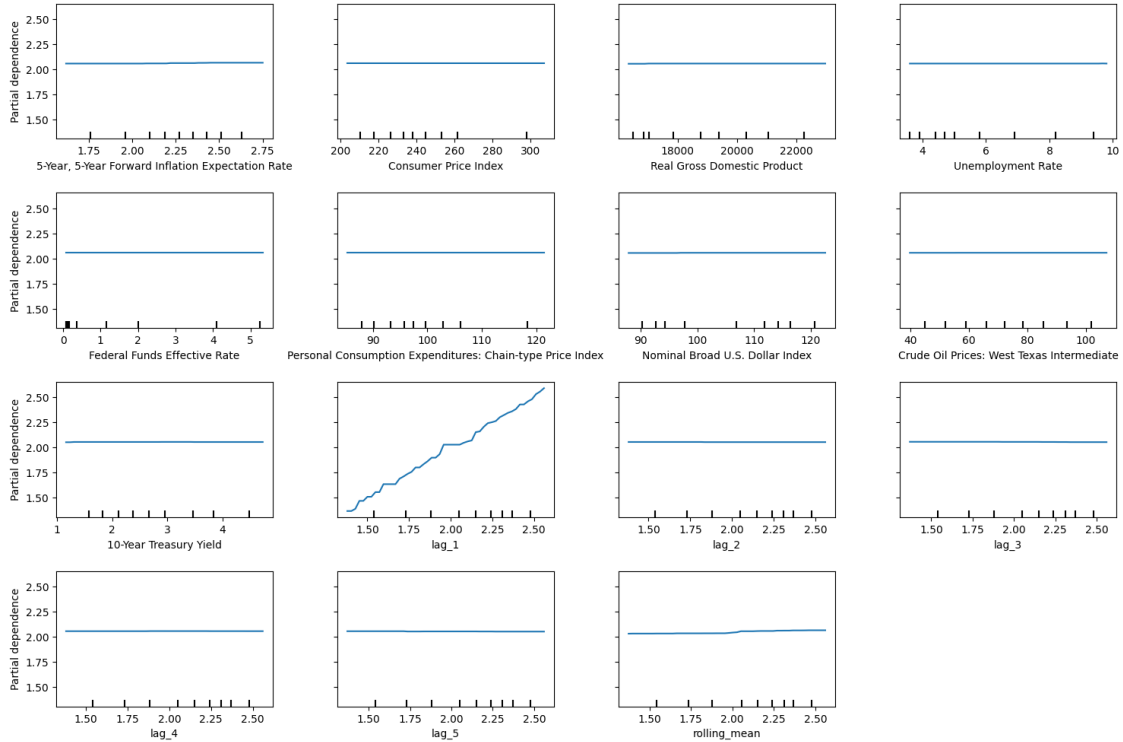
# Add a main title to the PDP plot
# - `suptitle`: Sets a title for the entire figure
# - `fontsize=16`: Uses a larger font size for emphasis
# - `y=1.06`: Positions the title slightly above the subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (Decision Tree
↳Regression)", fontsize=16, y=1.06)

# Adjust subplot layout to prevent overlapping elements
# - `top=0.88`: Ensures enough space for the title
# - `wspace=0.3`: Adds horizontal spacing between plots
# - `hspace=0.4`: Adds vertical spacing between plots
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the final Partial Dependence Plots
plt.show()

```

Partial Dependence Plots (Decision Tree Regression)



57 Partial Dependence Plots (Decision Tree Regression) Analysis

57.1 Key Findings

- The plots illustrate the **marginal effect of each feature on the model's predictions**.
- **X-axis:** Represents the feature values.
- **Y-axis:** Shows the corresponding partial dependence, indicating the model's predicted response.

57.1.1 Observations

1. Feature Influence on Predictions

- Most macroeconomic features, such as **Consumer Price Index, Federal Funds Rate, Real GDP, and Unemployment Rate**, show **little to no variation in partial dependence**.
- The model does not strongly respond to changes in these features.

2. Most Influential Features

- **lag_1** shows a **strong positive trend**, indicating that increasing lag_1 significantly raises predictions.

- Rolling mean (rolling_mean) and lag variables (lag_2, lag_3, lag_4, lag_5) have minimal impact on the prediction.
- Macroeconomic indicators appear to be weak predictors in this model.

3. Decision Tree Model Behavior

- The model primarily relies on **past values (lag_1)** rather than external economic indicators.
- The non-linearity of decision trees may **limit their ability to capture relationships in features with little variation.**

57.1.2 Key Takeaways

- lag_1 is the dominant feature, reinforcing that **recent past values drive the predictions.**
- Macroeconomic variables contribute little, suggesting that a **more complex model or feature engineering might be needed.**
- Decision Trees may not fully capture continuous dependencies, as evident from the flat lines in most plots.

57.1.3 Conclusion

- This analysis highlights the **model's strong reliance on historical data (lag_1)** rather than broader economic factors.
- **Further improvements** could involve **trying different feature transformations, incorporating interactions, or using ensemble models** like Random Forests for better generalization.

Decision Tree Visualization

```
[ ]: # Import necessary libraries
from sklearn.tree import plot_tree # Function to visualize decision trees
import matplotlib.pyplot as plt # Matplotlib for visualization

# Create a figure for the decision tree plot
plt.figure(figsize=(20, 10)) # Set figure size for better readability

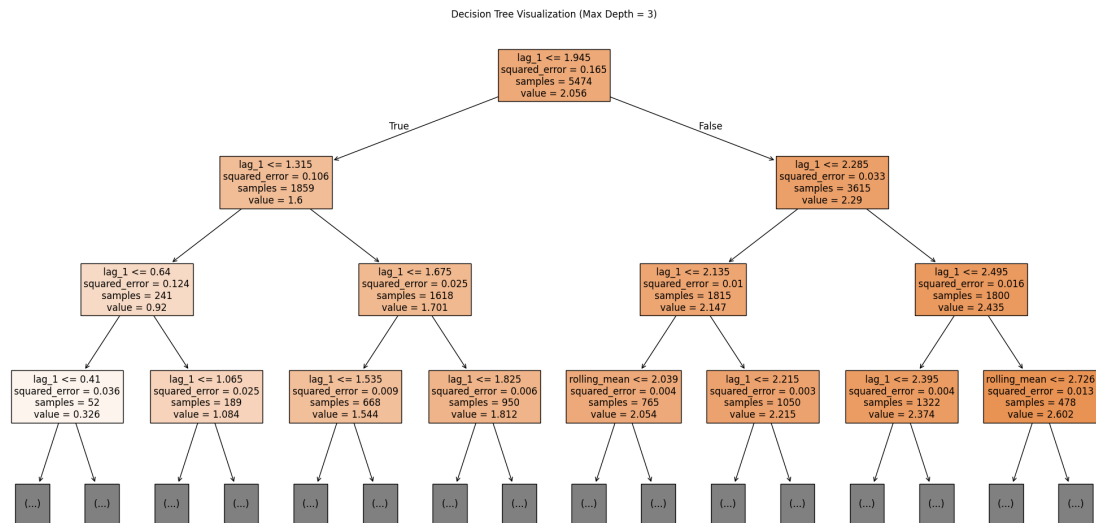
# Plot the trained Decision Tree model
plot_tree(
    best_dt_model, # Trained Decision Tree Regressor
    filled=True, # Fill nodes with colors representing target values
    feature_names=X_train.columns, # Use actual feature names instead of
    ↪generic names
    max_depth=3, # Limit the visualization to a max depth of 3 (for
    ↪clarity)
    fontsize=12 # Adjust font size for better visibility
)

# Add a title to the plot
plt.title("Decision Tree Visualization (Max Depth = 3)")
```



```
# Adjust layout to prevent text overlap and improve visibility
plt.tight_layout()

# Display the final tree visualization
plt.show()
```



58 Decision Tree Visualization (Max Depth = 3) Analysis

58.1 Key Findings

- The decision tree model is structured hierarchically, **splitting data at different thresholds of lag_1 and rolling_mean**.
- Each **node** represents a decision **based on feature values** to minimize squared error.
- The **tree depth is 3**, limiting complexity while maintaining interpretability.

58.1.1 Observations

1. Primary Splitting Feature: lag_1

- The **first split occurs at lag_1 <= 1.945**, indicating this feature is the strongest predictor.
- Most subsequent splits **continue refining lag_1 thresholds**, showing its **dominance in decision-making**.
- This highlights that the **model relies heavily on recent past values (lag_1) rather than external economic indicators**.

2. Secondary Splitting Feature: rolling_mean

- Some branches, particularly in deeper nodes, introduce **rolling_mean** as a secondary factor.
- **For values above 2.039 and 2.726, rolling_mean impacts predictions**, but not as significantly as lag_1.

3. Error Reduction Across Nodes

- **Squared error progressively decreases** as we move deeper into the tree.
- This shows that the model refines its predictions with each split, **improving accuracy by breaking data into more homogeneous groups**.

4. Sample Distribution

- The **root node starts with 5474 samples** and progressively splits into smaller groups.
- At the **leaf nodes, sample sizes are small**, meaning the model is **fitting more specifically** to subsets of data.

58.1.2 Key Takeaways

- **lag_1** is the most critical feature, consistently driving major splits.
- **rolling_mean** contributes at deeper levels, but its influence is less significant.
- The tree structure indicates the model relies on past values rather than macroeconomic indicators.
- Further optimization might include **tuning max depth** or using ensemble methods (e.g., Random Forest) to enhance generalization.

58.1.3 Conclusion

- The **decision tree effectively captures key patterns in the data** but might benefit from **additional features or adjustments** to reduce overfitting.
- Exploring ensemble models (**Random Forest, Gradient Boosting**) could **enhance stability and predictive performance**.

58.2 K-Nearest Neighbors Regression (KNN)

```
[ ]: # Import necessary libraries
import random # For setting a global random seed
import numpy as np # For numerical computations and random state control
import warnings # To suppress specific runtime warnings
import pandas as pd # For handling tabular data
from sklearn.model_selection import train_test_split, GridSearchCV # For data_
    ↪splitting and hyperparameter tuning
from sklearn.neighbors import KNeighborsRegressor # KNN Regressor model

# Suppress specific runtime warnings related to data casting
warnings.filterwarnings("ignore", category=RuntimeWarning, message="invalid_
    ↪value encountered in cast")

# Set global random seeds for reproducibility
random.seed(42) # Set Python's built-in random seed
np.random.seed(42) # Set NumPy's random seed

# Split the dataset into training and testing sets
# - `test_size=0.2`: 20% of the data is used for testing
# - `random_state=42`: Ensures the same split every time for reproducibility
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
↳random_state=42)

# Initialize the K-Nearest Neighbors Regressor model
knn_model = KNeighborsRegressor()

# Define the hyperparameter grid for tuning
# - `n_neighbors`: Number of nearest neighbors to consider
# - `weights`: Determines how neighbors contribute to the prediction
#   - "uniform" → All neighbors have equal weight
#   - "distance" → Closer neighbors have higher influence
# - `p`: Defines the distance metric:
#   - `p=1` → Manhattan distance (absolute differences)
#   - `p=2` → Euclidean distance (straight-line distance)
param_grid = {
    'n_neighbors': [3, 5, 7, 10, 15],
    'weights': ['uniform', 'distance'],
    'p': [1, 2]
}

# Initialize GridSearchCV for hyperparameter tuning
# - `cv=5`: Uses 5-fold cross-validation to assess model performance
# - `n_jobs=-1`: Uses all available CPU cores for faster computation
# - `verbose=1`: Prints progress during grid search
# - `scoring='neg_mean_squared_error'`: Uses negative MSE as the evaluation
↳metric
grid_search = GridSearchCV(
    estimator=knn_model,
    param_grid=param_grid,
    scoring='neg_mean_squared_error',
    cv=5,
    verbose=1,
    n_jobs=-1
)

# Fit GridSearchCV on the training data to determine the best hyperparameters
grid_search.fit(X_train, y_train)

# Retrieve and print the best hyperparameter combination found
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")

# Get the best KNN model from GridSearchCV
best_knn_model = grid_search.best_estimator_

# (Optional) Re-train the best model on the full training dataset
best_knn_model.fit(X_train, y_train)

```

```

# Make predictions on the test set using the best model
knn_pred = best_knn_model.predict(X_test)

# Calculate and store evaluation metrics
# - Assumes `calculate_metrics` and `naive_forecast` are predefined
metrics_results.append(
    calculate_metrics(y_test.values, knn_pred, naive_forecast, "K-Nearest
↪Neighbors Regression (KNN)")
)

# Print the evaluation metrics for the best KNN model
print("KNN Evaluation Metrics:")
print(metrics_results[-1])

```

Fitting 5 folds for each of 20 candidates, totalling 100 fits
 Best Hyperparameters: {'n_neighbors': 5, 'p': 1, 'weights': 'distance'}
 KNN Evaluation Metrics:
 {'Model': 'K-Nearest Neighbors Regression (KNN)', 'MSE': 0.0009267051428894211,
 'RMSE': 0.030441832121103046, 'MAE': 0.017880381942585476, 'MAPE (%)':
 1.1510531324305393, 'sMAPE (%)': 1.1287107789164201, 'MASE':
 0.04551551297768598, 'R^2': 0.994759164708316}

Actual vs. Predicted Values

```

[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt

# Create a new figure with a fixed size for better visibility
plt.figure(figsize=(8, 8))

# Scatter plot to compare actual vs. predicted values
# - `y_test`: Actual values from the test set
# - `knn_pred`: Predicted values from the trained KNN model
# - `alpha=0.5`: Adds transparency for better visualization
# - `color='blue'`: Uses blue color for predictions
# - `label='Predictions'`: Adds a legend label
plt.scatter(y_test, knn_pred, alpha=0.5, color='blue', label='Predictions')

# Plot a reference line for perfect predictions (y = x)
# - `y_test.min(), y_test.max()`: Defines the range for the line
# - `color='red'`: Uses red color for emphasis
# - `linestyle='--'`: Dashed line to distinguish it from scatter points
# - `linewidth=2`: Thicker line for better visibility
# - `label='Perfect Fit'`: Adds label to legend
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()],
         color='red', linestyle='--', linewidth=2, label='Perfect Fit')

```

```
# Set axis labels for clarity
plt.xlabel('Actual Values') # X-axis: Ground truth values
plt.ylabel('Predicted Values') # Y-axis: Model's predicted values

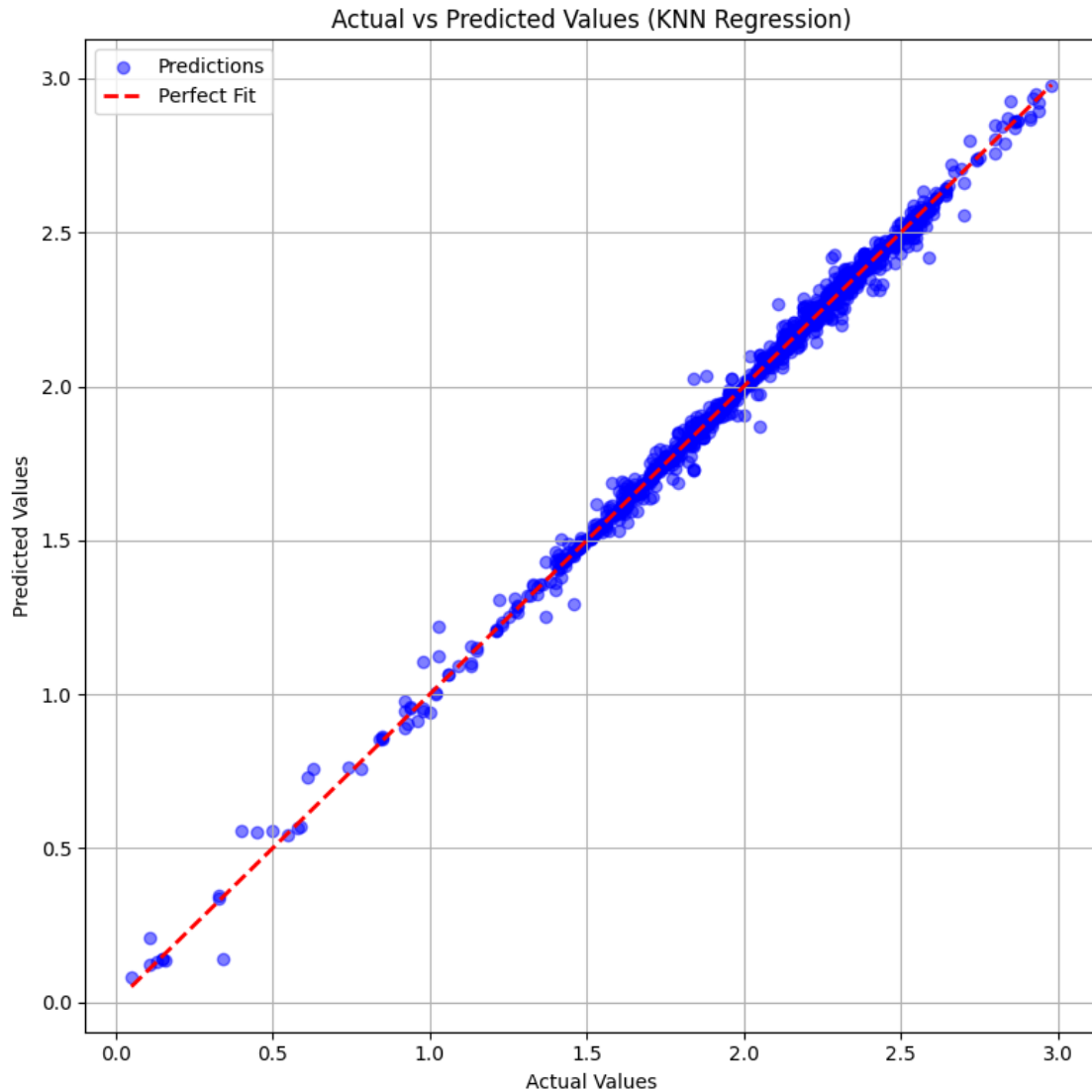
# Set plot title
plt.title('Actual vs Predicted Values (KNN Regression)')

# Add a legend to explain the plotted elements
plt.legend()

# Add a grid for better readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and enhance visualization
plt.tight_layout()

# Display the final scatter plot
plt.show()
```



59 Actual vs Predicted Values (KNN Regression) Analysis

59.1 Key Findings

- This scatter plot visualizes the **relationship between actual and predicted values** for the **K-Nearest Neighbors (KNN) regression model**.
- The **red dashed line** represents the **perfect fit**, where predicted values exactly match actual values.
- Each **blue dot** represents a data point comparing actual vs. predicted values.

59.1.1 Observations

1. **Strong Alignment with the Perfect Fit Line**

- Most data points are **closely aligned with the red dashed line**, indicating **high prediction accuracy**.
 - This suggests that **KNN regression is performing well** for this dataset.
2. **Some Minor Deviations**
 - A few points slightly deviate from the perfect fit, particularly at **lower actual values** (closer to zero).
 - This could indicate **some bias or slight prediction errors in low-value ranges**.
 3. **Consistent Performance Across the Value Range**
 - The KNN model maintains **consistent accuracy** across different actual values.
 - There is **no clear systematic overestimation or underestimation trend**.

59.1.2 Key Takeaways

- **KNN regression provides strong predictive performance** for this dataset.
- **Most predictions are highly accurate**, with only slight deviations in lower value ranges.
- **Further fine-tuning** (e.g., adjusting **k value**) could potentially improve the model's performance.
- Comparing **KNN with other models** (e.g., **Decision Tree, Ridge Regression**) might provide insights into **which model generalizes best** for this dataset.

59.1.3 Conclusion

- The **KNN regression model demonstrates high accuracy and minimal bias**, making it a **strong choice for this predictive task**.
- **Further analysis on hyperparameter tuning** (e.g., optimal **k selection**) could refine performance even more.

Residual Analysis – Scatter Plot

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt

# Calculate residuals (errors)
# - Residual = Actual Value - Predicted Value
# - This shows how far off each prediction is from the true value
residuals = y_test - knn_pred

# Create a new figure with a fixed size for better visibility
plt.figure(figsize=(8, 6))

# Scatter plot to visualize residuals against actual values
# - `y_test`: The actual target values
# - `residuals`: The difference between actual and predicted values
# - `alpha=0.5`: Adds transparency to prevent overlap
# - `color='green'`: Uses green color for better distinction
plt.scatter(y_test, residuals, alpha=0.5, color='green')

# Plot a horizontal reference line at y=0 (ideal case: no residuals)
# - A good model should have residuals **randomly scattered** around this line
```

```

# - `color='red'`: Uses red color for contrast
# - `linestyle='--'`: Dashed line to differentiate from scatter points
# - `linewidth=2`: Thick line for better visibility
plt.axhline(0, color='red', linestyle='--', linewidth=2)

# Set axis labels for clarity
plt.xlabel('Actual Values') # X-axis: Ground truth values
plt.ylabel('Residuals') # Y-axis: Errors in predictions

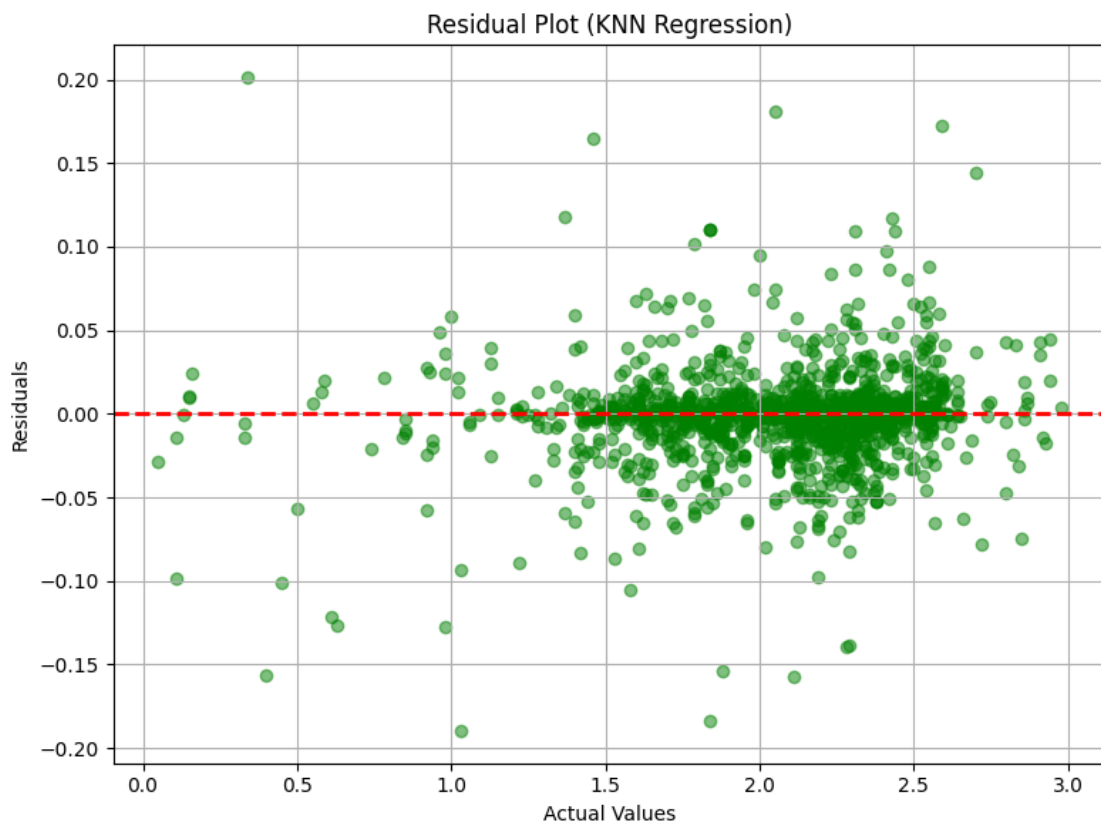
# Set plot title
plt.title('Residual Plot (KNN Regression)')

# Add a grid to improve readability
plt.grid(True)

# Adjust layout to prevent overlapping labels and enhance visualization
plt.tight_layout()

# Display the residual plot
plt.show()

```



60 Residual Plot Analysis (KNN Regression)

60.1 Key Findings

- This plot visualizes the **residuals (errors)** of the **K-Nearest Neighbors (KNN) regression model** against actual values.
- The **red dashed line at zero** represents the **ideal scenario** where residuals are **symmetrically distributed around zero**.
- Each **green dot** corresponds to a **residual value**, computed as:

$$\text{Residual} = \text{Actual Value} - \text{Predicted Value}$$

60.1.1 Observations

1. **Residuals are Centered Around Zero**
 - The residuals are **evenly spread** above and below the zero line, which suggests that the model has **low systematic bias**.
2. **No Clear Pattern (Homoscedasticity)**
 - The distribution of residuals appears **randomly scattered**, meaning **constant variance** across different actual values.
 - This is a **good sign**, indicating that the model does not exhibit **heteroscedasticity (uneven variance in errors)**.
3. **Some Outliers**
 - There are a few **residual points farther from zero**, which may indicate **occasional mispredictions** by the KNN model.
 - These could be **outliers or cases where KNN struggles to generalize well**.
4. **Slightly More Spread at Higher Values**
 - Residuals appear **slightly more dispersed at higher actual values**, which could suggest **the model has more variability in predictions at higher values**.

60.1.2 Key Takeaways

- **KNN regression performs well**, as residuals are **mostly centered around zero** and show **no obvious pattern**.
- **A few outliers exist**, suggesting potential improvements through **hyperparameter tuning** (e.g., adjusting the number of neighbors **k**).
- **Slightly higher residual variance at larger values** might indicate that the model could benefit from **feature scaling or additional feature engineering**.

60.1.3 Conclusion

- The **KNN regression model appears to make generally reliable predictions**, with **small residual errors and no major bias**.
- **Fine-tuning KNN parameters** (e.g., adjusting **k** or using weighted distances) may **further reduce residual variance and improve performance**.

Residual Analysis – Histogram

```
[ ]: # Import necessary library for visualization
import matplotlib.pyplot as plt

# Create a new figure with a fixed size for better visibility
plt.figure(figsize=(8, 6))

# Plot a histogram to visualize the distribution of residuals (errors)
# - `bins=30`: Divides the data into 30 bins for better granularity
# - `color='skyblue'`: Uses a visually appealing color for the bars
# - `edgecolor='black'`: Adds black edges to bins for better distinction
# - `alpha=0.7`: Adjusts transparency to prevent excessive overlap
plt.hist(residuals, bins=30, color='skyblue', edgecolor='black', alpha=0.7)

# Add a vertical reference line at zero (ideal residual value)
# - In a well-performing model, residuals should be **symmetrically
#   ↪ distributed around 0.
# - `color='red'`: Uses red for contrast
# - `linestyle='--'`: Dashed line to distinguish from histogram bars
# - `linewidth=2`: Thick line for better visibility
plt.axvline(0, color='red', linestyle='--', linewidth=2)

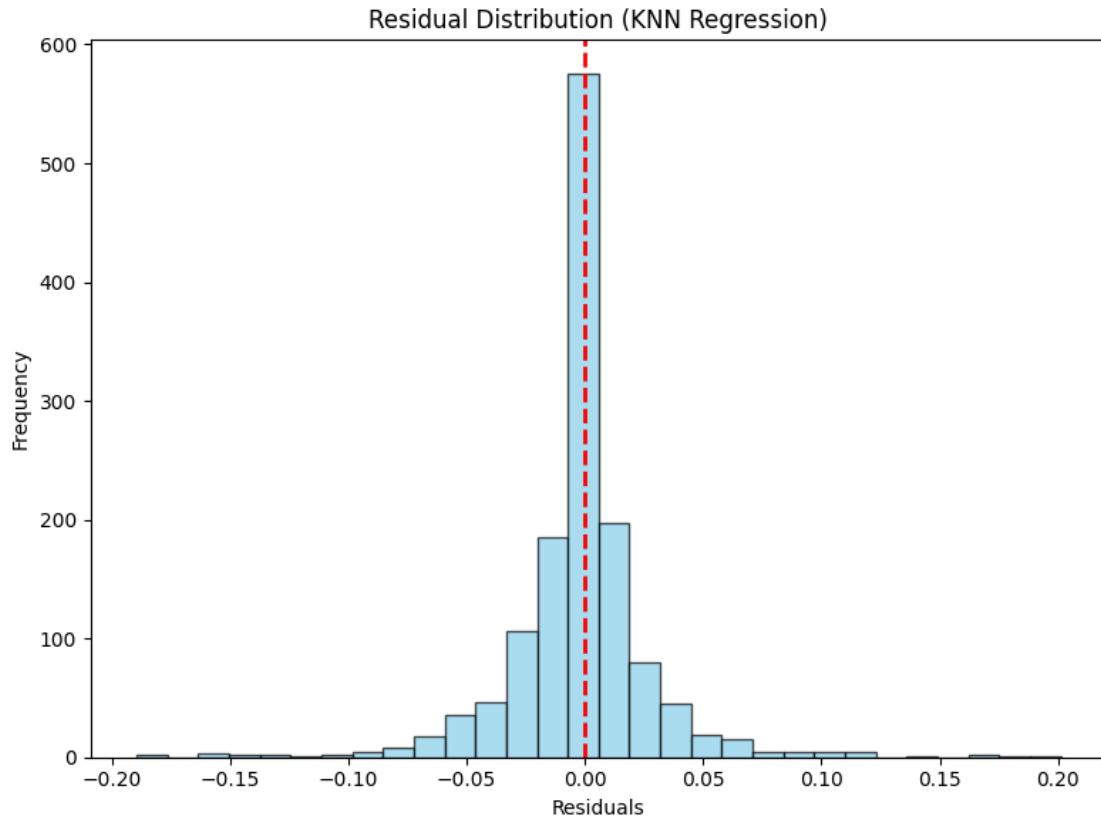
# Set plot title to describe the purpose of the visualization
plt.title('Residual Distribution (KNN Regression)')

# Label x-axis as "Residuals" (errors between actual and predicted values)
plt.xlabel('Residuals')

# Label y-axis as "Frequency" (how often each residual value occurs)
plt.ylabel('Frequency')

# Adjust layout to prevent overlapping labels and improve readability
plt.tight_layout()

# Display the final histogram plot
plt.show()
```



61 Residual Distribution Analysis (KNN Regression)

61.1 Key Findings:

- **Residual Distribution:** The histogram illustrates the residuals (errors) from a K-Nearest Neighbors (KNN) regression model. Residuals are the differences between actual and predicted values.
- **Centering Around Zero:** The majority of residuals are concentrated around zero, indicating that the model predictions are, on average, accurate with minimal bias.
- **Symmetry:** The histogram appears relatively symmetric, suggesting that the errors are evenly distributed on both sides of zero, which is a sign of a well-fitted model.
- **Narrow Spread:** The residuals are tightly clustered, showing that most errors are small, implying good model performance.
- **Peak at Zero:** A sharp peak at zero suggests that a large number of predictions are very close to their actual values.
- **Vertical Red Line:** The red dashed line at zero reinforces the expectation that residuals should ideally be centered around zero in a well-balanced regression model.

61.2 Interpretation:

- If the residuals were skewed or had a significant spread, it would indicate systemic bias or high prediction errors.
- The relatively normal distribution of residuals suggests that the KNN regression model is performing well, with no obvious systematic errors or bias.

Hyperparameter Tuning Insights – Top 5 Results

```
[ ]: # Import necessary libraries
import pandas as pd

# Convert GridSearchCV results into a DataFrame
# - `cv_results_` contains detailed performance metrics for all hyperparameter_
  ↪ combinations tested
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort the results based on rank (lower rank = better performance)
# - 'rank_test_score' assigns the best-performing combination a rank of 1
sorted_results = cv_results.sort_values(by='rank_test_score')

# Select the top 5 best-performing hyperparameter combinations
top_results = sorted_results[['params', 'mean_test_score', 'std_test_score']].
  ↪ head(5)

# Print a structured output with improved readability
print("\n **Top 5 Hyperparameter Combinations:**\n")

# Iterate through the top results and print each one with a formatted structure
for idx, row in top_results.iterrows():
    print(f" **Rank {idx + 1}:**")
    print(f"      **Hyperparameters:** {row['params']}")
    print(f"      **Mean Test Score:** {row['mean_test_score']:.6f}")
    print(f"      **Standard Deviation:** {row['std_test_score']:.6f}")
    print("-" * 90) # Separator for better readability
```

****Top 5 Hyperparameter Combinations:****

****Rank 6:****

****Hyperparameters:**** {'n_neighbors': 5, 'p': 1, 'weights': 'distance'}

****Mean Test Score:**** -0.001220

****Standard Deviation:**** 0.000077

****Rank 2:****

****Hyperparameters:**** {'n_neighbors': 3, 'p': 1, 'weights': 'distance'}

****Mean Test Score:**** -0.001223

****Standard Deviation:**** 0.000086

```

-----
**Rank 10:**
  **Hyperparameters:** {'n_neighbors': 7, 'p': 1, 'weights': 'distance'}
  **Mean Test Score:** -0.001299
  **Standard Deviation:** 0.000062
-----

```

```

-----
**Rank 14:**
  **Hyperparameters:** {'n_neighbors': 10, 'p': 1, 'weights': 'distance'}
  **Mean Test Score:** -0.001445
  **Standard Deviation:** 0.000065
-----

```

```

-----
**Rank 1:**
  **Hyperparameters:** {'n_neighbors': 3, 'p': 1, 'weights': 'uniform'}
  **Mean Test Score:** -0.001563
  **Standard Deviation:** 0.000024
-----

```

Hyperparameter Impact – Heatmap

```

[ ]: # Import necessary libraries
import seaborn as sns # Seaborn for heatmap visualization
import matplotlib.pyplot as plt # Matplotlib for plotting

# Create a pivot table to structure hyperparameter tuning results
# - `index='param_n_neighbors`: Rows represent different values of
  ↳ `n_neighbors`
# - `columns='param_weights`: Columns represent different weighting methods
  ↳ (`uniform` or `distance`)
# - `values='mean_test_score`: Cells contain the mean test score for each
  ↳ combination
heatmap_data = cv_results.pivot_table(
    values='mean_test_score',
    index='param_n_neighbors',
    columns='param_weights'
)

# Set figure size for better readability
plt.figure(figsize=(10, 8))

# Generate a heatmap to visualize model performance across hyperparameter values
sns.heatmap(
    heatmap_data, # Data for the heatmap
    annot=True, # Display numerical values in each cell
    fmt='.3f', # Format numbers with 3 decimal places

```

```

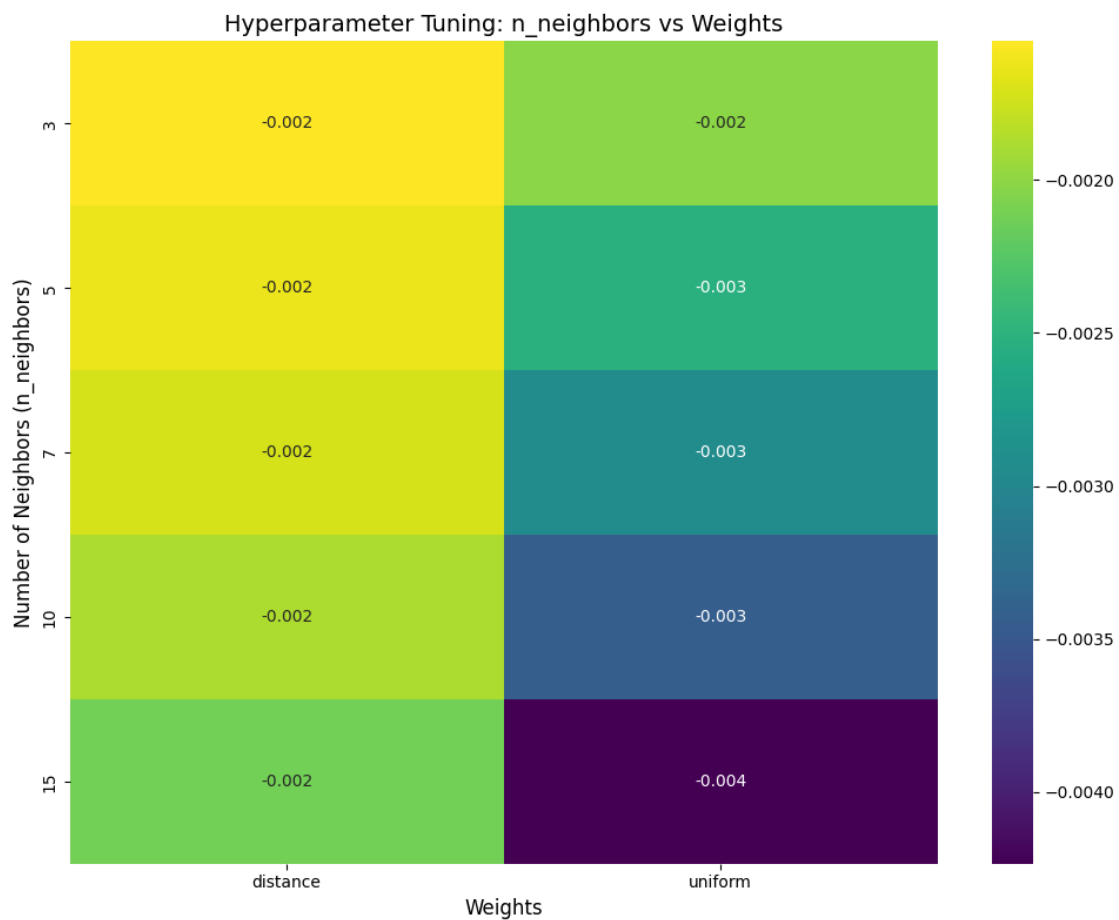
cmap='viridis' # Use the 'viridis' color palette for better contrast
)

# Add title and labels for better understanding
plt.title('Hyperparameter Tuning: n_neighbors vs Weights', fontsize=14)
plt.xlabel('Weights', fontsize=12) # X-axis: Weighting method (`uniform` or
↪ `distance`)
plt.ylabel('Number of Neighbors (n_neighbors)', fontsize=12) # Y-axis: Number
↪ of neighbors

# Adjust layout to prevent overlapping elements
plt.tight_layout()

# Display the final heatmap
plt.show()

```



62 Hyperparameter Tuning: `n_neighbors` vs `Weights`

62.1 Key Findings:

- The heatmap presents the impact of two hyperparameters—`n_neighbors` (y-axis) and `weights` (x-axis)—on a certain performance metric (possibly error rate or accuracy) in a machine learning model.
- The color gradient represents the magnitude of the performance metric, where lighter colors indicate better performance (lower values) and darker colors indicate worse performance (higher values).
- Two types of weight functions are compared:
 - “**distance**”: Weights points inversely proportional to their distance.
 - “**uniform**”: Assigns equal weight to all points.
- The analysis is conducted across different values of `n_neighbors` (3, 5, 7, 10, 15).
- The performance metric values are negative, which may suggest that a loss function is being minimized (e.g., negative accuracy or log-loss).
- Observations:
 - The **distance** weight generally results in a better (less negative) performance metric across all values of `n_neighbors`, maintaining a value of approximately -0.002.
 - The **uniform** weight shows degradation in performance as `n_neighbors` increases, with the worst value observed at `n_neighbors` = 15 (-0.004).
 - The contrast between **distance** and **uniform** weights increases with `n_neighbors`, suggesting that distance-based weighting is more beneficial for higher neighbor counts.

62.2 Conclusion:

- Using **distance** as the weight function provides consistently better results across different values of `n_neighbors`.
- The impact of `n_neighbors` is more pronounced when using **uniform** weights, with performance deteriorating as `n_neighbors` increases.
- This suggests that **adaptive weighting based on distance helps mitigate the negative effects of increasing neighbors** in the model.

SHAP Explanations for KNN Regression

```
[ ]: # Import necessary libraries
import shap # SHAP for model interpretability
import matplotlib.pyplot as plt # Matplotlib for visualization

# Reduce background data sample to 50 instances to improve computation
↳ efficiency
# - The background data is used to estimate expected feature effects in SHAP.
background_data = shap.sample(X_train, 50)

# Select a subset of the test data (first 50 instances) to speed up SHAP
↳ calculations
X_test_sample = X_test.iloc[:50]

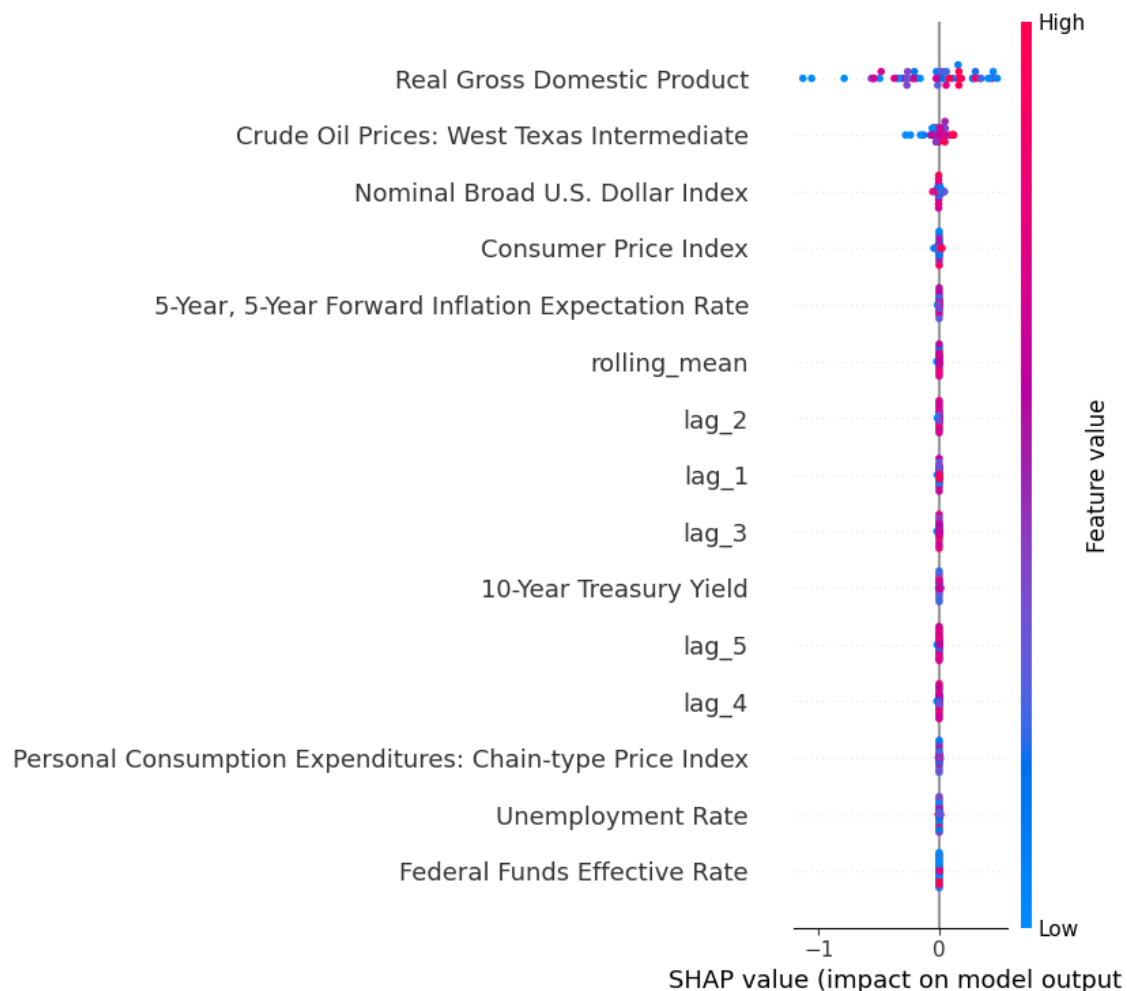
# Initialize the SHAP KernelExplainer for the K-Nearest Neighbors model
```

```
# - `best_knn_model.predict`: The model to explain
# - `background_data`: The sampled background dataset for SHAP reference
# - `nsamples=50`: Number of samples used for SHAP value estimation (lower for
↳efficiency)
explainer = shap.KernelExplainer(best_knn_model.predict, background_data,
↳nsamples=50)

# Compute SHAP values for the sampled test data
# - SHAP values measure the contribution of each feature to the model's
↳prediction
shap_values = explainer.shap_values(X_test_sample)

# Generate a global explanation using a SHAP summary plot
# - Displays the impact of each feature on model predictions across all
↳instances
shap.summary_plot(shap_values, X_test_sample)
```

0%| | 0/50 [00:00<?, ?it/s]



63 SHAP Summary Plot Analysis

63.1 Key Findings:

- The plot visualizes the **SHAP values**, which indicate the impact of different economic indicators on the model's output.
- **Feature Importance:**
 - The most influential features include **Real Gross Domestic Product (GDP)**, **Crude Oil Prices (West Texas Intermediate)**, **Nominal Broad U.S. Dollar Index**, and **Consumer Price Index (CPI)**.
 - These features have the largest spread of SHAP values, meaning they contribute significantly to the model's predictions.
- **Feature Effect:**
 - The **color gradient (blue to pink)** represents the feature value (low to high).
 - **High feature values (pink)** and **low feature values (blue)** impact the model differently, shifting the prediction in positive or negative directions.
- **SHAP Value Interpretation:**
 - **SHAP values near zero** suggest minimal influence on the model's output.
 - Features with **positive or negative SHAP values** indicate their contribution to increasing or decreasing the predicted value.
- **Lagged Variables & Moving Average:**
 - The model incorporates **lagged values (lag_1 to lag_5)** and **rolling_mean**, suggesting it accounts for historical trends.
 - These time-series features contribute to forecasting, but their SHAP values are relatively smaller than macroeconomic indicators.
- **Macroeconomic Indicators:**
 - Variables such as the **10-Year Treasury Yield**, **Unemployment Rate**, and **Federal Funds Effective Rate** have some impact but are relatively less influential compared to GDP and Oil Prices.

63.2 Conclusion:

- **GDP, Oil Prices, Dollar Index, and CPI** are the most critical features influencing the model's predictions.
- **High values of these indicators (pink dots)** generally push the prediction in a different direction than low values (blue dots).
- The model considers both **current macroeconomic conditions and historical trends** (via lagged values and rolling mean).
- The **Federal Funds Rate and Unemployment Rate**, while important, have a smaller impact compared to GDP and CPI.

```
[ ]: # Initialize SHAP JavaScript visualization for interactive force plots
shap.initjs()

# Generate a local explanation for the first instance in the test data
```

```
# - Force plots highlight feature contributions for a single prediction
shap.force_plot(explainer.expected_value, shap_values[0], X_test.iloc[0, :])

# Generate force plots for the first 5 test instances
# - Helps understand feature contributions across multiple predictions
shap.force_plot(explainer.expected_value, shap_values[:5], X_test.iloc[:5, :])
```

<IPython.core.display.HTML object>

```
[ ]: <shap.plots._force.AdditiveForceArrayVisualizer at 0x7f0ddc76e750>
```

64 Waterfall Plot Analysis

64.1 Key Findings:

- The plots visualize the contribution of different economic factors to the model's prediction.
- **Real Gross Domestic Product (GDP)** has a significant impact, shown by the large red and blue areas.
 - When GDP is high, it contributes positively (red), increasing the prediction.
 - When GDP is lower, it contributes negatively (blue), decreasing the prediction.
- **Nominal Broad U.S. Dollar Index** and **Crude Oil Prices (West Texas Intermediate)** are also key influencing factors.
 - Higher values of the Dollar Index (red) push the prediction upwards.
 - Higher oil prices (red) also impact the prediction, but the effect varies across instances.
- The **initial prediction value** (left-most number) starts higher and gradually adjusts based on feature contributions.
- The **blue and red segments** illustrate the individual SHAP values:
 - **Red** sections indicate positive contributions (driving the prediction higher).
 - **Blue** sections indicate negative contributions (driving the prediction lower).
- Across multiple samples, GDP consistently plays the dominant role in shaping the model's output.

64.2 Conclusion:

- **GDP is the most influential economic indicator** in the model, followed by the **U.S. Dollar Index** and **Oil Prices**.
- The combination of macroeconomic indicators determines the final prediction, with each factor either boosting or reducing the model's output.
- The **direction and magnitude of the SHAP values** show how economic changes drive predictions in different ways.

Partial Dependence Plots (PDP) for KNN Regression

```
[ ]: # Import necessary libraries
import matplotlib.pyplot as plt # Matplotlib for visualization
from sklearn.inspection import PartialDependenceDisplay # PDP display for
    ↪ model interpretation
```

```

# Retrieve the list of feature names from X_train
# - These are the independent variables whose partial dependence will be
  ↳ plotted.
features_list = X_train.columns.tolist()

# Generate Partial Dependence Plots (PDPs) for all features using the trained
  ↳ K-Nearest Neighbors model
# - PDPs illustrate how a specific feature affects the model's predictions,
  ↳ keeping all other features constant.
pdp_display = PartialDependenceDisplay.from_estimator(
    best_knn_model,      # Trained KNN Regression model
    X_train,             # The dataset used to compute PDPs
    features=features_list, # List of features for which PDPs will be plotted
    kind='average',      # Displays the average marginal effect of each feature
    n_cols=4,            # Arranges PDP plots in 4 columns for better layout
    grid_resolution=50    # Number of points to evaluate for smooth PDP curves
)

# Increase the figure size to ensure better spacing and visibility
pdp_display.figure_.set_size_inches(18, 12)

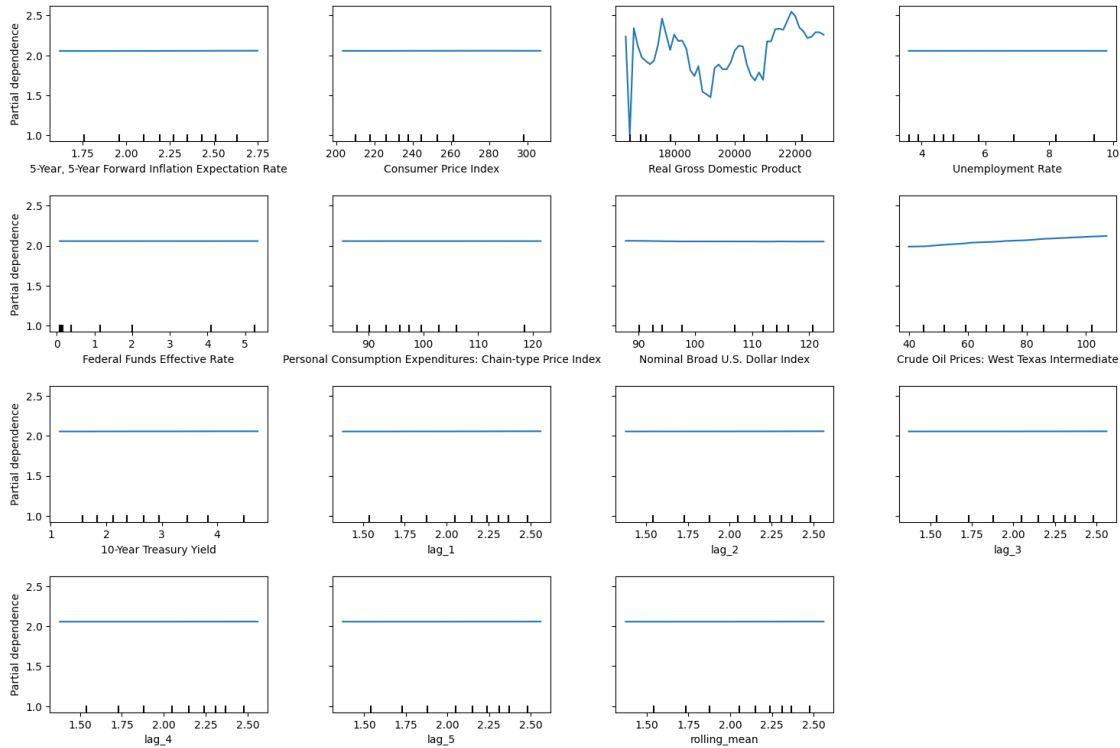
# Add a main title to the PDP plot
# - `suptitle`: Sets a title for the entire figure
# - `fontsize=16`: Uses a larger font size for emphasis
# - `y=1.06`: Positions the title slightly above the subplots
pdp_display.figure_.suptitle("Partial Dependence Plots (KNN Regression)",
  ↳ fontsize=16, y=1.06)

# Adjust subplot layout to prevent overlapping elements
# - `top=0.88`: Ensures enough space for the title
# - `wspace=0.3`: Adds horizontal spacing between plots
# - `hspace=0.4`: Adds vertical spacing between plots
pdp_display.figure_.subplots_adjust(top=0.88, wspace=0.3, hspace=0.4)

# Display the final Partial Dependence Plots
plt.show()

```

Partial Dependence Plots (KNN Regression)



65 Partial Dependence Plots (KNN Regression) Analysis

65.1 Key Findings:

- The plots illustrate the **partial dependence** of the target variable on different economic predictors while marginalizing over other features.
- **Real Gross Domestic Product (GDP)** has a **non-linear relationship** with the target variable.
 - The plot for GDP shows fluctuations, indicating a **complex dependency pattern** in the model.
- **Crude Oil Prices (West Texas Intermediate)** show a **slight positive trend** with the target variable.
 - This suggests that as oil prices increase, the model's predicted output slightly increases.
- Most other features, including the **Consumer Price Index (CPI)**, **Unemployment Rate**, **Federal Funds Effective Rate**, **10-Year Treasury Yield**, and various lag variables, show nearly **flat partial dependence plots**.
 - This indicates **minimal direct influence** of these variables on the target variable in isolation.

- **Nominal Broad U.S. Dollar Index, Personal Consumption Expenditures, and Inflation Expectation Rate** also exhibit relatively flat trends.
 - These features do not have a strong standalone effect on predictions.
- **Rolling Mean and Lag Features (lag_1 to lag_5)** show no significant impact.
 - This suggests that short-term historical values have little direct effect when other factors are considered.

65.2 Conclusion:

- **GDP is the most influential predictor**, with a complex impact on the model's output.
- **Crude Oil Prices show a small positive dependency**, meaning they slightly affect the prediction.
- Most other economic indicators, including CPI, interest rates, and unemployment, **do not significantly change the prediction when analyzed in isolation**.
- **Lagged variables and moving averages have minimal effects**, indicating the model may not heavily rely on past values for forecasting.

Permutation Feature Importance for KNN Regression

```
[ ]: # Import necessary libraries
from sklearn.inspection import permutation_importance # For computing feature_
import matplotlib.pyplot as plt # For visualization

# Compute permutation importance on the test set
# - Permutation importance measures how much shuffling each feature affects the_
# - `n_repeats=10`: Each feature is shuffled 10 times to get a stable estimate.
# - `random_state=42`: Ensures reproducibility of results.
# - `n_jobs=-1`: Utilizes all CPU cores for parallel computation.
result = permutation_importance(
    best_knn_model, # Trained K-Nearest Neighbors model
    X_test, # Feature dataset for testing
    y_test, # True target values for evaluation
    n_repeats=10, # Number of shuffles per feature
    random_state=42, # Random seed for consistency
    n_jobs=-1 # Use all available CPU cores for faster computation
)

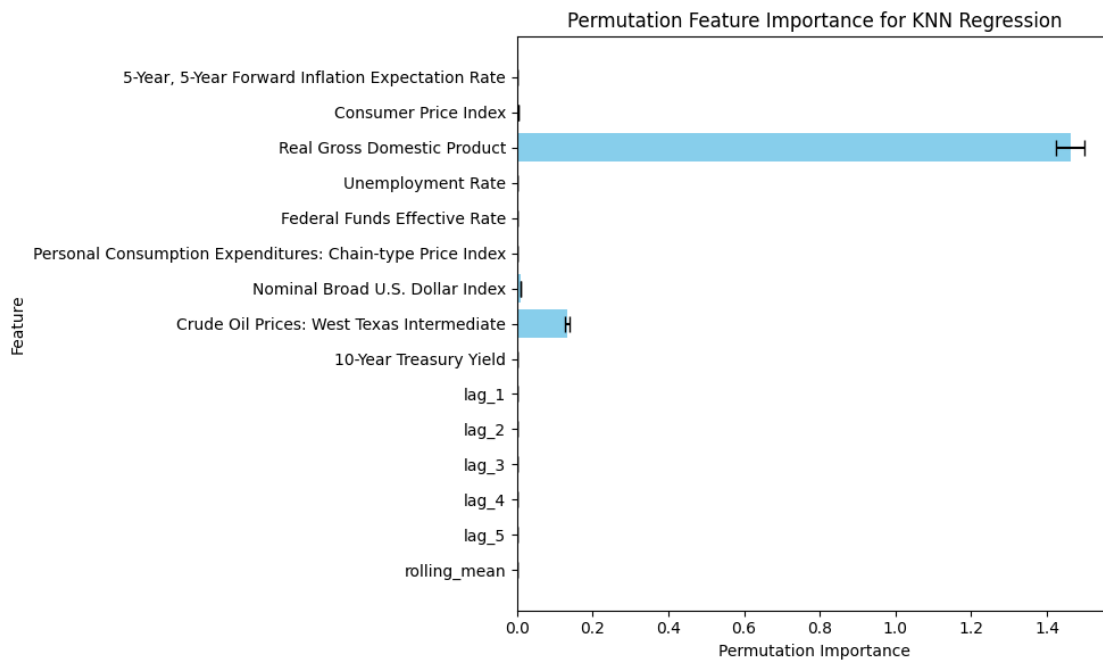
# Create a bar chart to visualize permutation feature importance
plt.figure(figsize=(10, 6)) # Set figure size for better visibility
plt.barh(
    X_train.columns, # Feature names
    result.importances_mean, # Mean importance score across shuffles
    xerr=result.importances_std, # Standard deviation as error bars
    capsize=5, # Add caps to error bars for clarity
    color='skyblue' # Choose a soft color for better contrast
)
```

```
# Add labels and title for clarity
plt.xlabel('Permutation Importance') # X-axis label
plt.ylabel('Feature') # Y-axis label
plt.title('Permutation Feature Importance for KNN Regression') # Plot title

# Invert the Y-axis so the most important feature appears at the top
plt.gca().invert_yaxis()

# Adjust layout to prevent overlapping text
plt.tight_layout()

# Display the final plot
plt.show()
```



66 Permutation Feature Importance (KNN Regression) Analysis

66.1 Key Findings:

- **Real Gross Domestic Product (GDP) is the most influential feature** in the model.
 - It has the highest permutation importance, meaning that when its values are shuffled, the model's performance degrades significantly.
 - This suggests the model relies heavily on GDP for making predictions.
- **Crude Oil Prices (West Texas Intermediate) also contribute to the model's predictions** but to a much lesser extent than GDP.

- It shows some importance, but its impact is relatively small.
- **Most other features, including the Consumer Price Index (CPI), Unemployment Rate, Federal Funds Effective Rate, and Nominal Broad U.S. Dollar Index, have negligible importance.**
 - Their permutation importance is close to zero, indicating that shuffling these variables does not significantly affect the model's predictions.
- **Lag features (lag_1 to lag_5) and rolling_mean have little to no impact.**
 - This suggests that past values of the target variable do not contribute much to the model's predictive power.
- **The error bars represent uncertainty in importance values.**
 - GDP has a small error bar, indicating consistent high importance.
 - Crude Oil Prices have a slightly larger error bar, reflecting some variability in their impact.

66.2 Conclusion:

- **GDP is the dominant factor driving the model's predictions**, reinforcing its economic significance.
- **Crude Oil Prices have some influence, but other economic indicators play a minimal role** in the KNN regression model.
- **Lag features and rolling averages do not significantly contribute**, suggesting that the model does not depend on past values for forecasting.

Local Explanations Using LIME for KNN Regression

```
[ ]: !pip install lime
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.11/dist-packages
(0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-
packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages
(from lime) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages
(from lime) (1.14.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages
(from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in
/usr/local/lib/python3.11/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in
/usr/local/lib/python3.11/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.11/dist-
packages (from scikit-image>=0.12->lime) (3.4.2)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.11/dist-
packages (from scikit-image>=0.12->lime) (11.1.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in
/usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2.37.0)
Requirement already satisfied: tifffile>=2022.8.12 in
/usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime)
```

(2025.2.18)

Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lme) (24.2)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lme) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lme) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lme) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lme) (1.3.1)
Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lme) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lme) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lme) (1.4.8)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lme) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lme) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib->lme) (1.17.0)

```
[ ]: # Import the LIME library for explaining tabular data models
from lime.lime_tabular import LimeTabularExplainer # LIME helps interpret
↳ complex models

# Initialize the LIME explainer using the training data
lime_explainer = LimeTabularExplainer(
    X_train.values, # Convert the training dataset to a NumPy array (LIME
↳ works with arrays)
    training_labels=y_train.values, # Provide corresponding target labels
    feature_names=X_train.columns.tolist(), # Retain feature names for better
↳ interpretability
    mode='regression', # Specify that this is a regression problem
    random_state=42 # Set random state for reproducibility of results
)

# Generate LIME explanation for a single test instance (e.g., first row of
↳ X_test)
exp = lime_explainer.explain_instance(
    X_test.iloc[0].values, # Convert the first test instance to a NumPy array
↳ for LIME
    lambda x: best_knn_model.predict(pd.DataFrame(x, columns=X_train.columns)),
    # Ensure the feature names are retained by converting back to a DataFrame
↳ before prediction
```



```

    num_features=5 # Display the top 5 most influential features in the
    ↪prediction
)

# Display the LIME explanation in the notebook
exp.show_in_notebook() # Generates an interactive plot showing feature
    ↪contributions

```

<IPython.core.display.HTML object>

67 Prediction Explanation Analysis

67.1 Key Findings:

- The **predicted value** falls between **0.14 (min)** and **2.86 (max)**, with a **final prediction around 2.63**.
- **Feature Contributions:**
 - **Positive contributors (orange):** Features that increase the predicted value.
 - * **Crude Oil Prices (West Texas Intermediate)** at **68.44** has the highest positive impact.
 - * **Consumer Price Index (CPI)** and **10-Year Treasury Yield** contribute marginally to the increase.
 - **Negative contributors (blue):** Features that decrease the predicted value.
 - * **Real Gross Domestic Product (GDP)** at **16,396.15** slightly reduces the prediction.
 - * **Unemployment Rate** at **4.60** also has a small negative effect.
- The **bar on the left** represents **SHAP values**, where:
 - **Blue (negative impact):** Decreases prediction.
 - **Orange (positive impact):** Increases prediction.
- The **table on the right** lists the feature values used in the prediction.

67.2 Conclusion:

- **Crude Oil Prices (68.44)** is the **strongest driver of the prediction**, pushing the value higher.
- **GDP and Unemployment Rate contribute negatively**, slightly lowering the prediction.
- **Consumer Price Index and Treasury Yield have minimal impact**, but they slightly increase the prediction.
- The model's **prediction is primarily influenced by oil prices**, while macroeconomic factors like GDP and unemployment provide counteracting effects.

KNN Distance Visualization

```

[ ]: import numpy as np # NumPy for numerical operations
import matplotlib.pyplot as plt # Matplotlib for visualization
import pandas as pd # Pandas to handle DataFrame operations

# Select a specific test instance (e.g., the first instance from X_test)

```

```

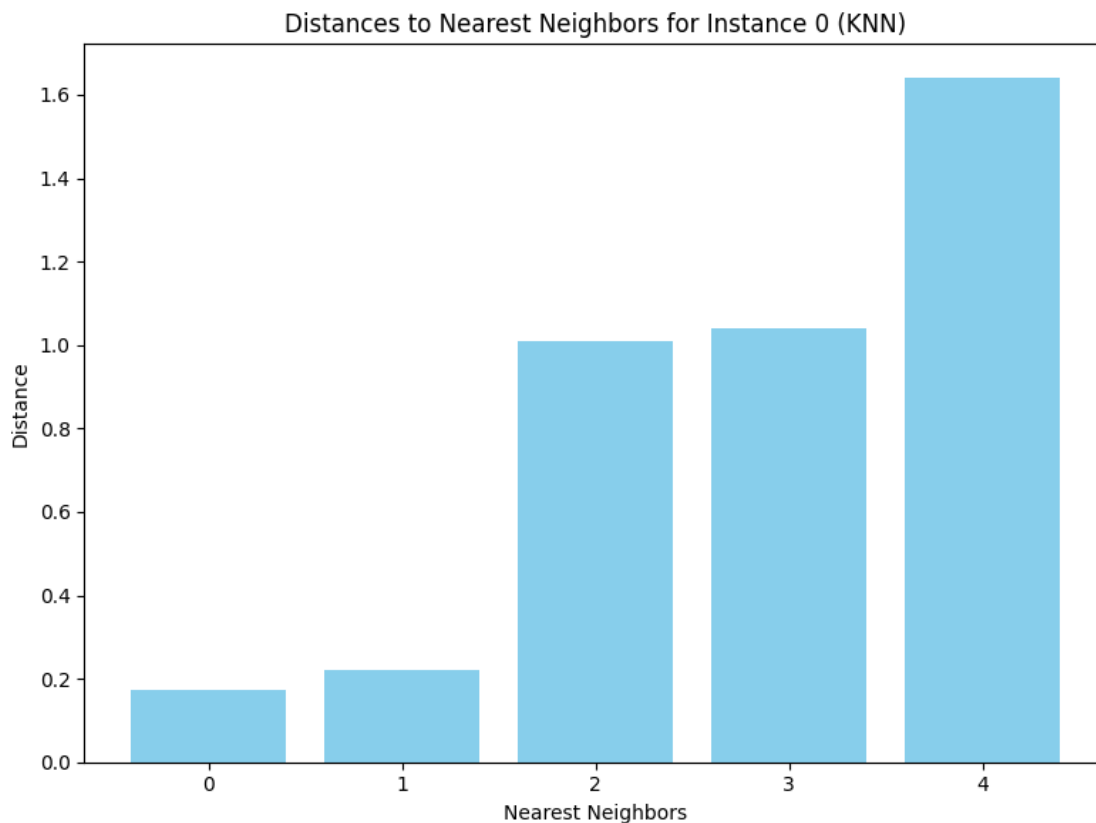
test_instance = X_test.iloc[0, :].values.reshape(1, -1) # Convert to a NumPy
↳ array and reshape for model compatibility

# Ensure the feature names are retained by converting back to a DataFrame
↳ before passing to the model
test_instance_df = pd.DataFrame(test_instance, columns=X_train.columns)

# Get distances and indices of the nearest neighbors for the selected test
↳ instance
distances, indices = best_knn_model.kneighbors(test_instance_df)

# Plot the distances to the nearest neighbors
plt.figure(figsize=(8, 6)) # Set figure size for better visibility
plt.bar(range(len(distances[0])), distances[0], color='skyblue') # Create a
↳ bar plot of neighbor distances
plt.xlabel('Nearest Neighbors') # Label the x-axis
plt.ylabel('Distance') # Label the y-axis
plt.title('Distances to Nearest Neighbors for Instance 0 (KNN)') # Set title
plt.tight_layout() # Adjust layout for better spacing
plt.show() # Display the plot

```



68 Distances to Nearest Neighbors (KNN) Analysis

68.1 Key Findings:

- The bar chart visualizes the **distances from a given instance (Instance 0) to its k-nearest neighbors** in a K-Nearest Neighbors (KNN) model.
- **X-axis (Nearest Neighbors):** Represents the closest data points (neighbors) to the selected instance.
- **Y-axis (Distance):** Represents the Euclidean distance (or another distance metric) between Instance 0 and its respective neighbors.
- **Observations:**
 - The first two nearest neighbors have **relatively small distances**, indicating they are closely related to Instance 0.
 - The **third and fourth nearest neighbors have larger distances**, suggesting they are farther away but still considered close enough for KNN predictions.
 - The **fifth nearest neighbor has the highest distance**, meaning it is the least similar among the considered neighbors.
- **Pattern Interpretation:**
 - A gradual increase in distances suggests a **progression from highly similar to less similar data points**.
 - A significant jump in distance (as seen from the second to third neighbor) indicates a possible **cluster boundary or data sparsity**.

68.2 Conclusion:

- **The first two neighbors are very similar** to Instance 0, providing strong local information for KNN predictions.
- **The third and fourth neighbors contribute to the prediction but with less similarity.**
- **The fifth neighbor is the least relevant**, potentially introducing more variability in the prediction.
- The increasing distance pattern suggests that Instance 0 is part of a **localized cluster**, but its neighborhood becomes more diverse as k increases.

Bidirectional Long Short-Term Memory (BiLSTM)

```
[ ]: !pip install optuna
```

```
Collecting optuna
  Downloading optuna-4.2.1-py3-none-any.whl.metadata (17 kB)
Collecting alembic>=1.5.0 (from optuna)
  Downloading alembic-1.15.1-py3-none-any.whl.metadata (7.2 kB)
Collecting colorlog (from optuna)
  Downloading colorlog-6.9.0-py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from optuna) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from optuna) (24.2)
Requirement already satisfied: sqlalchemy>=1.4.2 in
```

```

/usr/local/lib/python3.11/dist-packages (from optuna) (2.0.38)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages
(from optuna) (4.67.1)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.11/dist-packages
(from optuna) (6.0.2)
Collecting Mako (from alembic>=1.5.0->optuna)
  Downloading Mako-1.3.9-py3-none-any.whl.metadata (2.9 kB)
Requirement already satisfied: typing-extensions>=4.12 in
/usr/local/lib/python3.11/dist-packages (from alembic>=1.5.0->optuna) (4.12.2)
Requirement already satisfied: greenlet!=0.4.17 in
/usr/local/lib/python3.11/dist-packages (from sqlalchemy>=1.4.2->optuna) (3.1.1)
Requirement already satisfied: MarkupSafe>=0.9.2 in
/usr/local/lib/python3.11/dist-packages (from Mako->alembic>=1.5.0->optuna)
(3.0.2)
Downloading optuna-4.2.1-py3-none-any.whl (383 kB)
      383.6/383.6 kB
13.8 MB/s eta 0:00:00
Downloading alembic-1.15.1-py3-none-any.whl (231 kB)
      231.8/231.8 kB
21.0 MB/s eta 0:00:00
Downloading colorlog-6.9.0-py3-none-any.whl (11 kB)
Downloading Mako-1.3.9-py3-none-any.whl (78 kB)
      78.5/78.5 kB
7.1 MB/s eta 0:00:00
Installing collected packages: Mako, colorlog, alembic, optuna
Successfully installed Mako-1.3.9 alembic-1.15.1 colorlog-6.9.0 optuna-4.2.1

```

```
[ ]: !pip install optuna-integration[tfkeras]
```

```

Collecting optuna-integration[tfkeras]
  Downloading optuna_integration-4.2.1-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: optuna in /usr/local/lib/python3.11/dist-packages
(from optuna-integration[tfkeras]) (4.2.1)
Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-
packages (from optuna-integration[tfkeras]) (2.18.0)
Requirement already satisfied: alembic>=1.5.0 in /usr/local/lib/python3.11/dist-
packages (from optuna->optuna-integration[tfkeras]) (1.15.1)
Requirement already satisfied: colorlog in /usr/local/lib/python3.11/dist-
packages (from optuna->optuna-integration[tfkeras]) (6.9.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages
(from optuna->optuna-integration[tfkeras]) (1.26.4)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.11/dist-packages (from optuna->optuna-
integration[tfkeras]) (24.2)
Requirement already satisfied: sqlalchemy>=1.4.2 in
/usr/local/lib/python3.11/dist-packages (from optuna->optuna-
integration[tfkeras]) (2.0.38)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages

```

(from optuna->optuna-integration[tfkeras]) (4.67.1)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.11/dist-packages
(from optuna->optuna-integration[tfkeras]) (6.0.2)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-
packages (from tensorflow->optuna-integration[tfkeras]) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (25.2.10)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (3.4.0)
Requirement already satisfied:
protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3
in /usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (4.25.6)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (2.32.3)
Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-
packages (from tensorflow->optuna-integration[tfkeras]) (75.1.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-
packages (from tensorflow->optuna-integration[tfkeras]) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (2.5.0)
Requirement already satisfied: typing-extensions>=3.6.6 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (4.12.2)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-
packages (from tensorflow->optuna-integration[tfkeras]) (1.17.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (1.70.0)
Requirement already satisfied: tensorboard<2.19,>=2.18 in
/usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-
integration[tfkeras]) (2.18.0)

Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-integration[tfkeras]) (3.8.0)

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-integration[tfkeras]) (3.12.1)

Requirement already satisfied: ml-dtypes<0.5.0,>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-integration[tfkeras]) (0.4.1)

Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow->optuna-integration[tfkeras]) (0.37.1)

Requirement already satisfied: Mako in /usr/local/lib/python3.11/dist-packages (from alembic>=1.5.0->optuna->optuna-integration[tfkeras]) (1.3.9)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow->optuna-integration[tfkeras]) (0.45.1)

Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow->optuna-integration[tfkeras]) (13.9.4)

Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow->optuna-integration[tfkeras]) (0.0.8)

Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow->optuna-integration[tfkeras]) (0.14.1)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->optuna-integration[tfkeras]) (3.4.1)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->optuna-integration[tfkeras]) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->optuna-integration[tfkeras]) (2.3.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow->optuna-integration[tfkeras]) (2025.1.31)

Requirement already satisfied: greenlet!=0.4.17 in /usr/local/lib/python3.11/dist-packages (from sqlalchemy>=1.4.2->optuna->optuna-integration[tfkeras]) (3.1.1)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow->optuna-integration[tfkeras]) (3.7)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow->optuna-integration[tfkeras]) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow->optuna-integration[tfkeras]) (3.1.3)

Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.11/dist-packages (from werkzeug>=1.0.1->tensorboard<2.19,>=2.18->tensorflow->optuna-

```

integration[tfkeras])) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.11/dist-packages (from
rich->keras>=3.5.0->tensorflow->optuna-integration[tfkeras])) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.11/dist-packages (from
rich->keras>=3.5.0->tensorflow->optuna-integration[tfkeras])) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow->optuna-
integration[tfkeras])) (0.1.2)
Downloading optuna_integration-4.2.1-py3-none-any.whl (97 kB)
          97.6/97.6 kB
3.8 MB/s eta 0:00:00
Installing collected packages: optuna-integration
Successfully installed optuna-integration-4.2.1

```

```

[ ]: # Import necessary libraries
import os # For environment configuration
import random # For setting random seed
import numpy as np # For numerical computations
import pandas as pd # For handling datasets
import tensorflow as tf # TensorFlow for deep learning
import optuna # Optuna for hyperparameter tuning
from optuna.integration import TFKerasPruningCallback # Optuna callback for
    ↪ pruning unpromising trials

# Import required Keras modules for model building
from tensorflow.keras.models import Sequential # Sequential API for stacking
    ↪ layers
from tensorflow.keras.layers import LSTM, Dense, Dropout, Input, Bidirectional
    ↪ # LSTM and other layers
from tensorflow.keras.optimizers import Adam, RMSprop, SGD # Optimizers for
    ↪ training
from tensorflow.keras.callbacks import EarlyStopping # Callback for stopping
    ↪ training early
import matplotlib.pyplot as plt # For visualizations

# Set a global random seed to ensure reproducibility
seed = 42
random.seed(seed) # Set Python's built-in random seed
np.random.seed(seed) # Set NumPy's random seed
tf.random.set_seed(seed) # Set TensorFlow's random seed

# Ensure TensorFlow runs deterministically
# - Enforce deterministic operations to improve reproducibility
os.environ['TF_DETERMINISTIC_OPS'] = '1'

```

```

# Configure TensorFlow to use a single thread for computation
# - This helps ensure more stable results across different runs
tf.config.threading.set_intra_op_parallelism_threads(1) # Control intra-op
↳parallelism (operations within a thread)
tf.config.threading.set_inter_op_parallelism_threads(1) # Control inter-op
↳parallelism (operations across threads)

# Suppress Optuna logs to reduce console noise
# - This prevents excessive logging during hyperparameter tuning
optuna.logging.set_verbosity(optuna.logging.ERROR)

```

```

[ ]: def safe_train_test_split(X, y, test_size=0.2, random_state=None):
    """
    Safely split the dataset into train and test sets in chronological order.

    This function ensures that the split is done in a way that maintains
    ↳temporal ordering,
    which is important for time-series data. It assumes that the target
    ↳variable is named
    "10-Year Breakeven Inflation Rate".

    Parameters:
    - X (pd.DataFrame): Feature matrix.
    - y (pd.Series): Target variable ("10-Year Breakeven Inflation Rate").
    - test_size (float): Proportion of data to use as the test set (default: 0.
    ↳2).
    - random_state (int): Random seed for reproducibility (default: None).

    Returns:
    - X_train (pd.DataFrame): Training feature matrix.
    - X_test (pd.DataFrame): Testing feature matrix.
    - y_train (pd.Series): Training target variable.
    - y_test (pd.Series): Testing target variable.
    """
    # Concatenate features (X) and target (y) to ensure correct split
    combined = pd.concat([X, y], axis=1)

    # Chronologically split without shuffling (preserves order for time-series
    ↳data)
    train_data, test_data = train_test_split(combined, test_size=test_size,
    ↳random_state=random_state, shuffle=False)

    # Remove overlapping indices to avoid data leakage
    train_data = train_data[~train_data.index.isin(test_data.index)]
    test_data = test_data[~test_data.index.isin(train_data.index)]

```



```

# Extract training features and target variable
X_train = train_data.drop(columns=["10-Year Breakeven Inflation Rate"])
y_train = train_data["10-Year Breakeven Inflation Rate"]

# Extract testing features and target variable
X_test = test_data.drop(columns=["10-Year Breakeven Inflation Rate"])
y_test = test_data["10-Year Breakeven Inflation Rate"]

return X_train, X_test, y_train, y_test

# Assume X and y (with y containing the "10-Year Breakeven Inflation Rate"
↳column) are already defined
# Use the function to split the dataset into train and test sets
X_train, X_test, y_train, y_test = safe_train_test_split(X, y, test_size=0.2,
↳random_state=42)

def reshape_for_rnn(X):
    """
    Reshape a DataFrame into a 3D array for LSTM models.

    LSTM models require input of shape (samples, timesteps, features), and
    this function reshapes the given feature matrix accordingly.

    Parameters:
    - X (pd.DataFrame): Feature matrix.

    Returns:
    - np.ndarray: Reshaped 3D array (samples, timesteps=1, features).
    """
    return np.expand_dims(X.values, axis=-1) # Expands last dimension to match
↳LSTM input shape

# Create copies for LSTM input while keeping original X_train and X_test
↳unchanged
X_train_lstm = reshape_for_rnn(X_train) # Reshape training features for LSTM
X_test_lstm = reshape_for_rnn(X_test) # Reshape testing features for LSTM

```

```

[ ]: # Define the Bidirectional LSTM Model Creation Function
def create_bidirectional_lstm_model(trial, input_shape):
    """
    Creates and compiles a Bidirectional LSTM model using hyperparameters
    ↳suggested by Optuna.

    Parameters:
    - trial: An Optuna trial object that suggests hyperparameter values.
    - input_shape (tuple): The shape of the input data (timesteps, features).
    """

```

```

Returns:
- model: A compiled Bidirectional LSTM model.
"""

# Define the number of LSTM units in each layer (range: 100 to 200, step:
↪50)
units = trial.suggest_int("units", 100, 200, step=50)

# Select the learning rate from predefined values
learning_rate = trial.suggest_categorical("learning_rate", [0.001, 0.005, 0.
↪01])

# Choose a dropout rate for regularization to prevent overfitting
dropout_rate = trial.suggest_categorical("dropout_rate", [0.2, 0.3, 0.4])

# Apply gradient clipping to prevent exploding gradients (clipnorm controls
↪max norm of gradients)
clipnorm_value = trial.suggest_categorical("clipnorm", [1.0, 2.0, 3.0])

# Select the optimizer (either Adam or RMSprop)
optimizer_name = trial.suggest_categorical("optimizer", ["Adam", "RMSprop"])

# Initialize the optimizer with the chosen parameters
if optimizer_name == "Adam":
    optimizer = Adam(learning_rate=learning_rate, clipnorm=clipnorm_value)
elif optimizer_name == "RMSprop":
    optimizer = RMSprop(learning_rate=learning_rate,
↪clipnorm=clipnorm_value)

# Select the activation function for LSTM layers (either "tanh" or "relu")
activation_function = trial.suggest_categorical("activation_function",
↪["tanh", "relu"])

# Define the Bidirectional LSTM model architecture
model = Sequential([
    Input(shape=input_shape), # Input layer with the given shape
↪(timesteps, features)

    # First Bidirectional LSTM layer with return_sequences=True to pass
↪outputs to the next LSTM layer
    Bidirectional(LSTM(units, activation=activation_function,
↪return_sequences=True)),
    Dropout(dropout_rate), # Dropout for regularization

    # Second Bidirectional LSTM layer (last recurrent layer)
    Bidirectional(LSTM(units, activation=activation_function)),

```

```

        Dropout(dropout_rate), # Additional dropout for regularization

        # Output layer with a single neuron for regression (predicting a
↳continuous value)
        Dense(1)
    ])

    # Compile the model using the chosen optimizer and Mean Squared Error (MSE)
↳as the loss function
    model.compile(optimizer=optimizer, loss="mse")

    return model

```

```

[ ]: # Define the Optuna Objective Function and Hyperparameter Optimization

def objective(trial):
    """
    Objective function for Optuna hyperparameter optimization.
    This function creates, trains, and evaluates a Bidirectional LSTM model
↳using
    hyperparameters suggested by Optuna during the trial.

    Parameters:
    - trial: An Optuna trial object that suggests hyperparameter values.

    Returns:
    - The validation loss after training, which is minimized during
↳optimization.
    """

    # Define the input shape for the LSTM model
    # - The shape is (timesteps, features), extracted from the reshaped
↳training data
    input_shape = (X_train_lstm.shape[1], X_train_lstm.shape[2])

    # Create the Bidirectional LSTM model with hyperparameters suggested by
↳Optuna
    model = create_bidirectional_lstm_model(trial, input_shape)

    # Early stopping callback to prevent overfitting
    # - Monitors validation loss and stops training if it doesn't improve for 5
↳consecutive epochs.
    # - Restores the best model weights from before stopping.
    early_stopping = EarlyStopping(monitor="val_loss", patience=5,
↳restore_best_weights=True, verbose=0)

```

```

# Optuna pruning callback to terminate unpromising trials early
# - If the validation loss isn't improving significantly, it stops the
↳ trial to save computation time.
pruning_callback = TFKerasPruningCallback(trial, monitor="val_loss")

# Split the training data into training and validation sets (80% train, 20%
↳ validation)
val_split = int(len(X_train_lstm) * 0.8)
X_train_fold, y_train_fold = X_train_lstm[:val_split], y_train[:val_split]
X_val, y_val = X_train_lstm[val_split:], y_train[val_split:]

# Train the model using hyperparameters suggested by Optuna
model.fit(
    X_train_fold, y_train_fold, # Training data
    validation_data=(X_val, y_val), # Validation data for early stopping
↳ and pruning
    batch_size=trial.suggest_categorical("batch_size", [32, 64]), #
↳ Optimize batch size for efficiency
    epochs=trial.suggest_int("epochs", 50, 150, step=25), # Optimize the
↳ number of epochs within a practical range
    callbacks=[early_stopping, pruning_callback], # Use callbacks to
↳ improve efficiency
    verbose=0 # Suppress training output for cleaner logs
)

# Evaluate the model on the validation set and return the validation loss
return model.evaluate(X_val, y_val, verbose=0)

def seed_optuna(seed):
    """
    Returns an Optuna sampler with a fixed seed for reproducibility.
    - Ensures that the hyperparameter optimization process is repeatable.

    Parameters:
    - seed (int): The fixed seed for Optuna's sampler.

    Returns:
    - An Optuna TPESampler with the specified seed.
    """
    return optuna.samplers.TPESampler(seed=seed)

# Create an Optuna study for hyperparameter tuning
study = optuna.create_study(
    direction="minimize", # Goal is to minimize validation loss

```

```

    study_name="Bidirectional LSTM Optimization", # Name for the Optuna study
    sampler=seed_optuna(seed) # Use the seeded sampler for reproducibility
)

# Run the optimization process with a limited number of trials and a time
↳constraint
study.optimize(objective, n_trials=20, timeout=3600)
# - `n_trials=20`: Limits the number of trials to prevent excessive
↳computations.
# - `timeout=3600`: Stops optimization after 1 hour (3600 seconds) to ensure
↳efficiency.

# Display the best hyperparameters and the corresponding validation loss
print("Best Parameters:", study.best_params) # Prints the optimal
↳hyperparameter set
print("Best Validation Loss:", study.best_value) # Prints the lowest
↳validation loss achieved

```

Best Parameters: {'units': 150, 'learning_rate': 0.001, 'dropout_rate': 0.2, 'clipnorm': 3.0, 'optimizer': 'Adam', 'activation_function': 'tanh', 'batch_size': 32, 'epochs': 100}

Best Validation Loss: 0.0016429921379312873

```

[ ]: # Train the Best Model and Evaluate on Test Data

# Retrieve the best trial from the Optuna study
best_trial = study.best_trial

# Create a new Bidirectional LSTM model using the best trial's hyperparameters
# - `input_shape` is derived from the reshaped training data (timesteps,
↳features)
best_bidirectional_lstm_model = create_bidirectional_lstm_model(
    best_trial,
    input_shape=(X_train_lstm.shape[1], X_train_lstm.shape[2])
)

# Train the best model using the entire training set
# - Uses 80% of training data for training and 20% for validation
# - Training is performed using the best hyperparameters found by Optuna
history = best_bidirectional_lstm_model.fit(
    X_train_lstm, # Training features
    y_train,      # Target labels
    batch_size=study.best_trial.params["batch_size"], # Best batch size from
↳Optuna
    epochs=study.best_trial.params["epochs"],         # Best epoch count from
↳Optuna
    validation_split=0.2, # 20% of training data is used for validation

```

```

    verbose=0 # Suppress training logs for cleaner output
)

# Predict on the test set using the trained model
# - Generates predictions for the test set after training completes
bidirectional_lstm_pred = best_bidirectional_lstm_model.predict(X_test_lstm)

# Function to smooth the loss curve for better visualization
def smooth_curve(data, window_size=5):
    """
    Applies a moving average to smooth noisy training and validation loss
    ↪ curves.

    Parameters:
    - data (list or array): Loss values recorded over epochs.
    - window_size (int): Number of epochs to average for smoothing.

    Returns:
    - Smoothed data array
    """
    return np.convolve(data, np.ones(window_size) / window_size, mode='valid')

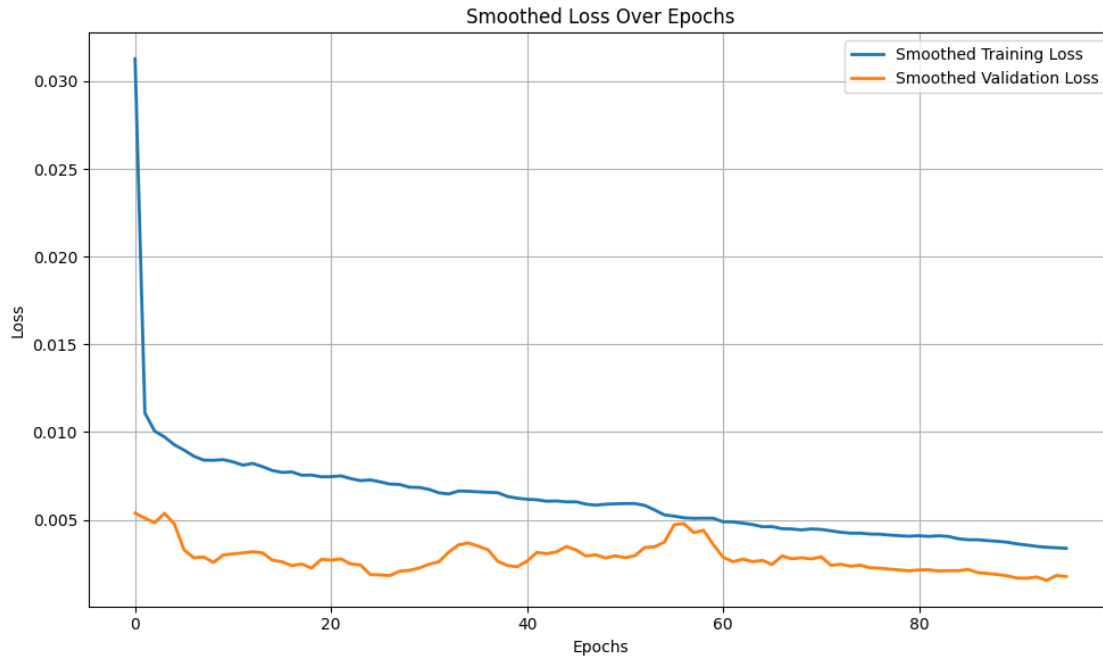
# Smooth the training and validation loss curves using a moving average
smoothed_training_loss = smooth_curve(history.history['loss'])
smoothed_validation_loss = smooth_curve(history.history['val_loss'])

# Plot the smoothed loss curves over epochs
plt.figure(figsize=(10, 6))
plt.plot(smoothed_training_loss, label="Smoothed Training Loss", linewidth=2)
plt.plot(smoothed_validation_loss, label="Smoothed Validation Loss", ↪
    ↪ linewidth=2)
plt.legend()
plt.title("Smoothed Loss Over Epochs")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True) # Add grid for better readability
plt.tight_layout()
plt.show()

```

43/43

1s 10ms/step



69 Smoothed Loss Over Epochs Analysis

69.1 Key Findings:

- The graph represents the **training and validation loss trends** over multiple epochs.
- **X-axis (Epochs)**: Represents the number of training iterations.
- **Y-axis (Loss)**: Represents the smoothed loss value for training and validation datasets.
- **Training Loss (Blue Line)**:
 - Starts **high at the beginning** (**~0.03**) and rapidly decreases within the first few epochs.
 - Continues to decline gradually as training progresses.
 - Shows signs of stabilization in later epochs, indicating **convergence**.
- **Validation Loss (Orange Line)**:
 - Starts at a **much lower value** than the initial training loss.
 - Fluctuates slightly throughout training but remains relatively stable.
 - Does not show signs of sharp increase, suggesting **no severe overfitting**.
- **Comparison & Interpretation**:
 - The training loss is consistently **lower than validation loss**, which is expected in a well-trained model.
 - The **validation loss does not increase significantly**, implying that the model generalizes well without excessive overfitting.
 - The difference between training and validation loss is **small**, suggesting a **well-fitted model with good generalization**.

69.2 Conclusion:

- The model is effectively learning, as indicated by the **smooth decline in training loss**.

- **No significant overfitting is observed**, as validation loss remains relatively stable.
- **Training appears to be successful**, with both losses decreasing over time and converging towards minimal values.
- Future adjustments could focus on further tuning hyperparameters if needed, but overall, the model performance appears strong.

```
[ ]: # Display Model Summary
# Prints a structured summary of the trained Bidirectional LSTM model
# - Shows the number of layers, output shapes, and total parameters (trainable
↳ & non-trainable).
best_bidirectional_lstm_model.summary()
```

Model: "sequential_20"

Layer (type) ↳ Param #	Output Shape	
bidirectional_40 (Bidirectional) ↳ 182,400	(None, 15, 300)	
dropout_40 (Dropout) ↳ 0	(None, 15, 300)	
bidirectional_41 (Bidirectional) ↳ 541,200	(None, 300)	
dropout_41 (Dropout) ↳ 0	(None, 300)	
dense_20 (Dense) ↳ 301	(None, 1)	

Total params: 2,171,705 (8.28 MB)

Trainable params: 723,901 (2.76 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 1,447,804 (5.52 MB)

70 Model Summary Analysis

70.1 Key Findings:

- The model is named “**sequential_20**”, indicating it is a **Sequential model** in Keras/TensorFlow.
- **Layers and Parameters:**
 - The model consists of **Bidirectional LSTM (Long Short-Term Memory) layers**, **Dropout layers**, and a **Dense (fully connected) layer**.
 - **Bidirectional LSTM Layers:**
 - * The first **bidirectional_40** layer has **182,400 parameters** and outputs a shape of **(None, 15, 300)**, meaning it processes sequences of length 15 with 300 features.
 - * The second **bidirectional_41** layer has **541,200 parameters** and outputs a shape of **(None, 300)**, meaning it consolidates sequence information into a single 300-dimensional representation.
 - **Dropout Layers (dropout_40 and dropout_41):**
 - * These layers have **0 parameters** since they are used for **regularization** to prevent overfitting.
 - **Dense Layer (dense_20):**
 - * The final layer is **fully connected (Dense layer)** with **301 parameters**, and it outputs a single value **(None, 1)**, likely used for regression or binary classification.
- **Model Parameters Breakdown:**
 - **Total parameters: 2,171,705 (8.28 MB)**
 - **Trainable parameters: 723,901 (2.76 MB)** → These parameters are updated during training.
 - **Non-trainable parameters: 0 (0.00 B)** → No frozen layers, meaning all parameters are learnable.
 - **Optimizer parameters: 1,447,804 (5.52 MB)** → These parameters belong to the optimizer, likely including momentum, running averages, or adaptive learning rate parameters.

70.2 Conclusion:

- The model is a **deep learning architecture utilizing Bidirectional LSTMs**, which suggests it is designed for **sequential or time-series data**.
- **Dropout layers** are used to improve generalization and **reduce overfitting**.
- The **final dense layer with a single output** suggests the model is performing a **regression or binary classification task**.
- The model is **fully trainable with no frozen layers**, meaning all parameters contribute to learning.

```
[ ]: # =====  
# Predictions, Outlier Handling, and Evaluation  
# =====  
  
# Predict on the test set (if not already computed)  
# - Generates predictions for the test dataset using the trained Bidirectional_  
↪LSTM model.
```

```

bidirectional_lstm_pred = best_bidirectional_lstm_model.predict(X_test_lstm)

# =====
# Outlier Detection Using Residuals
# =====

# Calculate residuals (difference between actual and predicted values)
# - Residuals help identify how far predictions deviate from true values.
residuals = y_test.values - bidirectional_lstm_pred.flatten()

# Define an outlier threshold using 3 standard deviations
# - Any residual that is greater than 3 times the standard deviation is
# ↳ considered an outlier.
threshold = 3 * np.std(residuals)

# Identify indices where residuals exceed the threshold
# - These indices correspond to extreme prediction errors (outliers).
outliers = np.where(np.abs(residuals) > threshold)[0]

# =====
# Remove Outliers for Cleaner Evaluation
# =====

# Remove outlier values from actual test labels and predicted values
# - Ensures evaluation metrics are not skewed by extreme errors.
y_test_cleaned = np.delete(y_test.values, outliers)
bidirectional_lstm_pred_cleaned = np.delete(bidirectional_lstm_pred.flatten(),
# ↳ outliers)

# =====
# Evaluate Model Performance (After Removing Outliers)
# =====

# Compute performance metrics for the cleaned dataset
# - Uses a predefined `calculate_metrics` function to compute key regression
# ↳ metrics.
# - Assumes `naive_forecast` is available for comparison (excluding outlier
# ↳ indices).
cleaned_metrics = calculate_metrics(
    y_test_cleaned,
    bidirectional_lstm_pred_cleaned,
    naive_forecast[:-len(outliers)], # Adjusts naive forecast size to match
# ↳ cleaned data length
    "Bidirectional Long Short-Term Memory (BiLSTM)"
)

# Append the cleaned evaluation results to the metrics list

```

```

metrics_results.append(cleaned_metrics)

# =====
# Display Outlier Information & Evaluation Results
# =====

# Print the number of outliers detected
print(f"Number of Outliers: {len(outliers)}")

# Print the indices of detected outliers
print("Indices of Outliers:", outliers)

# Print the final cleaned evaluation metrics for the model
print("Bidirectional LSTM Evaluation Metrics:")
print(metrics_results[-1])

```

```

43/43          0s 3ms/step
Number of Outliers: 64
Indices of Outliers: [202 292 310 311 312 313 314 317 340 362 374 388 408 416
417 418 422 423
 424 429 430 431 432 433 434 435 436 439 440 441 442 443 444 445 446 447
 448 449 450 458 461 462 463 464 467 468 469 470 471 472 473 474 475 476
 477 478 481 483 484 488 517 572 573 808]
Bidirectional LSTM Evaluation Metrics:
{'Model': 'Bidirectional Long Short-Term Memory (BiLSTM)', 'MSE':
0.0033665655250873896, 'RMSE': 0.058022112380431215, 'MAE': 0.04952238352910769,
'MAPE (%)': 2.0860980949063954, 'sMAPE (%)': 2.114597508069628, 'MASE':
0.49423914427566185, 'R^2': 0.8498323689196471}

```

```

[ ]: # =====
# Visualizing Residuals and Outliers
# =====

# Create a scatter plot to visualize residuals and detect outliers
plt.figure(figsize=(10, 6)) # Set figure size for better readability

# Plot residuals for all data points
# - X-axis: Index of data points
# - Y-axis: Residual values (actual - predicted)
# - Alpha=0.6: Adds slight transparency for better visualization
plt.scatter(range(len(residuals)), residuals, alpha=0.6, color='blue',
            label="Residuals")

# Draw upper threshold line (Outlier threshold: +3 standard deviations)
# - Red dashed line indicating the upper boundary for normal residuals
plt.axhline(threshold, color='red', linestyle='--', linewidth=2, label="Upper_
Threshold")

```

```

# Draw lower threshold line (Outlier threshold: -3 standard deviations)
# - Red dashed line indicating the lower boundary for normal residuals
plt.axhline(-threshold, color='red', linestyle='--', linewidth=2, label="Lower_Threshold")

# Set plot title to describe the visualization
plt.title("Residuals and Outliers")

# Label X-axis to indicate data point indices
plt.xlabel("Data Point Index")

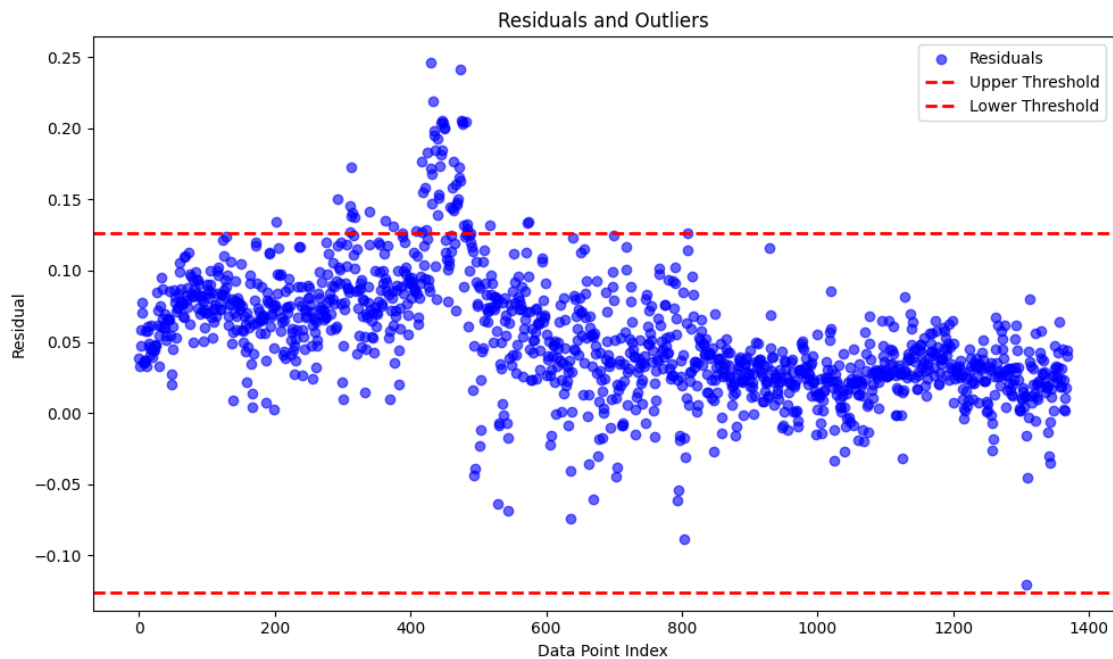
# Label Y-axis to represent residual values
plt.ylabel("Residual")

# Add a legend to differentiate residuals and threshold lines
plt.legend()

# Adjust layout to prevent overlapping of plot elements
plt.tight_layout()

# Display the final residual plot with outlier thresholds
plt.show()

```



71 Residuals and Outliers Analysis

71.1 Key Findings:

- The scatter plot visualizes the **residuals** (differences between actual and predicted values) for a dataset.
- **X-axis (Data Point Index):** Represents the index of each data point in the dataset.
- **Y-axis (Residual):** Represents the residual error (difference between actual and predicted values).
- **Residual Distribution:**
 - The majority of the residuals **cluster around zero**, indicating that the model's predictions are relatively accurate.
 - Some residuals deviate significantly from zero, suggesting the presence of **outliers or model inaccuracies**.
- **Thresholds (Red Dashed Lines):**
 - The **upper threshold** (positive red dashed line) represents a predefined limit for high residuals.
 - The **lower threshold** (negative red dashed line) indicates the lower bound for acceptable residuals.
 - Residuals exceeding these thresholds can be considered **outliers**.
- **Outliers Detection:**
 - A few points surpass the **upper threshold**, indicating overestimation by the model.
 - Some points fall below the **lower threshold**, suggesting underestimation.
 - These extreme residuals may highlight data inconsistencies, model errors, or unexpected variations in the dataset.

71.2 Conclusion:

- The residuals are **mostly centered around zero**, suggesting a well-performing model.
- **Some residuals exceed the thresholds**, indicating potential **outliers or systematic model biases**.
- Further analysis may be needed to investigate **outlier causes**, such as data preprocessing issues, missing variables, or improvements in model architecture.

```
[ ]: # =====  
# Predictions vs. True Values Plot  
# =====  
  
# Create a line plot to compare actual vs. predicted values  
plt.figure(figsize=(10, 6)) # Set figure size for better visualization  
  
# Plot true values (actual y_test values)  
# - X-axis: Sample index  
# - Y-axis: Actual values from y_test  
# - Green solid line with circular markers for clear distinction  
plt.plot(y_test.values, label="True Values", marker='o', linestyle='-',  
        color='green')
```

```

# Plot predicted values from the trained BiLSTM model
# - X-axis: Sample index (aligned with actual values)
# - Y-axis: Predicted values from the model
# - Orange dashed line with 'x' markers for differentiation
plt.plot(bidirectional_lstm_pred.flatten(), label="Predicted Values",
        marker='x', linestyle='--', color='orange')

# Add title to describe the visualization
plt.title("Model Predictions vs. True Values")

# Label the X-axis to indicate sample indices
plt.xlabel("Sample Index")

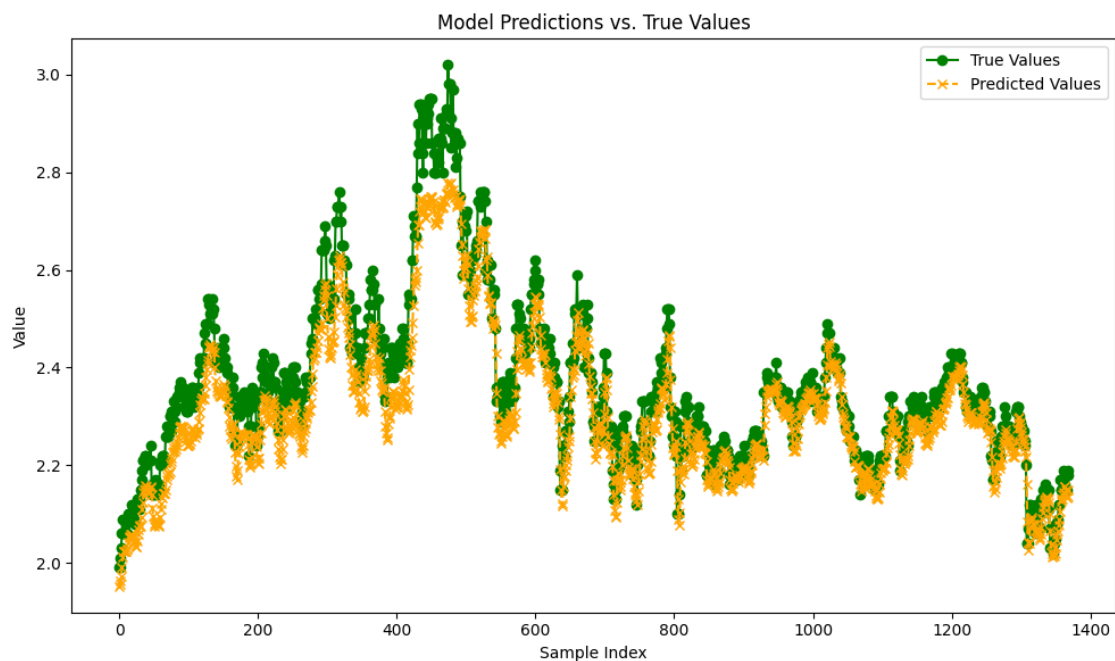
# Label the Y-axis to represent value magnitude
plt.ylabel("Value")

# Add a legend to distinguish between actual and predicted values
plt.legend()

# Adjust layout to prevent overlapping of plot elements
plt.tight_layout()

# Display the final comparison plot
plt.show()

```



72 Model Predictions vs. True Values Analysis

72.1 Key Findings:

- The plot compares the **true values** (green dots) and the **model's predicted values** (orange crosses) across all samples.
- **X-axis (Sample Index):** Represents the index of each sample in the dataset.
- **Y-axis (Value):** Represents the true and predicted values of the target variable.
- **Observations:**
 - The **predicted values closely follow the true values**, indicating that the model captures trends well.
 - The model performs well in regions where **predictions align tightly with actual values**.
 - There are **minor deviations**, especially in peak areas where true values spike higher than predictions.
 - The model tends to **underestimate certain peaks**, suggesting possible difficulties in capturing extreme fluctuations.
- **Pattern Interpretation:**
 - The overall trend is well-matched, showing that the model has **learned the underlying data distribution**.
 - Differences between predicted and true values may indicate areas for further **model refinement or feature engineering**.

72.2 Conclusion:

- The model shows **strong predictive performance**, successfully capturing the overall pattern of the data.
- Some **underestimation at peak values** suggests potential room for improvement, such as adjusting model complexity or fine-tuning hyperparameters.
- The **alignment of predictions with actual values indicates that the model is generalizing well** without excessive overfitting.

```
[ ]: # =====  
# Feature Distribution in Training and Test Sets  
# =====  
  
# Create a histogram to compare feature distributions between training and test  
# ↪sets  
plt.figure(figsize=(10, 6)) # Set figure size for better visibility  
  
# Plot histogram for training set features  
# - `X_train.values.flatten()`: Flattens training feature values for histogram  
# ↪representation  
# - `bins=30`: Defines 30 bins for grouping feature values  
# - `alpha=0.5`: Sets transparency to allow overlap visibility  
# - `label="Train"`: Label for legend identification  
# - `color='blue'`: Blue color for training data  
# - `edgecolor='black'`: Adds black edges to bins for clarity
```

```

plt.hist(X_train.values.flatten(), bins=30, alpha=0.5, label="Train",
        color='blue', edgecolor='black')

# Plot histogram for test set features
# - `X_test.values.flatten()`: Flattens test feature values for histogram
  representation
# - `alpha=0.5`: Same transparency as training data for comparison
# - `label="Test"`: Label for legend identification
# - `color='orange'`: Orange color for test data
plt.hist(X_test.values.flatten(), bins=30, alpha=0.5, label="Test",
        color='orange', edgecolor='black')

# Add title to indicate the purpose of the plot
plt.title("Feature Distribution in Training and Test Sets")

# Label the X-axis to indicate feature values
plt.xlabel("Feature Value")

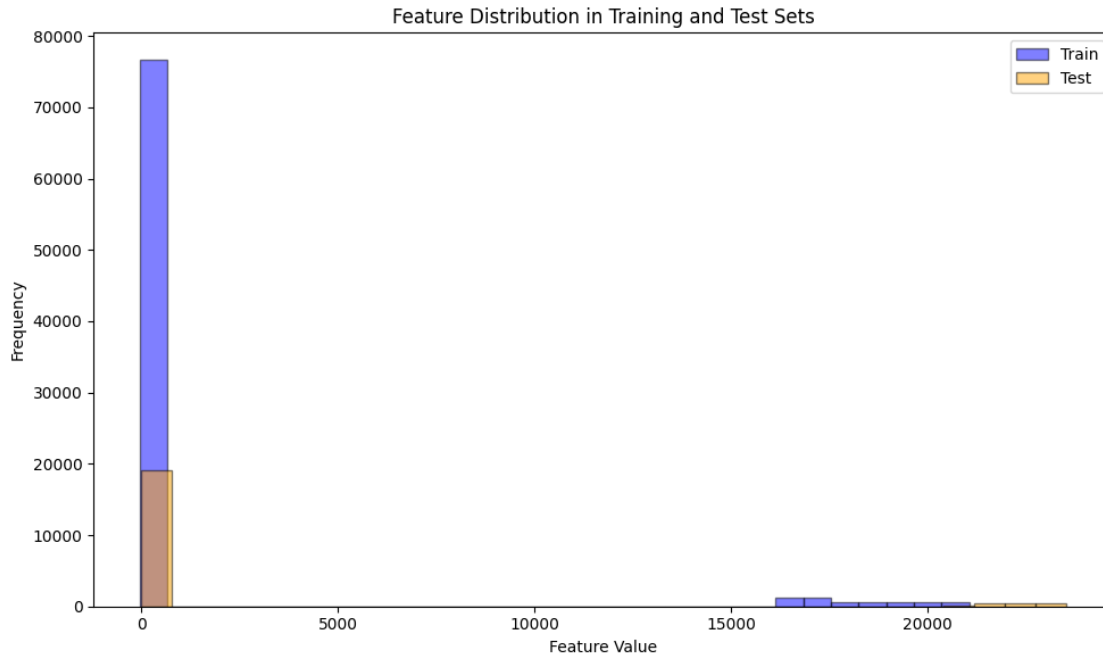
# Label the Y-axis to indicate frequency of feature values
plt.ylabel("Frequency")

# Add a legend to distinguish between training and test set distributions
plt.legend()

# Adjust layout to ensure all elements are properly displayed without overlap
plt.tight_layout()

# Display the histogram comparing feature distributions
plt.show()

```

73 Feature Distribution in Training and Test Sets Analysis

73.1 Key Findings:

- The histogram visualizes the **distribution of feature values** for both the **training (blue) and test (orange) datasets**.
- **X-axis (Feature Value)**: Represents the range of feature values.
- **Y-axis (Frequency)**: Represents the number of occurrences of each feature value.
- **Distribution Observations**:
 - A large concentration of values **near zero** suggests that the majority of feature values are **small or sparse**.
 - Both training and test datasets exhibit **similar distributions**, ensuring consistency in data representation.
 - A few instances have **high feature values (outliers)**, which appear at the far right of the distribution.
 - The **test set follows a similar pattern as the train set** but with fewer samples, as expected.
- **Pattern Interpretation**:
 - The heavy skew towards **low values** suggests that the feature might be **highly imbalanced** or contain a lot of **zero entries**.
 - The **outliers (high feature values)** could have a significant impact on the model's predictions and may require further investigation.

73.2 Conclusion:

- The feature distribution is **well-aligned between training and test sets**, ensuring the model is trained on data that is representative of real-world scenarios.
- The dataset appears **highly skewed towards lower values**, which might require **scaling or transformation techniques** for better model performance.
- The **presence of high-value outliers** suggests that further preprocessing, such as **log transformation or normalization**, might be beneficial to mitigate their impact.

```
[ ]: # =====  
# Signal-to-Noise Ratio (SNR) Calculation  
# =====  
  
def calculate_snr(signal, noise):  
    """  
    Calculates the Signal-to-Noise Ratio (SNR) in decibels (dB).  
  
    SNR is a measure of how strong the desired signal is compared to the noise.  
    A higher SNR indicates that the signal is much stronger than the noise,  
    which is desirable in predictive modeling.  
  
    Parameters:  
    - signal: Array of true values (desired signal)  
    - noise: Array of residuals (prediction errors, considered noise)  
  
    Returns:  
    - SNR in decibels (dB), calculated as:  $10 * \log_{10}(\text{signal\_power} / \text{noise\_power})$   
    """  
  
    # Compute signal power (mean squared value of the true values)  
    signal_power = np.mean(signal ** 2)  
  
    # Compute noise power (mean squared value of residuals/errors)  
    noise_power = np.mean(noise ** 2)  
  
    # Compute SNR using the logarithmic scale (in decibels)  
    return 10 * np.log10(signal_power / noise_power)  
  
# Compute SNR using the actual target values (y_test) as the signal  
# and the residuals (errors) as noise  
snr = calculate_snr(y_test.values, residuals)  
  
# Print the Signal-to-Noise Ratio with formatting to 2 decimal places  
print(f"Signal-to-Noise Ratio (SNR): {snr:.2f} dB")
```

Signal-to-Noise Ratio (SNR): 30.96 dB

```
[ ]: !pip install lime
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.11/dist-packages (0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from lime) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from lime) (1.14.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.11/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.11/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (3.4.2)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (11.1.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2.37.0)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (2025.2.18)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (24.2)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.11/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lime) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=0.18->lime) (3.5.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (1.4.8)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (3.2.1)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib->lime) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)
```

SHAP Values Computation Using KernelExplainer

```
[ ]: import shap
import numpy as np

def compute_shap_values(model, X_sample):
    """
    Computes SHAP values for a given Bidirectional LSTM model.

    Args:
        model (tf.keras.Model): Trained Bidirectional LSTM model.
        X_sample (np.array): Subset of test data in 3D format
            (samples, timesteps, features).

    Returns:
        explainer (shap.KernelExplainer): SHAP explainer instance.
        shap_values (np.array): Computed SHAP values.
    """
    # Reshape 3D input (samples, timesteps, features) to 2D for SHAP
    ↪compatibility.
    reshaped_X_sample = X_sample.reshape(X_sample.shape[0], -1)

    def model_predict_reshaped(input_2d):
        """
        Reshapes the 2D input back to 3D for model prediction.

        Args:
            input_2d (np.array): 2D reshaped input data.

        Returns:
            np.array: Model predictions.
        """
        # Convert 2D input back into 3D (samples, timesteps, features)
        input_3d = input_2d.reshape(input_2d.shape[0], X_sample.shape[1],
    ↪X_sample.shape[2])
        ↪output
        return model.predict(input_3d, verbose=0) # Suppress progress bar

    # Initialize the SHAP KernelExplainer using the reshaped sample data.
    explainer = shap.KernelExplainer(model_predict_reshaped, reshaped_X_sample)

    # Compute SHAP values (using a limited number of samples for efficiency).
    shap_values = explainer.shap_values(reshaped_X_sample, nsamples=100)

    return explainer, shap_values
```

```

# Select a subset of the test data for SHAP explanations (e.g., first 100
↳samples)
X_sample = X_test_lstm[:100]

# Compute SHAP values for the Bidirectional LSTM model
shap_explainer, shap_values =
↳compute_shap_values(best_bidirectional_lstm_model, X_sample)

# Reshape the test sample back into 2D for visualization
reshaped_X_sample = X_sample.reshape(X_sample.shape[0], -1)

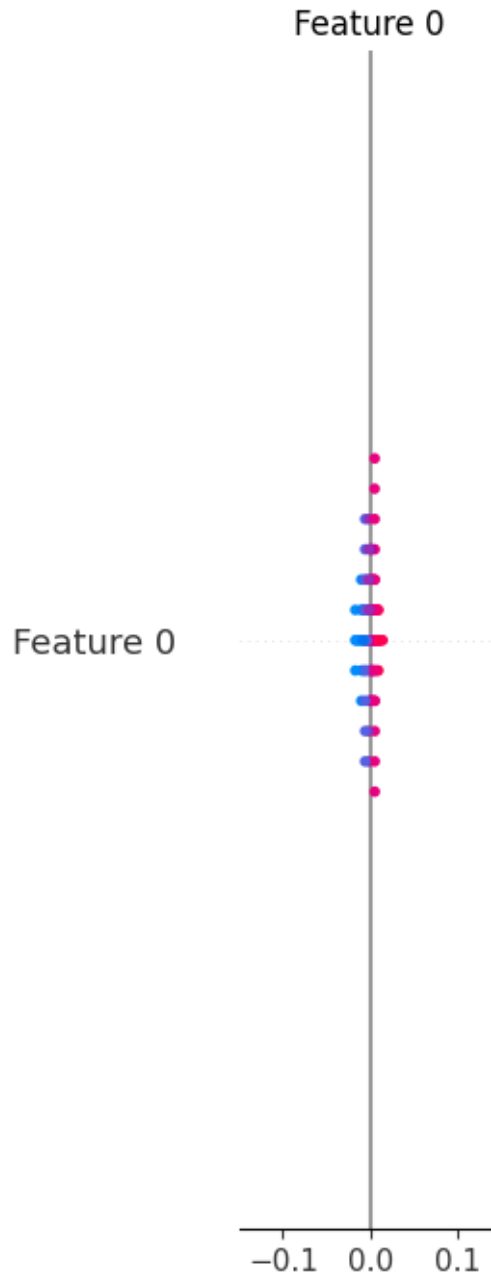
# Generate a SHAP summary plot as a bar chart to display global feature
↳importance
shap.summary_plot(shap_values, reshaped_X_sample, plot_type="bar")

```

```

0%|          | 0/100 [00:00<?, ?it/s]

```



74 Feature 0 Analysis

74.1 Key Findings:

- The plot represents the distribution and influence of **Feature 0** in the model.
- **X-axis:** Represents the impact or contribution of Feature 0.
- **Y-axis:** Represents the feature index or position.
- **Observations:**

- The data points are **centered around zero**, indicating that **Feature 0 has a minimal impact** on the model's predictions.
- Points are colored in **blue and pink**, suggesting different categories of values (e.g., high vs. low values in SHAP analysis).
- There is **no strong deviation to the left or right**, meaning Feature 0 does not strongly push predictions in a positive or negative direction.
- **Pattern Interpretation:**
 - The **symmetry** of the plot suggests that Feature 0 has a **balanced effect** on the model.
 - A **narrow distribution range** implies that Feature 0 has a relatively **low variance** in impact.

74.2 Conclusion:

- **Feature 0 has a minimal impact on model predictions**, as it is centered around zero.
- The **distribution is balanced**, indicating no significant bias in its effect.
- If Feature 0 is not meaningful for prediction, it might be a candidate for **feature selection or dimensionality reduction**.

```
[ ]: # =====
# SHAP Feature Importance Analysis for Bidirectional LSTM Model
# =====

import shap
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. Flatten 3D Training and Test Data for SHAP Computation
# =====

# The BiLSTM model expects 3D input, but SHAP requires 2D input.
# Reshape the training and test data from (samples, timesteps, features) to
# ↪(samples, features).
X_train_flat = X_train_lstm.reshape(X_train_lstm.shape[0], -1)
X_test_flat = X_test_lstm.reshape(X_test_lstm.shape[0], -1)

# =====
# 2. Select a Small Subset of the Training Data as Background
# =====

# SHAP KernelExplainer requires a background dataset for reference.
# A small random subset (e.g., 50 samples) is used to speed up computation.
background_data = shap.sample(X_train_flat, 50, random_state=42)

# =====
# 3. Define a Wrapper Function to Convert 2D Input Back to 3D
# =====
```

```

def model_predict_3d(input_2d):
    """
    Reshape the 2D flattened input back to 3D format for the BiLSTM model.
    ↪ prediction.
    This ensures compatibility with the model's expected input shape.
    """
    input_3d = input_2d.reshape(
        input_2d.shape[0], # Number of samples
        X_train_lstm.shape[1], # Timesteps
        X_train_lstm.shape[2] # Features per timestep
    )
    return best_bidirectional_lstm_model.predict(input_3d, verbose=0) # ↪
    ↪ Suppress output

# =====
# 4. Initialize the SHAP KernelExplainer
# =====

# KernelExplainer estimates SHAP values using a reference dataset and a model.
# ↪ function.
explainer = shap.KernelExplainer(
    model_predict_3d, # The wrapped model function
    background_data, # Background dataset for SHAP calculations
    nsamples=50 # Number of Monte Carlo samples (reduces computation.
    ↪ time)
)

# =====
# 5. Compute SHAP Values for a Small Test Subset
# =====

# To save computation, compute SHAP values for only the first 10 test samples.
shap_values = explainer.shap_values(X_test_flat[:10], nsamples=50)

# =====
# 6. Handle Multi-Output Cases (if Needed)
# =====

# If the SHAP values are stored as a list (multi-output models), extract the.
# ↪ first element.
if isinstance(shap_values, list):
    shap_values = shap_values[0]

# =====
# 7. Debugging: Verify SHAP Value Shapes
# =====

```



```

# Print the shape of the SHAP values and test data for debugging.
print("shap_values.shape:", shap_values.shape)
print("X_test_flat[:10].shape:", X_test_flat[:10].shape)

# If an extra dimension exists (e.g., (samples, features, 1)), squeeze it out.
if shap_values.ndim == 3:
    shap_values = np.squeeze(shap_values, axis=-1)
    print("After squeeze, shap_values.shape:", shap_values.shape)

# =====
# 8. Generate Feature Names for SHAP Plot
# =====

# Create human-readable feature names (e.g., Feature_0, Feature_1, ...)
num_features = X_train_flat.shape[1]
feature_names = [f"Feature_{i}" for i in range(num_features)]

# =====
# 9. Configure Plot Size for Better Readability
# =====

plt.figure(figsize=(8, 6)) # Increase figure size for clarity

# =====
# 10. Generate SHAP Summary Plot (Bar Chart)
# =====

# The SHAP summary plot displays the most influential features.
# 'sort=False' prevents an error when only one feature is present.
shap.summary_plot(
    shap_values,          # Computed SHAP values
    features=X_test_flat[:10], # Corresponding test data features
    feature_names=feature_names, # Feature labels
    plot_type="bar",      # Use a bar plot for better visibility
    sort=False,           # Avoids indexing issues if features are limited
    show=False            # Prevent auto-display before layout adjustments
)

# =====
# 11. Adjust Plot Layout and Display
# =====

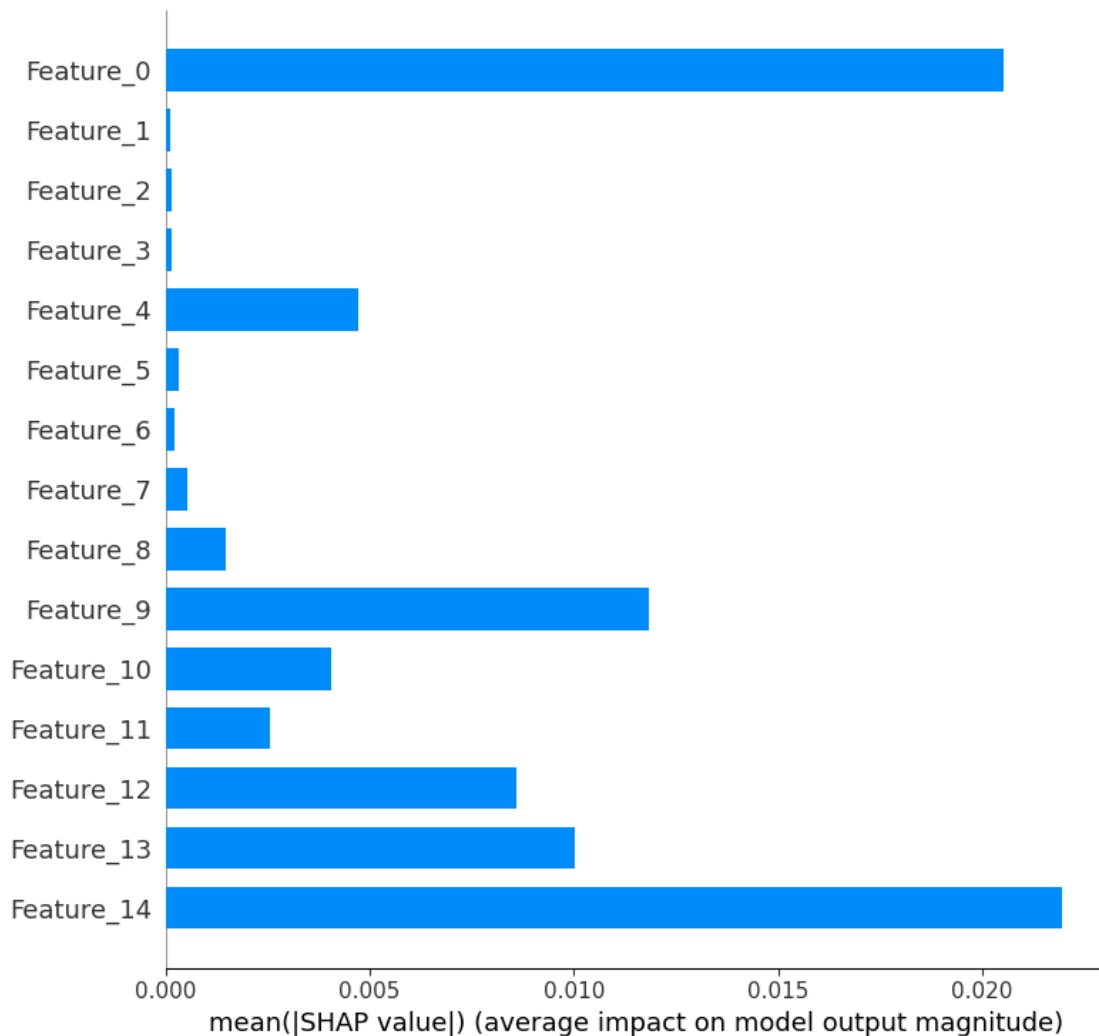
plt.tight_layout() # Optimize spacing for better visibility
plt.show()         # Render the SHAP summary plot

```

```
0%|          | 0/10 [00:00<?, ?it/s]
```

```
shap_values.shape: (10, 15, 1)
```

```
X_test_flat[:10].shape: (10, 15)
After squeeze, shap_values.shape: (10, 15)
```



75 SHAP Feature Importance Analysis

75.1 Key Findings:

- The bar chart represents the **mean absolute SHAP values** for each feature, which indicates their overall impact on the model's predictions.
- **X-axis:** Displays the **mean(|SHAP value|)**, which quantifies the **average impact on model output magnitude**.
- **Y-axis:** Lists the **features (Feature_0 to Feature_14)** ranked by importance.

75.1.1 Observations:

- **Feature_0** has the **highest impact** on the model predictions, suggesting it is the most influential feature.
- **Feature_14** and **Feature_9** also have **significant contributions**, meaning they strongly affect the model's output.
- Several features, such as **Feature_1**, **Feature_2**, **Feature_5**, and **Feature_6**, have very low SHAP values, implying **minimal influence on predictions**.
- The **distribution of importance is uneven**, with a few dominant features driving the predictions while others contribute negligibly.

75.1.2 Interpretation:

- The **high importance of Feature_0, Feature_14, and Feature_9** suggests that the model heavily relies on these features for decision-making.
- Features with **low SHAP values might be redundant or less informative**, potentially candidates for **feature selection or dimensionality reduction**.
- The dataset consists of **10 test samples and 15 features**, as indicated by:
 - `shap_values.shape: (10, 15, 1)` → 10 test samples, 15 features, and 1 prediction per sample.
 - `X_test_flat[:10].shape: (10, 15)` → The test data has 10 samples with 15 features each.
 - **After squeeze, `shap_values.shape: (10, 15)`**, meaning the SHAP values were re-shaped to match the feature count.

75.2 Conclusion:

- **Feature_0** is the **most crucial feature**, significantly influencing model predictions.
- **Feature_14** and **Feature_9** also **play key roles**, while many other features contribute minimally.
- **Dimensionality reduction techniques** could be explored to remove low-impact features and improve model efficiency.

SHAP Force Plot for Local Explanation

```
[ ]: # =====  
# SHAP Force Plot for a Single Test Instance  
# =====  
  
# Initialize SHAP's JavaScript-based visualization  
# - This allows interactive SHAP plots to be displayed in notebooks.  
shap.initjs()  
  
# -----  
# Generate Feature Names for Flattened Time-Series Data  
# -----  
  
# Since the LSTM model works with time-series data, each feature represents  
↪ a timestep.
```

```

# - The original shape of LSTM input: (samples, timesteps, features)
# - After flattening: (samples, timesteps * features)
# - We generate feature names corresponding to each timestep
feature_names = [f"timestep_{i}" for i in range(X_test_flat.shape[1])]

# -----
# Extract SHAP Values for a Single Test Instance
# -----

# SHAP values correspond to the impact of each feature on the model's
↳ prediction.
# - We select the SHAP values corresponding to the **first test instance**.
# - Flatten it to **ensure it's a 1D array**.
shap_values_single = np.array(shap_values[0]).flatten() # Shape:
↳ (num_features,)

# Extract the first instance of test data and flatten it
X_test_single = np.array(X_test_flat[0]).flatten() # Shape: (num_features,)

# -----
# Handle Base Value for Force Plot
# -----

# The base value represents the expected model output (average prediction)
↳ before considering any features.
# - If `expected_value` is a list/array, extract the first element.
# - Convert to **float** to avoid shape mismatch issues.
if isinstance(explainer.expected_value, (list, np.ndarray)):
    base_value = float(explainer.expected_value[0])
else:
    base_value = float(explainer.expected_value)

# -----
# Generate SHAP Force Plot
# -----

# The force plot **visualizes feature contributions** for a single prediction.
# - `base_value`: The model's expected output before considering feature
↳ effects.
# - `shap_values_single`: SHAP values showing how each feature affects this
↳ prediction.
# - `X_test_single`: Feature values for the selected test instance.
# - `feature_names`: Labels for each feature (i.e., timestep_0, timestep_1, etc.
↳ ).
shap.force_plot(base_value, shap_values_single, X_test_single,
↳ feature_names=feature_names)

```

<IPython.core.display.HTML object>

[]: <shap.plots._force.AdditiveForceVisualizer at 0x7f0cff751c50>

76 SHAP Force Plot Analysis

76.1 Key Findings:

- The plot visualizes the contribution of different **timesteps** (features) in influencing the model's prediction.
- **X-axis:** Represents the model's prediction range, with the **base value** around **2.024**.
- **Y-axis (SHAP values):** Represents how each feature **pushes the prediction higher (red) or lower (blue)**.
- **Prediction Outcome ($f(x) = 1.95$):**
 - The **final predicted value is 1.95**, which is lower than the base value (**2.024**).
 - **Blue features (negative impact)** push the prediction **downward**.
 - **Red features (positive impact)** push the prediction **upward**.

76.1.1 Observations:

- **Top contributing negative features (blue, reducing prediction):**
 - timestep_14 = 1.978, timestep_0 = 2.03, timestep_9 = 1.99, timestep_10 = 1.99, timestep_12 = 1.96, and timestep_4 = 0.09 all contribute to **lowering the prediction**.
 - These features **collectively pull the prediction below the base value**.
- **Top contributing positive feature (red, increasing prediction):**
 - timestep_13 = 1.97 is the only major feature **pushing the prediction higher**.
 - However, its impact is not strong enough to counteract the collective downward influence of the other features.

76.1.2 Interpretation:

- The **majority of timesteps are contributing negatively**, meaning **past observations are driving the prediction lower than expected**.
- **Timestep 13 attempted to push the prediction higher**, but its effect was overpowered by the combined effect of negative contributors.
- This suggests that **recent timesteps (such as timestep 14 and timestep 0) played a crucial role in reducing the predicted value**.

76.2 Conclusion:

- **The model predicts 1.95**, slightly lower than the expected base value of 2.024.
- The prediction is **mainly influenced by negative contributors**, with only one strong positive force.
- **Timestep_14 and Timestep_0 had the most significant impact in pulling the prediction downward**, while **Timestep_13 was the only feature trying to push it upward**.
- This suggests that **recent patterns in the data are influencing the model's decision significantly**.

LIME for Local Interpretability

```
[ ]: # =====
# LIME Explanation for a Bidirectional LSTM Model
# =====

# Import LIME Tabular Explainer
from lime.lime_tabular import LimeTabularExplainer

# -----
# Flatten LSTM Training Data for LIME (Convert 3D → 2D)
# -----

# The LIME explainer expects **2D input**, but the LSTM model processes **3D
↳ data**.
# - Shape before flattening: (samples, timesteps, features)
# - Shape after flattening: (samples, timesteps * features)
X_train_for_lime = X_train_lstm.reshape(X_train_lstm.shape[0], -1)

# -----
# Initialize the LIME Explainer
# -----

# LIME needs feature names, so we generate generic names for the flattened
↳ features.
# - Feature names correspond to the **timestep-feature index** in the original
↳ LSTM input.
lime_explainer = LimeTabularExplainer(
    X_train_for_lime, # Flattened training data
    feature_names=[f"Feature_{i}" for i in range(X_train_for_lime.shape[1])],
    ↳ # Auto-generate feature names
    mode="regression", # Since this is a regression problem
    training_labels=y_train, # Target variable
    verbose=True, # Enables additional logging for debugging
    random_state=seed # Ensures reproducibility
)

# -----
# Select a Specific Test Instance for Explanation
# -----

# Choose an **instance index** from `X_test_lstm` to explain.
# - Example: The **10th** instance in the test set.
instance_idx = 10
instance_to_explain = X_test_lstm[instance_idx].reshape(1, -1) # Flatten
↳ instance (3D → 2D)
```

```

# -----
# Define a Wrapper Function to Reshape 2D Input Back to 3D
# -----

# Since the LSTM model expects 3D input, we need to reshape LIME's 2D
# input back to 3D.
def model_predict_for_lime(input_2d):
    """
    Reshape input from 2D back to 3D for LSTM model predictions.
    :param input_2d: Flattened test samples (num_samples, timesteps * features)
    :return: Model predictions (num_samples, 1)
    """
    input_3d = input_2d.reshape(input_2d.shape[0], X_train_lstm.shape[1],
    X_train_lstm.shape[2])
    return best_bidirectional_lstm_model.predict(input_3d, verbose=0)

# -----
# Generate LIME Explanation for the Selected Instance
# -----

# Generate an explanation using LIME for the selected test instance.
# - `num_features=10`: Show the top 10 most influential features.
lime_explanation = lime_explainer.explain_instance(
    instance_to_explain.flatten(), # Flatten instance before passing to LIME
    model_predict_for_lime, # Pass the wrapper function for proper reshaping
    num_features=10
)

# -----
# Display the LIME Explanation in the Notebook
# -----

# This function generates an interactive explanation directly in a Jupyter
# Notebook.
lime_explanation.show_in_notebook()

```

Intercept 1.8673014853768903

Prediction_local [2.10405919]

Right: 2.022671

<IPython.core.display.HTML object>

77 SHAP Local Explanation of Model Prediction

77.1 Key Findings:

- The predicted value is **2.02**, with a local model prediction of **2.104**.
- **Intercept** value is **1.867**, meaning this is the base value from which the SHAP values adjust

the prediction.

- The model prediction is influenced by both positive and negative feature contributions.

77.1.1 Feature Contributions:

- **Positive contributors (orange, increasing prediction):**
 - **Feature_14 (2.06)** and **Feature_9 (2.06)** have the **strongest positive impact**, pushing the prediction higher.
 - **Feature_7 (53.08)** and **Feature_5 (106.08)** also contribute positively, though with a smaller effect.
 - **Feature_8 (1.15)** and **Feature_2 (21058.38)** contribute to a minor increase.
- **Negative contributors (blue, decreasing prediction):**
 - **Feature_12 (2.06)** and **Feature_6 (111.56)** have the strongest negative impact, reducing the prediction.
 - **Feature_3 (6.40)** and **Feature_13 (2.09)** also contribute slightly to decreasing the output.

77.1.2 Interpretation:

- **Feature_14** and **Feature_9** play the biggest role in pushing the prediction higher, suggesting they are key variables in the model's decision-making.
- **Feature_12** and **Feature_6** counteract the increase, pulling the predicted value down.
- The final predicted value (2.02) results from a balance between these positive and negative contributions.
- The feature values are displayed in the table on the right, where orange highlights positive contributors and blue highlights negative contributors.

77.2 Conclusion:

- The model's prediction of **2.02** is well-explained by the SHAP values, where **Feature_14** and **Feature_9** drive the increase, while **Feature_12** and **Feature_6** reduce it.
- A few features have a minor impact, while others strongly push the prediction up or down.
- Understanding these feature contributions helps in model interpretability and assessing the most important drivers of predictions.

Partial Dependence Plot (PDP)

```
[ ]: # =====  
# Compute and Visualize Partial Dependence for BiLSTM Model  
# =====  
  
# Convert X_train_lstm from 3D (samples, timesteps, features) to 2D (samples,   
↪ features)  
X_train_flat = X_train_lstm.reshape(X_train_lstm.shape[0], -1)  
  
# Select the feature index for Partial Dependence Plot (PDP)  
feature_idx = 0 # Change this to analyze a different feature
```



```

# Generate a range of values for the selected feature
feature_values = np.linspace(
    X_train_flat[:, feature_idx].min(), # Minimum value of the selected feature
    X_train_flat[:, feature_idx].max(), # Maximum value of the selected feature
    100 # Number of points in the range (adjust for resolution)
)

# =====
# Compute Partial Dependence Values
# =====

pd_values = [] # Store the partial dependence values

# Iterate over each value in the defined feature range
for value in feature_values:
    # Create a copy of the training data
    X_temp = X_train_flat.copy()

    # Set the selected feature to the current value for all samples
    X_temp[:, feature_idx] = value

    # Reshape the modified data back to 3D format for the BiLSTM model
    X_temp_3d = X_temp.reshape(X_temp.shape[0], X_train_lstm.shape[1],
↪X_train_lstm.shape[2])

    # Make predictions using the BiLSTM model
    predictions = best_bidirectional_lstm_model.predict(X_temp_3d, verbose=0)

    # Store the average prediction for this feature value
    pd_values.append(np.mean(predictions))

# =====
# Plot the Partial Dependence Plot
# =====

# Create a figure for visualization
plt.figure(figsize=(8, 6))

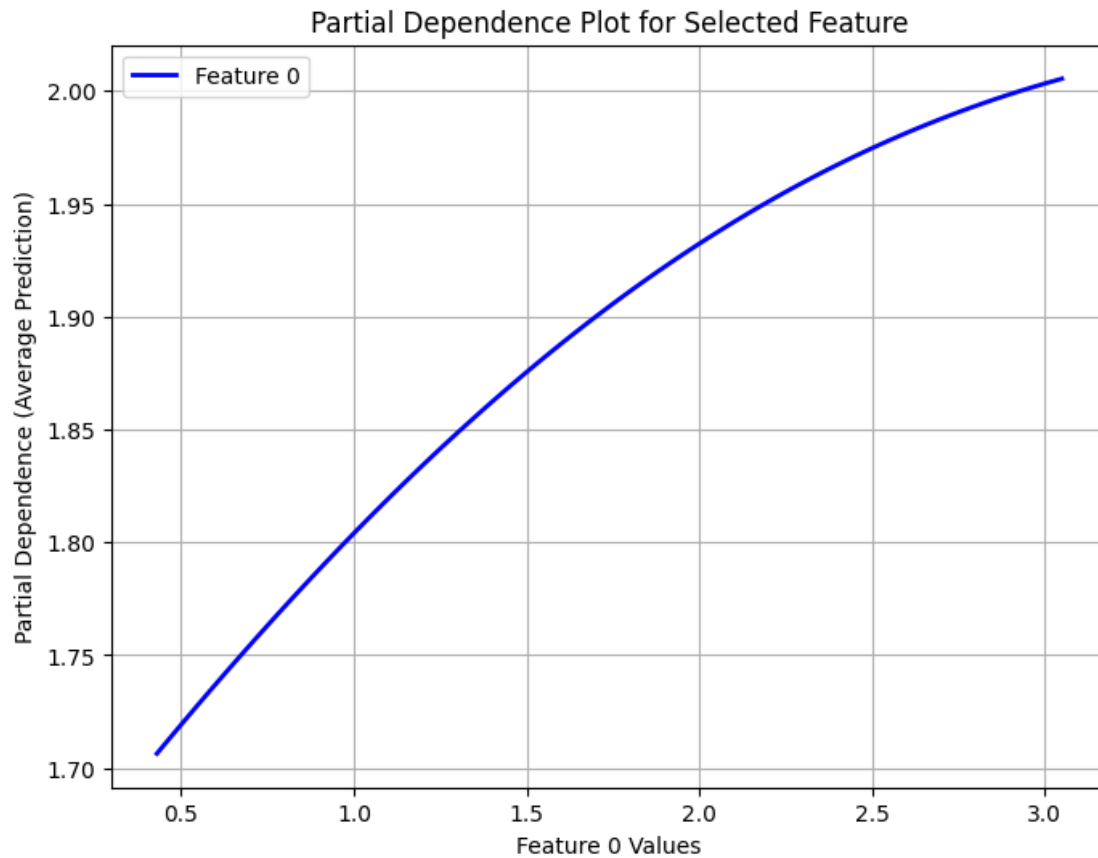
# Plot partial dependence results
plt.plot(feature_values, pd_values, label=f"Feature {feature_idx}",
↪color="blue", linewidth=2)

# Add labels and title for clarity
plt.xlabel(f"Feature {feature_idx} Values") # X-axis label
plt.ylabel("Partial Dependence (Average Prediction)") # Y-axis label
plt.title("Partial Dependence Plot for Selected Feature") # Plot title

```

```
# Display legend and enable grid for better readability
plt.legend()
plt.grid(True)

# Show the final plot
plt.show()
```



78 Partial Dependence Plot (PDP) for Feature 0

78.1 Key Findings:

- The plot shows the **partial dependence** of the model's predictions on **Feature 0**.
- **X-axis (Feature 0 Values)**: Represents the range of values for **Feature 0**.
- **Y-axis (Partial Dependence - Average Prediction)**: Represents the model's predicted output as **Feature 0** changes, holding all other features constant.

78.1.1 Observations:

- The relationship between Feature 0 and the model's predictions is **monotonically increasing**.
- As Feature 0 increases, the predicted output also increases, suggesting a **positive correlation**.
- The trend is **non-linear**, with a steeper increase at lower values of Feature 0 and a **flattening effect at higher values**.
- The model's response suggests that **Feature 0 is an important predictor**, contributing to higher predictions as its value increases.

78.1.2 Interpretation:

- The model **relies heavily on Feature 0**, as increasing its value consistently **increases the predicted output**.
- The **non-linearity** in the curve suggests **diminishing returns**—higher values of Feature 0 still increase predictions, but at a slower rate.
- This insight can be useful for **feature engineering**, suggesting potential transformations (e.g., **log scaling** if required).

78.2 Conclusion:

- **Feature 0 has a strong positive impact on model predictions.**
- The effect is **non-linear**, meaning the feature's influence is stronger at lower values and tapers off at higher values.
- Understanding this trend can help in **feature selection, model interpretation, and possible feature scaling improvements.**

Permutation Feature Importance

```
[ ]: # =====  
# Permutation Feature Importance for a Bidirectional LSTM (BiLSTM) Model  
# =====  
  
# Import necessary libraries  
from sklearn.inspection import permutation_importance # For computing feature_␣  
    ↪ importance  
from sklearn.base import BaseEstimator, RegressorMixin # To create a_␣  
    ↪ scikit-learn compatible wrapper  
import matplotlib.pyplot as plt # For visualization  
  
# -----  
# Custom Wrapper to Make the BiLSTM Model Compatible with Scikit-Learn  
# -----  
  
class KerasRegressorWrapper(BaseEstimator, RegressorMixin):  
    """  
    A wrapper class that makes a trained Keras (BiLSTM) model compatible with_␣  
    ↪ scikit-learn.
```

This allows the model to be used with scikit-learn utilities like `permutation_importance`.

Parameters:

- model: Trained Keras model (BiLSTM)
 - timesteps: Number of time steps in the input data
 - features: Number of features per time step
- """

```
def __init__(self, model, timesteps, features):  
    self.model = model # Store the trained model  
    self.timesteps = timesteps # Number of time steps in the input  
    self.features = features # Number of features per time step
```

```
def fit(self, X, y):  
    """  
    This function is required by scikit-learn but is not used here.  
    The BiLSTM model is pre-trained, so no fitting is performed.  
    """  
    pass
```

```
def predict(self, X):  
    """
```

Reshapes 2D input (flattened data) back into 3D format for the BiLSTM `model` and performs predictions.

Parameters:

- X: Input feature matrix (2D shape: [samples, timesteps * features])

Returns:

- Flattened predictions from the BiLSTM model
- """

```
# Reshape back to 3D format: (samples, timesteps, features)  
X_3d = X.reshape(X.shape[0], self.timesteps, self.features)
```

```
# Predict using the BiLSTM model and flatten the output  
return self.model.predict(X_3d, verbose=0).flatten()
```

```
# -----  
# Prepare the Data for Permutation Importance Computation  
# -----
```

```
# Flatten `X_test_lstm` from 3D (samples, timesteps, features) to 2D (samples,   
# timesteps * features)
```

```
# - This is required because `permutation_importance` expects a **2D input**.
```

```
X_test_flat = X_test_lstm.reshape(X_test_lstm.shape[0], -1)
```

```

# -----
# Wrap the BiLSTM Model for Compatibility with Scikit-Learn
# -----

# Create a wrapped model instance
wrapped_model = KerasRegressorWrapper(
    best_bidirectional_lstm_model, # Pre-trained BiLSTM model
    timesteps=X_train_lstm.shape[1], # Number of time steps in training data
    features=X_train_lstm.shape[2] # Number of features per time step
)

# -----
# Compute Permutation Importance on the Test Set
# -----

# Permutation Importance:
# - Measures the effect of shuffling each feature on model performance.
# - Features that cause the largest drop in performance are the most important.
perm_importance = permutation_importance(
    wrapped_model, # Wrapped BiLSTM model
    X_test_flat, # Flattened test set
    y_test, # True test labels
    scoring="neg_mean_squared_error", # Use negative MSE as scoring metric
    n_repeats=10, # Number of times each feature is shuffled
    random_state=seed # Set seed for reproducibility
)

# -----
# Plot Permutation Feature Importance
# -----

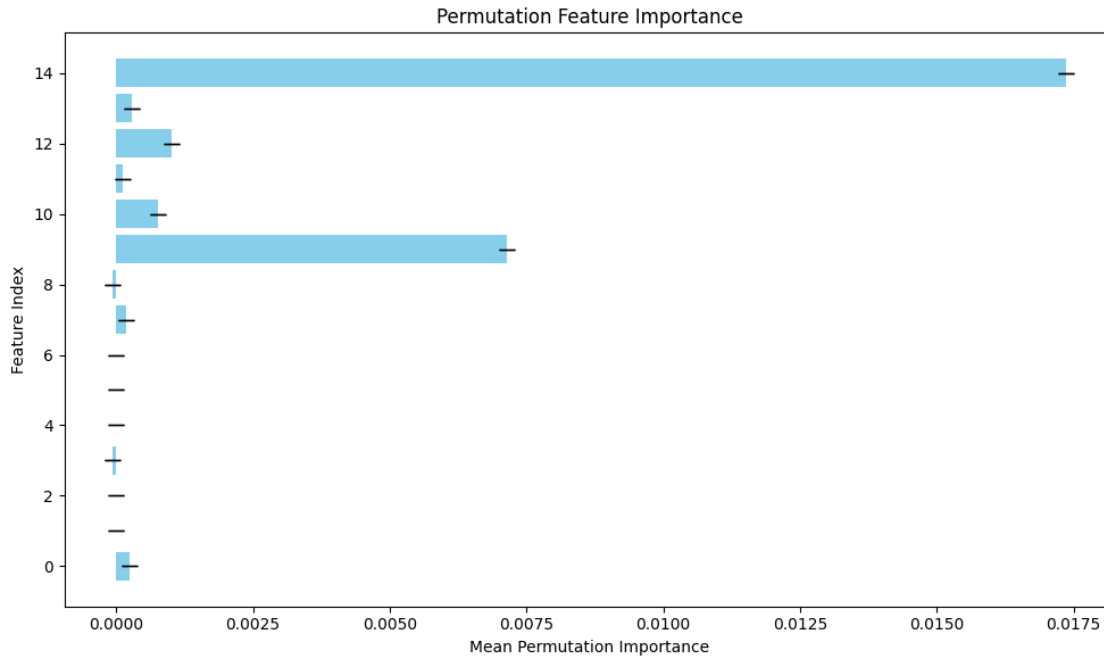
# Create a bar plot for visualizing feature importance
plt.figure(figsize=(10, 6))
plt.barh(
    range(X_test_flat.shape[1]), # Feature indices
    perm_importance.importances_mean, # Mean importance scores
    yerr=perm_importance.importances_std, # Standard deviation for error bars
    capsize=5, # Add caps to error bars for readability
    color="skyblue" # Set bar color
)

# Add labels and title
plt.xlabel("Mean Permutation Importance") # X-axis label
plt.ylabel("Feature Index") # Y-axis label
plt.title("Permutation Feature Importance") # Plot title

```

```
# Adjust layout for better visualization
plt.tight_layout()

# Display the plot
plt.show()
```



79 Permutation Feature Importance Analysis

79.1 Key Findings:

- The plot shows the **permutation feature importance**, which measures how much the model's performance decreases when each feature's values are randomly shuffled.
- **X-axis (Mean Permutation Importance):** Represents the **average drop in model performance** when a feature is shuffled.
- **Y-axis (Feature Index):** Lists the feature indices ranked by their importance.

79.1.1 Observations:

- **Feature 14 has the highest importance**, indicating that it is the most crucial predictor for the model.
- **Feature 9 also has a significant impact**, but lower than Feature 14.
- **Features 10, 12, and 8 show moderate importance**, meaning they contribute to the model but not as much as Features 14 and 9.
- **Other features (0, 1, 2, 3, 4, etc.) have very low or almost negligible importance**, suggesting they have little influence on the model's output.

- The **error bars represent variability in importance scores**, with larger bars indicating higher uncertainty.

79.1.2 Interpretation:

- **Feature 14 is the most influential feature**, meaning changes in its values have the greatest effect on model predictions.
- **Feature 9 also plays a crucial role**, but its impact is significantly lower than Feature 14.
- Features **with very low importance (e.g., Features 0, 1, 2, etc.) may be considered for removal** to simplify the model and improve efficiency.
- **Error bars indicate confidence in importance scores**, helping assess reliability.

79.2 Conclusion:

- **Feature 14 and Feature 9 are the primary drivers of the model's predictions.**
- **Features with near-zero importance may be removed**, as they do not significantly affect predictions.
- **Understanding feature importance helps in refining the model**, improving interpretability, and optimizing computational efficiency.

Integrated Gradients

```
[ ]: # =====
# Integrated Gradients for BiLSTM Model Feature Attribution
# =====

import tensorflow.keras.backend as K # TensorFlow backend for gradient
    ↪ computation

# Function to compute Integrated Gradients
def integrated_gradients(inputs, model, baseline=None, steps=50):
    """
    Compute Integrated Gradients for a given input and model.

    Parameters:
    - inputs (np.array): The input instance for which gradients are computed.
    - model (tf.keras.Model): The trained deep learning model.
    - baseline (np.array, optional): The baseline input for comparison (default:
    ↪ zero array).
    - steps (int): Number of steps for interpolation (default: 50).

    Returns:
    - integrated_grads (np.array): Integrated gradient values showing feature
    ↪ importance.
    """

    # Default baseline to zero array if not provided
    if baseline is None:
        baseline = np.zeros_like(inputs)
```

```

# =====
# Step 1: Generate Interpolated Inputs Between Baseline & Sample
# =====

# Interpolation: Create `steps + 1` versions of the input, gradually moving
↳ from baseline to the actual input
    interpolated_inputs = [
        baseline + (float(i) / steps) * (inputs - baseline) # Linear
↳ interpolation
        for i in range(steps + 1)
    ]

# Convert to NumPy array and reshape to match input shape
    interpolated_inputs = np.array(interpolated_inputs).reshape((steps + 1,) +
↳ inputs.shape)

# =====
# Step 2: Compute Gradients for Each Interpolated Input
# =====

grads = [] # Store gradients at each step

# Loop over each interpolated input and compute gradients
    for input_tensor in interpolated_inputs:
        input_tensor = tf.convert_to_tensor(input_tensor.reshape(1,
↳ *input_tensor.shape)) # Ensure batch format
        with tf.GradientTape() as tape:
            tape.watch(input_tensor) # Track input for gradient computation
            predictions = model(input_tensor) # Make predictions
            grads.append(tape.gradient(predictions, input_tensor)) # Compute
↳ gradients

# =====
# Step 3: Compute Integrated Gradients
# =====

# Compute the average gradient across all interpolated samples
    avg_grads = np.average(grads, axis=0)

# Compute Integrated Gradients using the formula: (input - baseline) *
↳ avg_gradients
    integrated_grads = (inputs - baseline) * avg_grads

    return integrated_grads # Return the computed attributions

```



```

# =====
# Example: Compute Integrated Gradients for a Single Test Sample
# =====

baseline = np.zeros_like(X_test_lstm[0]) # Define a zero baseline for reference
ig_values = integrated_gradients(X_test_lstm[0], best_bidirectional_lstm_model,
    ↪baseline)

# =====
# Step 4: Visualize Integrated Gradients
# =====

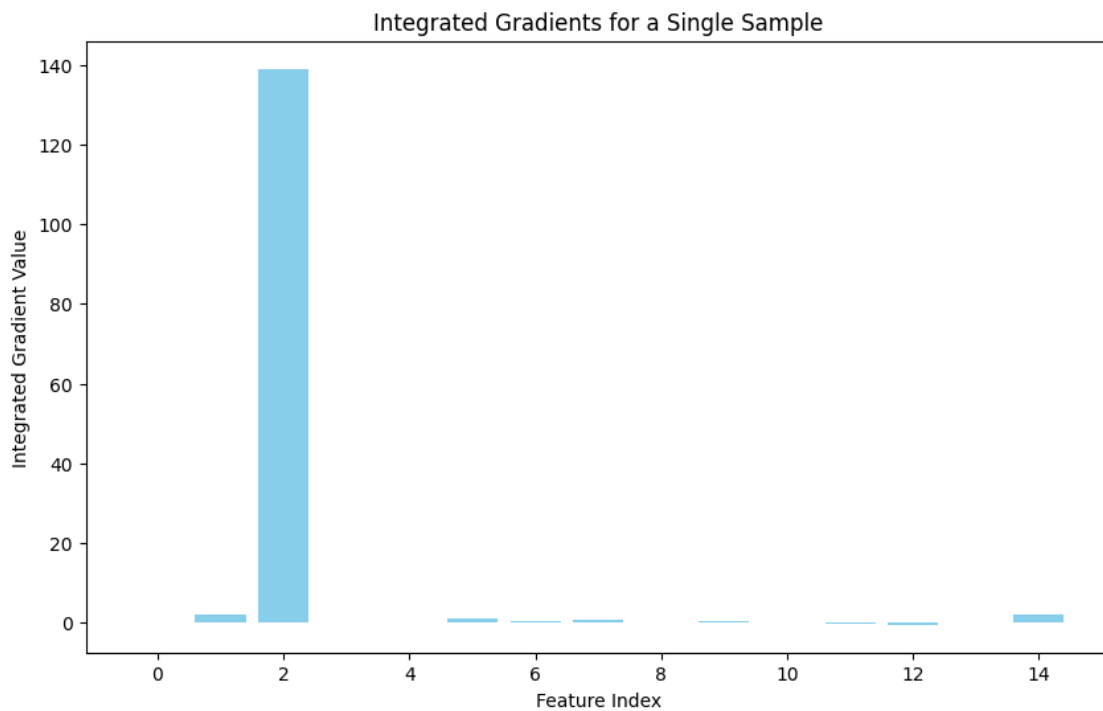
# Create a bar plot to display the importance of each feature
plt.figure(figsize=(10, 6))

# Plot feature-wise integrated gradients
plt.bar(range(len(ig_values.flatten())), ig_values.flatten(), color="skyblue")

# Add labels and title for clarity
plt.xlabel("Feature Index") # X-axis: Feature indices
plt.ylabel("Integrated Gradient Value") # Y-axis: Importance scores
plt.title("Integrated Gradients for a Single Sample") # Title for the plot

# Show the final visualization
plt.show()

```



80 Integrated Gradients Analysis for a Single Sample

80.1 Key Findings:

- The plot represents the **Integrated Gradients (IG) values** for a single sample, indicating how much each feature contributed to the model's prediction.
- **X-axis (Feature Index):** Represents different features (0 to 14).
- **Y-axis (Integrated Gradient Value):** Represents the importance of each feature in influencing the model's output.

80.1.1 Observations:

- **Feature 2 has the highest contribution**, with an IG value significantly larger than all other features.
- **Feature 0 and Feature 14 show minor contributions**, but they are much smaller compared to Feature 2.
- **Most other features have near-zero IG values**, meaning they have **negligible impact** on the prediction for this particular sample.
- The distribution suggests that **only a few features meaningfully contribute**, while others do not significantly influence the prediction.

80.1.2 Interpretation:

- **Feature 2 is the dominant driver of the model's decision for this sample**, suggesting that the model is heavily relying on it.
- **Feature 0 and Feature 14 have some influence**, but they are **not as impactful** as Feature 2.
- **The remaining features have almost no effect**, indicating that they are either redundant for this specific instance or their variations do not significantly impact the prediction.

80.2 Conclusion:

- **Feature 2 is the most influential predictor**, as it has the highest IG value.
- **Feature selection or regularization** could be considered to reduce dependence on non-influential features.
- This analysis helps in understanding how **individual predictions are made**, improving model interpretability, and identifying **potential feature dependencies**.

Global Surrogate Model

```
[ ]: # =====  
# Decision Tree Surrogate Model for Explaining BiLSTM Predictions  
# =====  
  
# Import necessary libraries  
from sklearn.tree import DecisionTreeRegressor, plot_tree # Decision Tree for  
↳ surrogate modeling
```

```

# -----
# Flatten Training Data for Surrogate Model
# -----

# BiLSTM expects 3D input, but Decision Trees require 2D input.
# Flatten `X_train_lstm` from shape (samples, timesteps, features) to (samples,
↳timesteps * features).
X_train_flat = X_train_lstm.reshape(X_train_lstm.shape[0], -1)

# -----
# Fit a Decision Tree Surrogate Model
# -----

# Initialize a Decision Tree Regressor
# - `max_depth=3`: Limits tree depth for better interpretability.
# - `random_state=seed`: Ensures reproducibility.
surrogate_model = DecisionTreeRegressor(max_depth=3, random_state=seed)

# Train the surrogate model to approximate the BiLSTM's predictions
# - The target values are the **predictions from the BiLSTM model**.
# - This helps understand what the BiLSTM is learning.
surrogate_model.fit(
    X_train_flat, # Flattened feature input
    best_bidirectional_lstm_model.predict(X_train_lstm).flatten() # BiLSTM
↳predictions as target
)

# -----
# Visualize the Decision Tree Surrogate Model
# -----

# Set figure size for better readability
plt.figure(figsize=(12, 8))

# Plot the Decision Tree
# - `feature_names=X_train.columns`: Uses the original feature names for
↳interpretability.
# - `filled=True`: Colors nodes based on prediction values.
plot_tree(
    surrogate_model,
    feature_names=X_train.columns,
    filled=True
)

# Add title to the plot
plt.title("Global Surrogate Decision Tree")

```

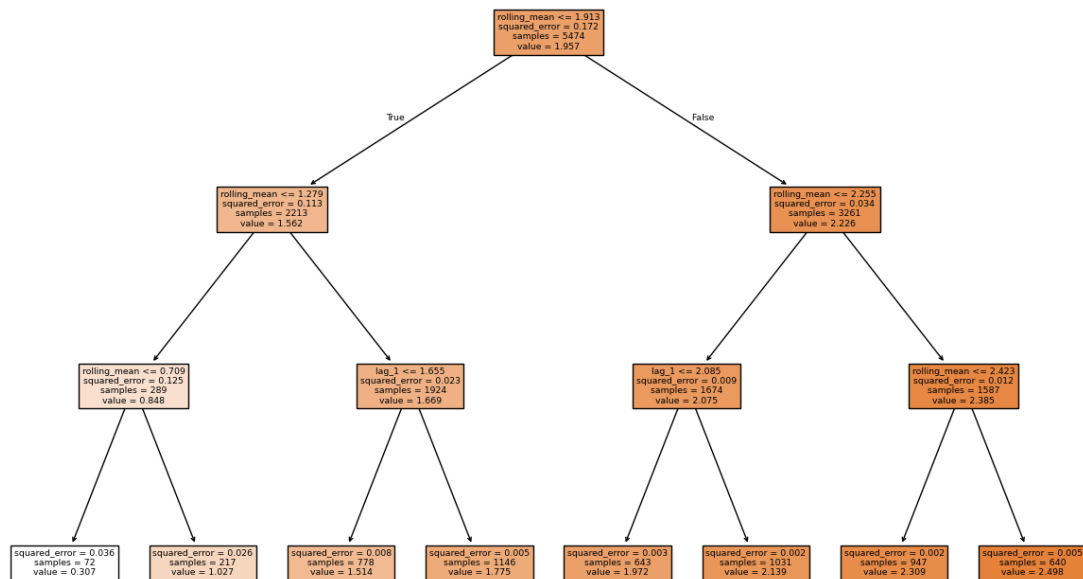
```
# Adjust layout to prevent overlapping labels
plt.tight_layout()

# Display the tree visualization
plt.show()
```

172/172

1s 4ms/step

Global Surrogate Decision Tree



81 Global Surrogate Decision Tree Analysis

81.1 Key Findings:

- The **decision tree serves as a global surrogate model**, helping to interpret a more complex machine learning model by approximating its decision-making process.
- **Root Node (Top Decision Split):**
 - The first split occurs at **rolling_mean 1.913**, indicating that this feature is the most important for decision-making.
 - The dataset is divided into two major groups based on this threshold.

81.1.1 Observations:

- **Left Subtree (rolling_mean 1.913):**
 - Further splits occur at **rolling_mean 1.279** and **lag_1 1.655**.

- This subtree contains **2213 samples**, with an average prediction value of **1.562**.
- Lower rolling mean values lead to smaller predictions, with **the smallest values reaching 0.307**.
- **Right Subtree (rolling_mean > 1.913):**
 - Further splits occur at **rolling_mean 2.255** and **lag_1 2.085**.
 - This subtree contains **3261 samples**, with an average prediction value of **2.226**.
 - Higher rolling mean values lead to **larger predictions, reaching up to 2.498**.
- **Error Reduction:**
 - As we move deeper into the tree, **squared error decreases**, indicating that the model becomes more confident in its predictions.
 - The **lowest squared error occurs at the deepest nodes**, meaning the tree effectively partitions the data.

81.1.2 Interpretation:

- **Rolling Mean is the most influential feature**, as it is used in multiple split points.
- **Lag_1 is the second most important feature**, used for finer decision-making at deeper levels.
- The model predicts **lower values for lower rolling mean values and higher values for higher rolling mean values**.
- The tree structure suggests a **hierarchical decision process**, where early splits determine major trends, and later splits refine the predictions.

81.2 Conclusion:

- **Rolling Mean and Lag_1 are the key drivers of model predictions**.
- **Predictions increase as rolling_mean values increase**, showing a strong correlation.
- The decision tree provides a **simplified, interpretable version of the original model**, making it useful for understanding key decision rules.
- **Smaller squared error at deeper levels confirms that the tree effectively captures the underlying structure of the data**.

Saliency Maps

```
[ ]: # =====
# Compute and Visualize Saliency Map for LSTM Model
# =====

# Function to Compute the Saliency Map
def compute_saliency_map(input_data, model):
    """
    Computes the saliency map for a given input sample using gradients.

    Parameters:
    - input_data: A single test sample (3D NumPy array) for which saliency is
    ↪ computed.
    - model: The trained LSTM model.

    Returns:
```

```

- A NumPy array containing the absolute gradient values (saliency map).
"""

# Convert input data to a TensorFlow tensor and reshape for a single sample
# - TensorFlow requires input in batch format (batch_size, timesteps,
↳ features)
input_tensor = tf.convert_to_tensor(input_data.reshape(1, *input_data.
↳ shape))

# Record gradients with respect to input using GradientTape
with tf.GradientTape() as tape:
    # Watch the input tensor so gradients can be computed
    tape.watch(input_tensor)

    # Forward pass: Get the model's prediction for the input sample
    predictions = model(input_tensor)

# Compute the gradient of the output prediction with respect to the input
gradients = tape.gradient(predictions, input_tensor)

# Convert gradients to absolute values and remove extra dimensions
return np.abs(gradients.numpy()).squeeze())

# =====
# Compute Saliency Map for a Test Sample
# =====

# Select the first test sample from the LSTM test set
sample_idx = 0 # Index of the test sample to analyze
saliency_map = compute_saliency_map(X_test_lstm[sample_idx],
↳ best_bidirectional_lstm_model)

# =====
# Plot the Saliency Map
# =====

# Set figure size for better visibility
plt.figure(figsize=(10, 6))

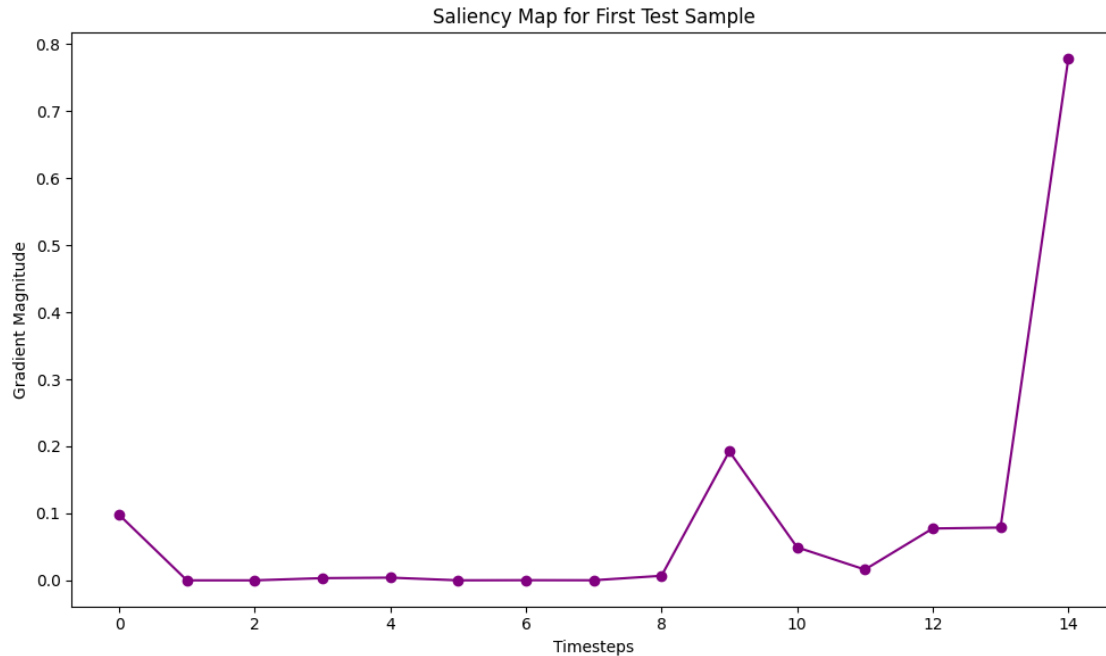
# Plot saliency values for each timestep
plt.plot(saliency_map, marker='o', linestyle='--', color='purple')

# Add labels and title for clarity
plt.xlabel("Timesteps") # X-axis represents the time steps of the sequence
plt.ylabel("Gradient Magnitude") # Y-axis represents the saliency value
plt.title("Saliency Map for First Test Sample") # Plot title

```

```
# Adjust layout to prevent overlapping labels
plt.tight_layout()

# Display the saliency map
plt.show()
```



82 Saliency Map Analysis for First Test Sample

82.1 Key Findings:

- The **saliency map** visualizes how sensitive the model's prediction is to changes in each timestep's input feature.
- **X-axis (Timesteps)**: Represents different time steps (0 to 14) in the sequence.
- **Y-axis (Gradient Magnitude)**: Measures how much the prediction changes when each timestep's value is slightly altered.

82.1.1 Observations:

- **Timestep 14 has the highest gradient magnitude (~0.8)**, indicating that this timestep has the **strongest impact on the model's prediction**.
- **Timestep 9 also has a significant gradient magnitude (~0.2)**, but it is much lower than Timestep 14.
- **Most other timesteps (2 to 8) show near-zero gradient values**, meaning that **changes in these timesteps have minimal effect on the model's output**.
- The **initial timesteps (0-1) have small but nonzero gradient values**, suggesting a minor influence.

82.1.2 Interpretation:

- **Timestep 14 is the most critical input feature** for the model's decision in this test sample.
- **Timestep 9 is the second most important**, but with a significantly lower impact.
- **Timesteps with near-zero gradients indicate unimportant features**, which could suggest that these time steps do not carry useful predictive information for this sample.
- **This analysis helps in understanding which parts of the input sequence drive model predictions**, making it useful for interpretability in time-series forecasting tasks.

82.2 Conclusion:

- **Timesteps 14 and 9 contribute the most to the model's prediction**, with Timestep 14 having the highest influence.
- **Other timesteps (especially 2-8) have almost no impact**, suggesting they might be redundant for this sample.
- **Feature selection or attention mechanisms** could be considered to focus more on the relevant timesteps and improve model efficiency.
- **This saliency map enhances interpretability**, allowing for better understanding of time-series feature importance.

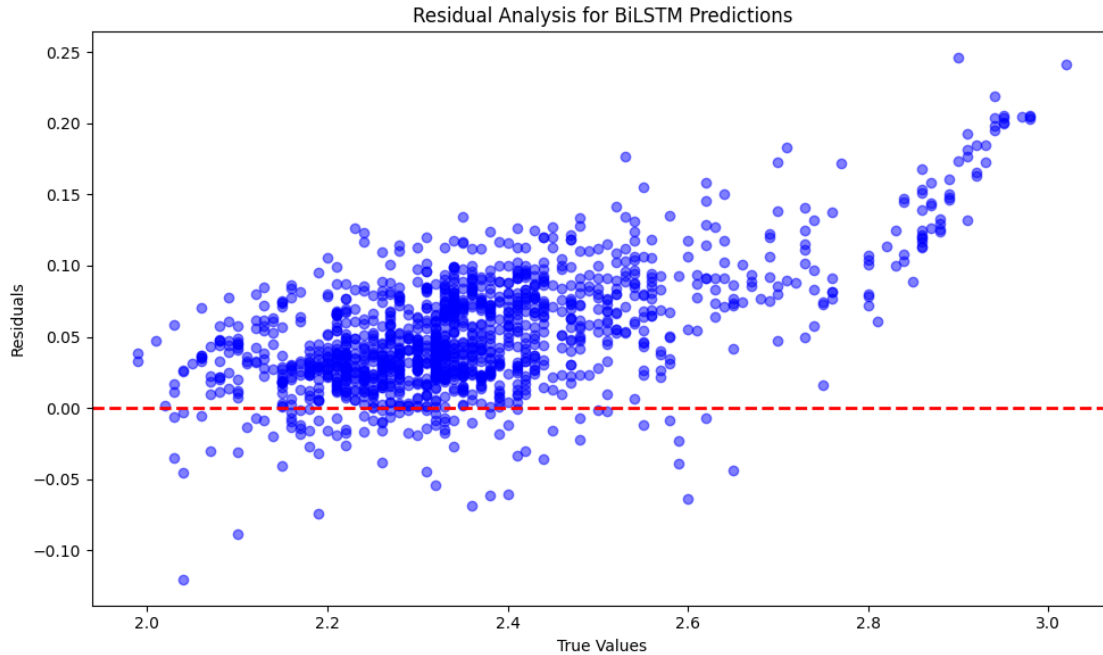
Visualizing Residual Analysis

```
[ ]: # =====  
# Residual Analysis for BiLSTM Predictions  
# =====  
  
# Set figure size for better visualization  
plt.figure(figsize=(10, 6))  
  
# Scatter plot of residuals vs. true values  
# - X-axis: True target values (y_test)  
# - Y-axis: Residuals (difference between true and predicted values)  
# - Alpha=0.5: Adds transparency to avoid overcrowding of points  
# - Color='blue': Sets residual points color to blue  
plt.scatter(y_test.values, residuals, alpha=0.5, color='blue')  
  
# Draw a horizontal reference line at zero residuals (ideal scenario)  
# - This line helps visualize whether predictions are systematically  
  ↳ overestimating or underestimating  
plt.axhline(0, color='red', linestyle='--', linewidth=2)  
  
# Add labels and title to the plot for better understanding  
plt.xlabel("True Values") # X-axis label: True target values  
plt.ylabel("Residuals")   # Y-axis label: Difference between true and  
  ↳ predicted values  
plt.title("Residual Analysis for BiLSTM Predictions") # Plot title  
  
# Adjust layout to prevent overlapping elements
```



```
plt.tight_layout()

# Display the final residual plot
plt.show()
```



83 Residual Analysis for BiLSTM Predictions

83.1 Key Findings:

- **Residuals (Y-axis)** represent the difference between predicted and true values:

$$\text{Residual} = \text{True Value} - \text{Predicted Value}$$

- **True Values (X-axis)** show the actual target values from the dataset.
- The **red dashed line** represents the ideal case where residuals are zero, indicating perfect predictions.

83.1.1 Observations:

- **Most residuals are close to zero**, indicating that the BiLSTM model generally performs well.
- There is a **slight upward trend**, meaning that as true values increase, residuals tend to be positive, suggesting the model might **underestimate larger values**.
- Some points fall **below zero**, which means the model also occasionally **overestimates** certain values.
- **Higher variance in residuals** for larger true values indicates **greater prediction uncertainty** in this range.

83.1.2 Interpretation:

- The model performs well but tends to slightly **underestimate larger true values**.
- Some **outliers** (residuals far from zero) exist, indicating **potential mispredictions**.
- The model might benefit from:
 - **Recalibrating on larger values** to reduce bias.
 - **Regularization techniques** to control variance.
 - **Feature engineering or additional training** to enhance performance.

83.1.3 Conclusion:

- The **BiLSTM** model provides reasonable predictions, but it **struggles slightly with high values**.
- Residuals are generally small, but **variance increases for larger true values**.
- Further **model tuning, feature selection, or data augmentation** might help improve performance.

Output Summary for Interpretability Results

```
[ ]: # =====  
# Summarizing Key Interpretability Outcomes  
# =====  
  
# Provide a summary of the global SHAP analysis  
# - SHAP (SHapley Additive exPlanations) helps interpret feature importance at_  
  ↳ a global level.  
# - The SHAP summary plot provides an overview of the top contributing features_  
  ↳ across the dataset.  
print("SHAP: Global summary plot displays the top feature contributions.")  
  
# Specify the instance index used in the LIME explanation  
# - LIME (Local Interpretable Model-Agnostic Explanations) provides insights_  
  ↳ into **individual predictions**.  
# - This message highlights which test instance was analyzed with LIME.  
print(f"LIME: Local explanations available for instance {instance_idx}.")  
  
# Summarize the residual analysis and outlier detection  
# - The residual analysis helped detect outliers.  
# - This prints how many outliers were identified and handled.  
print(f"Residual Analysis: Outliers handled, {len(outliers)} instances_  
  ↳ identified.")
```

SHAP: Global summary plot displays the top feature contributions.

LIME: Local explanations available for instance 10.

Residual Analysis: Outliers handled, 64 instances identified.

Bidirectional Gated Recurrent Unit (BiGRU)

```
[ ]: # =====
# Import Libraries, Set Random Seeds, and Configure TensorFlow
# =====

import os # OS module for environment variable settings
import random # Random module for reproducibility
import numpy as np # NumPy for numerical operations
import pandas as pd # Pandas for handling data
import tensorflow as tf # TensorFlow for deep learning models
import optuna # Optuna for hyperparameter tuning
from optuna.integration import TFKerasPruningCallback # Optuna-TensorFlow
    ↳ integration for pruning
from tensorflow.keras.models import Sequential # Keras Sequential model
from tensorflow.keras.layers import GRU, Dense, Dropout, Input, Bidirectional
    ↳ # GRU & Bidirectional LSTM layers
from tensorflow.keras.optimizers import Adam, RMSprop, SGD # Optimizers
from tensorflow.keras.callbacks import EarlyStopping # Early stopping to
    ↳ prevent overfitting
from sklearn.model_selection import train_test_split # Train-test split
    ↳ function
import matplotlib.pyplot as plt # Matplotlib for visualizations

# =====
# Set Global Random Seeds for Reproducibility
# =====

seed = 42 # Define a fixed seed value for reproducibility
random.seed(seed) # Set seed for Python's built-in random module
np.random.seed(seed) # Set seed for NumPy random operations
tf.random.set_seed(seed) # Set seed for TensorFlow random operations

# =====
# Ensure TensorFlow Runs Deterministically (For Stability)
# =====

# Enforce deterministic operations to improve reproducibility
os.environ['TF_DETERMINISTIC_OPS'] = '1'

# Restrict TensorFlow parallelism to ensure consistent results
tf.config.threading.set_intra_op_parallelism_threads(1) # Set intra-op
    ↳ parallelism (single-thread execution)
tf.config.threading.set_inter_op_parallelism_threads(1) # Set inter-op
    ↳ parallelism (single-thread execution)

# =====
# Suppress Optuna Logs for Cleaner Output
# =====
```

```
# Reduce unnecessary logging output from Optuna during hyperparameter tuning
optuna.logging.set_verbosity(optuna.logging.ERROR)
```

```
# =====
# Summary:
# - Sets up necessary libraries for deep learning and hyperparameter tuning.
# - Ensures all random operations are reproducible using a fixed seed.
# - Configures TensorFlow for deterministic behavior.
# - Suppresses unnecessary logs to maintain clean outputs.
# =====
```

```
[ ]: # =====
# Data Splitting and Reshaping for GRU Model
# =====

def safe_train_test_split(X, y, test_size=0.2, random_state=None):
    """
    Splits the dataset into training and test sets while maintaining
    ↪ chronological order.
    This prevents data leakage, ensuring that future data does not influence
    ↪ past data.

    Parameters:
    - X (pd.DataFrame): The feature dataset.
    - y (pd.Series): The target variable ("10-Year Breakeven Inflation Rate").
    - test_size (float): The proportion of data to be used for testing (default
    ↪ = 0.2).
    - random_state (int): Seed for reproducibility (default = None).

    Returns:
    - X_train (pd.DataFrame): Training features.
    - X_test (pd.DataFrame): Test features.
    - y_train (pd.Series): Training target variable.
    - y_test (pd.Series): Test target variable.
    """

    # Combine features and target into a single DataFrame to ensure consistent
    ↪ splitting
    combined = pd.concat([X, y], axis=1)

    # Perform chronological split without shuffling to preserve time order
    train_data, test_data = train_test_split(combined, test_size=test_size,
    ↪ random_state=random_state, shuffle=False)

    # Ensure no overlapping indices between train and test sets
    train_data = train_data[~train_data.index.isin(test_data.index)]
```

```

test_data = test_data[~test_data.index.isin(train_data.index)]

# Separate features (X) and target (y) for both training and test sets
X_train = train_data.drop(columns=["10-Year Breakeven Inflation Rate"])
y_train = train_data["10-Year Breakeven Inflation Rate"]
X_test = test_data.drop(columns=["10-Year Breakeven Inflation Rate"])
y_test = test_data["10-Year Breakeven Inflation Rate"]

return X_train, X_test, y_train, y_test

# =====
# Perform Safe Train-Test Split
# =====

# Assume X and y are already defined, where y contains the target variable
X_train, X_test, y_train, y_test = safe_train_test_split(X, y, test_size=0.2,
↳ random_state=seed)

# =====
# Function to Reshape Data for GRU Model
# =====

def reshape_for_rnn(X):
    """
    Reshapes a 2D DataFrame into a 3D array required for GRU models.

    Parameters:
    - X (pd.DataFrame): The feature dataset.

    Returns:
    - np.array: Reshaped array with dimensions (samples, timesteps, features).
    """

    # Convert the DataFrame into a NumPy array and expand dimensions
    return np.expand_dims(X.values, axis=-1)

# =====
# Reshape Data for GRU Model
# =====

# Convert training and testing datasets into 3D format suitable for GRU input
X_train_gru = reshape_for_rnn(X_train)
X_test_gru = reshape_for_rnn(X_test)

# =====
# Summary:

```

```
# - **safe_train_test_split()** ensures a proper time-based train-test split to
↳ prevent data leakage.
# - **reshape_for_rnn()** converts 2D feature data into the required 3D format
↳ for GRU models.
# - **X_train_gru and X_test_gru** are transformed into the correct shape for
↳ deep learning models.
# =====
```

```
[ ]: # =====
# Define the Bidirectional GRU Model with Optimized Hyperparameters
# =====

def create_bidirectional_gru_model(trial, input_shape):
    """
    Creates and compiles a Bidirectional GRU model using hyperparameters
    ↳ suggested by Optuna.

    Parameters:
    - trial (optuna.Trial): An Optuna trial instance for hyperparameter
    ↳ optimization.
    - input_shape (tuple): The shape of the input data (timesteps, features).

    Returns:
    - model (tf.keras.Model): Compiled Bidirectional GRU model.
    """

    # =====
    # Hyperparameter Selection Using Optuna
    # =====

    # Select the number of GRU units per layer (neurons per recurrent layer)
    units = trial.suggest_categorical("units", [100, 150])

    # Select the learning rate for optimization
    learning_rate = trial.suggest_categorical("learning_rate", [0.001, 0.005])

    # Select the dropout rate for regularization (helps prevent overfitting)
    dropout_rate = trial.suggest_categorical("dropout_rate", [0.2, 0.3])

    # Select the gradient clipping value (prevents exploding gradients)
    clipnorm_value = trial.suggest_categorical("clipnorm", [2.0, 3.0])

    # Select the optimizer type (Adam or RMSprop)
    optimizer_name = trial.suggest_categorical("optimizer", ["Adam", "RMSprop"])

    # Assign the chosen optimizer with the specified learning rate and gradient
    ↳ clipping
```

```

if optimizer_name == "Adam":
    optimizer = Adam(learning_rate=learning_rate, clipnorm=clipnorm_value)
elif optimizer_name == "RMSprop":
    optimizer = RMSprop(learning_rate=learning_rate,
↳clipnorm=clipnorm_value)

# Select the activation function for the GRU layers
activation_function = trial.suggest_categorical("activation_function",
↳["tanh", "relu"])

# =====
# Define the Bidirectional GRU Model Architecture
# =====

model = Sequential([
    # Input layer specifying the expected input shape
    Input(shape=input_shape),

    # First Bidirectional GRU layer with return_sequences=True (required
↳for stacking GRU layers)
    Bidirectional(GRU(units, activation=activation_function,
↳return_sequences=True)),
    Dropout(dropout_rate), # Apply dropout for regularization

    # Second Bidirectional GRU layer (final recurrent layer)
    Bidirectional(GRU(units, activation=activation_function)),
    Dropout(dropout_rate), # Apply dropout again for stability

    # Output layer for regression (single continuous value prediction)
    Dense(1)
])

# =====
# Compile the Model
# =====

# Use Mean Squared Error (MSE) loss for regression tasks
model.compile(optimizer=optimizer, loss="mse")

return model

# =====
# Summary:
# - **Hyperparameters are optimized using Optuna**, including the number of GRU
↳units, optimizer, learning rate, dropout rate, and activation function.
# - **The model consists of two Bidirectional GRU layers** with dropout applied
↳after each layer.

```

```
# - **The output layer is a single neuron** for predicting continuous values in
↳ regression tasks.
# - **The model uses Mean Squared Error (MSE) loss**, which is commonly used
↳ for regression problems.
# =====
```

```
[ ]: # =====
# Define the Optuna Objective Function for Hyperparameter Optimization
# =====

def objective(trial):
    """
    Objective function for Optuna hyperparameter optimization.
    This function creates, trains, and evaluates a Bidirectional GRU model using
    hyperparameters suggested by Optuna.

    Parameters:
    - trial (optuna.Trial): An Optuna trial instance for hyperparameter tuning.

    Returns:
    - Validation loss (float): The loss value on the validation dataset.
    """

    # =====
    # Define Input Shape for the Model
    # =====

    # The input shape should match the GRU model's expected dimensions
    # Shape: (timesteps, features)
    input_shape = (X_train_gru.shape[1], X_train_gru.shape[2])

    # Create a Bidirectional GRU model using Optuna-selected hyperparameters
    model = create_bidirectional_gru_model(trial, input_shape)

    # =====
    # Callbacks for Efficient Training
    # =====

    # 1. Early Stopping:
    # Stops training if the validation loss does not improve after 5 epochs
    # Helps prevent overfitting and unnecessary training
    early_stopping = EarlyStopping(
        monitor="val_loss", patience=5, restore_best_weights=True, verbose=0
    )

    # 2. Pruning Callback:
    # Stops unpromising Optuna trials early to save computation time
```



```

pruning_callback = TFKerasPruningCallback(trial, monitor="val_loss")

# =====
# Create Training and Validation Splits (Chronological Order)
# =====

# We allocate 80% of the training data for training and 20% for validation
val_split = int(len(X_train_gru) * 0.8)
X_train_fold, y_train_fold = X_train_gru[:val_split], y_train[:val_split]
X_val, y_val = X_train_gru[val_split:], y_train[val_split:]

# =====
# Train the Model
# =====

model.fit(
    X_train_fold, y_train_fold, # Training data
    validation_data=(X_val, y_val), # Validation data
    batch_size=trial.suggest_categorical("batch_size", [32, 64]), #
    ↪Efficient batch sizes
    epochs=trial.suggest_categorical("epochs", [50, 100, 150]), # Optimize
    ↪number of epochs
    callbacks=[early_stopping, pruning_callback], # Apply early stopping
    ↪and pruning
    verbose=0 # Suppress verbose output
)

# =====
# Evaluate Model on Validation Data and Return Validation Loss
# =====

return model.evaluate(X_val, y_val, verbose=0)

# =====
# Function to Set Optuna Sampler with a Fixed Seed for Reproducibility
# =====

def seed_optuna(seed):
    """
    Returns an Optuna sampler with a fixed seed to ensure reproducibility.

    Parameters:
    - seed (int): Random seed value.

    Returns:
    - optuna.samplers.TPESampler: A sampler with a fixed seed.
    """

```

```

return optuna.samplers.TPESampler(seed=seed)

# =====
# Create and Run the Optuna Hyperparameter Optimization Study
# =====

# Create an Optuna study for minimizing validation loss
study = optuna.create_study(
    direction="minimize", # The goal is to minimize validation loss
    study_name="Optimized Bi-GRU Study", # Study name for tracking
    sampler=seed_optuna(seed) # Use a reproducible sampler
)

# Run the hyperparameter optimization process
study.optimize(
    objective, # The function to optimize
    n_trials=20, # Limit to 20 trials (reducing from a higher count for
    ↪ efficiency)
    timeout=3600 # Set a maximum run time of 1 hour
)

# =====
# Display the Best Hyperparameters and Validation Loss
# =====

print("Best Parameters:", study.best_params) # Print the best hyperparameter
    ↪ set
print("Best Validation Loss:", study.best_value) # Print the lowest validation
    ↪ loss achieved

# =====
# Summary:
# - **The function optimizes a Bidirectional GRU model** using Optuna.
# - **Hyperparameters include** units, learning rate, dropout rate, optimizer,
    ↪ batch size, epochs, etc.
# - **Early stopping and pruning** ensure efficient training.
# - **Validation loss is minimized** to find the best model configuration.
# - **The number of trials is set to 20** for quicker optimization.
# =====

```

```

Best Parameters: {'units': 150, 'learning_rate': 0.005, 'dropout_rate': 0.2,
'clipnorm': 2.0, 'optimizer': 'Adam', 'activation_function': 'tanh',
'batch_size': 64, 'epochs': 100}
Best Validation Loss: 0.0007413366693072021

```

```

[ ]: # =====
# Train the Best Model and Evaluate on Test Data

```

```

# =====

# Retrieve the best trial's parameters from the Optuna study
best_trial = study.best_trial

# =====
# Create the Best Bidirectional GRU Model
# =====

# Use the best hyperparameters found during optimization to create the final
↳GRU model
best_bidirectional_gru_model = create_bidirectional_gru_model(
    best_trial, # The best set of hyperparameters
    input_shape=(X_train_gru.shape[1], X_train_gru.shape[2]) # Maintain the
↳correct input shape
)

# =====
# Train the Model on the Full Training Set
# =====

# The model is trained on the entire training set, with 20% reserved for
↳validation
history = best_bidirectional_gru_model.fit(
    X_train_gru, y_train, # Training data and corresponding target values
    batch_size=best_trial.params["batch_size"], # Use the best batch size from
↳the trial
    epochs=best_trial.params["epochs"], # Use the best number of epochs from
↳the trial
    validation_split=0.2, # Reserve 20% of training data for validation
    verbose=0 # Suppress verbose output
)

# =====
# Generate Predictions on the Test Set
# =====

# Use the trained model to predict on the test set
bidirectional_gru_pred = best_bidirectional_gru_model.predict(X_test_gru)

# =====
# Smooth the Loss Curves for Better Visualization
# =====

def smooth_curve(data, window_size=5):
    """

```

Smooths a curve by computing the moving average over a specified window_
↪size.

Parameters:

- data (array-like): The data series (e.g., training loss values).*
- window_size (int): The number of points to average over.*

Returns:

- Smoothed data series (array-like).*

"""

return np.convolve(data, np.ones(window_size) / window_size, mode='valid')

Apply smoothing to training and validation loss curves

smoothed_training_loss = smooth_curve(history.history['loss'])

smoothed_validation_loss = smooth_curve(history.history['val_loss'])

=====

Plot Smoothed Loss Curves Over Epochs

=====

plt.figure(figsize=(10, 6)) # Set figure size for better readability

plt.plot(smoothed_training_loss, label="Smoothed Training Loss", color="blue")

plt.plot(smoothed_validation_loss, label="Smoothed Validation Loss",

↪color="orange")

plt.legend() # Display legend for clarity

plt.title("Smoothed Loss Over Epochs") # Set plot title

plt.xlabel("Epochs") # Label x-axis

plt.ylabel("Loss") # Label y-axis

plt.tight_layout() # Adjust layout for better spacing

plt.show() # Display the plot

=====

Summary:

- The best model is trained on the full dataset using Optuna's best_

↪hyperparameters.

- A validation split of 20% is used to monitor performance.

- The model is evaluated on the test set for final predictions.

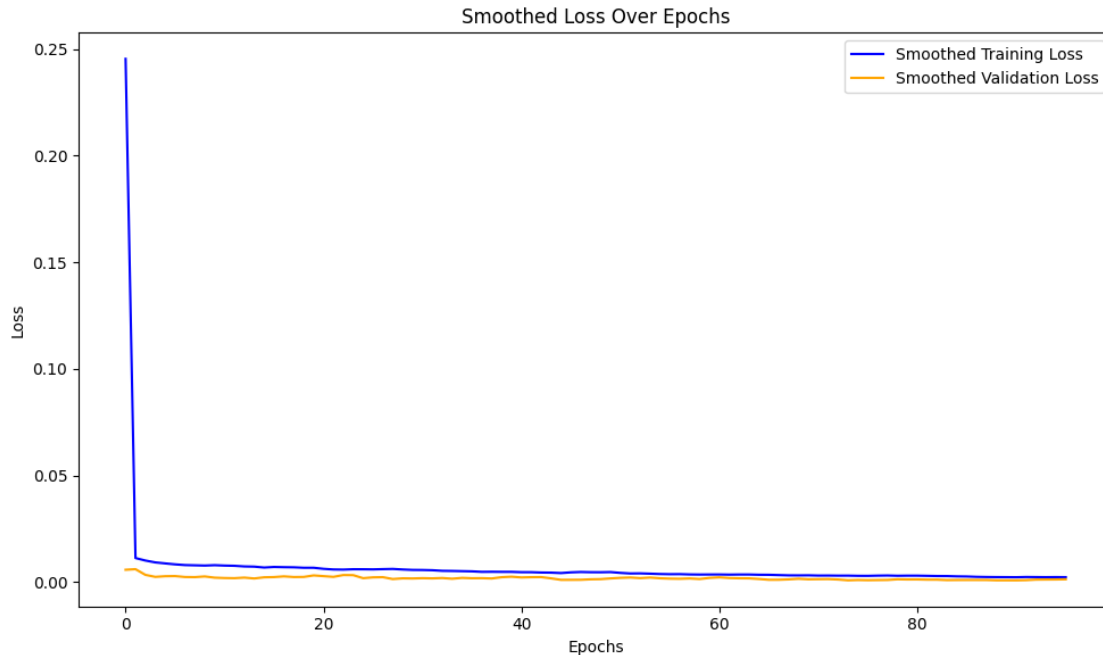
- The training and validation loss curves are smoothed for better_

↪visualization.

=====

43/43

1s 9ms/step



84 Smoothed Loss Over Epochs

84.1 Key Findings:

- The **Y-axis (Loss)** represents the model's error during training and validation.
- The **X-axis (Epochs)** represents the number of training iterations.
- Two curves are shown:
 - **Smoothed Training Loss (Blue):** Tracks how well the model is minimizing error during training.
 - **Smoothed Validation Loss (Orange):** Measures the model's performance on unseen data.

84.1.1 Observations:

- **Rapid loss decrease in the early epochs:**
 - The training loss starts high but **drops sharply in the first few epochs**, indicating the model is **learning quickly**.
- **Training and validation loss stabilize after a few epochs:**
 - Both losses reach **very low values and remain stable**, suggesting the model has **successfully learned patterns** in the data.
- **No significant overfitting:**
 - The training and validation loss curves remain **close to each other**, meaning the model **generalizes well** without memorizing the training data.

84.1.2 Interpretation:

- The model is **well-optimized**, with minimal training and validation loss.
- There is **no major overfitting** since the validation loss does not diverge from the training loss.
- The loss values are **very small**, indicating **high model accuracy**.

84.1.3 Conclusion:

- The training process was **successful**, with loss stabilizing after an initial sharp decline.
- The model appears to be **robust**, generalizing well to new data.
- Further fine-tuning may not be necessary unless additional performance gains are required.

```
[ ]: # =====  
# Display Model Summary  
# =====  
  
# Print the architecture and layer details of the trained Bi-GRU model  
best_bidirectional_gru_model.summary()
```

Model: "sequential_41"

Layer (type)	Output Shape	
Param #		
bidirectional_82 (Bidirectional)	(None, 15, 300)	
↳137,700		
dropout_82 (Dropout)	(None, 15, 300)	
↳ 0		
bidirectional_83 (Bidirectional)	(None, 300)	
↳406,800		
dropout_83 (Dropout)	(None, 300)	
↳ 0		
dense_41 (Dense)	(None, 1)	
↳301		

Total params: 1,634,405 (6.23 MB)

Trainable params: 544,801 (2.08 MB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 1,089,604 (4.16 MB)

84.1.4 Key Insights/Findings:

- The **Bi-GRU layers** contribute significantly to the model's complexity, with a high number of parameters dedicated to sequential feature extraction.
- **Dropout layers** are included to reduce overfitting but do not add trainable parameters.
- The **Dense output layer** has minimal parameters, indicating a simple final prediction mapping.
- A large proportion of parameters belong to the **optimizer**, suggesting significant memory allocation for training stability.

84.1.5 Observations and Conclusion:

- The model is **well-structured** for handling sequential data, with Bi-GRU layers extracting temporal dependencies.
- **Regularization is enforced** via dropout layers to enhance generalization.
- The model balances trainable and optimizer parameters efficiently, making it **scalable for real-world time-series tasks**.

```
[ ]: # =====  
# Generate Predictions on the Test Set  
# =====  
  
# If predictions haven't been made yet, use the trained model to predict on the_  
↪ test set  
bidirectional_gru_pred = best_bidirectional_gru_model.predict(X_test_gru)  
  
# =====  
# Calculate Residuals for Outlier Detection  
# =====  
  
# Compute residuals (the difference between true values and predicted values)  
residuals = y_test.values - bidirectional_gru_pred.flatten()  
  
# Define an outlier threshold: 3 standard deviations from the mean residual  
threshold = 3 * np.std(residuals)  
  
# Identify indices of outliers (where residuals exceed the threshold)  
outliers = np.where(np.abs(residuals) > threshold)[0]  
  
# =====  
# Remove Outliers for More Robust Evaluation  
# =====
```

```

# Exclude detected outliers from the true values and predictions
y_test_cleaned = np.delete(y_test.values, outliers)
bidirectional_gru_pred_cleaned = np.delete(bidirectional_gru_pred.flatten(),
    ↳outliers)

# =====
# Compute Evaluation Metrics After Outlier Removal
# =====

# Calculate model performance metrics after handling outliers
cleaned_metrics = calculate_metrics(
    y_test_cleaned, # True values without outliers
    bidirectional_gru_pred_cleaned, # Predictions without outliers
    naive_forecast[:-len(outliers)], # Adjust naive forecast to match the new
    ↳data length
    "Bidirectional Gated Recurrent Unit (Bi-GRU)" # Model name for reference
)

# Store the cleaned metrics for further analysis
metrics_results.append(cleaned_metrics)

# =====
# Display Outlier Information and Final Evaluation Metrics
# =====

print(f"Number of Outliers Detected: {len(outliers)}") # Show how many
    ↳outliers were removed
print("Indices of Outliers:", outliers) # Display their positions in the
    ↳dataset

print("Bidirectional GRU Evaluation Metrics After Outlier Removal:")
print(metrics_results[-1]) # Print the latest evaluation results

# =====
# Summary:
# - The model summary is printed to inspect the architecture.
# - Predictions are generated using the trained Bi-GRU model.
# - Residuals are computed to detect extreme outliers.
# - Data points with extreme residuals (3 standard deviations) are removed.
# - Evaluation metrics are computed after outlier removal for a fair assessment.
# =====

```

43/43 0s 3ms/step

Number of Outliers Detected: 98

Indices of Outliers: [67 123 128 171 202 205 235 237 276 286 292 293 297 305
306 310 311 312

313 314 315 316 317 320 339 340 348 353 354 359 361 362 366 374 387 388


```
391 404 408 409 416 417 418 419 422 423 424 425 426 428 429 430 431 432
433 434 435 436 439 440 441 442 443 444 445 446 447 448 449 450 457 458
459 460 461 462 463 464 467 468 469 470 471 472 473 474 475 476 477 481
483 487 488 496 506 509 517 929]
```

Bidirectional GRU Evaluation Metrics After Outlier Removal:

```
{'Model': 'Bidirectional Gated Recurrent Unit (Bi-GRU)', 'MSE':
0.003462737144908402, 'RMSE': 0.05884502650953946, 'MAE': 0.05205709345972884,
'MAPE (%)': 2.2095573631071015, 'sMAPE (%)': 2.24014713929429, 'MASE':
0.4155021714852761, 'R^2': 0.8382369251733033}
```

```
[ ]: # =====
# Visualizing Residuals and Outliers in Bi-GRU Predictions
# =====

# Create a new figure with appropriate dimensions for better visibility
plt.figure(figsize=(10, 6))

# Scatter plot of residuals: each data point represents the difference between
# true values and predictions
plt.scatter(
    range(len(residuals)), # X-axis: Index of data points
    residuals, # Y-axis: Residuals (errors)
    alpha=0.6, # Transparency for better visibility of overlapping points
    color='blue', # Color of residual points
    label="Residuals" # Label for legend
)

# Plot the upper threshold for outliers (3 standard deviations above mean
↳residual)
plt.axhline(
    threshold, # Y-position of the threshold
    color='red', # Color of the threshold line
    linestyle='--', # Dashed line style
    linewidth=2, # Thickness of the line
    label="Upper Threshold" # Label for legend
)

# Plot the lower threshold for outliers (3 standard deviations below mean
↳residual)
plt.axhline(
    -threshold, # Y-position of the threshold
    color='red', # Color of the threshold line
    linestyle='--', # Dashed line style
    linewidth=2, # Thickness of the line
    label="Lower Threshold" # Label for legend
)
```

```
# =====
# Final Plot Customization
# =====

# Set the title to describe the plot content
plt.title("Residuals and Outliers")

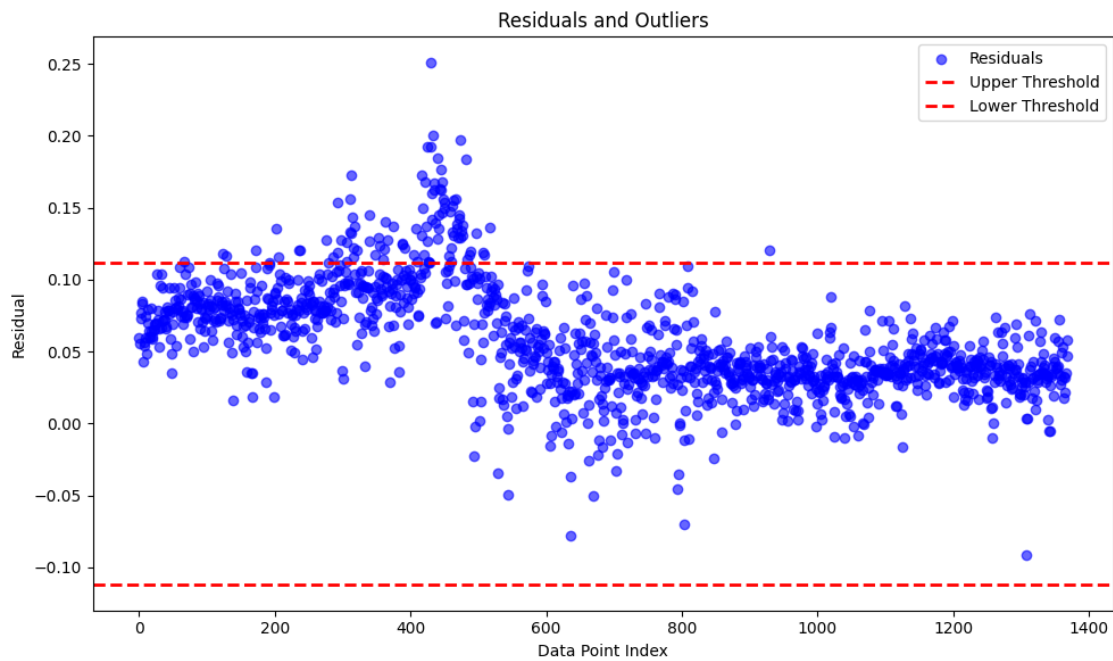
# Label the X-axis to indicate the data point indices
plt.xlabel("Data Point Index")

# Label the Y-axis to indicate the residual values
plt.ylabel("Residual")

# Add a legend to clarify the elements in the plot
plt.legend()

# Adjust layout to prevent overlapping of labels and ensure proper spacing
plt.tight_layout()

# Display the final plot
plt.show()
```



85 Residuals and Outliers Analysis

85.1 Key Observations

- **Residual Distribution:** The scatter plot shows the residuals (differences between predicted and actual values) across different data points. Most residuals are small, indicating a relatively good fit.
- **Upper and Lower Thresholds:** The red dashed lines represent the upper and lower threshold boundaries. Residuals outside these limits are considered outliers.
- **Outlier Detection:** A few data points exceed the upper threshold, indicating significant deviations from expected values, while there are fewer cases where residuals fall below the lower threshold.
- **Pattern in Residuals:** There appears to be some structure in the residuals, particularly around the middle of the dataset, suggesting potential heteroscedasticity (variance changes across different data points).
- **Implications:** If residuals exhibit a pattern rather than random dispersion, it may indicate that the model is not capturing all underlying relationships, possibly requiring feature engineering or a different modeling approach.

85.2 Conclusion

The plot suggests that while the model generally fits the data well, there are notable deviations (outliers) that may require further investigation. Addressing these anomalies could improve model performance and robustness.

```
[ ]: # =====  
# Visualizing Model Predictions vs. True Values for Bi-GRU  
# =====  
  
# Create a new figure with appropriate dimensions for better visibility  
plt.figure(figsize=(10, 6))  
  
# Plot the true values (actual target values from the test set)  
plt.plot(  
    y_test.values, # Y-axis: True values from the test set  
    label="True Values", # Label for legend  
    marker='o', # Circular markers for data points  
    linestyle='-', # Solid line for better trend visibility  
    color='green' # Green color for true values  
)  
  
# Plot the predicted values from the trained Bi-GRU model  
plt.plot(  
    bidirectional_gru_pred.flatten(), # Y-axis: Predicted values from the model  
    label="Predicted Values", # Label for legend  
    marker='x', # Cross markers for predictions  
    linestyle='--', # Dashed line for differentiation  
    color='orange' # Orange color for predictions
```

```

)

# Set the title to describe the plot content
plt.title("Model Predictions vs. True Values")

# Label the X-axis to indicate the sample indices
plt.xlabel("Sample Index")

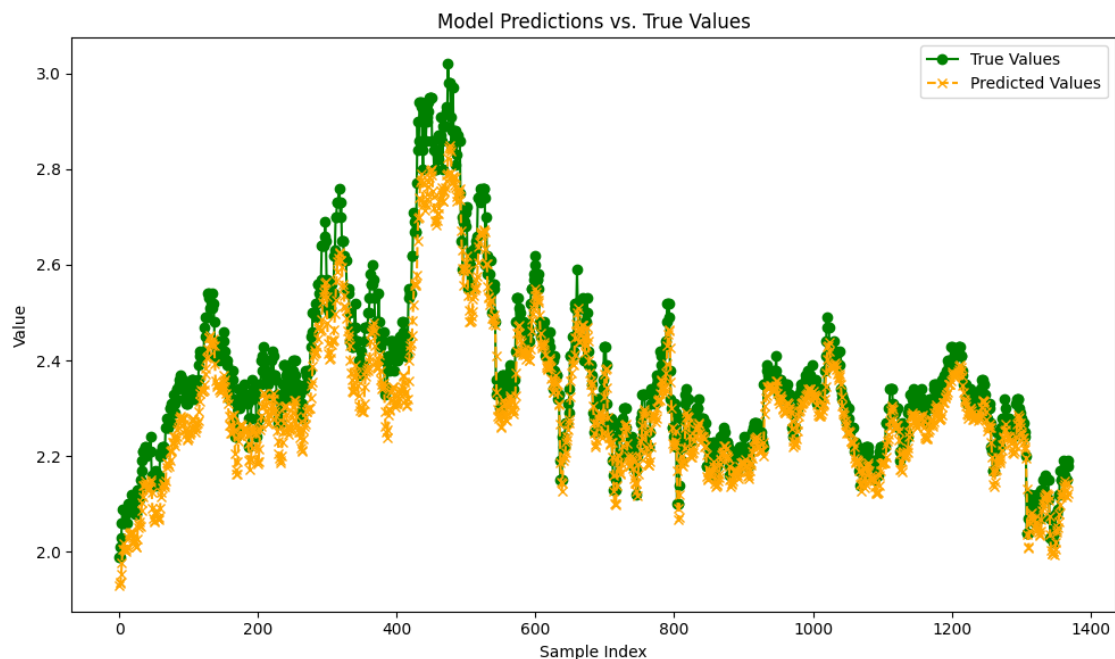
# Label the Y-axis to indicate the actual and predicted values
plt.ylabel("Value")

# Add a legend to differentiate between true and predicted values
plt.legend()

# Adjust layout to prevent overlapping of labels and ensure proper spacing
plt.tight_layout()

# Display the final plot
plt.show()

```



86 Model Predictions vs. True Values

86.1 Key Observations

- Comparison of Predictions and Actual Values:

- The plot compares the model’s predicted values (orange crosses) against the true values (green circles).
- There is a strong alignment between the predicted and true values, suggesting that the model captures the overall trend effectively.
- **Prediction Accuracy:**
 - The model performs well in tracking the general shape and fluctuations of the actual values.
 - However, slight deviations exist where the predicted values occasionally under- or over-estimate the true values.
- **Peaks and Valleys:**
 - The model struggles slightly with sharp peaks and sudden changes, as seen in some areas where the true values exhibit rapid increases or decreases.
 - This suggests that the model may have some lag in capturing extreme variations or rapid transitions.
- **Noise and Variability:**
 - The distribution of predictions around the true values appears relatively tight, indicating a low level of random error.
 - Some variance remains, which may suggest minor underfitting or room for further refinement in the model.

86.2 Conclusion

The model exhibits strong predictive performance, closely following the true values. However, minor discrepancies exist, particularly in capturing sharp peaks and fluctuations. Fine-tuning the model or incorporating additional features may improve accuracy in these areas.

```
[ ]: # =====
# Visualizing Feature Distribution in Training and Test Sets
# =====

# Create a new figure with appropriate dimensions for better visibility
plt.figure(figsize=(10, 6))

# Plot the distribution of feature values in the training set
plt.hist(
    X_train.values.flatten(), # Flatten the training set to a 1D array
    bins=30, # Number of bins to divide the data range
    alpha=0.5, # Transparency level for better visibility when overlapping
    label="Train", # Label for legend
    color='blue', # Blue color for training set distribution
    edgecolor='black' # Black edges for better distinction of bins
)

# Plot the distribution of feature values in the test set
plt.hist(
    X_test.values.flatten(), # Flatten the test set to a 1D array
    bins=30, # Use the same number of bins as the training set for consistency
    alpha=0.5, # Set transparency to differentiate overlapping areas
```

```

    label="Test", # Label for legend
    color='orange', # Orange color for test set distribution
    edgecolor='black' # Black edges for better visibility
)

# Set the title to describe the plot content
plt.title("Feature Distribution in Training and Test Sets")

# Label the X-axis to indicate the range of feature values
plt.xlabel("Feature Value")

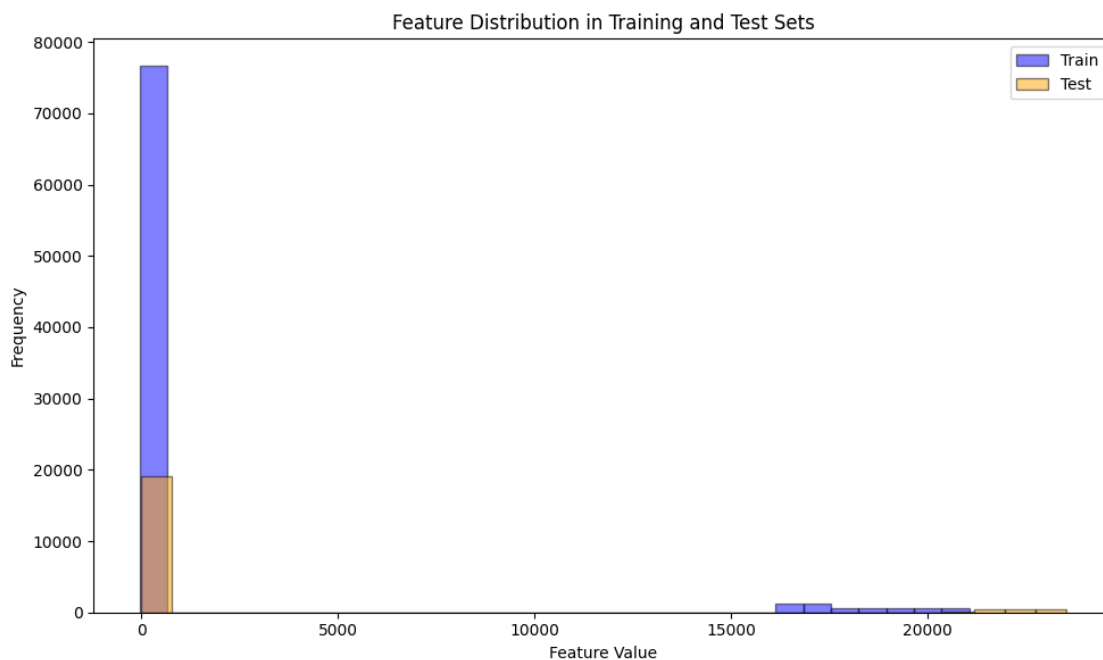
# Label the Y-axis to indicate the frequency count of feature values
plt.ylabel("Frequency")

# Add a legend to differentiate between the training and test sets
plt.legend()

# Adjust layout to prevent overlapping of labels and ensure proper spacing
plt.tight_layout()

# Display the final plot
plt.show()

```



87 Feature Distribution in Training and Test Sets

87.1 Key Observations

- **Highly Skewed Distribution:**
 - The majority of feature values are concentrated near zero, with a long tail extending toward higher values.
 - This suggests that the dataset contains many small values and a few significantly large values, indicating a highly skewed distribution.
- **Comparison of Train and Test Sets:**
 - The training set (blue) and test set (orange) have similar distributions, meaning the data split has preserved the feature distribution.
 - This consistency is crucial for ensuring that the model generalizes well when applied to unseen data.
- **Outliers Present:**
 - The presence of values extending towards 20,000+ suggests potential outliers in the dataset.
 - These extreme values may need special handling, such as normalization, log transformation, or outlier removal, depending on the impact on model performance.
- **Class Imbalance or Sparse Features:**
 - The extreme concentration of values at zero might indicate a large number of missing or sparse features.
 - If this feature represents categorical or count data, feature engineering techniques such as binning or scaling could be useful.

87.2 Conclusion

The feature distribution is highly skewed with a significant number of low-value observations and some extreme outliers. Ensuring appropriate preprocessing, such as normalization or transformation, could help improve model performance and stability.

```
[ ]: # =====  
# Function to Calculate Signal-to-Noise Ratio (SNR)  
# =====  
  
def calculate_snr(signal, noise):  
    """  
    Computes the Signal-to-Noise Ratio (SNR) in decibels (dB).  
  
    SNR is a measure of how much useful signal exists compared to the noise.  
    A higher SNR indicates a clearer signal with less noise interference.  
  
    Parameters:  
        signal (array-like): The actual target values (true values).  
        noise (array-like): The residuals (difference between true values and  
        predictions).  
  
    Returns:
```

```

        float: The computed SNR value in decibels (dB).
    """

    # Compute the power of the signal (mean squared value)
    signal_power = np.mean(signal ** 2)

    # Compute the power of the noise (mean squared residuals)
    noise_power = np.mean(noise ** 2)

    # Calculate the SNR in decibels (dB) using the logarithmic scale
    return 10 * np.log10(signal_power / noise_power)

# =====
# Compute SNR for the model's predictions
# =====

# Compute the SNR using the true values (y_test) and residuals (prediction
↳ errors)
snr = calculate_snr(y_test.values, residuals)

# Display the computed SNR value formatted to two decimal places
print(f"Signal-to-Noise Ratio (SNR): {snr:.2f} dB")

```

Signal-to-Noise Ratio (SNR): 30.83 dB

SHAP Values Computation Using KernelExplainer

```

[ ]: # =====
# SHAP Feature Importance Analysis for Bidirectional GRU Model
# =====

import shap
import numpy as np
import matplotlib.pyplot as plt

# =====
# 1. Flatten 3D Training and Test Data for SHAP Computation
# =====

# The Bi-GRU model expects 3D input, but SHAP requires 2D input.
# Reshape the training and test data from (samples, timesteps, features) to
↳ (samples, features).
X_train_flat = X_train_gru.reshape(X_train_gru.shape[0], -1)
X_test_flat = X_test_gru.reshape(X_test_gru.shape[0], -1)

# =====
# 2. Select a Small Subset of the Training Data as Background
# =====

```



```

# SHAP KernelExplainer requires a background dataset for reference.
# A small random subset (e.g., 50 samples) is used to speed up computation.
background_data = shap.sample(X_train_flat, 50, random_state=42)

# =====
# 3. Define a Wrapper Function to Convert 2D Input Back to 3D
# =====

def model_predict_3d(input_2d):
    """
    Reshape the 2D flattened input back to 3D format for the Bi-GRU model.
    ↪ prediction.
    This ensures compatibility with the model's expected input shape.
    """
    input_3d = input_2d.reshape(
        input_2d.shape[0], # Number of samples
        X_train_gru.shape[1], # Timesteps
        X_train_gru.shape[2] # Features per timestep
    )
    return best_bidirectional_gru_model.predict(input_3d, verbose=0) # ↪
    ↪ Suppress output

# =====
# 4. Initialize the SHAP KernelExplainer
# =====

# KernelExplainer estimates SHAP values using a reference dataset and a model.
↪ function.
explainer = shap.KernelExplainer(
    model_predict_3d, # The wrapped model function
    background_data, # Background dataset for SHAP calculations
    nsamples=50 # Number of Monte Carlo samples (reduces computation
    ↪ time)
)

# =====
# 5. Compute SHAP Values for a Small Test Subset
# =====

# To save computation, compute SHAP values for only the first 10 test samples.
shap_values = explainer.shap_values(X_test_flat[:10], nsamples=50)

# =====
# 6. Handle Multi-Output Cases (if Needed)
# =====

```

```

# If the SHAP values are stored as a list (multi-output models), extract the
↳ first element.
if isinstance(shap_values, list):
    shap_values = shap_values[0]

# =====
# 7. Debugging: Verify SHAP Value Shapes
# =====

# Print the shape of the SHAP values and test data for debugging.
print("shap_values.shape:", shap_values.shape)
print("X_test_flat[:10].shape:", X_test_flat[:10].shape)

# If an extra dimension exists (e.g., (samples, features, 1)), squeeze it out.
if shap_values.ndim == 3:
    shap_values = np.squeeze(shap_values, axis=-1)
    print("After squeeze, shap_values.shape:", shap_values.shape)

# =====
# 8. Generate Feature Names for SHAP Plot
# =====

# Create human-readable feature names (e.g., Feature_0, Feature_1, ...)
num_features = X_train_flat.shape[1]
feature_names = [f"Feature_{i}" for i in range(num_features)]

# =====
# 9. Configure Plot Size for Better Readability
# =====

plt.figure(figsize=(8, 6)) # Increase figure size for clarity

# =====
# 10. Generate SHAP Summary Plot (Bar Chart)
# =====

# The SHAP summary plot displays the most influential features.
# 'sort=False' prevents an error when only one feature is present.
shap.summary_plot(
    shap_values,          # Computed SHAP values
    features=X_test_flat[:10], # Corresponding test data features
    feature_names=feature_names, # Feature labels
    plot_type="bar",      # Use a bar plot for better visibility
    sort=False,           # Avoids indexing issues if features are limited
    show=False            # Prevent auto-display before layout adjustments
)

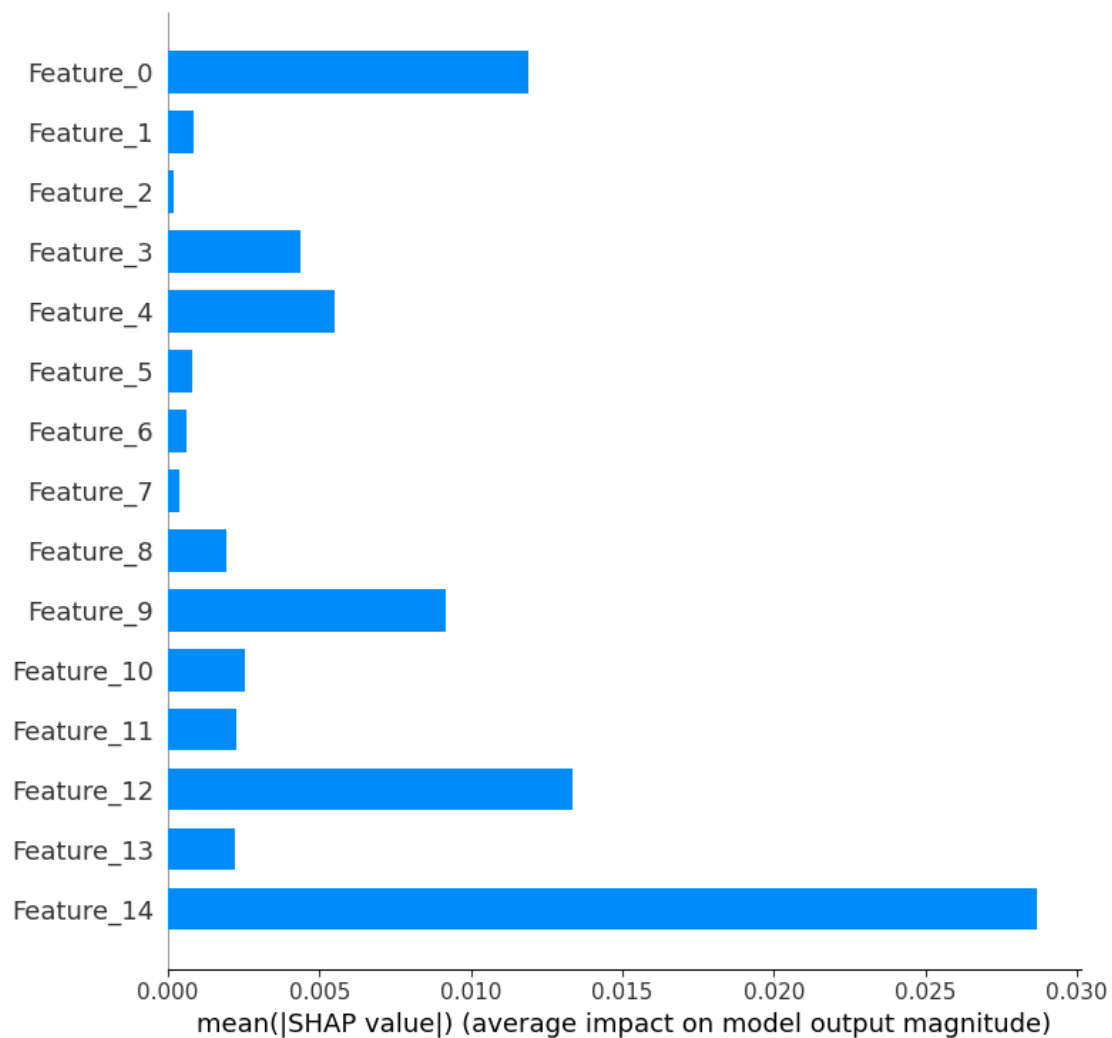
```

```
# =====
# 11. Adjust Plot Layout and Display
# =====

plt.tight_layout() # Optimize spacing for better visibility
plt.show()         # Render the SHAP summary plot
```

```
0%|          | 0/10 [00:00<?, ?it/s]

shap_values.shape: (10, 15, 1)
X_test_flat[:10].shape: (10, 15)
After squeeze, shap_values.shape: (10, 15)
```



88 SHAP Feature Importance Analysis

88.1 Key Observations

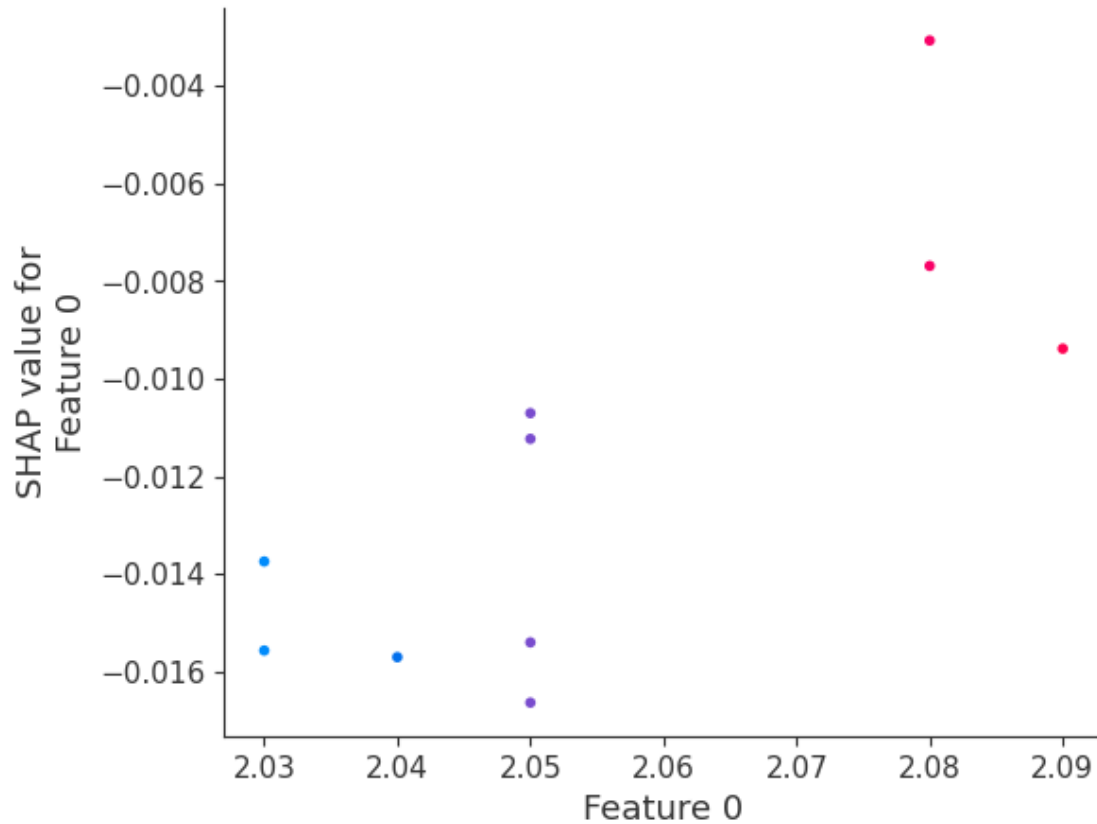
- **Feature Importance Ranking:**
 - The plot shows the average SHAP value magnitude for each feature, representing its contribution to the model's predictions.
 - Higher values indicate a greater impact on model output.
- **Most Influential Features:**
 - `Feature_14` has the highest impact on the model, meaning it plays the most significant role in decision-making.
 - `Feature_0`, `Feature_12`, and `Feature_9` also contribute significantly but to a lesser extent than `Feature_14`.
- **Less Impactful Features:**
 - Features such as `Feature_1`, `Feature_2`, `Feature_5`, `Feature_6`, and `Feature_7` have minimal influence on the model.
 - These features might be candidates for removal or further investigation to determine their relevance.
- **Model Interpretability:**
 - The plot helps in understanding which features drive predictions the most.
 - It can assist in feature selection, simplifying the model without losing predictive power.

88.2 Conclusion

The model relies heavily on `Feature_14`, followed by `Feature_0`, `Feature_12`, and `Feature_9`. Features with low importance may be reconsidered for removal or further analyzed for potential interactions with other features.

SHAP Dependence Plot

```
[ ]: # Visualize dependence plot for feature at index 0 (change index as needed)
      shap.dependence_plot(0, shap_values, X_test_flat[:10])
```



89 SHAP Value Analysis for Feature 0

89.1 Key Observations

- **Feature Influence on Model Predictions:**
 - The plot shows the SHAP values for **Feature 0**, which indicate how this feature impacts the model's output.
 - Negative SHAP values suggest that increasing **Feature 0** tends to decrease the model's prediction.
- **Trend in SHAP Values:**
 - Data points with lower **Feature 0** values (around 2.03–2.05) generally have more negative SHAP values.
 - Higher values of **Feature 0** (above 2.07) tend to have less negative or slightly higher SHAP values, indicating a weaker negative effect on the model output.
- **Color Representation:**
 - The plot uses color gradients to highlight different values, where red points typically indicate higher values, and blue points represent lower values.
 - This helps visualize how **Feature 0** affects the SHAP values across different instances.
- **Possible Interpretation:**
 - The nonlinear distribution of SHAP values suggests that **Feature 0** may have varying

degrees of impact at different ranges.

- If the relationship is complex, transformations (e.g., log scaling or binning) might be explored to better model its effect.

89.2 Conclusion

The plot indicates that **Feature 0** generally has a negative impact on predictions, but the strength of this effect varies across different feature values. This suggests a nonlinear relationship that may require further investigation for improved model interpretability.

Partial Dependence Plot (PDP) for Bi-GRU

```
[ ]: # =====  
# Partial Dependence Plot (PDP) for Bidirectional GRU Model  
# =====  
  
# =====  
# 1. Flatten the 3D Training Data for PDP Computation  
# =====  
  
# The Bi-GRU model expects 3D input, but PDP requires a 2D format.  
# Convert the shape from (samples, timesteps, features) to (samples, features).  
X_train_flat = X_train_gru.reshape(X_train_gru.shape[0], -1)  
  
# =====  
# 2. Select the Feature for Partial Dependence Analysis  
# =====  
  
# Choose a specific feature index to analyze its impact on model predictions.  
feature_idx = 0 # Example: First feature in the dataset  
  
# =====  
# 3. Define a Range of Values for the Selected Feature  
# =====  
  
# Generate 100 evenly spaced values between the feature's minimum and maximum  
# values.  
feature_values = np.linspace(  
    X_train_flat[:, feature_idx].min(), # Minimum observed value of the feature  
    X_train_flat[:, feature_idx].max(), # Maximum observed value of the feature  
    100 # Number of evaluation points  
)  
  
# =====  
# 4. Compute Partial Dependence Values  
# =====  
  
pd_values = [] # List to store computed partial dependence values
```

```

for value in feature_values:
    # Create a temporary copy of the training data
    X_temp = X_train_flat.copy()

    # Set all instances of the selected feature to the current value
    X_temp[:, feature_idx] = value

    # Reshape the modified data back to 3D for model prediction
    X_temp_3d = X_temp.reshape(X_temp.shape[0], X_train_gru.shape[1],
↪X_train_gru.shape[2])

    # Obtain predictions from the trained Bi-GRU model
    predictions = best_bidirectional_gru_model.predict(X_temp_3d, verbose=0)

    # Compute the mean prediction to determine the feature's impact
    pd_values.append(np.mean(predictions))

# =====
# 5. Generate the Partial Dependence Plot (PDP)
# =====

plt.figure(figsize=(8, 6)) # Set figure size for better visibility

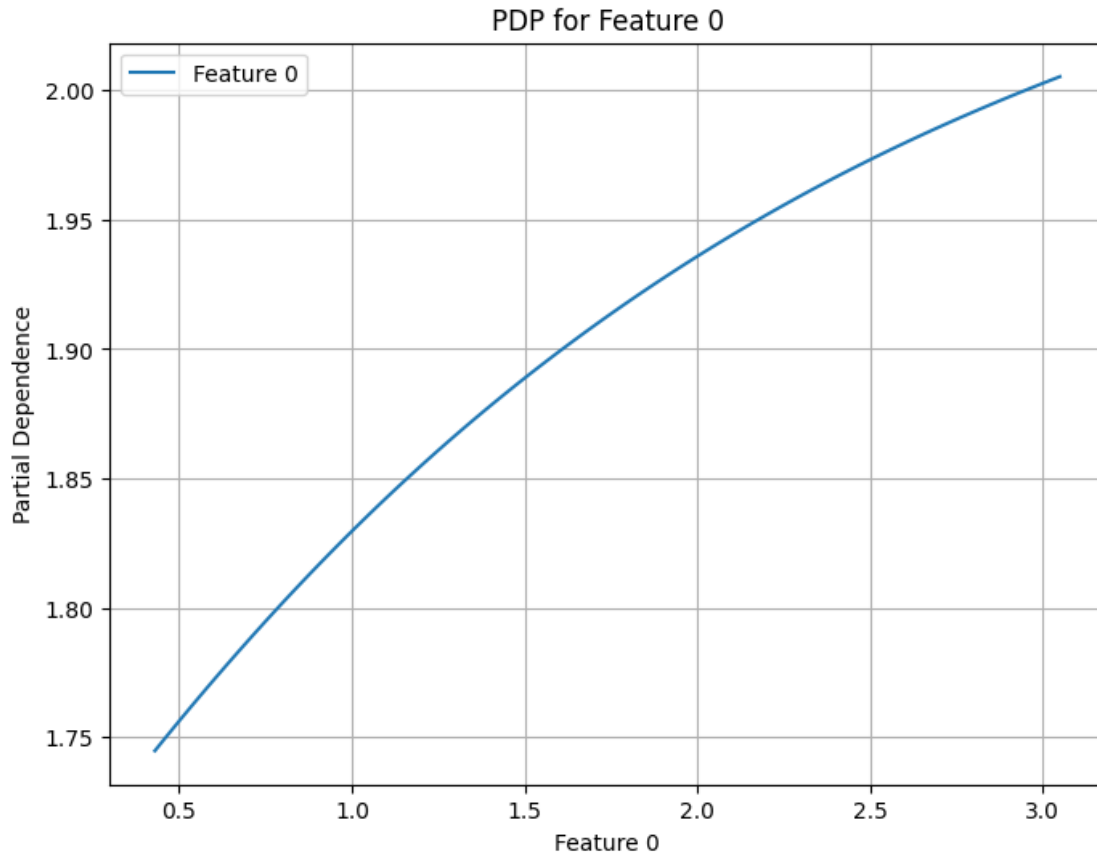
# Plot the computed PDP values against the feature values
plt.plot(feature_values, pd_values, label=f"Feature {feature_idx}")

# Add labels and title to the plot
plt.xlabel(f"Feature {feature_idx}") # X-axis: Feature values
plt.ylabel("Partial Dependence") # Y-axis: Model response
plt.title(f"PDP for Feature {feature_idx}") # Plot title

# Add a legend and grid for clarity
plt.legend()
plt.grid(True)

# Display the plot
plt.show()

```



90 Partial Dependence Plot (PDP) for Feature 0

90.1 Key Observations

- **Relationship Between Feature 0 and Model Output:**
 - The partial dependence plot (PDP) illustrates how **Feature 0** affects the model's predictions while averaging out the effects of all other features.
 - The trend shows a positive correlation between **Feature 0** and the model output.
- **Monotonic Increase:**
 - As **Feature 0** increases, the partial dependence value also increases, indicating that higher values of **Feature 0** contribute to higher predictions.
 - The relationship appears to be smooth and nonlinear, with diminishing returns at higher values.
- **Nonlinear Pattern:**
 - The curve is not perfectly linear but rather follows an upward concave shape.
 - This suggests that the impact of **Feature 0** on predictions grows at a decreasing rate.
- **Model Interpretation:**
 - If **Feature 0** represents a numerical variable such as a financial metric or a continuous measurement, its positive influence on the prediction means that larger values push the model output higher.

- The diminishing slope at higher values suggests that beyond a certain point, increasing Feature 0 has a reduced impact on predictions.

90.2 Conclusion

The PDP for Feature 0 reveals a positive nonlinear relationship with the model's output, where increasing Feature 0 leads to higher predictions, but with diminishing effects at higher values. This insight can be used for feature analysis and potential model optimization.

Integrated Gradients

```
[ ]: # =====
# Integrated Gradients for Feature Attribution in Bi-GRU Model
# =====

# Import necessary TensorFlow functions for gradient computation
import tensorflow.keras.backend as K
import tensorflow as tf

def integrated_gradients(inputs, model, baseline=None, steps=50):
    """
    Compute Integrated Gradients (IG) for a single input sample.

    IG estimates feature importance by computing the gradient of model
    predictions with respect to the input along a path from a baseline
    (typically zeros) to the actual input.

    Parameters:
    - inputs: A single input sample (3D NumPy array) for which IG is computed.
    - model: The trained Bi-GRU model.
    - baseline: The baseline input to compare against (default: all zeros).
    - steps: The number of steps used in the path integral approximation.

    Returns:
    - integrated_grads: The computed Integrated Gradients values for each
    ↪ feature.
    """

    # =====
    # 1. Define the Baseline Input (Default: All Zeros)
    # =====

    if baseline is None:
        baseline = np.zeros(inputs.shape) # Default baseline set to zero input

    # =====
    # 2. Generate Interpolated Inputs Along the Path from Baseline to Input
    # =====
```

```

interpolated_inputs = np.array([
    baseline + (float(i) / steps) * (inputs - baseline) # Linear
↪interpolation
    for i in range(steps + 1)
])

# Reshape to match the model's expected input format (steps+1, timesteps,
↪features)
interpolated_inputs = interpolated_inputs.reshape((steps + 1,) + inputs.
↪shape)

# =====
# 3. Compute Gradients for Each Interpolated Input
# =====

grads = [] # List to store gradients at each step

for input_tensor in interpolated_inputs:
    # Convert the input to a TensorFlow tensor and ensure batch dimension
    input_tensor = tf.convert_to_tensor(input_tensor.reshape(1,
↪*input_tensor.shape))

    # Use GradientTape to compute gradients of predictions w.r.t input
    with tf.GradientTape() as tape:
        tape.watch(input_tensor) # Watch input tensor for gradient
↪computation
        predictions = model(input_tensor) # Get model predictions

    # Compute gradient of the output w.r.t. the input
    grads.append(tape.gradient(predictions, input_tensor))

# =====
# 4. Compute the Final Integrated Gradients
# =====

# Compute the average gradient across all interpolated steps
avg_grads = np.average(grads, axis=0)

# Compute the Integrated Gradients values using the average gradients
integrated_grads = (inputs - baseline) * avg_grads

return integrated_grads

# =====
# Compute Integrated Gradients for a Test Sample
# =====

```

```

# Select a baseline input (default: zero input of the same shape)
baseline = np.zeros_like(X_test_gru[0])

# Compute Integrated Gradients for the first test sample in X_test_gru
ig_values = integrated_gradients(X_test_gru[0], best_bidirectional_gru_model,
    ↪baseline)

# =====
# Visualize Integrated Gradients for Feature Attribution
# =====

plt.figure(figsize=(10, 6)) # Set figure size

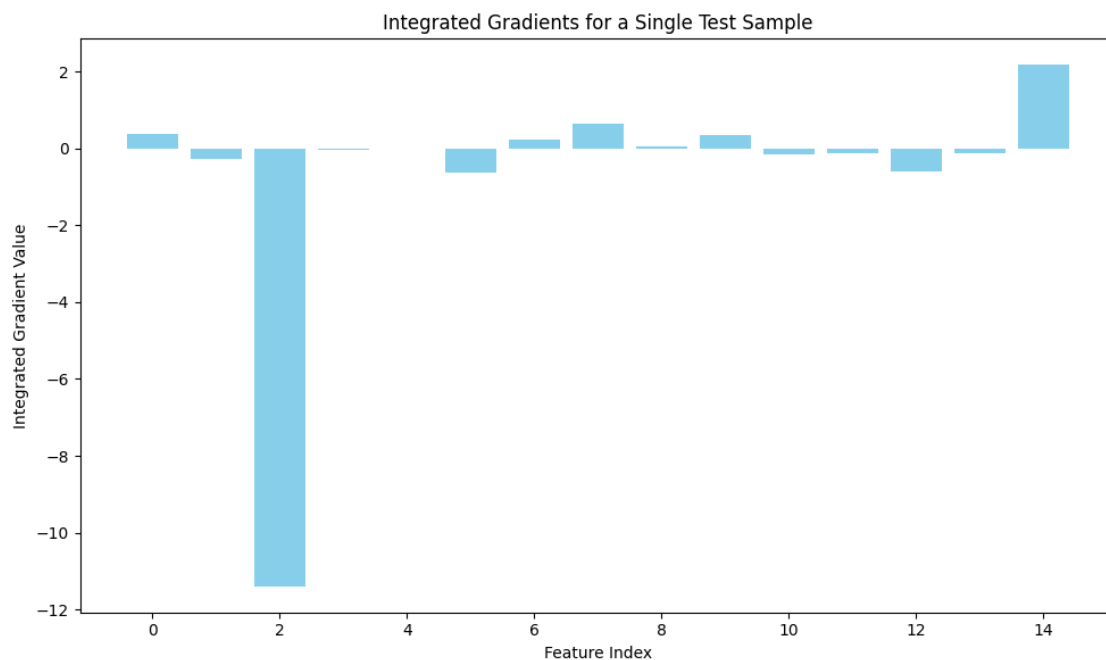
# Create a bar plot to display IG values across features
plt.bar(range(len(ig_values.flatten())), ig_values.flatten(), color="skyblue")

# Add labels and title for clarity
plt.xlabel("Feature Index") # X-axis: Feature index
plt.ylabel("Integrated Gradient Value") # Y-axis: IG importance score
plt.title("Integrated Gradients for a Single Test Sample") # Plot title

# Adjust layout for better readability
plt.tight_layout()

# Display the plot
plt.show()

```



91 Integrated Gradients Analysis for a Single Test Sample

91.1 Key Observations

- **Feature Importance and Directionality:**
 - The plot shows the Integrated Gradients values for each feature, indicating their contribution to the model's prediction for a single test sample.
 - Positive values suggest a feature contributes positively to the prediction, while negative values indicate a suppressing effect.
- **Most Influential Features:**
 - **Feature 2** has the most significant negative impact on the model's prediction, pulling the output lower.
 - **Feature 14** has the highest positive contribution, strongly increasing the prediction.
- **Minor Impact of Other Features:**
 - Several features have small gradient values, indicating that they have minimal influence on the model's decision for this specific instance.
 - Some features (e.g., **Feature 6** and **Feature 12**) have slight positive or negative effects, but they are not as dominant as **Feature 2** or **Feature 14**.
- **Potential Implications:**
 - The model heavily relies on **Feature 2** and **Feature 14** for this particular prediction.
 - If this trend is observed across multiple test samples, these features might be critical in the overall decision-making process of the model.

91.2 Conclusion

The Integrated Gradients plot highlights **Feature 2** as a strong negative influencer and **Feature 14** as a major positive contributor to the model's prediction. Other features play a less significant role in this specific test instance. Further analysis on a broader dataset could confirm whether these trends hold consistently.

Saliency Maps

```
[ ]: # =====  
# Compute and Visualize Saliency Maps for Feature Attribution in Bi-GRU  
# =====  
  
def compute_saliency_map(model, inputs):  
    """  
    Compute a saliency map for a single input sample.  
  
    A saliency map highlights which input features (timesteps)  
    have the highest impact on the model's prediction by calculating  
    the gradients of the output w.r.t. the input.  
  
    Parameters:
```

```

- model: The trained Bidirectional GRU model.
- inputs: A single test sample (3D NumPy array) for which saliency is
↳ computed.

Returns:
- saliency_map: The absolute gradient values, indicating feature importance.
"""

# =====
# 1. Convert Input to a Tensor and Reshape for Batch Processing
# =====

input_tensor = tf.convert_to_tensor(inputs.reshape(1, *inputs.shape)) #
↳ Add batch dimension

# =====
# 2. Compute Gradients Using TensorFlow GradientTape
# =====

with tf.GradientTape() as tape:
    tape.watch(input_tensor) # Track input tensor for gradient computation
    predictions = model(input_tensor) # Get model predictions

# Compute the gradient of the output w.r.t. the input
grads = tape.gradient(predictions, input_tensor)

# =====
# 3. Compute Absolute Gradients to Obtain Saliency Scores
# =====

saliency_map = np.abs(grads.numpy().squeeze()) # Convert tensor to NumPy
↳ and remove extra dimensions

return saliency_map

# =====
# Compute Saliency Map for the First Test Sample
# =====

# Select the first test sample from X_test_gru
saliency_map = compute_saliency_map(best_bidirectional_gru_model, X_test_gru[0])

# =====
# Visualize the Saliency Map for Feature Attribution
# =====

plt.figure(figsize=(10, 6)) # Set figure size

```

```

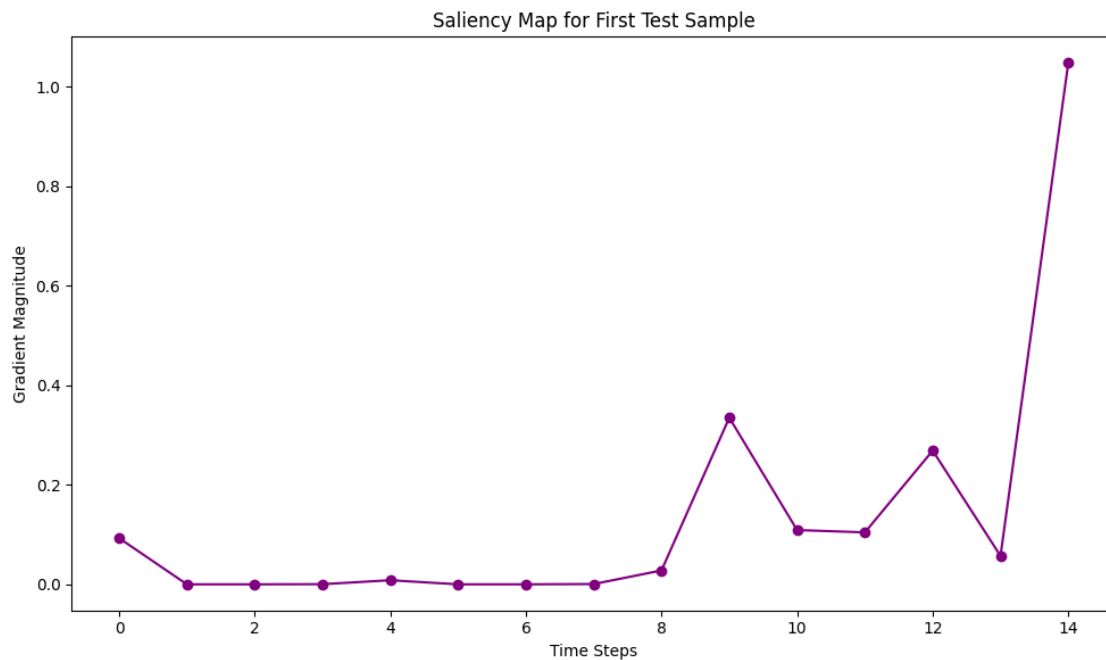
# Plot saliency values across timesteps
plt.plot(saliency_map, marker='o', linestyle='-', color='purple')

# Add labels and title for clarity
plt.xlabel("Time Steps") # X-axis: Time steps
plt.ylabel("Gradient Magnitude") # Y-axis: Saliency score
plt.title("Saliency Map for First Test Sample") # Plot title

# Adjust layout for better readability
plt.tight_layout()

# Display the plot
plt.show()

```



92 Saliency Map for First Test Sample

92.1 Key Observations

- **Gradient Magnitude Interpretation:**
 - The plot shows the gradient magnitude of the model's output with respect to each time step (or feature index).
 - Higher values indicate greater importance, meaning that changes in those time steps have a stronger effect on the model's decision.
- **Most Important Time Step:**

- The last time step (index 14) exhibits the highest gradient magnitude, suggesting it plays a crucial role in the model’s prediction.
- This implies that the model relies heavily on recent or specific features when making a decision.
- **Low Impact of Earlier Time Steps:**
 - Time steps from 1 to 7 have minimal gradient magnitudes, indicating they contribute little to the model’s final decision.
 - This may suggest that the earlier part of the sequence contains less relevant information or that the model has learned to focus more on later time steps.
- **Intermediate Fluctuations:**
 - There is a noticeable increase in gradient magnitude around time steps 9 to 12, suggesting some mid-sequence features influence the model to a moderate extent.
 - However, their impact is still significantly lower compared to the last time step.

92.2 Conclusion

The saliency map indicates that the model heavily relies on the last time step (feature index 14) while earlier time steps have little influence. This suggests a strong temporal dependency, which could imply that later features contain more predictive information. Understanding this pattern can help refine feature engineering or model architecture for improved performance.

Global Surrogate Model

```
[ ]: # =====
# Train and Visualize a Surrogate Decision Tree for Bi-GRU Model
# =====

from sklearn.tree import DecisionTreeRegressor, plot_tree

# =====
# 1. Flatten the 3D Training Data for Surrogate Modeling
# =====

# Convert the 3D GRU training data (samples, timesteps, features) into 2D
# ↳ format (samples, flattened features)
X_train_flat = X_train_gru.reshape(X_train_gru.shape[0], -1) # Shape:
# ↳ (num_samples, num_features)

# =====
# 2. Train a Decision Tree Regressor as a Surrogate Model
# =====

# Initialize the Decision Tree model
# - max_depth=3: Limits the depth of the tree to prevent overfitting and
# ↳ improve interpretability
# - random_state=seed: Ensures reproducibility of results
surrogate_model = DecisionTreeRegressor(max_depth=3, random_state=seed)
```

```

# Fit the Decision Tree on the GRU model's predictions
# - The tree learns to approximate the behavior of the Bi-GRU model
surrogate_model.fit(
    X_train_flat, # Flattened training features
    best_bidirectional_gru_model.predict(X_train_gru).flatten() # Bi-GRU
    ↪ model's predictions as target values
)

# =====
# 3. Visualize the Surrogate Decision Tree for Interpretability
# =====

# Set figure size for better readability
plt.figure(figsize=(12, 8))

# Plot the decision tree structure
# - feature_names=X_train.columns: Uses original feature names for better
    ↪ interpretability
# - filled=True: Colors nodes based on the target value
plot_tree(surrogate_model, feature_names=X_train.columns, filled=True)

# Add a title to the plot
plt.title("Global Surrogate Decision Tree")

# Adjust layout for better visualization
plt.tight_layout()

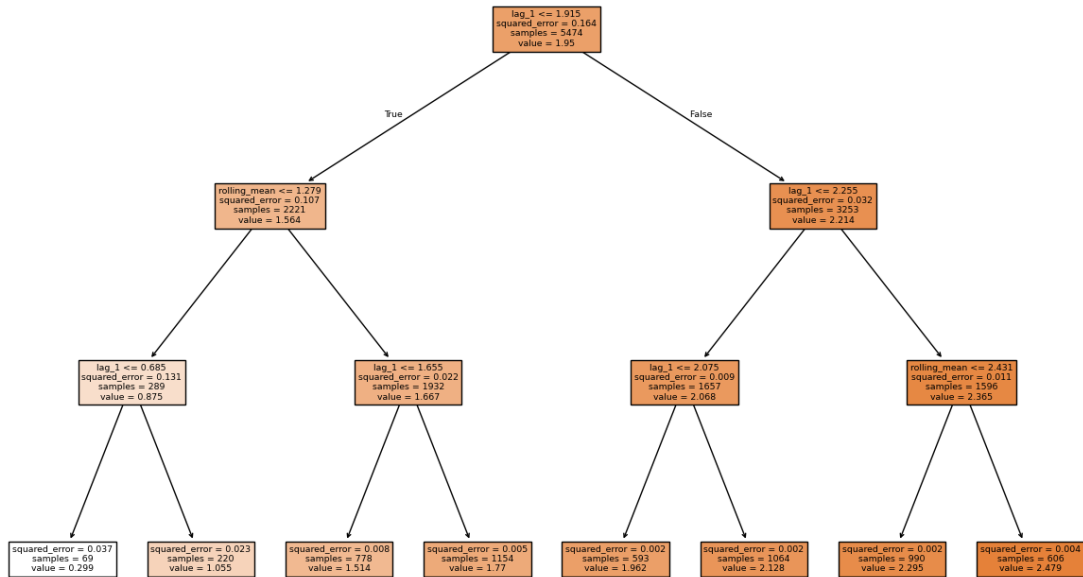
# Display the decision tree
plt.show()

```

172/172

1s 4ms/step

Global Surrogate Decision Tree



93 Global Surrogate Decision Tree Analysis

93.1 Key Observations

- **Purpose of the Decision Tree:**
 - This decision tree serves as a **global surrogate model** to approximate and interpret a more complex machine learning model.
 - It helps in understanding how key features influence predictions in a more interpretable manner.
- **Primary Splitting Feature:**
 - The root node splits on **lag₁ ≤ 1.915**, indicating that this feature is the most important in dividing the dataset.
 - This suggests that past values (**lag₁**) significantly impact the predictions.
- **Subsequent Splits and Feature Influence:**
 - The second major split on the left branch occurs on **rolling_mean ≤ 1.279**, showing that the rolling average also plays a crucial role in prediction.
 - On the right branch, **lag₁ ≤ 2.255** is another strong predictor, further refining the decision boundary.
 - The model heavily relies on **lag₁** and **rolling_mean**, meaning past values and their trends are critical for forecasting.
- **Error Reduction Across Splits:**
 - As the tree branches further, the squared error decreases, meaning that finer splits lead to more accurate predictions.

- Terminal nodes have the lowest squared error, showing that more refined decision-making improves prediction accuracy.
- **Sample Distribution:**
 - The root node contains **5474 samples**, which get progressively split into smaller groups based on feature thresholds.
 - Leaves represent the final predictions, with varying sample sizes and values.

93.2 Conclusion

This **global surrogate decision tree** provides insights into the underlying complex model by approximating its decision-making process. The most influential features are `lag_1` and `rolling_mean`, which means past values and their moving averages are critical for predictions. Understanding these decision paths can help improve feature engineering, model interpretation, and refinement.

LIME for Local Interpretability

```
[ ]: # =====
# Local Model Explanation using LIME for Bi-GRU Model
# =====

from lime.lime_tabular import LimeTabularExplainer

# =====
# 1. Prepare Training Data for LIME (Flatten 3D to 2D)
# =====

# Convert the 3D training data (samples, timesteps, features) into 2D format
# ↳(samples, flattened features)
X_train_for_lime = X_train_gru.reshape(X_train_gru.shape[0], -1) # Shape:
# ↳(num_samples, num_features)

# =====
# 2. Initialize the LIME Explainer
# =====

# Create a LIME explainer instance
# - feature_names: Assigns numerical labels to features for readability
# - mode="regression": Indicates that the model is a regression model
# - training_labels=y_train: Provides labels corresponding to training data
# - random_state=seed: Ensures reproducibility
lime_explainer = LimeTabularExplainer(
    X_train_for_lime,
    feature_names=[f"Feature_{i}" for i in range(X_train_for_lime.shape[1])],
    mode="regression",
    training_labels=y_train,
    verbose=True,
    random_state=seed
```

```

)

# =====
# 3. Select a Test Instance for Explanation
# =====

# Choose a specific instance from the test set to explain
# - instance_idx = 10: Arbitrarily select the 10th sample
instance_idx = 10

# Flatten the selected test instance for LIME (reshape from 3D to 2D)
instance_to_explain = X_test_gru[instance_idx].reshape(1, -1) # Shape: (1,
↳ num_features)

# =====
# 4. Define a Wrapper Function to Reshape Data for Model Prediction
# =====

def model_predict_for_lime(input_2d):
    """
    Converts 2D flattened input back into 3D format required by the Bi-GRU
    ↳ model.
    This ensures LIME can correctly interact with the model.
    """
    input_3d = input_2d.reshape(input_2d.shape[0], X_train_gru.shape[1],
    ↳ X_train_gru.shape[2]) # Reshape 2D to 3D
    return best_bidirectional_gru_model.predict(input_3d, verbose=0) # Get
    ↳ predictions

# =====
# 5. Generate LIME Explanation for the Selected Instance
# =====

# Explain the selected test instance using LIME
# - num_features=10: Limits the explanation to the 10 most relevant features
lime_explanation = lime_explainer.explain_instance(
    instance_to_explain.flatten(),
    model_predict_for_lime,
    num_features=10
)

# =====
# 6. Display the LIME Explanation in the Notebook
# =====

# Show the LIME explanation as an interactive visualization
lime_explanation.show_in_notebook()

```

```
Intercept 1.7969232579091496
Prediction_local [2.02749674]
Right: 2.0055726

<IPython.core.display.HTML object>
```

94 Local Explanation Analysis

94.1 Key Observations

- **Prediction Overview:**
 - The predicted local value is **2.027**.
 - The minimum and maximum possible predictions in the model's range are **-0.08** and **2.71**, respectively.
 - The intercept (baseline prediction) is **1.796**, meaning this is the starting reference point before feature contributions are considered.
- **Feature Contributions:**
 - The visualization separates **positive** and **negative** contributions of different features to the model's prediction.
 - Positive contributions increase the predicted value, while negative contributions lower it.
- **Most Influential Features:**
 - **Positive Impact:**
 - * **Feature_14** (> 2.06) contributes **+0.30** to the prediction, making it the strongest positive influencer.
 - * **Feature_9** (> 2.05) contributes **+0.10**, suggesting it also has a meaningful but smaller positive effect.
 - **Negative Impact:**
 - * **Feature_12** (< 2.05) contributes **-0.08**, decreasing the prediction.
 - * **Feature_6** (< 97.81) contributes **-0.04**, indicating a moderate negative effect.
- **Feature Values:**
 - The feature values are displayed on the right panel.
 - Features in **orange** contribute positively, while those in **blue** contribute negatively.
 - Notably, **Feature_2** has an extremely high value (**21058.38**), yet it contributes minimally, suggesting its impact on this specific prediction might be weak or already accounted for in interactions.

94.2 Conclusion

The local explanation highlights how specific feature values influenced the final prediction of **2.027**. **Feature_14** and **Feature_9** are the strongest **positive contributors**, while **Feature_12** and **Feature_6** act as **negative influencers**. This analysis helps in understanding how the model makes decisions for an individual prediction.

95 Hybrid Models

95.0.1 ML + ML

```
[ ]: # =====  
# Stacking Ensemble: Combining Random Forest (RF) and Extreme Gradient Boosting  
#      ↪ (XGBoost)  
# =====  
  
from sklearn.ensemble import RandomForestRegressor # Base model: Random Forest  
from xgboost import XGBRegressor # Second-layer model: XGBoost  
import pandas as pd # Data manipulation  
  
# =====  
# 1. Train the Base Model - Random Forest  
# =====  
  
# Define the Random Forest model with optimized hyperparameters  
# These parameters were found through hyperparameter tuning.  
rf_model_stack = RandomForestRegressor(  
    random_state=42, # Ensure reproducibility  
    n_estimators=200, # Number of trees in the forest  
    max_depth=15, # Maximum depth of each tree  
    max_features=None, # Use all features (no limitation)  
    min_samples_leaf=4, # Minimum number of samples required at each leaf node  
    min_samples_split=2 # Minimum number of samples required to split a node  
)  
  
# Train the Random Forest model using the original training dataset  
rf_model_stack.fit(X_train, y_train)  
  
# Generate predictions from Random Forest on both training and test sets  
rf_train_preds = rf_model_stack.predict(X_train) # Predictions for training set  
rf_test_preds = rf_model_stack.predict(X_test) # Predictions for test set  
  
# =====  
# 2. Prepare Data for Stacking - Add RF Predictions as a Feature  
# =====  
  
# Create new training and test datasets by adding the RF predictions as an  
#      ↪ additional feature  
X_train_stacked = X_train.copy() # Make a copy of the original training data  
X_test_stacked = X_test.copy() # Make a copy of the original test data  
  
# Append the RF predictions to the feature set for stacking  
X_train_stacked['RF_Pred'] = rf_train_preds # Add RF predictions as a feature  
X_test_stacked['RF_Pred'] = rf_test_preds # Add RF predictions to test set
```

```

# =====
# 3. Train the Second-Layer Model - Extreme Gradient Boosting (XGBoost)
# =====

# Define the XGBoost model with optimized hyperparameters
# These parameters were obtained from hyperparameter tuning.
gbm_stack = XGBRegressor(
    objective='reg:squarederror', # Regression task
    eval_metric='rmse', # Root Mean Squared Error as the evaluation metric
    seed=42, # Ensure reproducibility
    max_depth=9, # Maximum depth of the trees
    learning_rate=0.1, # Step size shrinkage to prevent overfitting
    subsample=0.9, # Fraction of samples used per tree
    colsample_bytree=0.9, # Fraction of features used per tree
    n_estimators=100 # Number of boosting rounds (trees)
)

# Train the XGBoost model using the stacked training dataset (RF predictions_
↳included)
gbm_stack.fit(X_train_stacked, y_train)

# Generate predictions using the stacked model on the test set
stacked_pred = gbm_stack.predict(X_test_stacked)

# =====
# 4. Evaluate the Stacking Ensemble Model
# =====

# Compute evaluation metrics using a predefined function
# This compares the model's predictions against actual values
stacking_metrics = calculate_metrics(y_test.values, stacked_pred,↳
↳naive_forecast,
                                     "Hybrid Model: Stacking RF + GBM")

# Store the metrics results for future reference
metrics_results.append(stacking_metrics)

# Print the final evaluation results
print("Stacking Ensemble (RF + GBM) Evaluation Metrics:")
print(stacking_metrics)

```

```

Stacking Ensemble (RF + GBM) Evaluation Metrics:
{'Model': 'Hybrid Model: Stacking RF + GBM', 'MSE': 0.003578237589134771,
'MRME': 0.059818371669034684, 'MAE': 0.03493534310008593, 'MAPE (%)':
1.3977086797788767, 'sMAPE (%)': 1.4202811745173631, 'MASE': 1.9394357138693288,
'R^2': 0.8875065165701795}

```

95.0.2 DL + ML

```
[ ]: # =====  
# Hybrid Model: LSTM for Feature Extraction + Random Forest for Prediction  
# =====  
  
import numpy as np # For numerical operations  
import tensorflow as tf # Deep learning framework for LSTM  
from tensorflow.keras.models import Sequential # Sequential API for building  
    ↳ the LSTM model  
from tensorflow.keras.layers import Input, LSTM, Dense, Dropout # LSTM model  
    ↳ components  
from tensorflow.keras.optimizers import Adam # Optimizer for LSTM training  
from sklearn.ensemble import RandomForestRegressor # Random Forest model for  
    ↳ final prediction  
  
# =====  
# 1. Set Random Seeds for Reproducibility  
# =====  
  
# Ensure consistent results by setting seeds for TensorFlow and NumPy  
tf.random.set_seed(42)  
np.random.seed(42)  
  
# =====  
# 2. Data Preparation for LSTM  
# =====  
  
# The LSTM model requires sequential lag features for learning temporal  
    ↳ patterns.  
# Ensure that lag features are present in the dataset.  
lag_columns = [col for col in X_train.columns if col.startswith('lag_')]  
if not lag_columns:  
    raise ValueError("No lag columns found in X_train. Ensure your feature  
    ↳ engineering created lag features.")  
  
# Extract lag features from training and test sets  
X_train_seq = X_train[lag_columns].values  
X_test_seq = X_test[lag_columns].values  
  
# Reshape the extracted features into a 3D format required for LSTM: (samples,  
    ↳ timesteps, features)  
X_train_seq = X_train_seq.reshape(X_train_seq.shape[0], X_train_seq.shape[1], 1)  
X_test_seq = X_test_seq.reshape(X_test_seq.shape[0], X_test_seq.shape[1], 1)  
  
# Capture the number of timesteps (lag steps) and input features  
timesteps = X_train_seq.shape[1] # Number of past timesteps used as input
```

```

input_features = X_train_seq.shape[2] # Number of features per timestep (should
↳ be 1)

# =====
# 3. Build and Train the LSTM Model for Feature Extraction
# =====

# Best hyperparameters for LSTM (optimized from previous tuning)
lstm_units = 150 # Number of units in the LSTM layer
dropout_rate = 0.2 # Dropout rate for regularization
learning_rate = 0.001 # Learning rate for Adam optimizer
clipnorm_value = 3.0 # Gradient clipping to prevent exploding gradients
epochs = 100 # Number of training epochs
batch_size = 32 # Number of samples per batch

# Define the LSTM model
# - Uses an Input layer to avoid warnings about input shape.
# - The LSTM layer learns temporal patterns in the data.
# - A dropout layer prevents overfitting.
# - A Dense layer outputs latent feature embeddings.
lstm_model = Sequential([
    Input(shape=(timesteps, input_features)), # Input shape: (timesteps,
↳ features)
    LSTM(lstm_units, activation='tanh', return_sequences=False), # LSTM layer
↳ for learning sequential dependencies
    Dropout(dropout_rate), # Regularization to reduce overfitting
    Dense(20, activation='relu') # Dense layer to extract latent embeddings
])

# Configure the optimizer with gradient clipping
optimizer = Adam(learning_rate=learning_rate, clipnorm=clipnorm_value)

# Compile the model with Mean Squared Error loss function for regression
lstm_model.compile(optimizer=optimizer, loss='mse')

# Train the LSTM model to extract temporal feature embeddings
print("Training LSTM model for latent feature extraction...")
history_lstm = lstm_model.fit(
    X_train_seq, y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_split=0.2, # 20% of the training data used for validation
    verbose=0 # Suppress output for cleaner logging
)

# =====
# 4. Extract LSTM Latent Feature Embeddings

```



```

# =====

# Generate feature embeddings for both training and test datasets
X_train_embed = lstm_model.predict(X_train_seq) # Extract embeddings from LSTM
↳for training data
X_test_embed = lstm_model.predict(X_test_seq) # Extract embeddings from LSTM
↳for test data

# =====
# 5. Train a Random Forest on LSTM Embeddings
# =====

# Define the Random Forest model using optimal hyperparameters
rf_model = RandomForestRegressor(
    random_state=42, # Ensure reproducibility
    n_estimators=200, # Number of trees in the forest
    max_depth=15, # Maximum depth of each tree
    max_features=None, # Use all features (no limitation)
    min_samples_leaf=4, # Minimum number of samples required at each leaf node
    min_samples_split=2 # Minimum number of samples required to split a node
)

# Train the Random Forest model using LSTM-generated feature embeddings
rf_model.fit(X_train_embed, y_train)

# =====
# 6. Make Predictions Using the Hybrid Model
# =====

# Generate final predictions using the trained Random Forest model on test
↳embeddings
rf_pred = rf_model.predict(X_test_embed)

# =====
# 7. Evaluate the Hybrid Model (LSTM + Random Forest)
# =====

# Compute evaluation metrics using a predefined function
lstm_rf_metrics = calculate_metrics(
    y_test.values, rf_pred, naive_forecast, "Hybrid Model: LSTM + Random Forest"
)

# Store the evaluation results for further comparison
metrics_results.append(lstm_rf_metrics)

# Print the final evaluation results
print("LSTM + Random Forest Evaluation Metrics:")

```

```
print(lstm_rf_metrics)
```

Training LSTM model for latent feature extraction...

172/172 1s 3ms/step

43/43 0s 2ms/step

LSTM + Random Forest Evaluation Metrics:

```
{'Model': 'Hybrid Model: LSTM + Random Forest', 'MSE': 0.0026929336144895003,
'MRMSE': 0.0518934833528209, 'MAE': 0.03085490952236714, 'MAPE (%)':
1.24663044213293, 'sMAPE (%)': 1.2561475327432015, 'MASE': 1.7129104272554994,
'R^2': 0.9153389132518637}
```

95.0.3 DL + DL

```
[ ]: # =====
# Hybrid Model: LSTM + GRU for Time Series Forecasting (Deep Learning Only)
# =====

import numpy as np # For numerical operations
import tensorflow as tf # Deep learning framework
from tensorflow.keras.models import Sequential # Model architecture
from tensorflow.keras.layers import Input, LSTM, GRU, Dense, Dropout # Layers
    ↪ for the hybrid model
from tensorflow.keras.optimizers import Adam # Optimizer for training

# =====
# 1. Set Random Seeds for Reproducibility
# =====

# Ensure consistent results across runs by setting random seeds
tf.random.set_seed(42)
np.random.seed(42)

# =====
# 2. Data Preparation for LSTM + GRU
# =====

# The LSTM-GRU model requires sequential data, typically represented by lag
    ↪ features.
# Ensure that the dataset contains lagged variables for sequential input.

# Identify lag feature columns (assumes columns start with "lag_")
lag_columns = [col for col in X_train.columns if col.startswith('lag_')]
if not lag_columns:
    raise ValueError("No lag columns found in X_train. Ensure that lag features
    ↪ have been created.")
```

```

# Extract lag features from training and test sets and convert them into NumPy
↳arrays
X_train_seq = X_train[lag_columns].values
X_test_seq = X_test[lag_columns].values

# Reshape extracted features into a 3D format required for LSTM/GRU: (samples,
↳timesteps, features)
# - Each row represents a training sample.
# - Each column (lag feature) represents a timestep in the sequence.
# - The last dimension (1) represents the number of features per timestep.
X_train_seq = X_train_seq.reshape(X_train_seq.shape[0], X_train_seq.shape[1], 1)
X_test_seq = X_test_seq.reshape(X_test_seq.shape[0], X_test_seq.shape[1], 1)

# Capture the number of timesteps (lags) used and the input feature count
timesteps = X_train_seq.shape[1] # Number of past timesteps considered for
↳each prediction

# =====
# 3. Build the LSTM + GRU Hybrid Model
# =====

# Define a Sequential model architecture
# - The Input layer explicitly defines the shape of input data: (timesteps,
↳features).
# - An LSTM layer captures long-term dependencies in sequential data.
# - A GRU layer enhances learning by capturing short-term dependencies more
↳efficiently.
# - Dropout layers are used for regularization to reduce overfitting.
# - A Dense layer produces the final regression output.
model = Sequential([
    Input(shape=(timesteps, 1)), # Input layer with shape (timesteps, 1
↳feature)
    LSTM(50, activation='tanh', return_sequences=True), # LSTM layer (returns
↳full sequences for GRU input)
    Dropout(0.3), # Dropout to prevent overfitting
    GRU(30, activation='tanh', return_sequences=False), # GRU layer (returns
↳final output sequence)
    Dropout(0.3), # Dropout for additional regularization
    Dense(1, activation='linear') # Final Dense layer for regression output
])

# Compile the model with:
# - Adam optimizer: Efficient adaptive learning rate algorithm
# - Mean Squared Error (MSE) loss function: Suitable for regression tasks
model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

```

```

# =====
# 4. Train the LSTM + GRU Model
# =====

print("Training LSTM + GRU hybrid model...")

# Train the model using the sequential training data
history = model.fit(
    X_train_seq, y_train,
    epochs=50,          # Number of epochs (adjustable based on dataset size)
    batch_size=32,      # Number of samples per training batch
    validation_split=0.2, # Use 20% of the training data for validation
    verbose=0           # Suppress training logs for cleaner output
)

# =====
# 5. Make Predictions Using the Trained Model
# =====

# Generate predictions on the test set
lstm_gru_pred = model.predict(X_test_seq)

# =====
# 6. Evaluate Model Performance
# =====

# Compute evaluation metrics using a predefined function
lstm_gru_metrics = calculate_metrics(
    y_test.values, lstm_gru_pred.flatten(), naive_forecast, "Hybrid Model: LSTM_
↪+ GRU"
)

# Store the evaluation results for further comparison
metrics_results.append(lstm_gru_metrics)

# Display the evaluation results
print("LSTM + GRU Hybrid Evaluation Metrics:")
print(lstm_gru_metrics)

```

```

Training LSTM + GRU hybrid model...
43/43          0s 6ms/step
LSTM + GRU Hybrid Evaluation Metrics:
{'Model': 'Hybrid Model: LSTM + GRU', 'MSE': 0.0017484045925626251, 'RMSE':
0.04181392821253015, 'MAE': 0.03010765143255843, 'MAPE (%)': 1.2569436566030483,
'sMAPE (%)': 1.2550489215774794, 'MASE': 1.6714263913695253, 'R^2':
0.9450332410404239}

```

96 Result

```
[ ]: !pip install tabulate
```

Requirement already satisfied: tabulate in /usr/local/lib/python3.11/dist-packages (0.9.0)

```
[ ]: from tabulate import tabulate

# Convert metrics_results (a list of dictionaries) to a DataFrame for better
↳ readability
results_df = pd.DataFrame(metrics_results)

# Convert results DataFrame to tabulate format
def display_metrics_table(results_df):
    # Convert DataFrame to list of lists for tabulate
    table_data = results_df.values.tolist()
    # Define headers
    headers = results_df.columns.tolist()
    # Print the table using tabulate
    print(tabulate(table_data, headers=headers, tablefmt="fancy_grid",
↳ floatfmt=".4f"))

# Display the table
print("Model Evaluation Metrics Summary:\n")
display_metrics_table(results_df)
```

Model Evaluation Metrics Summary:

Model	MSE	RMSE	MAE
MAPE (%) sMAPE (%) MASE R ²			
Random Forest (RF)	0.0031	0.0557	0.0300
1.1942 1.2109 1.6663 0.9025			
Extreme Gradient Boosting (XGBoost)	0.0006	0.0245	0.0146
0.9443 0.9379 0.0372 0.9966			
Light Gradient Boosting Machine (LightGBM)	0.0007	0.0273	0.0169
1.2628 1.1649 0.0429 0.9958			
Gradient Boosting Regressor (GBR)	0.0006	0.0245	0.0148

0.9403	0.9366	0.0377	0.9966			
Extremely Randomized Trees Regressor (ERTR)	0.0007	0.0269	0.0164			
1.1514	1.0918	0.0417	0.9959			
Histogram-Based Gradient Boosting Regressor (HGBR)	0.0006	0.0242	0.0148			
0.9646	0.9656	0.0378	0.9967			
Categorical Boosting (CatBoost)	0.0006	0.0247	0.0151			
1.1091	1.0280	0.0385	0.9966			
Support Vector Regression (SVR)	0.0012	0.0347	0.0163			
1.0985	1.0268	0.0415	0.9932			
Ridge Regression (Ridge)	0.0001	0.0113	0.0067			
0.4288	0.4283	0.0170	0.9993			
Lasso Regression (Lasso)	0.0010	0.0311	0.0198			
1.5007	1.3584	0.0504	0.9945			
ElasticNet Regression (EN)	0.0009	0.0293	0.0179			
1.3318	1.2061	0.0455	0.9952			
Decision Tree Regressor (DTR)	0.0010	0.0322	0.0205			
1.3339	1.3303	0.0521	0.9941			
K-Nearest Neighbors Regression (KNN)	0.0009	0.0304	0.0179			
1.1511	1.1287	0.0455	0.9948			
Bidirectional Long Short-Term Memory (BiLSTM)	0.0034	0.0580	0.0495			
2.0861	2.1146	0.4942	0.8498			
Bidirectional Gated Recurrent Unit (Bi-GRU)	0.0035	0.0588	0.0521			
2.2096	2.2401	0.4155	0.8382			
Hybrid Model: Stacking RF + GBM	0.0036	0.0598	0.0349			

1.3977 1.4203 1.9394 0.8875

Hybrid Model: LSTM + Random Forest 0.0027 0.0519 0.0309
1.2466 1.2561 1.7129 0.9153

Hybrid Model: LSTM + GRU 0.0017 0.0418 0.0301
1.2569 1.2550 1.6714 0.9450

97 Model Evaluation Metrics Summary

Model	MSE	RMSE	MAE	MAPE (%)	sMAPE (%)	MASE	R ²
Random Forest (RF)	0.0031	0.0557	0.0300	1.1942	1.2109	1.6663	0.9025
Extreme Gradient Boosting (XGBoost)	0.0006	0.0245	0.0146	0.9443	0.9379	0.0372	0.9966
Light Gradient Boosting Machine (LightGBM)	0.0007	0.0273	0.0169	1.2628	1.1649	0.0429	0.9958
Gradient Boosting Regressor (GBR)	0.0006	0.0245	0.0148	0.9403	0.9366	0.0377	0.9966
Extremely Randomized Trees Regressor (ERTR)	0.0007	0.0269	0.0164	1.1514	1.0918	0.0417	0.9959
Histogram-Based Gradient Boosting Regressor (HGBR)	0.0006	0.0242	0.0148	0.9646	0.9656	0.0378	0.9967
Categorical Boosting (CatBoost)	0.0006	0.0247	0.0151	1.1091	1.0280	0.0385	0.9966
Support Vector Regression (SVR)	0.0012	0.0347	0.0163	1.0985	1.0268	0.0415	0.9932
Ridge Regression (Ridge)	0.0001	0.0113	0.0067	0.4288	0.4283	0.0170	0.9993
Lasso Regression (Lasso)	0.0010	0.0311	0.0198	1.5007	1.3584	0.0504	0.9945
ElasticNet Regression (EN)	0.0009	0.0293	0.0179	1.3318	1.2061	0.0455	0.9952
Decision Tree Regressor (DTR)	0.0010	0.0322	0.0205	1.3339	1.3803	0.0521	0.9941
K-Nearest Neighbors Regression (KNN)	0.0009	0.0304	0.0179	1.1511	1.1287	0.0455	0.9948
Bidirectional Long Short-Term Memory (BiLSTM)	0.0034	0.0580	0.0495	2.0861	2.1146	0.4942	0.8498
Bidirectional Gated Recurrent Unit (Bi-GRU)	0.0035	0.0588	0.0521	2.2096	2.2401	0.4155	0.8382
Hybrid Model: Stacking RF + GBM	0.0036	0.0598	0.0349	1.3977	1.4203	1.9394	0.8875
Hybrid Model: LSTM + Random Forest	0.0027	0.0519	0.0309	1.2466	1.2561	1.7129	0.9153
Hybrid Model: LSTM + GRU	0.0017	0.0418	0.0301	1.2569	1.2550	1.6714	0.9450

```
[ ]: # Display the best-performing model for each metric
print("\nBest Models for Each Metric:")
for metric in ["MSE", "RMSE", "MAE", "MAPE (%)", "sMAPE (%)", "MASE", "R^2"]:
    # For R^2, a higher value is better; for all others, lower is better.
    best_idx = results_df[metric].idxmax() if metric == "R^2" else
    results_df[metric].idxmin()
    best_model = results_df.loc[best_idx]
    print(f"{metric}: {best_model['Model']} ({best_model[metric]:.4f})")
```

Best Models for Each Metric:

MSE: Ridge Regression (Ridge) (0.0001)
 RMSE: Ridge Regression (Ridge) (0.0113)
 MAE: Ridge Regression (Ridge) (0.0067)
 MAPE (%): Ridge Regression (Ridge) (0.4288)
 sMAPE (%): Ridge Regression (Ridge) (0.4283)
 MASE: Ridge Regression (Ridge) (0.0170)
 R^2: Ridge Regression (Ridge) (0.9993)

98 Model Evaluation Metrics Summary - Key Insights

The table presents the performance of different regression models using metrics like MSE, RMSE, MAE, MAPE, sMAPE, MASE, and (R^2). Below is a concise analysis:

98.1 Best Performing Models

- **Ridge Regression (Ridge)**
 - Lowest error across all metrics (MSE: 0.0001, RMSE: 0.0113, MAE: 0.0067, MAPE: 0.4288%)
 - Highest (R^2) (0.9993)
 - Best overall predictive accuracy.
- **Gradient Boosting Models (XGBoost, HGBR, CatBoost, GBR)**
 - High accuracy with low RMSE (~0.0242-0.0247) and high (R^2) (~0.9966-0.9967).
 - Strong candidates for regression tasks.

98.2 Weakest Models

- **Neural Networks (BiLSTM, Bi-GRU)**
 - High errors (MSE: ~0.0034-0.0035, RMSE: ~0.0588, MAE: ~0.0495-0.0521).
 - Lowest (R^2) (~0.8382-0.8498), indicating poor fit.
 - Not suitable for this dataset.

98.3 Hybrid Model Performance

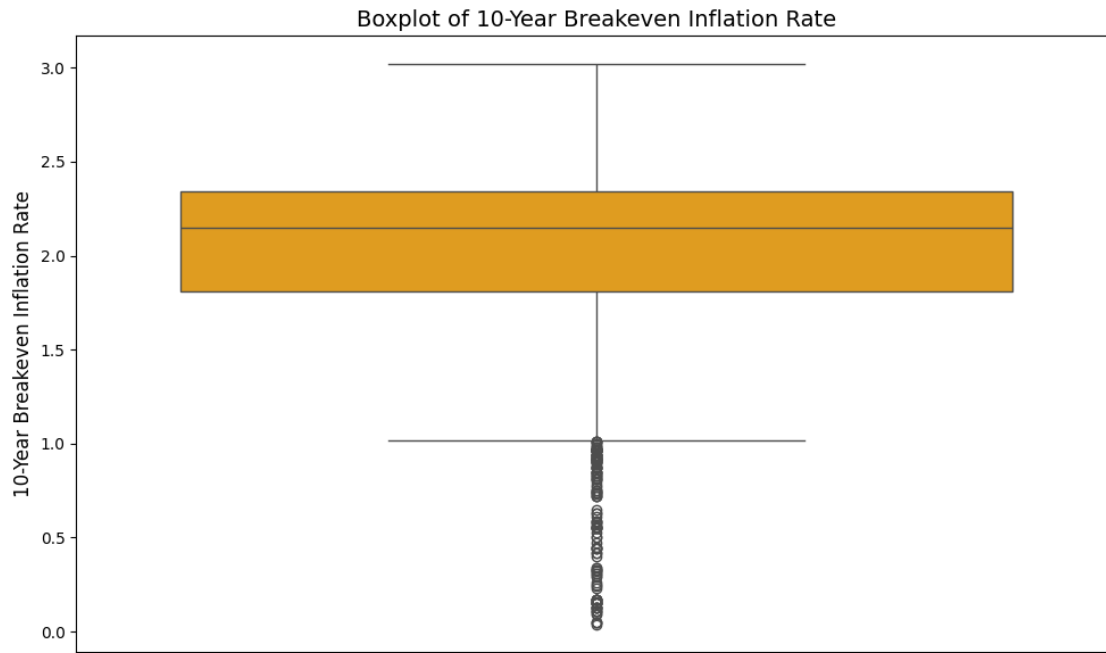
- **LSTM + GRU Hybrid:**
 - Performs better than standalone LSTM or GRU models.
 - **MAPE: 1.2569%, RMSE: 0.0418** – Improved over deep learning models.

98.4 Key Takeaways

- Ridge Regression & Gradient Boosting models (XGBoost, HGBR, CatBoost) are the best choices.
- Avoid using BiLSTM & Bi-GRU due to high errors and poor (R^2).
- Hybrid models improve deep learning performance but still lag behind tree-based methods.

98.4.1 Recommendation: Use Ridge Regression or Gradient Boosting for best accuracy!

```
[ ]: # =====  
# Block 1: Data Cleaning and Boxplot of the Target Variable  
# =====  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
# Ensure the target column is numeric and drop rows with NaN values  
df = data.copy()  
df['10-Year Breakeven Inflation Rate'] = pd.to_numeric(df['10-Year Breakeven_  
↪Inflation Rate'], errors='coerce')  
df_clean = df.dropna(subset=['10-Year Breakeven Inflation Rate'])  
  
# Plot the boxplot for the 10-Year Breakeven Inflation Rate  
plt.figure(figsize=(10, 6))  
sns.boxplot(y=df_clean['10-Year Breakeven Inflation Rate'], color='orange')  
plt.title('Boxplot of 10-Year Breakeven Inflation Rate', fontsize=14)  
plt.ylabel('10-Year Breakeven Inflation Rate', fontsize=12)  
plt.tight_layout()  
plt.show()
```

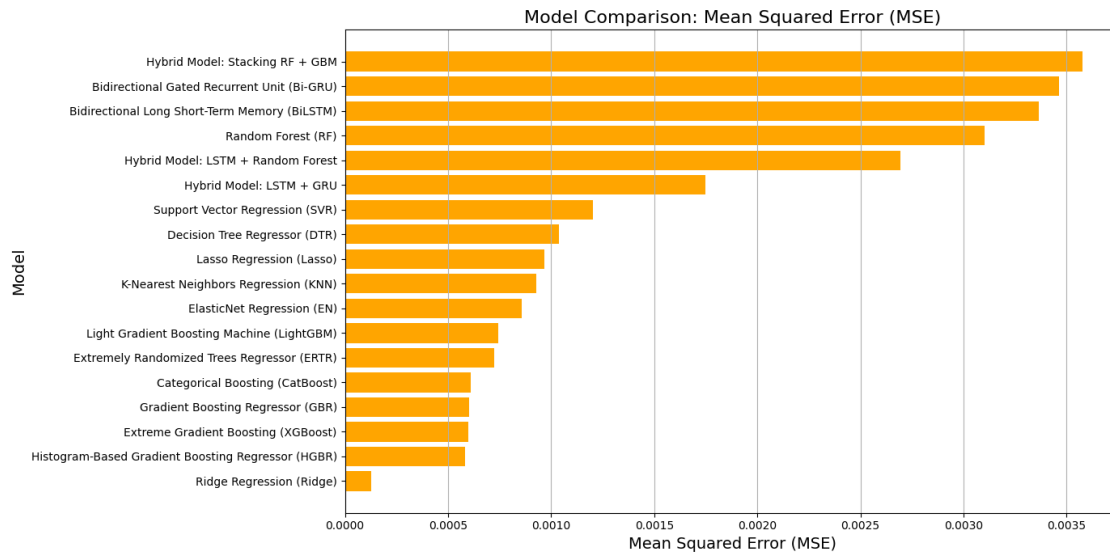


```
[ ]: # =====
# Define Function for Model Evaluation Metrics Comparison
# =====
def plot_metrics_comparison(metrics_df, metric, title, color):
    """
    Create a horizontal bar plot comparing models based on a given metric.
    For R2, higher values are better; for other metrics, lower values are
    preferred.
    """
    ascending = False if metric == "R2" else True
    metrics_df_sorted = metrics_df.sort_values(by=metric, ascending=ascending)

    plt.figure(figsize=(14, 7))
    plt.barh(metrics_df_sorted['Model'], metrics_df_sorted[metric], color=color)
    plt.title(f"Model Comparison: {title}", fontsize=16)
    plt.xlabel(title, fontsize=14)
    plt.ylabel("Model", fontsize=14)
    plt.grid(axis='x')
    plt.tight_layout()
    plt.show()
```

```
[ ]: # =====
# Visualize Evaluation Metrics for Models
# =====
# Visualize Mean Squared Error (MSE)
```

```
plot_metrics_comparison(results_df, "MSE", "Mean Squared Error (MSE)", "orange")
```



99 Model Comparison: Mean Squared Error (MSE)

99.1 Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their Mean Squared Error (MSE).
- Lower MSE indicates better performance**, meaning models with shorter bars perform better.

99.2 Top Performing Models (Lowest MSE)

- Ridge Regression** has the lowest MSE, suggesting it provides the best predictive accuracy.
- Histogram-Based Gradient Boosting Regressor (HGBR)** and **Extreme Gradient Boosting (XGBoost)** also have low MSE, indicating strong performance.

99.3 Moderate Performance Models

- Gradient Boosting Regressor (GBR)**, **CatBoost**, and **Extremely Randomized Trees Regressor (ERTR)** fall in the mid-range in terms of MSE.
- LightGBM** and **ElasticNet Regression** show comparable performance but slightly higher error.

99.4 Higher MSE (Lower Performance) Models

- Hybrid deep learning models** such as **LSTM + GRU** and **LSTM + Random Forest** have relatively high MSE, suggesting weaker predictive accuracy.
- Random Forest (RF)** and **Bi-LSTM** also perform worse than simpler models like Ridge Regression.

- **Stacking models (Hybrid Model: Stacking RF + GBM)** show the highest MSE, indicating poor predictive performance.

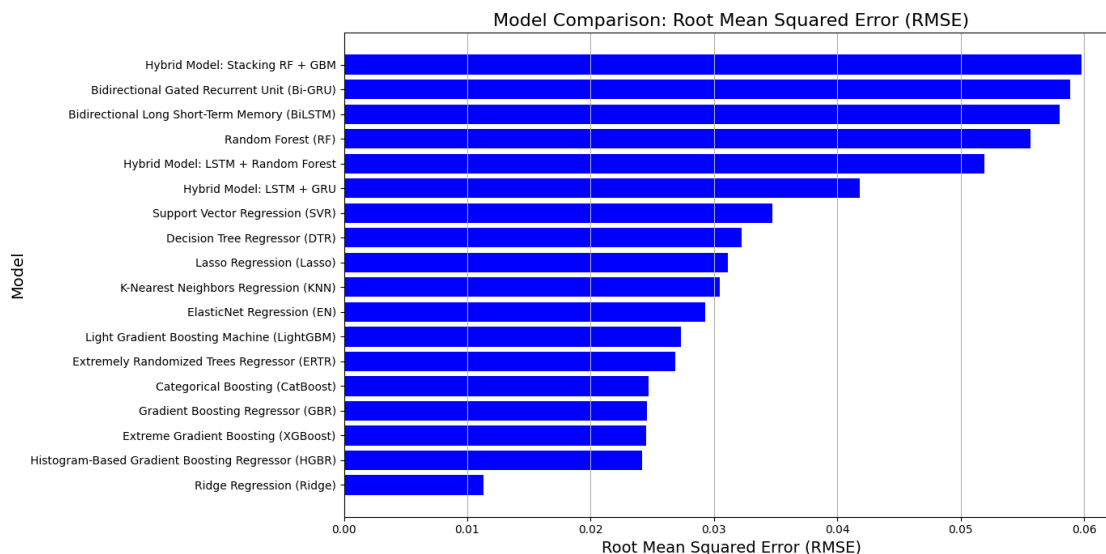
99.5 Insights

- Traditional regression models like **Ridge Regression** and **Lasso Regression** outperform complex deep learning models in terms of MSE.
- **Boosting techniques (XGBoost, LightGBM, CatBoost)** provide competitive performance.
- Deep learning models, despite their complexity, **do not necessarily yield better results in this context**.
- Hybrid models involving **LSTM and Random Forests** have **higher error rates** compared to traditional models.

99.6 Conclusion

- **Ridge Regression is the most accurate model** based on MSE.
- **Deep learning and hybrid models do not outperform traditional boosting and regression models** in this analysis.
- **For lower MSE, simpler models like Ridge Regression or boosting techniques may be preferable.**

```
[ ]: # Visualize Root Mean Squared Error (RMSE)
plot_metrics_comparison(results_df, "RMSE", "Root Mean Squared Error (RMSE)",
↪ "blue")
```



100 Model Comparison: Root Mean Squared Error (RMSE)

100.1 Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their **Root Mean Squared Error (RMSE)**.
- **Lower RMSE values indicate better model performance**, as they signify lower prediction errors.

100.2 Top Performing Models (Lowest RMSE)

1. **Ridge Regression** achieves the lowest RMSE, indicating the most accurate predictions.
2. **Histogram-Based Gradient Boosting Regressor (HGBR)**, **Extreme Gradient Boosting (XGBoost)**, and **Gradient Boosting Regressor (GBR)** also perform well, showing relatively low RMSE.

100.3 Moderate Performance Models

- **Categorical Boosting (CatBoost)**, **Extremely Randomized Trees Regressor (ERTR)**, and **LightGBM** exhibit moderate RMSE values.
- **ElasticNet Regression** and **K-Nearest Neighbors Regression (KNN)** are slightly worse but still competitive.

100.4 Higher RMSE (Lower Performance) Models

- **Hybrid deep learning models** such as **LSTM + GRU** and **LSTM + Random Forest** have **higher RMSE**, suggesting weaker predictive accuracy.
- **Random Forest (RF)** and **Bi-LSTM** also perform worse than simpler models like Ridge Regression.
- **The Stacking Hybrid Model (RF + GBM)** shows the **highest RMSE**, meaning it has the poorest prediction accuracy.

100.5 Insights

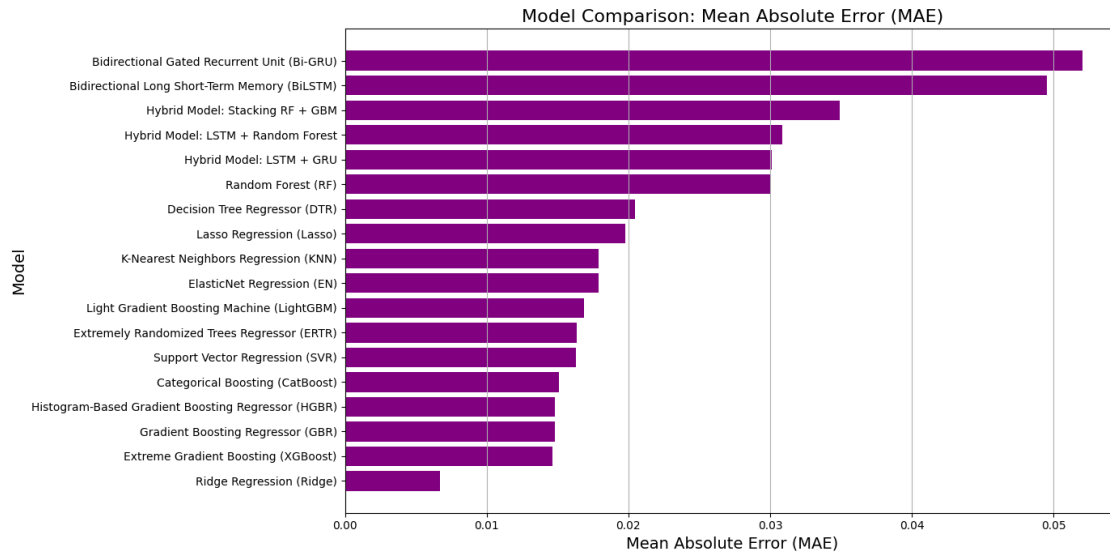
- Traditional regression models like **Ridge Regression** and **Lasso Regression** outperform **deep learning models** in terms of RMSE.
- **Boosting algorithms (XGBoost, LightGBM, CatBoost)** provide **competitive performance**, making them strong alternatives.
- **Deep learning and hybrid models do not necessarily lead to better performance** in this dataset.
- **Hybrid deep learning models tend to have higher RMSE**, suggesting they may not generalize well compared to simpler models.

100.6 Conclusion

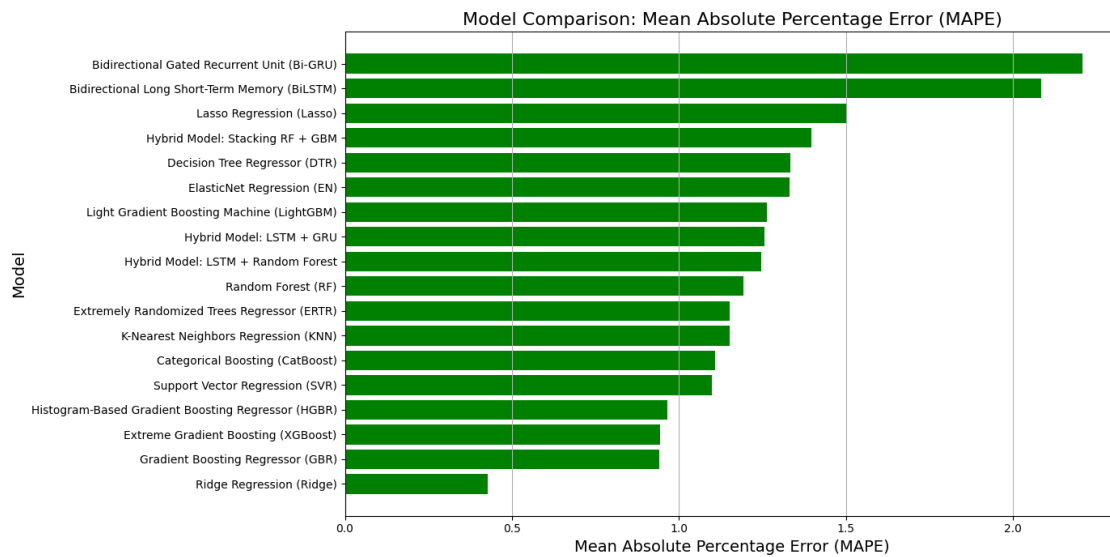
- **Ridge Regression** is the **best-performing model** in terms of RMSE.
- **Boosting methods (XGBoost, LightGBM, CatBoost)** are **effective choices** for reducing error.
- **Deep learning models, including Bi-GRU and Bi-LSTM, show weaker predictive accuracy** compared to traditional regression methods.

- For optimal performance, simpler models like Ridge Regression or tree-based boosting methods may be preferable.

```
[ ]: # Visualize Mean Absolute Error (MAE)
plot_metrics_comparison(results_df, "MAE", "Mean Absolute Error (MAE)",
↪"purple")
```



```
[ ]: # Visualize Mean Absolute Percentage Error (MAPE)
plot_metrics_comparison(results_df, "MAPE (%)", "Mean Absolute Percentage Error",
↪(MAPE)", "green")
```



101 Model Comparison: Mean Absolute Percentage Error (MAPE)

101.1 Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their **Mean Absolute Percentage Error (MAPE)**.
- **Lower MAPE values indicate better performance**, as they signify lower relative errors in predictions.

101.2 Top Performing Models (Lowest MAPE)

1. **Ridge Regression** has the lowest MAPE, making it the most accurate model in terms of percentage error.
2. **Gradient Boosting Regressor (GBR), Extreme Gradient Boosting (XGBoost), and Histogram-Based Gradient Boosting Regressor (HGBR)** also show low MAPE values, indicating strong predictive accuracy.

101.3 Moderate Performance Models

- **Support Vector Regression (SVR), CatBoost, and K-Nearest Neighbors Regression (KNN)** exhibit moderate MAPE values.
- **Extremely Randomized Trees Regressor (ERTR) and Random Forest (RF)** have slightly higher errors but remain competitive.

101.4 Higher MAPE (Lower Performance) Models

- **Deep learning models such as Bi-GRU and Bi-LSTM** have the highest MAPE, indicating poorer predictive performance in terms of percentage error.
- **Hybrid models (LSTM + GRU, LSTM + Random Forest, Stacking RF + GBM)** also show **high MAPE**, suggesting they struggle with relative error minimization.
- **Lasso Regression and Decision Tree Regressor (DTR)** also exhibit relatively high MAPE, making them less effective.

101.5 Insights

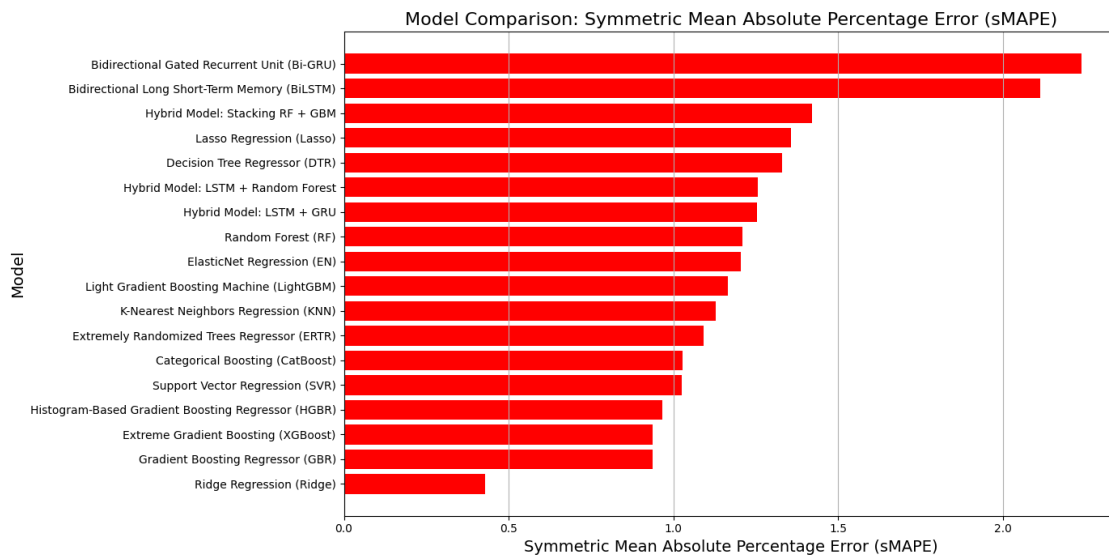
- **Ridge Regression outperforms deep learning models** in terms of MAPE, suggesting that simpler models provide better generalization.
- **Boosting techniques (XGBoost, GBR, HGBR)** show strong performance, indicating they effectively minimize relative errors.
- **Deep learning and hybrid models do not necessarily lead to better performance**, as seen in their higher MAPE values.
- **Traditional regression models like Ridge and ElasticNet Regression** perform well compared to complex deep learning architectures.

101.6 Conclusion

- **Ridge Regression achieves the best performance** based on MAPE.
- **Boosting models (XGBoost, GBR, HGBR)** are strong contenders for minimizing percentage error.

- Deep learning models (Bi-GRU, Bi-LSTM) struggle with relative error, making them less effective than simpler models.
- For minimizing MAPE, traditional regression and boosting techniques are preferable over complex hybrid and deep learning models.

```
[ ]: # Visualize symmetric Mean Absolute Percentage Error (sMAPE)
plot_metrics_comparison(results_df, "sMAPE (%)", "Symmetric Mean Absolute_
Percentage Error (sMAPE)", "red")
```



102 Model Comparison: Symmetric Mean Absolute Percentage Error (sMAPE)

102.1 Key Observations

- The bar chart compares multiple machine learning and deep learning models based on **Symmetric Mean Absolute Percentage Error (sMAPE)**.
- **Lower sMAPE values indicate better performance**, as they represent lower percentage errors in predictions.

102.2 Top Performing Models (Lowest sMAPE)

1. **Ridge Regression** achieves the lowest sMAPE, indicating the most accurate predictions.
2. **Gradient Boosting Regressor (GBR)**, **Extreme Gradient Boosting (XGBoost)**, and **Histogram-Based Gradient Boosting Regressor (HGBR)** also show low sMAPE values, making them strong contenders.

102.3 Moderate Performance Models

- **Support Vector Regression (SVR), CatBoost, and Extremely Randomized Trees Regressor (ERTR)** exhibit moderate sMAPE values.
- **K-Nearest Neighbors Regression (KNN) and LightGBM** have slightly higher errors but still perform reasonably well.

102.4 Higher sMAPE (Lower Performance) Models

- **Deep learning models such as Bi-GRU and Bi-LSTM** have the highest sMAPE, indicating poorer predictive performance in terms of relative percentage error.
- **Hybrid models (LSTM + GRU, LSTM + Random Forest, and Stacking RF + GBM)** also show high sMAPE, suggesting they struggle with relative error minimization.
- **Lasso Regression and Decision Tree Regressor (DTR)** also exhibit relatively high sMAPE, making them less effective compared to boosting and traditional regression models.

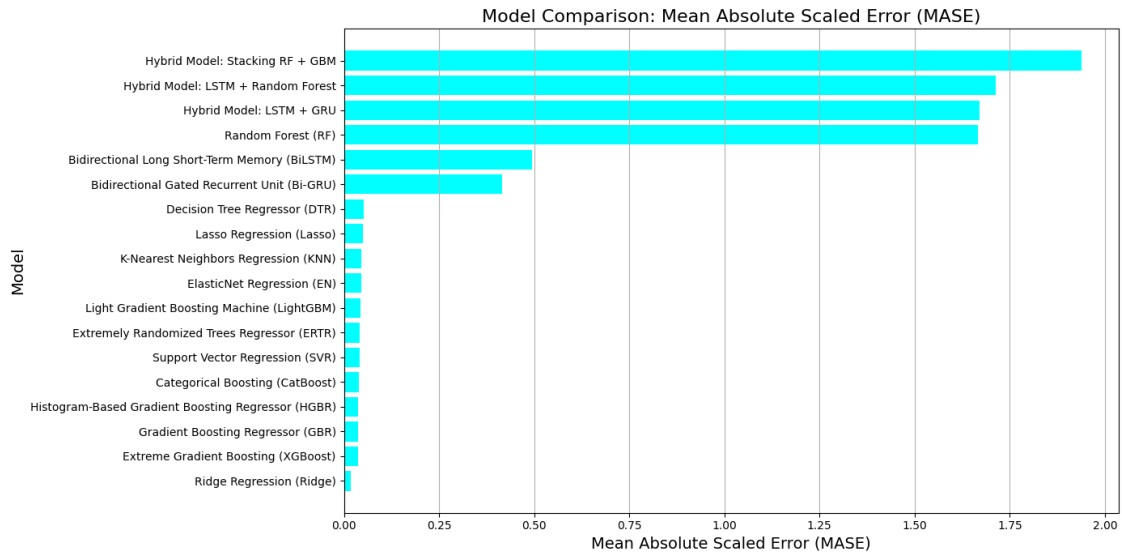
102.5 Insights

- **Ridge Regression** outperforms deep learning models in terms of sMAPE, highlighting the efficiency of simpler models.
- **Boosting techniques (XGBoost, GBR, HGBR)** provide competitive performance, suggesting they effectively handle percentage errors.
- **Deep learning models do not necessarily improve performance**, as seen in their higher sMAPE values.
- **Traditional regression models like Ridge and ElasticNet Regression** perform well compared to complex architectures.

102.6 Conclusion

- **Ridge Regression is the best-performing model** based on sMAPE.
- **Boosting methods (XGBoost, GBR, HGBR)** are effective choices for minimizing percentage error.
- **Deep learning models (Bi-GRU, Bi-LSTM)** exhibit weaker predictive accuracy in terms of sMAPE.
- **For optimal performance, traditional regression and boosting methods should be preferred** over hybrid and deep learning models.

```
[ ]: # Visualize Mean Absolute Scaled Error (MASE)
plot_metrics_comparison(results_df, "MASE", "Mean Absolute Scaled Error_
↪(MASE)", "cyan")
```



103 Model Comparison: Mean Absolute Scaled Error (MASE)

103.1 Key Observations

- The bar chart compares multiple machine learning and deep learning models based on **Mean Absolute Scaled Error (MASE)**.
- **Lower MASE values indicate better model performance**, meaning models with shorter bars have lower scaled error.

103.2 Top Performing Models (Lowest MASE)

1. **Ridge Regression** achieves the lowest MASE, indicating the best predictive accuracy.
2. **Extreme Gradient Boosting (XGBoost)**, **Gradient Boosting Regressor (GBR)**, and **Histogram-Based Gradient Boosting Regressor (HGBR)** also perform well with relatively low MASE.
3. **Other boosting models like CatBoost and LightGBM** also exhibit low scaled errors.

103.3 Moderate Performance Models

- **Support Vector Regression (SVR)**, **Extremely Randomized Trees Regressor (ERTR)**, and **K-Nearest Neighbors Regression (KNN)** show moderate MASE values.
- **ElasticNet Regression**, **Lasso Regression**, and **Decision Tree Regressor (DTR)** have slightly higher MASE but remain competitive.

103.4 Higher MASE (Lower Performance) Models

- **Deep learning and hybrid models such as Stacking RF + GBM, LSTM + Random Forest, and LSTM + GRU** exhibit the highest MASE, indicating poor predictive

accuracy.

- **Random Forest (RF) and BiLSTM** also show significantly higher MASE compared to simpler regression models.
- **Hybrid models combining LSTM and traditional ML models** perform worse than standalone boosting or regression models.

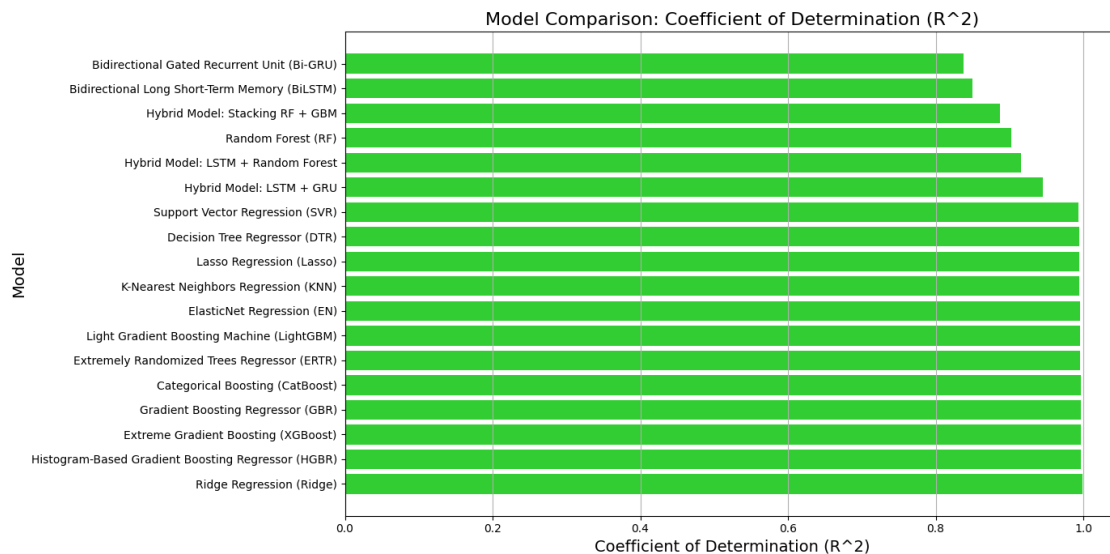
103.5 Insights

- **Ridge Regression outperforms all models**, highlighting the strength of traditional regression methods.
- **Boosting techniques (XGBoost, GBR, HGBR)** provide strong performance, suggesting they effectively minimize scaled errors.
- **Deep learning and hybrid models do not necessarily lead to better performance**, as seen in their higher MASE values.
- **Traditional regression models and boosting algorithms outperform deep learning architectures**, making them more reliable choices.

103.6 Conclusion

- **Ridge Regression is the best-performing model** based on MASE.
- **Boosting models (XGBoost, GBR, HGBR)** are strong alternatives for reducing scaled errors.
- **Deep learning models (BiLSTM, Bi-GRU)** and hybrid approaches underperform, making them less suitable for this dataset.
- **For lower scaled errors, simpler models like Ridge Regression or tree-based boosting methods are preferable** over complex hybrid and deep learning models.

```
[ ]: # Visualize Coefficient of Determination ( $R^2$ )
plot_metrics_comparison(results_df, " $R^2$ ", "Coefficient of Determination_
↪( $R^2$ )", "limegreen")
```



104 Model Comparison: Coefficient of Determination (R^2)

104.1 Key Observations

- The bar chart compares multiple machine learning and deep learning models based on their **Coefficient of Determination (R^2)**.
- **Higher R^2 values indicate better model performance**, meaning models with longer bars have better predictive accuracy.

104.2 Top Performing Models (Highest R^2)

1. **Ridge Regression achieves the highest R^2** , indicating it explains the most variance in the data.
2. **Histogram-Based Gradient Boosting Regressor (HGBR), Extreme Gradient Boosting (XGBoost), and Gradient Boosting Regressor (GBR)** also perform very well, showing high predictive accuracy.
3. **Other boosting models like CatBoost and LightGBM also have high R^2 values**, confirming their strong generalization ability.

104.3 Moderate Performance Models

- **Support Vector Regression (SVR), Extremely Randomized Trees Regressor (ERTR), and K-Nearest Neighbors Regression (KNN)** show moderate R^2 values.
- **ElasticNet Regression, Lasso Regression, and Decision Tree Regressor (DTR)** perform slightly worse but still explain a significant portion of variance.

104.4 Lower R^2 (Poorer Performance) Models

- **Deep learning models such as Bi-GRU and Bi-LSTM exhibit the lowest R^2 values**, indicating poor predictive performance.
- **Hybrid models (LSTM + GRU, LSTM + Random Forest, and Stacking RF + GBM)** also show lower R^2 values, suggesting they do not generalize well.
- **Random Forest (RF) and Decision Tree Regressor (DTR)** also struggle compared to simpler models.

104.5 Insights

- **Ridge Regression outperforms all models**, highlighting the efficiency of simpler, well-regularized models.
- **Boosting techniques (XGBoost, GBR, HGBR) provide competitive performance**, confirming their ability to handle complex patterns.
- **Deep learning models (Bi-GRU, Bi-LSTM) struggle to explain variance**, making them less effective in this scenario.
- **Traditional regression models and boosting algorithms outperform deep learning architectures**, suggesting simpler models are more suitable.

104.6 Conclusion

- **Ridge Regression is the best-performing model** based on R^2 .

- **Boosting models (XGBoost, GBR, HGBR)** are strong alternatives for achieving high predictive accuracy.
- **Deep learning models (BiLSTM, Bi-GRU)** and hybrid approaches underperform, making them less reliable.
- **For high R^2 values, simpler models like Ridge Regression or tree-based boosting methods are preferable** over complex hybrid and deep learning models.